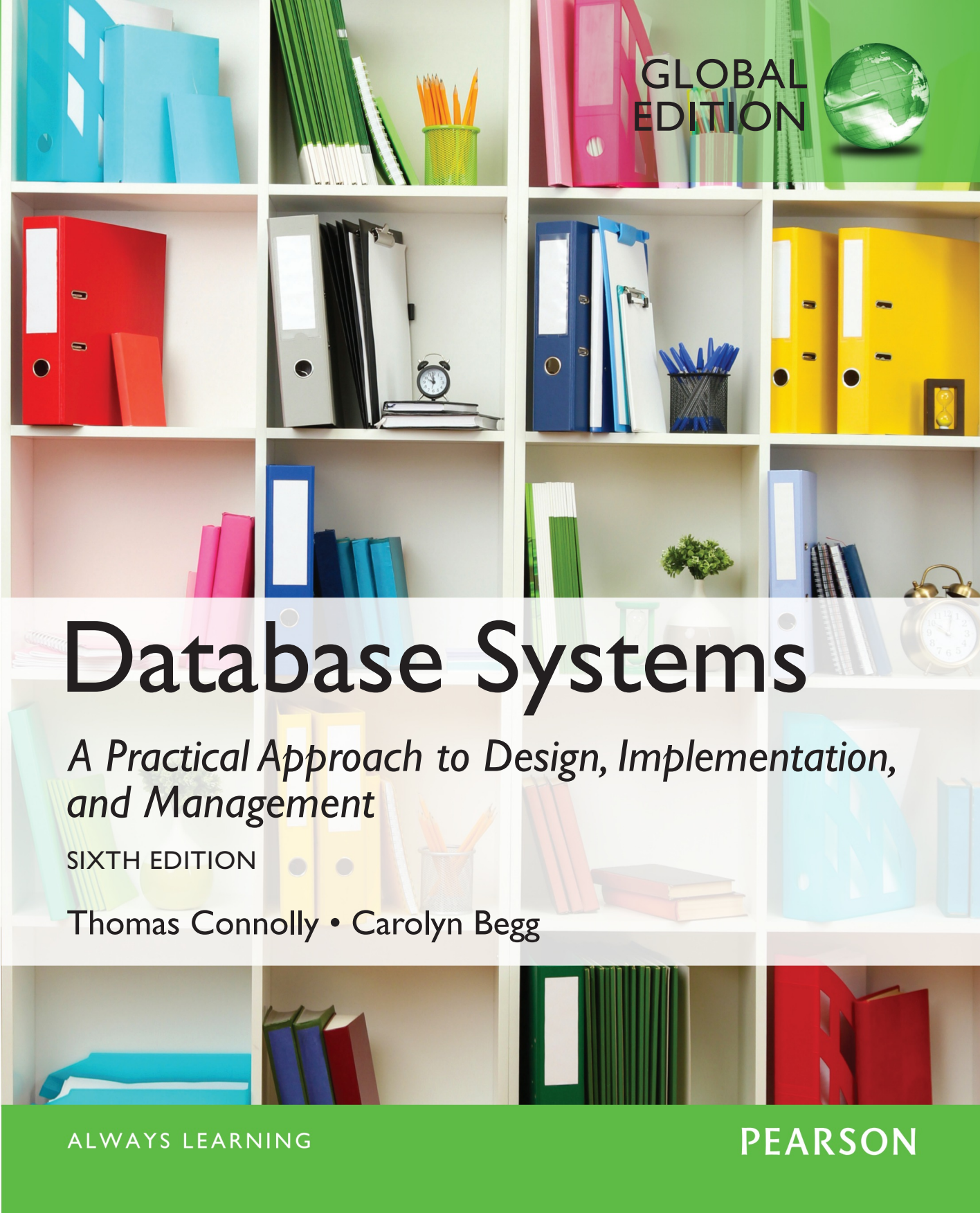




GLOBAL
EDITION



Database Systems

*A Practical Approach to Design, Implementation,
and Management*

SIXTH EDITION

Thomas Connolly • Carolyn Begg

ALWAYS LEARNING

PEARSON

ONLINE ACCESS

Thank you for purchasing a new copy of Database Systems, Sixth Edition. Your textbook includes one year of prepaid access to the book's Companion Website. This prepaid subscription provides you with full access to the following student support areas:

- online appendices
- tutorials on selected chapters
- DreamHome web implementation

**Use a coin to scratch off the coating and reveal your student access code.
Do not use a knife or other sharp object as it may damage the code.**

To access the Database Systems, Sixth Edition., Companion Website for the first time, you will need to register online using a computer with an Internet connection and a web browser. The process takes just a couple of minutes and only needs to be completed once.

1. Go to **www.pearsonglobaleditions.com/connolly**
2. Click on **Companion Website**.
3. Click on the **Register** button.
4. On the registration page, enter your student access code* found beneath the scratch-off panel. Do not type the dashes. You can use lower- or uppercase.
5. Follow the on-screen instructions. If you need help at any time during the online registration process, simply click the **Need Help?** icon.
6. Once your personal Login Name and Password are confirmed, you can begin using the Database Systems Companion Website!

To log in after you have registered:

You only need to register for this Companion Website once. After that, you can log in any time at **www.pearsonglobaleditions.com/connolly** by providing your Login Name and Password when prompted.

*Important: The access code can only be used once. This subscription is valid for one year upon activation and is not transferable. If this access code has already been revealed, it may no longer be valid. If this is the case, you can purchase a subscription by going to **www.pearsonglobaleditions.com/connolly** and following the on-screen instructions.

DATABASE SYSTEMS

A Practical Approach to Design, Implementation, and Management

SIXTH EDITION
GLOBAL EDITION

DATABASE SYSTEMS

A Practical Approach to Design, Implementation, and Management

SIXTH EDITION
GLOBAL EDITION

THOMAS M. CONNOLLY | CAROLYN E. BEGG
UNIVERSITY OF THE WEST OF SCOTLAND

PEARSON

Boston Columbus Indianapolis New York San Francisco Upper Saddle River
Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montréal Toronto
Delhi Mexico City São Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo

Editorial Director, ECS:	Marcia Horton	Senior Production Manufacturing	
Head of Learning Asset		Controller, Global Editions:	Trudy Kimber
Acquisition, Global Editions:	Laura Dent	Manufacturing Buyer:	Linda Sager
Acquisitions Editor:	Matt Goldstein	Art Director:	Jayne Conte
Acquisitions Editor,		Cover Designer:	Lumina Datamatics
Global Editions:	Subhasree Patra	Text Designer:	Susan Raymond
Program Manager:	Kayla Smith-Tarbox	Manager, Text Permissions:	Tim Nicholls
Director of Marketing:	Christy Lesko	Text Permission Project Manager:	Jenell Forschler
Marketing Manager:	Yezan Alayan	Cover Image:	© Africa Studio/ Shutterstock
Marketing Assistant:	Jon Bryant	Media Project Manager:	Renata Butera
Director of Production:	Erin Gregg	Full-Service Project Management:	Vasundhara Sawhney/ Cenveo® Publisher Services
Senior Managing Editor:	Scott Disanno		
Senior Project Manager:	Marilyn Lloyd		
Media Producer, Global Editions:	M Vikram Kumar		
Project Editor, Global Editions:	K.K. Neelakantan		

Credits and acknowledgments borrowed from other sources and reproduced, with permission, in this textbook appear on the Credits page at the end of the book.

Pearson Education Limited
Edinburgh Gate
Harlow
Essex CM20 2JE
England

and Associated Companies throughout the world

Visit us on the World Wide Web at:
www.pearsonglobaleditions.com

© Pearson Education Limited 2015

The rights of Thomas Connolly and Carolyn Begg to be identified as the authors of this work have been asserted by them in accordance with the Copyright, Designs and Patents Act 1988.

Authorized adaptation from the United States edition, entitled Database Systems: A Practical Approach to Design, Implementation, and Management, 6th edition, ISBN 978-0-13-294326-0, by Thomas Connolly and Carolyn Begg, published by Pearson Education © 2015.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without either the prior written permission of the publisher or a license permitting restricted copying in the United Kingdom issued by the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS.

All trademarks used herein are the property of their respective owners. The use of any trademark in this text does not vest in the author or publisher any trademark ownership rights in such trademarks, nor does the use of such trademarks imply any affiliation with or endorsement of this book by such owners.

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library

10 9 8 7 6 5 4 3 2 1
15 14 13 12 11

ISBN 10: 1-292-06118-9
ISBN 13: 978-1-292-06118-4

Typeset by Cenveo Publishing Services.

Printed and bound by Courier Westford in The United States of America.

To Sheena, for her patience, understanding, and love.

To our beautiful children Kathryn, Michael and Stephen with all our love.

And to my brother, Francis, who died during the writing of this book.

Thomas M. Connolly

To my past, present, and future students at UWS.

Carolyn E. Begg

Brief Contents

<i>Preface</i>	35
Part I Background	49
Chapter 1 Introduction to Databases	51
Chapter 2 Database Environment	83
Chapter 3 Database Architectures and the Web	105
Part 2 The Relational Model and Languages	147
Chapter 4 The Relational Model	149
Chapter 5 Relational Algebra and Relational Calculus	167
Chapter 6 SQL: Data Manipulation	191
Chapter 7 SQL: Data Definition	233
Chapter 8 Advanced SQL	271
Chapter 9 Object-Relational DBMSs	291
Part 3 Database Analysis and Design	343
Chapter 10 Database System Development Lifecycle	345
Chapter 11 Database Analysis and the <i>DreamHome</i> Case Study	375
Chapter 12 Entity–Relationship Modeling	405
Chapter 13 Enhanced Entity–Relationship Modeling	433
Chapter 14 Normalization	451
Chapter 15 Advanced Normalization	481
Part 4 Methodology	501
Chapter 16 Methodology—Conceptual Database Design	503
Chapter 17 Methodology—Logical Database Design for the Relational Model	527

Chapter 18	Methodology—Physical Database Design for Relational Databases	561
Chapter 19	Methodology—Monitoring and Tuning the Operational System	585
Part 5	Selected Database Issues	605
Chapter 20	Security and Administration	607
Chapter 21	Professional, Legal, and Ethical Issues in Data Management	641
Chapter 22	Transaction Management	667
Chapter 23	Query Processing	727
Part 6	Distributed DBMSs and Replication	783
Chapter 24	Distributed DBMSs—Concepts and Design	785
Chapter 25	Distributed DBMSs—Advanced Concepts	831
Chapter 26	Replication and Mobile Databases	875
Part 7	Object DBMSs	939
Chapter 27	Object-Oriented DBMSs—Concepts and Design	941
Chapter 28	Object-Oriented DBMSs—Standards and Systems	995
Part 8	The Web and DBMSs	1045
Chapter 29	Web Technology and DBMSs	1047
Chapter 30	Semistructured Data and XML	1129
Part 9	Business Intelligence	1221
Chapter 31	Data Warehousing Concepts	1223
Chapter 32	Data Warehousing Design	1257
Chapter 33	OLAP	1285
Chapter 34	Data Mining	1315

Appendices**I329**

A Users' Requirements Specification for <i>DreamHome</i> Case Study	A-1
B Other Case Studies	B-1
C Alternative ER Modeling Notations	C-1
D Summary of the Database Design Methodology for Relational Databases	D-1
E Introduction to Pyrrho: A Lightweight RDBMS	E-1
F File Organizations and Indexes (Online)	F-1
G When Is a DBMS Relational? (Online)	G-1
H Commercial DBMSs: Access® and Oracle® (Online)	H-1
I Programmatic SQL (Online)	I-1
J Estimating Disk Space Requirements (Online)	J-1
K Introduction to Object-Oriented Concepts (Online)	K-1
L Example Web Scripts (Online)	L-1
M Query-By-Example (QBE) (Online)	M-1
N Third Generation Manifestos (Online)	N-1
O Postgres—An Early ORDBMS (Online)	O-1

References**R-I****Further Reading****FR-I****Index****IN-I**

Contents

<i>Preface</i>	35
Part I Background	49
Chapter 1 Introduction to Databases	51
1.1 Introduction	52
1.2 Traditional File-Based Systems	55
1.2.1 File-Based Approach	55
1.2.2 Limitations of the File-Based Approach	60
1.3 Database Approach	62
1.3.1 The Database	63
1.3.2 The Database Management System (DBMS)	64
1.3.3 (Database) Application Programs	65
1.3.4 Components of the DBMS Environment	66
1.3.5 Database Design: The Paradigm Shift	69
1.4 Roles in the Database Environment	69
1.4.1 Data and Database Administrators	69
1.4.2 Database Designers	70
1.4.3 Application Developers	71
1.4.4 End-Users	71
1.5 History of Database Management Systems	71
1.6 Advantages and Disadvantages of DBMSs	75
<i>Chapter Summary</i>	79
<i>Review Questions</i>	80
<i>Exercises</i>	80
Chapter 2 Database Environment	83
2.1 The Three-Level ANSI-SPARC Architecture	84
2.1.1 External Level	85
2.1.2 Conceptual Level	86
2.1.3 Internal Level	86
2.1.4 Schemas, Mappings, and Instances	87
2.1.5 Data Independence	88
2.2 Database Languages	89
2.2.1 The Data Definition Language (DDL)	90

2.2.2	The Data Manipulation Language (DML)	90
2.2.3	Fourth-Generation Languages (4GLs)	92
2.3	Data Models and Conceptual Modeling	93
2.3.1	Object-Based Data Models	94
2.3.2	Record-Based Data Models	94
2.3.3	Physical Data Models	97
2.3.4	Conceptual Modeling	97
2.4	Functions of a DBMS	97
	<i>Chapter Summary</i>	102
	<i>Review Questions</i>	103
	<i>Exercises</i>	104
Chapter 3	Database Architectures and the Web	105
3.1	Multi-user DBMS Architectures	106
3.1.1	Teleprocessing	106
3.1.2	File-Server Architecture	107
3.1.3	Traditional Two-Tier Client–Server Architecture	108
3.1.4	Three-Tier Client–Server Architecture	111
3.1.5	<i>N</i> -Tier Architectures	112
3.1.6	Middleware	113
3.1.7	Transaction Processing Monitors	115
3.2	Web Services and Service-Oriented Architectures	117
3.2.1	Web Services	117
3.2.2	Service-Oriented Architectures (SOA)	119
3.3	Distributed DBMSs	120
3.4	Data Warehousing	123
3.5	Cloud Computing	125
3.5.1	Benefits and Risks of Cloud Computing	127
3.5.2	Cloud-based database solutions	130
3.6	Components of a DBMS	134
3.7	Oracle Architecture	137
3.7.1	Oracle’s Logical Database Structure	137
3.7.2	Oracle’s Physical Database Structure	140
	<i>Chapter Summary</i>	144
	<i>Review Questions</i>	145
	<i>Exercises</i>	145
Part 2	The Relational Model and Languages	147
Chapter 4	The Relational Model	149
4.1	Brief History of the Relational Model	150
4.2	Terminology	152
4.2.1	Relational Data Structure	152

4.2.2 Mathematical Relations	155
4.2.3 Database Relations	156
4.2.4 Properties of Relations	156
4.2.5 Relational Keys	158
4.2.6 Representing Relational Database Schemas	159
4.3 Integrity Constraints	161
4.3.1 Nulls	161
4.3.2 Entity Integrity	162
4.3.3 Referential Integrity	162
4.3.4 General Constraints	163
4.4 Views	163
4.4.1 Terminology	163
4.4.2 Purpose of Views	164
4.4.3 Updating Views	165
<i>Chapter Summary</i>	165
<i>Review Questions</i>	166
<i>Exercises</i>	166
Chapter 5 Relational Algebra and Relational Calculus	167
5.1 The Relational Algebra	168
5.1.1 Unary Operations	168
5.1.2 Set Operations	171
5.1.3 Join Operations	174
5.1.4 Division Operation	177
5.1.5 Aggregation and Grouping Operations	178
5.1.6 Summary of the Relational Algebra Operations	180
5.2 The Relational Calculus	181
5.2.1 Tuple Relational Calculus	181
5.2.2 Domain Relational Calculus	184
5.3 Other Languages	186
<i>Chapter Summary</i>	187
<i>Review Questions</i>	187
<i>Exercises</i>	188
Chapter 6 SQL: Data Manipulation	191
6.1 Introduction to SQL	192
6.1.1 Objectives of SQL	192
6.1.2 History of SQL	193
6.1.3 Importance of SQL	195
6.1.4 Terminology	195
6.2 Writing SQL Commands	195

6.3	Data Manipulation	196
6.3.1	Simple Queries	197
6.3.2	Sorting Results (ORDER BY Clause)	205
6.3.3	Using the SQL Aggregate Functions	207
6.3.4	Grouping Results (GROUP BY Clause)	209
6.3.5	Subqueries	212
6.3.6	ANY and ALL	214
6.3.7	Multi-table Queries	216
6.3.8	EXISTS and NOT EXISTS	222
6.3.9	Combining Result Tables (UNION, INTERSECT, EXCEPT)	223
6.3.10	Database Updates	225
	<i>Chapter Summary</i>	229
	<i>Review Questions</i>	230
	<i>Exercises</i>	230
Chapter 7	SQL: Data Definition	233
7.1	The ISO SQL Data Types	234
7.1.1	SQL Identifiers	234
7.1.2	SQL Scalar Data Types	235
7.2	Integrity Enhancement Feature	240
7.2.1	Required Data	240
7.2.2	Domain Constraints	240
7.2.3	Entity Integrity	241
7.2.4	Referential Integrity	242
7.2.5	General Constraints	243
7.3	Data Definition	244
7.3.1	Creating a Database	244
7.3.2	Creating a Table (CREATE TABLE)	245
7.3.3	Changing a Table Definition (ALTER TABLE)	248
7.3.4	Removing a Table (DROP TABLE)	249
7.3.5	Creating an Index (CREATE INDEX)	250
7.3.6	Removing an Index (DROP INDEX)	250
7.4	Views	251
7.4.1	Creating a View (CREATE VIEW)	251
7.4.2	Removing a View (DROP VIEW)	253
7.4.3	View Resolution	254
7.4.4	Restrictions on Views	255
7.4.5	View Updatability	255
7.4.6	WITH CHECK OPTION	256
7.4.7	Advantages and Disadvantages of Views	258
7.4.8	View Materialization	260

7.5	Transactions	261
7.5.1	Immediate and Deferred Integrity Constraints	262
7.6	Discretionary Access Control	262
7.6.1	Granting Privileges to Other Users (GRANT)	264
7.6.2	Revoking Privileges from Users (REVOKE)	265
	<i>Chapter Summary</i>	267
	<i>Review Questions</i>	268
	<i>Exercises</i>	268
Chapter 8	Advanced SQL	271
8.1	The SQL Programming Language	272
8.1.1	Declarations	272
8.1.2	Assignments	273
8.1.3	Control Statements	274
8.1.4	Exceptions in PL/SQL	276
8.1.5	Cursors in PL/SQL	277
8.2	Subprograms, Stored Procedures, Functions, and Packages	280
8.3	Triggers	281
8.4	Recursion	287
	<i>Chapter Summary</i>	288
	<i>Review Questions</i>	289
	<i>Exercises</i>	289
Chapter 9	Object-Relational DBMSs	291
9.1	Advanced Database Applications	292
9.2	Weaknesses of RDBMSs	297
9.3	Storing Objects in a Relational Database	302
9.3.1	Mapping Classes to Relations	303
9.3.2	Accessing Objects in the Relational Database	304
9.4	Introduction to Object-Relational Database Systems	305
9.5	SQL:2011	308
9.5.1	Row Types	309
9.5.2	User-Defined Types	310
9.5.3	Subtypes and Supertypes	313
9.5.4	User-Defined Routines	314
9.5.5	Polymorphism	317
9.5.6	Reference Types and Object Identity	318
9.5.7	Creating Tables	318
9.5.8	Querying Data	321

9.5.9 Collection Types	323
9.5.10 Typed Views	326
9.5.11 Persistent Stored Modules	327
9.5.12 Triggers	327
9.5.13 Large Objects	330
9.5.14 Recursion	331
9.6 Object-Oriented Extensions in Oracle	331
9.6.1 User-Defined Data Types	332
9.6.2 Manipulating Object Tables	337
9.6.3 Object Views	338
9.6.4 Privileges	339
<i>Chapter Summary</i>	340
<i>Review Questions</i>	340
<i>Exercises</i>	341

Part 3 Database Analysis and Design 343

Chapter 10 Database System Development Lifecycle	345
10.1 The Information Systems Lifecycle	346
10.2 The Database System Development Lifecycle	347
10.3 Database Planning	347
10.4 System Definition	350
10.4.1 User Views	350
10.5 Requirements Collection and Analysis	350
10.5.1 Centralized Approach	352
10.5.2 View Integration Approach	352
10.6 Database Design	354
10.6.1 Approaches to Database Design	355
10.6.2 Data Modeling	355
10.6.3 Phases of Database Design	356
10.7 DBMS Selection	359
10.7.1 Selecting the DBMS	359
10.8 Application Design	363
10.8.1 Transaction Design	364
10.8.2 User Interface Design Guidelines	365
10.9 Prototyping	367
10.10 Implementation	367
10.11 Data Conversion and Loading	368

10.12	Testing	368
10.13	Operational Maintenance	369
10.14	CASE Tools	370
	<i>Chapter Summary</i>	372
	<i>Review Questions</i>	373
	<i>Exercises</i>	374
Chapter 11	Database Analysis and the <i>DreamHome</i> Case Study	375
11.1	When Are Fact-Finding Techniques Used?	376
11.2	What Facts Are Collected?	377
11.3	Fact-Finding Techniques	378
	11.3.1 Examining Documentation	378
	11.3.2 Interviewing	378
	11.3.3 Observing the Enterprise in Operation	379
	11.3.4 Research	380
	11.3.5 Questionnaires	380
11.4	Using Fact-Finding Techniques: A Worked -Example	381
	11.4.1 The <i>DreamHome</i> Case Study—An Overview of the Current System	382
	11.4.2 The <i>DreamHome</i> Case Study—Database Planning	386
	11.4.3 The <i>DreamHome</i> Case Study—System Definition	392
	11.4.4 The <i>DreamHome</i> Case Study—Requirements Collection and Analysis	393
	11.4.5 The <i>DreamHome</i> Case Study—Database Design	401
	<i>Chapter Summary</i>	402
	<i>Review Questions</i>	402
	<i>Exercises</i>	402
Chapter 12	Entity–Relationship Modeling	405
12.1	Entity Types	406
12.2	Relationship Types	408
	12.2.1 Degree of Relationship Type	410
	12.2.2 Recursive Relationship	412
12.3	Attributes	413
	12.3.1 Simple and Composite Attributes	413
	12.3.2 Single-valued and Multi-valued Attributes	414
	12.3.3 Derived Attributes	414
	12.3.4 Keys	415
12.4	Strong and Weak Entity Types	417
12.5	Attributes on Relationships	418

12.6	Structural Constraints	419
12.6.1	One-to-One (1:1) Relationships	420
12.6.2	One-to-Many (1:*) Relationships	421
12.6.3	Many-to-Many (*:*) Relationships	422
12.6.4	Multiplicity for Complex Relationships	423
12.6.5	Cardinality and Participation Constraints	424
12.7	Problems with ER Models	426
12.7.1	Fan Traps	426
12.7.2	Chasm Traps	428
	<i>Chapter Summary</i>	430
	<i>Review Questions</i>	430
	<i>Exercises</i>	431
Chapter 13	Enhanced Entity–Relationship Modeling	433
13.1	Specialization/Generalization	434
13.1.1	Superclasses and Subclasses	434
13.1.2	Superclass/Subclass Relationships	435
13.1.3	Attribute Inheritance	436
13.1.4	Specialization Process	436
13.1.5	Generalization Process	437
13.1.6	Constraints on Specialization/Generalization	440
13.1.7	Worked Example of using Specialization/ Generalization to Model the Branch View of the <i>DreamHome</i> Case Study	441
13.2	Aggregation	445
13.3	Composition	446
	<i>Chapter Summary</i>	447
	<i>Review Questions</i>	448
	<i>Exercises</i>	448
Chapter 14	Normalization	451
14.1	The Purpose of Normalization	452
14.2	How Normalization Supports Database Design	453
14.3	Data Redundancy and Update Anomalies	454
14.3.1	Insertion Anomalies	455
14.3.2	Deletion Anomalies	455
14.3.3	Modification Anomalies	456
14.4	Functional Dependencies	456
14.4.1	Characteristics of Functional Dependencies	456
14.4.2	Identifying Functional Dependencies	460
14.4.3	Identifying the Primary Key for a Relation Using Functional Dependencies	463
14.5	The Process of Normalization	464

14.6	First Normal Form (1NF)	466
14.7	Second Normal Form (2NF)	470
14.8	Third Normal Form (3NF)	471
14.9	General Definitions of 2NF and 3NF	473
	<i>Chapter Summary</i>	475
	<i>Review Questions</i>	475
	<i>Exercises</i>	476
Chapter 15	Advanced Normalization	481
15.1	More on Functional Dependencies	482
15.1.1	Inference Rules for Functional Dependencies	482
15.1.2	Minimal Sets of Functional Dependencies	484
15.2	Boyce–Codd Normal Form (BCNF)	485
15.2.1	Definition of BCNF	485
15.3	Review of Normalization Up to BCNF	440
15.4	Fourth Normal Form (4NF)	493
15.4.1	Multi-Valued Dependency	494
15.4.2	Definition of Fourth Normal Form	495
15.5	Fifth Normal Form (5NF)	495
15.5.1	Lossless-Join Dependency	496
15.5.2	Definition of Fifth Normal Form	496
	<i>Chapter Summary</i>	498
	<i>Review Questions</i>	498
	<i>Exercises</i>	499
Part 4	Methodology	501
Chapter 16	Methodology—Conceptual Database Design	503
16.1	Introduction to the Database Design Methodology	504
16.1.1	What Is a Design Methodology?	504
16.1.2	Conceptual, Logical, and Physical Database Design	505
16.1.3	Critical Success Factors in Database Design	505
16.2	Overview of the Database Design Methodology	506
16.3	Conceptual Database Design Methodology	508
	Step 1: Build Conceptual Data Model	508
	<i>Chapter Summary</i>	524
	<i>Review Questions</i>	524
	<i>Exercises</i>	525

Chapter 17	Methodology—Logical Database Design for the Relational Model	527
17.1	Logical Database Design Methodology for the Relational Model	528
	Step 2: Build Logical Data Model	528
	<i>Chapter Summary</i>	556
	<i>Review Questions</i>	557
	<i>Exercises</i>	557
Chapter 18	Methodology—Physical Database Design for Relational Databases	561
18.1	Comparison of Logical and Physical Database Design	562
18.2	Overview of the Physical Database Design Methodology	563
18.3	The Physical Database Design Methodology for Relational Databases	564
	Step 3: Translate Logical Data Model for Target DBMS	564
	Step 4: Design File Organizations and Indexes	569
	Step 5: Design User Views	582
	Step 6: Design Security Mechanisms	582
	<i>Chapter Summary</i>	583
	<i>Review Questions</i>	584
	<i>Exercises</i>	584
Chapter 19	Methodology—Monitoring and Tuning the Operational System	585
19.1	Denormalizing and Introducing Controlled Redundancy	585
	Step 7: Consider the Introduction of Controlled Redundancy	585
19.2	Monitoring the System to Improve Performance	598
	Step 8: Monitor and Tune the Operational System	598
	<i>Chapter Summary</i>	602
	<i>Review Questions</i>	603
	<i>Exercises</i>	603
Part 5	Selected Database Issues	605
Chapter 20	Security and Administration	607
20.1	Database Security	608
	20.1.1 Threats	609

20.2	Countermeasures—Computer-Based Controls	611
20.2.1	Authorization	612
20.2.2	Access Controls	613
20.2.3	Views	616
20.2.4	Backup and Recovery	616
20.2.5	Integrity	617
20.2.6	Encryption	617
20.2.7	RAID (Redundant Array of Independent Disks)	618
20.3	Security in Microsoft Office Access DBMS	621
20.4	Security in Oracle DBMS	623
20.5	DBMSs and Web Security	627
20.5.1	Proxy Servers	628
20.5.2	Firewalls	628
20.5.3	Message Digest Algorithms and Digital Signatures	629
20.5.4	Digital Certificates	629
20.5.5	Kerberos	630
20.5.6	Secure Sockets Layer and Secure HTTP	630
20.5.7	Secure Electronic Transactions and Secure Transaction Technology	631
20.5.8	Java Security	632
20.5.9	ActiveX Security	634
20.6	Data Administration and Database Administration	634
20.6.1	Data Administration	635
20.6.2	Database Administration	636
20.6.3	Comparison of Data and Database Administration	636
	<i>Chapter Summary</i>	637
	<i>Review Questions</i>	638
	<i>Exercises</i>	638
Chapter 21	Professional, Legal, and Ethical Issues in Data Management	641
21.1	Defining Legal and Ethical Issues in IT	642
21.1.1	Defining Ethics in the Context of IT	642
21.1.2	The Difference Between Ethical and Legal Behavior	643
21.1.3	Ethical Behavior in IT	644
21.2	Legislation and Its Impact on the IT Function	645
21.2.1	Securities and Exchange Commission (SEC) Regulation National Market System (NMS)	645
21.2.2	The Sarbanes-Oxley Act, COBIT, and COSO	646
21.2.3	The Health Insurance Portability and Accountability Act	649
21.2.4	The European Union (EU) Directive on Data Protection of 1995	650
21.2.5	The United Kingdom's Data Protection Act of 1998	651

21.2.6	Access to Information Laws	652
21.2.7	International Banking—Basel II Accords	654
21.3	Establishing a Culture of Legal and Ethical	
	Data Stewardship	655
21.3.1	Developing an Organization-Wide Policy for Legal and Ethical Behavior	655
21.3.2	Professional Organizations and Codes of Ethics	656
21.3.3	Developing an Organization-Wide Policy for Legal and Ethical Behavior for <i>DreamHome</i>	659
21.4	Intellectual Property	660
21.4.1	Patent	661
21.4.2	Copyright	661
21.4.3	Trademark	662
21.4.4	Intellectual Property Rights Issues for Software	662
21.4.5	Intellectual Property Rights Issues for Data	664
	<i>Chapter Summary</i>	664
	<i>Review Questions</i>	665
	<i>Exercises</i>	666
Chapter 22	Transaction Management	667
22.1	Transaction Support	668
22.1.1	Properties of Transactions	671
22.1.2	Database Architecture	671
22.2	Concurrency Control	672
22.2.1	The Need for Concurrency Control	672
22.2.2	Serializability and Recoverability	675
22.2.3	Locking Methods	683
22.2.4	Deadlock	689
22.2.5	Timestamping Methods	692
22.2.6	Multiversion Timestamp Ordering	695
22.2.7	Optimistic Techniques	696
22.2.8	Granularity of Data Items	697
22.3	Database Recovery	700
22.3.1	The Need for Recovery	700
22.3.2	Transactions and Recovery	701
22.3.3	Recovery Facilities	704
22.3.4	Recovery Techniques	707
22.3.5	Recovery in a Distributed DBMS	709
22.4	Advanced Transaction Models	709
22.4.1	Nested Transaction Model	711
22.4.2	Sagas	712
22.4.3	Multilevel Transaction Model	713
22.4.4	Dynamic Restructuring	714
22.4.5	Workflow Models	715

22.5	Concurrency Control and Recovery in Oracle	716
22.5.1	Oracle's Isolation Levels	717
22.5.2	Multiversion Read Consistency	717
22.5.3	Deadlock Detection	719
22.5.4	Backup and Recovery	719
	<i>Chapter Summary</i>	722
	<i>Review Questions</i>	723
	<i>Exercises</i>	724
Chapter 23	Query Processing	727
23.1	Overview of Query Processing	729
23.2	Query Decomposition	732
23.3	Heuristical Approach to Query Optimization	736
23.3.1	Transformation Rules for the Relational Algebra Operations	736
23.3.2	Heuristical Processing Strategies	741
23.4	Cost Estimation for the Relational Algebra Operations	742
23.4.1	Database Statistics	742
23.4.2	Selection Operation ($S = \sigma_p(R)$)	743
23.4.3	Join Operation ($T = (R \bowtie_F S)$)	750
23.4.4	Projection Operation ($S = \Pi_{A_1, A_2, \dots, A_m}(R)$)	757
23.4.5	The Relational Algebra Set Operations ($T = R \cup S$, $T = R \cap S$, $T = R - S$)	759
23.5	Enumeration of Alternative Execution Strategies	760
23.5.1	Pipelining	761
23.5.2	Linear Trees	761
23.5.3	Physical Operators and Execution Strategies	762
23.5.4	Reducing the Search Space	764
23.5.5	Enumerating Left-Deep Trees	765
23.5.6	Semantic Query Optimization	766
23.5.7	Alternative Approaches to Query Optimization	767
23.5.8	Distributed Query Optimization	768
23.6	Query Processing and Optimization	768
23.6.1	New Index Types	771
23.7	Query Optimization in Oracle	772
23.7.1	Rule-Based and Cost-Based Optimization	772
23.7.2	Histograms	776
23.7.3	Viewing the Execution Plan	778
	<i>Chapter Summary</i>	779
	<i>Review Questions</i>	780
	<i>Exercises</i>	781

Part 6 Distributed DBMSs and Replication **783**

Chapter 24 Distributed DBMSs—Concepts and Design **785**

24.1	Introduction	786
24.1.1	Concepts	787
24.1.2	Advantages and Disadvantages of DDBMSs	791
24.1.3	Homogeneous and Heterogeneous DDBMSs	794
24.2	Overview of Networking	797
24.3	Functions and Architectures of a DDBMS	801
24.3.1	Functions of a DDBMS	801
24.3.2	Reference Architecture for a DDBMS	801
24.3.3	Reference Architecture for a Federated MDBS	803
24.3.4	Component Architecture for a DDBMS	804
24.4	Distributed Relational Database Design	805
24.4.1	Data Allocation	806
24.4.2	Fragmentation	807
24.5	Transparencies in a DDBMS	816
24.5.1	Distribution Transparency	816
24.5.2	Transaction Transparency	819
24.5.3	Performance Transparency	822
24.5.4	DBMS Transparency	824
24.5.5	Summary of Transparencies in a DDBMS	824
24.6	Date's Twelve Rules for a DDBMS	825
	<i>Chapter Summary</i>	827
	<i>Review Questions</i>	828
	<i>Exercises</i>	828

Chapter 25 Distributed DBMSs—Advanced Concepts **831**

25.1	Distributed Transaction Management	832
25.2	Distributed Concurrency Control	833
25.2.1	Objectives	833
25.2.2	Distributed Serializability	834
25.2.3	Locking Protocols	834
25.3	Distributed Deadlock Management	837
25.4	Distributed Database Recovery	840
25.4.1	Failures in a Distributed Environment	841
25.4.2	How Failures Affect Recovery	842
25.4.3	Two-Phase Commit (2PC)	842
25.4.4	Three-Phase Commit (3PC)	849
25.4.5	Network Partitioning	852
25.5	The X/Open Distributed Transaction Processing Model	854
25.6	Distributed Query Optimization	856

25.6.1 Data Localization	858
25.6.2 Distributed Joins	861
25.6.3 Global Optimization	862
25.7 Distribution in Oracle	866
25.7.1 Oracle's DDBMS Functionality	866
Chapter Summary	872
Review Questions	872
Exercises	873
Chapter 26 Replication and Mobile Databases	875
26.1 Introduction to Data Replication	876
26.1.1 Applications of Replication	877
26.1.2 Replication Model	878
26.1.3 Functional Model of Replication Protocols	879
26.1.4 Consistency	880
26.2 Replication Architecture	880
26.2.1 Kernel-Based Replication	880
26.2.2 Middleware-Based Replication	881
26.2.3 Processing of Updates	882
26.2.4 Propagation of Updates	884
26.2.5 Update Location (Data Ownership)	884
26.2.6 Termination Protocols	888
26.3 Replication Schemes	888
26.3.1 Eager Primary Copy	889
26.3.2 Lazy Primary Copy	894
26.3.3 Eager Update Anywhere	898
26.3.4 Lazy Update Anywhere	899
26.3.5 Update Anywhere with Uniform Total Order Broadcast	903
26.3.6 SI and Uniform Total Order Broadcast Replication	907
26.4 Introduction to Mobile Databases	913
26.4.1 Mobile DBMSs	915
26.4.2 Issues with Mobile DBMSs	916
26.5 Oracle Replication	929
26.5.1 Oracle's Replication Functionality	929
Chapter Summary	936
Review Questions	937
Exercises	937
Part 7 Object DBMSs	939
Chapter 27 Object-Oriented DBMSs—Concepts and Design	941
27.1 Next-Generation Database Systems	943

27.2	Introduction to OODBMSs	945
27.2.1	Definition of Object-Oriented DBMSs	945
27.2.2	Functional Data Models	946
27.2.3	Persistent Programming Languages	951
27.2.4	Alternative Strategies for Developing an OODBMS	953
27.3	Persistence in OODBMSs	954
27.3.1	Pointer Swizzling Techniques	956
27.3.2	Accessing an Object	959
27.3.3	Persistence Schemes	961
27.3.4	Orthogonal Persistence	962
27.4	Issues in OODBMSs	964
27.4.1	Transactions	964
27.4.2	Versions	965
27.4.3	Schema Evolution	966
27.4.4	Architecture	969
27.4.5	Benchmarking	971
27.5	Advantages and Disadvantages of OODBMSs	974
27.5.1	Advantages	974
27.5.2	Disadvantages	976
27.6	Comparison of ORDBMS and OODBMS	978
27.7	Object-Oriented Database Design	979
27.7.1	Comparison of Object-Oriented Data Modeling and Conceptual Data Modeling	979
27.7.2	Relationships and Referential Integrity	980
27.7.3	Behavioral Design	982
27.8	Object-Oriented Analysis and Design with UML	984
27.8.1	UML Diagrams	985
27.8.2	Usage of UML in the Methodology for Database Design	990
	<i>Chapter Summary</i>	992
	<i>Review Questions</i>	993
	<i>Exercises</i>	993
Chapter 28	Object-Oriented DBMSs—Standards and Systems	995
28.1	Object Management Group	996
28.1.1	Background	996
28.1.2	The Common Object Request Broker Architecture	999
28.1.3	Other OMG Specifications	1004
28.1.4	Model-Driven Architecture	1007
28.2	Object Data Standard ODMG 3.0, 1999	1007
28.2.1	Object Data Management Group	1009
28.2.2	The Object Model	1010

28.2.3 The Object Definition Language	1018
28.2.4 The Object Query Language	1021
28.2.5 Other Parts of the ODMG Standard	1027
28.2.6 Mapping the Conceptual Design to a Logical (Object-Oriented) Design	1030
28.3 ObjectStore	1031
28.3.1 Architecture	1031
28.3.2 Building an ObjectStore Application	1034
28.3.3 Data Definition in ObjectStore	1035
28.3.4 Data Manipulation in ObjectStore	1039
<i>Chapter Summary</i>	<i>1042</i>
<i>Review Questions</i>	<i>1043</i>
<i>Exercises</i>	<i>1043</i>

Part 8 The Web and DBMSs **I045**

Chapter 29 Web Technology and DBMSs **I047**

29.1 Introduction to the Internet and the Web	1048
29.1.1 Intranets and Extranets	1050
29.1.2 e-Commerce and e-Business	1051
29.2 The Web	1052
29.2.1 HyperText Transfer Protocol	1053
29.2.2 HyperText Markup Language	1055
29.2.3 Uniform Resource Locators	1057
29.2.4 Static and Dynamic Web Pages	1058
29.2.5 Web Services	1058
29.2.6 Requirements for Web–DBMS Integration	1059
29.2.7 Advantages and Disadvantages of the Web–DBMS Approach	1060
29.2.8 Approaches to Integrating the Web and DBMSs	1064
29.3 Scripting Languages	1065
29.3.1 JavaScript and JScript	1065
29.3.2 VBScript	1066
29.3.3 Perl and PHP	1067
29.4 Common Gateway Interface (CGI)	1067
29.4.1 Passing Information to a CGI Script	1069
29.4.2 Advantages and Disadvantages of CGI	1071
29.5 HTTP Cookies	1072
29.6 Extending the Web Server	1073
29.6.1 Comparison of CGI and API	1074
29.7 Java	1074
29.7.1 JDBC	1078
29.7.2 SQLJ	1084

29.7.3	Comparison of JDBC and SQLJ	1084
29.7.4	Container-Managed Persistence (CMP)	1085
29.7.5	Java Data Objects (JDO)	1089
29.7.6	JPA (Java Persistence API)	1096
29.7.7	Java Servlets	1104
29.7.8	JavaServer Pages	1104
29.7.9	Java Web Services	1105
29.8	Microsoft's Web Platform	1107
29.8.1	Universal Data Access	1108
29.8.2	Active Server Pages and ActiveX Data Objects	1109
29.8.3	Remote Data Services	1110
29.8.4	Comparison of ASP and JSP	1113
29.8.5	Microsoft .NET	1113
29.8.6	Microsoft Web Services	1118
29.9	Oracle Internet Platform	1119
29.9.1	Oracle WebLogic Server	1120
29.9.2	Oracle Metadata Repository	1121
29.9.3	Oracle Identity Management	1121
29.9.4	Oracle Portal	1122
29.9.5	Oracle WebCenter	1122
29.9.6	Oracle Business Intelligence (BI) Discoverer	1122
29.9.7	Oracle SOA (Service-Oriented Architecture) Suite	1123
	<i>Chapter Summary</i>	1126
	<i>Review Questions</i>	1127
	<i>Exercises</i>	1127
Chapter 30	Semistructured Data and XML	1129
30.1	Semistructured Data	1130
30.1.1	Object Exchange Model (OEM)	1132
30.1.2	Lore and Lorel	1133
30.2	Introduction to XML	1137
30.2.1	Overview of XML	1140
30.2.2	Document Type Definitions (DTDs)	1142
30.3	XML-Related Technologies	1145
30.3.1	DOM and SAX Interfaces	1146
30.3.2	Namespaces	1147
30.3.3	XSL and XSLT	1147
30.3.4	XPath (XML Path Language)	1148
30.3.5	XPointer (XML Pointer Language)	1149
30.3.6	XLink (XML Linking Language)	1150
30.3.7	XHTML	1150
30.3.8	Simple Object Access Protocol (SOAP)	1151
30.3.9	Web Services Description Language (WSDL)	1152

30.3.10 Universal Discovery, Description, and Integration (UDDI)	1152
30.3.11 JSON (JavaScript Object Notation)	1154
30.4 XML Schema	1156
30.4.1 Resource Description Framework (RDF)	1162
30.5 XML Query Languages	1166
30.5.1 Extending Lore and Lorel to Handle XML	1167
30.5.2 XML Query Working Group	1168
30.5.3 XQuery—A Query Language for XML	1169
30.5.4 XML Information Set	1179
30.5.5 XQuery 1.0 and XPath 2.0 Data Model (XDM)	1180
30.5.6 XQuery Update Facility 1.0	1186
30.5.7 Formal Semantics	1188
30.6 XML and Databases	1196
30.6.1 Storing XML in Databases	1196
30.6.2 XML and SQL	1199
30.6.3 Native XML Databases	1213
30.7 XML in Oracle	1214
Chapter Summary	1217
Review Questions	1219
Exercises	1220

Part 9 Business Intelligence

1221

Chapter 31 Data Warehousing Concepts	1223
31.1 Introduction to Data Warehousing	1224
31.1.1 The Evolution of Data Warehousing	1224
31.1.2 Data Warehousing Concepts	1225
31.1.3 Benefits of Data Warehousing	1226
31.1.4 Comparison of OLTP Systems and Data Warehousing	1226
31.1.5 Problems of Data Warehousing	1228
31.1.6 Real-Time Data Warehouse	1230
31.2 Data Warehouse Architecture	1231
31.2.1 Operational Data	1231
31.2.2 Operational Data Store	1231
31.2.3 ETL Manager	1232
31.2.4 Warehouse Manager	1232
31.2.5 Query Manager	1233
31.2.6 Detailed Data	1233
31.2.7 Lightly and Highly Summarized Data	1233
31.2.8 Archive/Backup Data	1233
31.2.9 Metadata	1234
31.2.10 End-User Access Tools	1234

31.3	Data Warehousing Tools and Technologies	1235
31.3.1	Extraction, Transformation, and Loading (ETL)	1236
31.3.2	Data Warehouse DBMS	1237
31.3.3	Data Warehouse Metadata	1240
31.3.4	Administration and Management Tools	1242
31.4	Data Mart	1242
31.4.1	Reasons for Creating a Data Mart	1243
31.5	Data Warehousing and Temporal Databases	1243
31.5.1	Temporal Extensions to the SQL Standard	1246
31.6	Data Warehousing Using Oracle	1248
31.6.1	Warehouse Features in Oracle 11g	1251
31.6.2	Oracle Support for Temporal Data	1252
	<i>Chapter Summary</i>	1253
	<i>Review Questions</i>	1254
	<i>Exercises</i>	1255
Chapter 32	Data Warehousing Design	1257
32.1	Designing a Data Warehouse Database	1258
32.2	Data Warehouse Development Methodologies	1258
32.3	Kimball's Business Dimensional Lifecycle	1260
32.4	Dimensionality Modeling	1261
32.4.1	Comparison of DM and ER models	1264
32.5	The Dimensional Modeling Stage of Kimball's Business Dimensional Lifecycle	1265
32.5.1	Create a High-Level Dimensional Model (Phase I)	1265
32.5.2	Identify All Dimension Attributes for the Dimensional Model (Phase II)	1270
32.6	Data Warehouse Development Issues	1273
32.7	Data Warehousing Design Using Oracle	1274
32.7.1	Oracle Warehouse Builder Components	1274
32.7.2	Using Oracle Warehouse Builder	1275
32.7.3	Warehouse Builder Features in Oracle 11g	1279
	<i>Chapter Summary</i>	1280
	<i>Review Questions</i>	1281
	<i>Exercises</i>	1282
Chapter 33	OLAP	1285
33.1	Online Analytical Processing	1286
33.1.1	OLAP Benchmarks	1287

33.2	OLAP Applications	1287
33.3	Multidimensional Data Model	1289
33.3.1	Alternative Multidimensional Data Representations	1289
33.3.2	Dimensional Hierarchy	1291
33.3.3	Multidimensional Operations	1293
33.3.4	Multidimensional Schemas	1293
33.4	OLAP Tools	1293
33.4.1	Codd's Rules for OLAP Tools	1294
33.4.2	OLAP Server—Implementation Issues	1295
33.4.3	Categories of OLAP Servers	1296
33.5	OLAP Extensions to the SQL Standard	1300
33.5.1	Extended Grouping Capabilities	1300
33.5.2	Elementary OLAP Operators	1305
33.6	Oracle OLAP	1307
33.6.1	Oracle OLAP Environment	1307
33.6.2	Platform for Business Intelligence Applications	1308
33.6.3	Oracle Database	1308
33.6.4	Oracle OLAP	1310
33.6.5	Performance	1311
33.6.6	System Management	1312
33.6.7	System Requirements	1312
33.6.8	OLAP Features in Oracle 11g	1312
	<i>Chapter Summary</i>	1313
	<i>Review Questions</i>	1313
	<i>Exercises</i>	1313
Chapter 34	Data Mining	1315
34.1	Data Mining	1316
34.2	Data Mining Techniques	1316
34.2.1	Predictive Modeling	1318
34.2.2	Database Segmentation	1319
34.2.3	Link Analysis	1320
34.2.4	Deviation Detection	1321
34.3	The Data Mining Process	1322
34.3.1	The CRISP-DM Model	1322
34.4	Data Mining Tools	1323
34.5	Data Mining and Data Warehousing	1324
34.6	Oracle Data Mining (ODM)	1325
34.6.1	Data Mining Capabilities	1325

34.6.2 Enabling Data Mining Applications	1325
34.6.3 Predictions and Insights	1326
34.6.4 Oracle Data Mining Environment	1326
34.6.5 Data Mining Features in Oracle 11g	1327
<i>Chapter Summary</i>	1327
<i>Review Questions</i>	1328
<i>Exercises</i>	1328

Appendices **1329**

A Users' Requirements Specification for DreamHome Case Study	A-I
A.1 Branch User Views of <i>DreamHome</i>	A-1
A.1.1 Data Requirements	A-1
A.1.2 Transaction Requirements (Sample)	A-3
A.2 Staff User Views of <i>DreamHome</i>	A-4
A.2.1 Data Requirements	A-4
A.2.2 Transaction Requirements (Sample)	A-5
B Other Case Studies	B-I
B.1 The <i>University Accommodation Office</i> Case Study	B-1
B.1.1 Data Requirements	B-1
B.1.2 Query Transactions (Sample)	B-3
B.2 The <i>EasyDrive School of Motoring</i> Case Study	B-4
B.2.1 Data Requirements	B-4
B.2.2 Query Transactions (Sample)	B-5
B.3 The <i>Wellmeadows Hospital</i> Case Study	B-5
B.3.1 Data Requirements	B-5
B.3.2 Transaction Requirements (Sample)	B-12
C Alternative ER Modeling Notations	C-I
C.1 ER Modeling Using the Chen Notation	C-1
C.2 ER Modeling Using the Crow's Feet Notation	C-1
D Summary of the Database Design Methodology for Relational Databases	D-I
Step 1: Build Conceptual Data Model	D-1
Step 2: Build Logical Data Model	D-2
Step 3: Translate Logical Data Model for Target DBMS	D-5
Step 4: Design File Organizations and Indexes	D-5

Step 5: Design User Views	D-5
Step 6: Design Security Mechanisms	D-5
Step 7: Consider the Introduction of Controlled Redundancy	D-6
Step 8: Monitor and Tune the Operational System	D-6

E Introduction to Pyrrho: A Lightweight RDBMS	E-I
--	------------

E.1 Pyrrho Features	E-2
E.2 Download and Install Pyrrho	E-2
E.3 Getting Started	E-3
E.4 The Connection String	E-3
E.5 Pyrrho's Security Model	E-4
E.6 Pyrrho SQL Syntax	E-4

F File Organizations and Indexes (Online)	F-I
--	------------

G When Is a DBMS Relational? (Online)	G-I
--	------------

H Commercial DBMSs: Access and Oracle (Online)	H-I
---	------------

I Programmatic SQL (Online)	I-I
------------------------------------	------------

J Estimating Disk Space Requirements (Online)	J-I
--	------------

K Introduction to Object-Oriented Concepts (Online)	K-I
--	------------

L Example Web Scripts (Online)	L-I
---------------------------------------	------------

M Query-By-Example (QBE) (Online)	M-I
--	------------

N Third Generation Manifestos (Online)	N-I
---	------------

O Postgres—An Early ORDBMS (Online)	O-I
--	------------

References	R-I
-------------------	------------

Further Reading	FR-I
------------------------	-------------

Index	IN-I
--------------	-------------

Preface



Background

The history of database research over the past 30 years is one of exceptional productivity that has led to the database system becoming arguably the most important development in the field of software engineering. The database is now the underlying framework of the information system and has fundamentally changed the way many organizations operate. In particular, the developments in this technology over the last few years have produced systems that are more powerful and more intuitive to use. This development has resulted in increasing availability of database systems for a wider variety of users. Unfortunately, the apparent simplicity of these systems has led to users creating databases and applications without the necessary knowledge to produce an effective and efficient system. And so the “software crisis” or, as it is sometimes referred to, the “software depression” continues.

The original stimulus for this book came from the authors’ work in industry, providing consultancy on database design for new software systems or, as often as not, resolving inadequacies with existing systems. In addition, the authors’ move to academia brought similar problems from different users—students. The objectives of this book, therefore, are to provide a textbook that introduces the theory behind databases as clearly as possible and, in particular, to provide a methodology for database design that can be used by both technical and nontechnical readers.

The methodology presented in this book for relational Database Management Systems (DBMSs)—the predominant system for business applications at present—has been tried and tested over the years in both industrial and academic environments. It consists of three main phases: conceptual, logical, and physical database design. The first phase starts with the production of a conceptual data model that is independent of all physical considerations. This model is then refined in the second phase into a logical data model by removing constructs that cannot be represented in relational systems. In the third phase, the logical data model is translated into a physical design for the target DBMS. The physical design phase considers the storage structures and access methods required for efficient and secure access to the database on secondary storage.



The methodology in each phase is presented as a series of steps. For the inexperienced designer, it is expected that the steps will be followed in the order described, and guidelines are provided throughout to help with this process. For the experienced designer, the methodology can be less prescriptive, acting more as a framework or checklist. To help the reader use the methodology and understand the important issues, the methodology has been described using a realistic worked example, based on an integrated case study, *DreamHome*. In addition, three additional case studies are provided in Appendix B to allow readers to try out the methodology for themselves.

UML (Unified Modeling Language)

Increasingly, companies are standardizing the way in which they model data by selecting a particular approach to data modeling and using it throughout their database development projects. A popular high-level data model used in conceptual/logical database design, and the one we use in this book, is based on the concepts of the Entity–Relationship (ER) model. Currently there is no standard notation for an ER model. Most books that cover database design for relational DBMSs tend to use one of two conventional notations:

- Chen’s notation, consisting of rectangles representing entities and diamonds representing relationships, with lines linking the rectangles and diamonds; or
- Crow’s Feet notation, again consisting of rectangles representing entities and lines between entities representing relationships, with a crow’s foot at one end of a line representing a one-to-many relationship.

Both notations are well supported by current Computer-Aided Software Engineering (CASE) tools. However, they can be quite cumbersome to use and a bit difficult to explain. In previous editions, we used Chen’s notation. However, following an extensive questionnaire carried out by Pearson Education, there was a general consensus that the notation should be changed to the latest object-oriented modeling language, called UML (Unified Modeling Language). UML is a notation that combines elements from the three major strands of object-oriented design: Rumbaugh’s OMT modeling, Booch’s Object-Oriented Analysis and Design, and Jacobson’s Objectory.

There are three primary reasons for adopting a different notation: (1) UML is becoming an industry standard; for example, the Object Management Group (OMG) has adopted UML as the standard notation for object methods; (2) UML is arguably clearer and easier to use; and (3) UML is now being adopted within academia for teaching object-oriented analysis and design, and using UML in database modules provides more synergy. Therefore, in this edition we have adopted the class diagram notation from UML. We believe that you will find this notation easier to understand and use.

What's New in the Sixth Edition

- Extended chapter on database architectures and the Web, covering cloud computing.
- Updated chapter on professional, legal, and ethical issues in IT and databases.
- New section on data warehousing and temporal databases.
- New review questions and exercises at the end of chapters.
- Updated treatment to cover the latest version of the SQL standard, which was released in late 2011 (SQL:2011).
- Revised chapter on replication and mobile databases.
- Updated chapters on Web-DBMS integration and XML.
- Coverage updated to Oracle 11g.

Intended Audience

This book is intended as a textbook for a one- or two-semester course in database management or database design in an introductory undergraduate, graduate, or advanced undergraduate course. Such courses are usually required in an information systems, business IT, or computer science curriculum.

The book is also intended as a reference book for IT professionals, such as systems analysts or designers, application programmers, systems programmers, database practitioners, and for independent self-teachers. Owing to the widespread use of database systems nowadays, these professionals could come from any type of company that requires a database.

It would be helpful for students to have a good background in the file organization and data structures concepts covered in Appendix F before covering the material in Chapter 18 on physical database design and Chapter 23 on query processing. This background ideally will have been obtained from a prior course. If this is not possible, then the material in Appendix F can be presented near the beginning of the database course, immediately following Chapter 1.

An understanding of a high-level programming language, such as C, would be advantageous for Appendix I on embedded and dynamic SQL and Section 28.3 on ObjectStore.

Distinguishing Features

- (1) An easy-to-use, step-by-step methodology for conceptual and logical database design, based on the widely accepted Entity–Relationship model, with normalization used as a validation technique. There is an integrated case study showing how to use the methodology.
- (2) An easy-to-use, step-by-step methodology for physical database design, covering the mapping of the logical design to a physical implementation, the selection

of file organizations and indexes appropriate for the applications, and when to introduce controlled redundancy. Again, there is an integrated case study showing how to use the methodology.

- (3) Separate chapters showing how database design fits into the overall database systems development lifecycle, how fact-finding techniques can be used to identify the system requirements, and how UML fits into the methodology.
- (4) A clear and easy-to-understand presentation, with definitions clearly highlighted, chapter objectives clearly stated, and chapters summarized. Numerous examples and diagrams are provided throughout each chapter to illustrate the concepts. There is a realistic case study integrated throughout the book and additional case studies that can be used as student projects.
- (5) Extensive treatment of the latest formal and de facto standards: Structured Query Language (SQL), Query-By-Example (QBE), and the Object Data Management Group (ODMG) standard for object-oriented databases.
- (6) Three tutorial-style chapters on the SQL standard, covering both interactive and embedded SQL.
- (7) A chapter on legal, professional and ethical issues related to IT and databases.
- (8) Comprehensive coverage of the concepts and issues relating to distributed DBMSs and replication servers.
- (9) Comprehensive introduction to the concepts and issues relating to object-based DBMSs including a review of the ODMG standard and a tutorial on the object management facilities within the latest release of the SQL standard, SQL:2011.
- (10) Extensive treatment of the Web as a platform for database applications with many code samples of accessing databases on the Web. In particular, we cover persistence through Container-Managed Persistence (CMP), Java Data Objects (JDO), Java Persistence API (JPA), JDBC, SQLJ, ActiveX Data Objects (ADO), ADO.NET, and Oracle PL/SQL Pages (PSP).
- (11) An introduction to semistructured data and its relationship to XML and extensive coverage of XML and its related technologies. In particular, we cover XML Schema, XQuery, and the XQuery Data Model and Formal Semantics. We also cover the integration of XML into databases and examine the extensions added to SQL:2008 and SQL:2011 to enable the publication of XML.
- (12) Comprehensive introduction to data warehousing, Online Analytical Processing (OLAP), and data mining.
- (13) Comprehensive introduction to dimensionality modeling for designing a data warehouse database. An integrated case study is used to demonstrate a methodology for data warehouse database design.
- (14) Coverage of DBMS system implementation concepts, including concurrency and recovery control, security, and query processing and query optimization.

Pedagogy

Before starting to write any material for this book, one of the objectives was to produce a textbook that would be easy for the readers to follow and understand,

whatever their background and experience. From the authors' experience of using textbooks, which was quite considerable before undertaking a project of this size, and also from listening to colleagues, clients, and students, we knew there were a number of design features that readers liked and disliked. With these comments in mind, the following style and structure features were adopted:

- A set of objectives is clearly identified at the start of each chapter.
- Each important concept that is introduced is clearly defined and highlighted by setting the definition apart from the text.
- Diagrams are liberally used throughout to support and clarify concepts.
- A very practical orientation: each chapter contains many worked examples to illustrate the concepts covered.
- A summary at the end of each chapter covers the main concepts introduced.
- A set of review questions falls at the end of each chapter, the answers to which can be found in the text.
- A set of exercises at the end of each chapter, can be used by teachers or by individuals to demonstrate and test the individual's understanding of the chapter, the answers to which can be found in the accompanying Instructor's Solutions Manual.

Support Materials

A comprehensive set of supplements are available for this textbook:

- Lecture slides in PowerPoint® format
- Instructor's Solutions Manual, including sample solutions to all review questions and exercises
- A companion Web site with additional resources, located at:
www.pearsonglobaleditions/connolly

Supplements are available to qualified instructors only at www.pearsonglobaleditions/connolly. Please contact your local sales representative.

Organization of this Book

Part I: Background

Part 1 of the book serves to introduce the field of database systems and database design.

Chapter 1 introduces the field of database management, examining the problems with the precursor to the database system, the file-based system, and the advantages offered by the database approach.

Chapter 2 examines the database environment, discussing the advantages offered by the three-level ANSI-SPARC architecture, introducing the most popular data models and outlining the functions that should be provided by a multi-user DBMS.

Chapter 3 examines multi-user DBMS architectures and discusses the different types of middleware that exist in the database field. It also examines Web services that can be used to provide new types of business services to users and service-oriented architecture (SOA). The chapter briefly introduces the architecture for a distributed DBMS and data warehousing presented more fully in later chapters. The chapter also looks at the underlying software architecture for DBMSs and the logical and physical structures in the Oracle DBMS, which could be omitted for a first course in database management.

Part 2: The Relational Model and Languages

Part 2 of the book serves to introduce the relational model and relational languages, namely the relational algebra and relational calculus, QBE (Query-By-Example), and SQL (Structured Query Language). This part also examines two highly popular commercial systems: Microsoft Office Access and Oracle.

Chapter 4 introduces the concepts behind the relational model, the most popular data model at present, and the one most often chosen for standard business applications. After introducing the terminology and showing the relationship with mathematical relations, the relational integrity rules, entity integrity, and referential integrity are discussed. The chapter concludes with a section on views, which is expanded upon in Chapter 7.

Chapter 5 introduces the relational algebra and relational calculus with examples to illustrate all the operations. This could be omitted for a first course in database management. However, relational algebra is required to understand Query Processing in Chapter 23 and fragmentation in Chapter 24 on distributed DBMSs. In addition, the comparative aspects of the procedural algebra and the non-procedural calculus act as a useful precursor for the study of SQL in Chapters 6 and 7, although not essential.

Chapter 6 introduces the data manipulation statements of the SQL standard: SELECT, INSERT, UPDATE, and DELETE. The chapter is presented as a tutorial, giving a series of worked examples that demonstrate the main concepts of these statements.

Chapter 7 covers the main data definition facilities of the SQL standard. Again, the chapter is presented as a worked tutorial. The chapter introduces the SQL data types and the data definition statements, the Integrity Enhancement Feature (IEF), and the more advanced features of the data definition statements, including the access control statements GRANT and REVOKE. It also examines views and how they can be created in SQL.

Chapter 8 covers some of the advanced features of SQL, including the SQL programming language (SQL/PSM), triggers, and stored procedures.

Chapter 9 examines the object-relational DBMS and provides a detailed overview of the object management features that have been added to the new release of the SQL

standard, SQL:2011. The chapter also discusses how query processing and query optimization need to be extended to handle data type extensibility efficiently. The chapter concludes by examining some of the object-relational features within Oracle.

Part 3: Database Analysis and Design

Part 3 of the book discusses the main techniques for database analysis and design and how they can be applied in a practical way.

Chapter 10 presents an overview of the main stages of the database system development lifecycle. In particular, it emphasizes the importance of database design and shows how the process can be decomposed into three phases: conceptual, logical, and physical database design. It also describes how the design of the application (*the functional approach*) affects database design (*the data approach*). A crucial stage in the database system development lifecycle is the selection of an appropriate DBMS. This chapter discusses the process of DBMS selection and provides some guidelines and recommendations.

Chapter 11 discusses when a database developer might use fact-finding techniques and what types of facts should be captured. The chapter describes the most commonly used fact-finding techniques and identifies the advantages and disadvantages of each. The chapter also demonstrates how some of these techniques may be used during the earlier stages of the database system lifecycle using the *DreamHome* case study.

Chapters 12 and 13 cover the concepts of the Entity–Relationship (ER) model and the Enhanced Entity–Relationship (EER) model, which allows more advanced data modeling using subclasses and superclasses and categorization. The EER model is a popular high-level conceptual data model and is a fundamental technique of the database design methodology presented herein. The reader is also introduced to UML to represent ER diagrams.

Chapters 14 and 15 examine the concepts behind normalization, which is another important technique used in the logical database design methodology. Using a series of worked examples drawn from the integrated case study, they demonstrate how to transition a design from one normal form to another and show the advantages of having a logical database design that conforms to particular normal forms up to and including fifth normal form.

Part 4: Methodology

This part of the book covers a methodology for database design. The methodology is divided into three parts covering conceptual, logical, and physical database design. Each part of the methodology is illustrated using the *DreamHome* case study.

Chapter 16 presents a step-by-step methodology for conceptual database design. It shows how to decompose the design into more manageable areas based on

individual user views, and then provides guidelines for identifying entities, attributes, relationships, and keys.

Chapter 17 presents a step-by-step methodology for logical database design for the relational model. It shows how to map a conceptual data model to a logical data model and how to validate it to ensure that it supports the required transactions and follows the rules of normalization. For database systems with multiple user views, this chapter shows how to merge the resulting local data models together into a global data model that represents all the user views of the part of the enterprise being modeled.

Chapters 18 and 19 present a step-by-step methodology for physical database design for relational systems. It shows how to translate the logical data model developed during logical database design into a physical design for a relational system. The methodology addresses the performance of the resulting implementation by providing guidelines for choosing file organizations and storage structures and when to introduce controlled redundancy.

Part 5: Selected Database Issues

Part 5 of the book examines four specific topics that the authors consider necessary for a modern course in database management.

Chapter 20 considers database security and administration. Security considers both the DBMS and its environment. It illustrates security provision with Microsoft Office Access and Oracle. The chapter also examines the security problems that can arise in a Web environment and presents some approaches to overcoming them. The chapter concludes with a discussion of the tasks of data administration and database administration.

Chapter 21 considers professional, legal, and ethical issues related to IT and data management and data governance. It distinguishes between legal and ethical issues and situations that data/database administrators face, discusses how new regulations are placing additional requirements and responsibilities on data/database administrators, and how legislation, such as the Sarbanes-Oxley Act and the Basel II accords, affect data/database administration functions.

Chapter 22 concentrates on three functions that a Database Management System should provide, namely transaction management, concurrency control, and recovery. These functions are intended to ensure that the database is reliable and remains in a consistent state when multiple users are accessing the database and in the presence of failures of both hardware and software components. The chapter also discusses advanced transaction models that are more appropriate for transactions that may be of a long duration. The chapter concludes by examining transaction management within Oracle.

Chapter 23 examines query processing and query optimization. The chapter considers the two main techniques for query optimization: the use of heuristic rules that

order the operations in a query and the other technique that compares different strategies based on their relative costs and selects the one that minimizes resource usage. The chapter concludes by examining query processing within Oracle.

Part 6: Distributed DBMSs and Replication

Part 6 of the book examines distributed DBMSs and object-based DBMSs. Distributed database management system (DDMS) technology is one of the current major developments in the database systems area. The previous chapters of this book concentrate on centralized database systems, that is, systems with a single logical database located at one site under the control of a single DBMS.

Chapter 24 discusses the concepts and problems of distributed DBMSs, with which users can access the database at their own site, and also access data stored at remote sites.

Chapter 25 examines various advanced concepts associated with distributed DBMSs. In particular, it concentrates on the protocols associated with distributed transaction management, concurrency control, deadlock management, and database recovery. The chapter also examines the X/Open Distributed Transaction Processing (DTP) protocol. The chapter concludes by examining data distribution within Oracle.

Chapter 26 discusses replication servers as an alternative to distributed DBMSs and examines the issues associated with mobile databases. The chapter also examines the data replication facilities in Oracle.

Part 7: Object DBMSs

The preceding chapters of this book concentrate on the relational model and relational systems. The justification for this is that such systems are currently the predominant DBMS for traditional business database applications. However, relational systems are not without their failings, and the object-based DBMS is a major development in the database systems area that attempts to overcome these failings. Chapters 27–28 examine this development in some detail.

Chapter 27 introduces the object-based DBMSs and first examines the types of advanced database applications that are emerging and discusses the weaknesses of the relational data model that makes it unsuitable for these types of applications. It then examines the object-oriented DBMS (OODBMS) and starts by providing an introduction to object-oriented data models and persistent programming languages. The chapter discusses the difference between the two-level storage model used by conventional DBMSs and the single-level model used by OODBMSs and how this affects data access. It also discusses the various approaches to providing persistence in programming languages and the different techniques for pointer swizzling and examines version management, schema evolution, and OODBMS architectures. The

chapter concludes by briefly showing how the methodology presented in Part 4 of this book may be extended for object-oriented databases.

Chapter 28 addresses the object model proposed by the Object Data Management Group (ODMG), which has become a de facto standard for OODBMSs. The chapter also examines ObjectStore, a commercial OODBMS.

Part 8: The Web and DBMSs

Part 8 of the book deals with the integration of the DBMS into the Web environment, semistructured data and its relationship to XML, XML query languages, and mapping XML to databases.

Chapter 29 examines the integration of the DBMS into the Web environment. After providing a brief introduction to Internet and Web technology, the chapter examines the appropriateness of the Web as a database application platform and discusses the advantages and disadvantages of this approach. It then considers a number of the different approaches to integrating DBMSs into the Web environment, including scripting languages, CGI, server extensions, Java, ADO and ADO.NET, and Oracle's Internet Platform.

Chapter 30 examines semistructured data and then discusses XML and how XML is an emerging standard for data representation and interchange on the Web. The chapter then discusses XML-related technologies such as namespaces, XSL, XPath, XPointer, XLink, SOAP, WSDL, and UDDI. It also examines how XML Schema can be used to define the content model of an XML document and how the Resource Description Framework (RDF) provides a framework for the exchange of metadata. The chapter examines query languages for XML and, in particular, concentrates on XQuery, as proposed by W3C. It also examines the extensions added to SQL:2011 to enable the publication of XML and, more generally, mapping and storing XML in databases.

Part 9: Business Intelligence

The final part of the book considers the main technologies associated with Business Intelligence (BI): the data warehouse, Online Analytical Processing (OLAP), and data mining.

Chapter 31 discusses data warehousing, what it is, how it has evolved, and describes the potential benefits and problems associated with this system. The chapter examines the architecture, the main components, and the associated tools and technologies of a data warehouse. The chapter also discusses data marts and the issues associated with the development and management of data marts. This chapter examines the concepts and practicalities associated with the management of temporal data in a data warehouse. The chapter concludes by describing the data warehousing facilities of the Oracle DBMS.

Chapter 32 describes alternative approaches for the development of the decision-support database of a data warehouse or data mart. The chapter describes the basic concepts associated with dimensionality modeling and compares this technique with traditional Entity–Relationship (ER) modeling. It also describes and demonstrates a step-by-step methodology for designing a data warehouse using worked examples taken from an extended version of the *DreamHome* case study. The chapter concludes by describing how to design a data warehouse using the Oracle Warehouse Builder.

Chapter 33 describes Online Analytical Processing (OLAP). It discusses what OLAP is and the main features of OLAP applications. The chapter discusses how multidimensional data can be represented and the main categories of OLAP tools. It also discusses the OLAP extensions to the SQL standard and how Oracle supports OLAP.

Chapter 34 describes data mining (DM). It discusses what DM is and the main features of DM applications. The chapter describes the main characteristics of data mining operations and associated techniques. It describes the process of DM and the main features of DM tools, with particular coverage of Oracle DM.

Appendices—In Book

Appendix A provides a description of *DreamHome*, a case study that is used extensively throughout the book.



Appendix B provides three additional case studies, which can be used as student projects.

Appendix C describes two alternative data modeling notations to UML: Chen's notation and Crow's Foot.

Appendix D summarizes the steps in the methodology presented in Chapters 16–19 for conceptual, logical, and physical database design.

Appendix E examines an open-source lightweight RDBMS called Pyrrho implemented in C# that demonstrates many of the concepts discussed in this book and can be downloaded and used.

Appendices—Online, at www.pearsonglobaleditions/connolly

Appendix F provides some background information on file organization and storage structures that is necessary for an understanding of the physical database design methodology presented in Chapter 18 and query processing in Chapter 23.

Appendix G describes Codd's 12 rules for a relational DBMS, which form a yardstick against which the “real” relational DBMS products can be identified.

Appendix H provides introductions to two popular commercial relational DBMSs: Microsoft Office Access and Oracle. Elsewhere in the book, we examine how these systems implement various database facilities, such as security and query processing.

Appendix I examines embedded and dynamic SQL, with sample programs in C. The chapter also examines the Open Database Connectivity (ODBC) standard, which has emerged as a de facto industry standard for accessing heterogeneous SQL databases.

Appendix J discusses how to estimate the disk space requirements for an Oracle database.

Appendix K provides an overview of the main object-oriented concepts.

Appendix L provides some sample Web scripts to complement Chapter 29 on Web technology and DBMSs.

Appendix M examines the interactive query language, Query-By-Example (QBE), which has acquired the reputation of being one of the easiest ways for nontechnical computer users to access information in a database. The QBE language is demonstrated using Microsoft Office Access.

Appendix N covers the Third Generation DBMS Manifestos.

Appendix O covers Postgres, an early Object-Relational DBMS.

The logical organization of the book and the suggested paths through it are illustrated in Figure P.1.

Corrections and Suggestions

As a textbook of this size is vulnerable to errors, disagreements, omissions, and confusion, your input is solicited for future reprints and editions. Comments, corrections, and constructive suggestions should be sent to me at:

thomas.connolly@uws.ac.uk

Acknowledgments

This book is the outcome of our many years of work in industry, research, and academia. It is difficult to name everyone who has helped us in our efforts. We apologize to anyone we may happen to omit. We first give special thanks and apologies to our families, who over the years have been neglected, even ignored, during our deepest concentrations.

We would like to thank the reviewers of the previous editions of the book: William H. Gwinn, Texas Tech University; Adrian Larner, De Montfort University, Leicester; Andrew McGettrick, University of Strathclyde; Dennis McLeod, University of Southern California; Josephine DeGuzman Mendoza, California State University; Jeff Naughton, University of Oklahoma; Junping Sun, Nova Southeastern University; Donovan Young, Georgia Tech; Barry Eaglestone, University of Bradford; John Wade, IBM; Stephano Ceri, Politecnico di Milano; Lars Gillberg, Mid Sweden University, Oestersund; Dawn Jutla, St Mary's University, Halifax; Julie

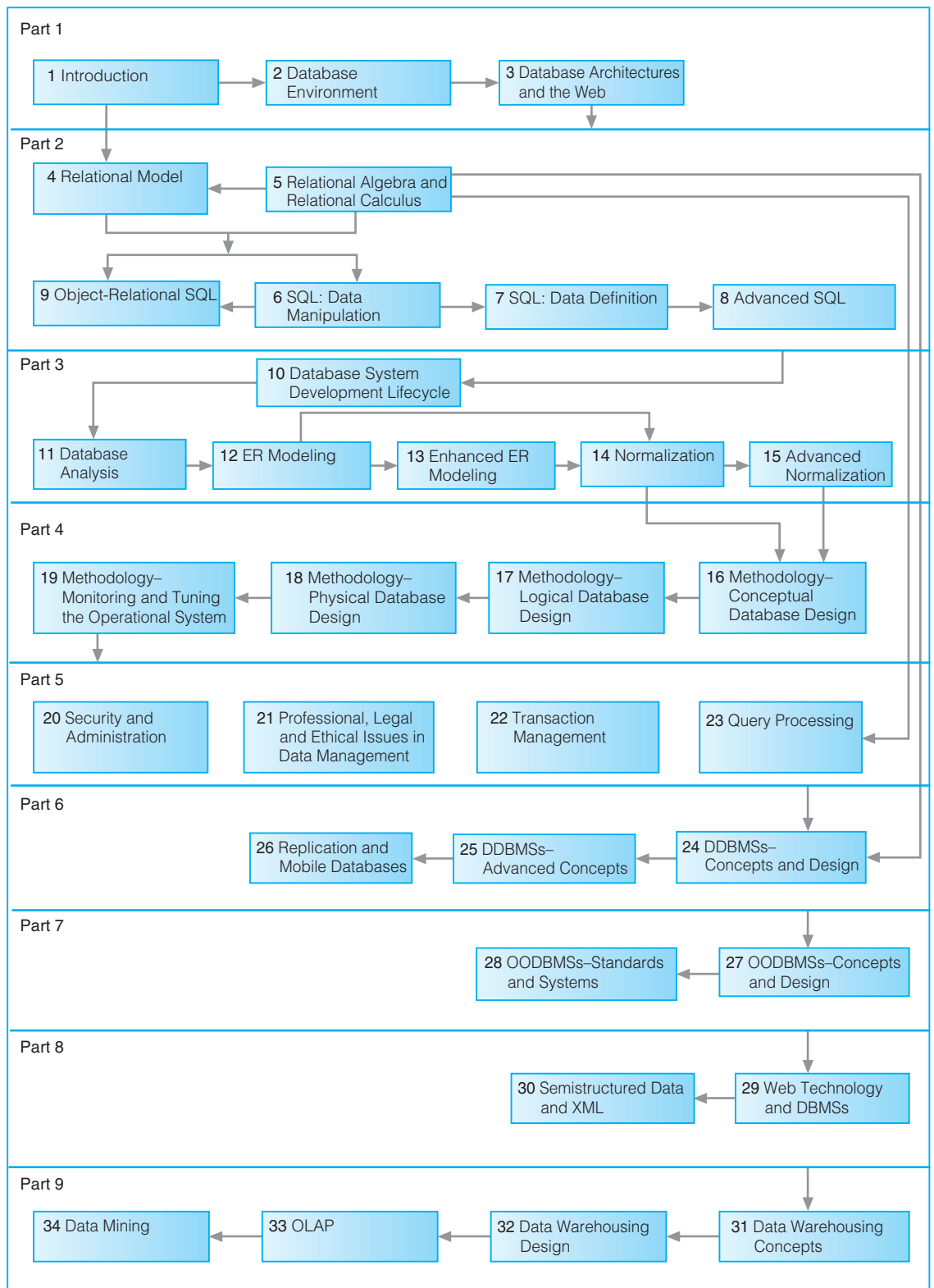


Figure P.1 Logical organization of the book and suggested paths through it.

McCann, City University, London; Munindar Singh, North Carolina State University; Hugh Darwen, Hursely, UK; Claude Delobel, Paris, France; Dennis Murray, Reading, UK; Richard Cooper, University of Glasgow; Emma Eliason, University of Orebro; Sari Hakkarainen, Stockholm University and the Royal Institute of Technology; Nenad Jukic, Loyola University Chicago; Jan Paredaens, University of Antwerp; Stephen Priest, Daniel Webster College; and from our own department, John Kawala and Peter Knaggs. Many others are still anonymous to us—we thank you for the time you must have spent on the manuscript. We would also like to acknowledge Anne Strachan for her contribution to the first edition.

I would also like to thank Kakoli Bandyopadhyay, Lamar University; Jiangping Chen, University of North Texas; Robert Donnelly, Goldey-Beacom College; Cyrus Grant, Dominican University; David G. Hendry, University of Washington; Amir Hussain, University of Stirling; Marilyn G. Kletke, Oklahoma State University; Farhi Marir, Knowledge Management Research Group, CCTM Department, London Metropolitan University; Javed Mostafa, Indiana University, Bloomington; Goran Nenadic, University of Manchester; Robert C. Nickerson, San Francisco State University; Amy Phillips, University of Denver; Pamela Smith, Lawrence Technological University.

For the sixth edition, we would like to thank Marcia Horton, our editor at Pearson, Kayla Smith-Tarbox and Marilyn Lloyd in the production team at Pearson and Vasundhara Sawhney, our project manager at Cenveo. We would also like to thank Nikos Dimitrakas, Stockholm University, Sweden; Tom Carnduff, Cardiff University, UK; David Kingston, Pittsburgh Technical Institute; Catherine Anderson, University of MD-College Park; Xumin Liu, Rochester Institute of Technology; Dr. Mohammed Yamin, Australian National U; Cahit Aybet, Izmir University of Economics; Martti Laiho, Haaga-Helia University of Applied Sciences in Finland; Fritz Laux and Tim Lessner, Reutlingen University in Germany; and Malcolm Crowe, University of the West of Scotland, UK for their contributions.

We should also like to thank Malcolm Bronte-Stewart for the *DreamHome* concept, Moira O'Donnell for ensuring the accuracy of the *Wellmeadows Hospital* case study, Alistair McMonnies, and Richard Beeby, for their help with material for the Web site.

Thomas M. Connolly
Carolyn E. Begg
Glasgow, February 2013

Pearson would like to thank and acknowledge Simon Msanjila and Mohamed Ghasia, Mzumbe University for their contributions to the Global Edition, and Preethi Subramanian, Malaysian University of Technology and Science, Vincent Ng, The Hong Kong Polytechnic University, and Issam El-Moughrabi, Gulf University for reviewing the Global Edition.

Background

Chapter 1	Introduction to Databases	51
Chapter 2	Database Environment	83
Chapter 3	Database Architectures and the Web	105

Introduction to Databases

Chapter Objectives

In this chapter you will learn:

- The importance of database systems.
- Some common uses of database systems.
- The characteristics of file-based systems.
- The problems with the file-based approach.
- The meaning of the term “database.”
- The meaning of the term “database management system” (DBMS).
- The typical functions of a DBMS.
- The major components of the DBMS environment.
- The personnel involved in the DBMS environment.
- The history of the development of DBMSs.
- The advantages and disadvantages of DBMSs.

While barely 50 years old, database research has had a profound impact on the economy and society, creating an industry sector valued at between US\$35-US\$50 billion annually. The database system, the subject of this book, is arguably the most important development in the field of software engineering, and the database is now the underlying framework of the information system, fundamentally changing the way that many organizations operate. Indeed the importance of the database system has increased in recent years with significant developments in hardware capability, hardware capacity, and communications, including the emergence of the Internet, electronic commerce, business intelligence, mobile communications, and grid computing.

Database technology has been an exciting area to work in and, since its emergence, has been the catalyst for many important developments in software engineering. Database research is not over and there are still many problems that need

to be addressed. Moreover, as the applications of database systems become even more complex we will have to rethink many of the algorithms currently being used, such as the algorithms for file storage, file access, and query optimization. These original algorithms have made significant contributions in software engineering and, without doubt, the development of new algorithms will have similar effects. In this first chapter, we introduce the database system.

“As a result of its importance, many computing and business courses cover the study of database systems. In this book, we explore a range of issues associated with the implementation and applications of database systems. We also focus on how to design a database and present a methodology that should help you design both simply and complex databases.

Structure of this Chapter In Section 1.1, we examine some uses of database systems that we find in everyday life but are not necessarily aware of. In Sections 1.2 and 1.3, we compare the early file-based approach to computerizing the manual file system with the modern, and more usable, database approach. In Section 1.4, we discuss the four types of role that people perform in the database environment, namely: data and database administrators, database designers, application developers, and end-users. In Section 1.5, we provide a brief history of database systems, and follow that in Section 1.6 with a discussion of the advantages and disadvantages of database systems.

Throughout this book, we illustrate concepts using a case study based on a fictitious property management company called *DreamHome*. We provide a detailed description of this case study in Section 11.4 and Appendix A. In Appendix B, we present further case studies that are intended to provide additional realistic projects for the reader. There will be exercises based on these case studies at the end of many chapters.



1.1 Introduction

The database is now such an integral part of our day-to-day life that often we are not aware that we are using one. To start our discussion of databases, in this section we examine some applications of database systems. For the purposes of this discussion, we consider a **database** to be a collection of related data and a **database management system (DBMS)** to be the software that manages and controls access to the database. A **database application** is simply a program that interacts with the database at some point in its execution. We also use the more inclusive term **database system** as a collection of application programs that interact with the database along with the DBMS and the database itself. We provide more accurate definitions in Section 1.3.

Purchases from the supermarket

When you purchase goods from your local supermarket, it is likely that a database is accessed. The checkout assistant uses a bar code reader to scan each of your purchases. This reader is linked to a database application that uses the bar code to find out the price of the item from a product database. The application then reduces the number of such items in stock and displays the price on the cash register. If the reorder level falls below a specified threshold, the database system may automatically place an order to obtain more of that item. If a customer telephones the supermarket, an assistant can check whether an item is in stock by running an application program that determines availability from the database.

Purchases using your credit card

When you purchase goods using your credit card, the assistant normally checks whether you have sufficient credit left to make the purchase. This check may be carried out by telephone or automatically by a card reader linked to a computer system. In either case, there is a database somewhere that contains information about the purchases that you have made using your credit card. To check your credit, there is a database application that uses your credit card number to check that the price of the goods you wish to buy, together with the sum of the purchases that you have already made this month, is within your credit limit. When the purchase is confirmed, the details of the purchase are added to this database. The database application also accesses the database to confirm that the credit card is not on the list of stolen or lost cards before authorizing the purchase. There are other database applications to send out monthly statements to each cardholder and to credit accounts when payment is received.

Booking a vacation with a travel agent

When you make inquiries about a vacation, your travel agent may access several databases containing vacation and flight details. When you book your vacation, the database system has to make all the necessary booking arrangements. In this case, the system has to ensure that two different agents do not book the same vacation or overbook the seats on the flight. For example, if there is only one seat left on the flight from New York to London and two agents try to reserve the last seat at the same time, the system has to recognize this situation, allow one booking to proceed, and inform the other agent that there are now no seats available. The travel agent may have another, usually separate, database for invoicing.

Using the local library

Your local library probably has a database containing details of the books in the library, details of the readers, reservations, and so on. There will be a computerized index that allows readers to find a book based on its title, authors, or subject area. The database system handles reservations to allow a reader to reserve a book and to be informed by mail or email when the book is available. The system also sends reminders to borrowers who have failed to return books by the due date. Typically, the system will have a bar code reader, similar to that used by the

supermarket described earlier, that is used to keep track of books coming in and going out of the library.

Taking out insurance

Whenever you wish to take out insurance—for example personal insurance, property insurance, or auto insurance, your agent may access several databases containing figures for various insurance organizations. The personal details that you supply, such as name, address, age, and whether you drink or smoke, are used by the database system to determine the cost of the insurance. An insurance agent can search several databases to find the organization that gives you the best deal.

Renting a DVD

When you wish to rent a DVD from a DVD rental company, you will probably find that the company maintains a database consisting of the DVD titles that it stocks, details on the copies it has for each title, whether the copy is available for rent or is currently on loan, details of its members (the renters), and which DVDs they are currently renting and date they are returned. The database may even store more detailed information on each DVD, such as its director and its actors. The company can use this information to monitor stock usage and predict future buying trends based on historic rental data.

Using the Internet

Many of the sites on the Internet are driven by database applications. For example, you may visit an online bookstore that allows you to browse and buy books, such as Amazon.com. The bookstore allows you to browse books in different categories, such as computing or management, or by author name. In either case, there is a database on the organization's Web server that consists of book details, availability, shipping information, stock levels, and order history. Book details include book titles, ISBNs, authors, prices, sales histories, publishers, reviews, and detailed descriptions. The database allows books to be cross-referenced: for example, a book may be listed under several categories, such as computing, programming languages, bestsellers, and recommended titles. The cross-referencing also allows Amazon to give you information on other books that are typically ordered along with the title you are interested in.

As with an earlier example, you can provide your credit card details to purchase one or more books online. Amazon.com personalizes its service for customers who return to its site by keeping a record of all previous transactions, including items purchased, shipping, and credit card details. When you return to the site, you might be greeted by name and presented with a list of recommended titles based on previous purchases.

Studying at College

If you are at college, there will be a database system containing information about yourself, your major and minor fields, the courses you are enrolled in, details about your financial aid, the classes you have taken in previous years or are taking

this year, and details of all your examination results. There may also be a database containing details relating to the next year's admissions and a database containing details of the staff working at the university, giving personal details and salary-related details for the payroll office.

These are only a few of the applications for database systems, and you will no doubt know of plenty of others. Though we now take such applications for granted, the database system is a highly complex technology that has been developed over the past 40 years. In the next section, we discuss the precursor to the database system: the file-based system.

1.2 Traditional File-Based Systems

It is almost a tradition that comprehensive database books introduce the database system with a review of its predecessor, the file-based system. We will not depart from this tradition. Although the file-based approach is largely obsolete, there are good reasons for studying it:

- Understanding the problems inherent in file-based systems may prevent us from repeating these problems in database systems. In other words, we should learn from our earlier mistakes. Actually, using the word “mistakes” is derogatory and does not give any cognizance to the work that served a useful purpose for many years. However, we have learned from this work that there are better ways to handle data.
- If you wish to convert a file-based system to a database system, understanding how the file system works will be extremely useful, if not essential.

1.2.1 File-Based Approach

File-based system

A collection of application programs that perform services for the end-users, such as the production of reports. Each program defines and manages its own data.

File-based systems were an early attempt to computerize the manual filing system that we are all familiar with. For example, an organization might have physical files set up to hold all external and internal correspondence relating to a project, product, task, client, or employee. Typically, there are many such files, and for safety they are labeled and stored in one or more cabinets. For security, the cabinets may have locks or may be located in secure areas of the building. In our own home, we probably have some sort of filing system that contains receipts, warranties, invoices, bank statements, and so on. When we need to look something up, we go to the filing system and search through the system, starting at the first entry, until we find what we want. Alternatively, we might have an indexing system that helps locate what we want more quickly. For example, we might have divisions in the filing system or separate folders for different types of items that are in some way *logically related*.

The manual filing system works well as long as the number of items to be stored is small. It even works adequately when there are large numbers of items and we only have to store and retrieve them. However, the manual filing system breaks down when we have to cross-reference or process the information in the files. For

example, a typical real estate agent's office might have a separate file for each property for sale or rent, each potential buyer and renter, and each member of staff. Consider the effort that would be required to answer the following questions:

- What three-bedroom properties do you have for sale with an acre of land and a garage?
- What apartments do you have for rent within three miles of downtown?
- What is the average rent for a two-bedroom apartment?
- What is the annual total for staff salaries?
- How does last month's net income compare with the projected figure for this month?
- What is the expected monthly net income for the next financial year?

Increasingly nowadays, clients, senior managers, and staff want more and more information. In some business sectors, there is a legal requirement to produce detailed monthly, quarterly, and annual reports. Clearly, the manual system is inadequate for this type of work. The file-based system was developed in response to the needs of industry for more efficient data access. However, rather than establish a centralized store for the organization's operational data, a decentral-ized approach was taken, where each department, with the assistance of **Data Processing (DP)** staff, stored and controlled its own data. To understand what this means, consider the *DreamHome* example.



The Sales Department is responsible for the selling and renting of properties. For example, whenever a client who wishes to offer his or her property as a rental approaches the Sales Department, a form similar to the one shown in Figure 1.1(a) is completed. The completed form contains details on the property, such as address, number of rooms, and the owner's contact information. The Sales Department also handles inquiries from clients, and a form similar to the one shown in Figure 1.1(b) is completed for each one. With the assistance of the DP Department, the Sales Department creates an information system to handle the renting of property. The system consists of three files containing property, owner, and client details, as illustrated in Figure 1.2. For simplicity, we omit details relating to members of staff, branch offices, and business owners.

The Contracts Department is responsible for handling the lease agreements associated with properties for rent. Whenever a client agrees to rent a property, a form with the client and property details is filled in by one of the sales staff, as shown in Figure 1.3. This form is passed to the Contracts Department, which allocates a lease number and completes the payment and rental period details. Again, with the assistance of the DP Department, the Contracts Department creates an information system to handle lease agreements. The system consists of three files that store lease, property, and client details, and that contain similar data to that held by the Sales Department, as illustrated in Figure 1.4

The process is illustrated in Figure 1.5. It shows each department accessing their own files through application programs written especially for them. Each set of departmental application programs handles data entry, file maintenance, and the generation of a fixed set of specific reports. More important, the physical structure and storage of the data files and records are defined in the application code.

DreamHome Property for Rent Details Property Number: PG21	Allocated to Branch: 163 Main St, Glasgow Branch No. B003 Address 18 Dale Rd City Glasgow Postcode G12 Type House Rent 600 No. of Rooms 5 Staff Responsible Ann Beech
DreamHome Client Details Client Number: CR74	First Name Mike Last Name Ritchie Address 18 Tain St PA1G 1YQ Tel. No. 01475-392178 Property Requirement Details Preferred Property Type House Maximum Monthly Rent 750 General Comments Currently living at home with parents Getting married in August Seen By Ann Beech Date 24-Mar-13 Branch No. B003 Branch City Glasgow

Name

Carol Farrel

Address

6 Achray St

Glasgow G32 9DX

Tel. No.

0141-357-7419

Owner No.

C087

Business Name

Address

Tel. No.

Owner No.

Contact Name

Business Type

(a)

Figure I.1 Sales Department forms; (a) Property for Rent Details form; (b) Client Details form.

Lease

leaseNo	propertyNo	clientNo	rent	payment Method	deposit	paid	rentStart	rentFinish	duration
10024	PA14	CR62	650	Visa	1300	Y	1-Jun-13	31-May-14	12
10075	PL94	CR76	400	Cash	800	N	1-Aug-13	31-Jan-14	6
10012	PG21	CR74	600	Cheque	1200	Y	1-Jul-13	30-Jun-14	12

PropertyForRent

propertyNo	street	city	postcode	rent
PA14	16 Holhead	Aberdeen	AB7 5SU	650
PL94	6 Argyll St	London	NW2	400
PG21	18 Dale Rd	Glasgow	G12	600

Client

clientNo	fName	lName	address	telNo
CR76	John	Kay	56 High St, London SW1 4EH	0171-774-5632
CR74	Mike	Ritchie	18 Tain St, PA1G 1YQ	01475-392178
CR62	Mary	Tregear	5 Tarbot Rd, Aberdeen AB9 3ST	01224-196720

Figure 1.4
The Lease, PropertyForRent, and Client files used by the Contracts Department.

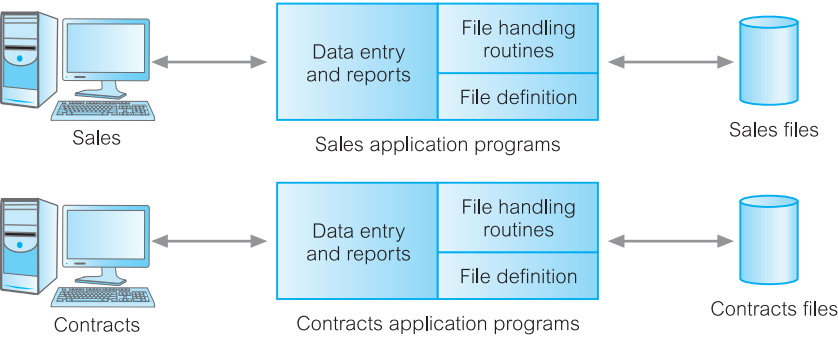


Figure 1.5
File-based processing.

Sales Files

- PropertyForRent** (propertyNo, street, city, postcode, type, rooms, rent, ownerNo)
- PrivateOwner** (ownerNo, fName, lName, address, telNo)
- Client** (clientNo, fName, lName, address, telNo, prefType, maxRent)

Contracts Files

- Lease** (leaseNo, propertyNo, clientNo, rent, paymentMethod, deposit, paid, rentStart, rentFinish, duration)
- PropertyForRent** (propertyNo, street, city, postcode, rent)
- Client** (clientNo, fName, lName, address, telNo)

We can find similar examples in other departments. For example, the Payroll Department stores details relating to each member of staff's salary:

StaffSalary(staffNo, fName, lName, sex, salary, branchNo)

The Human Resources (HR) Department also stores staff details:

Staff(staffNo, fName, lName, position, sex, dateOfBirth, salary, branchNo)

It can be seen quite clearly that there is a significant amount of duplication of data in these departments, and this is generally true of file-based systems. Before we discuss the limitations of this approach, it may be useful to understand the terminology used in file-based systems. A file is simply a collection of **records**, which contains **logically related data**. For example, the `PropertyForRent` file in Figure 1.2 contains six records, one for each property. Each record contains a logically connected set of one or more **fields**, where each field represents some characteristic of the real-world object that is being modeled. In Figure 1.2, the fields of the `PropertyForRent` file represent characteristics of properties, such as address, property type, and number of rooms.

1.2.2 Limitations of the File-Based Approach

This brief description of traditional file-based systems should be sufficient to discuss the limitations of this approach. We list five problems in Table 1.1.

Separation and isolation of data

When data is isolated in separate files, it is more difficult to access data that should be available. For example, if we want to produce a list of all houses that match the requirements of clients, we first need to create a temporary file of those clients who have “house” as the preferred type. We then search the `PropertyForRent` file for those properties where the property type is “house” and the rent is less than the client’s maximum rent. With file systems, such processing is difficult. The application developer must synchronize the processing of two files to ensure that the correct data is extracted. This difficulty is compounded if we require data from more than two files.

Duplication of data

Owing to the decentralized approach taken by each department, the file-based approach encouraged, if not necessitated, the uncontrolled duplication of data. For example, in Figure 1.5 we can clearly see that there is duplication of both property and client details in the Sales and Contracts Departments. Uncontrolled duplication of data is undesirable for several reasons, including:

- Duplication is wasteful. It costs time and money to enter the data more than once.
- It takes up additional storage space, again with associated costs. Often, the duplication of data can be avoided by sharing data files.

TABLE 1.1 Limitations of file-based systems.

Separation and isolation of data
Duplication of data
Data dependence
Incompatible file formats
Fixed queries/proliferation of application programs

- Perhaps more importantly, duplication can lead to loss of data integrity; in other words, the data is no longer consistent. For example, consider the duplication of data between the Payroll and HR Departments described previously. If a member of staff moves and the change of address is communicated only to HR and not to Payroll, the person's paycheck will be sent to the wrong address. A more serious problem occurs if an employee is promoted with an associated increase in salary. Again, let's assume that the change is announced to HR, but the change does not filter through to Payroll. Now, the employee is receiving the wrong salary. When this error is detected, it will take time and effort to resolve. Both these examples illustrate inconsistencies that may result from the duplication of data. As there is no automatic way for HR to update the data in the Payroll files, it is not difficult to foresee such inconsistencies arising. Even if Payroll is notified of the changes, it is possible that the data will be entered incorrectly.

Data dependence

As we have already mentioned, the physical structure and storage of the data files and records are defined in the application code. This means that changes to an existing structure are difficult to make. For example, increasing the size of the `PropertyForRent` address field from 40 to 41 characters sounds like a simple change, but it requires the creation of a one-off program (that is, a program that is run only once and can then be discarded) that converts the `PropertyForRent` file to the new format. This program has to:

- Open the original `PropertyForRent` file for reading
- Open a temporary file with the new structure
- Read a record from the original file, convert the data to conform to the new structure, and write it to the temporary file, then repeat this step for all records in the original file
- Delete the original `PropertyForRent` file
- Rename the temporary file as `PropertyForRent`

In addition, all programs that access the `PropertyForRent` file must be modified to conform to the new file structure. There might be many such programs that access the `PropertyForRent` file. Thus, the programmer needs to identify all the affected programs, modify them, and then retest them. Note that a program does not even have to use the address field to be affected: it only has to use the `PropertyForRent` file. Clearly, this process could be very time-consuming and subject to error. This characteristic of file-based systems is known as **program–data dependence**.

Incompatible file formats

Because the structure of files is embedded in the application programs, the structures are dependent on the application programming language. For example, the structure of a file generated by a COBOL program may be different from the structure of a file generated by a C program. The direct incompatibility of such files makes them difficult to process jointly.

For example, suppose that the Contracts Department wants to find the names and addresses of all owners whose property is currently rented out. Unfortunately,

Contracts does not hold the details of property owners; only the Sales Department holds these. However, Contracts has the property number (`propertyNo`), which can be used to find the corresponding property number in the Sales Department's `PropertyForRent` file. This file holds the owner number (`ownerNo`), which can be used to find the owner details in the `PrivateOwner` file. The Contracts Department programs in COBOL and the Sales Department programs in C. Therefore, matching `propertyNo` fields in the two `PropertyForRent` files requires that an application developer write software to convert the files to some common format to facilitate processing. Again, this process can be time-consuming and expensive.

Fixed queries/proliferation of application programs

From the end-user's point of view, file-based systems were a great improvement over manual systems. Consequently, the requirement for new or modified queries grew. However, file-based systems are very dependent upon the application developer, who has to write any queries or reports that are required. As a result, two things happened. In some organizations, the type of query or report that could be produced was fixed. There was no facility for asking unplanned (that is, spur-of-the-moment or *ad hoc*) queries either about the data itself or about which types of data were available.

In other organizations, there was a proliferation of files and application programs. Eventually, this reached a point where the DP Department, with its existing resources, could not handle all the work. This put tremendous pressure on the DP staff, resulting in programs that were inadequate or inefficient in meeting the demands of the users, limited documentation, and difficult maintenance. Often, certain types of functionality were omitted:

- There was no provision for security or integrity.
- Recovery, in the event of a hardware or software failure, was limited or nonexistent.
- Access to the files was restricted to one user at a time—there was no provision for shared access by staff in the same department.

In either case, the outcome was unacceptable. Another solution was required.

1.3 Database Approach

All of the previously mentioned limitations of the file-based approach can be attributed to two factors:

- (1) The definition of the data is embedded in the application programs, rather than being stored separately and independently.
- (2) There is no control over the access and manipulation of data beyond that imposed by the application programs.

To become more effective, a new approach was required. What emerged were the database and the Database Management System (DBMS). In this section, we provide a more formal definition of these terms, and examine the components that we might expect in a DBMS environment.

1.3.1 The Database

Database

A shared collection of logically related data and its description, designed to meet the information needs of an organization.

We now examine the definition of a database so that you can understand the concept fully. The database is a single, possibly large repository of data that can be used simultaneously by many departments and users. Instead of disconnected files with redundant data, all data items are integrated with a minimum amount of duplication. The database is no longer owned by one department but is a shared corporate resource. The database holds not only the organization's operational data, but also a description of this data. For this reason, a database is also defined as a *self-describing collection of integrated records*. The description of the data is known as the **system catalog** (or **data dictionary** or **metadata**—the “data about data”). It is the self-describing nature of a database that provides program–data independence.

The approach taken with database systems, where the definition of data is separated from the application programs, is similar to the approach taken in modern software development, where an internal definition of an object and a separate external definition are provided. The users of an object see only the external definition and are unaware of how the object is defined and how it functions. One advantage of this approach, known as **data abstraction**, is that we can change the internal definition of an object without affecting the users of the object, provided that the external definition remains the same. In the same way, the database approach separates the structure of the data from the application programs and stores it in the database. If new data structures are added or existing structures are modified, then the application programs are unaffected, provided that they do not directly depend upon what has been modified. For example, if we add a new field to a record or create a new file, existing applications are unaffected. However, if we remove a field from a file that an application program uses, then that application program is affected by this change and must be modified accordingly.

Another expression in the definition of a database that we should explain is “logically related.” When we analyze the information needs of an organization, we attempt to identify entities, attributes, and relationships. An **entity** is a distinct object (a person, place, thing, concept, or event) in the organization that is to be represented in the database. An **attribute** is a property that describes some aspect of the object that we wish to record, and a **relationship** is an association between entities. For example, Figure 1.6 shows an Entity–Relationship (ER) diagram for part of the *DreamHome* case study. It consists of:

- six entities (the rectangles): Branch, Staff, PropertyForRent, Client, PrivateOwner, and Lease;
- seven relationships (the names adjacent to the lines): *Has*, *Offers*, *Oversees*, *Views*, *Owns*, *LeasedBy*, and *Holds*;
- six attributes, one for each entity: *branchNo*, *staffNo*, *propertyNo*, *clientNo*, *ownerNo*, and *leaseNo*.



The database represents the entities, the attributes, and the logical relationships between the entities. In other words, the database holds data that is logically related. We discuss the ER model in detail in Chapters 12 and 13.

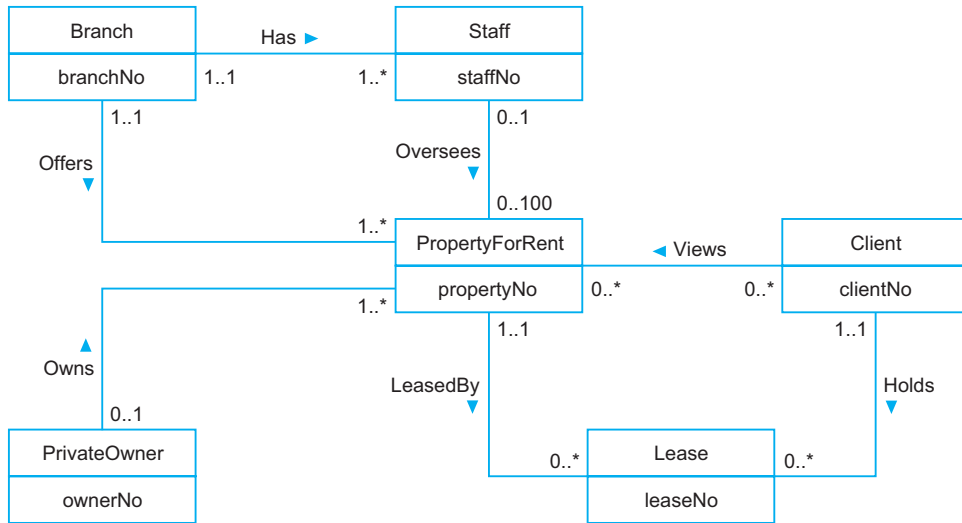


Figure 1.6 Example Entity–Relationship diagram.

1.3.2 The Database Management System (DBMS)

DBMS

A software system that enables users to define, create, maintain, and control access to the database.

The DBMS is the software that interacts with the users’ application programs and the database. Typically, a DBMS provides the following facilities:

- It allows users to define the database, usually through a **Data Definition Language (DDL)**. The DDL allows users to specify the data types and structures and the constraints on the data to be stored in the database.
- It allows users to insert, update, delete, and retrieve data from the database, usually through a **Data Manipulation Language (DML)**. Having a central repository for all data and data descriptions allows the DML to provide a general inquiry facility to this data, called a **query language**. The provision of a query language alleviates the problems with file-based systems where the user has to work with a fixed set of queries or there is a proliferation of programs, causing major software management problems. The most common query language is the **Structured Query Language (SQL)**, pronounced “S-Q-L”, or sometimes “See-Quel”), which is now both the formal and de facto standard language for relational DBMSs. To emphasize the importance of SQL, we devote Chapters 6–9 and Appendix I to a comprehensive study of this language.
- It provides controlled access to the database. For example, it may provide:
 - a security system, which prevents unauthorized users accessing the database;
 - an integrity system, which maintains the consistency of stored data;
 - a concurrency control system, which allows shared access of the database;
 - a recovery control system, which restores the database to a previous consistent state following a hardware or software failure;
 - a user-accessible catalog, which contains descriptions of the data in the database.

1.3.3 (Database) Application Programs

Application Programs

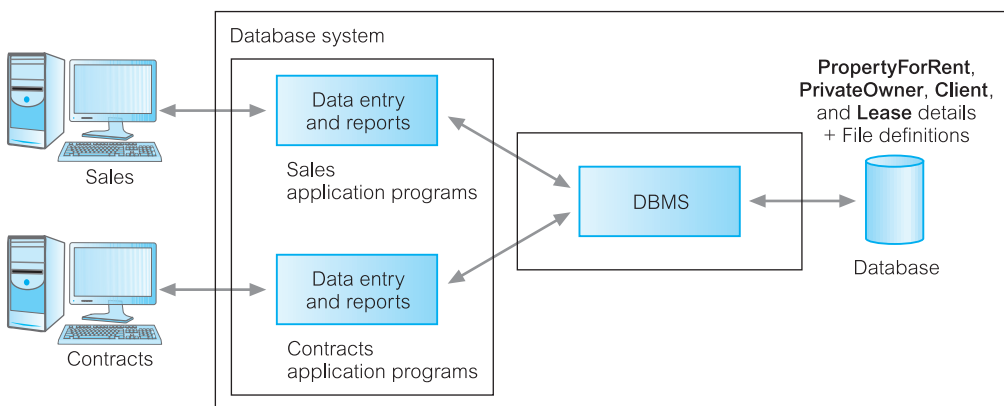
A computer program that interacts with the database by issuing an appropriate request (typically an SQL statement) to the DBMS.

Users interact with the database through a number of **application programs** that are used to create and maintain the database and to generate information. These programs can be conventional batch applications or, more typically nowadays, online applications. The application programs may be written in a programming language or in higher-level fourth-generation language.

The database approach is illustrated in Figure 1.7, based on the file approach of Figure 1.5. It shows the Sales and Contracts Departments using their application programs to access the database through the DBMS. Each set of departmental application programs handles data entry, data maintenance, and the generation of reports. However, as opposed to the file-based approach, the physical structure and storage of the data are now managed by the DBMS.

Views

With this functionality, the DBMS is an extremely powerful and useful tool. However, as the end-users are not too interested in how complex or easy a task is for the system, it could be argued that the DBMS has made things more complex, because they now see more data than they actually need or want. For example, the details that the Contracts Department wants to see for a rental property, as shown in Figure 1.5, have changed in the database approach, shown in Figure 1.7. Now the database also holds the property type, the number of rooms, and the owner details. In recognition of this problem, a DBMS provides another facility known as a **view mechanism**, which allows each user to have his or her own view of the



PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo)

PrivateOwner (ownerNo, fName, lName, address, telNo)

Client (clientNo, fName, lName, address, telNo, prefType, maxRent)

Lease (leaseNo, propertyNo, clientNo, paymentMethod, deposit, paid, rentStart, rentFinish)

Figure 1.7 Database processing.

database (a **view** is, in essence, some subset of the database). For example, we could set up a view that allows the Contracts Department to see only the data that they want to see for rental properties.

As well as reducing complexity by letting users see the data in the way they want to see it, views have several other benefits:

- *Views provide a level of security.* Views can be set up to exclude data that some users should not see. For example, we could create a view that allows a branch manager and the Payroll Department to see all staff data, including salary details, and we could create a second view that other staff would use that excludes salary details.
- *Views provide a mechanism to customize the appearance of the database.* For example, the Contracts Department may wish to call the monthly rent field (**rent**) by the more obvious name, **Monthly Rent**.
- *A view can present a consistent, unchanging picture of the structure of the database,* even if the underlying database is changed (for example, fields added or removed, relationships changed, files split, restructured, or renamed). If fields are added or removed from a file, and these fields are not required by the view, the view is not affected by this change. Thus, a view helps provide the program–data independence we mentioned previously.

The previous discussion is general and the actual level of functionality offered by a DBMS differs from product to product. For example, a DBMS for a personal computer may not support concurrent shared access, and may provide only limited security, integrity, and recovery control. However, modern, large multi-user DBMS products offer all the functions mentioned and much more. Modern systems are extremely complex pieces of software consisting of millions of lines of code, with documentation comprising many volumes. This complexity is a result of having to provide software that handles requirements of a more general nature. Furthermore, the use of DBMSs nowadays requires a system that provides almost total reliability and 24/7 availability (24 hours a day, 7 days a week), even in the presence of hardware or software failure. The DBMS is continually evolving and expanding to cope with new user requirements. For example, some applications now require the storage of graphic images, video, sound, and so on. To reach this market, the DBMS must change. It is likely that new functionality will always be required, so the functionality of the DBMS will never become static. We discuss the basic functions provided by a DBMS in later chapters.

1.3.4 Components of the DBMS Environment

We can identify five major components in the DBMS environment: hardware, software, data, procedures, and people, as illustrated in Figure 1.8.

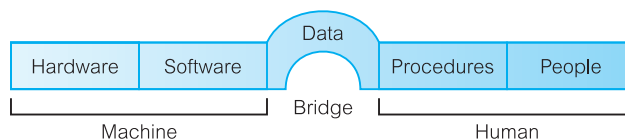


Figure 1.8 The DBMS environment.

Hardware

The DBMS and the applications require hardware to run. The hardware can range from a single personal computer to a single mainframe or a network of computers. The particular hardware depends on the organization's requirements and the DBMS used. Some DBMSs run only on particular hardware or operating systems, while others run on a wide variety of hardware and operating systems. A DBMS requires a minimum amount of main memory and disk space to run, but this minimum configuration may not necessarily give acceptable performance. A simplified hardware configuration for *DreamHome* is illustrated in Figure 1.9. It consists of a network of small servers, with a central server located in London running the **backend** of the DBMS, that is, the part of the DBMS that manages and controls access to the database. It also shows several computers at various locations running the **frontend** of the DBMS, that is, the part of the DBMS that interfaces with the user. This is called a **client-server** architecture: the backend is the server and the frontends are the clients. We discuss this type of architecture in Section 3.1.

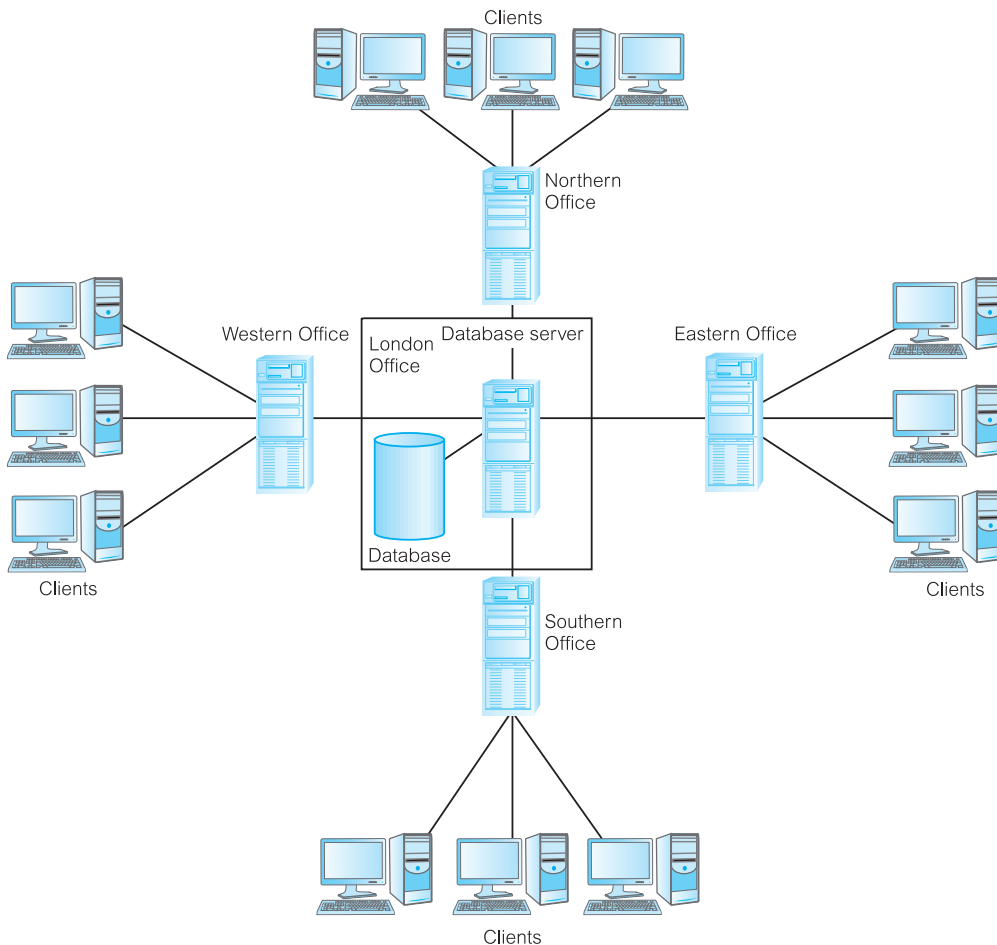


Figure 1.9 *DreamHome* hardware configuration.

Software

The software component comprises the DBMS software itself and the application programs, together with the operating system, including network software if the DBMS is being used over a network. Typically, application programs are written in a third-generation programming language (3GL), such as C, C++, C#, Java, Visual Basic, COBOL, Fortran, Ada, or Pascal, or a fourth-generation language (4GL), such as SQL, embedded in a third-generation language. The target DBMS may have its own fourth-generation tools that allow rapid development of applications through the provision of nonprocedural query languages, reports generators, forms generators, graphics generators, and application generators. The use of fourth-generation tools can improve productivity significantly and produce programs that are easier to maintain. We discuss fourth-generation tools in Section 2.2.3.

Data

Perhaps the most important component of the DBMS environment—certainly from the end-users' point of view—is the data. In Figure 1.8, we observe that the data acts as a bridge between the machine components and the human components. The database contains both the operational data and the metadata, the “data about data.” The structure of the database is called the **schema**. In Figure 1.7, the schema consists of four files, or **tables**, namely: PropertyForRent, PrivateOwner, Client, and Lease. The PropertyForRent table has eight fields, or **attributes**, namely: propertyNo, street, city, zipCode, type (the property type), rooms (the number of rooms), rent (the monthly rent), and ownerNo. The ownerNo attribute models the relationship between PropertyForRent and PrivateOwner: that is, an owner *Owns* a property for rent, as depicted in the ER diagram of Figure 1.6. For example, in Figure 1.2 we observe that owner CO46, Joe Keogh, owns property PA14.

The data also incorporates the system catalog, which we discuss in detail in Section 2.4.

Procedures

Procedures refer to the instructions and rules that govern the design and use of the database. The users of the system and the staff who manage the database require documented procedures on how to use or run the system. These may consist of instructions on how to:

- Log on to the DBMS.
- Use a particular DBMS facility or application program.
- Start and stop the DBMS.
- Make backup copies of the database.
- Handle hardware or software failures. This may include procedures on how to identify the failed component, how to fix the failed component (for example, telephone the appropriate hardware engineer), and, following the repair of the fault, how to recover the database.
- Change the structure of a table, reorganize the database across multiple disks, improve performance, or archive data to secondary storage.

People

The final component is the people involved with the system. We discuss this component in Section 1.4.

1.3.5 Database Design: The Paradigm Shift

Until now, we have taken it for granted that there is a structure to the data in the database. For example, we have identified four tables in Figure 1.7: *PropertyForRent*, *PrivateOwner*, *Client*, and *Lease*. But how did we get this structure? The answer is quite simple: the structure of the database is determined during **database design**. However, carrying out database design can be extremely complex. To produce a system that will satisfy the organization's information needs requires a different approach from that of file-based systems, where the work was driven by the application needs of individual departments. For the database approach to succeed, the organization now has to think of the data first and the application second. This change in approach is sometimes referred to as a *paradigm shift*. For the system to be acceptable to the end-users, the database design activity is crucial. A poorly designed database will generate errors that may lead to bad decisions, which may have serious repercussions for the organization. On the other hand, a well-designed database produces a system that provides the correct information for the decision-making process to succeed in an efficient way.

The objective of this book is to help effect this paradigm shift. We devote several chapters to the presentation of a complete methodology for database design (see Chapters 16–19). It is presented as a series of simple-to-follow steps, with guidelines provided throughout. For example, in the ER diagram of Figure 1.6, we have identified six entities, seven relationships, and six attributes. We provide guidelines to help identify the entities, attributes, and relationships that have to be represented in the database.

Unfortunately, database design methodologies are not very popular. Many organizations and individual designers rely very little on methodologies for conducting the design of databases, and this is commonly considered a major cause of failure in the development of database systems. Owing to the lack of structured approaches to database design, the time or resources required for a database project are typically underestimated, the databases developed are inadequate or inefficient in meeting the demands of applications, documentation is limited, and maintenance is difficult.

1.4 Roles in the Database Environment

In this section, we examine what we listed in the previous section as the fifth component of the DBMS environment: the **people**. We can identify four distinct types of people who participate in the DBMS environment: data and database administrators, database designers, application developers, and end-users.

1.4.1 Data and Database Administrators

The database and the DBMS are corporate resources that must be managed like any other resource. Data and database administration are the roles generally associated with the management and control of a DBMS and its data. The

Data Administrator (DA) is responsible for the management of the data resource, including database planning; development and maintenance of standards, policies and procedures; and conceptual/logical database design. The DA consults with and advises senior managers, ensuring that the direction of database development will ultimately support corporate objectives.

The **Database Administrator (DBA)** is responsible for the physical realization of the database, including physical database design and implementation, security and integrity control, maintenance of the operational system, and ensuring satisfactory performance of the applications for users. The role of the DBA is more technically oriented than the role of the DA, requiring detailed knowledge of the target DBMS and the system environment. In some organizations there is no distinction between these two roles; in others, the importance of the corporate resources is reflected in the allocation of teams of staff dedicated to each of these roles. We discuss data and database administration in more detail in Section 20.15.

1.4.2 Database Designers

In large database design projects, we can distinguish between two types of designer: logical database designers and physical database designers. The **logical database designer** is concerned with identifying the data (that is, the entities and attributes), the relationships between the data, and the constraints on the data that is to be stored in the database. The logical database designer must have a thorough and complete understanding of the organization's data and any constraints on this data (the constraints are sometimes called **business rules**). These constraints describe the main characteristics of the data as viewed by the organization. Examples of constraints for *DreamHome* are:



- a member of staff cannot manage more than 100 properties for rent or sale at the same time;
- a member of staff cannot handle the sale or rent of his or her own property;
- a solicitor cannot act for both the buyer and seller of a property.

To be effective, the logical database designer must involve all prospective database users in the development of the data model, and this involvement should begin as early in the process as possible. In this book, we split the work of the logical database designer into two stages:

- conceptual database design, which is independent of implementation details, such as the target DBMS, application programs, programming languages, or any other physical considerations;
- logical database design, which targets a specific data model, such as relational, network, hierarchical, or object-oriented.

The **physical database designer** decides how the logical database design is to be physically realized. This involves:

- mapping the logical database design into a set of tables and integrity constraints;
- selecting specific storage structures and access methods for the data to achieve good performance;
- designing any security measures required on the data.

Many parts of physical database design are highly dependent on the target DBMS, and there may be more than one way of implementing a mechanism. Consequently, the physical database designer must be fully aware of the functionality of the target DBMS and must understand the advantages and disadvantages of each alternative implementation. The physical database designer must be capable of selecting a suitable storage strategy that takes account of usage. Whereas conceptual and logical database design are concerned with the *what*, physical database design is concerned with the *how*. It requires different skills, which are often found in different people. We present a methodology for conceptual database design in Chapter 16, for logical database design in Chapter 17, and for physical database design in Chapters 18 and 19.

1.4.3 Application Developers

Once the database has been implemented, the application programs that provide the required functionality for the end-users must be implemented. This is the responsibility of the **application developers**. Typically, the application developers work from a specification produced by systems analysts. Each program contains statements that request the DBMS to perform some operation on the database, which includes retrieving data, inserting, updating, and deleting data. The programs may be written in a third-generation or fourth-generation programming language, as discussed previously.

1.4.4 End-Users

The end-users are the “clients” of the database, which has been designed and implemented and is being maintained to serve their information needs. End-users can be classified according to the way they use the system:

- **Naïve users** are typically unaware of the DBMS. They access the database through specially written application programs that attempt to make the operations as simple as possible. They invoke database operations by entering simple commands or choosing options from a menu. This means that they do not need to know anything about the database or the DBMS. For example, the checkout assistant at the local supermarket uses a bar code reader to find out the price of the item. However, there is an application program present that reads the bar code, looks up the price of the item in the database, reduces the database field containing the number of such items in stock, and displays the price on the till.
- **Sophisticated users.** At the other end of the spectrum, the sophisticated end-user is familiar with the structure of the database and the facilities offered by the DBMS. Sophisticated end-users may use a high-level query language such as SQL to perform the required operations. Some sophisticated end-users may even write application programs for their own use.

1.5 History of Database Management Systems

We have already seen that the predecessor to the DBMS was the file-based system. However, there was never a time when the database approach began and the file-based system ceased. In fact, the file-based system still exists in specific areas. It has been suggested that the DBMS has its roots in the 1960s Apollo

moon-landing project, which was initiated in response to President Kennedy's objective of landing a man on the moon by the end of that decade. At that time, there was no system available that would be able to handle and manage the vast amounts of information that the project would generate.

As a result, North American Aviation (NAA, now Rockwell International), the prime contractor for the project, developed software known as **GUAM** (for Generalized Update Access Method). GUAM was based on the concept that smaller components come together as parts of larger components, and so on, until the final product is assembled. This structure, which conforms to an upside-down tree, is also known as a **hierarchical structure**. In the mid-1960s, IBM joined NAA to develop GUAM into what is now known as **IMS** (for Information Management System). The reason that IBM restricted IMS to the management of hierarchies of records was to allow the use of serial storage devices; most notably magnetic tape, which was a market requirement at that time. This restriction was subsequently dropped. Although one of the earliest commercial DBMSs, IMS is still the main hierarchical DBMS used by most large mainframe installations.

In the mid-1960s, another significant development was the emergence of **IDS** (or Integrated Data Store) from General Electric. This work was headed by one of the early pioneers of database systems, Charles Bachmann. This development led to a new type of database system known as the **network DBMS**, which had a profound effect on the information systems of that generation. The network database was developed partly to address the need to represent more complex data relationships than could be modeled with hierarchical structures, and partly to impose a database standard. To help establish such standards, the Conference on Data Systems Languages (**CODASYL**), comprising representatives of the U.S. government and the world of business and commerce, formed a List Processing Task Force in 1965, subsequently renamed the **Data Base Task Group (DBTG)** in 1967. The terms of reference for the DBTG were to define standard specifications for an environment that would allow database creation and data manipulation. A draft report was issued in 1969, and the first definitive report was issued in 1971. The DBTG proposal identified three components:

- the network **schema**—the logical organization of the entire database as seen by the DBA—which includes a definition of the database name, the type of each record, and the components of each record type;
- the **subschema**—the part of the database as seen by the user or application program;
- a data management language to define the data characteristics and the data structure, and to manipulate the data.

For standardization, the DBTG specified three distinct languages:

- a schema DDL, which enables the DBA to define the schema;
- a subschema DDL, which allows the application programs to define the parts of the database they require;
- a DML, to manipulate the data.

Although the report was not formally adopted by the American National Standards Institute (ANSI), a number of systems were subsequently developed following the DBTG proposal. These systems are now known as CODASYL or DBTG systems. The CODASYL and hierarchical approaches represented the **first generation** of DBMSs.

We look more closely at these systems on the Web site for this book (see the Preface for the URL). However, these two models have some fundamental disadvantages:

- complex programs have to be written to answer even simple queries based on navigational record-oriented access;
- there is minimal data independence;
- there is no widely accepted theoretical foundation.

In 1970, E. F. Codd of the IBM Research Laboratory produced his highly influential paper on the relational data model (“A relational model of data for large shared data banks,” Codd, 1970). This paper was very timely and addressed the disadvantages of the former approaches. Many experimental relational DBMSs were implemented thereafter, with the first commercial products appearing in the late 1970s and early 1980s. Of particular note is the System R project at IBM’s San José Research Laboratory in California, which was developed during the late 1970s (Astrahan et al., 1976). This project was designed to prove the practicality of the relational model by providing an implementation of its data structures and operations, and led to two major developments:

- the development of a structured query language called SQL, which has since become the standard language for relational DBMSs;
- the production of various commercial relational DBMS products during the 1980s, for example DB2 and SQL/DS from IBM and Oracle from Oracle Corporation.

Now there are several hundred relational DBMSs for both mainframe and PC environments, though many are stretching the definition of the relational model. Other examples of multi-user relational DBMSs are MySQL from Oracle Corporation, Ingres from Actian Corporation, SQL Server from Microsoft, and Informix from IBM. Examples of PC-based relational DBMSs are Office Access and Visual FoxPro from Microsoft, Interbase from Embarcadero Technologies, and R:Base from R:Base Technologies. Relational DBMSs are referred to as **second-generation DBMSs**. We discuss the relational data model in Chapter 4.

The relational model is not without its failings; in particular, its limited modeling capabilities. There has been much research since then attempting to address this problem. In 1976, Chen presented the Entity–Relationship model, which is now a widely accepted technique for database design and the basis for the methodology presented in Chapters 16 and 17 of this book. In 1979, Codd himself attempted to address some of the failings in his original work with an extended version of the relational model called RM/T (1979) and subsequently RM/V2 (1990). The attempts to provide a data model that represents the “real world” more closely have been loosely classified as **semantic data modeling**.

In response to the increasing complexity of database applications, two “new” systems have emerged: the **object-oriented DBMS (OODBMS)** and the **object-relational DBMS (ORDBMS)**. However, unlike previous models, the actual composition of these models is not clear. This evolution represents **third-generation DBMSs**, which we discuss in Chapters 9 and 27–28.

The 1990s also saw the rise of the Internet, the three-tier client-server architecture, and the demand to allow corporate databases to be integrated with Web applications. The late 1990s saw the development of XML (eXtensible Markup Language), which has had a profound effect on many aspects of IT, including

database integration, graphical interfaces, embedded systems, distributed systems, and database systems. We will discuss Web–database integration and XML in Chapters 29–30.

Specialized DBMSs have also been created, such as **data warehouses**, which can store data drawn from several data sources, possibly maintained by different operating units of an organization. Such systems provide comprehensive data analysis facilities to allow strategic decisions to be made based on, for example, historical trends. All the major DBMS vendors provide data warehousing solutions. We discuss data warehouses in Chapters 31–32. Another example is the **enterprise resource planning (ERP)** system, an application layer built on top of a DBMS that integrates all the business functions of an organization, such as manufacturing, sales, finance, marketing, shipping, invoicing, and human resources. Popular ERP systems are SAP R/3 from SAP and PeopleSoft from Oracle. Figure 1.10 provides a summary of historical development of database systems.

TIMEFRAME	DEVELOPMENT	COMMENTS
1960s (onwards)	File-based systems	Precursor to the database system. Decentralized approach: each department stored and controlled its own data.
Mid-1960s	Hierarchical and network data models	Represents first-generation DBMSs. Main hierarchical system is IMS from IBM and the main network system is IDMS/R from Computer Associates. Lacked data independence and required complex programs to be developed to process the data.
1970	Relational model proposed	Publication of E. F. Codd's seminal paper "A relational model of data for large shared data banks," which addresses the weaknesses of first-generation systems.
1970s	Prototype RDBMSs developed	During this period, two main prototypes emerged: the Ingres project at the University of California at Berkeley (started in 1970) and the System R project at IBM's San José Research Laboratory in California (started in 1974), which led to the development of SQL.
1976	ER model proposed	Publication of Chen's paper "The Entity-Relationship model—Toward a unified view of data." ER modeling becomes a significant component in methodologies for database design.
1979	Commercial RDBMSs appear	Commercial RDBMSs like Oracle, Ingres, and DB2 appear. These represent the second generation of DBMSs.
1987	ISO SQL standard	SQL is standardized by the ISO (International Standards Organization). There are subsequent releases of the standard in 1989, 1992 (SQL2), 1999 (SQL:1999), 2003 (SQL:2003), 2008 (SQL:2008), and 2011 (SQL:2011).
1990s	OODBMS and ORDBMSs appear	This period initially sees the emergence of OODBMSs and later ORDBMSs (Oracle 8, with object features released in 1997).
1990s	Data warehousing systems appear	This period also see releases from the major DBMS vendors of data warehousing systems and thereafter data mining products.
Mid-1990s	Web–database integration	The first Internet database applications appear. DBMS vendors and third-party vendors recognize the significance of the Internet and support web–database integration.
1998	XML	XML 1.0 ratified by the W3C. XML becomes integrated with DBMS products and native XML databases are developed.

Figure 1.10 Historical development of database systems.

1.6 Advantages and Disadvantages of DBMSs

The database management system has promising potential advantages. Unfortunately, there are also disadvantages. In this section, we examine these advantages and disadvantages.

Advantages

The advantages of database management systems are listed in Table 1.2.

Control of data redundancy As we discussed in Section 1.2, traditional file-based systems waste space by storing the same information in more than one file. For example, in Figure 1.5, we stored similar data for properties for rent and clients in both the Sales and Contracts Departments. In contrast, the database approach attempts to eliminate the redundancy by integrating the files so that multiple copies of the same data are not stored. However, the database approach does not eliminate redundancy entirely, but controls the amount of redundancy inherent in the database. Sometimes it is necessary to duplicate key data items to model relationships; at other times, it is desirable to duplicate some data items to improve performance. The reasons for controlled duplication will become clearer as you read the next few chapters.

Data consistency By eliminating or controlling redundancy, we reduce the risk of inconsistencies occurring. If a data item is stored only once in the database, any update to its value has to be performed only once and the new value is available immediately to all users. If a data item is stored more than once and the system is aware of this, the system can ensure that all copies of the item are kept consistent. Unfortunately, many of today's DBMSs do not automatically ensure this type of consistency.

More information from the same amount of data With the integration of the operational data, it may be possible for the organization to derive additional information from the same data. For example, in the file-based system illustrated in Figure 1.5, the Contracts Department does not know who owns a leased property. Similarly, the Sales Department has no knowledge of lease details. When we

TABLE 1.2 Advantages of DBMSs.

Control of data redundancy	Economy of scale
Data consistency	Balance of conflicting requirements
More information from the same amount of data	Improved data accessibility and responsiveness
Sharing of data	Increased productivity
Improved data integrity	Improved maintenance through data independence
Improved security	Increased concurrency
Enforcement of standards	Improved backup and recovery services

integrate these files, the Contracts Department has access to owner details and the Sales Department has access to lease details. We may now be able to derive more information from the same amount of data.

Sharing of data Typically, files are owned by the people or departments that use them. On the other hand, the database belongs to the entire organization and can be shared by all authorized users. In this way, more users share more of the data. Furthermore, new applications can build on the existing data in the database and add only data that is not currently stored, rather than having to define all data requirements again. The new applications can also rely on the functions provided by the DBMS, such as data definition and manipulation, and concurrency and recovery control, rather than having to provide these functions themselves.

Improved data integrity Database integrity refers to the validity and consistency of stored data. Integrity is usually expressed in terms of **constraints**, which are consistency rules that the database is not permitted to violate. Constraints may apply to data items within a single record or to relationships between records. For example, an integrity constraint could state that a member of staff's salary cannot be greater than \$40,000 or that the branch number contained in a staff record, representing the branch where the member of staff works, must correspond to an existing branch office. Again, integration allows the DBA to define integrity constraints, and the DBMS to enforce them.

Improved security Database security is the protection of the database from unauthorized users. Without suitable security measures, integration makes the data more vulnerable than file-based systems. However, integration allows the DBA to define database security, and the DBMS to enforce it. This security may take the form of user names and passwords to identify people authorized to use the database. The access that an authorized user is allowed on the data may be restricted by the operation type (retrieval, insert, update, delete). For example, the DBA has access to all the data in the database; a branch manager may have access to all data that relates to his or her branch office; and a sales assistant may have access to all data relating to properties but no access to sensitive data such as staff salary details.

Enforcement of standards Again, integration allows the DBA to define and the DBMS to enforce the necessary standards. These may include departmental, organizational, national, or international standards for such things as data formats to facilitate exchange of data between systems, naming conventions, documentation standards, update procedures, and access rules.

Economy of scale Combining all the organization's operational data into one database and creating a set of applications that work on this one source of data can result in cost savings. In this case, the budget that would normally be allocated to each department for the development and maintenance of its file-based system can be combined, possibly resulting in a lower total cost, leading to an economy of scale. The combined budget can be used to buy a system configuration that is more suited to the organization's needs. This may consist of one large, powerful computer or a network of smaller computers.

Balance of conflicting requirements Each user or department has needs that may be in conflict with the needs of other users. Because the database is under the control of the DBA, the DBA can make decisions about the design and operational use of the database that provide the best use of resources for the organization as a whole. These decisions will provide optimal performance for important applications, possibly at the expense of less-critical ones.

Improved data accessibility and responsiveness Again, as a result of integration, data that crosses departmental boundaries is directly accessible to the end-users. This provides a system with potentially much more functionality that can, for example, be used to provide better services to the end-user or the organization's clients. Many DBMSs provide query languages or report writers that allow users to ask *ad hoc* questions and to obtain the required information almost immediately at their terminal, without requiring a programmer to write some software to extract this information from the database. For example, a branch manager could list all flats with a monthly rent greater than £400 by entering the following SQL command at a terminal:

```
SELECT *  
FROM PropertyForRent  
WHERE type = 'Flat' AND rent > 400;
```

Increased productivity As mentioned previously, the DBMS provides many of the standard functions that the programmer would normally have to write in a file-based application. At a basic level, the DBMS provides all the low-level file-handling routines that are typical in application programs. The provision of these functions allows the programmer to concentrate on the specific functionality required by the users without having to worry about low-level implementation details. Many DBMSs also provide a fourth-generation environment, consisting of tools to simplify the development of database applications. This results in increased programmer productivity and reduced development time (with associated cost savings).

Improved maintenance through data independence In file-based systems, the descriptions of the data and the logic for accessing the data are built into each application program, making the programs dependent on the data. A change to the structure of the data—such as making an address 41 characters instead of 40 characters, or a change to the way the data is stored on disk—can require substantial alterations to the programs that are affected by the change. In contrast, a DBMS separates the data descriptions from the applications, thereby making applications immune to changes in the data descriptions. This is known as **data independence** and is discussed further in Section 2.1.5. The provision of data independence simplifies database application maintenance.

Increased concurrency In some file-based systems, if two or more users are allowed to access the same file simultaneously, it is possible that the accesses will interfere with each other, resulting in loss of information or even loss of integrity. Many DBMSs manage concurrent database access and ensure that such problems cannot occur. We discuss concurrency control in Chapter 22.

Improved backup and recovery services Many file-based systems place the responsibility on the user to provide measures to protect the data from failures to the computer system or application program. This may involve performing a nightly backup of the data. In the event of a failure during the next day, the backup is restored and the work that has taken place since this backup is lost and has to be re-entered. In contrast, modern DBMSs provide facilities to minimize the amount of processing that is lost following a failure. We discuss database recovery in Section 22.3.

Disadvantages

The disadvantages of the database approach are summarized in Table 1.3.

Complexity The provision of the functionality that we expect of a good DBMS makes the DBMS an extremely complex piece of software. Database designers and developers, data and database administrators, and end-users must understand this functionality to take full advantage of it. Failure to understand the system can lead to bad design decisions, which can have serious consequences for an organization.

Size The complexity and breadth of functionality makes the DBMS an extremely large piece of software, occupying many megabytes of disk space and requiring substantial amounts of memory to run efficiently.

Cost of DBMSs The cost of DBMSs varies significantly, depending on the environment and functionality provided. For example, a single-user DBMS for a personal computer may only cost \$100. However, a large mainframe multi-user DBMS servicing hundreds of users can be extremely expensive, perhaps \$100,000 or even \$1,000,000. There is also the recurrent annual maintenance cost, which is typically a percentage of the list price.

Additional hardware costs The disk storage requirements for the DBMS and the database may necessitate the purchase of additional storage space. Furthermore, to achieve the required performance, it may be necessary to purchase a larger machine, perhaps even a machine dedicated to running the DBMS. The procurement of additional hardware results in further expenditure.

TABLE 1.3 Disadvantages of DBMSs.

Complexity
Size
Cost of DBMSs
Additional hardware costs
Cost of conversion
Performance
Greater impact of a failure

Cost of conversion In some situations, the cost of the DBMS and extra hardware may be relatively small compared with the cost of converting existing applications to run on the new DBMS and hardware. This cost also includes the cost of training staff to use these new systems, and possibly the employment of specialist staff to help with the conversion and running of the systems. This cost is one of the main reasons why some organizations feel tied to their current systems and cannot switch to more modern database technology. The term **legacy system** is sometimes used to refer to an older, and usually inferior, system.

Performance Typically, a file-based system is written for a specific application, such as invoicing. As a result, performance is generally very good. However, the DBMS is written to be more general, to cater for many applications rather than just one. The result is that some applications may not run as fast as they used to.

Greater impact of a failure The centralization of resources increases the vulnerability of the system. Because all users and applications rely on the availability of the DBMS, the failure of certain components can bring operations to a halt.



Chapter Summary

- **The Database Management System (DBMS)** is now the underlying framework of the information system and has fundamentally changed the way in which many organizations operate. The database system remains a very active research area and many significant problems remain.
- The predecessor to the DBMS was the **file-based system**, which is a collection of application programs that perform services for the end-users, usually the production of reports. Each program defines and manages its own data. Although the file-based system was a great improvement over the manual filing system, it still has significant problems, mainly the amount of data redundancy present and program—data dependence.
- The database approach emerged to resolve the problems with the file-based approach. A **database** is a shared collection of logically related data and a description of this data, designed to meet the information needs of an organization. A **DBMS** is a software system that enables users to define, create, maintain, and control access to the database. An **application program** is a computer program that interacts with the database by issuing an appropriate request (typically a SQL statement) to the DBMS. The more inclusive term **database system** is used to define a collection of application programs that interact with the database along with the DBMS and database itself.
- All access to the database is through the DBMS. The DBMS provides a **Data Definition Language (DDL)**, which allows users to define the database, and a **Data Manipulation Language (DML)**, which allows users to insert, update, delete, and retrieve data from the database.
- The DBMS provides controlled access to the database. It provides security, integrity, concurrency and recovery control, and a user-accessible catalog. It also provides a view mechanism to simplify the data that users have to deal with.
- The DBMS environment consists of hardware (the computer), software (the DBMS, operating system, and applications programs), data, procedures, and people. The people include data and database administrators, database designers, application developers, and end-users.
- The roots of the DBMS lie in file-based systems. The hierarchical and CODASYL systems represent the first generation of DBMSs. The **hierarchical model** is typified by IMS (Information Management System) and the

network or **CODASYL model** by IDS (Integrated Data Store), both developed in the mid-1960s. The **relational model**, proposed by E. F. Codd in 1970, represents the second generation of DBMSs. It has had a fundamental effect on the DBMS community and there are now over one hundred relational DBMSs. The third generation of DBMSs are represented by the **Object-Relational DBMS** and the **Object-Oriented DBMS**.

- Some advantages of the database approach include control of data redundancy, data consistency, sharing of data, and improved security and integrity. Some disadvantages include complexity, cost, reduced performance, and higher impact of a failure.

Review Questions

- 1.1 List four government sectors in your country that use database systems..
- 1.2 Discuss each of the following terms:
 - (a) data
 - (b) database
 - (c) database management system
 - (d) database application program
 - (e) data independence
 - (f) security
 - (g) integrity
 - (h) views
- 1.3 Describe the role of database management systems (DBMS) in the database approach. Discuss why knowledge of DBMS is important for database administrators.
- 1.4 Describe the main characteristics of the database approach and contrast it with the file-based approach.
- 1.5 Describe the five components of the DBMS environment and discuss how they relate to each other.
- 1.6 Discuss the roles of the following personnel in the database environment:
 - (a) data administrator
 - (b) database administrator
 - (c) logical database designer
 - (d) physical database designer
 - (e) application developer
 - (f) end-users
- 1.7 Discuss the three generations of DBMSs.
- 1.8 Why are views an important aspect of database management systems?

Exercises

- 1.9 Interview some users of database systems. Which DBMS features do they find most useful and why? Which DBMS facilities do they find least useful and why? What do these users perceive to be the advantages and disadvantages of the DBMS?
- 1.10 A database approach addresses several of the problems and challenges associated with the traditional file-based approach. Using a DBMS to control how data is shared with different applications and users, through applications such as views, has a number of advantages. However, the implementation of a database approach has its own

challenges, such as expense. Discuss the various costs associated with the implementation of a database approach.



I.11 Study the *DreamHome* case study presented in Section 11.4 and Appendix A.

- (a) In what ways would a DBMS help this organization?
- (b) What do you think are the main objects that need to be represented in the database?
- (c) What relationships do you think exist between these main objects?
- (d) For each of the objects, what details do you think need to be stored in the database?
- (e) What queries do you think are required?

I.12 Study the *Wellmeadows Hospital* case study presented in Appendix B.3.

- (a) In what ways would a DBMS help this organization?
- (b) What do you think are the main objects that need to be represented in the database?
- (c) What relationships do you think exist between these main objects?
- (d) For each of the objects, what details do you think need to be stored in the database?
- (e) What queries do you think are required?



I.13 Discuss what you consider to be the three most important advantages for the use of a DBMS for a company like *DreamHome* and provide a justification for your selection. Discuss what you consider to be the three most important disadvantages for the use of a DBMS for a company like *DreamHome* and provide a justification for your selection.

I.14 Organizations have a vital need for quality information. Discuss how the following database roles relate to each other.

- (a) Data Administrator
- (b) Database Administrator
- (c) Database Designer
- (d) Application Developer
- (e) End-Users

Chapter Objectives

In this chapter you will learn:

- The purpose and origin of the three-level database architecture.
- The contents of the external, conceptual, and internal levels.
- The purpose of the external/conceptual and the conceptual/internal mappings.
- The meaning of logical and physical data independence.
- The distinction between a Data Definition Language (DDL) and a Data Manipulation Language (DML).
- A classification of data models.
- The purpose and importance of conceptual modeling.
- The typical functions and services that a DBMS should provide.
- The function and importance of the system catalog.

A major aim of a database system is to provide users with an abstract view of data, hiding certain details of how data is stored and manipulated. Therefore, the starting point for the design of a database must be an abstract and general description of the information requirements of the organization that is to be represented in the database. In this chapter, and throughout this book, we use the term “organization” loosely to mean the whole organization or part of the organization. For example, in the *DreamHome* case study, we may be interested in modeling:



- the “real-world” **entities** Staff, PropertyforRent, PrivateOwner, and Client;
- **attributes** describing properties or qualities of each entity (for example, each Staff entry has a name, position, and salary);
- **relationships** between these entities (for example, Staff *Manages* PropertyforRent).

Furthermore, because a database is a shared resource, each user may require a different view of the data held in the database. To satisfy these needs, the architecture of most commercial DBMSs available today is based to some extent on the so-called ANSI-SPARC architecture. In this chapter, we discuss various architectural and functional characteristics of DBMSs.



Structure of this Chapter In Section 2.1, we examine the three-level ANSI-SPARC architecture and its associated benefits. In Section 2.2, we consider the types of language that are used by DBMSs, and in Section 2.3, we introduce the concepts of data models and conceptual modeling, which we expand on in later parts of the book. In Section 2.4, we discuss the functions that we would expect a DBMS to provide. The examples in this chapter are drawn from the *DreamHome* case study, which we discuss more fully in Section 11.4 and Appendix A.

Much of the material in this chapter provides important background information on DBMSs. However, the reader who is new to the area of database systems may find some of the material difficult to comprehend fully on first reading. Do not be too concerned about this, but be prepared to revisit parts of this chapter at a later date when you have read subsequent chapters of the book.

2.1 The Three-Level ANSI-SPARC Architecture

An early proposal for a standard terminology and general architecture for database systems was produced in 1971 by the DBTG appointed by CODASYL in 1971. The DBTG recognized the need for a two-level approach with a system view called the **schema** and user views called **subschemas**. The American National Standards Institute (ANSI) Standards Planning and Requirements Committee (SPARC), or ANSI/X3/SPARC, produced a similar terminology and architecture in 1975 (ANSI, 1975). The ANSI-SPARC architecture recognized the need for a three-level approach with a system catalog. These proposals reflected those published by the IBM user organizations Guide and Share some years previously, and concentrated on the need for an implementation-independent layer to isolate programs from underlying representational issues (Guide/Share, 1970). Although the ANSI-SPARC model did not become a standard, it still provides a basis for understanding some of the functionality of a DBMS.

For our purposes, the fundamental point of these and later reports is the identification of three levels of abstraction, that is, three distinct levels at which data items can be described. The levels form a **three-level architecture** comprising an **external**, a **conceptual**, and an **internal** level, as depicted in Figure 2.1. The way users perceive the data is called the **external level**. The way the DBMS and the operating system perceive the data is the **internal level**, where the data is actually stored using the data structures and file organizations described in Appendix F. The **conceptual level** provides both the **mapping** and the desired **independence** between the external and internal levels.

The objective of the three-level architecture is to separate each user's view of the database from the way the database is physically represented. There are several reasons why this separation is desirable:

- Each user should be able to access the same data, but have a different customized view of the data. Each user should be able to change the way he or she views the data, and this change should not affect other users.

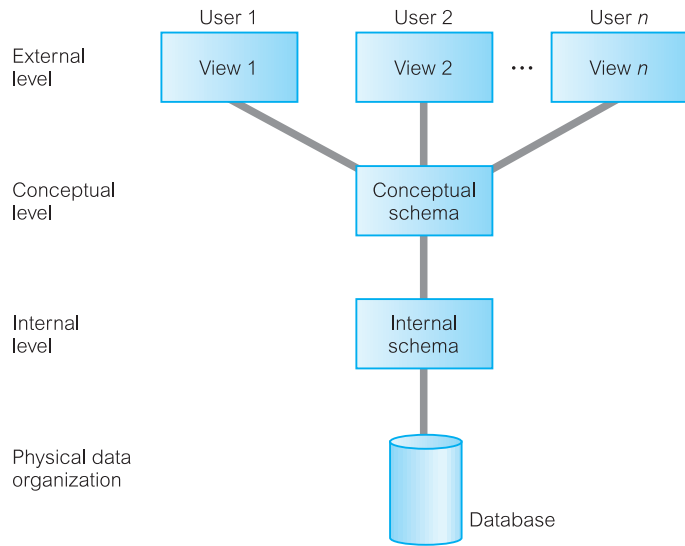


Figure 2.1
The ANSI-SPARC three-level architecture.

- Users should not have to deal directly with physical database storage details, such as indexing or hashing (see Appendix F). In other words, a user's interaction with the database should be independent of storage considerations.
- The DBA should be able to change the database storage structures without affecting the users' views.
- The internal structure of the database should be unaffected by changes to the physical aspects of storage, such as the changeover to a new storage device.
- The DBA should be able to change the conceptual structure of the database without affecting all users.

2.1.1 External Level

External level

The users' view of the database. This level describes that part of the database that is relevant to each user.

The external level consists of a number of different external views of the database. Each user has a view of the “real world” represented in a form that is familiar for that user. The external view includes only those entities, attributes, and relationships in the “real world” that the user is interested in. Other entities, attributes, or relationships that are not of interest may be represented in the database, but the user will be unaware of them.

In addition, different views may have different representations of the same data. For example, one user may view dates in the form (day, month, year), while another may view dates as (year, month, day). Some views might include derived or calculated data: data not actually stored in the database as such, but created when needed. For example, in the *DreamHome* case study, we may wish to view the age of a member of staff. However, it is unlikely that ages would be stored, as this data would have to be updated daily. Instead, the member of staff's date of birth would be stored and age would be calculated by the DBMS when it is referenced.



Views may even include data combined or derived from several entities. We discuss views in more detail in Sections 4.4 and 7.4.

2.1.2 Conceptual Level

Conceptual level

The community view of the database. This level describes *what* data is stored in the database and the relationships among the data.

The middle level in the three-level architecture is the conceptual level. This level contains the logical structure of the entire database as seen by the DBA. It is a complete view of the data requirements of the organization that is independent of any storage considerations. The conceptual level represents:

- all entities, their attributes, and their relationships;
- the constraints on the data;
- semantic information about the data;
- security and integrity information.

The conceptual level supports each external view, in that any data available to a user must be contained in, or derivable from, the conceptual level. However, this level must not contain any storage-dependent details. For instance, the description of an entity should contain only data types of attributes (for example, integer, real, character) and their length (such as the maximum number of digits or characters), but not any storage considerations, such as the number of bytes occupied.

2.1.3 Internal Level

Internal level

The physical representation of the database on the computer. This level describes *how* the data is stored in the database.

The internal level covers the physical implementation of the database to achieve optimal runtime performance and storage space utilization. It covers the data structures and file organizations used to store data on storage devices. It interfaces with the operating system access methods (file management techniques for storing and retrieving data records) to place the data on the storage devices, build the indexes, retrieve the data, and so on. The internal level is concerned with such things as:

- storage space allocation for data and indexes;
- record descriptions for storage (with stored sizes for data items);
- record placement;
- data compression and data encryption techniques.

Below the internal level there is a **physical level** that may be managed by the operating system under the direction of the DBMS. However, the functions of the DBMS and the operating system at the physical level are not clear-cut and vary from system to system. Some DBMSs take advantage of many of the operating

system access methods, and others use only the most basic ones and create their own file organizations. The physical level below the DBMS consists of items that only the operating system knows, such as exactly how the sequencing is implemented and whether the fields of internal records are stored as contiguous bytes on the disk.

2.1.4 Schemas, Mappings, and Instances

The overall description of the database is called the **database schema**. There are three different types of schema in the database and these are defined according to the levels of abstraction of the three-level architecture illustrated in Figure 2.1. At the highest level, we have multiple **external schemas** (also called **subschemas**) that correspond to different views of the data. At the conceptual level, we have the **conceptual schema**, which describes all the entities, attributes, and relationships together with integrity constraints. At the lowest level of abstraction we have the **internal schema**, which is a complete description of the internal model, containing the definitions of stored records, the methods of representation, the data fields, and the indexes and storage structures used. There is only one conceptual schema and one internal schema per database.

The DBMS is responsible for mapping between these three types of schema. It must also check the schemas for consistency; in other words, the DBMS must confirm that each external schema is derivable from the conceptual schema, and it must use the information in the conceptual schema to map between each external schema and the internal schema. The conceptual schema is related to the internal schema through a **conceptual/internal mapping**. This mapping enables the DBMS to find the actual record or combination of records in physical storage that constitute a **logical record** in the conceptual schema, together with any constraints to be enforced on the operations for that logical record. It also allows any differences in entity names, attribute names, attribute order, data types, and so on to be resolved. Finally, each external schema is related to the conceptual schema by the **external/conceptual mapping**. This mapping enables the DBMS to map names in the user's view to the relevant part of the conceptual schema.

An example of the different levels is shown in Figure 2.2. Two different external views of staff details exist: one consisting of a staff number (sNo), first name (fName), last name (lName), age, and salary; a second consisting of a staff number (staffNo), last name (lName), and the number of the branch the member of staff works at (branchNo). These external views are merged into one conceptual view. In this merging process, the major difference is that the age field has been changed into a date of birth field, DOB. The DBMS maintains the external/conceptual mapping; for example, it maps the sNo field of the first external view to the staffNo field of the conceptual record. The conceptual level is then mapped to the internal level, which contains a physical description of the structure for the conceptual record. At this level, we see a definition of the structure in a high-level language. The structure contains a pointer, next, which allows the list of staff records to be physically linked together to form a chain. Note that the order of fields at the internal level is different from that at the conceptual level. Again, the DBMS maintains the conceptual/internal mapping.

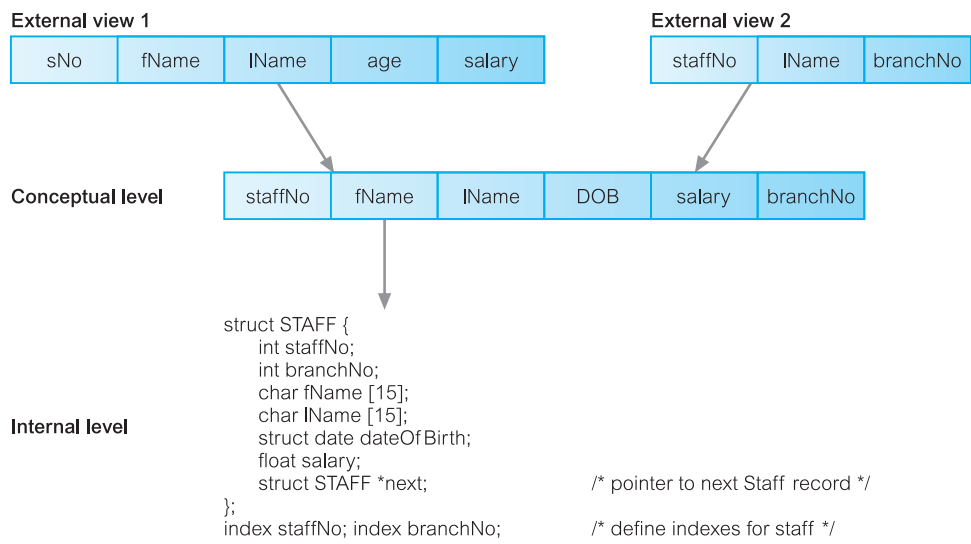


Figure 2.2 Differences between the three levels.

It is important to distinguish between the description of the database and the database itself. The description of the database is the **database schema**. The schema is specified during the database design process and is not expected to change frequently. However, the actual data in the database may change frequently; for example, it changes every time we insert details of a new member of staff or a new property. The data in the database at any particular point in time is called a **database instance**. Therefore, many database instances can correspond to the same database schema. The schema is sometimes called the **intension** of the database; an instance is called an **extension** (or **state**) of the database.

2.1.5 Data Independence

A major objective for the three-level architecture is to provide **data independence**, which means that upper levels are unaffected by changes to lower levels. There are two kinds of data independence: **logical and physical**.

- Logical data independence

The immunity of the external schemas to changes in the conceptual schema.
- Changes to the conceptual schema, such as the addition or removal of new entities, attributes, or relationships, should be possible without having to change existing external schemas or having to rewrite application programs. Clearly, the users for whom the changes have been made need to be aware of them, but what is important is that other users should not be.
- Physical data independence

The immunity of the conceptual schema to changes in the internal schema.

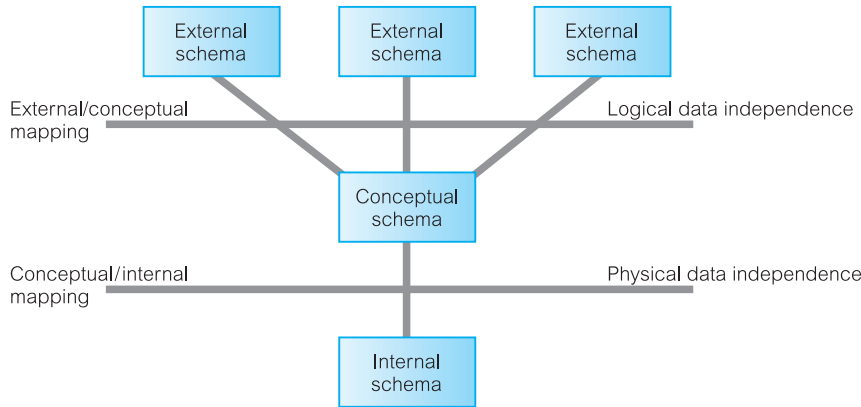


Figure 2.3
Data independence and the ANSI-SPARC three-level architecture.

Changes to the internal schema, such as using different file organizations or storage structures, using different storage devices, modifying indexes or hashing algorithms, should be possible without having to change the conceptual or external schemas. From the users' point of view, the only effect that may be noticed is a change in performance. In fact, deterioration in performance is the most common reason for internal schema changes. Figure 2.3 illustrates where each type of data independence occurs in relation to the three-level architecture.

The two-stage mapping in the ANSI-SPARC architecture may be inefficient, but it also provides greater data independence. However, for more efficient mapping, the ANSI-SPARC model allows the direct mapping of external schemas onto the internal schema, thus by-passing the conceptual schema. This mapping of course reduces data independence, so that every time the internal schema changes, the external schema and any dependent application programs may also have to change.

2.2 Database Languages

A data sublanguage consists of two parts: a Data Definition Language (DDL) and a Data Manipulation Language (DML). The DDL is used to specify the database schema and the DML is used to both read and update the database. These languages are called *data sublanguages* because they do not include constructs for all computing needs, such as conditional or iterative statements, which are provided by the high-level programming languages. Many DBMSs have a facility for *embedding* the sublanguage in a high-level programming language such as COBOL, Fortran, Pascal, Ada, C, C++, C#, Java, or Visual Basic. In this case, the high-level language is sometimes referred to as the *host language*. To compile the embedded file, the commands in the data sublanguage are first removed from the host-language program and replaced by function calls. The preprocessed file is then compiled, placed in an object module, linked with a DBMS-specific library containing the replaced functions, and executed when required. Most data sublanguages also provide *nonembedded* or *interactive* commands that can be input directly from a terminal.

2.2.1 The Data Definition Language (DDL)

DDL

A language that allows the DBA or user to describe and name the entities, attributes, and relationships required for the application, together with any associated integrity and security constraints.

The database schema is specified by a set of definitions expressed by means of a special language called a Data Definition Language. The DDL is used to define a schema or to modify an existing one. It cannot be used to manipulate data.

The result of the compilation of the DDL statements is a set of tables stored in special files collectively called the **system catalog**. The system catalog integrates the **metadata**, which is data that describes the objects in the database and makes it easier for those objects to be accessed or manipulated. The metadata contains definitions of records, data items, and other objects that are of interest to users or are required by the DBMS. The DBMS normally consults the system catalog before the actual data is accessed in the database. The terms **data dictionary** and **data directory** are also used to describe the system catalog, although the term “data dictionary” usually refers to a more general software system than a catalog for a DBMS. We discuss the system catalog further in Section 2.4.

At a theoretical level, we could identify different DDLs for each schema in the three-level architecture: namely, a DDL for the external schemas, a DDL for the conceptual schema, and a DDL for the internal schema. However, in practice, there is one comprehensive DDL that allows specification of at least the external and conceptual schemas.

2.2.2 The Data Manipulation Language (DML)

DML

A language that provides a set of operations to support the basic data manipulation operations on the data held in the database.

Data manipulation operations usually include the following:

- insertion of new data into the database;
- modification of data stored in the database;
- retrieval of data contained in the database;
- deletion of data from the database.

Therefore, one of the main functions of the DBMS is to support a Data Manipulation Language in which the user can construct statements that will cause such data manipulation to occur. Data manipulation applies to the external, conceptual, and internal levels. However, at the internal level we must define rather complex low-level procedures that allow efficient data access. In contrast, at higher levels, emphasis is placed on ease of use and effort is directed at providing efficient user interaction with the system.

The part of a DML that involves data retrieval is called a **query language**. A query language can be defined as a high-level special-purpose language used to satisfy diverse requests for the retrieval of data held in the database. The term

“query” is therefore reserved to denote a retrieval statement expressed in a query language. The terms “query language” and “DML” are commonly used interchangeably, although this is technically incorrect.

DMLs are distinguished by their underlying retrieval constructs. We can distinguish between two types of DML: **procedural** and **nonprocedural**. The prime difference between these two data manipulation languages is that procedural languages specify *how* the output of a DML statement is to be obtained, while nonprocedural DMLs describe only *what* output is to be obtained. Typically, procedural languages treat records individually, whereas nonprocedural languages operate on sets of records.

Procedural DMLs

Procedural DML

A language that allows the user to tell the system what data is needed and exactly how to retrieve the data.

With a procedural DML, the user, or more often the programmer, specifies what data is needed and how to obtain it. This means that the user must express all the data access operations that are to be used by calling appropriate procedures to obtain the information required. Typically, such a procedural DML retrieves a record, processes it and, based on the results obtained by this processing, retrieves another record that would be processed similarly, and so on. This process of retrievals continues until the data requested from the retrieval has been gathered. Typically, procedural DMLs are embedded in a high-level programming language that contains constructs to facilitate iteration and handle navigational logic. Network and hierarchical DMLs are normally procedural (see Section 2.3).

Nonprocedural DMLs

Nonprocedural DML

A language that allows the user to state what data is needed rather than how it is to be retrieved.

Nonprocedural DMLs allow the required data to be specified in a single retrieval or update statement. With nonprocedural DMLs, the user specifies what data is required without specifying how it is to be obtained. The DBMS translates a DML statement into one or more procedures that manipulate the required sets of records, which frees the user from having to know how data structures are internally implemented and what algorithms are required to retrieve and possibly transform the data, thus providing users with a considerable degree of data independence. Nonprocedural languages are also called *declarative languages*. Relational DBMSs usually include some form of nonprocedural language for data manipulation, typically SQL or QBE (Query-By-Example). Nonprocedural DMLs are normally easier to learn and use than procedural DMLs, as less work is done by the user and more by the DBMS. We examine SQL in detail in Chapters 6–9 and Appendix I, and QBE in Appendix M.

2.2.3 Fourth-Generation Languages (4GLs)

There is no consensus about what constitutes a **fourth-generation language**; in essence, it is a shorthand programming language. An operation that requires hundreds of lines of code in a third-generation language (3GL), such as COBOL, generally requires significantly fewer lines in a 4GL.

Compared with a 3GL, which is procedural, a 4GL is nonprocedural: the user defines *what* is to be done, not *how*. A 4GL is expected to rely largely on much higher-level components known as fourth-generation tools. The user does not define the steps that a program needs to perform a task, but instead defines parameters for the tools that use them to generate an application program. It is claimed that 4GLs can improve productivity by a factor of ten, at the cost of limiting the types of problem that can be tackled. Fourth-generation languages encompass:

- presentation languages, such as query languages and report generators;
- speciality languages, such as spreadsheets and database languages;
- application generators that define, insert, update, and retrieve data from the database to build applications;
- very high-level languages that are used to generate application code.

SQL and QBE, mentioned previously, are examples of 4GLs. We now briefly discuss some of the other types of 4GL.

Forms generators

A forms generator is an interactive facility for rapidly creating data input and display layouts for screen forms. The forms generator allows the user to define what the screen is to look like, what information is to be displayed, and where on the screen it is to be displayed. It may also allow the definition of colors for screen elements and other characteristics, such as bold, underline, blinking, reverse video, and so on. The better forms generators allow the creation of derived attributes, perhaps using arithmetic operators or aggregates, and the specification of validation checks for data input.

Report generators

A **report generator** is a facility for creating reports from data stored in the database. It is similar to a query language in that it allows the user to ask questions of the database and retrieve information from it for a report. However, in the case of a report generator, we have much greater control over what the output looks like. We can let the report generator automatically determine how the output should look or we can create our own customized output reports using special report-generator command instructions.

There are two main types of report generator: language-oriented and visually oriented. In the first case, we enter a command in a sublanguage to define what data is to be included in the report and how the report is to be laid out. In the second case, we use a facility similar to a forms generator to define the same information.

Graphics generators

A **graphics generator** is a facility to retrieve data from the database and display the data as a graph showing trends and relationships in the data. Typically, it allows the user to create bar charts, pie charts, line charts, scatter charts, and so on.

Application generators

An **application generator** is a facility for producing a program that interfaces with the database. The use of an application generator can reduce the time it takes to design an entire software application. Application generators typically consist of prewritten modules that comprise fundamental functions that most programs use. These modules, usually written in a high-level language, constitute a library of functions to choose from. The user specifies *what* the program is supposed to do; the application generator determines *how* to perform the tasks.

2.3 Data Models and Conceptual Modeling

We mentioned earlier that a schema is written using a DDL. In fact, it is written in the DDL of a particular DBMS. Unfortunately, this type of language is too low level to describe the data requirements of an organization in a way that is readily understandable by a variety of users. What we require is a higher-level description of the schema: that is, a **data model**.

Data model

An integrated collection of concepts for describing and manipulating data, relationships between data, and constraints on the data in an organization.

A model is a representation of real-world objects and events, and their associations. It is an abstraction that concentrates on the essential, inherent aspects of an organization and ignores the accidental properties. A data model represents the organization itself. It should provide the basic concepts and notations that will allow database designers and end-users to communicate unambiguously and accurately their understanding of the organizational data. A data model can be thought of as comprising three components:

- (1) a **structural part**, consisting of a set of rules according to which databases can be constructed;
- (2) a **manipulative part**, defining the types of operation that are allowed on the data (this includes the operations that are used for updating or retrieving data from the database and for changing the structure of the database);
- (3) a **set of integrity constraints**, which ensures that the data is accurate.

The purpose of a data model is to represent data and to make the data understandable. If it does this, then it can be easily used to design a database. To reflect the ANSI-SPARC architecture introduced in Section 2.1, we can identify three related data models:

- (1) an external data model, to represent each user's view of the organization, sometimes called the **Universe of Discourse (UoD)**;

- (2) a conceptual data model, to represent the logical (or community) view that is DBMS-independent;
- (3) an internal data model, to represent the conceptual schema in such a way that it can be understood by the DBMS.

There have been many data models proposed in the literature. They fall into three broad categories: **object-based**, **record-based**, and **physical** data models. The first two are used to describe data at the conceptual and external levels, the third is used to describe data at the internal level.

2.3.1 Object-Based Data Models

Object-based data models use concepts such as entities, attributes, and relationships. An **entity** is a distinct object (a person, place, thing, concept, event) in the organization that is to be represented in the database. An **attribute** is a property that describes some aspect of the object that we wish to record, and a **relationship** is an association between entities. Some of the more common types of object-based data model are:

- Entity-Relationship (ER)
- Semantic
- Functional
- Object-oriented

The ER model has emerged as one of the main techniques for database design and forms the basis for the database design methodology used in this book. The object-oriented data model extends the definition of an entity to include not only the attributes that describe the **state** of the object but also the actions that are associated with the object, that is, its **behavior**. The object is said to **encapsulate** both state and behavior. We look at the ER model in depth in Chapters 12 and 13 and the object-oriented model in Chapters 27–28. We also examine the functional data model in Section 27.5.2.

2.3.2 Record-Based Data Models

In a record-based model, the database consists of a number of fixed-format records, possibly of differing types. Each record type defines a fixed number of fields, typically of a fixed length. There are three principal types of record-based logical data model: the **relational data model**, the **network data model**, and the **hierarchical data model**. The hierarchical and network data models were developed almost a decade before the relational data model, so their links to traditional file processing concepts are more evident.

Relational data model



The relational data model is based on the concept of mathematical relations. In the relational model, data and relationships are represented as tables, each of which has a number of columns with a unique name. Figure 2.4 is a sample instance of a relational schema for part of the *DreamHome* case study, showing branch and staff

Branch

branchNo	street	city	postCode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

Figure 2.4

A sample instance of a relational schema.

details. For example, it shows that employee John White is a manager with a salary of £30,000, who works at branch (branchNo) B005, which, from the first table, is at 22 Deer Rd in London. It is important to note that there is a relationship between **Staff** and **Branch**: a branch office *has* staff. However, there is no explicit link between these two tables; it is only by knowing that the attribute **branchNo** in the **Staff** relation is the same as the **branchNo** of the **Branch** relation that we can establish that a relationship exists.

Note that the relational data model requires only that the database be perceived by the user as tables. However, this perception applies only to the logical structure of the database, that is, the external and conceptual levels of the ANSI-SPARC architecture. It does not apply to the physical structure of the database, which can be implemented using a variety of storage structures. We discuss the relational data model in Chapter 4.

Network data model

In the network model, data is represented as collections of **records**, and relationships are represented by **sets**. Compared with the relational model, relationships are explicitly modeled by the sets, which become pointers in the implementation. The records are organized as generalized graph structures with records appearing as **nodes** (also called **segments**) and sets as **edges** in the graph. Figure 2.5 illustrates an instance of a network schema for the same data set presented in Figure 2.4. The most popular network DBMS is Computer Associates' IDMS/R. We discuss the network data model in more detail on the Web site for this book (see the Preface for the URL).

Hierarchical data model

The hierarchical model is a restricted type of network model. Again, data is represented as collections of **records** and relationships are represented by **sets**.

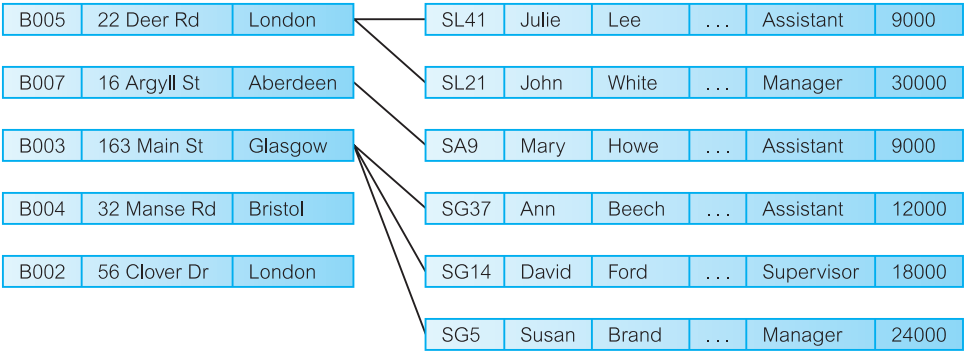


Figure 2.5 A sample instance of a network schema.

However, the hierarchical model allows a node to have only one parent. A hierarchical model can be represented as a tree graph, with records appearing as **nodes** (also called **segments**) and sets as **edges**. Figure 2.6 illustrates an instance of a hierarchical schema for the same data set presented in Figure 2.4. The main hierarchical DBMS is IBM’s IMS, although IMS also provides nonhierarchical features. We discuss the hierarchical data model in more detail on the Web site for this book (see the Preface for the URL).

Record-based (logical) data models are used to specify the overall structure of the database and a higher-level description of the implementation. Their main drawback is that they do not provide adequate facilities for explicitly specifying constraints on the data, whereas the object-based data models lack the means of logical structure specification but provide more semantic substance by allowing the user to specify constraints on the data.

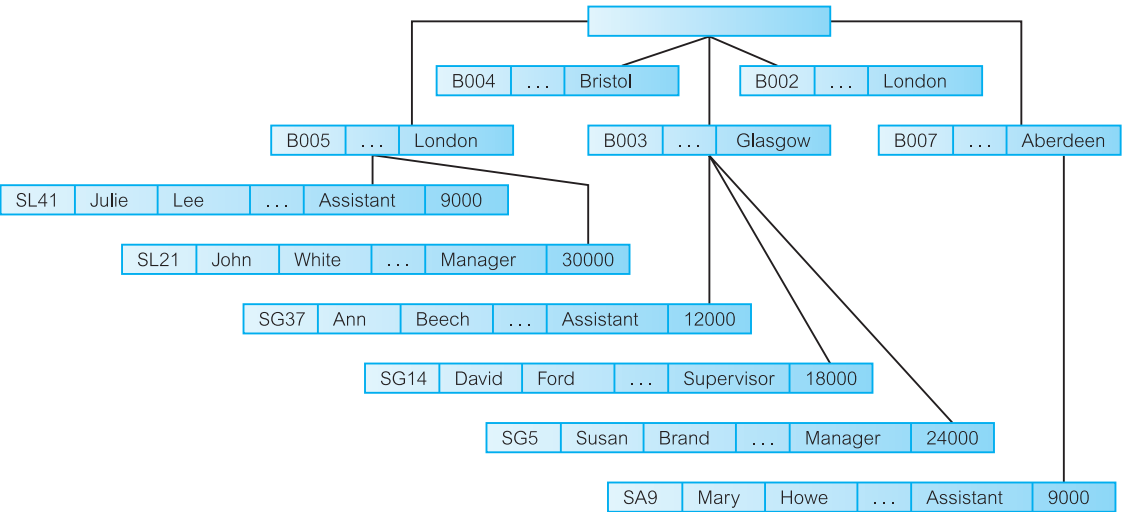


Figure 2.6 A sample instance of a hierarchical schema.

The majority of modern commercial systems are based on the relational paradigm, whereas the early database systems were based on either the network or hierarchical data models. The latter two models require the user to have knowledge of the physical database being accessed, whereas the former provides a substantial amount of data independence. Hence, relational systems adopt a **declarative** approach to database processing (that is, they specify *what* data is to be retrieved), but network and hierarchical systems adopt a **navigational** approach (that is, they specify *how* the data is to be retrieved).

2.3.3 Physical Data Models

Physical data models describe how data is stored in the computer, representing information such as record structures, record orderings, and access paths. There are not as many physical data models as logical data models; the most common ones are the *unifying model* and the *frame memory*.

2.3.4 Conceptual Modeling

From an examination of the three-level architecture, we see that the conceptual schema is the heart of the database. It supports all the external views and is, in turn, supported by the internal schema. However, the internal schema is merely the physical implementation of the conceptual schema. The conceptual schema should be a complete and accurate representation of the data requirements of the enterprise.[†] If this is not the case, some information about the enterprise will be missing or incorrectly represented and we will have difficulty fully implementing one or more of the external views.

Conceptual modeling or **conceptual database design** is the process of constructing a model of the information use in an enterprise that is independent of implementation details, such as the target DBMS, application programs, programming languages, or any other physical considerations. This model is called a **conceptual data model**. Conceptual models are also referred to as “logical models” in the literature. However, in this book we make a distinction between conceptual and logical data models. The conceptual model is independent of all implementation details, whereas the logical model assumes knowledge of the underlying data model of the target DBMS. In Chapters 16 and 17 we present a methodology for database design that begins by producing a conceptual data model, which is then refined into a logical model based on the relational data model. We discuss database design in more detail in Section 10.6.

2.4 Functions of a DBMS

In this section, we look at the types of function and service that we would expect a DBMS to provide. Codd (1982) lists eight services that should be provided by any full-scale DBMS, and we have added two more that might reasonably be expected to be available.

[†]When we are discussing the organization in the context of database design we normally refer to the business or organization as the *enterprise*.

(1) Data storage, retrieval, and update

A DBMS must furnish users with the ability to store, retrieve, and update data in the database.

This is the fundamental function of a DBMS. From the discussion in Section 2.1, clearly in providing this functionality, clearly the DBMS should hide the internal physical implementation details (such as file organization and storage structures) from the user.

(2) A user-accessible catalog

A DBMS must furnish a catalog in which descriptions of data items are stored and which is accessible to users.

A key feature of the ANSI-SPARC architecture is the recognition of an integrated **system catalog** to hold data about the schemas, users, applications, and so on. The catalog is expected to be accessible to users as well as to the DBMS. A system catalog, or data dictionary, is a repository of information describing the data in the database; it is the “data about the data” or the **metadata**. The amount of information and the way the information is used vary with the DBMS. Typically, the system catalog stores:

- names, types, and sizes of data items;
- names of relationships;
- integrity constraints on the data;
- names of authorized users who have access to the data;
- the data items that each user can access and the types of access allowed; for example, insert, update, delete, or read access;
- external, conceptual, and internal schemas and the mappings between the schemas, as described in 2.1.4;
- usage statistics, such as the frequencies of transactions and counts on the number of accesses made to objects in the database.

The DBMS system catalog is one of the fundamental components of the system. Many of the software components that we describe in the next section rely on the system catalog for information. Some benefits of a system catalog are:

- Information about data can be collected and stored centrally. This helps to maintain control over the data as a resource.
- The meaning of data can be defined, which will help other users understand the purpose of the data.
- Communication is simplified, because exact meanings are stored. The system catalog may also identify the user or users who own or access the data.
- Redundancy and inconsistencies can be identified more easily as the data is centralized.
- Changes to the database can be recorded.

- The impact of a change can be determined before it is implemented because the system catalog records each data item, all its relationships, and all its users.
- Security can be enforced.
- Integrity can be ensured.
- Audit information can be provided.

Some authors make a distinction between system catalog and data directory, in that a data directory holds information relating to where data is stored and how it is stored. The ISO has adopted a standard for data dictionaries called Information Resource Dictionary System (IRDS) (ISO 1990, 1993). IRDS is a software tool that can be used to control and document an organization's information sources. It provides a definition for the tables that comprise the data dictionary and the operations that can be used to access these tables. We use the term "system catalog" in this book to refer to all repository information. We discuss other types of statistical information stored in the system catalog to assist with query optimization in Section 23.4.1.

(3) Transaction support

A DBMS must furnish a mechanism that will ensure either that all the updates corresponding to a given transaction are made or that none of them is made.

A transaction is a series of actions, carried out by a single user or application program, which accesses or changes the contents of the database. For example, some simple transactions for the *DreamHome* case study might be to add a new member of staff to the database, to update the salary of a member of staff, or to delete a property from the register. A more complicated example might be to delete a member of staff from the database *and* to reassign the properties that he or she managed to another member of staff. In this case, there is more than one change to be made to the database. If the transaction fails during execution, perhaps because of a computer crash, the database will be in an **inconsistent** state: some changes will have been made and others will not. Consequently, the changes that have been made will have to be undone to return the database to a consistent state again. We discuss transaction support in Section 22.1.



(4) Concurrency control services

A DBMS must furnish a mechanism to ensure that the database is updated correctly when multiple users are updating the database concurrently.

One major objective in using a DBMS is to enable many users to access shared data concurrently. Concurrent access is relatively easy if all users are only reading data, as there is no way that they can interfere with one another. However, when two or more users are accessing the database simultaneously and at least one of them is updating data, there may be interference that can result in inconsistencies. For example, consider two transactions T_1 and T_2 , which are executing concurrently, as illustrated in Figure 2.7.

Time	T ₁	T ₂	bal _x
t ₁		read(bal _x)	100
t ₂	read(bal _x)	bal _x = bal _x + 100	100
t ₃	bal _x = bal _x - 10	write(bal _x)	200
t ₄	write(bal _x)		90
t ₅			90

Figure 2.7 The lost update problem.

T₁ is withdrawing £10 from an account (with balance bal_x) and T₂ is depositing £100 into the same account. If these transactions were executed **serially**, one after the other with no interleaving of operations, the final balance would be £190, regardless of which was performed first. However, in this example transactions T₁ and T₂ start at nearly the same time and both read the balance as £100. T₂ then increases bal_x by £100 to £200 and stores the update in the database. Meanwhile, transaction T₁ decrements its copy of bal_x by £10 to £90 and stores this value in the database, overwriting the previous update and thereby “losing” £100.

The DBMS must ensure that when multiple users are accessing the database, interference cannot occur. We discuss this issue fully in Section 22.2.

(5) Recovery services

A DBMS must furnish a mechanism for recovering the database in the event that the database is damaged in any way.

When discussing transaction support, we mentioned that if the transaction fails, then the database has to be returned to a consistent state. This failure may be the result of a system crash, media failure, a hardware or software error causing the DBMS to stop, or it may be the result of the user detecting an error during the transaction and aborting the transaction before it completes. In all these cases, the DBMS must provide a mechanism to restore the database to a consistent state. We discuss database recovery in Section 22.3.

(6) Authorization services

A DBMS must furnish a mechanism to ensure that only authorized users can access the database.

It is not difficult to envision instances where we would want to prevent some of the data stored in the database from being seen by all users. For example, we may want only branch managers to see salary-related information for staff and to prevent all other users from seeing this data. Additionally, we may want to protect the database from unauthorized access. The term “security” refers to the protection of the database against unauthorized access, either intentional or accidental. We expect the DBMS to provide mechanisms to ensure that the data is secure. We discuss security in Chapter 20.

(7) Support for data communication

A DBMS must be capable of integrating with communication software.

Most users access the database from workstations. Sometimes these workstations are connected directly to the computer hosting the DBMS. In other cases, the workstations are at remote locations and communicate with the computer hosting the DBMS over a network. In either case, the DBMS receives requests as **communications messages** and responds in a similar way. All such transmissions are handled by a data communication manager (DCM). Although the DCM is not part of the DBMS, it is necessary for the DBMS to be capable of being integrated with a variety of DCMs if the system is to be commercially viable. Even DBMSs for personal computers should be capable of being run on a local area network so that one centralized database can be established for users to share, rather than having a series of disparate databases, one for each user. This does not imply that the database has to be distributed across the network, but rather that users should be able to access a centralized database from remote locations. We refer to this type of topology as *distributed processing* (see Section 24.1.1).

(8) Integrity services

A DBMS must furnish a means to ensure that both the data in the database and changes to the data follow certain rules.

“Database integrity” refers to the correctness and consistency of stored data: it can be considered as another type of database protection. Although integrity is related to security, it has wider implications: integrity is concerned with the quality of data itself. Integrity is usually expressed in terms of *constraints*, which are consistency rules that the database is not permitted to violate. For example, we may want to specify a constraint that no member of staff can manage more than 100 properties at any one time. Here, we would want the DBMS to check when we assign a property to a member of staff whether this limit would be exceeded and to prevent the assignment from occurring if the limit has been reached.

In addition to these eight services, we could also reasonably expect the following two services to be provided by a DBMS.

(9) Services to promote data independence

A DBMS must include facilities to support the independence of programs from the actual structure of the database.

We discussed the concept of data independence in Section 2.1.5. Data independence is normally achieved through a view or subschema mechanism. Physical data independence is easier to achieve: there are usually several types of change that

can be made to the physical characteristics of the database without affecting the views. However, complete logical data independence is more difficult to achieve. The addition of a new entity, attribute, or relationship can usually be accommodated, but not their removal. In some systems, any type of change to an existing component in the logical structure is prohibited.

(10) Utility services

A DBMS should provide a set of utility services.

Utility programs help the DBA administer the database effectively. Some utilities work at the external level, and consequently can be produced by the DBA. Other utilities work at the internal level and can be provided only by the DBMS vendor. Examples of utilities of the latter kind are:

- import facilities, to load the database from flat files, and export facilities, to unload the database to flat files;
- monitoring facilities, to monitor database usage and operation;
- statistical analysis programs, to examine performance or usage statistics;
- index reorganization facilities, to reorganize indexes and their overflows;
- garbage collection and reallocation, to remove deleted records physically from the storage devices, to consolidate the space released, and to reallocate it where it is needed.

Chapter Summary

- The ANSI-SPARC database architecture uses **three levels** of abstraction: **external**, **conceptual**, and **internal**. The **external level** consists of the users' views of the database. The **conceptual level** is the community view of the database: it specifies the information content of the entire database, independent of storage considerations. The conceptual level represents all entities, their attributes, and their relationships, as well as the constraints on the data, and security and integrity information. The **internal level** is the computer's view of the database: it specifies how data is represented, how records are sequenced, what indexes and pointers exist, and so on.
- The **external/conceptual mapping** transforms requests and results between the external and conceptual levels. The **conceptual/internal mapping** transforms requests and results between the conceptual and internal levels.
- A **database schema** is a description of the database structure. There are three different types of schema in the database; these are defined according to the three levels of the ANSI-SPARC architecture. Data independence makes each level immune to changes to lower levels. **Logical data independence** refers to the immunity of the external schemas to changes in the conceptual schema. **Physical data independence** refers to the immunity of the conceptual schema to changes in the internal schema.
- A data sublanguage consists of two parts: a **Data Definition Language (DDL)** and a **Data Manipulation Language (DML)**. The DDL is used to specify the database schema and the DML is used to both read and update the database. The part of a DML that involves data retrieval is called a **query language**.

- A **data model** is a collection of concepts that can be used to describe a set of data, the operations to manipulate the data, and a set of integrity constraints for the data. They fall into three broad categories: **object-based** data models, **record-based** data models, and **physical** data models. The first two are used to describe data at the conceptual and external levels; the latter is used to describe data at the internal level.
- Object-based data models include the Entity–Relationship, semantic, functional, and object-oriented models. Record-based data models include the relational, network, and hierarchical models.
- **Conceptual modeling** is the process of constructing a detailed architecture for a database that is independent of implementation details, such as the target DBMS, application programs, programming languages, or any other physical considerations. The design of the conceptual schema is critical to the overall success of the system. It is worth spending the time and effort necessary to produce the best possible conceptual design.
- **Functions** and **services** of a multi-user DBMS include data storage, retrieval, and update; a user-accessible catalog; transaction support; concurrency control and recovery services; authorization services; support for data communication; integrity services; services to promote data independence; and utility services.
- The **system catalog** is one of the fundamental components of a DBMS. It contains “data about the data,” or **metadata**. The catalog should be accessible to users. The Information Resource Dictionary System is an ISO standard that defines a set of access methods for a data dictionary. This standard allows dictionaries to be shared and transferred from one system to another.



Review Questions

- 2.1 Explain the concept of database schema and discuss the three types of schema in a database.
- 2.2 What are data sublanguages? Why are they important?
- 2.3 What is a data model? Discuss the main types of data model.
- 2.4 Discuss the function and importance of conceptual modeling.
- 2.5 Describe the types of facility that you would expect to be provided in a multi-user DBMS.
- 2.6 Of the facilities described in your answer to Question 2.5, which ones do you think would *not* be needed in a standalone PC DBMS? Provide justification for your answer.
- 2.7 Discuss the function and importance of the system catalog.
- 2.8 Discuss the differences between DDL and DML. What operations would you typically expect to be available in each language?
- 2.9 Discuss the differences between procedural DMLs and nonprocedural DMLs.
- 2.10 Name four object-based data models.
- 2.11 Name three record-based data models. Discuss the main differences between these data models.
- 2.12 What is a transaction? Give an example of a transaction.
- 2.13 What is concurrency control and why does a DBMS need a concurrency control facility?
- 2.14 Define the term “database integrity”. How does database integrity differ from database security?



Exercises

- 2.15 Analyze the DBMSs that you are currently using. Determine each system's compliance with the functions that we would expect to be provided by a DBMS. What type of language does each system provide? What type of architecture does each DBMS use? Check the accessibility and extensibility of the system catalog. Is it possible to export the system catalog to another system?
- 2.16 Write a program that stores names and telephone numbers in a database. Write another program that stores names and addresses in a database. Modify the programs to use external, conceptual, and internal schemas. What are the advantages and disadvantages of this modification?
- 2.17 Write a program that stores names and dates of birth in a database. Extend the program so that it stores the format of the data in the database: in other words, create a system catalog. Provide an interface that makes this system catalog accessible to external users.
- 2.17 Write a program that stores names and dates of birth in a database. Extend the program so that it stores the format of the data in the database: in other words, create a system catalog. Provide an interface that makes this system catalog accessible to external users.
- 2.18 A database approach uses different data models. Common database models include the relational model, the network model and the hierarchical model. Which data model should be chosen under which circumstances and why?

Database Architectures and the Web

Chapter Objectives

In this chapter you will learn:

- The meaning of the client–server architecture and the advantages of this type of architecture for a DBMS.
- The difference between two-tier, three-tier and n -tier client–server architectures.
- The function of an application server.
- The meaning of middleware and the different types of middleware that exist.
- The function and uses of Transaction Processing (TP) Monitors.
- The purpose of a Web service and the technological standards used to develop a Web service.
- The meaning of service-oriented architecture (SOA).
- The difference between distributed DBMSs, and distributed processing.
- The architecture of a data warehouse.
- About cloud computing and cloud databases.
- The software components of a DBMS.
- About Oracle's logical and physical structure.

In Chapter 1, we provided a summary of the historical development of database systems from the 1960s onwards. During this period, although a better understanding of the functionality that users required was gained and new underlying data models were proposed and implemented, the software paradigm used to develop software systems more generally was undergoing significant change. Database system vendors had to recognize these changes and adapt to them to ensure that their systems were current with the latest thinking. In this chapter, we examine the different architectures that have been used and examine emerging developments in Web services and service-oriented architectures (SOA).

Structure of this Chapter In Section 3.1 we examine multi-user DBMS architectures focusing on the two-tier client-server architecture and the three-tier client-server architecture. We also examine the concept of middleware and discuss the different types of middleware that exist in the database field. In Section 3.2 we examine Web services which can be used to provide new types of business services to users, and SOA that promotes the design of loosely coupled and autonomous services that can be combined to provide more flexible composite business processes and applications. In Section 3.3 we briefly describe the architecture for a distributed DBMS that we consider in detail in Chapters 24 and 25, and distinguish it from distributed processing. In Section 3.4 we briefly describe the architecture for a data warehouse and associated tools that we consider in detail in Chapters 31–34. In Section 3.5 we discuss cloud computing and cloud databases. In Section 3.6 we examine an abstract internal architecture for a DBMS and in Section 3.7 we examine the logical and physical architecture of the Oracle DBMS.

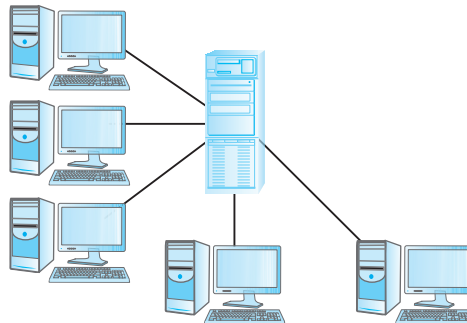
3.1 Multi-user DBMS Architectures

In this section we look at the common architectures that are used to implement multi-user database management systems: teleprocessing, file-server, and client-server.

3.1.1 Teleprocessing

The traditional architecture for multi-user systems was teleprocessing, where there is one computer with a single central processing unit (CPU) and a number of terminals, as illustrated in Figure 3.1. All processing is performed within the boundaries of the same physical computer. User terminals are typically “dumb” ones, incapable of functioning on their own, and cabled to the central computer. The terminals send messages via the communications control subsystem of the operating system to the user’s application program, which in turn uses the services of

Figure 3.1
Teleprocessing
topology.



the DBMS. In the same way, messages are routed back to the user's terminal. Unfortunately, this architecture placed a tremendous burden on the central computer, which had to not only run the application programs and the DBMS, but also carry out a significant amount of work on behalf of the terminals (such as formatting data for display on the screen).

In recent years, there have been significant advances in the development of high-performance personal computers and networks. There is now an identifiable trend in industry towards **downsizing**, that is, replacing expensive main-frame computers with more cost-effective networks of personal computers that achieve the same, or even better, results. This trend has given rise to the next two architectures: file-server and client-server.

3.1.2 File-Server Architecture

File server

A computer attached to a network with the primary purpose of providing shared storage for computer files such as documents, spreadsheets, images, and databases.

In a file-server environment, the processing is distributed about the network, typically a local area network (LAN). The file-server holds the files required by the applications and the DBMS. However, the applications and the DBMS run on each workstation, requesting files from the file-server when necessary, as illustrated in Figure 3.2. In this way, the file-server acts simply as a shared hard disk drive. The DBMS on each workstation sends requests to the file-server for all data that the DBMS requires that is stored on disk. This approach can generate a significant amount of network traffic, which can lead to performance problems. For example, consider a user request that requires the names of staff who work in the branch at 163 Main St. We can express this request in SQL (see Chapter 6) as:

```
SELECT fName, lName
FROM Branch b, Staff s
WHERE b.branchNo = s.branchNo AND b.street = '163 Main St';
```

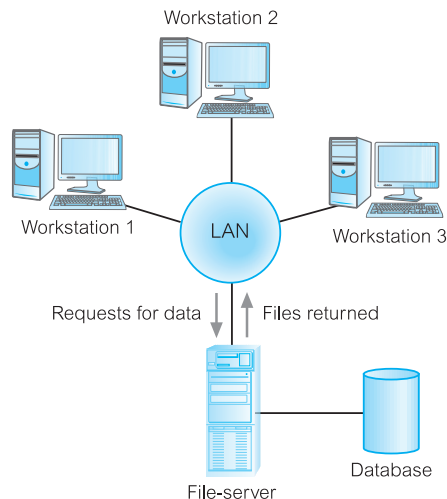


Figure 3.2
File-server
architecture.

As the file-server has no knowledge of SQL, the DBMS must request the files corresponding to the **Branch** and **Staff** relations from the file-server, rather than just the staff names that satisfy the query.

The file-server architecture, therefore, has three main disadvantages:

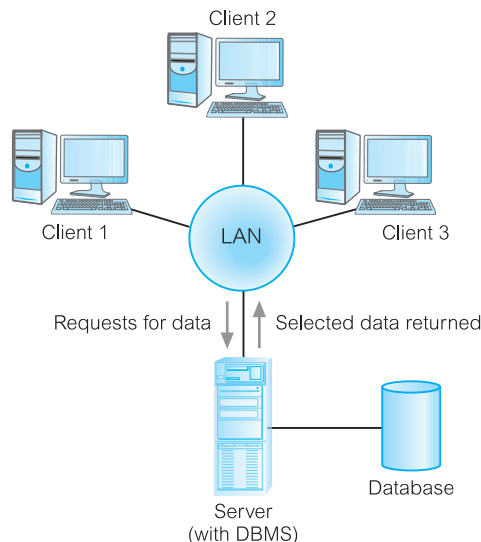
- (1) There is a large amount of network traffic.
- (2) A full copy of the DBMS is required on each workstation.
- (3) Concurrency, recovery, and integrity control are more complex, because there can be multiple DBMSs accessing the same files.

3.1.3 Traditional Two-Tier Client–Server Architecture

To overcome the disadvantages of the first two approaches and accommodate an increasingly decentralized business environment, the client–server architecture was developed. Client–server refers to the way in which software components interact to form a system. As the name suggests, there is a **client** process, which requires some resource, and a **server**, which provides the resource. There is no requirement that the client and server must reside on the same machine. In practice, it is quite common to place a server at one site in a LAN and the clients at the other sites. Figure 3.3 illustrates the client–server architecture and Figure 3.4 shows some possible combinations of the client–server topology.

Data-intensive business applications consist of four major components: the database, the transaction logic, the business and data application logic, and the user interface. The traditional two-tier client–server architecture provides a very basic separation of these components. The client (tier 1) is primarily responsible for the *presentation* of data to the user, and the server (tier 2) is primarily responsible for supplying *data services* to the client, as illustrated in Figure 3.5. Presentation services handle user interface actions and the main business and data application logic. Data services provide limited business application logic, typically validation

Figure 3.3
Client–server
architecture.



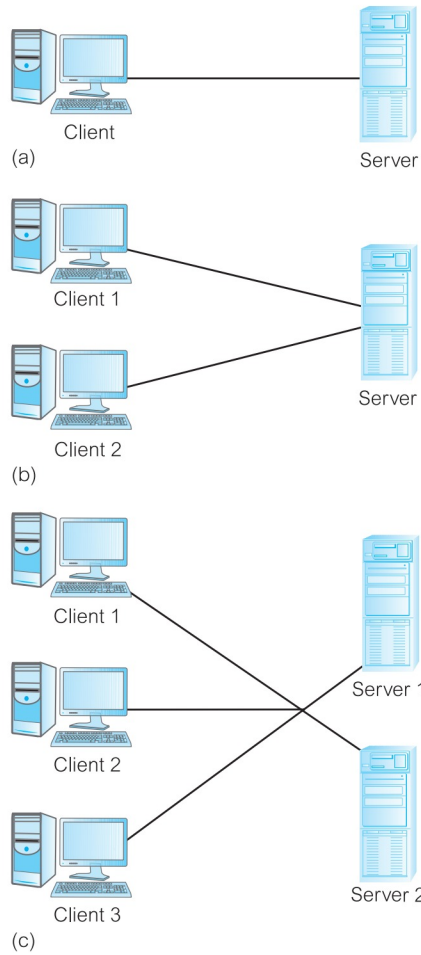


Figure 3.4
Alternative client-server topologies: (a) single client, single server; (b) multiple clients, single server; (c) multiple clients, multiple servers.

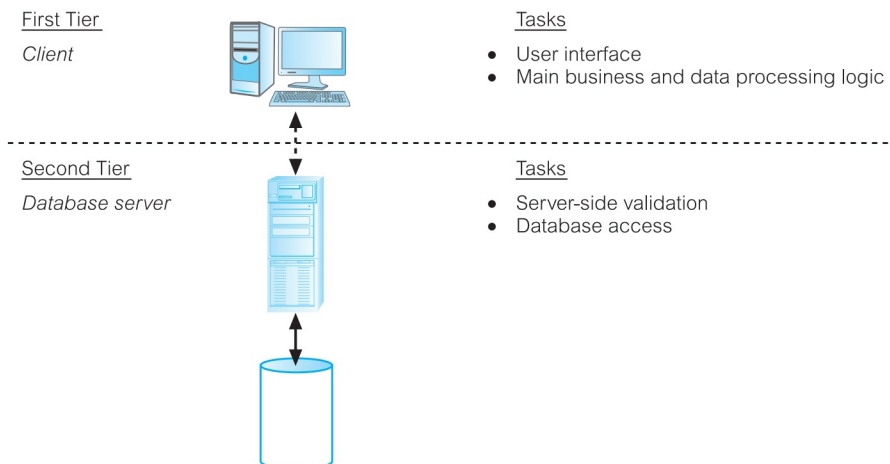


Figure 3.5
The traditional two-tier client-server architecture.

that the client is unable to carry out due to lack of information, and access to the requested data, independent of its location. The data can come from relational DBMSs, object-relational DBMSs, object-oriented DBMSs, legacy DBMSs, or proprietary data access systems. Typically, the client would run on end-user desktops and interact with a centralized database server over a network.

A typical interaction between client and server is as follows. The client takes the user’s request, checks the syntax, and generates database requests in SQL or another database language appropriate to the application logic. It then transmits the message to the server, waits for a response, and formats the response for the end-user. The server accepts and processes the database requests, then transmits the results back to the client. The processing involves checking authorization, ensuring integrity, maintaining the system catalog, and performing query and update processing. In addition, it also provides concurrency and recovery control. The operations of client and server are summarized in Table 3.1.

There are many advantages to this type of architecture. For example:

- It enables wider access to existing databases.
- Increased performance: If the clients and server reside on different computers, then different CPUs can be processing applications in parallel. It should also be easier to tune the server machine if its only task is to perform database processing.
- Hardware costs may be reduced: It is only the server that requires storage and processing power sufficient to store and manage the database.
- Communication costs are reduced: Applications carry out part of the operations on the client and send only requests for database access across the network, resulting in less data being sent across the network.
- Increased consistency: The server can handle integrity checks, so that constraints need be defined and validated only in the one place, rather than having each application program perform its own checking.
- It maps on to open systems architecture quite naturally.

Some database vendors have used this architecture to indicate distributed database capability, that is, a collection of multiple, logically interrelated databases distributed over a computer network. However, although the client–server architecture can be used to provide distributed DBMSs, by itself it does not constitute a distributed DBMS. We discuss distributed DBMSs briefly in Section 3.3 and more fully in Chapters 24 and 25.

TABLE 3.1 Summary of client–server functions.

CLIENT	SERVER
Manages the user interface	Accepts and processes database requests from clients
Accepts and checks syntax of user input	Checks authorization
Processes application logic	Ensures integrity constraints not violated
Generates database requests and transmits to server	Performs query/update processing and transmits response to client
Passes response back to user	Maintains system catalog Provides concurrent database access Provides recovery control

3.1.4 Three-Tier Client–Server Architecture

The need for enterprise scalability challenged the traditional two-tier client–server model. In the mid 1990s, as applications became more complex and could potentially be deployed to hundreds or thousands of end-users, the client side presented two problems that prevented true scalability:

- A “fat” client, requiring considerable resources on the client’s computer to run effectively. This includes disk space, RAM, and CPU power.
- A significant client-side administration overhead.

By 1995, a new variation of the traditional two-tier client–server model appeared to solve the problem of enterprise scalability. This new architecture proposed three layers, each potentially running on a different platform:

- (1) The user interface layer, which runs on the end-user’s computer (the *client*).
- (2) The business logic and data processing layer. This middle tier runs on a server and is often called the *application server*.
- (3) A DBMS, which stores the data required by the middle tier. This tier may run on a separate server called the *database server*.

As illustrated in Figure 3.6, the client is now responsible for only the application’s user interface and perhaps performing some simple logic processing, such as input validation, thereby providing a “thin” client. The core business logic of the application now resides in its own layer, physically connected to the client and database server over a LAN or wide area network (WAN). One application server is designed to serve multiple clients.

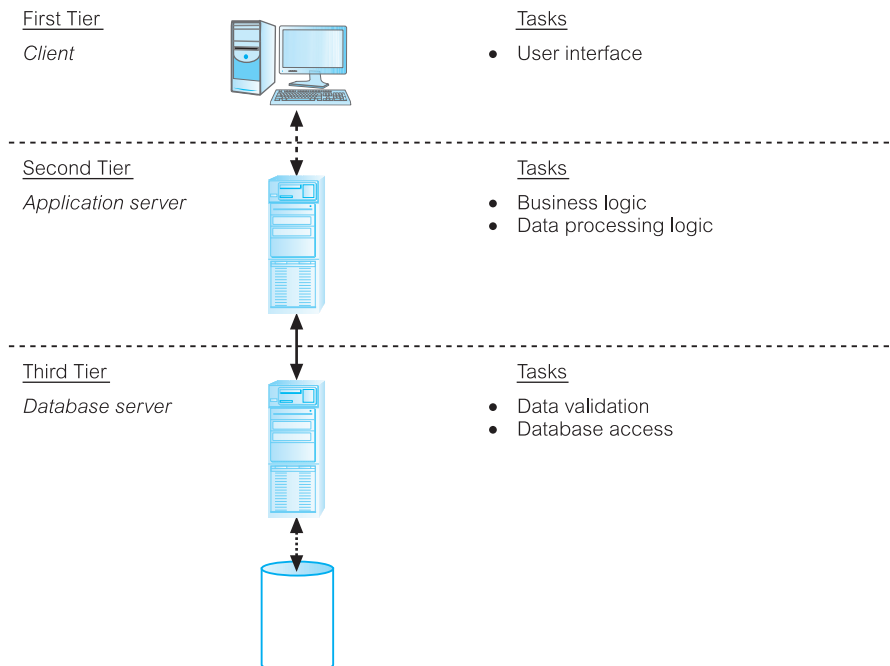


Figure 3.6
The three-tier architecture.

The three-tier design has many advantages over traditional two-tier or single-tier designs, which include:

- The need for less expensive hardware because the client is “thin.”
- Application maintenance is centralized with the transfer of the business logic for many end-users into a single application server. This eliminates the concerns of software distribution that are problematic in the traditional two-tier client–server model.
- The added modularity makes it easier to modify or replace one tier without affecting the other tiers.
- Load balancing is easier with the separation of the core business logic from the database functions.

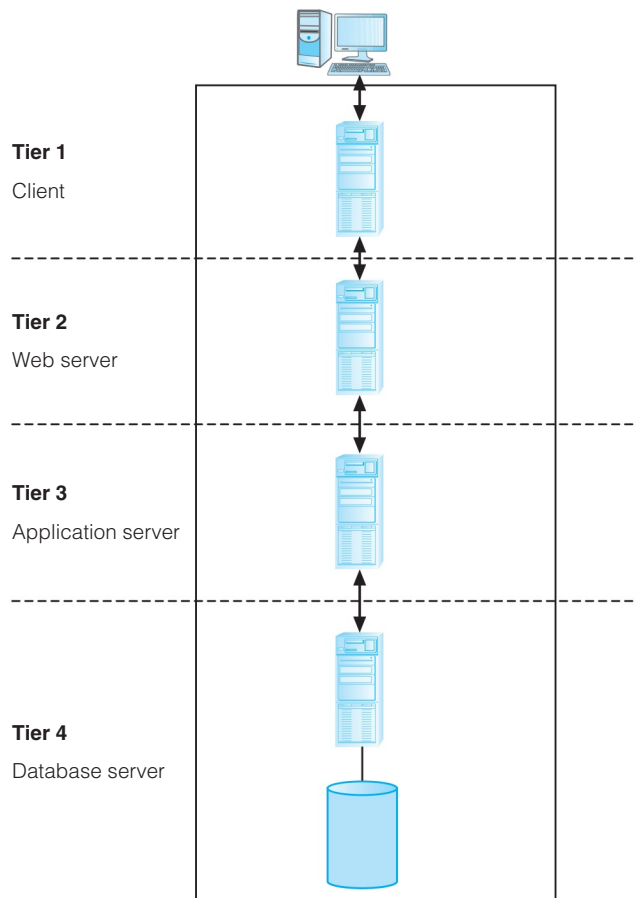
An additional advantage is that the three-tier architecture maps quite naturally to the Web environment, with a Web browser acting as the “thin” client, and a Web server acting as the application server.

3.1.5 *N*-Tier Architectures

The three-tier architecture can be expanded to n tiers, with additional tiers providing more flexibility and scalability. For example, as illustrated in Figure 3.7, the middle tier of the architecture could be split into two, with one tier for the Web

Figure 3.7

Four-tier architecture with the middle tier split into a Web server and application server.



server and another tier for the application server. In environments with a high volume of throughput, the single Web server could be replaced by a set of Web servers (or a *Web farm*) to achieve efficient load balancing.

Application servers

Application server

Hosts an application programming interface (API) to expose business logic and business processes for use by other applications.

An application server must handle a number of complex issues:

- concurrency;
- network connection management;
- providing access to all the database servers;
- database connection pooling;
- legacy database support;
- clustering support;
- load balancing;
- failover.

In Chapter 29 we will examine a number of application servers:

- Java Platform, Enterprise Edition (JEE), previously known as J2EE, is a specification for a platform for server programming in the Java programming language. As with other Java Community Process specifications, JEE is also considered informally to be a standard, as providers must agree to certain conformance requirements in order to declare their products to be “JEE-compliant.” A JEE application server can handle the transactions, security, scalability, concurrency, and management of the components that are deployed to it, meaning that the developers should be able to concentrate more on the business logic of the components rather than on infrastructure and integration tasks.

Some well known JEE application servers are WebLogic Server and Oracle GoldFish Server from Oracle Corporation, JBoss from Red Hat, WebSphere Application Server from IBM, and the open source Glassfish Application Server. We discuss the JEE platform and the technologies associated with accessing databases in Section 29.7.

- .NET Framework is Microsoft’s offering for supporting the development of the middle tier. We discuss Microsoft .NET in Section 29.8.
- Oracle Application Server provides a set of services for assembling a scalable multitier infrastructure to support e-Business. We discuss the Oracle Application Server in Section 29.9.

3.1.6 Middleware

Middleware

Computer software that connects software components or applications.

Middleware is a generic term used to describe software that mediates with other software and allows for communication between disparate applications in a

heterogeneous system. The need for middleware arises when distributed systems become too complex to manage efficiently without a common interface. The need to make heterogeneous systems work efficiently across a network and be flexible enough to incorporate frequent modifications led to the development of middleware, which hides the underlying complexity of distributed systems.

Hurwitz (1998) defines six main types of middleware:

- *Asynchronous Remote Procedure Call (RPC)*: An interprocess communication technology that allows a client to request a service in another address space (typically on another computer across a network) without waiting for a response. An RPC is initiated by the client sending a request message to a known remote server in order to execute a specified procedure using supplied parameters. This type of middleware tends to be highly scalable, as very little information about the connection and the session are maintained by either the client or the server. On the other hand, if the connection is broken, the client has to start over again from the beginning, so the protocol has low recoverability. Asynchronous RPC is most appropriate when transaction integrity is not required.
- *Synchronous RPC*: Similar to asynchronous RPC, however, while the server is processing the call, the client is blocked (it has to wait until the server has finished processing before resuming execution). This type of middleware is the least scalable but has the best recoverability.

There are a number of analogous protocols to RPC, such as:

- Java’s Remote Method Invocation (Java RMI) API provides similar functionality to standard UNIX RPC methods;
- XML-RPC is an RPC protocol that uses XML to encode its calls and HTTP as a transport mechanism. We will discuss HTTP in Chapter 29 and XML in Chapter 30.
- Microsoft .NET Remoting offers RPC facilities for distributed systems implemented on the Windows platform. We will discuss the .NET platform in Section 29.8.
- CORBA provides remote procedure invocation through an intermediate layer called the “Object Request Broker.” We will discuss CORBA in Section 28.1.
- The Thrift protocol and framework for the social networking Web site Facebook.
- *Publish/subscribe*: An asynchronous messaging protocol where *subscribers* subscribe to messages produced by *publishers*. Messages can be categorized into classes and subscribers express interest in one or more classes, and receive only messages that are of interest, without knowledge of what (if any) publishers there are. This decoupling of publishers and subscribers allows for greater scalability and a more dynamic network topology. Examples of publish/subscribe middleware include TIBCO Rendezvous from TIBCO Software Inc. and Ice (Internet Communications Engine) from ZeroC Inc.
- *Message-oriented middleware (MOM)*: Software that resides on both the client and server and typically supports asynchronous calls between the client and server applications. Message queues provide temporary storage when the destination application is busy or not connected. There are many MOM products on the market, including WebSphere MQ from IBM, MSMQ (Microsoft Message Queuing), JMS (Java Messaging Service), which is part of JEE and enables the development

of portable, message-based applications in Java, Sun Java System Message Queue (SJSMQ), which implements JMS, and MessageQ from Oracle Corporation.

- *Object-request broker (ORB)*: Manages communication and data exchange between objects. ORBs promote interoperability of distributed object systems by allowing developers to build systems by integrating together objects, possibly from different vendors, that communicate with each other via the ORB. The Common Object Requesting Broker Architecture (CORBA) is a standard defined by the Object Management Group (OMG) that enables software components written in multiple computer languages and running on multiple computers to work together. An example of a commercial ORB middleware product is Orbix from Progress Software.
- *SQL-oriented data access*: Connects applications with databases across the network and translates SQL requests into the database's native SQL or other database language. SQL-oriented middleware eliminates the need to code SQL-specific calls for each database and to code the underlying communications. More generally, *database-oriented middleware* connects applications to any type of database (not necessarily a relational DBMS through SQL). Examples include Microsoft's ODBC (Open Database Connectivity) API, which exposes a single interface to facilitate access to a database and then uses drivers to accommodate differences between databases, and the JDBC API, which uses a single set of Java methods to facilitate access to multiple databases. Within this category we would also include *gateways*, which act as mediators in distributed DBMSs to translate one database language or dialect of a language into to another language or dialect (for example, Oracle SQL into IBM's DB2 SQL or Microsoft SQL Server SQL into Object Query Language, or OQL). We will discuss gateways in Chapter 24 and ODBC and JDBC in Chapter 29.

In the next section, we examine one particular type of middleware for transaction processing.

3.1.7 Transaction Processing Monitors

TP Monitor

A program that controls data transfer between clients and servers in order to provide a consistent environment, particularly for online transaction processing (OLTP).

Complex applications are often built on top of several **resource managers** (such as DBMSs, operating systems, user interfaces, and messaging software). A Transaction Processing Monitor, or TP Monitor, is a middleware component that provides access to the services of a number of resource managers and provides a uniform interface for programmers who are developing transactional software. A TP Monitor forms the middle tier of a three-tier architecture, as illustrated in Figure 3.8. TP Monitors provide significant advantages, including:

- *Transaction routing*: The TP Monitor can increase scalability by directing transactions to specific DBMSs.
- *Managing distributed transactions*: The TP Monitor can manage transactions that require access to data held in multiple, possibly heterogeneous, DBMSs. For example, a transaction may require to update data items held in an Oracle DBMS at site 1, an Informix DBMS at site 2, and an IMS DBMS at site 3. TP Monitors normally

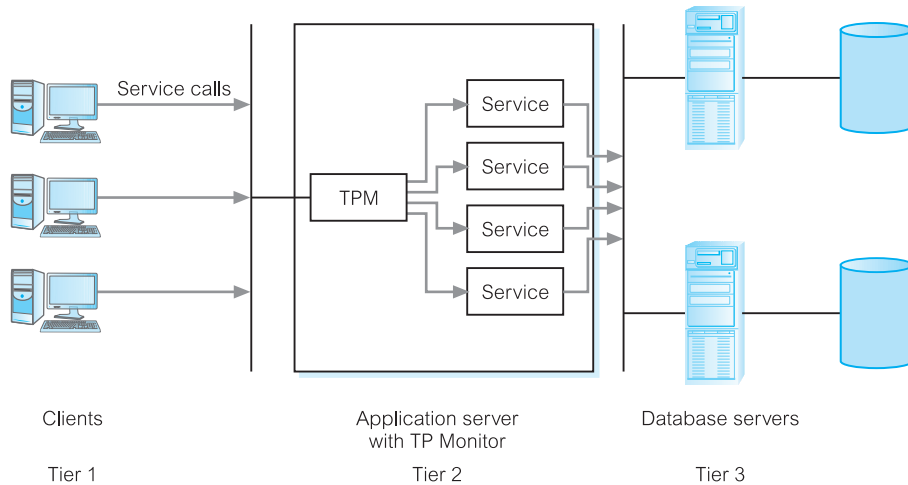


Figure 3.8 The Transaction Processing Monitor as the middle tier of a three-tier client-server architecture.

control transactions using the X/Open Distributed Transaction Processing (DTP) standard. A DBMS that supports this standard can function as a resource manager under the control of a TP Monitor acting as a transaction manager. We discuss distributed transactions and the DTP standard in Chapters 24 and 25.

- *Load balancing:* The TP Monitor can balance client requests across multiple DBMSs on one or more computers by directing client service calls to the least loaded server. In addition, it can dynamically bring in additional DBMSs as required to provide the necessary performance.
- *Funneling:* In environments with a large number of users, it may sometimes be difficult for all users to be logged on simultaneously to the DBMS. In many cases, we would find that users generally do not need continuous access to the DBMS. Instead of each user connecting to the DBMS, the TP Monitor can establish connections with the DBMSs as and when required, and can funnel user requests through these connections. This allows a larger number of users to access the available DBMSs with a potentially much smaller number of connections, which in turn would mean less resource usage.
- *Increased reliability:* The TP Monitor acts as a *transaction manager*, performing the necessary actions to maintain the consistency of the database, with the DBMS acting as a *resource manager*. If the DBMS fails, the TP Monitor may be able to resubmit the transaction to another DBMS or can hold the transaction until the DBMS becomes available again.

TP Monitors are typically used in environments with a very high volume of transactions, where the TP Monitor can be used to offload processes from the DBMS server. Prominent examples of TP Monitors include CICS (which is used primarily on IBM mainframes under z/OS and z/VSE) and Tuxedo from Oracle Corporation. In addition, the Java Transaction API (JTA), one of the Java Enterprise Edition (JEE) APIs, enables distributed transactions to be performed across multiple X/Open XA resources in a Java environment. Open-source implementations of JTA include JBossTS, formerly known as Arjuna Transaction Service, from Red Hat and Bitronix Transaction Manager from Bitronix.

3.2 Web Services and Service-Oriented Architectures

3.2.1 Web Services

Web service

A software system designed to support interoperable machine-to-machine interaction over a network.

Although it has been only about 20 years since the conception of the Internet, in this relatively short period of time it has profoundly changed many aspects of society, including business, government, broadcasting, shopping, leisure, communication, education, and training. Though the Internet has allowed companies to provide a wide range of services to users, sometimes called B2C (Business to Consumer), Web services allow applications to integrate with other applications across the Internet and may be a key technology that supports B2B (Business to Business) interaction.

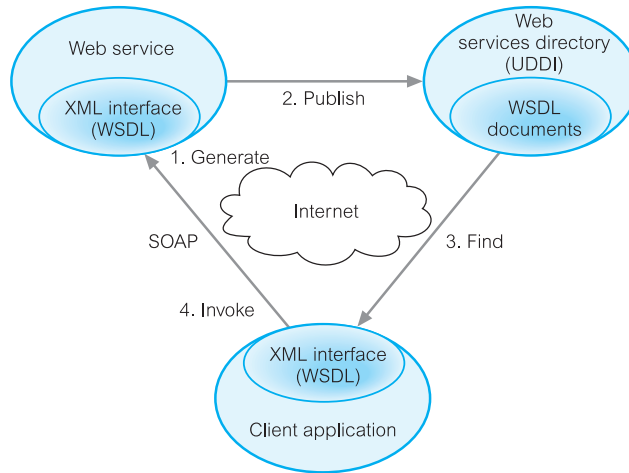
Unlike other Web-based applications, Web services have no user interface and are not aimed at Web browsers. Web services instead share business logic, data, and processes through a programmatic interface across a network. In this way, it is the applications that interface and not the users. Developers can then add the Web service to a Web page (or an executable program) to offer specific functionality to users. Examples of Web services include:

- Microsoft Bing Maps and Google Maps Web services provide access to location-based services, such as maps, driving directions, proximity searches, and geocoding (that is, converting addresses into geographic coordinates) and reverse geocoding.
- Amazon Simple Storage Service (Amazon S3) is a simple Web services interface that can be used to store and retrieve large amounts of data, at any time, from anywhere on the Web. It gives any developer access to the same highly scalable, reliable, fast, inexpensive data storage infrastructure that Amazon uses to run its own global network of Web sites. Charges are based on the “pay-as-you-go” policy, currently \$0.125 per GB for the first 50TB/month of storage used.
- Geonames provides a number of location-related Web services; for example, to return a set of Wikipedia entries as XML documents for a given place name or to return the time zone for a given latitude/longitude.
- DOTS Web services from Service Objects Inc., an early adopter of Web services, provide a range of services such as company information, reverse telephone number lookup, email address validation, weather information, IP address-to-location determination.
- Xignite is a B2B Web service that allows companies to incorporate financial information into their applications. Services include US equities information, real-time securities quotes, US equities pricing, and financial news.

Key to the Web services approach is the use of widely accepted technologies and standards, such as:

- XML (extensible Markup Language).
- SOAP (Simple Object Access Protocol) is a communication protocol for exchanging structured information over the Internet and uses a message format based on XML. It is both platform- and language-independent.

Figure 3.9
Relationship
between WSDL,
UDDI, and
SOAP.



- WSDL (Web Services Description Language) protocol, again based on XML, is used to describe and locate a Web service.
- UDDI (Universal Discovery, Description, and Integration) protocol is a platform-independent, XML-based registry for businesses to list themselves on the Internet. It was designed to be interrogated by SOAP messages and to provide access to WSDL documents describing the protocol bindings and message formats required to interact with the Web services listed in its directory.

Figure 3.9 illustrates the relationship between these technologies. From the database perspective, Web services can be used both from within the database (to invoke an external Web service as a *consumer*) and the Web service itself can access its own database (as a *provider*) to maintain the data required to provide the requested service.

RESTful Web services

Web API is a development in Web services where emphasis has been moving away from SOAP-based services towards Representational State Transfer (REST) based communications. REST services do not require XML, SOAP, WSDL, or UDDI definitions. REST is an architectural style that specifies constraints, such as a uniform interface, that if applied to a Web service creates desirable properties, such as performance, scalability, and modifiability, that enable services to work best on the Web. In the REST architectural style, data and functionality are considered resources and are accessed using Uniform Resource Identifiers (URIs), generally links on the Web. The resources are acted upon by using a set of simple, well-defined HTML operations for create, read, update, and delete: PUT, GET, POST, and DELETE. PUT creates a new resource, which can be then deleted by using DELETE. GET retrieves the current state of a resource in some representation. POST transfers a new state into a resource.

REST adopts a client-server architecture and is designed to use a stateless communication protocol, typically HTTP. In the REST architecture style, clients and servers exchange representations of resources by using a standardized interface and protocol.

We will discuss Web services, HTML, and URIs in Section 29.2.5 and SOAP, WSDL, UDDI in Chapters 29 and 30.

3.2.2 Service-Oriented Architectures (SOA)

SOA

A business-centric software architecture for building applications that implement business processes as sets of services published at a granularity relevant to the service consumer. Services can be invoked, published, and discovered, and are abstracted away from the implementation using a single standards-based form of interface.

Flexibility is recognized as a key requirement for businesses in a time when IT is providing business opportunities that were never envisaged in the past while at the same time the underlying technologies are rapidly changing. Reusability has often been seen as a major goal of software development and underpins the object-oriented paradigm: object-oriented programming (OOP) may be viewed as a collection of cooperating *objects*, as opposed to a traditional view in which a program may be seen as a group of tasks to compute. In OOP, each object is capable of receiving messages, processing data, and sending messages to other objects. In traditional IT architectures, business process activities, applications, and data tend to be locked in independent, often-incompatible “silos,” where users have to navigate separate networks, applications, and databases to conduct the chain of activities that complete a business process. Unfortunately, independent silos absorb an inordinate amount of IT budget and staff time to maintain. This architecture is illustrated in Figure 3.10(a) where we have three processes: Service Scheduling, Order Processing, and Account Management, each accessing a number of databases. Clearly there are common “services” in the activities to be performed by these processes. If the business requirements change or new opportunities present themselves, the lack of independence among these processes may lead to difficulties in quickly adapting these processes.

The SOA approach attempts to overcome this difficulty by designing loosely coupled and autonomous *services* that can be combined to provide flexible composite business processes and applications. Figure 3.10(b) illustrates the same business processes rearchitected to use a service-oriented approach.

The essence of a service, therefore, is that the provision of the service is independent of the application using the service. Service providers can develop specialized services and offer these to a range of service users from different organizations. Applications may be constructed by linking services from various providers using either a standard programming language or a specialized service orchestration language such as BPEL (Business Process Execution Language).

What makes Web services designed for SOA different from other Web services is that they typically follow a number of distinct conventions. The following are a set of common SOA principles that provide a unique design approach for building Web services for SOA:

- *Loose coupling*: Services must be designed to interact on a loosely coupled basis;
- *Reusability*: Logic that can potentially be reused is designed as a separate service;
- *Contract*: Services adhere to a communications contract that defines the information exchange and any additional service description information, specified by one or more service description documents;
- *Abstraction*: Beyond what is described in the service contract, services hide logic from the outside world;

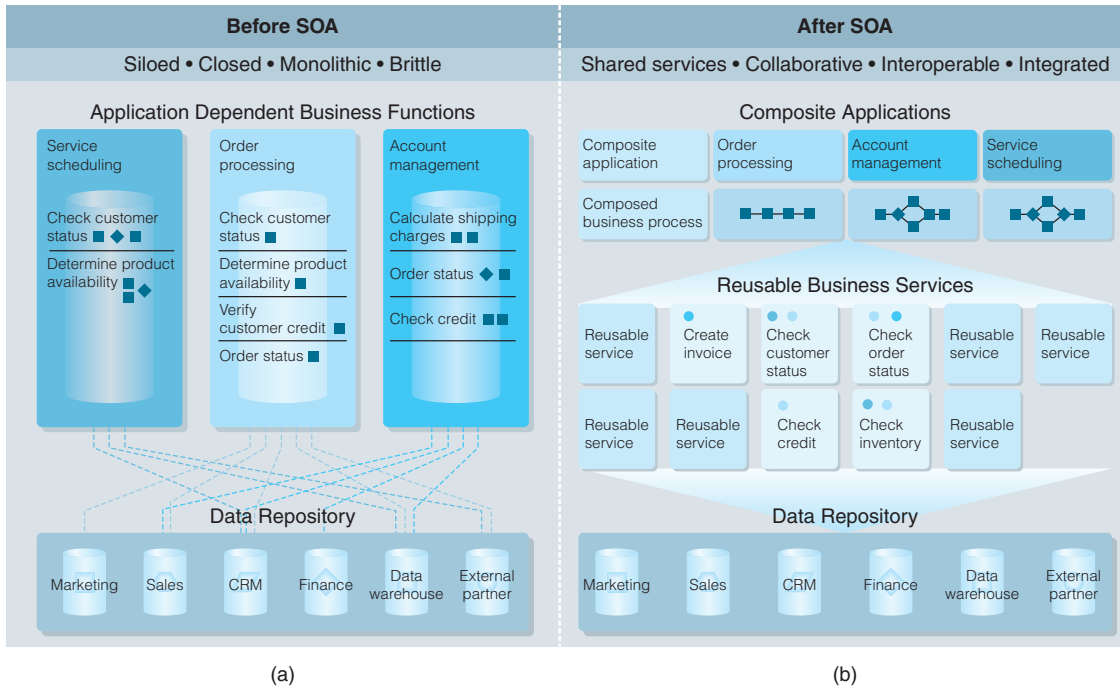


Figure 3.10 (a) Traditional IT architecture for three business processes; (b) service-oriented architecture that splits the processes into a number of reusable services.

- *Composability*: Services may compose other services, so that logic can be represented at different levels of granularity thereby promoting reusability and the creation of abstraction layers;
- *Autonomy*: Services have control over the logic they encapsulate and are not dependent upon other services to execute this governance;
- *Stateless*: Services should not be required to manage state information, as this can affect their ability to remain loosely-coupled;
- *Discoverability*: Services are designed to be outwardly descriptive so that they can be found and assessed via available discovery mechanisms.

Note that SOA is not restricted to Web services and could be used with other technologies. Further discussion of SOA is beyond the scope of this book and the interested reader is referred to the additional reading listed at the end of the book for this chapter.

3.3 Distributed DBMSs

As discussed in Chapter 1, a major motivation behind the development of database systems is the desire to integrate the operational data of an organization and to provide controlled access to the data. Although we may think that integration and controlled access implies centralization, this is not the intention. In fact, the development of computer networks promotes a decentralized mode of work. This

decentralized approach mirrors the organizational structure of many companies, which are logically distributed into divisions, departments, projects, and so on, and physically distributed into offices, plants, or factories, where each unit maintains its own operational data. The development of a distributed DBMS that reflects this organizational structure, makes the data in all units accessible, and stores data proximate to the location where it is most frequently used, should improve the ability to share the data and should improve the efficiency with which we can access the data.

Distributed database

A logically interrelated collection of shared data (and a description of this data), physically distributed over a computer network.

Distributed DBMS

The software system that permits the management of the distributed database and makes the distribution transparent to users.

A **distributed database management system (DDBMS)** consists of a single logical database that is split into a number of *fragments*. Each fragment is stored on one or more computers (*replicas*) under the control of a separate DBMS, with the computers connected by a communications network. Each site is capable of independently processing user requests that require access to local data (that is, each site has some degree of local autonomy) and is also capable of processing data stored on other computers in the network.

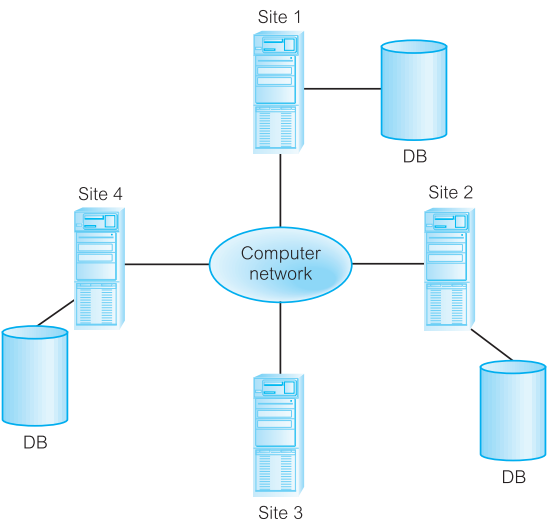
Users access the distributed database via applications. Applications are classified as those that do not require data from other sites (*local applications*) and those that do require data from other sites (*global applications*). We require a DDBMS to have at least one global application. A DDBMS therefore has the following characteristics:

- a collection of logically related shared data;
- data split into a number of fragments;
- fragments may be replicated;
- fragments/replicas are allocated to sites;
- sites are linked by a communications network;
- data at each site is under the control of a DBMS;
- DBMS at each site can handle local applications, autonomously;
- each DBMS participates in at least one global application.

It is not necessary for every site in the system to have its own local database, as illustrated by the topology of the DDBMS shown in Figure 3.11.

From the definition of the DDBMS, the system is expected to make the distribution *transparent* (invisible) to the user. Thus, the fact that a distributed database is split into fragments that can be stored on different computers and perhaps replicated should be hidden from the user. The objective of transparency is to make the distributed system appear like a centralized system. This is sometimes referred to as the *fundamental principle* of distributed DBMSs. This requirement provides significant functionality for the end-user but, unfortunately, creates many additional problems that have to be handled by the DDBMS.

Figure 3.11
Distributed
database
management
system.



Distributed processing

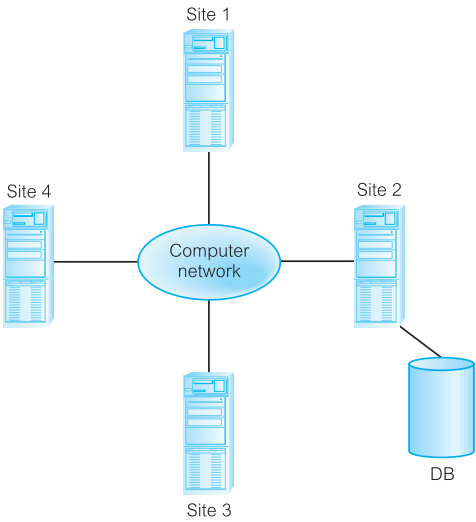
It is important to make a distinction between a distributed DBMS and distributed processing:

Distributed processing

A centralized database that can be accessed over a computer network.

The key point with the definition of a distributed DBMS is that the system consists of data that is physically distributed across a number of sites in the network. If the data is centralized, even though other users may be accessing the data over the network, we do not consider this to be a distributed DBMS simply distributed processing. We illustrate the topology of distributed processing in Figure 3.12. Compare this figure, which has a central database at site 2, with Figure 3.11, which shows several sites each with their own database. We will discuss distributed DBMSs in depth in Chapters 24 and 25.

Figure 3.12
Distributed
processing.



3.4 Data Warehousing

Since the 1970s, organizations have largely focused their investment in new computer systems (called online transaction processing or OLTP systems) that automate business processes. In this way, organizations gained competitive advantage through systems that offered more efficient and cost-effective services to the customer. Throughout this period, organizations accumulated growing amounts of data stored in their operational databases. However, in recent times, where such systems are commonplace, organizations are focusing on ways to use operational data to support decision making as a means of regaining competitive advantage.

Operational systems were never primarily designed to support business decision making and so using such systems may never be an easy solution. The legacy is that a typical organization may have numerous operational systems with overlapping and sometimes contradictory definitions, such as data types. The challenge for an organization is to turn its archives of data into a source of knowledge, so that a single integrated/consolidated view of the organization's data is presented to the user. The concept of a **data warehouse** was deemed the solution to meet the requirements of a system capable of supporting decision making, receiving data from multiple operational data sources.

Data warehouse

A consolidated/integrated view of corporate data drawn from disparate operational data sources and a range of end-user access tools capable of supporting simple to highly complex queries to support decision making.

The data held in a data warehouse is described as being subject-oriented, integrated, time-variant, and nonvolatile (Inmon, 1993).

- *Subject-oriented*, as the warehouse is organized around the major subjects of the organization (such as customers, products, and sales) rather than the major application areas (such as customer invoicing, stock control, and product sales). This is reflected in the need to store decision-support data rather than application-oriented data.
- *Integrated*, because of the coming together of source data from different organization-wide applications systems. The source data is often inconsistent, using, for example, different data types and/or formats. The integrated data source must be made consistent to present a unified view of the data to the users.
- *Time-variant*, because data in the warehouse is accurate and valid only at some point in time or over some time interval.
- *Nonvolatile*, as the data is not updated in real time but is refreshed from operational systems on a regular basis. New data is always added as a supplement to the database, rather than a replacement.

The typical architecture of a data warehouse is shown in Figure 3.13.

The source of *operational data* for the data warehouse is supplied from mainframes, proprietary file systems, private workstations and servers, and external systems such as the Internet. An *operational data store* (ODS) is a repository of current and integrated operational data used for analysis. It is often structured and supplied with data in the same way as the data warehouse, but may in fact act simply as a staging area for data to be moved into the warehouse. The *load manager* performs all the

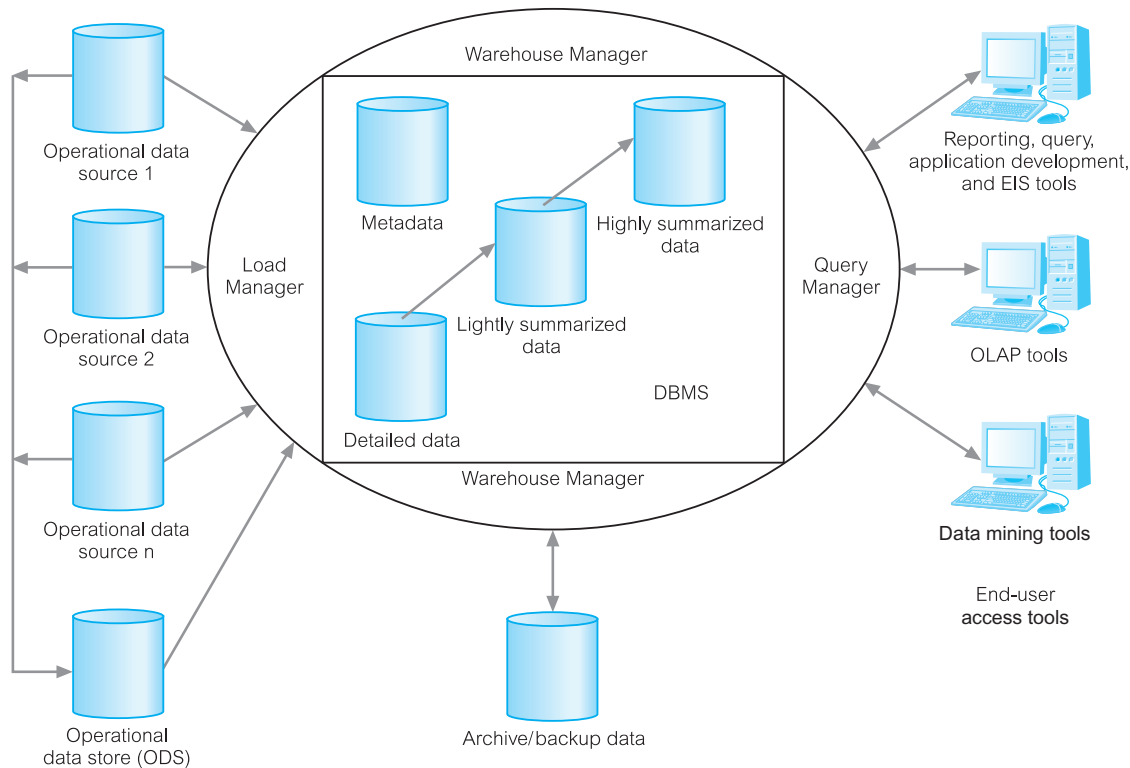


Figure 3.13 The typical architecture of a data warehouse.

operations associated with the extraction and loading of data into the warehouse. The *warehouse manager* performs all the operations associated with the management of the data, such as the transformation and merging of source data; creation of indexes and views on base tables; generation of aggregations, and backing up and archiving data. The *query manager* performs all the operations associated with the management of user queries. *Detailed data* is not stored online but is made available by summarizing the data to the next level of detail. However, on a regular basis, detailed data is added to the warehouse to supplement the summarized data. The warehouse stores all the predefined *lightly and highly summarized data* generated by the warehouse manager. The purpose of summary information is to speed up the performance of queries. Although there are increased operational costs associated with initially summarizing the data, this cost is offset by removing the requirement to continually perform summary operations (such as sorting or grouping) in answering user queries. The summary data is updated continuously as new data is loaded into the warehouse. Detailed and summarized data is stored offline for the purposes of archiving and backup. *Metadata* (data about data) definitions are used by all the processes in the warehouse, including the extraction and loading processes; the warehouse management process; and as part of the query management process.

The principal purpose of data warehousing is to provide information to business users for strategic decision making. These users interact with the warehouse using *end-user access tools*. The data warehouse must efficiently support *ad hoc* and routine

analysis as well as more complex data analysis. The types of end-user access tools typically include reporting and query tools, application development tools, executive information system (EIS) tools, online analytical processing (OLAP) tools, and data mining tools. We will discuss data warehousing, OLAP, and data mining tools in depth in Chapters 31–34.

3.5 Cloud Computing

Cloud computing

A model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction (NIST, 2011)¹.

Cloud computing is the term given to the use of multiple servers over a digital network as if they were one computer. The ‘Cloud’ itself is a virtualization of resources—networks, servers, applications, data storage, and services—which the end-user has on-demand access to. *Virtualization* is the creation of a virtual version of something, such as a server, operating system, storage device, or network resource. The aim is to provide these resources with minimal management or service provider interaction. Cloud computing offers users resources without the requirement of having knowledge of the systems or the location of the systems that deliver them. Additionally, the cloud can provide users with a far greater range of applications and services. Therefore, the aim of the cloud is to provide users and businesses with scalable and tailored services.

NIST views the cloud model as consisting of five essential characteristics, three service models, and four deployment models. The essential characteristics are as follows:

- *On-demand self-service.* Consumers can obtain, configure, and deploy cloud services themselves using cloud service catalogues, without requiring the assistance of anyone from the cloud provider.
- *Broad network access.* The most vital characteristic of cloud computing, namely that it is network based, and accessible from anywhere, from any standardized platform (e.g., desktop computers, laptops, mobile devices).
- *Resource pooling.* The cloud provider’s computing resources are pooled to serve multiple consumers, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand. Examples of resources include storage, processing, memory, and network bandwidth.
- *Rapid elasticity.* Resource pooling avoids the capital expenditure required for the establishment of network and computing infrastructure. By outsourcing to a cloud, consumers can cater for the spikes in demand for their services by using the cloud provider’s computing capacity, and the risk of outages and service interruptions are significantly reduced. Moreover, capabilities can be elastically provisioned and released, in some cases automatically, to scale rapidly based on demand. To the consumer, the capabilities available for provisioning often appear to be unlimited and can be called on in any quantity at any time.

¹ NIST, 2011. The NIST Definition of Cloud Computing, NIST Special Publication 800-145, National Institute of Standards, September 2011.

- *Measured service.* Cloud systems automatically control and optimize resource use by leveraging a metering capability appropriate to the type of service (e.g., storage, processing, bandwidth, and active user accounts). Resource usage can be monitored, controlled, and charged for.

The three service models defined by NIST are as follows:

- *Software as a Service (SaaS).* Software and associated data are centrally hosted on the cloud. SaaS is typically accessed from various client devices through a thin client interface, such as a Web browser. The consumer does not manage or control the underlying cloud infrastructure with the possible exception of limited user-specific application configuration settings. SaaS may be considered the oldest and most mature type of cloud computing. Examples include Salesforce.com sales management applications, NetSuite's integrated business management software, Google's Gmail, and Cornerstone OnDemand.
- *Platform as a Service (PaaS).* PaaS a computing platform that allows the creation of web applications quickly and easily and without the complexity of buying and maintaining the software and infrastructure underneath it. Sometimes, PaaS is used to extend the capabilities of applications developed as SaaS. While earlier application development required hardware, an operating system, a database, middleware, Web servers, and other software, with the PaaS model only the knowledge to integrate them is required. The rest is handled by the PaaS provider. Examples of PaaS include Salesforce.com's Force.com, Google's App Engine, and Microsoft's Azure. The consumer does not manage or control the underlying cloud infrastructure including network, servers, operating systems, or storage, but has control over the deployed applications and possibly configuration settings for the hosting environment.
- *Infrastructure as a Service (IaaS).* IaaS delivers servers, storage, network and operating systems—typically a platform virtualization environment—to consumers as an on-demand service, in a single bundle and billed according to usage. A popular use of IaaS is in hosting Web sites, where the in-house infrastructure is not burdened with this task but left free to manage the business. Amazon's Elastic Compute Cloud (EC2), Rackspace, and GoGrid are examples of IaaS. The consumer does not manage or control the underlying cloud infrastructure but has control over operating systems, storage, and deployed applications, and possibly limited control of select networking components (e.g., firewalls).

These models are illustrated in Figure 3.14.

The four main deployment models for the cloud are:

- *Private cloud.* Cloud infrastructure is operated solely for a single organization, whether managed internally by the organization, a third party, or some combination of them, and it may be hosted internally or externally.
- *Community cloud.* Cloud infrastructure is shared for exclusive use by a specific community of organizations that have common concerns (e.g., security requirements, compliance, jurisdiction). It may be owned and managed by one or more of the organizations in the community, a third party, or some combination of them, and it may be hosted internally or externally.
- *Public cloud.* Cloud infrastructure is made available to the general public by a service provider. These services are free or offered on a pay-per-use model. It may be owned and managed by a business, academic, or government organization, or some combination of these. It exists on the premises of the cloud provider.

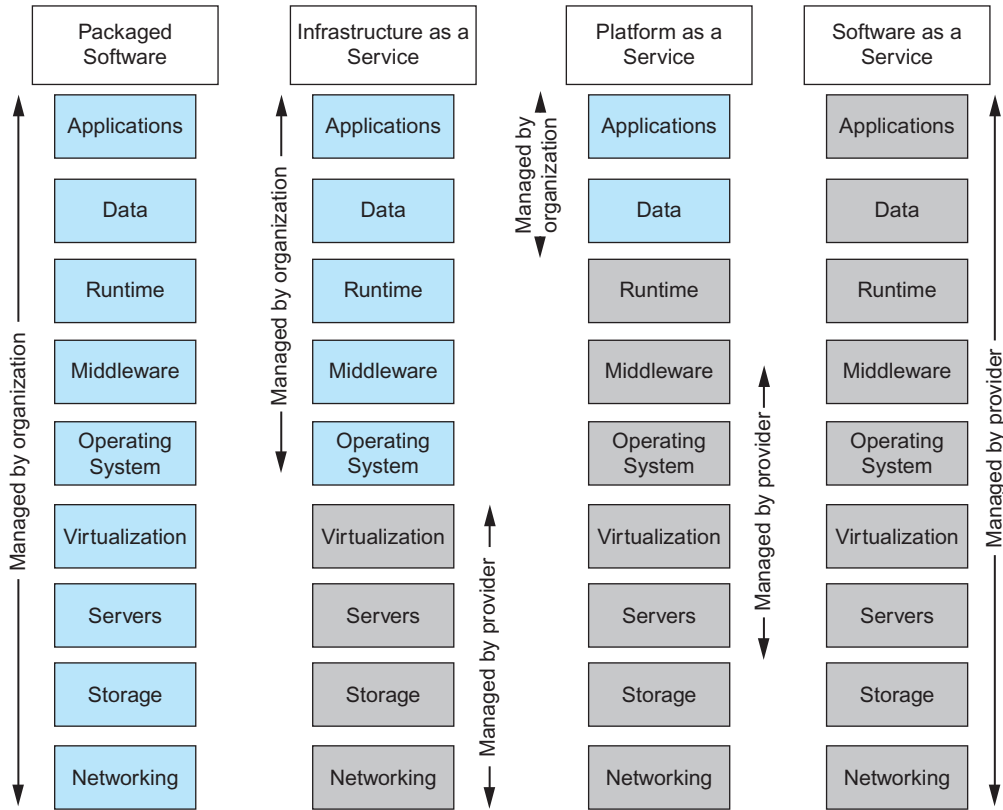


Figure 3.14 Differences between packaged software, IaaS, PaaS, and SaaS.

Generally, public cloud service providers such as Amazon AWS, Microsoft, and Google own and operate the infrastructure and offer access only via the Internet (direct connectivity is not offered).

- *Hybrid cloud.* Cloud infrastructure is a composition of two or more distinct cloud infrastructures (private, community, or public) that remain unique entities, but are bound together by standardized or proprietary technology, offering the benefits of multiple deployment models.

3.5.1 Benefits and Risks of Cloud Computing

Cloud computing offers a number of distinct benefits to companies compared to more traditional approaches:

- *Cost-reduction.* Cloud computing allows organizations to avoid the up-front capital expenditure associated with expensive servers, software licenses, and IT personnel, shifting this responsibility to the cloud provider. This allows smaller firms to have access to powerful computing infrastructure that they might otherwise not have been able to afford. Moreover, organizations only pay for the computing facilities they use rather than pay regardless of whether or not the facilities are being used to full capacity.

- *Scalability/Agility.* Provisioning-on-demand enables organizations to set up the resources they require and remove unwanted resources on an as-needed basis. When a project is funded, the organization initiates the service, and when the project terminates the resources can be relinquished. This supports business growth without requiring expensive changes to an organization's existing IT systems. This is also known as elasticity and is behind Amazon's name Elastic Computing Cloud (EC2).
- *Improved security.* Security can be as good as or even better than in-house systems, because cloud providers can devote resources to solving security issues that many organizations cannot afford. The decentralization of data can also provide increases in security, since not all of an organization's data is stored in one place. Moreover, many smaller organizations do not have the resources needed to manage audit and certification processes for internal data centers. Cloud providers that address compliance standards such as Sarbanes-Oxley, the Payment Card Industry Data Security Standard (PCI DSS), and Health Insurance Portability and Accountability Act (HIPAA) can help organizations address regulatory and compliance processes.
- *Improved reliability.* Again, with devoted resources a cloud provider can focus on increasing the reliability of their systems. The provider can give 24/7 support and high-level computing for all their clients at once, rather than each client requiring its own dedicated in-house staff. This leads to a more redundant and safer system.
- *Access to new technologies.* Cloud computing can help ensure that organizations have access to the latest technology, including access to hardware, software, and IT functionality that would otherwise be too expensive to buy.
- *Faster development.* Cloud computing platforms provide many of the core services that might normally be built in-house. These services, plus templates and other tools, can significantly accelerate the development cycle.
- *Large-scale prototyping/load testing.* Cloud computing makes large scale prototyping and load testing much easier. It would be possible to spawn 1,000 servers in the cloud to load test an application and then release them as soon as the tests were completed, which would be extremely difficult and expensive with in-house servers.
- *More flexible working practices.* Cloud computing allows staff to work more flexibly. Staff can access files using Web-enabled devices such as laptops, netbooks and smartphones. The ability to simultaneously share documents and other files over the Internet can also help support global collaboration and knowledge sharing as well as group decision making.
- *Increased competitiveness.* Delegating commodity infrastructure and services allows organizations to focus on their core competencies and further develop capabilities that can differentiate their organizations in their respective business markets, thereby increasing competitiveness.

As well as benefits, cloud computing also presents some risks to organizations such as:

- *Network dependency.* Perhaps the most obvious drawback to cloud computing is its dependency on the network. Power outages and service interruptions will prevent access to the cloud facilities. In addition, cloud computing is also susceptible to simple bandwidth issues, for example, at peak hours of demand. If the cloud provider's servers are being strained at peak hours, both the performance and availability of its services will suffer. While many cloud providers will have the redundancy to handle their peak load, there is always the risk of unforeseen events, server failures, or outside attack; any of which would threaten client access.

- *System dependency.* An organization may be highly dependent on the availability and reliability of the cloud provider's systems. Should the provider fail to provide adequate disaster recovery, the result could be catastrophic for the organization.
- *Cloud provider dependency.* Ideally, the cloud provider will never go broke or get acquired by a larger company. If this were to happen, it is possible that the service would suddenly terminate and organizations need to ensure they are protected against such an eventuality.
- *Lack of control.* By committing data to the systems managed by a cloud provider, organizations may no longer be in full control of this data and cannot deploy the technical and organisational measures necessary to safeguard the data. As a result, the following problems may arise:
 - *Lack of availability.* If the cloud provider relies on proprietary technology, it may prove difficult for an organization to shift data and documents between different cloud-based systems (data portability) or to exchange information with applications that use cloud services managed by different providers (interoperability).
 - *Lack of integrity.* A cloud is made up of shared systems and infrastructures. Cloud providers process personal data coming from a wide range of data subjects and organisations and it is possible that conflicting interests and/or different objectives might arise.
 - *Lack of confidentiality.* Data being processed in the cloud may be subject to requests from law enforcement agencies. There is a risk that data could be disclosed to (foreign) law enforcement agencies without a valid legal basis and thus a breach of law could occur.
 - *Lack of intervenability.* Due to the complexity and dynamics of the outsourcing chain, the cloud service offered by one provider might be produced by combining services from a range of other providers, which may be dynamically added or removed during the duration of the organization's contract. In addition, a cloud provider may not provide the necessary measures and tools to safeguard organizational data or an individual's personal data in terms of, e.g., access, deletion, or correction of data.
 - *Lack of isolation.* A cloud provider may use its physical control over data from different clients to link personal data. If administrators are facilitated with sufficiently privileged access rights, they could link information from different clients.
- *Lack of information on processing (transparency).* Insufficient information about a cloud service's processing operations poses a risk to data controllers as well as to data subjects because they might not be aware of potential threats and risks and thus cannot take measures they deem appropriate. Some potential threats may arise from the data controller not knowing that:
 - Chain processing is taking place involving multiple processors and subcontractors.
 - Personal data is processed in different geographic locations. This impacts directly on the law applicable to any data protection disputes that may arise between user and provider.
 - Personal data is transferred to third countries outside the organization's control. Third countries may not provide an adequate level of data protection and transfers may not be safeguarded by appropriate measures (e.g., standard contractual clauses or binding corporate rules) and thus may be illegal.

3.5.2 Cloud-Based Database Solutions

As a type of SaaS, cloud-based database solutions fall into two basic categories: Data as a Service (DaaS) and Database as a Service (DBaaS). The key difference between the two is mainly how the data is managed:

- *DaaS*. DaaS offers the ability to define data in the cloud and subsequently query that data on demand. Unlike traditional database solutions, DaaS does not implement typical DBMS interfaces such as SQL (see Chapter 6). Instead, the data is accessed via a common set of APIs. DaaS enables any organization with valuable data to create new revenue lines based on data they already own. Examples of DaaS are Urban Mapping, a geography data service, which provides data for customers to embed into their own Web sites and applications; Xignite, which makes financial data available to customers; and Hoovers, a Dun & Bradstreet company, which provides business data on various organizations.
- *DBaaS*. DBaaS offers full database functionality to application developers. In DBaaS, a management layer is responsible for the continuous monitoring and configuring of the database to achieve optimized scaling, high availability, multi-tenancy (that is, serving multiple client organizations), and effective resource allocation in the cloud, thereby sparing the developer from ongoing database administration tasks. Database as a Service architectures support the following necessary capabilities:
 - Consumer-based provisioning and management of database instances using on-demand, self-service mechanisms;
 - Automated monitoring of and compliance with provider-defined service definitions, attributes and quality of service levels;
 - Fine-grained metering of database usage enabling show-back reporting or charge-back functionality for each individual consumer.

There are a number of architectural issues associated with DBaaS to do with how one organization's data is isolated from other organizations' data. We consider the following options:

- Separate servers
- Shared server, separate database server process
- Shared database server, separate databases
- Shared database, separate schema
- Shared database, shared schema

Separate servers architecture

With the separate server architecture, each tenant has a hosted server machine that only serves their database. This architecture may be appropriate for organizations who require a high degree of isolation, have large databases, a large number of users, or who have very specific performance requirements. This approach tends to lead to higher costs for maintaining equipment and backing up tenant data. The architecture is illustrated in Figure 3.15.

Shared server, separate database server process architecture

With this architecture, each tenant has their own database, however, several tenants share a single server machine, each with their own server process. This architecture

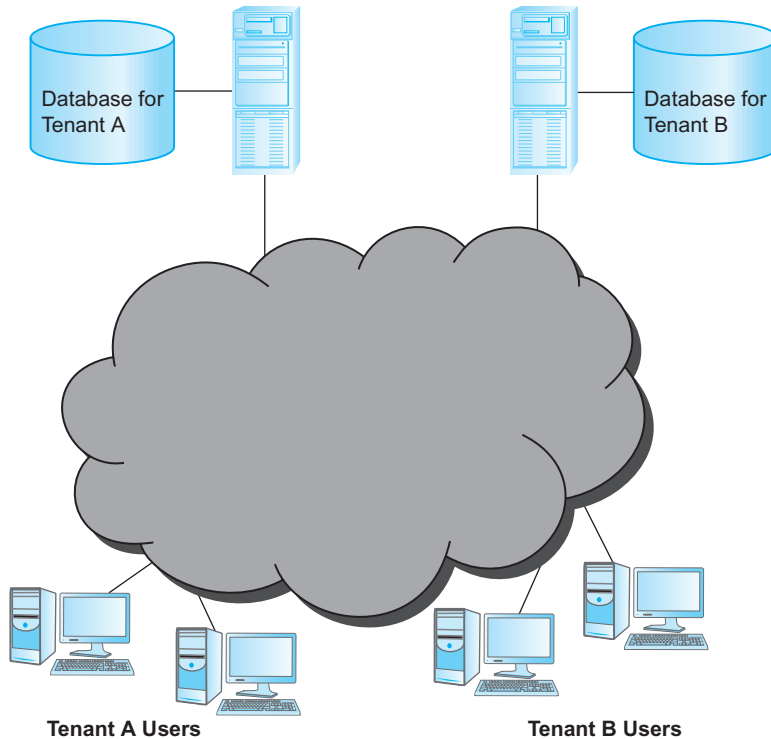


Figure 3.15 Multi-tenant cloud database—separate servers architecture.

represents a common virtualization environment. The resources on a given server machine are subdivided for each tenant. Each virtualized environment reserves its own memory and disk resources, and is unable to share unused resources with other virtualized environments. With this approach, memory and storage may become an issue as each tenant has their own database files and server process. Moreover, performance may be an issue as tenants share the same server although the cloud provider can reduce the risk of this by allocating only a small number of tenants to each server. Security is not an issue because tenants are completely isolated from each other. This architecture is illustrated in Figure 3.16.

Shared database server, separate databases

With this architecture, each tenant has their own separate database but shares a single database server (and single server process) with all other tenants. An improvement over the previous approach is that the single database server process can share resources such as database cache effectively between tenants, supporting better utilization of machine resources. This architecture is illustrated in Figure 3.17.

Shared database, separate schema architecture

With this architecture, there is a single shared database server and each tenant has its data grouped into a schema created specifically for the tenant. The DBMS must

Figure 3.16
Multi-tenant cloud database—shared server, separate database server process architecture

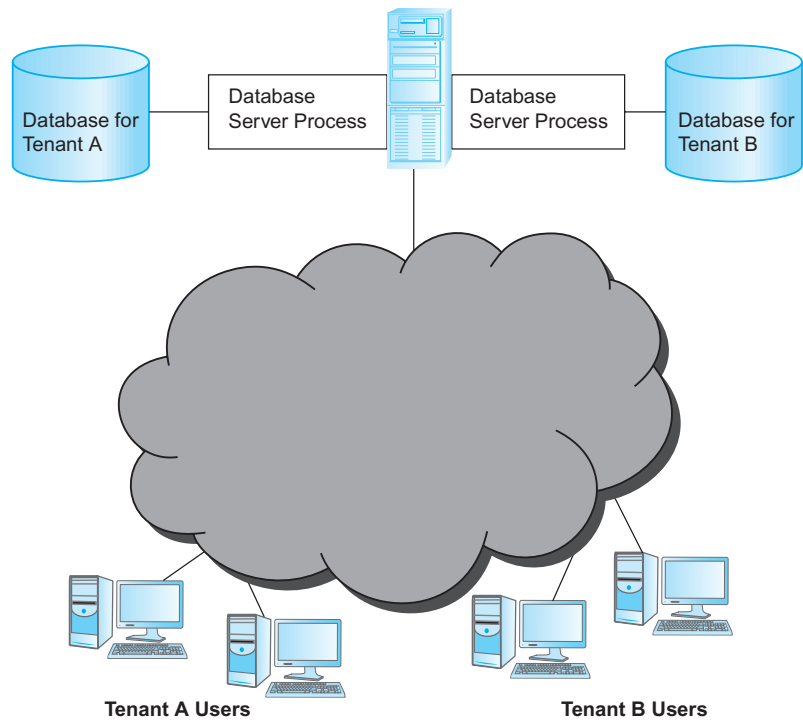
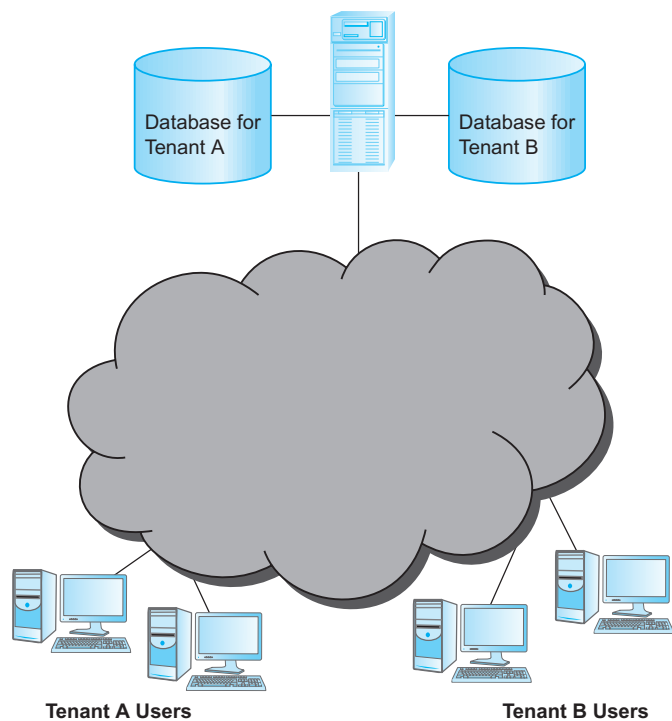


Figure 3.17
Multi-tenant cloud database—shared DBMS server, separate databases.



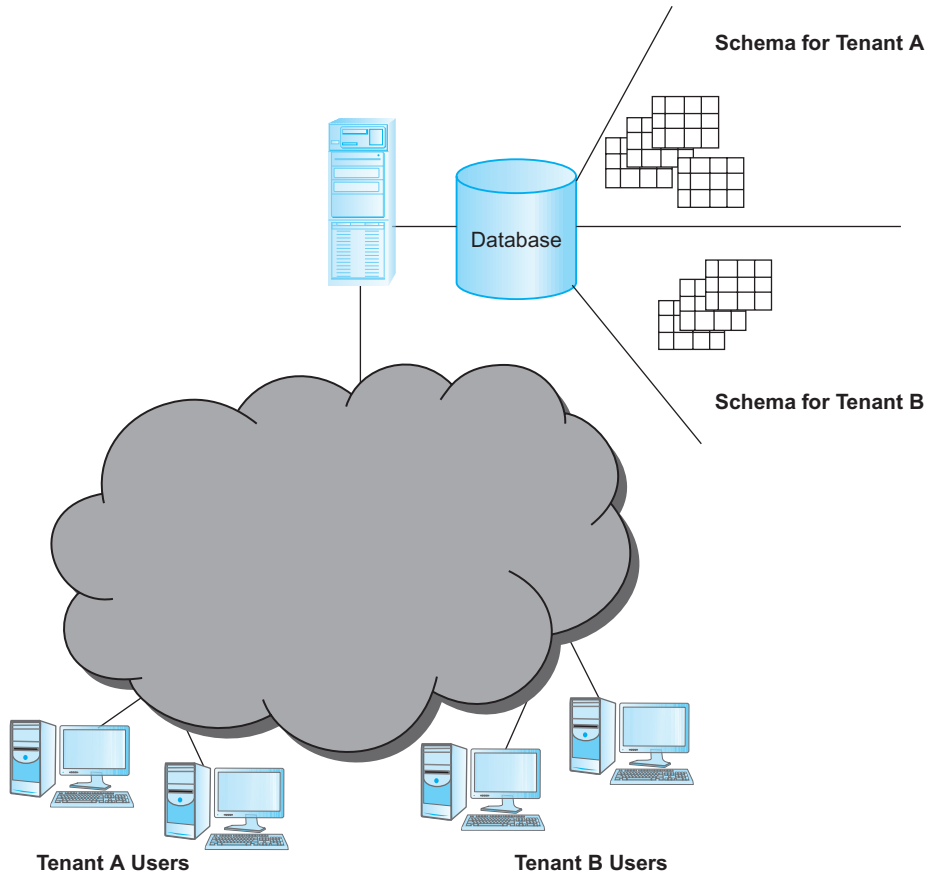


Figure 3.18 Multi-tenant cloud database—shared database, separate schema architecture.

have a permission structure to ensure that users only have access to the data to which they are entitled. This architecture is illustrated in Figure 3.18.

Shared database, shared schema architecture

With this architecture, there is a single shared database server and each tenant has its data grouped into a single shared schema. Each database table must have a column used to identify the owner of the row. Any application accessing the row must refer to this column in every query to ensure that one tenant is not able to see another tenant's data. This approach has the lowest hardware and backup costs, because it supports the largest number of tenants per database server. However, because multiple tenants share the same database tables, this approach may incur additional development effort in security, to ensure that tenants can never access other tenants' data. This approach is appropriate when it is important that the application must serve a large number of tenants with a small number of servers, and prospective customers are willing to surrender data isolation in exchange for the lower costs that this approach makes possible. This architecture is illustrated in Figure 3.19.

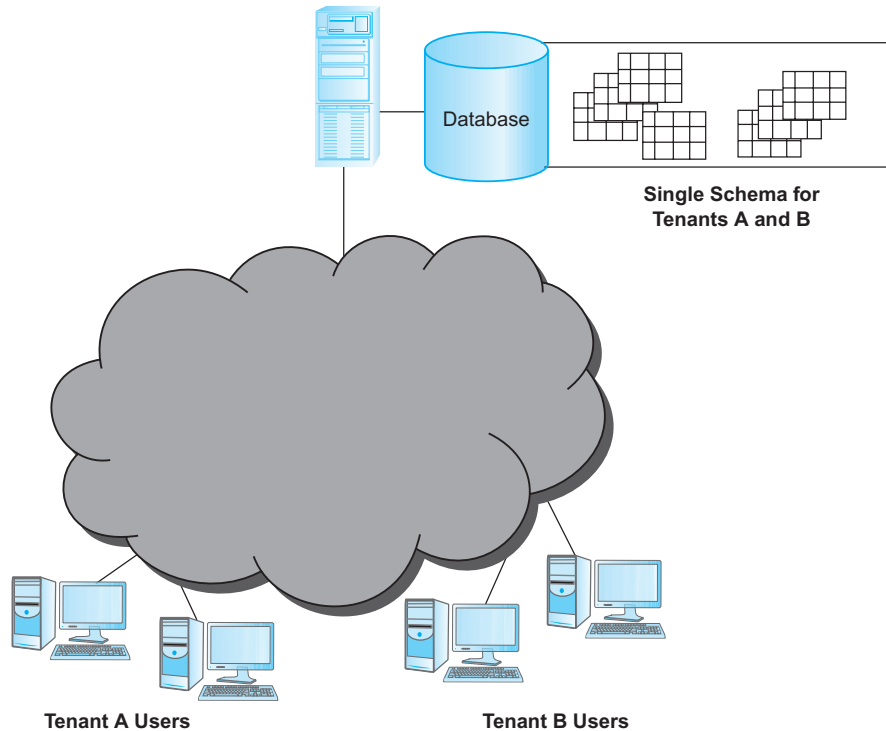


Figure 3.19 Multi-tenant cloud database—shared schema architecture.

3.6 Components of a DBMS

DBMSs are highly complex and sophisticated pieces of software that aim to provide the services discussed in Section 2.4. It is not possible to generalize the component structure of a DBMS, as it varies greatly from system to system. However, it is useful when trying to understand database systems to try to view the components and the relationships between them. In this section, we present a possible architecture for a DBMS. We examine the architecture of the Oracle DBMS in the next section.

A DBMS is partitioned into several software components (or *modules*), each of which is assigned a specific operation. As stated previously, some of the functions of the DBMS are supported by the underlying operating system. However, the operating system provides only basic services and the DBMS must be built on top of it. Thus, the design of a DBMS must take into account the interface between the DBMS and the operating system.

The major software components in a DBMS environment are depicted in Figure 3.20. This diagram shows how the DBMS interfaces with other software components, such as user queries and access methods (file management techniques for storing and retrieving data records). We will provide an overview of file organizations and access methods in Appendix F. For a more comprehensive treatment, the interested reader is referred to Teorey and Fry (1982), Weiderhold (1983), Smith and Barnes (1987), and Ullman (1988).

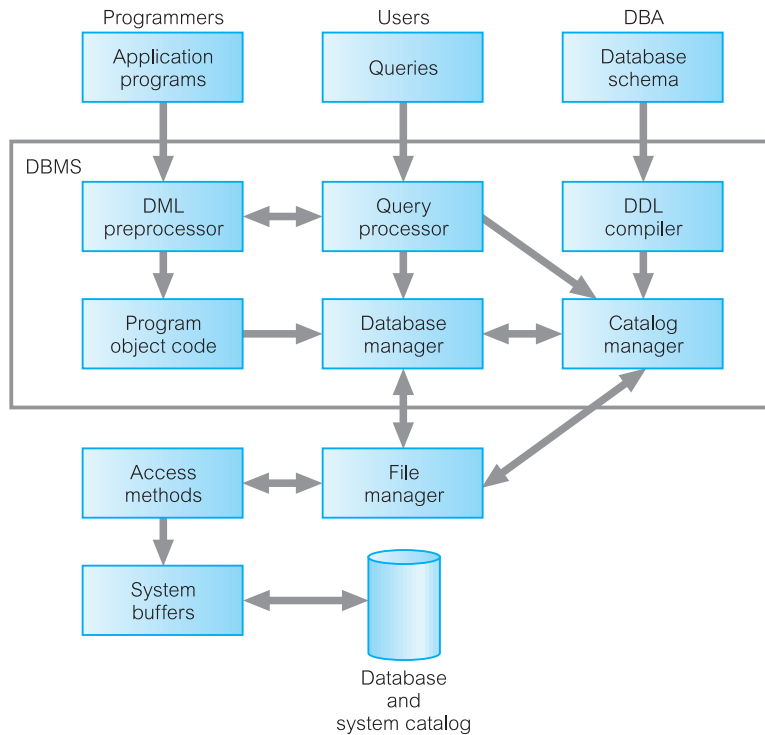


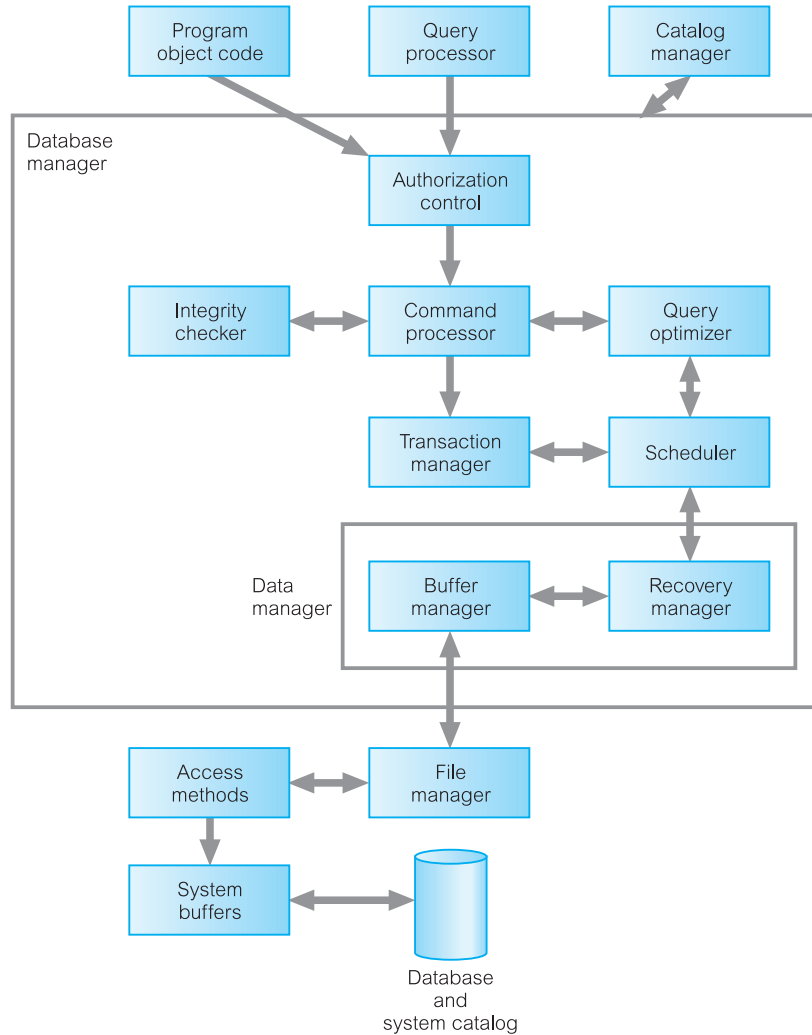
Figure 3.20
Major
components
of a DBMS.

Figure 3.20 shows the following components:

- *Query processor*. This is a major DBMS component that transforms queries into a series of low-level instructions directed to the database manager. We discuss query processing in Chapter 23.
- *Database manager (DM)*. The DM interfaces with user-submitted application programs and queries. The DM accepts queries and examines the external and conceptual schemas to determine what conceptual records are required to satisfy the request. The DM then places a call to the file manager to perform the request. The components of the DM are shown in Figure 3.21.
- *File manager*. The file manager manipulates the underlying storage files and manages the allocation of storage space on disk. It establishes and maintains the list of structures and indexes defined in the internal schema. If hashed files are used, it calls on the hashing functions to generate record addresses. However, the file manager does not directly manage the physical input and output of data. Rather, it passes the requests on to the appropriate access methods, which either read data from or write data into the system buffer (or *cache*).
- *DML preprocessor*. This module converts DML statements embedded in an application program into standard function calls in the host language. The DML preprocessor must interact with the query processor to generate the appropriate code.
- *DDL compiler*. The DDL compiler converts DDL statements into a set of tables containing metadata. These tables are then stored in the system catalog while control information is stored in data file headers.

Figure 3.21

Components of a database manager.



- *Catalog manager.* The catalog manager manages access to and maintains the system catalog. The system catalog is accessed by most DBMS components.

The major software components for the *database manager* are as follows:

- *Authorization control.* This module confirms whether the user has the necessary authorization to carry out the required operation.
- *Command processor.* Once the system has confirmed that the user has authority to carry out the operation, control is passed to the command processor.
- *Integrity checker.* For an operation that changes the database, the integrity checker checks whether the requested operation satisfies all necessary integrity constraints (such as key constraints).
- *Query optimizer.* This module determines an optimal strategy for the query execution. We discuss query optimization in Chapter 23.

- *Transaction manager*. This module performs the required processing of operations that it receives from transactions.
- *Scheduler*. This module is responsible for ensuring that concurrent operations on the database proceed without conflicting with one another. It controls the relative order in which transaction operations are executed.
- *Recovery manager*. This module ensures that the database remains in a consistent state in the presence of failures. It is responsible for transaction commit and abort.
- *Buffer manager*. This module is responsible for the transfer of data between main memory and secondary storage, such as disk and tape. The recovery manager and the buffer manager are sometimes referred to collectively as the *data manager*. The buffer manager is sometimes known as the *cache manager*.

We discuss the last four modules in Chapter 22. In addition to the previously mentioned modules, several other data structures are required as part of the physical-level implementation. These structures include data and index files and the system catalog. An attempt has been made to standardize DBMSs, and a reference model was proposed by the Database Architecture Framework Task Group (DAFTG, 1986). The purpose of this reference model was to define a conceptual framework aiming to divide standardization attempts into manageable pieces and to show at a very broad level how these pieces could be interrelated.

3.7 Oracle Architecture

Oracle is based on the client–server architecture examined in Section 3.1.3. The Oracle server consists of the *database* (the raw data, including log and control files) and the *instance* (the processes and system memory on the server that provide access to the database). An instance can connect to only one database. The database consists of a *logical structure*, such as the database schema, and a *physical structure*, containing the files that make up an Oracle database. We now discuss the logical and physical structure of the database and the system processes in more detail.

3.7.1 Oracle’s Logical Database Structure

At the logical level, Oracle maintains *tablespaces*, *schemas*, and *data blocks* and *extents/segments*.

Tablespaces

An Oracle database is divided into logical storage units called **tablespaces**. A tablespace is used to group related logical structures together. For example, tablespaces commonly group all the application’s objects to simplify some administrative operations.

Every Oracle database contains a tablespace named SYSTEM, which is created automatically when the database is created. The SYSTEM tablespace always contains the system catalog tables (called the *data dictionary* in Oracle) for the entire database. A small database might need only the SYSTEM tablespace; however, it is recommended that at least one additional tablespace is created to store user data separate from the data dictionary, thereby reducing contention among dictionary objects and schema objects for the same datafiles (see Figure 19.11). Figure 3.22

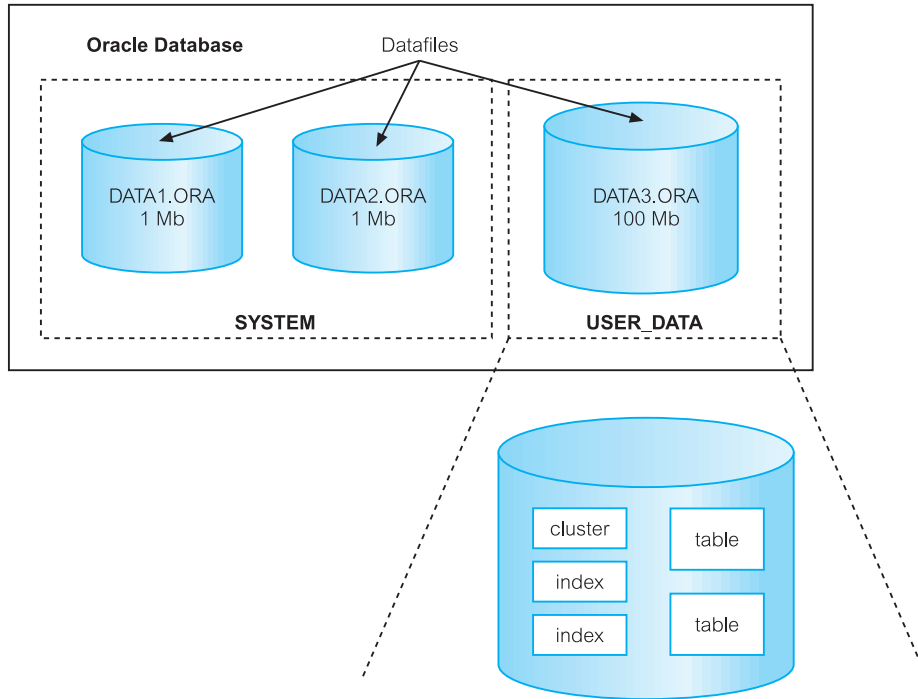


Figure 3.22 Relationship between an Oracle database, tablespaces, and datafiles.

illustrates an Oracle database consisting of the SYSTEM tablespace and a USER_DATA tablespace.

A new tablespace can be created using the CREATE TABLESPACE command, for example:

```
CREATE TABLESPACE user_data
DATAFILE 'DATA3.ORA' SIZE 100K
EXTENT MANAGEMENT LOCAL
SEGMENT SPACE MANAGEMENT AUTO;
```

A table can then be associated with a specific tablespace using the CREATE TABLE or ALTER TABLE statement, for example:

```
CREATE TABLE PropertyForRent (propertyNo VARCHAR2(5) NOT NULL, . . . )
TABLESPACE user_data;
```

If no tablespace is specified when creating a new table, the default tablespace associated with the user when the user account was set up is used.

Users, schemas, and schema objects

A **user** (sometimes called a **username**) is a name defined in the database that can connect to, and access, objects. A **schema** is a named collection of schema objects, such as tables, views, indexes, clusters, and procedures, associated with a particular user. Schemas and users help DBAs manage database security.

To access a database, a user must run a database application (such as Oracle Forms or SQL*Plus) and connect using a username defined in the database. When a database user is created, a corresponding schema of the same name is created for the user. By default, once a user connects to a database, the user has access to all objects contained in the corresponding schema. As a user is associated only with the schema of the same name; the terms “user” and “schema” are often used interchangeably. (Note that there is no relationship between a tablespace and a schema: Objects in the same schema can be in different tablespaces, and a tablespace can hold objects from different schemas.)

Data blocks, extents, and segments

The **data block** is the smallest unit of storage that Oracle can use or allocate. One data block corresponds to a specific number of bytes of physical disk space. The data block size can be set for each Oracle database when it is created. This data block size should be a multiple of the operating system’s block size (within the system’s maximum operating limit) to avoid unnecessary I/O. A data block has the following structure:

- *Header*. contains general information such as block address and type of segment.
- *Table directory*. contains information about the tables that have data in the data block.
- *Row directory*. contains information about the rows in the data block.
- *Row data*. contains the actual rows of table data. A row can span blocks.
- *Free space*. allocated for the insertion of new rows and updates to rows that require additional space. As of Oracle8i, Oracle can manage free space automatically, although there is an option to manage it manually.

We show how to estimate the size of an Oracle table using these components in Appendix J on the Web site for this book. The next level of logical database space is called an **extent**. An extent is a specific number of contiguous data blocks allocated for storing a specific type of information. The level above an extent is called a **segment**. A segment is a set of extents allocated for a certain logical structure. For example, each table’s data is stored in its own data segment, and each index’s data is stored in its own index segment. Figure 3.23 shows the relationship between data

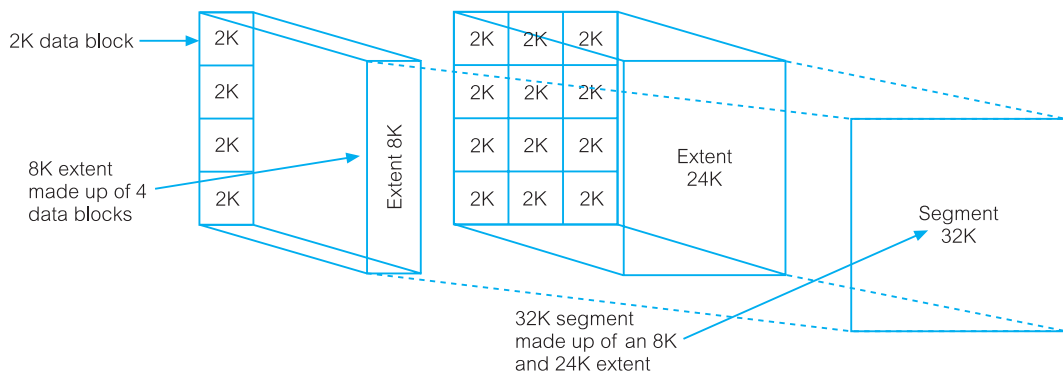


Figure 3.23 Relationship between Oracle data blocks, extents, and segments.

blocks, extents, and segments. Oracle dynamically allocates space when the existing extents of a segment become full. Because extents are allocated as needed, the extents of a segment may or may not be contiguous on disk.

3.7.2 Oracle's Physical Database Structure

The main physical database structures in Oracle are datafiles, redo log files, and control files.

Datafiles

Every Oracle database has one or more physical datafiles. The data of logical database structures (such as tables and indexes) is physically stored in these datafiles. As shown in Figure 3.16, one or more datafiles form a tablespace. The simplest Oracle database would have one tablespace and one datafile. A more complex database might have four tablespaces, each consisting of two datafiles, giving a total of eight datafiles.

Redo log files

Every Oracle database has a set of two or more redo log files that record all changes made to data for recovery purposes. Should a failure prevent modified data from being permanently written to the datafiles, the changes can be obtained from the redo log, thus preventing work from being lost. We discuss recovery in detail in Section 22.3.

Control files

Every Oracle database has a control file that contains a list of all the other files that make up the database, such as the datafiles and redo log files. For added protection, it is recommended that the control file should be multiplexed (multiple copies may be written to multiple devices). Similarly, it may be advisable to multiplex the redo log files as well.

The Oracle instance

The Oracle instance consists of the Oracle processes and shared memory required to access information in the database. The instance is made up of the Oracle background processes, the user processes, and the shared memory used by these processes, as illustrated in Figure 3.18. Among other things, Oracle uses shared memory for caching data and indexes as well as storing shared program code. Shared memory is broken into various *memory structures*, of which the basic ones are the System Global Area (SGA) and the Program Global Area (PGA).

- *System global area.* The SGA is an area of shared memory that is used to store data and control information for one Oracle instance. The SGA is allocated when the Oracle instance starts and deallocated when the Oracle instance shuts down. The information in the SGA consists of the following memory structures, each of which has a fixed size and is created at instance startup:

- *Database buffer cache.* This contains the most recently used data blocks from the database. These blocks can contain modified data that has not yet been written to disk (*dirty blocks*), blocks that have not been modified, or blocks that have been written to disk since modification (*clean blocks*). By storing the most recently used blocks, the most active buffers stay in memory to reduce I/O and improve performance. We discuss buffer management policies in Section 22.3.2.
- *Redo log buffer.* This contains the redo log file entries, which are used for recovery purposes (see Section 22.3). The background process LGWR (explained shortly) writes the redo log buffer to the active online redo log file on disk.
- *Shared pool.* This contains the shared memory structures, such as shared SQL areas in the library cache and internal information in the data dictionary. The shared SQL areas contain parse trees and execution plans for the SQL queries. If multiple applications issue the same SQL statement, each can access the shared SQL area to reduce the amount of memory needed and to reduce the processing time used for parsing and execution. We discuss query processing in Chapter 23.
- *Large pool.* This is an optional memory area intended for large memory allocations (eg. for buffers for Recovery Manager (RMAN) I/O slaves).
- *Java pool.* This area stores all session-specific Java code and data within the Java Virtual Machine (JVM).
- *Streams pool.* This area stores buffered queue messages and provides memory for Oracle Streams processes. Oracle Streams allows information flow (such as database events and database changes) to be managed and potentially propagated to other databases.
- *Fixed SGA.* This is an internal housekeeping area that contains various data such as general information about the state of the database and the Oracle instance and information communicated between Oracle processes, such as information about locks.
- *Program global area.* The PGA is an area of shared memory that is used to store data and control information for an Oracle process. The PGA is created by Oracle Database when an Oracle process is started. One PGA exists for each server process and background process. The size and content of the PGA depends on the Oracle server options installed.
- *Client processes.* Each client process represents a user's connection to the Oracle server (for example, through SQL*Plus or an Oracle Forms application). The user process manipulates the user's input, communicates with the Oracle server process, displays the information requested by the user and, if required, processes this information into a more useful form.
- *Oracle processes.* Oracle (server) processes perform functions for users. Oracle processes can be split into two groups: *server processes* (which handle requests from connected user processes) and *background processes* (which perform asynchronous I/O and provide increased parallelism for improved performance and reliability). From Figure 3.24, we have the following background processes:
 - *Database Writer (DBWR).* The DBWR process is responsible for writing the modified (*dirty*) blocks from the buffer cache in the SGA to datafiles on disk. An Oracle instance can have up to ten DBWR processes, named DBW0 to DBW9, to handle I/O to multiple datafiles. Oracle employs a technique known as write-ahead logging (see Section 22.3.4), which means that the DBWR process performs *batched writes* whenever the buffers need to be freed, not necessarily at the point the transaction commits.

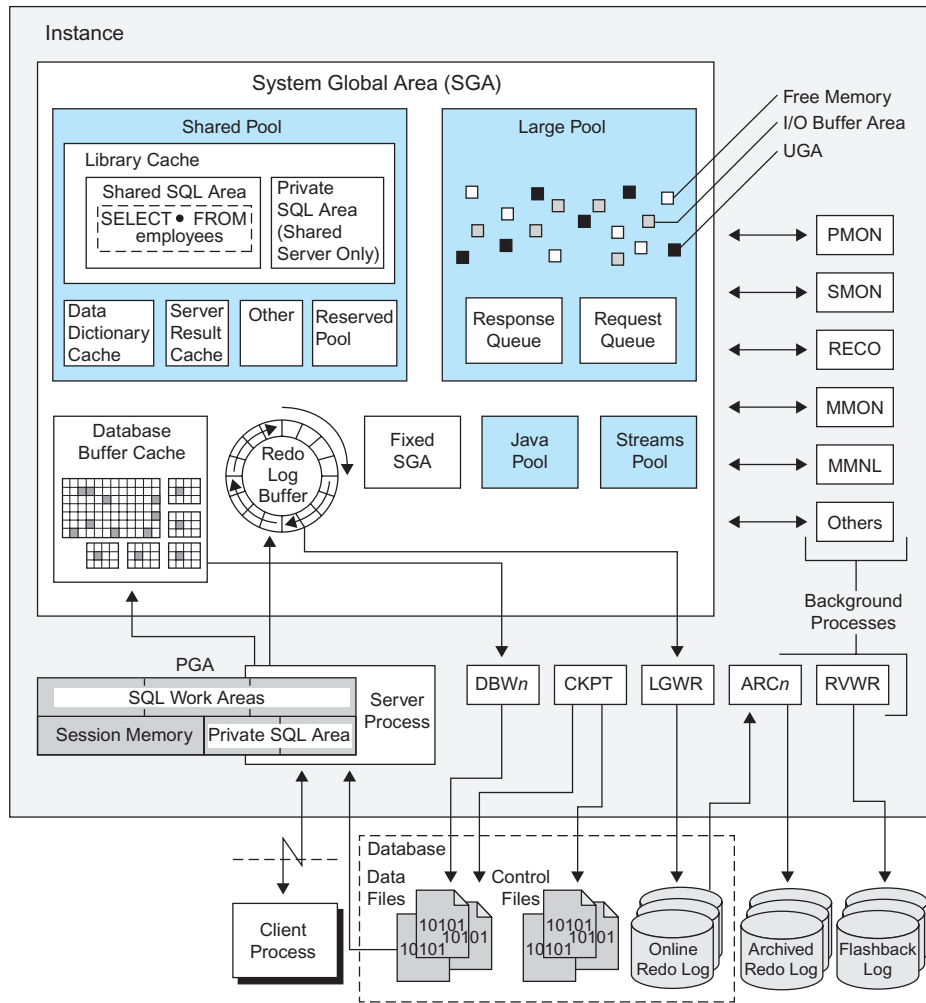


Figure 3.24 The Oracle architecture (from the Oracle documentation set).

- *Log Writer (LGWR)*. The LGWR process is responsible for writing data from the log buffer to the redo log.
- *Checkpoint (CKPT)*. A checkpoint is an event in which all modified database buffers are written to the datafiles by the DBWR (see Section 22.3.2). The CKPT process is responsible for telling the DBWR process to perform a checkpoint and to update all the datafiles and control files for the database to indicate the most recent checkpoint. The CKPT process is optional and, if omitted, these responsibilities are assumed by the LGWR process.
- *System Monitor (SMON)*. The SMON process is responsible for crash recovery when the instance is started following a failure. This includes recovering transactions that have died because of a system crash. SMON also defragments the database by merging free extents within the datafiles.

- *Process Monitor (PMON)*. The PMON process is responsible for tracking user processes that access the database and recovering them following a crash. This includes cleaning up any resources left behind (such as memory) and releasing any locks held by the failed process.
- *Archiver (ARCH)*. The ARCH process is responsible for copying the online redo log files to archival storage when they become full. The system can be configured to run up to 10 ARCH processes, named ARC0 to ARC9. The additional archive processes are started by the LWGR when the load dictates.
- *Recoverer (RECO)*. The RECO process is responsible for cleaning up failed or suspended distributed transactions (see Section 25.4).
- *Flashback Writer or Recovery Writer (RVWR)*. When flashback is enabled or when there are guaranteed restore points, the RVWR process writes flashback data to flashback database logs in the flash recovery area. Flashback tools allow administrators and users to view and manipulate past states of an Oracle instance's data without recovering the database to a fixed point in time.
- *Manageability Monitor (MMON)*. This process performs many tasks related to the *Automatic Workload Repository (AWR)*. AWR is a repository of historical performance data that includes cumulative statistics for the system, sessions, individual SQL statements, segments, and services. Among other things, by default, the MMON process gathers statistics every hour and creates an AWR snapshot.
- *Manageability Monitor Lite (MMNL)*. This process writes statistics from the Active Session History (ASH) buffer in the SGA to disk. MMNL writes to disk when the ASH buffer is full.

In the previous descriptions we have used the term “process” generically. Nowadays, some systems implement processes as *threads*.

Example of how these processes interact

The following example illustrates an Oracle configuration with the server process running on one machine and a user process connecting to the server from a separate machine. Oracle uses a communication mechanism called Oracle Net Services to allow processes on different physical machines to communicate with each other. Oracle Net Services supports a variety of network protocols, such as TCP/IP. The services can also perform network protocol interchanges, allowing clients that use one protocol to interact with a database server using another protocol.

- (1) The client workstation runs an application in a user process. The client application attempts to establish a connection to the server using the Oracle Net Services driver.
- (2) The server detects the connection request from the application and creates a (dedicated) server process on behalf of the user process.
- (3) The user executes an SQL statement to change a row of a table and commits the transaction.
- (4) The server process receives the statement and checks the shared pool for any shared SQL area that contains an identical SQL statement. If a shared SQL area is found, the server process checks the user's access privileges to the requested data and the previously existing shared SQL area is used to process

the statement; if not, a new shared SQL area is allocated for the statement so that it can be parsed and processed.

- (5) The server process retrieves any necessary data values from the actual datafile (table) or those stored in the SGA.
- (6) The server process modifies data in the SGA. The DBWR process writes modified blocks permanently to disk when doing so is efficient. Because the transaction committed, the LGWR process immediately records the transaction in the online redo log file.
- (7) The server process sends a success/failure message across the network to the application.
- (8) During this time, the other background processes run, watching for conditions that require intervention. In addition, the Oracle server manages other users' transactions and prevents contention between transactions that request the same data.

Chapter Summary

- **Client-server** architecture refers to the way in which software components interact. There is a **client** process that requires some resource, and a **server** that provides the resource. In the two-tier model, the client handles the user interface and business processing logic and the server handles the database functionality. In the Web environment, the traditional two-tier model has been replaced by a three-tier model, consisting of a user interface layer (the **client**), a business logic and data processing layer (the **application server**), and a DBMS (the **database server**), distributed over different machines.
- The three-tier architecture can be extended to n tiers, will additional tiers added to provide more flexibility and scalability.
- **Middleware** is computer software that connects software components or applications. Middleware types include RPC (synchronous and asynchronous), publish/subscribe, message-oriented middleware (MOM), object-request brokers (ORB), and database middleware.
- A **Web service** is a software system designed to support interoperable machine-to-machine interaction over a network. They are based on standards such as XML, SOAP, WSDL, and UDDI.
- **Service-Oriented Architecture(SOA)** is a business-centric software architecture for building applications that implement business processes as sets of services published at a granularity relevant to the service consumer.
- **Cloud computing** is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. The three main service models are: Software as a Service (SaaS), Platform as a Service (PaaS), and Infrastructure as a Service (IaaS). Cloud-based database solutions fall into two basic categories: Data as a Service (DaaS) and Database as a Service (DBaaS).
- A **Transaction Processing (TP) Monitor** is a program that controls data transfer between clients and servers in order to provide a consistent environment, particularly for online transaction processing (OLTP). The advantages include transaction routing, distributed transactions, load balancing, funneling, and increased reliability.



Review Questions

- 3.1 What is meant by the term 'client-server architecture' and what are the advantages of this approach? Compare the client-server architecture with two other architectures.
- 3.2 Compare and contrast the two-tier client-server architecture for traditional DBMSs with the three-tier client-server architecture. Why is the latter architecture more appropriate for the Web?
- 3.3 How is an application server different from a file server?
- 3.4 What is a data warehouse? How is it different from an OLTP system?
- 3.5 What is a TP Monitor? What advantages does a TP Monitor bring to an OLTP environment?
- 3.6 Describe the features of a distributed database management system (DDBMS).
- 3.7 What technologies and standards are used to develop Web services and how do they relate to each other?
- 3.8 What is a service-oriented architecture?
- 3.9 Describe the functions of a database manager?
- 3.10 What is Cloud computing?
- 3.11 Discuss the five essential characteristics of cloud computing.
- 3.12 Discuss the three main service models of cloud computing.
- 3.13 Compare and contrast the four main deployment models for the cloud.
- 3.14 What is the difference between Data as a service (DaaS) and Database as a service (DBaaS)?
- 3.15 Discuss the different architectural models for Database as a service.
- 3.16 Describe the main components in a DBMS.
- 3.17 Describe the internal architecture of Oracle.



Exercises

- 3.18 Examine the documentation sets of Microsoft SQL Server, Oracle, and IBM's DB2 system to identify their support for the following:
 - (a) client-server architecture
 - (b) Web services
 - (c) service-oriented architecture
- 3.19 Search the Web for a number of Web services other than the ones discussed in Section 3.2. What do these services have in common? Identify whether the services access a database.
- 3.20 Based on the Oracle architecture described in section 3.7, examine the structure of two other DBMSs of your choice. Describe features common to all three DBMSs.

The Relational Model and Languages

Chapter	4	The Relational Model	149
Chapter	5	Relational Algebra and Relational Calculus	167
Chapter	6	SQL: Data Manipulation	191
Chapter	7	SQL: Data Definition	233
Chapter	8	Advanced SQL	271
Chapter	9	Object-Relational DBMSs	291

Chapter Objectives

In this chapter you will learn:

- The origins of the relational model.
- The terminology of the relational model.
- How tables are used to represent data.
- The connection between mathematical relations and relations in the relational model.
- Properties of database relations.
- How to identify candidate, primary, alternate, and foreign keys.
- The meaning of entity integrity and referential integrity.
- The purpose and advantages of views in relational systems.

The Relational Database Management System (RDBMS) has become the dominant data-processing software in use today, with an estimated total software revenue worldwide of US\$24 billion in 2011 and estimated to grow to about US\$37 billion by 2016. This software represents the second generation of DBMSs and is based on the relational data model proposed by E. F. Codd (1970). In the relational model, all data is logically structured within relations (tables). Each **relation** has a name and is made up of named **attributes** (columns) of data. Each **tuple** (row) contains one value per attribute. A great strength of the relational model is this simple logical structure. Yet behind this simple structure is a sound theoretical foundation that is lacking in the first generation of DBMSs (the network and hierarchical DBMSs).

We devote a significant amount of this book to the RDBMS, in recognition of the importance of these systems. In this chapter, we discuss the terminology and basic structural concepts of the relational data model. In the next chapter, we examine the relational languages that can be used for update and data retrieval.

Structure of this Chapter To put our treatment of the RDBMS into perspective, in Section 4.1 we provide a brief history of the relational model. In Section 4.2 we discuss the underlying concepts and terminology of the relational model. In Section 4.3 we discuss the relational integrity rules, including entity integrity and referential integrity. In Section 4.4 we introduce the concept of views, which are important features of relational DBMSs, although not a concept of the relational model *per se*.

Looking ahead, in Chapters 5–9 we examine SQL (Structured Query Language), the formal and *de facto* standard language for RDBMSs, and in Appendix M we examine QBE (Query-By-Example), another highly popular visual query language for RDBMSs. In Chapters 16–19 we present a complete methodology for relational database design. In Chapter 9, we discuss an extension to relational DBMSs to incorporate object-oriented features, and in particular examine the object-oriented features of SQL. In Appendix G, we examine Codd’s twelve rules, which form a yardstick against which RDBMS products can be identified. The examples in this chapter are drawn from the *DreamHome* case study, which is described in detail in Section 11.4 and Appendix A.



4.1 Brief History of the Relational Model

The relational model was first proposed by E. F. Codd in his seminal paper “A relational model of data for large shared data banks” (Codd, 1970). This paper is now generally accepted as a landmark in database systems, although a set-oriented model had been proposed previously (Childs, 1968). The relational model’s objectives were specified as follows:

- To allow a high degree of data independence. Application programs must not be affected by modifications to the internal data representation, particularly by changes to file organizations, record orderings, or access paths.
- To provide substantial grounds for dealing with data semantics, consistency, and redundancy problems. In particular, Codd’s paper introduced the concept of **normalized** relations, that is, relations that have no repeating groups. (The process of normalization is discussed in 14 and 15.)
- To enable the expansion of set-oriented data manipulation languages.

Although interest in the relational model came from several directions, the most significant research may be attributed to three projects with rather different perspectives. The first of these, at IBM’s San José Research Laboratory in California, was the prototype relational DBMS System R, which was developed during the late 1970s (Astrahan *et al.*, 1976). This project was designed to prove the practicality of the relational model by providing an implementation of its data structures

and operations. It also proved to be an excellent source of information about implementation concerns such as transaction management, concurrency control, recovery techniques, query optimization, data security and integrity, human factors, and user interfaces, and led to the publication of many research papers and to the development of other prototypes. In particular, the System R project led to two major developments:

- the development of a structured query language called SQL, which has since become the formal International Organization for Standardization (ISO) and de facto standard language for relational DBMSs;
- the production of various commercial relational DBMS products during the late 1970s and the 1980s: for example, DB2 and SQL/DS from IBM and Oracle from Oracle Corporation.

The second project to have been significant in the development of the relational model was the INGRES (Interactive Graphics Retrieval System) project at the University of California at Berkeley, which was active at about the same time as the System R project. The INGRES project involved the development of a prototype RDBMS, with the research concentrating on the same overall objectives as the System R project. This research led to an academic version of INGRES, which contributed to the general appreciation of relational concepts, and spawned the commercial products INGRES from Relational Technology Inc. (now Actian Corporation) and the Intelligent Database Machine from Britton Lee Inc.

The third project was the Peterlee Relational Test Vehicle at the IBM UK Scientific Centre in Peterlee (Todd, 1976). This project had a more theoretical orientation than the System R and INGRES projects and was significant, principally for research into such issues as query processing and optimization as well as functional extension.

Commercial systems based on the relational model started to appear in the late 1970s and early 1980s. Now there are several hundred RDBMSs for both mainframe and PC environments, even though many do not strictly adhere to the definition of the relational model. Examples of PC-based RDBMSs are Office Access and Visual FoxPro from Microsoft, InterBase from Embarcadero Technologies, and R:Base from R:Base Technologies.

Owing to the popularity of the relational model, many nonrelational systems now provide a relational user interface, irrespective of the underlying model. IDMS, the principal network DBMS, has become CA-IDMS from CA Inc. (formerly Computer Associates), supporting a relational view of data. Other mainframe DBMSs that support some relational features are Model 204 from Computer Corporation of America (a subsidiary of Rocket Software Inc) and Software AG's ADABAS.

Some extensions to the relational model have also been proposed; for example, extensions to:

- capture more closely the meaning of data (for example, Codd, 1979);
- support object-oriented concepts (for example, Stonebraker and Rowe, 1986);
- support deductive capabilities (for example, Gardarin and Valduriez, 1989).

We discuss some of these extensions in Chapters 27–28, as we discuss Object DBMSs.

4.2 Terminology

The relational model is based on the mathematical concept of a **relation**, which is physically represented as a **table**. Codd, a trained mathematician, used terminology taken from mathematics, principally set theory and predicate logic. In this section we explain the terminology and structural concepts of the relational model.

4.2.1 Relational Data Structure

Relation

A relation is a table with columns and rows.

An RDBMS requires only that the database be perceived by the user as tables. Note, however, that this perception applies only to the logical structure of the database: that is, the external and conceptual levels of the ANSI-SPARC architecture discussed in Section 2.1. It does not apply to the physical structure of the database, which can be implemented using a variety of storage structures (see Appendix F).

Attribute

An attribute is a named column of a relation.

In the relational model, **relations** are used to hold information about the objects to be represented in the database. A relation is represented as a two-dimensional table in which the rows of the table correspond to individual records and the table columns correspond to **attributes**. Attributes can appear in any order and the relation will still be the same relation, and therefore will convey the same meaning.

For example, the information on branch offices is represented by the **Branch** relation, with columns for attributes **branchNo** (the branch number), **street**, **city**, and **postcode**. Similarly, the information on staff is represented by the **Staff** relation, with columns for attributes **staffNo** (the staff number), **fName**, **lName**, **position**, **sex**, **DOB** (date of birth), **salary**, and **branchNo** (the number of the branch the staff member works at). Figure 4.1 shows instances of the **Branch** and **Staff** relations. As you can see from this example, a column contains values of a single attribute; for example, the **branchNo** columns contain only numbers of existing branch offices.

Domain

A domain is the set of allowable values for one or more attributes.

Domains are an extremely powerful feature of the relational model. Every attribute in a relation is defined on a **domain**. Domains may be distinct for each attribute, or two or more attributes may be defined on the same domain. Figure 4.2 shows the domains for some of the attributes of the **Branch** and **Staff** relations. Note that at any given time, typically there will be values in a domain that do not currently appear as values in the corresponding attribute.

The domain concept is important, because it allows the user to define in a central place the meaning and source of values that attributes can hold. As a result, more

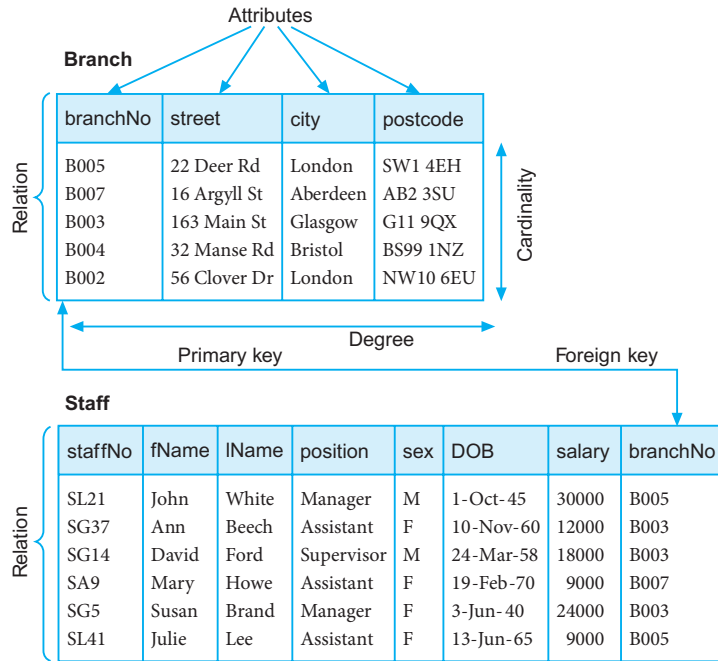


Figure 4.1
Instances of the
Branch and Staff
relations.

Attribute	Domain Name	Meaning	Domain Definition
branchNo	BranchNumbers	The set of all possible branch numbers	character: size 4, range B001–B999
street	StreetNames	The set of all street names in Britain	character: size 25
city	CityNames	The set of all city names in Britain	character: size 15
postcode	Postcodes	The set of all postcodes in Britain	character: size 8
sex	Sex	The sex of a person	character: size 1, value M or F
DOB	DatesOfBirth	Possible values of staff birth dates	date, range from 1-Jan-20, format dd-mmm-yy
salary	Salaries	Possible values of staff salaries	monetary: 7 digits, range 6000.00–40000.00

Figure 4.2
Domains for
some attributes
of the Branch and
Staff relations.

information is available to the system when it undertakes the execution of a relational operation, and operations that are semantically incorrect can be avoided. For example, it is not sensible to compare a street name with a telephone number, even though the domain definitions for both these attributes are character strings. On the other hand, the monthly rental on a property and the number of months a property has been leased have different domains (the first a monetary value, the second an integer value), but it is still a legal operation to multiply two values from these domains. As these two examples illustrate, a complete implementation of domains is not straightforward, and as a result, many RDBMSs do not support them fully.

Tuple A tuple is a row of a relation.

The elements of a relation are the rows or **tuples** in the table. In the Branch relation, each row contains four values, one for each attribute. Tuples can appear in

any order and the relation will still be the same relation, and therefore convey the same meaning.

The structure of a relation, together with a specification of the domains and any other restrictions on possible values, is sometimes called its **intension**, which is usually fixed, unless the meaning of a relation is changed to include additional attributes. The tuples are called the **extension** (or **state**) of a relation, which changes over time.

Degree

The degree of a relation is the number of attributes it contains.

The Branch relation in Figure 4.1 has four attributes or degree four. This means that each row of the table is a four-tuple, containing four values. A relation with only one attribute would have degree one and be called a **unary** relation or one-tuple. A relation with two attributes is called **binary**, one with three attributes is called **ternary**, and after that the term **n-ary** is usually used. The degree of a relation is a property of the *intension* of the relation.

Cardinality

The cardinality of a relation is the number of tuples it contains.

By contrast, the number of tuples is called the **cardinality** of the relation and this changes as tuples are added or deleted. The cardinality is a property of the *extension* of the relation and is determined from the particular instance of the relation at any given moment. Finally, we define a relational database.

Relational database

A collection of normalized relations with distinct relation names.

A relational database consists of relations that are appropriately structured. We refer to this appropriateness as *normalization*. We defer the discussion of normalization until Chapters 14 and 15.

Alternative terminology

The terminology for the relational model can be quite confusing. We have introduced two sets of terms. In fact, a third set of terms is sometimes used: a relation may be referred to as a **file**, the tuples as **records**, and the attributes as **fields**. This terminology stems from the fact that, physically, the RDBMS may store each relation in a file. Table 4.1 summarizes the different terms for the relational model.

TABLE 4.1 Alternative terminology for relational model terms.

FORMAL TERMS	ALTERNATIVE 1	ALTERNATIVE 2
Relation	Table	File
Tuple	Row	Record
Attribute	Column	Field

4.2.2 Mathematical Relations

To understand the true meaning of the term *relation*, we have to review some concepts from mathematics. Suppose that we have two sets, D_1 and D_2 , where $D_1 = \{2, 4\}$ and $D_2 = \{1, 3, 5\}$. The **Cartesian product** of these two sets, written $D_1 \times D_2$, is the set of all ordered pairs such that the first element is a member of D_1 and the second element is a member of D_2 . An alternative way of expressing this is to find all combinations of elements with the first from D_1 and the second from D_2 . In our case, we have:

$$D_1 \times D_2 = \{(2, 1), (2, 3), (2, 5), (4, 1), (4, 3), (4, 5)\}$$

Any subset of this Cartesian product is a relation. For example, we could produce a relation R such that:

$$R = \{(2, 1), (4, 1)\}$$

We may specify which ordered pairs will be in the relation by giving some condition for their selection. For example, if we observe that R includes all those ordered pairs in which the second element is 1, then we could write R as:

$$R = \{(x, y) \mid x \in D_1, y \in D_2, \text{ and } y = 1\}$$

Using these same sets, we could form another relation S in which the first element is always twice the second. Thus, we could write S as:

$$S = \{(x, y) \mid x \in D_1, y \in D_2, \text{ and } x = 2y\}$$

or, in this instance,

$$S = \{(2, 1)\}$$

as there is only one ordered pair in the Cartesian product that satisfies this condition. We can easily extend the notion of a relation to three sets. Let D_1 , D_2 , and D_3 be three sets. The Cartesian product $D_1 \times D_2 \times D_3$ of these three sets is the set of all ordered triples such that the first element is from D_1 , the second element is from D_2 , and the third element is from D_3 . Any subset of this Cartesian product is a relation. For example, suppose we have:

$$D_1 = \{1, 3\} \quad D_2 = \{2, 4\} \quad D_3 = \{5, 6\}$$

$$D_1 \times D_2 \times D_3 = \{(1, 2, 5), (1, 2, 6), (1, 4, 5), (1, 4, 6), (3, 2, 5), (3, 2, 6), (3, 4, 5), (3, 4, 6)\}$$

Any subset of these ordered triples is a relation. We can extend the three sets and define a general relation on n domains. Let $D_1, D_2, \dots, D_n$ be n sets. Their Cartesian product is defined as:

$$D_1 \times D_2 \times \dots \times D_n = \{(d_1, d_2, \dots, d_n) \mid d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n\}$$

and is usually written as:

$$\prod_{i=1}^n D_i$$

Any set of n -tuples from this Cartesian product is a relation on the n sets. Note that in defining these relations we have to specify the sets, or **domains**, from which we choose values.

4.2.3 Database Relations

Applying the previously discussed concepts to databases, we can define a relation schema.

Relation schema

A named relation defined by a set of attribute and domain name pairs.

Let $A_1, A_2, \dots, A_n$ be attributes with domains $D_1, D_2, \dots, D_n$. Then the set $\{A_1:D_1, A_2:D_2, \dots, A_n:D_n\}$ is a relation schema. A relation R defined by a relation schema S is a set of mappings from the attribute names to their corresponding domains. Thus, relation R is a set of n -tuples:

$$(A_1:d_1, A_2:d_2, \dots, A_n:d_n) \text{ such that } d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$$

Each element in the n -tuple consists of an attribute and a value for that attribute. Normally, when we write out a relation as a table, we list the attribute names as column headings and write out the tuples as rows having the form $(d_1, d_2, \dots, d_n)$, where each value is taken from the appropriate domain. In this way, we can think of a relation in the relational model as any subset of the Cartesian product of the domains of the attributes. A table is simply a physical representation of such a relation.

In our example, the Branch relation shown in Figure 4.1 has attributes branchNo, street, city, and postcode, each with its corresponding domain. The Branch relation is any subset of the Cartesian product of the domains, or any set of four-tuples in which the first element is from the domain BranchNumbers, the second is from the domain StreetNames, and so on. One of the four-tuples is:

$$\{(B005, 22 \text{ Deer Rd, London, SW1 4EH})\}$$

or more correctly:

$$\{(\text{branchNo: B005, street: 22 Deer Rd, city: London, postcode: SW1 4EH})\}$$

We refer to this as a **relation instance**. The Branch table is a convenient way of writing out all the four-tuples that form the relation at a specific moment in time, which explains why table rows in the relational model are called “tuples”. In the same way that a relation has a schema, so too does the relational database.

Relational database schema

A set of relation schemas, each with a distinct name.

If $R_1, R_2, \dots, R_n$ are a set of relation schemas, then we can write the *relational database schema*, or simply *relational schema*, R , as:

$$R = \{R_1, R_2, \dots, R_n\}$$

4.2.4 Properties of Relations

A relation has the following properties:

- the relation has a name that is distinct from all other relation names in the relational schema;
- each cell of the relation contains exactly one atomic (single) value;

- each attribute has a distinct name;
- the values of an attribute are all from the same domain;
- each tuple is distinct; there are no duplicate tuples;
- the order of attributes has no significance;
- the order of tuples has no significance, theoretically. (However, in practice, the order may affect the efficiency of accessing tuples.)

To illustrate what these restrictions mean, consider again the Branch relation shown in Figure 4.1. Because each cell should contain only one value, it is illegal to store two postcodes for a single branch office in a single cell. In other words, relations do not contain repeating groups. A relation that satisfies this property is said to be **normalized** or in **first normal form**. (Normal forms are discussed in chapters 14 and 15.)

The column names listed at the tops of columns correspond to the attributes of the relation. The values in the branchNo attribute are all from the BranchNumbers domain; we should not allow a postcode value to appear in this column. There can be no duplicate tuples in a relation. For example, the row (B005, 22 Deer Rd, London, SW1 4EH) appears only once.

Provided that an attribute name is moved along with the attribute values, we can interchange columns. The table would represent the same relation if we were to put the city attribute before the postcode attribute, although for readability it makes more sense to keep the address elements in the normal order. Similarly, tuples can be interchanged, so the records of branches B005 and B004 can be switched and the relation will still be the same.

Most of the properties specified for relations result from the properties of mathematical relations:

- When we derived the Cartesian product of sets with simple, single-valued elements such as integers, each element in each tuple was single-valued. Similarly, each cell of a relation contains exactly one value. However, a mathematical relation need not be normalized. Codd chose to disallow repeating groups to simplify the relational data model.
- In a relation, the possible values for a given position are determined by the set, or domain, on which the position is defined. In a table, the values in each column must come from the same attribute domain.
- In a set, no elements are repeated. Similarly, in a relation, there are no duplicate tuples.
- Because a relation is a set, the order of elements has no significance. Therefore, in a relation, the order of tuples is immaterial.

However, in a mathematical relation, the order of elements in a tuple is important. For example, the ordered pair (1, 2) is quite different from the ordered pair (2, 1). This is not the case for relations in the relational model, which specifically requires that the order of attributes be immaterial. The reason is that the column headings define which attribute the value belongs to. This means that the order of column headings in the intension is immaterial, but once the structure of the relation is chosen, the order of elements within the tuples of the extension must match the order of attribute names.

4.2.5 Relational Keys

As stated earlier, there are no duplicate tuples within a relation. Therefore, we need to be able to identify one or more attributes (called **relational keys**) that uniquely identifies each tuple in a relation. In this section, we explain the terminology used for relational keys.

Superkey

An attribute, or set of attributes, that uniquely identifies a tuple within a relation.

A superkey uniquely identifies each tuple within a relation. However, a superkey may contain additional attributes that are not necessary for unique identification, and we are interested in identifying superkeys that contain only the minimum number of attributes necessary for unique identification.

Candidate key

A superkey such that no proper subset is a superkey within the relation.

A candidate key K for a relation R has two properties:

- *Uniqueness.* In each tuple of R , the values of K uniquely identify that tuple.
- *Irreducibility.* No proper subset of K has the uniqueness property.

There may be several candidate keys for a relation. When a key consists of more than one attribute, we call it a **composite key**. Consider the **Branch** relation shown in Figure 4.1. Given a value of city, we can determine several branch offices (for example, London has two branch offices). This attribute cannot be a candidate key. On the other hand, because *DreamHome* allocates each branch office a unique branch number, given a branch number value, *branchNo*, we can determine at most one tuple, so that *branchNo* is a candidate key. Similarly, *postcode* is also a candidate key for this relation.



Now consider a relation **Viewing**, which contains information relating to properties viewed by clients. The relation comprises a client number (*clientNo*), a property number (*propertyNo*), a date of viewing (*viewDate*) and, optionally, a comment (*comment*). Given a client number, *clientNo*, there may be several corresponding viewings for different properties. Similarly, given a property number, *propertyNo*, there may be several clients who viewed this property. Therefore, *clientNo* by itself or *propertyNo* by itself cannot be selected as a candidate key. However, the combination of *clientNo* and *propertyNo* identifies at most one tuple, so for the **Viewing** relation, *clientNo* and *propertyNo* together form the (composite) candidate key. If we need to take into account the possibility that a client may view a property more than once, then we could add *viewDate* to the composite key. However, we assume that this is not necessary.

Note that an instance of a relation cannot be used to prove that an attribute or combination of attributes is a candidate key. The fact that there are no duplicates for the values that appear at a particular moment in time does not guarantee that duplicates are not possible. However, the presence of duplicates in an instance can be used to show that some attribute combination is not a candidate key. Identifying a candidate key requires that we know the “real-world” meaning of the attribute(s) involved so that we can decide whether duplicates are possible. Only by using this semantic information can we be certain that an attribute combination is a candidate key. For example, from the data presented in Figure 4.1, we may think that

a suitable candidate key for the *Staff* relation would be *IName*, the employee's surname. However, although there is only a single value of "White" in this instance of the *Staff* relation, a new member of staff with the surname "White" may join the company, invalidating the choice of *IName* as a candidate key.

Primary key

The candidate key that is selected to identify tuples uniquely within the relation.

Because a relation has no duplicate tuples, it is always possible to identify each row uniquely. This means that a relation always has a primary key. In the worst case, the entire set of attributes could serve as the primary key, but usually some smaller subset is sufficient to distinguish the tuples. The candidate keys that are not selected to be the primary key are called **alternate keys**. For the *Branch* relation, if we choose *branchNo* as the primary key, *postcode* would then be an alternate key. For the *Viewing* relation, there is only one candidate key, comprising *clientNo* and *propertyNo*, so these attributes would automatically form the primary key.

Foreign key

An attribute, or set of attributes, within one relation that matches the candidate key of some (possibly the same) relation.

When an attribute appears in more than one relation, its appearance usually represents a relationship between tuples of the two relations. For example, the inclusion of *branchNo* in both the *Branch* and *Staff* relations is quite deliberate and links each branch to the details of staff working at that branch. In the *Branch* relation, *branchNo* is the primary key. However, in the *Staff* relation, the *branchNo* attribute exists to match staff to the branch office they work in. In the *Staff* relation, *branchNo* is a foreign key. We say that the attribute *branchNo* in the *Staff* relation **targets** the primary key attribute *branchNo* in the **home relation**, *Branch*. These common attributes play an important role in performing data manipulation, as we see in the next chapter.

4.2.6 Representing Relational Database Schemas

A relational database consists of any number of normalized relations. The relational schema for part of the *DreamHome* case study is:



Branch	(<u>branchNo</u> , street, city, postcode)
Staff	(<u>staffNo</u> , fName, IName, position, sex, DOB, salary, branchNo)
PropertyForRent	(<u>propertyNo</u> , street, city, postcode, type, rooms, rent, ownerNo, staffNo, branchNo)
Client	(<u>clientNo</u> , fName, IName, telNo, prefType, maxRent, eMail)
PrivateOwner	(<u>ownerNo</u> , fName, IName, address, telNo, eMail, password)
Viewing	(<u>clientNo</u> , <u>propertyNo</u> , viewDate, comment)
Registration	(<u>clientNo</u> , <u>branchNo</u> , staffNo, dateJoined)

The common convention for representing a relation schema is to give the name of the relation followed by the attribute names in parentheses. Normally, the primary key is underlined.

The *conceptual model*, or *conceptual schema*, is the set of all such schemas for the database. Figure 4.3 shows an instance of this relational schema.

Figure 4-3
Instance of the
DreamHome
rental database.

Branch										
branchNo	street	city	postcode							
B005	22 Deer Rd	London	SW1 4EH							
B007	16 Argyll St	Aberdeen	AB2 3SU							
B003	163 Main St	Glasgow	G11 9QX							
B004	32 Manse Rd	Bristol	BS99 1NZ							
B002	56 Clover Dr	London	NW10 6EU							

Staff										
staffNo	fName	lName	position	sex	DOB	salary	branchNo			
SL21	John	White	Manager	M	1-Oct-45	30000	B005			
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003			
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003			
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007			
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003			
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005			

PropertyForRent										
propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo	
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46	SA9	B007	
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87	SL41	B005	
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40		B003	
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93	SG37	B003	
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87	SG37	B003	
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93	SG14	B003	

Client						
clientNo	fName	lName	telNo	prefType	maxRent	eMail
CR76	John	Kay	0207-774-5632	Flat	425	john.kay@gmail.com
CR56	Aline	Stewart	0141-848-1825	Flat	350	astewart@hotmail.com
CR74	Mike	Ritchie	01475-392178	House	750	mritchie01@yahoo.co.uk
CR62	Mary	Tregear	01224-196720	Flat	600	maryt@hotmail.co.uk

PrivateOwner							
ownerNo	fName	lName	address	telNo	eMail	password	
CO46	Joe	Keogh	2 Fergus Dr, Aberdeen AB2 7SX	01224-861212	jkeogh@lhh.com	*****	
CO87	Carol	Farrel	6 Achray St, Glasgow G32 9DX	0141-357-7419	cfarrel@gmail.com	*****	
CO40	Tina	Murphy	63 Well St, Glasgow G42	0141-943-1728	tinam@hotmail.com	*****	
CO93	Tony	Shaw	12 Park Pl, Glasgow G4 0QR	0141-225-7025	tony.shaw@ark.com	*****	

Viewing				Registration			
clientNo	propertyNo	viewDate	comment	clientNo	branchNo	staffNo	dateJoined
CR56	PA14	24-May-13	too small	CR76	B005	SL41	2-Jan-13
CR76	PG4	20-Apr-13	too remote	CR56	B003	SG37	11-Apr-12
CR56	PG4	26-May-13		CR74	B003	SG37	16-Nov-11
CR62	PA14	14-May-13	no dining room	CR62	B007	SA9	7-Mar-12
CR56	PG36	28-Apr-13					

4.3 Integrity Constraints

In the previous section, we discussed the structural part of the relational data model. As stated in Section 2.3, a data model has two other parts: a manipulative part, defining the types of operation that are allowed on the data, and a set of integrity constraints, which ensure that the data is accurate. In this section, we discuss the relational integrity constraints, and in the next chapter, we discuss the relational manipulation operations.

We have already seen an example of an integrity constraint in Section 4.2.1: because every attribute has an associated domain, there are constraints (called **domain constraints**) that form restrictions on the set of values allowed for the attributes of relations. In addition, there are two important **integrity rules**, which are constraints or restrictions that apply to all instances of the database. The two principal rules for the relational model are known as entity integrity and **referential integrity**. Other types of integrity constraint are **multiplicity**, which we discuss in Section 12.6, and **general constraints**, which we introduce in Section 4.3.4. Before we define entity and referential integrity, it is necessary to understand the concept of nulls.

4.3.1 Nulls

Null

Represents a value for an attribute that is currently unknown or is not applicable for this tuple.

A null can be taken to mean the logical value “unknown.” It can mean that a value is not applicable to a particular tuple, or it could merely mean that no value has yet been supplied. Nulls are a way to deal with incomplete or exceptional data. However, a null is not the same as a zero numeric value or a text string filled with spaces; zeros and spaces are values, but a null represents the absence of a value. Therefore, nulls should be treated differently from other values. Some authors use the term “null value”; however, as a null is not a value but represents the absence of a value, the term “null value” is deprecated.

For example, in the Viewing relation shown in Figure 4.3, the comment attribute may be undefined until the potential renter has visited the property and returned his or her comment to the agency. Without nulls, it becomes necessary to introduce false data to represent this state or to add additional attributes that may not be meaningful to the user. In our example, we may try to represent a null comment with the value -1 . Alternatively, we may add a new attribute `hasCommentBeenSupplied` to the Viewing relation, which contains a Y (Yes) if a comment has been supplied, and N (No) otherwise. Both these approaches can be confusing to the user.

Nulls can cause implementation problems, arising from the fact that the relational model is based on first-order predicate calculus, which is a two-valued or Boolean logic—the only values allowed are true or false. Allowing nulls means that we have to work with a higher-valued logic, such as three- or four-valued logic (Codd, 1986, 1987, 1990).

The incorporation of nulls in the relational model is a contentious issue. Codd later regarded nulls as an integral part of the model (Codd, 1990). Others consider this approach to be misguided, believing that the missing information problem is not fully understood, that no fully satisfactory solution has been found and, consequently, that the incorporation of nulls in the relational model is premature (see, for example, Date, 1995).

We are now in a position to define the two relational integrity rules.

4.3.2 Entity Integrity

The first integrity rule applies to the primary keys of base relations. For the present, we define a base relation as a relation that corresponds to an entity in the conceptual schema (see Section 2.1). We provide a more precise definition in Section 4.4.

Entity integrity

In a base relation, no attribute of a primary key can be null.

By definition, a primary key is a minimal identifier that is used to identify tuples uniquely. This means that no subset of the primary key is sufficient to provide unique identification of tuples. If we allow a null for any part of a primary key, we are implying that not all the attributes are needed to distinguish between tuples, which contradicts the definition of the primary key. For example, as `branchNo` is the primary key of the `Branch` relation, we should not be able to insert a tuple into the `Branch` relation with a null for the `branchNo` attribute. As a second example, consider the composite primary key of the `Viewing` relation, comprising the client number (`clientNo`) and the property number (`propertyNo`). We should not be able to insert a tuple into the `Viewing` relation with a null for the `clientNo` attribute, or a null for the `propertyNo` attribute, or nulls for both attributes.

If we were to examine this rule in detail, we would find some anomalies. First, why does the rule apply only to primary keys and not more generally to candidate keys, which also identify tuples uniquely? Second, why is the rule restricted to base relations? For example, using the data of the `Viewing` relation shown in Figure 4.3, consider the query “List all comments from viewings.” This query will produce a unary relation consisting of the attribute `comment`. By definition, this attribute must be a primary key, but it contains nulls (corresponding to the viewings on PG36 and PG4 by client CR56). Because this relation is not a base relation, the model allows the primary key to be null. There have been several attempts to redefine this rule (see, for example, Codd, 1988, and Date, 1990).

4.3.3 Referential Integrity

The second integrity rule applies to foreign keys.

Referential integrity

If a foreign key exists in a relation, either the foreign key value must match a candidate key value of some tuple in its home relation or the foreign key value must be wholly null.

For example, `branchNo` in the `Staff` relation is a foreign key targeting the `branchNo` attribute in the home relation, `Branch`. It should not be possible to create a staff

record with branch number B025, for example, unless there is already a record for branch number B025 in the **Branch** relation. However, we should be able to create a new staff record with a null branch number to allow for the situation where a new member of staff has joined the company but has not yet been assigned to a particular branch office.

4.3.4 General Constraints

General constraints

Additional rules specified by the users or database administrators of a database that define or constrain some aspect of the enterprise.

It is also possible for users to specify additional constraints that the data must satisfy. For example, if an upper limit of 20 has been placed upon the number of staff that may work at a branch office, then the user must be able to specify this general constraint and expect the DBMS to enforce it. In this case, it should not be possible to add a new member of staff at a given branch to the **Staff** relation if the number of staff currently assigned to that branch is 20. Unfortunately, the level of support for general constraints varies from system to system. We discuss the implementation of relational integrity in 7 and 18.

4.4 Views

In the three-level ANSI-SPARC architecture presented in Chapter 2, we described an external view as the structure of the database as it appears to a particular user. In the relational model, the word “view” has a slightly different meaning. Rather than being the entire external model of a user’s view, a view is a **virtual** or **derived relation**: a relation that does not necessarily exist in its own right, but may be dynamically derived from one or more base relations. Thus, an external model can consist of both base (conceptual-level) relations and views derived from the base relations. In this section, we briefly discuss views in relational systems. In Section 7.4 we examine views in more detail and show how they can be created and used within SQL.

4.4.1 Terminology

The relations we have been dealing with so far in this chapter are known as base relations.

Base relation

A named relation corresponding to an entity in the conceptual schema, whose tuples are physically stored in the database.

We can define a view in terms of base relations.

View

The dynamic result of one or more relational operations operating on the base relations to produce another relation. A view is a *virtual relation* that does not necessarily exist in the database but can be produced upon request by a particular user, at the time of request.

A view is a relation that appears to the user to exist, can be manipulated as if it were a base relation, but does not necessarily exist in storage in the sense that the base relations do (although its definition is stored in the system catalog). The contents of a view are defined as a query on one or more base relations. Any operations on the view are automatically translated into operations on the relations from which it is derived. Views are **dynamic**, meaning that changes made to the base relations that affect the view are immediately reflected in the view. When users make permitted changes to the view, these changes are made to the underlying relations. In this section, we describe the purpose of views and briefly examine restrictions that apply to updates made through views. However, we defer treatment of how views are defined and processed until Section 7.4.

4.4.2 Purpose of Views

The view mechanism is desirable for several reasons:

- It provides a powerful and flexible security mechanism by hiding parts of the database from certain users. Users are not aware of the existence of any attributes or tuples that are missing from the view.
- It permits users to access data in a way that is customized to their needs, so that the same data can be seen by different users in different ways, at the same time.
- It can simplify complex operations on the base relations. For example, if a view is defined as a combination (join) of two relations (see Section 5.1), users may now perform more simple operations on the view, which will be translated by the DBMS into equivalent operations on the join.

A view should be designed to support the external model that the user finds familiar. For example:

- A user might need Branch tuples that contain the names of managers as well as the other attributes already in Branch. This view is created by combining the Branch relation with a restricted form of the Staff relation where the staff position is “Manager.”
- Some members of staff should see Staff tuples without the salary attribute.
- Attributes may be renamed or the order of attributes changed. For example, the user accustomed to calling the `branchNo` attribute of branches by the full name Branch Number may see that column heading.
- Some members of staff should see property records only for those properties that they manage.

Although all these examples demonstrate that a view provides *logical data independence* (see Section 2.1.5), views allow a more significant type of logical data independence that supports the reorganization of the conceptual schema. For example, if a new attribute is added to a relation, existing users can be unaware of its existence if their views are defined to exclude it. If an existing relation is rearranged or split up, a view may be defined so that users can continue to see their original views. We will see an example of this in Section 7.4.7 when we discuss the advantages and disadvantages of views in more detail.

4.4.3 Updating Views

All updates to a base relation should be immediately reflected in all views that reference that base relation. Similarly, if a view is updated, then the underlying base relation should reflect the change. However, there are restrictions on the types of modification that can be made through views. We summarize here the conditions under which most systems determine whether an update is allowed through a view:

- Updates are allowed through a view defined using a simple query involving a single base relation and containing either the primary key or a candidate key of the base relation.
- Updates are not allowed through views involving multiple base relations.
- Updates are not allowed through views involving aggregation or grouping operations.

Classes of views have been defined that are **theoretically not updatable**, **theoretically updatable**, and **partially updatable**. A survey on updating relational views can be found in Furtado and Casanova (1985).

Chapter Summary

- The Relational Database Management System (RDBMS) has become the dominant data-processing software in use today, with estimated new licence sales of between US\$6 billion and US\$10 billion per year (US\$25 billion with tools sales included). This software represents the second generation of DBMSs and is based on the relational data model proposed by E. F. Codd.
- A mathematical **relation** is a subset of the Cartesian product of two or more sets. In database terms, a relation is any subset of the Cartesian product of the domains of the attributes. A relation is normally written as a set of n -tuples, in which each element is chosen from the appropriate domain.
- Relations are physically represented as **tables**, with the rows corresponding to individual tuples and the columns to attributes.
- The structure of the relation, with domain specifications and other constraints, is part of the **intension** of the database; the relation with all its tuples written out represents an **instance** or **extension** of the database.
- Properties of database relations are: each cell contains exactly one atomic value, attribute names are distinct, attribute values come from the same domain, attribute order is immaterial, tuple order is immaterial, and there are no duplicate tuples.
- The **degree** of a relation is the number of attributes, and the **cardinality** is the number of tuples. A **unary** relation has one attribute, a **binary** relation has two, a **ternary** relation has three, and an **n -ary** relation has n attributes.
- A **superkey** is an attribute, or set of attributes, that identifies tuples of a relation uniquely, and a **candidate key** is a minimal superkey. A **primary key** is the candidate key chosen for use in identification of tuples. A relation must always have a primary key. A **foreign key** is an attribute, or set of attributes, within one relation that is the candidate key of another relation.
- A **null** represents a value for an attribute that is unknown at the present time or is not applicable for this tuple.

- **Entity integrity** is a constraint that states that in a base relation no attribute of a primary key can be null. **Referential integrity** states that foreign key values must match a candidate key value of some tuple in the home relation or be wholly null. Apart from relational integrity, integrity constraints include required data, domain, and multiplicity constraints; other integrity constraints are called **general constraints**.
- A **view** in the relational model is a **virtual** or **derived relation** that is dynamically created from the underlying base relation(s) when required. Views provide security and allow the designer to customize a user's model. Not all views are updatable.

Review Questions

- 4.1 Discuss each of the following concepts in the context of the relational data model:
 - (a) relation
 - (b) attribute
 - (c) domain
 - (d) tuple
 - (e) intension and extension
 - (f) degree and cardinality.
- 4.2 Describe the relationship between mathematical relations and relations in the relational data model.
- 4.3 Describe the term "normalized reaction." Why are constraints so important in a relational database?
- 4.4 Discuss the properties of a relation.
- 4.5 Discuss the differences between the candidate keys and the primary key of a relation. Explain what is meant by a foreign key. How do foreign keys of relations relate to candidate keys? Give examples to illustrate your answer.
- 4.6 Define the two principal integrity rules for the relational model. Discuss why it is desirable to enforce these rules.
- 4.7 Define "views." Why are they important in a database approach?

Exercises

The following tables form part of a database held in a relational DBMS:

Hotel (hotelNo, hotelName, city)
Room (roomNo, hotelNo, type, price)
Booking (hotelNo, guestNo, dateFrom, dateTo, roomNo)
Guest (guestNo, guestName, guestAddress)

where Hotel contains hotel details and hotelNo is the primary key;

Room contains room details for each hotel and (roomNo, hotelNo) forms the primary key;

Booking contains details of bookings and (hotelNo, guestNo, dateFrom) forms the primary key;

Guest contains guest details and guestNo is the primary key.

- 4.8 Identify the foreign keys in this schema. Explain how the entity and referential integrity rules apply to these relations.
- 4.9 Produce some sample tables for these relations that observe the relational integrity rules. Suggest some general constraints that would be appropriate for this schema.
- 4.10 Analyze the RDBMSs that you are currently using. Determine the support the system provides for primary keys, alternate keys, foreign keys, relational integrity, and views.
- 4.11 Implement the above schema in one of the RDBMSs you currently use. Generate two user-views that are accessible and updatable as well as two other user-views that cannot be updated.

Relational Algebra and Relational Calculus

Chapter Objectives

In this chapter you will learn:

- The meaning of the term “relational completeness.”
- How to form queries in the relational algebra.
- How to form queries in the tuple relational calculus.
- How to form queries in the domain relational calculus.
- The categories of relational Data Manipulation Languages (DMLs).

In the previous chapter we introduced the main structural components of the relational model. As we discussed in Section 2.3, another important part of a data model is a manipulation mechanism, or *query language*, to allow the underlying data to be retrieved and updated. In this chapter we examine the query languages associated with the relational model. In particular, we concentrate on the relational algebra and the relational calculus as defined by Codd (1971) as the basis for relational languages. Informally, we may describe the relational algebra as a (high-level) procedural language: it can be used to tell the DBMS how to build a new relation from one or more relations in the database. Again, informally, we may describe the relational calculus as a nonprocedural language: it can be used to formulate the definition of a relation in terms of one or more database relations. However, the relational algebra and relational calculus are formally equivalent to one another: for every expression in the algebra, there is an equivalent expression in the calculus (and vice versa).

Both the algebra and the calculus are formal, non-user-friendly languages. They have been used as the basis for other, higher-level Data Manipulation Languages (DMLs) for relational databases. They are of interest because they illustrate the basic operations required of any DML and because they serve as the standard of comparison for other relational languages.

The relational calculus is used to measure the selective power of relational languages. A language that can be used to produce any relation that can be derived using the relational calculus is said to be **relationally complete**. Most relational query languages are relationally complete but have more expressive

power than the relational algebra or relational calculus, because of additional operations such as calculated, summary, and ordering functions.



Structure of this Chapter In Section 5.1 we examine the relational algebra and in Section 5.2 we examine two forms of the relational calculus: tuple relational calculus and domain relational calculus. In Section 5.3 we briefly discuss some other relational languages. We use the *DreamHome* rental database instance shown in Figure 4.3 to illustrate the operations.

In Chapters 6–9 we examine SQL, the formal and *de facto* standard language for RDBMSs, which has constructs based on the tuple relational calculus. In Appendix M we examine QBE (Query-By-Example), another highly popular visual query language for RDBMSs, which is based in part on the domain relational calculus.

5.1 The Relational Algebra

The relational algebra is a theoretical language with operations that work on one or more relations to define another relation without changing the original relation(s). Thus both the operands and the results are relations, and so the output from one operation can become the input to another operation. This ability allows expressions to be nested in the relational algebra, just as we can nest arithmetic operations. This property is called **closure**: relations are closed under the algebra, just as numbers are closed under arithmetic operations.

The relational algebra is a relation-at-a-time (or set) language in which all tuples, possibly from several relations, are manipulated in one statement without looping. There are several variations of syntax for relational algebra commands and we use a common symbolic notation for the commands and present it informally. The interested reader is referred to Ullman (1988) for a more formal treatment.

There are many variations of the operations that are included in relational algebra. Codd (1972a) originally proposed eight operations, but several others have been developed. The five fundamental operations in relational algebra—Selection, Projection, Cartesian product, Union, and Set difference—perform most of the data retrieval operations that we are interested in. In addition, there are also the Join, Intersection, and Division operations, which can be expressed in terms of the five basic operations. The function of each operation is illustrated in Figure 5.1.

The Selection and Projection operations are **unary** operations, as they operate on one relation. The other operations work on pairs of relations and are therefore called **binary** operations. In the following definitions, let R and S be two relations defined over the attributes $A = (a_1, a_2, \dots, a_N)$ and $B = (b_1, b_2, \dots, b_M)$, respectively.

5.1.1 Unary Operations

We start the discussion of the relational algebra by examining the two unary operations: Selection and Projection.

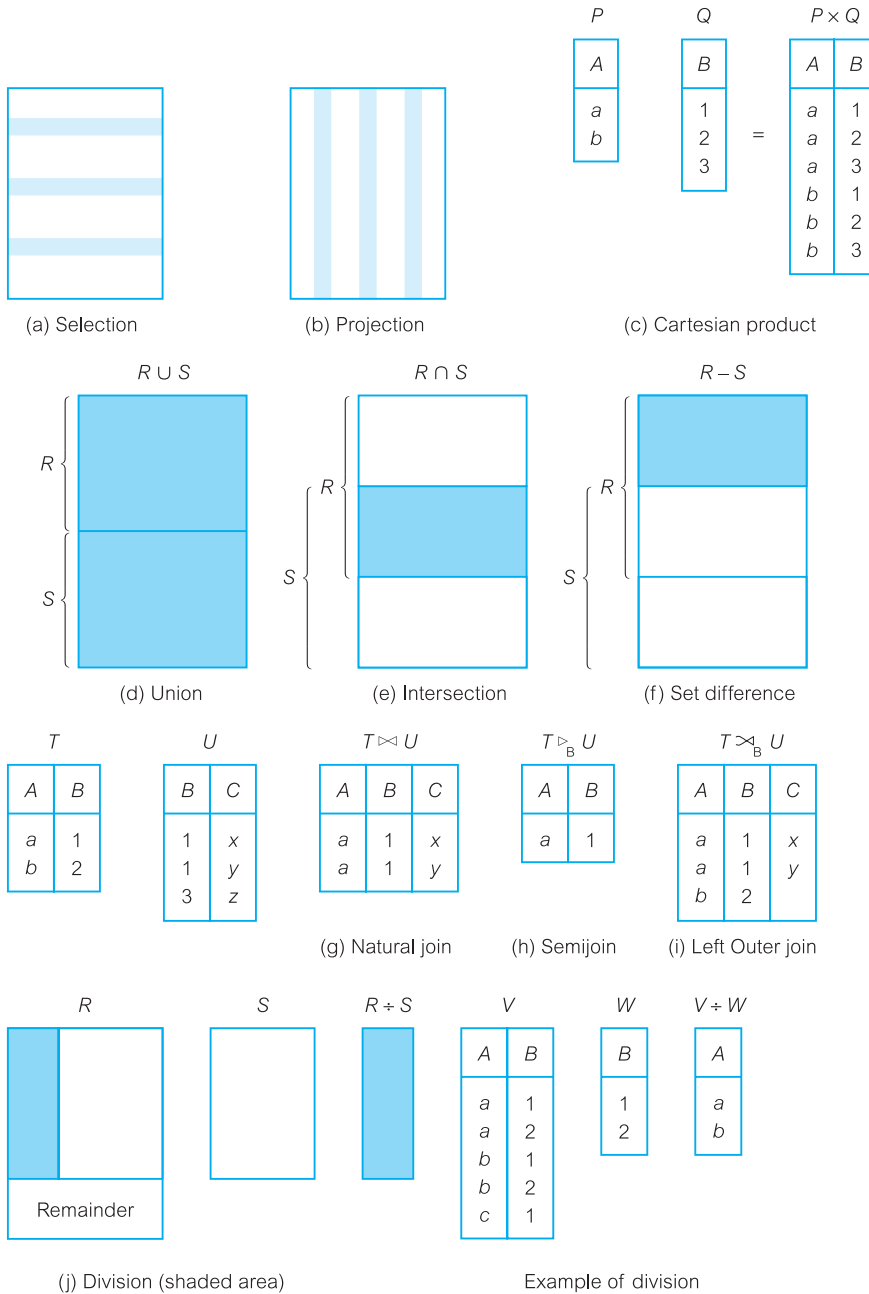


Figure 5.1
Illustration
showing the
function of the
relational algebra
operations.

Selection (or Restriction)

$\sigma_{\text{predicate}}(\mathbf{R})$

The Selection operation works on a single relation R and defines a relation that contains only those tuples of R that satisfy the specified condition (*predicate*).

EXAMPLE 5.1 Selection operation

List all staff with a salary greater than £10000.

$\sigma_{\text{salary} > 10000}(\text{Staff})$

Here, the input relation is **Staff** and the predicate is **salary > 10000**. The Selection operation defines a relation containing only those **Staff** tuples with a salary greater than £10000. The result of this operation is shown in Figure 5.2. More complex predicates can be generated using the logical operators **(AND)**, **(OR)**, and **~ (NOT)**.

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003

Figure 5.2 Selecting salary > 10000 from the Staff relation.

Projection

$\Pi_{a_1, \dots, a_n}(\mathbf{R})$

The Projection operation works on a single relation **R** and defines a relation that contains a vertical subset of **R**, extracting the values of specified attributes and eliminating duplicates.

EXAMPLE 5.2 Projection operation

Produce a list of salaries for all staff, showing only the staffNo, fName, lName, and salary details.

$\Pi_{\text{staffNo, fName, lName, salary}}(\text{Staff})$

In this example, the Projection operation defines a relation that contains only the designated **Staff** attributes **staffNo**, **fName**, **lName**, and **salary**, in the specified order. The result of this operation is shown in Figure 5.3.

staffNo	fName	lName	salary
SL21	John	White	30000
SG37	Ann	Beech	12000
SG14	David	Ford	18000
SA9	Mary	Howe	9000
SG5	Susan	Brand	24000
SL41	Julie	Lee	9000

Figure 5.3 Projecting the Staff relation over the staffNo, fName, lName, and salary attributes.

5.1.2 Set Operations

The Selection and Projection operations extract information from only one relation. There are obviously cases where we would like to combine information from several relations. In the remainder of this section, we examine the binary operations of the relational algebra, starting with the set operations of Union, Set difference, Intersection, and Cartesian product.

Union

$R \cup S$

The union of two relations R and S defines a relation that contains all the tuples of R , or S , or both R and S , duplicate tuples being eliminated. R and S must be union-compatible.

If R and S have I and J tuples, respectively, their union is obtained by concatenating them into one relation with a maximum of $(I + J)$ tuples. Union is possible only if the schemas of the two relations match, that is, if they have the same number of attributes with each pair of corresponding attributes having the same domain. In other words, the relations must be **union-compatible**. Note that attributes names are not used in defining union-compatibility. In some cases, the Projection operation may be used to make two relations union-compatible.

EXAMPLE 5.3 Union operation

List all cities where there is either a branch office or a property for rent.

$$\Pi_{\text{city}}(\text{Branch}) \cup \Pi_{\text{city}}(\text{PropertyForRent})$$

To produce union-compatible relations, we first use the Projection operation to project the Branch and PropertyForRent relations over the attribute city, eliminating duplicates where necessary. We then use the Union operation to combine these new relations to produce the result shown in Figure 5.4.

city
London
Aberdeen
Glasgow
Bristol

Figure 5.4

Union based on the city attribute from the Branch and Property-ForRent relations

Set difference

$R - S$

The Set difference operation defines a relation consisting of the tuples that are in relation R , but not in S . R and S must be union-compatible.

EXAMPLE 5.4 Set difference operation

List all cities where there is a branch office but no properties for rent.

$$\Pi_{\text{city}}(\text{Branch}) - \Pi_{\text{city}}(\text{PropertyForRent})$$

As in the previous example, we produce union-compatible relations by projecting the Branch and PropertyForRent relations over the attribute city. We then use the Set difference operation to combine these new relations to produce the result shown in Figure 5.5.

city
Bristol

Figure 5.5

Set difference based on the city attribute from the Branch and PropertyForRent relations.

Intersection

city
Aberdeen
London
Glasgow

Figure 5.6
Intersection based on city attribute from the Branch and PropertyForRent relations.

$R \cap S$

The Intersection operation defines a relation consisting of the set of all tuples that are in both R and S. R and S must be union-compatible.

EXAMPLE 5.5 Intersection operation

List all cities where there is both a branch office and at least one property for rent.

$$\Pi_{\text{city}}(\text{Branch}) \cap \Pi_{\text{city}}(\text{PropertyForRent})$$

As in the previous example, we produce union-compatible relations by projecting the Branch and PropertyForRent relations over the attribute city. We then use the Intersection operation to combine these new relations to produce the result shown in Figure 5.6.

Note that we can express the Intersection operation in terms of the Set difference operation:

$$R \cap S = R - (R - S)$$

Cartesian product

$R \times S$

The Cartesian product operation defines a relation that is the concatenation of every tuple of relation R with every tuple of relation S.

The Cartesian product operation multiplies two relations to define another relation consisting of all possible pairs of tuples from the two relations. Therefore, if one relation has *I* tuples and *N* attributes and the other has *J* tuples and *M* attributes, the Cartesian product relation will contain (*I* * *J*) tuples with (*N* + *M*) attributes. It is possible that the two relations may have attributes with the same name. In this case, the attribute names are prefixed with the relation name to maintain the uniqueness of attribute names within a relation.

EXAMPLE 5.6 Cartesian product operation

List the names and comments of all clients who have viewed a property for rent.

The names of clients are held in the Client relation and the details of viewings are held in the Viewing relation. To obtain the list of clients and the comments on properties they have viewed, we need to combine these two relations:

$$\Pi_{\text{clientNo}, \text{fName}, \text{lName}}(\text{Client}) \times (\Pi_{\text{clientNo}, \text{propertyNo}, \text{comment}}(\text{Viewing}))$$

The result of this operation is shown in Figure 5.7. In its present form, this relation contains more information than we require. For example, the first tuple of this relation contains different clientNo values. To obtain the required list, we need to carry out a Selection operation on this relation to extract those tuples where Client.clientNo = Viewing.clientNo. The complete operation is thus:

$$\sigma_{\text{Client.clientNo} = \text{Viewing.clientNo}}((\Pi_{\text{clientNo}, \text{fName}, \text{lName}}(\text{Client})) \times (\Pi_{\text{clientNo}, \text{propertyNo}, \text{comment}}(\text{Viewing})))$$

The result of this operation is shown in Figure 5.8.

client.clientNo	fName	lName	Viewing.clientNo	propertyNo	comment
CR76	John	Kay	CR56	PA14	too small
CR76	John	Kay	CR76	PG4	too remote
CR76	John	Kay	CR56	PG4	
CR76	John	Kay	CR62	PA14	no dining room
CR76	John	Kay	CR56	PG36	
CR56	Aline	Stewart	CR56	PA14	too small
CR56	Aline	Stewart	CR76	PG4	too remote
CR56	Aline	Stewart	CR56	PG4	
CR56	Aline	Stewart	CR62	PA14	no dining room
CR56	Aline	Stewart	CR56	PG36	
CR74	Mike	Ritchie	CR56	PA14	too small
CR74	Mike	Ritchie	CR76	PG4	too remote
CR74	Mike	Ritchie	CR56	PG4	
CR74	Mike	Ritchie	CR62	PA14	no dining room
CR74	Mike	Ritchie	CR56	PG36	
CR62	Mary	Tregear	CR56	PA14	too small
CR62	Mary	Tregear	CR76	PG4	too remote
CR62	Mary	Tregear	CR56	PG4	
CR62	Mary	Tregear	CR62	PA14	no dining room
CR62	Mary	Tregear	CR56	PG36	

Figure 5.7 Cartesian product of reduced Client and Viewing relations.

client.clientNo	fName	lName	Viewing.clientNo	propertyNo	comment
CR76	John	Kay	CR76	PG4	too remote
CR56	Aline	Stewart	CR56	PA14	too small
CR56	Aline	Stewart	CR56	PG4	
CR56	Aline	Stewart	CR56	PG36	
CR62	Mary	Tregear	CR62	PA14	no dining room

Figure 5.8 Restricted Cartesian product of reduced Client and Viewing relations.

Decomposing complex operations

The relational algebra operations can be of arbitrary complexity. We can decompose such operations into a series of smaller relational algebra operations and give a name to the results of intermediate expressions. We use the assignment operation, denoted by $\leftarrow$, to name the results of a relational algebra operation. This works in a similar manner to the assignment operation in a programming language: in this case, the right-hand side of the operation is assigned to the left-hand side. For instance, in the previous example we could rewrite the operation as follows:

```

TempViewing(clientNo, propertyNo, comment)  $\leftarrow$   $\Pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing})$ 
TempClient(clientNo, fName, lName)  $\leftarrow$   $P_{\text{clientNo, fName, lName}}(\text{Client})$ 
Comment(clientNo, fName, lName, vclientNo, propertyNo, comment)  $\leftarrow$ 
    TempClient  $\times$  TempViewing
Result  $\leftarrow$   $\sigma_{\text{clientNo} = \text{vclientNo}}(\text{Comment})$ 

```

Alternatively, we can use the Rename operation ρ (rho), which gives a name to the result of a relational algebra operation. Rename allows an optional name for each of the attributes of the new relation to be specified.

$\rho_S(E)$ or
 $\rho_{S(a_1, a_2, \dots, a_n)}(E)$

The Rename operation provides a new name S for the expression E , and optionally names the attributes as $a_1, a_2, \dots, a_n$.

5.1.3 Join Operations

Typically, we want only combinations of the Cartesian product that satisfy certain conditions and so we would normally use a **Join operation** instead of the Cartesian product operation. The Join operation, which combines two relations to form a new relation, is one of the essential operations in the relational algebra. Join is a derivative of Cartesian product, equivalent to performing a Selection operation, using the join predicate as the selection formula, over the Cartesian product of the two operand relations. Join is one of the most difficult operations to implement efficiently in an RDBMS and is one of the reasons why relational systems have intrinsic performance problems. We examine strategies for implementing the Join operation in Section 23.4.3.

There are various forms of the Join operation, each with subtle differences, some more useful than others:

- Theta join
- Equijoin (a particular type of Theta join)
- Natural join
- Outer join
- Semijoin

Theta join (θ -join)

$R \bowtie_F S$

The Theta join operation defines a relation that contains tuples satisfying the predicate F from the Cartesian product of R and S . The predicate F is of the form $R.a_i \theta S.b_j$, where θ may be one of the comparison operators ($<$, $\leq$, $>$, $\geq$, $=$, $\neq$).

We can rewrite the Theta join in terms of the basic Selection and Cartesian product operations:

$$R \bowtie_F S = \sigma_F(R \times S)$$

As with a Cartesian product, the degree of a Theta join is the sum of the degrees of the operand relations R and S . In the case where the predicate F contains only equality ($=$), the term **Equijoin** is used instead. Consider again the query of Example 5.6.

EXAMPLE 5.7 Equijoin operation

List the names and comments of all clients who have viewed a property for rent.

In Example 5.6 we used the Cartesian product and Selection operations to obtain this list. However, the same result is obtained using the Equijoin operation:

$$(\Pi_{\text{clientNo}, \text{fName}, \text{lName}}(\text{Client})) \bowtie_{\text{Client.clientNo} = \text{Viewing.clientNo}} (\Pi_{\text{clientNo}, \text{propertyNo}, \text{comment}}(\text{Viewing}))$$

or

$$\text{Result} \leftarrow \text{TempClient} \bowtie_{\text{TempClient.clientNo} = \text{TempViewing.clientNo}} \text{TempViewing}$$

The result of these operations was shown in Figure 5.8.

Natural join

R $\bowtie$ **S**

The Natural join is an Equijoin of the two relations R and S over all common attributes x . One occurrence of each common attribute is eliminated from the result.

The Natural join operation performs an Equijoin over all the attributes in the two relations that have the same name. The degree of a Natural join is the sum of the degrees of the relations R and S less the number of attributes in x .

EXAMPLE 5.8 Natural join operation

List the names and comments of all clients who have viewed a property for rent.

In Example 5.7 we used the Equijoin to produce this list, but the resulting relation had two occurrences of the join attribute `clientNo`. We can use the Natural join to remove one occurrence of the `clientNo` attribute:

$$(\Pi_{\text{clientNo}, \text{fName}, \text{lName}}(\text{Client})) \bowtie (\Pi_{\text{clientNo}, \text{propertyNo}, \text{comment}}(\text{Viewing}))$$

or

$$\text{Result} \leftarrow \text{TempClient} \bowtie \text{TempViewing}$$

The result of this operation is shown in Figure 5.9.

clientNo	fName	lName	propertyNo	comment
CR76	John	Kay	PG4	too remote
CR56	Aline	Stewart	PA14	too small
CR56	Aline	Stewart	PG4	
CR56	Aline	Stewart	PG36	
CR62	Mary	Tregear	PA14	no dining room

Figure 5.9 Natural join of restricted Client and Viewing relations.

Outer join

Often in joining two relations, a tuple in one relation does not have a matching tuple in the other relation; in other words, there is no matching value in the join attributes. We may want tuples from one of the relations to appear in the result even when there are no matching values in the other relation. This may be accomplished using the Outer join.

$R \bowtie S$

The (left) Outer join is a join in which tuples from R that do not have matching values in the common attributes of S are also included in the result relation. Missing values in the second relation are set to null.

The Outer join is becoming more widely available in relational systems and is a specified operator in the SQL standard (see Section 6.3.7). The advantage of an Outer join is that information is preserved; that is, the Outer join preserves tuples that would have been lost by other types of join.

EXAMPLE 5.9 Left Outer join operation

Produce a status report on property viewings.

In this example, we want to produce a relation consisting of the properties that have been viewed with comments and those that have not been viewed. This can be achieved using the following Outer join:

$(\Pi_{\text{propertyNo, street, city}}(\text{PropertyForRent})) \bowtie \text{Viewing}$

The resulting relation is shown in Figure 5.10. Note that properties PL94, PG21, and PG16 have no viewings, but these tuples are still contained in the result with nulls for the attributes from the Viewing relation.

propertyNo	street	city	clientNo	viewDate	comment
PA14	16 Holhead	Aberdeen	CR56	24-May-13	too small
PA14	16 Holhead	Aberdeen	CR62	14-May-13	no dining room
PL94	6 Argyll St	London	null	null	null
PG4	6 Lawrence St	Glasgow	CR76	20-Apr- 13	too remote
PG4	6 Lawrence St	Glasgow	CR56	26-May-13	
PG36	2 Manor Rd	Glasgow	CR56	28-Apr- 13	
PG21	18 Dale Rd	Glasgow	null	null	null
PG16	5 Novar Dr	Glasgow	null	null	null

Figure 5.10 Left (natural) Outer join of PropertyForRent and Viewing relations.

Strictly speaking, Example 5.9 is a **Left (natural) Outer join**, as it keeps every tuple in the left-hand relation in the result. Similarly, there is a **Right Outer join** that keeps every tuple in the right-hand relation in the result. There is also a **Full Outer join** that keeps all tuples in both relations, padding tuples with nulls when no matching tuples are found.

Semijoin

$R \bowtie_F S$

The Semijoin operation defines a relation that contains the tuples of R that participate in the join of R with S satisfying the predicate F .

The Semijoin operation performs a join of the two relations and then projects over the attributes of the first operand. One advantage of a Semijoin is that it decreases the number of tuples that need to be handled to form the join. It is particularly useful for computing joins in distributed systems (see Sections 24.4.2 and 25.6.2). We can rewrite the Semijoin using the Projection and Join operations:

$$R \bowtie_F S = \Pi_A(R \bowtie_F S) \quad A \text{ is the set of all attributes for } R$$

This is actually a Semi-Theta join. There are variants for the Semi-Equijoin and the Semi-Natural join.

EXAMPLE 5.10 Semijoin operation

List complete details of all staff who work at the branch in Glasgow.

If we are interested in seeing only the attributes of the **Staff** relation, we can use the following Semijoin operation, producing the relation shown in Figure 5.11.

Staff $\bowtie_{\text{Staff branchNo} = \text{Branch branchNo} (\sigma_{\text{city} = \text{'Glasgow'}} (\text{Branch}))}$

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003

Figure 5.11 Semijoin of Staff and Branch relations.

5.1.4 Division Operation

The Division operation is useful for a particular type of query that occurs quite frequently in database applications. Assume that relation R is defined over the attribute set A and relation S is defined over the attribute set B such that $B \subseteq A$ (B is a subset of A). Let $C = A - B$, that is, C is the set of attributes of R that are not attributes of S . We have the following definition of the Division operation.

$R \div S$

The Division operation defines a relation over the attributes C that consists of the set of tuples from R that match the combination of **every** tuple in S .

We can express the Division operation in terms of the basic operations:

$$\begin{aligned} T_1 &\leftarrow \Pi_C(R) \\ T_2 &\leftarrow \Pi_C((T_1 \times S) - R) \\ T &\leftarrow T_1 - T_2 \end{aligned}$$

EXAMPLE 5.11 Division operation

Identify all clients who have viewed all properties with three rooms.

We can use the Selection operation to find all properties with three rooms followed by the Projection operation to produce a relation containing only these property numbers. We can then use the following Division operation to obtain the new relation shown in Figure 5.12.

$$(\Pi_{\text{clientNo, propertyNo}}(\text{Viewing})) \div (\Pi_{\text{propertyNo}}(\sigma_{\text{rooms} = 3}(\text{PropertyForRent})))$$

$\Pi_{\text{clientNo, propertyNo}}(\text{Viewing})$		$\Pi_{\text{propertyNo}}(\sigma_{\text{rooms} = 3}(\text{PropertyForRent}))$	RESULT
clientNo	propertyNo	propertyNo	clientNo
CR56	PA14	PG4	CR56
CR76	PG4	PG36	
CR56	PG4		
CR62	PA14		
CR56	PG36		

Figure 5.12 Result of the Division operation on the Viewing and PropertyForRent relations.

5.1.5 Aggregation and Grouping Operations

As well as simply retrieving certain tuples and attributes of one or more relations, we often want to perform some form of summation or **aggregation** of data, similar to the totals at the bottom of a report, or some form of **grouping** of data, similar to subtotals in a report. These operations cannot be performed using the basic relational algebra operations considered earlier. However, additional operations have been proposed, as we now discuss.

Aggregate operations

AL(R)	Applies the aggregate function list, AL, to the relation R to define a relation over the aggregate list. AL contains one or more (<aggregate_function>, <attribute>) pairs.
--------------	---

The main aggregate functions are:

- COUNT – returns the number of values in the associated attribute.
- SUM – returns the sum of the values in the associated attribute.
- AVG – returns the average of the values in the associated attribute.
- MIN – returns the smallest value in the associated attribute.
- MAX – returns the largest value in the associated attribute.

EXAMPLE 5.12 Aggregate operations

(a) How many properties cost more than £350 per month to rent?

We can use the aggregate function COUNT to produce the relation R shown in Figure 5.13(a):

myCount	myMin	myMax	myAverage
5	9000	30000	17000

(a) (b)

Figure 5.13 Result of the Aggregate operations: (a) finding the number of properties whose rent is greater than £350; (b) finding the minimum, maximum, and average staff salary.

$\rho_R(\text{myCount}) \text{ COUNT propertyNo } (\sigma_{\text{rent} > 350} (\text{PropertyForRent}))$

(b) Find the minimum, maximum, and average staff salary.

We can use the aggregate functions—MIN, MAX, and AVERAGE—to produce the relation R shown in Figure 5.13(b) as follows:

$\rho_R(\text{myMin, myMax, myAverage}) \text{ MIN salary, MAX salary, AVERAGE salary } (\text{Staff})$

Grouping operation

$\text{GA AL}(\mathbf{R})$

Groups the tuples of relation R by the grouping attributes, GA, and then applies the aggregate function list AL to define a new relation. AL contains one or more ($\langle \text{aggregate_function} \rangle, \langle \text{attribute} \rangle$) pairs. The resulting relation contains the grouping attributes, GA, along with the results of each of the aggregate functions.

The general form of the grouping operation is as follows:

$a_1, a_2, \dots, a_n \text{ } \langle A_{p_1} \rangle, \langle A_{q_1} \rangle, \dots, \langle A_{z_1} \rangle (\mathbf{R})$

where R is any relation, $a_1, a_2, \dots, a_n$ are attributes of R on which to group, $a_p, a_q, \dots, a_z$ are other attributes of R, and $A_p, A_q, \dots, A_z$ are aggregate functions. The tuples of R are partitioned into groups such that:

- all tuples in a group have the same value for $a_1, a_2, \dots, a_n$;
- tuples in different groups have different values for $a_1, a_2, \dots, a_n$.

We illustrate the use of the grouping operation with the following example.

EXAMPLE 5.13 Grouping operation

Find the number of staff working in each branch and the sum of their salaries.

We first need to group tuples according to the branch number, `branchNo`, and then use the aggregate functions COUNT and SUM to produce the required relation. The relational algebra expression is as follows:

$\rho_R(\text{branchNo, myCount, mySum}) \text{ branchNo COUNT staffNo, SUM salary } (\text{Staff})$

The resulting relation is shown in Figure 5.14.

branchNo	myCount	mySum
B003	3	54000
B005	2	39000
B007	1	9000

Figure 5.14 Result of the grouping operation to find the number of staff working in each branch and the sum of their salaries.

5.1.6 Summary of the Relational Algebra Operations

The relational algebra operations are summarized in Table 5.1.

TABLE 5.1 Operations in the relational algebra.

OPERATION	NOTATION	FUNCTION
Selection	$\sigma_{\text{predicate}}(R)$	Produces a relation that contains only those tuples of R that satisfy the specified <i>predicate</i> .
Projection	$\Pi_{a_1, \dots, a_n}(R)$	Produces a relation that contains a vertical subset of R, extracting the values of specified attributes and eliminating duplicates.
Union	$R \cup S$	Produces a relation that contains all the tuples of R, or S, or both R and S, duplicate tuples being eliminated. R and S must be union-compatible.
Set difference	$R - S$	Produces a relation that contains all the tuples in R that are not in S. R and S must be union-compatible.
Intersection	$R \cap S$	Produces a relation that contains all the tuples in both R and S. R and S must be union-compatible.
Cartesian product	$R \times S$	Produces a relation that is the concatenation of every tuple of relation R with every tuple of relation S.
Theta join	$R \bowtie_F S$	Produces a relation that contains tuples satisfying the predicate <i>F</i> from the Cartesian product of R and S.
Equijoin	$R \bowtie_F S$	Produces a relation that contains tuples satisfying the predicate <i>F</i> (which contains only equality comparisons) from the Cartesian product of R and S.
Natural join	$R \bowtie S$	An Equijoin of the two relations R and S over all common attributes <i>x</i> . One occurrence of each common attribute is eliminated.
(Left) Outer join	$R \rhd S$	A join in which tuples from R that do not have matching values in the common attributes of S are also included in the result relation.
Semijoin	$R \bowtie_F S$	Produces a relation that contains the tuples of R that participate in the join of R with S satisfying the predicate <i>F</i> .
Division	$R \div S$	Produces a relation that consists of the set of tuples from R defined over the attributes <i>C</i> that match the combination of every tuple in S, where <i>C</i> is the set of attributes that are in R but not in S.
Aggregate	$\alpha_{AL}(R)$	Applies the aggregate function list, AL, to the relation R to define a relation over the aggregate list. AL contains one or more (<aggregate_function>, <attribute>) pairs.
Grouping	$\gamma_{GA, AL}(R)$	Groups the tuples of relation R by the grouping attributes, GA, and then applies the aggregate function list AL to define a new relation. AL contains one or more (<aggregate_function>, <attribute>) pairs. The resulting relation contains the grouping attributes, GA, along with the results of each of the aggregate functions.

5.2 The Relational Calculus

A certain order is always explicitly specified in a relational algebra expression and a strategy for evaluating the query is implied. In the relational calculus, there is no description of how to evaluate a query; a relational calculus query specifies *what* is to be retrieved rather than *how* to retrieve it.

The relational calculus is not related to differential and integral calculus in mathematics, but takes its name from a branch of symbolic logic called **predicate calculus**. When applied to databases, it is found in two forms: **tuple** relational calculus, as originally proposed by Codd (1972a), and **domain** relational calculus, as proposed by Lacroix and Pirotte (1977).

In first-order logic or predicate calculus, a **predicate** is a truth-valued function with arguments. When we substitute values for the arguments, the function yields an expression, called a **proposition**, which can be either true or false. For example, the sentences, “John White is a member of staff” and “John White earns more than Ann Beech” are both propositions, because we can determine whether they are true or false. In the first case, we have a function, “is a member of staff,” with one argument (John White); in the second case, we have a function, “earns more than,” with two arguments (John White and Ann Beech).

If a predicate contains a variable, as in “ x is a member of staff,” there must be an associated **range** for x . When we substitute some values of this range for x , the proposition may be true; for other values, it may be false. For example, if the range is the set of all people and we replace x by John White, the proposition “John White is a member of staff” is true. If we replace x by the name of a person who is not a member of staff, the proposition is false.

If P is a predicate, then we can write the set of all x such that P is true for x , as:

$$\{x \mid P(x)\}$$

We may connect predicates by the logical connectives (AND), (OR), and $\sim$ (NOT) to form compound predicates.

5.2.1 Tuple Relational Calculus

In the tuple relational calculus, we are interested in finding tuples for which a predicate is true. The calculus is based on the use of **tuple variables**. A tuple variable is a variable that “ranges over” a named relation: that is, a variable whose only permitted values are tuples of the relation. (The word “range” here does not correspond to the mathematical use of range, but corresponds to a mathematical domain.) For example, to specify the range of a tuple variable S as the Staff relation, we write:

$$\text{Staff}(S)$$

To express the query “Find the set of all tuples S such that $F(S)$ is true,” we can write:

$$\{S \mid F(S)\}$$

F is called a **formula** (**well-formed formula**, or **wff** in mathematical logic). For example, to express the query “Find the staffNo, fName, lName, position,

sex, DOB, salary, and branchNo of all staff earning more than £10,000,” we can write:

$$\{S \mid \text{Staff}(S) \text{ } S.\text{salary} > 10000\}$$

$S.\text{salary}$ means the value of the salary attribute for the tuple variable S . To retrieve a particular attribute, such as salary, we would write:

$$\{S.\text{salary} \mid \text{Staff}(S) \text{ } S.\text{salary} > 10000\}$$

The existential and universal quantifiers

There are two **quantifiers** we can use with formulae to tell how many instances the predicate applies to. The **existential quantifier** $\exists$ (“there exists”) is used in formulae that must be true for at least one instance, such as:

$$\text{Staff}(S) \text{ } (\exists B) (\text{Branch}(B) \text{ } (B.\text{branchNo} = S.\text{branchNo}) \text{ } B.\text{city} = \text{'London'})$$

This means, “There exists a Branch tuple that has the same branchNo as the branchNo of the current Staff tuple, S , and is located in London.” The **universal quantifier** $\forall$ (“for all”) is used in statements about every instance, such as:

$$(\forall B) (B.\text{city} \neq \text{'Paris'})$$

This means, “For all Branch tuples, the address is not in Paris.” We can apply a generalization of De Morgan’s laws to the existential and universal quantifiers. For example:

$$(\exists X)(F(X)) \sim (\forall X)(\sim(F(X)))$$

$$(\forall X)(F(X)) \sim (\exists X)(\sim(F(X)))$$

$$(\exists X)(F_1(X) \text{ } F_2(X)) \sim (\forall X)(\sim(F_1(X)) \text{ } \sim(F_2(X)))$$

$$(\forall X)(F_1(X) \text{ } F_2(X)) \sim (\exists X)(\sim(F_1(X)) \text{ } \sim(F_2(X)))$$

Using these equivalence rules, we can rewrite the previous formula as:

$$\sim(\exists B) (B.\text{city} = \text{'Paris'})$$

which means, “There are no branches with an address in Paris.”

Tuple variables that are qualified by $\forall$ or $\exists$ are called **bound variables**; the other tuple variables are called **free variables**. The only free variables in a relational calculus expression should be those on the left side of the bar ($|$). For example, in the following query:

$$\{S.\text{fName}, S.\text{IName} \mid \text{Staff}(S) \text{ } (\exists B) (\text{Branch}(B) \text{ } (B.\text{branchNo} = S.\text{branchNo}) \text{ } B.\text{city} = \text{'London'})\}$$

S is the only free variable and S is then bound successively to each tuple of Staff.

Expressions and formulae

As with the English alphabet, in which some sequences of characters do not form a correctly structured sentence, in calculus not every sequence of formulae is acceptable. The formulae should be those sequences that are unambiguous and make sense. An expression in the tuple relational calculus has the following general form:

$$\{S_1.a_1, S_2.a_2, \dots, S_n.a_n \mid F(S_1, S_2, \dots, S_n)\} \quad m \geq n$$

where $S_1, S_2, \dots, S_n$ are tuple variables; each a_i is an attribute of the relation over which S_i ranges; and F is a formula. A (well-formed) formula is made out of one or more *atoms*, where an atom has one of the following forms:

- $R(S_i)$, where S_i is a tuple variable and R is a relation.
- $S_i.a_1 \theta S_j.a_2$, where S_i and S_j are tuple variables, a_1 is an attribute of the relation over which S_i ranges, a_2 is an attribute of the relation over which S_j ranges, and θ is one of the comparison operators ($<$, $\leq$, $>$, $\geq$, $=$, $\neq$); the attributes a_1 and a_2 must have domains whose members can be compared by θ .
- $S_i.a_1 \theta c$, where S_i is a tuple variable, a_1 is an attribute of the relation over which S_i ranges, c is a constant from the domain of attribute a_1 , and θ is one of the comparison operators.

We recursively build up formulae from atoms using the following rules:

- An atom is a formula.
- If F_1 and F_2 are formulae, so are their conjunction $F_1 \wedge F_2$, their disjunction $F_1 \vee F_2$, and the negation $\neg F_1$.
- If F is a formula with free variable X , then $(\exists X)(F)$ and $(\forall X)(F)$ are also formulae.

EXAMPLE 5.14 Tuple relational calculus

(a) List the names of all managers who earn more than £25,000.

$$\{S.fName, S.lName \mid Staff(S) \wedge S.position = 'Manager' \wedge S.salary > 25000\}$$

(b) List the staff who manage properties for rent in Glasgow.

$$\{S \mid Staff(S) \wedge (\exists P) (PropertyForRent(P) \wedge (P.staffNo = S.staffNo) \wedge P.city = 'Glasgow')\}$$

The *staffNo* attribute in the *PropertyForRent* relation holds the staff number of the member of staff who manages the property. We could reformulate the query as: “For each member of staff whose details we want to list, there exists a tuple in the relation *PropertyForRent* for that member of staff with the value of the attribute *city* in that tuple being Glasgow.”

Note that in this formulation of the query, there is no indication of a strategy for executing the query—the DBMS is free to decide the operations required to fulfil the request and the execution order of these operations. On the other hand, the equivalent relational algebra formulation would be: “Select tuples from *PropertyForRent* such that the city is Glasgow and perform their join with the *Staff* relation,” which has an implied order of execution.

(c) List the names of staff who currently do not manage any properties.

$$\{S.fName, S.lName \mid Staff(S) \wedge (\neg(\exists P) (PropertyForRent(P) \wedge (S.staffNo = P.staffNo)))\}$$

Using the general transformation rules for quantifiers given previously, we can rewrite this as:

$$\{S.fName, S.lName \mid Staff(S) \wedge ((\forall P) (\neg PropertyForRent(P) \vee \neg(S.staffNo = P.staffNo)))\}$$

(d) List the names of clients who have viewed a property for rent in Glasgow.

$$\{C.fName, C.lName \mid Client(C) \wedge ((\exists V) (\exists P) (Viewing(V) \wedge PropertyForRent(P) \wedge (C.clientNo = V.clientNo) \wedge (V.propertyNo = P.propertyNo) \wedge P.city = 'Glasgow'))\}$$

To answer this query, note that we can rephrase “clients who have viewed a property in Glasgow” as “clients for whom there exists some viewing of some property in Glasgow.”

(e) *List all cities where there is either a branch office or a property for rent.*

$$\{T.city \mid (\exists B) (Branch(B) \wedge (B.city = T.city)) \vee (\exists P) (PropertyForRent(P) \wedge (P.city = T.city))\}$$

Compare this with the equivalent relational algebra expression given in Example 5.3.

(f) *List all the cities where there is a branch office but no properties for rent.*

$$\{B.city \mid Branch(B) \wedge \neg(\exists P) (PropertyForRent(P) \wedge (B.city = P.city))\}$$

Compare this with the equivalent relational algebra expression given in Example 5.4.

(g) *List all the cities where there is both a branch office and at least one property for rent.*

$$\{B.city \mid Branch(B) \wedge (\exists P) (PropertyForRent(P) \wedge (B.city = P.city))\}$$

Compare this with the equivalent relational algebra expression given in Example 5.5.

Safety of expressions

Before we complete this section, we should mention that it is possible for a calculus expression to generate an infinite set. For example:

$$\{S \mid \neg Staff(S)\}$$

would mean the set of all tuples that are not in the **Staff** relation. Such an expression is said to be **unsafe**. To avoid this, we have to add a restriction that all values that appear in the result must be values in the *domain* of the expression E , denoted $dom(E)$. In other words, the domain of E is the set of all values that appear explicitly in E or that appear in one or more relations whose names appear in E . In this example, the domain of the expression is the set of all values appearing in the **Staff** relation.

An expression is *safe* if all values that appear in the result are values from the domain of the expression. The previous expression is not safe, as it will typically include tuples from outside the **Staff** relation (and so outside the domain of the expression). All other examples of tuple relational calculus expressions in this section are safe. Some authors have avoided this problem by using range variables that are defined by a separate **RANGE** statement. The interested reader is referred to Date (2000).

5.2.2 Domain Relational Calculus

In the tuple relational calculus, we use variables that range over tuples in a relation. In the domain relational calculus, we also use variables, but in this case the variables take their values from *domains* of attributes rather than tuples of relations. An expression in the domain relational calculus has the following general form:

$$\{d_1, d_2, \dots, d_n \mid F(d_1, d_2, \dots, d_n)\} \quad m \geq n$$

where $d_1, d_2, \dots, d_n$ represent domain variables and $F(d_1, d_2, \dots, d_n)$ represents a formula composed of atoms, where each atom has one of the following forms:

- $R(d_1, d_2, \dots, d_n)$, where R is a relation of degree n and each d_i is a domain variable.

- $d_i \theta d_j$, where d_i and d_j are domain variables and θ is one of the comparison operators ($<$, $\leq$, $>$, $\geq$, $=$, $\neq$); the domains d_i and d_j must have members that can be compared by θ .
- $d_i \theta c$, where d_i is a domain variable, c is a constant from the domain of d_i , and θ is one of the comparison operators.

We recursively build up formulae from atoms using the following rules:

- An atom is a formula.
- If F_1 and F_2 are formulae, so are their conjunction $F_1 \wedge F_2$, their disjunction $F_1 \vee F_2$, and the negation $\neg F_1$.
- If F is a formula with domain variable X , then $(\exists X)(F)$ and $(\forall X)(F)$ are also formulae.

EXAMPLE 5.15 Domain relational calculus

In the following examples, we use the following shorthand notation:

$(\exists d_1, d_2, \dots, d_n)$ in place of $(\exists d_1), (\exists d_2), \dots, (\exists d_n)$

(a) Find the names of all managers who earn more than £25,000.

$\{fN, IN \mid (\exists sN, posn, sex, DOB, sal, bN) (Staff(sN, fN, IN, posn, sex, DOB, sal, bN) \wedge posn = 'Manager' \wedge sal > 25000))\}$

If we compare this query with the equivalent tuple relational calculus query in Example 5.14(a), we see that each attribute is given a (variable) name. The condition $Staff(sN, fN, \dots, bN)$ ensures that the domain variables are restricted to be attributes of the same tuple. Thus, we can use the formula $posn = 'Manager'$, rather than $Staff.position = 'Manager'$. Also note the difference in the use of the existential quantifier. In the tuple relational calculus, when we write $\exists posn$ for some tuple variable $posn$, we bind the variable to the relation $Staff$ by writing $Staff(posn)$. On the other hand, in the domain relational calculus $posn$ refers to a domain value and remains unconstrained until it appears in the subformula $Staff(sN, fN, IN, posn, sex, DOB, sal, bN)$, when it becomes constrained to the position values that appear in the $Staff$ relation.

For conciseness, in the remaining examples in this section we quantify only those domain variables that actually appear in a condition (in this example, $posn$ and sal).

(b) List the staff who manage properties for rent in Glasgow.

$\{sN, fN, IN, posn, sex, DOB, sal, bN \mid (\exists sN1, ctY) (Staff(sN, fN, IN, posn, sex, DOB, sal, bN) \wedge PropertyForRent(pN, st, ctY, pc, typ, rms, rnt, oN, sN1, bN1) \wedge (sN = sN1) \wedge ctY = 'Glasgow'))\}$

This query can also be written as:

$\{sN, fN, IN, posn, sex, DOB, sal, bN \mid (Staff(sN, fN, IN, posn, sex, DOB, sal, bN) \wedge PropertyForRent(pN, st, 'Glasgow', pc, typ, rms, rnt, oN, sN, bN1))\}$

In this version, the domain variable ctY in $PropertyForRent$ has been replaced with the constant "Glasgow" and the same domain variable sN , which represents the staff number, has been repeated for $Staff$ and $PropertyForRent$.

(c) List the names of staff who currently do not manage any properties for rent.

$\{fN, IN \mid (\exists sN) (Staff(sN, fN, IN, posn, sex, DOB, sal, bN) \wedge \neg (\exists sN1) (PropertyForRent(pN, st, ctY, pc, typ, rms, rnt, oN, sN1, bN1) \wedge (sN = sN1))))\}$

(d) List the names of clients who have viewed a property for rent in Glasgow.

$$\{fN, IN \mid (\exists cN, cN1, pN, pN1, cty) (Client(cN, fN, IN, tel, pT, mR, eM) \\ Viewing(cN1, pN1, dt, cmt) \text{ PropertyForRent}(pN, st, cty, pc, typ, rms, rnt, oN, sN, bN) \\ (cN = cN1) (pN = pN1) \text{ cty} = \text{'Glasgow'})\}$$

(e) List all cities where there is either a branch office or a property for rent.

$$\{cty \mid (Branch(bN, st, cty, pc) \\ \text{PropertyForRent}(pN, st1, cty, pc1, typ, rms, rnt, oN, sN, bN1))\}$$

(f) List all the cities where there is a branch office but no properties for rent.

$$\{cty \mid (Branch(bN, st, cty, pc) \\ (\neg(\exists cty1)(\text{PropertyForRent}(pN, st1, cty1, pc1, typ, rms, rnt, oN, sN, bN1) \text{ cty} = cty1))))\}$$

(g) List all the cities where there is both a branch office and at least one property for rent.

$$\{cty \mid (Branch(bN, st, cty, pc) \\ (\exists cty1) (\text{PropertyForRent}(pN, st1, cty1, pc1, typ, rms, rnt, oN, sN, bN1) \text{ cty} = cty1)))\}$$

These queries are **safe**. When the domain relational calculus is restricted to safe expressions, it is equivalent to the tuple relational calculus restricted to safe expressions, which in turn is equivalent to the relational algebra. This means that for every relational algebra expression there is an equivalent expression in the relational calculus, and for every tuple or domain relational calculus expression there is an equivalent relational algebra expression.

5.3 Other Languages

Although the relational calculus is hard to understand and use, it was recognized that its nonprocedural property is exceedingly desirable, and this resulted in a search for other easy-to-use nonprocedural techniques. This led to another two categories of relational languages: transform-oriented and graphical.

Transform-oriented languages are a class of nonprocedural languages that use relations to transform input data into required outputs. These languages provide easy-to-use structures for expressing what is desired in terms of what is known. SQUARE (Boyce *et al.*, 1975), SEQUEL (Chamberlin *et al.*, 1976), and SEQUEL's offspring, SQL, are all transform-oriented languages. We discuss SQL in Chapters 6–9.

Graphical languages provide the user with a picture or illustration of the structure of the relation. The user fills in an example of what is wanted and the system returns the required data in that format. QBE (Query-By-Example) is an example of a graphical language (Zloof, 1977). We demonstrate the capabilities of QBE in Appendix M.

Another category is **fourth-generation languages (4GLs)**, which allow a complete customized application to be created using a limited set of commands in a user-friendly, often menu-driven environment (see Section 2.2). Some systems accept a form of *natural language*, a restricted version of natural English, sometimes called a **fifth-generation language (5GL)**, although this development is still at an early stage.

Chapter Summary

- The **relational algebra** is a (high-level) procedural language: it can be used to tell the DBMS how to build a new relation from one or more relations in the database. The **relational calculus** is a nonprocedural language: it can be used to formulate the definition of a relation in terms of one or more database relations. However, formally the relational algebra and relational calculus are equivalent to one another: for every expression in the algebra, there is an equivalent expression in the calculus (and vice versa).
- The relational calculus is used to measure the selective power of relational languages. A language that can be used to produce any relation that can be derived using the relational calculus is said to be **relationally complete**. Most relational query languages are relationally complete but have more expressive power than the relational algebra or relational calculus because of additional operations such as calculated, summary, and ordering functions.
- The five fundamental operations in relational algebra—Selection, Projection, Cartesian product, Union, and Set difference—perform most of the data retrieval operations that we are interested in. In addition, there are also the Join, Intersection, and Division operations, which can be expressed in terms of the five basic operations.
- The **relational calculus** is a formal nonprocedural language that uses predicates. There are two forms of the relational calculus: tuple relational calculus and domain relational calculus.
- In the **tuple relational calculus**, we are interested in finding tuples for which a predicate is true. A tuple variable is a variable that “ranges over” a named relation: that is, a variable whose only permitted values are tuples of the relation.
- In the **domain relational calculus**, domain variables take their values from domains of attributes rather than tuples of relations.
- The relational algebra is logically equivalent to a safe subset of the relational calculus (and vice versa).
- Relational data manipulation languages are sometimes classified as **procedural** or **nonprocedural**, **transform-oriented**, **graphical**, **fourth-generation**, or **fifth-generation**.

Review Questions

- 5.1 What is the difference between a procedural and a nonprocedural language? How would you classify the relational algebra and relational calculus?
- 5.2 Explain the following terms:
 - (a) tuple relational calculus
 - (b) domain relational calculus
- 5.3 Define the five basic relational algebra operations. Define the Join, Intersection, and Division operations in terms of these five basic operations.
- 5.4 Discuss the differences between the five Join operations: Theta join, Equijoin, Natural join, Outer join, and Semijoin. Give examples to illustrate your answer.
- 5.5 There are different types of join operations that can be used to retrieve data, based on different relations. Describe the relation between theta and equal join.
- 5.6 Define the structure of a (well-formed) formula in both the tuple relational calculus and domain relational calculus.
- 5.7 What is the difference between existential and universal quantifiers in relational calculus? Give examples to elaborate how the two are applied in a statement.

Exercises

For the following exercises, use the *Hotel* schema defined at the start of the Exercises at the end of Chapter 4.

5.8 Describe the relations that would be produced by the following relational algebra operations:

- $\Pi_{\text{hotelNo}}(\sigma_{\text{price} > 50}(\text{Room}))$
- $\sigma_{\text{Hotel.hotelNo} = \text{Room.hotelNo}}(\text{Hotel} \times \text{Room})$
- $\Pi_{\text{hotelName}}(\text{Hotel} \bowtie_{\text{Hotel.hotelNo} = \text{Room.hotelNo}}(\sigma_{\text{price} > 50}(\text{Room})))$
- $\text{Guest} \rhd (\sigma_{\text{dateTo} \geq \text{'1-Jan-2007'}}(\text{Booking}))$
- $\text{Hotel} \bowtie_{\text{Hotel.hotelNo} = \text{Room.hotelNo}}(\sigma_{\text{price} > 50}(\text{Room}))$
- $\Pi_{\text{guestName, hotelNo}}(\text{Booking} \bowtie_{\text{Booking.guestNo} = \text{Guest.guestNo}} \text{Guest}) \div \Pi_{\text{hotelNo}}(\sigma_{\text{city} = \text{'London'}}(\text{Hotel}))$

5.9 Provide the equivalent tuple relational calculus and domain relational calculus expressions for each of the relational algebra queries given in Exercise 5.8.

5.10 Describe the relations that would be produced by the following tuple relational calculus expressions:

- $\{H.\text{hotelName} \mid \text{Hotel}(H) \wedge H.\text{city} = \text{'London'}\}$
- $\{H.\text{hotelName} \mid \text{Hotel}(H) \wedge (\exists R) (\text{Room}(R) \wedge H.\text{hotelNo} = R.\text{hotelNo} \wedge R.\text{price} > 50)\}$
- $\{H.\text{hotelName} \mid \text{Hotel}(H) \wedge (\exists B) (\exists G) (\text{Booking}(B) \wedge \text{Guest}(G) \wedge H.\text{hotelNo} = B.\text{hotelNo} \wedge B.\text{guestNo} = G.\text{guestNo} \wedge G.\text{guestName} = \text{'John Smith'})\}$
- $\{H.\text{hotelName}, G.\text{guestName}, B1.\text{dateFrom}, B2.\text{dateFrom} \mid \text{Hotel}(H) \wedge \text{Guest}(G) \wedge \text{Booking}(B1) \wedge \text{Booking}(B2) \wedge H.\text{hotelNo} = B1.\text{hotelNo} \wedge G.\text{guestNo} = B1.\text{guestNo} \wedge B2.\text{hotelNo} = B1.\text{hotelNo} \wedge B2.\text{guestNo} = B1.\text{guestNo} \wedge B2.\text{dateFrom} \neq B1.\text{dateFrom}\}$

5.11 Provide the equivalent domain relational calculus and relational algebra expressions for each of the tuple relational calculus expressions given in Exercise 5.10.

5.12 Generate the relational algebra, tuple relational calculus, and domain relational calculus expressions for the following queries:

- List all hotels.
- List all single rooms with a price below £20 per night.
- List the names and cities of all guests.
- List the price and type of all rooms at the Grosvenor Hotel.
- List all guests currently staying at the Grosvenor Hotel.
- List the details of all rooms at the Grosvenor Hotel, including the name of the guest staying in the room, if the room is occupied.
- List the guest details (**guestNo**, **guestName**, and **guestAddress**) of all guests staying at the Grosvenor Hotel.

5.13 Using relational algebra, produce a report of all employees from the IT and planning departments who are born after 1990.

The following tables form part of a database held in an RDBMS:

Employee	(<u>empNo</u> , fName, lName, address, DOB, sex, position, deptNo)
Department	(<u>deptNo</u> , deptName, mgrEmpNo)
Project	(<u>projNo</u> , projName, deptNo)
WorksOn	(<u>empNo</u> , <u>projNo</u> , <u>dateWorked</u> , hoursWorked)

where	Employee	contains employee details and empNo is the key.
	Department	contains department details and deptNo is the key. mgrEmpNo identifies the employee who is the manager of the department. There is only one manager for each department.
	Project	contains details of the projects in each department and the key is projNo (no two departments can run the same project).
and	WorksOn	contains details of the hours worked by employees on each project, and empNo / projNo / dateWorked form the key.

Formulate the following queries in relational algebra, tuple relational calculus, and domain relational calculus.

- 5.14 List all employees.
- 5.15 List all the details of employees who are female and born after 1990.
- 5.16 List all employees who are not managers and are paid more than \$1500.
- 5.17 Produce a list of the names and addresses of all employees who work for the IT department.
- 5.18 Produce a list of the names of all employees who work on the SCCS project.
- 5.19 Produce a complete list of all managers who are due to retire this year, in alphabetical order of surname.

Formulate the following queries in relational algebra.

- 5.20 Find out how many managers are female.
- 5.21 Produce a report of all projects under the IT department.
- 5.22 Using the union operator, retrieve the list of employees who are neither managers nor supervisors. Attributes to be retrieved are first name, last name, position, sex and department number.
- 5.23 List the total number of employees in each department for those departments with more than 10 employees. Create an appropriate heading for the columns of the results table.

The following tables form part of a Library database held in an RDBMS:

Book	(ISBN, title, edition, year)
BookCopy	(copyNo, ISBN, available)
Borrower	(borrowerNo, borrowerName, borrowerAddress)
BookLoan	(copyNo, dateOut, dateDue, borrowerNo)
where	
Book	contains details of book titles in the library and the ISBN is the key.
BookCopy	contains details of the individual copies of books in the library and copyNo is the key. ISBN is a foreign key identifying the book title.
Borrower	contains details of library members who can borrow books and borrowerNo is the key.
BookLoan	contains details of the book copies that are borrowed by library members and copyNo/dateOut forms the key. borrowerNo is a foreign key identifying the borrower.

Formulate the following queries in relational algebra, tuple relational calculus, and domain relational calculus.

- 5.24 List all book titles.
- 5.25 List all borrower details.
- 5.26 List all book titles published in the year 2012.
- 5.27 List all copies of book titles that are available for borrowing.
- 5.28 List all copies of the book title *Lord of the Rings* that are available for borrowing.
- 5.29 List the names of borrowers who currently have the book title *Lord of the Rings* on loan.
- 5.30 List the names of borrowers with overdue books.

Formulate the following queries in relational algebra.

- 5.31 How many copies of ISBN "0-321-52306-7" are there?
- 5.32 How many copies of ISBN "0-321-52306-7" are currently available?
- 5.33 How many times has the book title with ISBN "0-321-52306-7" been borrowed?
- 5.34 Produce a report of book titles that have been borrowed by "Peter Bloomfield."

- 5.35 For each book title with more than three copies, list the names of library members who have borrowed them.
- 5.36 Produce a report with the details of borrowers who currently have books overdue.
- 5.37 Produce a report detailing how many times each book title has been borrowed.
- 5.38 Analyze the RDBMSs that you are currently using. What types of relational language does the system provide?
For each of the languages provided, what are the equivalent operations for the eight relational algebra operations defined in Section 5.1?

Chapter Objectives

In this chapter you will learn:

- The purpose and importance of the Structured Query Language (SQL).
- The history and development of SQL.
- How to write an SQL command.
- How to retrieve data from the database using the SELECT statement.
- How to build SQL statements that:
 - use the WHERE clause to retrieve rows that satisfy various conditions;
 - sort query results using ORDER BY;
 - use the aggregate functions of SQL;
 - group data using GROUP BY;
 - use subqueries;
 - join tables together;
 - perform set operations (UNION, INTERSECT, EXCEPT).
- How to perform database updates using INSERT, UPDATE, and DELETE.

In Chapters 4 and 5 we described the relational data model and relational languages in some detail. A particular language that has emerged from the development of the relational model is the Structured Query Language, or SQL as it is commonly called. Over the last few years, SQL has become the standard relational database language. In 1986, a standard for SQL was defined by the American National Standards Institute (ANSI) and was subsequently adopted in 1987 as an international standard by the International Organization for Standardization (ISO, 1987). More than one hundred DBMSs now support SQL, running on various hardware platforms from PCs to mainframes.

Owing to the current importance of SQL, we devote four chapters and an appendix of this book to examining the language in detail, providing a comprehensive treatment for both technical and nontechnical users including programmers, database professionals, and managers. In these chapters we largely

concentrate on the ISO definition of the SQL language. However, owing to the complexity of this standard, we do not attempt to cover all parts of the language. In this chapter, we focus on the data manipulation statements of the language.

Structure of this Chapter In Section 6.1 we introduce SQL and discuss why the language is so important to database applications. In Section 6.2 we introduce the notation used in this book to specify the structure of an SQL statement. In Section 6.3 we discuss how to retrieve data from relations using SQL, and how to insert, update, and delete data from relations.

Looking ahead, in Chapter 7 we examine other features of the language, including data definition, views, transactions, and access control. In Chapter 8 we examine more advanced features of the language, including triggers and stored procedures. In Chapter 9 we examine in some detail the features that have been added to the SQL specification to support object-oriented data management. In Appendix I we discuss how SQL can be embedded in high-level programming languages to access constructs that were not available in SQL until very recently. The two formal languages, relational algebra and relational calculus, that we covered in Chapter 5 provide a foundation for a large part of the SQL standard and it may be useful to refer to this chapter occasionally to see the similarities. However, our presentation of SQL is mainly independent of these languages for those readers who have omitted Chapter 5. The examples in this chapter use the *DreamHome* rental database instance shown in Figure 4.3.



6.1 Introduction to SQL

In this section we outline the objectives of SQL, provide a short history of the language, and discuss why the language is so important to database applications.

6.1.1 Objectives of SQL

Ideally, a database language should allow a user to:

- create the database and relation structures;
- perform basic data management tasks, such as the insertion, modification, and deletion of data from the relations;
- perform both simple and complex queries.

A database language must perform these tasks with minimal user effort, and its command structure and syntax must be relatively easy to learn. Finally, the language must be portable; that is, it must conform to some recognized standard so that we can use the same command structure and syntax when we move from one DBMS to another. SQL is intended to satisfy these requirements.

SQL is an example of a **transform-oriented language**, or a language designed to use relations to transform inputs into required outputs. As a language, the ISO SQL standard has two major components:

- a Data Definition Language (DDL) for defining the database structure and controlling access to the data;
- a Data Manipulation Language (DML) for retrieving and updating data.

Until the 1999 release of the standard, known as SQL:1999 or SQL3, SQL contained only these definitional and manipulative commands; it did not contain flow of control commands, such as IF . . . THEN . . . ELSE, GO TO, or DO . . . WHILE. These commands had to be implemented using a programming or job-control language, or interactively by the decisions of the user. Owing to this lack of *computational completeness*, SQL can be used in two ways. The first way is to use SQL *interactively* by entering the statements at a terminal. The second way is to *embed* SQL statements in a procedural language, as we discuss in Appendix I. We also discuss the latest release of the standard, SQL:2011 in Chapter 9.

SQL is a relatively easy language to learn:

- It is a nonprocedural language; you specify *what* information you require, rather than *how* to get it. In other words, SQL does not require you to specify the access methods to the data.
- Like most modern languages, SQL is essentially free-format, which means that parts of statements do not have to be typed at particular locations on the screen.
- The command structure consists of standard English words such as CREATE TABLE, INSERT, SELECT. For example:
 - **CREATE TABLE** Staff (staffNo **VARCHAR**(5), lName **VARCHAR**(15), salary **DECIMAL**(7,2));
 - **INSERT INTO** Staff **VALUES** ('SG16', 'Brown', 8300);
 - **SELECT** staffNo, lName, salary
FROM Staff
WHERE salary > 10000;
- SQL can be used by a range of users including database administrators (DBA), management personnel, application developers, and many other types of end-user.

An international standard now exists for the SQL language making it both the formal and de facto standard language for defining and manipulating relational databases (ISO, 1992, 2011a).

6.1.2 History of SQL

As stated in Chapter 4, the history of the relational model (and indirectly SQL) started with the publication of the seminal paper by E. F. Codd, written during his work at IBM's Research Laboratory in San José (Codd, 1970). In 1974, D. Chamberlin, also from the IBM San José Laboratory, defined a language called the Structured English Query Language, or SEQUEL. A revised version, SEQUEL/2, was defined in 1976, but the name was subsequently changed to SQL for legal reasons (Chamberlin and Boyce, 1974; Chamberlin *et al.*, 1976). Today, many people still pronounce SQL as "See-Quel," though the official pronunciation is "S-Q-L."

IBM produced a prototype DBMS based on SEQUEL/2 called System R (Astrahan *et al.*, 1976). The purpose of this prototype was to validate the feasibility of the relational model. Besides its other successes, one of the most important results that has been attributed to this project was the development of SQL. However, the roots of

SQL are in the language SQUARE (Specifying Queries As Relational Expressions), which predates the System R project. SQUARE was designed as a research language to implement relational algebra with English sentences (Boyce *et al.*, 1975).

In the late 1970s, the database system Oracle was produced by what is now called the Oracle Corporation, and was probably the first commercial implementation of a relational DBMS based on SQL. INGRES followed shortly afterwards, with a query language called QUEL, which although more “structured” than SQL, was less English-like. When SQL emerged as the standard database language for relational systems, INGRES was converted to an SQL-based DBMS. IBM produced its first commercial RDBMS, called SQL/DS, for the DOS/VSE and VM/CMS environments in 1981 and 1982, respectively, and subsequently as DB2 for the MVS environment in 1983.

In 1982, ANSI began work on a Relational Database Language (RDL) based on a concept paper from IBM. ISO joined in this work in 1983, and together they defined a standard for SQL. (The name RDL was dropped in 1984, and the draft standard reverted to a form that was more like the existing implementations of SQL.)

The initial ISO standard published in 1987 attracted a considerable degree of criticism. Date, an influential researcher in this area, claimed that important features such as referential integrity constraints and certain relational operators had been omitted. He also pointed out that the language was extremely redundant; in other words, there was more than one way to write the same query (Date, 1986, 1987a, 1990). Much of the criticism was valid, and had been recognized by the standards bodies before the standard was published. It was decided, however, that it was more important to release a standard as early as possible to establish a common base from which the language and the implementations could develop than to wait until all the features that people felt should be present could be defined and agreed.

In 1989, ISO published an addendum that defined an “Integrity Enhancement Feature” (ISO, 1989). In 1992, the first major revision to the ISO standard occurred, sometimes referred to as SQL2 or SQL-92 (ISO, 1992). Although some features had been defined in the standard for the first time, many of these had already been implemented in part or in a similar form in one or more of the many SQL implementations. It was not until 1999 that the next release of the standard, commonly referred to as SQL:1999 (ISO, 1999a), was formalized. This release contains additional features to support object-oriented data management, which we examine in Chapter 9. There have been further releases of the standard in late 2003 (SQL:2003), in summer 2008 (SQL:2008), and in late 2011 (SQL:2011).

Features that are provided on top of the standard by the vendors are called **extensions**. For example, the standard specifies six different data types for data in an SQL database. Many implementations supplement this list with a variety of extensions. Each implementation of SQL is called a **dialect**. No two dialects are exactly alike, and currently no dialect exactly matches the ISO standard. Moreover, as database vendors introduce new functionality, they are expanding their SQL dialects and moving them even further apart. However, the central core of the SQL language is showing signs of becoming more standardized. In fact, SQL now has a set of features called **Core SQL** that a vendor must implement to claim **conformance** with the SQL standard. Many of the remaining features are divided into packages; for example, there are **packages** for object features and OLAP (OnLine Analytical Processing).

Although SQL was originally an IBM concept, its importance soon motivated other vendors to create their own implementations. Today there are literally hundreds of SQL-based products available, with new products being introduced regularly.

6.1.3 Importance of SQL

SQL is the first and, so far, only standard database language to gain wide acceptance. The only other standard database language, the Network Database Language (NDL), based on the CODASYL network model, has few followers. Nearly every major current vendor provides database products based on SQL or with an SQL interface, and most are represented on at least one of the standard-making bodies. There is a huge investment in the SQL language both by vendors and by users. It has become part of application architectures such as IBM's Systems Application Architecture (SAA) and is the strategic choice of many large and influential organizations, for example, the Open Group consortium for UNIX standards. SQL has also become a Federal Information Processing Standard (FIPS) to which conformance is required for all sales of DBMSs to the U.S. government. The SQL Access Group, a consortium of vendors, defined a set of enhancements to SQL that would support interoperability across disparate systems.

SQL is used in other standards and even influences the development of other standards as a definitional tool. Examples include ISO's Information Resource Dictionary System (IRDS) standard and Remote Data Access (RDA) standard. The development of the language is supported by considerable academic interest, providing both a theoretical basis for the language and the techniques needed to implement it successfully. This is especially true in query optimization, distribution of data, and security. There are now specialized implementations of SQL that are directed at new markets, such as OnLine Analytical Processing (OLAP).

6.1.4 Terminology

The ISO SQL standard does not use the formal terms of relations, attributes, and tuples, instead using the terms tables, columns, and rows. In our presentation of SQL we mostly use the ISO terminology. It should also be noted that SQL does not adhere strictly to the definition of the relational model described in Chapter 4. For example, SQL allows the table produced as the result of the SELECT statement to contain duplicate rows, it imposes an ordering on the columns, and it allows the user to order the rows of a result table.

6.2 Writing SQL Commands

In this section we briefly describe the structure of an SQL statement and the notation we use to define the format of the various SQL constructs. An SQL statement consists of **reserved words** and **user-defined words**. Reserved words are a fixed part of the SQL language and have a fixed meaning. They must be spelled *exactly* as required and cannot be split across lines. User-defined words are made up by the user (according to certain syntax rules) and represent the names of various database objects such as tables, columns, views, indexes, and so on. The words in a statement are also built according to a set of syntax rules. Although the standard

does not require it, many dialects of SQL require the use of a statement terminator to mark the end of each SQL statement (usually the semicolon “;” is used).

Most components of an SQL statement are **case-insensitive**, which means that letters can be typed in either upper- or lowercase. The one important exception to this rule is that literal character data must be typed *exactly* as it appears in the database. For example, if we store a person’s surname as “SMITH” and then search for it using the string “Smith,” the row will not be found.

Although SQL is free-format, an SQL statement or set of statements is more readable if indentation and lineation are used. For example:

- each clause in a statement should begin on a new line;
- the beginning of each clause should line up with the beginning of other clauses;
- if a clause has several parts, they should each appear on a separate line and be indented under the start of the clause to show the relationship.

Throughout this and the next three chapters, we use the following extended form of the Backus Naur Form (BNF) notation to define SQL statements:

- uppercase letters are used to represent reserved words and must be spelled exactly as shown;
- lowercase letters are used to represent user-defined words;
- a vertical bar (|) indicates a **choice** among alternatives; for example, a | b | c;
- curly braces indicate a **required element**; for example, {a};
- square brackets indicate an **optional element**; for example, [a];
- an ellipsis (. . .) is used to indicate **optional repetition** of an item zero or more times.

For example:

{a|b} (, c . . .)

means either a or b followed by zero or more repetitions of c separated by commas.

In practice, the DDL statements are used to create the database structure (that is, the tables) and the access mechanisms (that is, what each user can legally access), and then the DML statements are used to populate and query the tables. However, in this chapter we present the DML before the DDL statements to reflect the importance of DML statements to the general user. We discuss the main DDL statements in the next chapter.

6.3 Data Manipulation

This section looks at the following SQL DML statements:

- SELECT – to query data in the database
- INSERT – to insert data into a table
- UPDATE – to update data in a table
- DELETE – to delete data from a table

Owing to the complexity of the SELECT statement and the relative simplicity of the other DML statements, we devote most of this section to the SELECT

statement and its various formats. We begin by considering simple queries, and successively add more complexity to show how more complicated queries that use sorting, grouping, aggregates, and also queries on multiple tables can be generated. We end the chapter by considering the INSERT, UPDATE, and DELETE statements.

We illustrate the SQL statements using the instance of the *DreamHome* case study shown in Figure 4.3, which consists of the following tables:



Branch	(<u>branchNo</u> , street, city, postcode)
Staff	(<u>staffNo</u> , fName, lName, position, sex, DOB, salary, branchNo)
PropertyForRent	(<u>propertyNo</u> , street, city, postcode, type, rooms, rent, ownerNo, staffNo, branchNo)
Client	(<u>clientNo</u> , fName, lName, telNo, prefType, maxRent, eMail)
PrivateOwner	(<u>ownerNo</u> , fName, lName, address, telNo, eMail, password)
Viewing	(<u>clientNo</u> , propertyNo, viewDate, comment)

Literals

Before we discuss the SQL DML statements, it is necessary to understand the concept of **literals**. Literals are **constants** that are used in SQL statements. There are different forms of literals for every data type supported by SQL (see Section 7.1.1). However, for simplicity, we can distinguish between literals that are enclosed in single quotes and those that are not. All nonnumeric data values must be enclosed in single quotes; all numeric data values must **not** be enclosed in single quotes. For example, we could use literals to insert data into a table:

```
INSERT INTO PropertyForRent(propertyNo, street, city, postcode, type, rooms, rent,
                             ownerNo, staffNo, branchNo)
VALUES ('PA14', '16 Holhead', 'Aberdeen', 'AB7 5SU', 'House', 6, 650.00,
       'CO46', 'SA9', 'B007');
```

The value in column *rooms* is an integer literal and the value in column *rent* is a decimal number literal; they are not enclosed in single quotes. All other columns are character strings and are enclosed in single quotes.

6.3.1 Simple Queries

The purpose of the SELECT statement is to retrieve and display data from one or more database tables. It is an extremely powerful command, capable of performing the equivalent of the relational algebra's *Selection*, *Projection*, and *Join* operations in a single statement (see Section 5.1). SELECT is the most frequently used SQL command and has the following general form:

```
SELECT      [DISTINCT | ALL] { * | [columnExpression [AS newName]] [, . . .] }
FROM        TableName [alias] [, . . .]
[WHERE      condition]
[GROUP BY  columnList] [HAVING condition]
[ORDER BY  columnList]
```

columnExpression represents a column name or an expression, *TableName* is the name of an existing database table or view that you have access to, and *alias* is an optional abbreviation for *TableName*. The sequence of processing in a SELECT statement is:

- FROM specifies the table or tables to be used
- WHERE filters the rows subject to some condition
- GROUP BY forms groups of rows with the same column value
- HAVING filters the groups subject to some condition
- SELECT specifies which columns are to appear in the output
- ORDER BY specifies the order of the output

The order of the clauses in the SELECT statement *cannot* be changed. The only two mandatory clauses are the first two: SELECT and FROM; the remainder are optional. The SELECT operation is **closed**: the result of a query on a table is another table (see Section 5.1). There are many variations of this statement, as we now illustrate.

Retrieve all rows

EXAMPLE 6.1 Retrieve all columns, all rows

List full details of all staff.

Because there are no restrictions specified in this query, the WHERE clause is unnecessary and all columns are required. We write this query as:

```
SELECT staffNo, fName, lName, position, sex, DOB, salary, branchNo
FROM Staff;
```

Because many SQL retrievals require all columns of a table, there is a quick way of expressing “all columns” in SQL, using an asterisk (*) in place of the column names. The following statement is an equivalent and shorter way of expressing this query:

```
SELECT *
FROM Staff;
```

The result table in either case is shown in Table 6.1.

TABLE 6.1 Result table for Example 6.1.

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000.00	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000.00	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000.00	B005

EXAMPLE 6.2 Retrieve specific columns, all rows

Produce a list of salaries for all staff, showing only the staff number, the first and last names, and the salary details.

```
SELECT staffNo, fName, lName, salary
FROM Staff;
```

In this example a new table is created from **Staff** containing only the designated columns **staffNo**, **fName**, **lName**, and **salary**, in the specified order. The result of this operation is shown in Table 6.2. Note that, unless specified, the rows in the result table may not be sorted. Some DBMSs do sort the result table based on one or more columns (for example, Microsoft Office Access would sort this result table based on the primary key **staffNo**). We describe how to sort the rows of a result table in the next section.

TABLE 6.2 Result table for Example 6.2.

staffNo	fName	lName	salary
SL21	John	White	30000.00
SG37	Ann	Beech	12000.00
SG14	David	Ford	18000.00
SA9	Mary	Howe	9000.00
SG5	Susan	Brand	24000.00
SL41	Julie	Lee	9000.00

EXAMPLE 6.3 Use of DISTINCT

List the property numbers of all properties that have been viewed.

```
SELECT propertyNo
FROM Viewing;
```

The result table is shown in Table 6.3(a). Notice that there are several duplicates, because unlike the relational algebra Projection operation (see Section 5.1.1), **SELECT** does not

TABLE 6.3(a) Result table for Example 6.3 with duplicates.

propertyNo
PA14
PG4
PG4
PA14
PG36

eliminate duplicates when it projects over one or more columns. To eliminate the duplicates, we use the `DISTINCT` keyword. Rewriting the query as:

```
SELECT DISTINCT propertyNo
FROM Viewing;
```

we get the result table shown in Table 6.3(b) with the duplicates eliminated.

TABLE 6.3(b) Result table for Example 6.3 with duplicates eliminated.

propertyNo
PA14
PG4
PG36

EXAMPLE 6.4 Calculated fields

Produce a list of monthly salaries for all staff, showing the staff number, the first and last names, and the salary details.

```
SELECT staffNo, fName, lName, salary/12
FROM Staff;
```

This query is almost identical to Example 6.2, with the exception that monthly salaries are required. In this case, the desired result can be obtained by simply dividing the salary by 12, giving the result table shown in Table 6.4.

This is an example of the use of a **calculated field** (sometimes called a **computed** or **derived field**). In general, to use a calculated field, you specify an SQL expression in the `SELECT` list. An SQL expression can involve addition, subtraction, multiplication, and division, and parentheses can be used to build complex expressions. More than one table column can be used in a calculated column; however, the columns referenced in an arithmetic expression must have a numeric type.

The fourth column of this result table has been output as *col4*. Normally, a column in the result table takes its name from the corresponding column of the database table from which it has been retrieved. However, in this case, SQL does not know how to label the column. Some dialects give the column a name corresponding to its position

TABLE 6.4 Result table for Example 6.4.

staffNo	fName	lName	col4
SL21	John	White	2500.00
SG37	Ann	Beech	1000.00
SG14	David	Ford	1500.00
SA9	Mary	Howe	750.00
SG5	Susan	Brand	2000.00
SL41	Julie	Lee	750.00

in the table (for example, col4); some may leave the column name blank or use the expression entered in the SELECT list. The ISO standard allows the column to be named using an AS clause. In the previous example, we could have written:

```
SELECT staffNo, fName, lName, salary/12 AS monthlySalary
FROM Staff;
```

In this case, the column heading of the result table would be monthlySalary rather than col4.

Row selection (WHERE clause)

The previous examples show the use of the SELECT statement to retrieve all rows from a table. However, we often need to restrict the rows that are retrieved. This can be achieved with the WHERE clause, which consists of the keyword WHERE followed by a search condition that specifies the rows to be retrieved. The five basic search conditions (*or predicates*, using the ISO terminology) are as follows:

- *Comparison* Compare the value of one expression to the value of another expression.
- *Range* Test whether the value of an expression falls within a specified range of values.
- *Set membership* Test whether the value of an expression equals one of a set of values.
- *Pattern match* Test whether a string matches a specified pattern.
- *Null* Test whether a column has a null (unknown) value.

The WHERE clause is equivalent to the relational algebra Selection operation discussed in Section 5.1.1. We now present examples of each of these types of search conditions.

EXAMPLE 6.5 Comparison search condition

List all staff with a salary greater than £10,000.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > 10000;
```

Here, the table is Staff and the predicate is salary > 10000. The selection creates a new table containing only those Staff rows with a salary greater than £10,000. The result of this operation is shown in Table 6.5.

TABLE 6.5 Result table for Example 6.5.

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG37	Ann	Beech	Assistant	12000.00
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00

In SQL, the following simple comparison operators are available:

=	equals		
<>	is not equal to (ISO standard)	!=	is not equal to (allowed in some dialects)
<	is less than	<=	is less than or equal to
>	is greater than	>=	is greater than or equal to

More complex predicates can be generated using the logical operators **AND**, **OR**, and **NOT**, with parentheses (if needed or desired) to show the order of evaluation. The rules for evaluating a conditional expression are:

- an expression is evaluated left to right;
- subexpressions in brackets are evaluated first;
- NOTs are evaluated before ANDs and ORs;
- ANDs are evaluated before ORs.

The use of parentheses is always recommended, in order to remove any possible ambiguities.

EXAMPLE 6.6 Compound comparison search condition

List the addresses of all branch offices in London or Glasgow.

```
SELECT *
FROM Branch
WHERE city = 'London' OR city = 'Glasgow';
```

In this example the logical operator OR is used in the WHERE clause to find the branches in London (city = 'London') or in Glasgow (city = 'Glasgow'). The result table is shown in Table 6.6.

TABLE 6.6 Result table for Example 6.6.

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B003	163 Main St	Glasgow	G11 9QX
B002	56 Clover Dr	London	NW10 6EU

EXAMPLE 6.7 Range search condition (BETWEEN/NOT BETWEEN)

List all staff with a salary between £20,000 and £30,000.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary BETWEEN 20000 AND 30000;
```

The BETWEEN test includes the endpoints of the range, so any members of staff with a salary of £20,000 or £30,000 would be included in the result. The result table is shown in Table 6.7.

TABLE 6.7 Result table for Example 6.7.

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG5	Susan	Brand	Manager	24000.00

There is also a negated version of the range test (NOT BETWEEN) that checks for values outside the range. The BETWEEN test does not add much to the expressive power of SQL, because it can be expressed equally well using two comparison tests. We could have expressed the previous query as:

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary >= 20000 AND salary <= 30000;
```

However, the BETWEEN test is a simpler way to express a search condition when considering a range of values.

EXAMPLE 6.8 Set membership search condition (IN/NOT IN)

List all managers and supervisors.

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE position IN ('Manager', 'Supervisor');
```

The set membership test (IN) tests whether a data value matches one of a list of values, in this case either 'Manager' or 'Supervisor'. The result table is shown in Table 6.8.

TABLE 6.8 Result table for Example 6.8.

staffNo	fName	lName	position
SL21	John	White	Manager
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

There is a negated version (NOT IN) that can be used to check for data values that do not lie in a specific list of values. Like BETWEEN, the IN test does not add much to the expressive power of SQL. We could have expressed the previous query as:

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE position = 'Manager' OR position = 'Supervisor';
```

However, the IN test provides a more efficient way of expressing the search condition, particularly if the set contains many values.

EXAMPLE 6.9 Pattern match search condition (LIKE/NOT LIKE)

Find all owners with the string ‘Glasgow’ in their address.

For this query, we must search for the string ‘Glasgow’ appearing somewhere within the address column of the PrivateOwner table. SQL has two special pattern-matching symbols:

- The % percent character represents any sequence of zero or more characters (*wildcard*).
- The _ underscore character represents any single character.

All other characters in the pattern represent themselves. For example:

- address LIKE ‘H%’ means the first character must be *H*, but the rest of the string can be anything.
- address LIKE ‘H____’ means that there must be exactly four characters in the string, the first of which must be an *H*.
- address LIKE ‘%e’ means any sequence of characters, of length at least 1, with the last character an *e*.
- address LIKE ‘%Glasgow%’ means a sequence of characters of any length containing *Glasgow*.
- address NOT LIKE ‘H%’ means the first character cannot be an *H*.

If the search string can include the pattern-matching character itself, we can use an **escape character** to represent the pattern-matching character. For example, to check for the string ‘15%’, we can use the predicate:

LIKE ‘15#%’ ESCAPE ‘#’

Using the pattern-matching search condition of SQL, we can find all owners with the string “Glasgow” in their address using the following query, producing the result table shown in Table 6.9:

```
SELECT ownerNo, fName, lName, address, telNo
FROM PrivateOwner
WHERE address LIKE '%Glasgow%';
```

Note that some RDBMSs, such as Microsoft Office Access, use the wildcard characters * and ? instead of % and _ .

TABLE 6.9 Result table for Example 6.9.

ownerNo	fName	lName	address	telNo
CO87	Carol	Farrel	6 Achray St, Glasgow G32 9DX	0141-357-7419
CO40	Tina	Murphy	63 Well St, Glasgow G42	0141-943-1728
CO93	Tony	Shaw	12 Park Pl, Glasgow G4 0QR	0141-225-7025

EXAMPLE 6.10 NULL search condition (IS NULL/IS NOT NULL)

List the details of all viewings on property PG4 where a comment has not been supplied.

From the Viewing table of Figure 4.3, we can see that there are two viewings for property PG4: one with a comment, the other without a comment. In this simple example,

you may think that the latter row could be accessed by using one of the search conditions:

```
(propertyNo = 'PG4' AND comment = ' ')
or
(propertyNo = 'PG4' AND comment < > 'too remote')
```

However, neither of these conditions would work. A null comment is considered to have an unknown value, so we cannot test whether it is equal or not equal to another string. If we tried to execute the SELECT statement using either of these compound conditions, we would get an empty result table. Instead, we have to test for null explicitly using the special keyword IS NULL:

```
SELECT clientNo, viewDate
FROM Viewing
WHERE propertyNo = 'PG4' AND comment IS NULL;
```

The result table is shown in Table 6.10. The negated version (IS NOT NULL) can be used to test for values that are not null.

TABLE 6.10 Result table for Example 6.10.

clientNo	viewDate
CR56	26-May-13

6.3.2 Sorting Results (ORDER BY Clause)

In general, the rows of an SQL query result table are not arranged in any particular order (although some DBMSs may use a default ordering based, for example, on a primary key). However, we can ensure the results of a query are sorted using the ORDER BY clause in the SELECT statement. The ORDER BY clause consists of a list of **column identifiers** that the result is to be sorted on, separated by commas. A column identifier may be either a column name or a column number[†] that identifies an element of the SELECT list by its position within the list, 1 being the first (leftmost) element in the list, 2 the second element in the list, and so on. Column numbers could be used if the column to be sorted on is an expression and no AS clause is specified to assign the column a name that can subsequently be referenced. The ORDER BY clause allows the retrieved rows to be ordered in ascending (ASC) or descending (DESC) order on any column or combination of columns, regardless of whether that column appears in the result. However, some dialects insist that the ORDER BY elements appear in the SELECT list. In either case, the ORDER BY clause must always be the last clause of the SELECT statement.

[†]Column numbers are a deprecated feature of the ISO standard and should not be used.

EXAMPLE 6.11 Single-column ordering

Produce a list of salaries for all staff, arranged in descending order of salary.

```
SELECT staffNo, fName, lName, salary
FROM Staff
ORDER BY salary DESC;
```

This example is very similar to Example 6.2. The difference in this case is that the output is to be arranged in descending order of **salary**. This is achieved by adding the **ORDER BY** clause to the end of the **SELECT** statement, specifying **salary** as the column to be sorted, and **DESC** to indicate that the order is to be descending. In this case, we get the result table shown in Table 6.11. Note that we could have expressed the **ORDER BY** clause as: **ORDER BY 4 DESC**, with the 4 relating to the fourth column name in the **SELECT** list, namely **salary**.

TABLE 6.11 Result table for Example 6.11.

staffNo	fName	lName	salary
SL21	John	White	30000.00
SG5	Susan	Brand	24000.00
SG14	David	Ford	18000.00
SG37	Ann	Beech	12000.00
SA9	Mary	Howe	9000.00
SL41	Julie	Lee	9000.00

It is possible to include more than one element in the **ORDER BY** clause. The **major sort key** determines the overall order of the result table. In Example 6.11, the major sort key is **salary**. If the values of the major sort key are unique, there is no need for additional keys to control the sort. However, if the values of the major sort key are not unique, there may be multiple rows in the result table with the same value for the major sort key. In this case, it may be desirable to order rows with the same value for the major sort key by some additional sort key. If a second element appears in the **ORDER BY** clause, it is called a **minor sort key**.

EXAMPLE 6.12 Multiple column ordering

Produce an abbreviated list of properties arranged in order of property type.

```
SELECT propertyNo, type, rooms, rent
FROM PropertyForRent
ORDER BY type;
```

In this case we get the result table shown in Table 6.12(a).

There are four flats in this list. As we did not specify any minor sort key, the system arranges these rows in any order it chooses. To arrange the properties in order of **rent**, we specify a minor order, as follows:

```
SELECT propertyNo, type, rooms, rent
FROM PropertyForRent
ORDER BY type, rent DESC;
```

TABLE 6.12(a) Result table for Example 6.12 with one sort key.

propertyNo	type	rooms	rent
PL94	Flat	4	400
PG4	Flat	3	350
PG36	Flat	3	375
PG16	Flat	4	450
PA14	House	6	650
PG21	House	5	600

Now, the result is ordered first by property type, in ascending alphabetic order (ASC being the default setting), and within property type, in descending order of rent. In this case, we get the result table shown in Table 6.12(b).

TABLE 6.12(b) Result table for Example 6.12 with two sort keys.

propertyNo	type	rooms	rent
PG16	Flat	4	450
PL94	Flat	4	400
PG36	Flat	3	375
PG4	Flat	3	350
PA14	House	6	650
PG21	House	5	600

The ISO standard specifies that nulls in a column or expression sorted with ORDER BY should be treated as either less than all nonnull values or greater than all nonnull values. The choice is left to the DBMS implementor.

6.3.3 Using the SQL Aggregate Functions

As well as retrieving rows and columns from the database, we often want to perform some form of summation or **aggregation** of data, similar to the totals at the bottom of a report. The ISO standard defines five **aggregate functions**:

- COUNT – returns the number of values in a specified column
- SUM – returns the sum of the values in a specified column
- AVG – returns the average of the values in a specified column
- MIN – returns the smallest value in a specified column
- MAX – returns the largest value in a specified column

These functions operate on a single column of a table and return a single value. COUNT, MIN, and MAX apply to both numeric and nonnumeric fields, but SUM and AVG may be used on numeric fields only. Apart from COUNT(*), each function eliminates nulls first and operates only on the remaining nonnull values. COUNT(*) is a special use of COUNT that counts all the rows of a table, regardless of whether nulls or duplicate values occur.

If we want to eliminate duplicates before the function is applied, we use the keyword DISTINCT before the column name in the function. The ISO standard allows the keyword ALL to be specified if we do not want to eliminate duplicates, although ALL is assumed if nothing is specified. DISTINCT has no effect with the MIN and MAX functions. However, it may have an effect on the result of SUM or AVG, so consideration must be given to whether duplicates should be included or excluded in the computation. In addition, DISTINCT can be specified only once in a query.

It is important to note that an aggregate function can be used only in the SELECT list and in the HAVING clause (see Section 6.3.4). It is incorrect to use it elsewhere. If the SELECT list includes an aggregate function and no GROUP BY clause is being used to group data together (see Section 6.3.4), then no item in the SELECT list can include any reference to a column unless that column is the argument to an aggregate function. For example, the following query is illegal:

```
SELECT staffNo, COUNT(salary)
FROM Staff;
```

because the query does not have a GROUP BY clause and the column staffNo in the SELECT list is used outside an aggregate function.

TABLE 6.13
Result table for
Example 6.13.

myCount
5

EXAMPLE 6.13 Use of COUNT(*)

How many properties cost more than £350 per month to rent?

```
SELECT COUNT(*) AS myCount
FROM PropertyForRent
WHERE rent > 350;
```

Restricting the query to properties that cost more than £350 per month is achieved using the WHERE clause. The total number of properties satisfying this condition can then be found by applying the aggregate function COUNT. The result table is shown in Table 6.13.

TABLE 6.14
Result table for
Example 6.14.

myCount
2

EXAMPLE 6.14 Use of COUNT(DISTINCT)

How many different properties were viewed in May 2013?

```
SELECT COUNT(DISTINCT propertyNo) AS myCount
FROM Viewing
WHERE viewDate BETWEEN '1-May-13' AND '31-May-13';
```

Again, restricting the query to viewings that occurred in May 2013 is achieved using the WHERE clause. The total number of viewings satisfying this condition can then be found by applying the aggregate function COUNT. However, as the same property may be viewed many times, we have to use the DISTINCT keyword to eliminate duplicate properties. The result table is shown in Table 6.14.

EXAMPLE 6.15 Use of COUNT and SUM

Find the total number of Managers and the sum of their salaries.

```
SELECT COUNT(staffNo) AS myCount, SUM(salary) AS mySum
FROM Staff
WHERE position = 'Manager';
```

Restricting the query to Managers is achieved using the WHERE clause. The number of Managers and the sum of their salaries can be found by applying the COUNT and the SUM functions respectively to this restricted set. The result table is shown in Table 6.15.

TABLE 6.15 Result table for Example 6.15.

myCount	mySum
2	54000.00

EXAMPLE 6.16 Use of MIN, MAX, AVG

Find the minimum, maximum, and average staff salary.

```
SELECT MIN(salary) AS myMin, MAX(salary) AS myMax, AVG(salary) AS myAvg
FROM Staff;
```

In this example, we wish to consider all staff and therefore do not require a WHERE clause. The required values can be calculated using the MIN, MAX, and AVG functions based on the salary column. The result table is shown in Table 6.16.

TABLE 6.16 Result table for Example 6.16.

myMin	myMax	myAvg
9000.00	30000.00	17000.00

6.3.4 Grouping Results (GROUP BY Clause)

The previous summary queries are similar to the totals at the bottom of a report. They condense all the detailed data in the report into a single summary row of data. However, it is often useful to have subtotals in reports. We can use the GROUP BY clause of the SELECT statement to do this. A query that includes the GROUP BY clause is called a **grouped query**, because it groups the data from the SELECT table(s) and produces a single summary row for each group. The columns named in the GROUP BY clause are called the **grouping columns**. The ISO standard requires the SELECT clause and the GROUP BY clause to be closely integrated. When GROUP BY is used, each item in the SELECT list must be **single-valued per group**. In addition, the SELECT clause may contain only:

- column names;
- aggregate functions;

- constants;
- an expression involving combinations of these elements.

All column names in the SELECT list must appear in the GROUP BY clause unless the name is used only in an aggregate function. The contrary is not true: there may be column names in the GROUP BY clause that do not appear in the SELECT list. When the WHERE clause is used with GROUP BY, the WHERE clause is applied first, then groups are formed from the remaining rows that satisfy the search condition.

The ISO standard considers two nulls to be equal for purposes of the GROUP BY clause. If two rows have nulls in the same grouping columns and identical values in all the nonnull grouping columns, they are combined into the same group.

EXAMPLE 6.17 Use of GROUP BY

Find the number of staff working in each branch and the sum of their salaries.

```
SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum
FROM Staff
GROUP BY branchNo
ORDER BY branchNo;
```

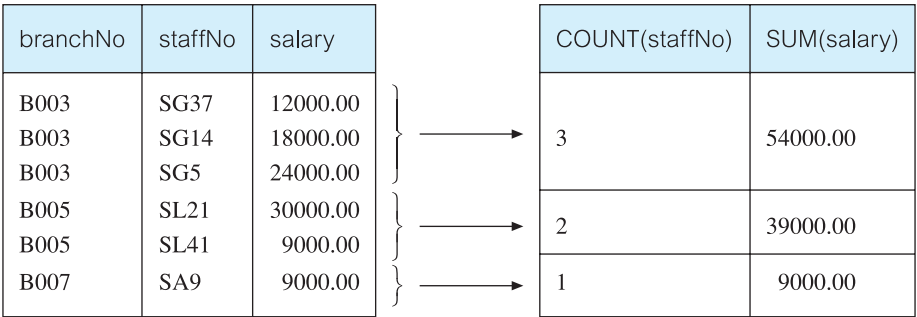
It is not necessary to include the column names `staffNo` and `salary` in the GROUP BY list, because they appear only in the SELECT list within aggregate functions. On the other hand, `branchNo` is not associated with an aggregate function and so must appear in the GROUP BY list. The result table is shown in Table 6.17.

TABLE 6.17 Result table for Example 6.17.

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00
B007	1	9000.00

Conceptually, SQL performs the query as follows:

- (1) SQL divides the staff into groups according to their respective branch numbers. Within each group, all staff have the same branch number. In this example, we get three groups:



- (2) For each group, SQL computes the number of staff members and calculates the sum of the values in the salary column to get the total of their salaries. SQL generates a single summary row in the query result for each group.
- (3) Finally, the result is sorted in ascending order of branch number, branchNo.

The SQL standard allows the SELECT list to contain nested queries (see Section 6.3.5). Therefore, we could also express the previous query as:

```
SELECT branchNo, (SELECT COUNT(staffNo) AS myCount
                  FROM Staff s
                  WHERE s.branchNo = b.branchNo),
              (SELECT SUM(salary) AS mySum
               FROM Staff s
               WHERE s.branchNo = b.branchNo)
FROM Branch b
ORDER BY branchNo;
```

With this version of the query, however, the two aggregate values are produced for each branch office in Branch; in some cases possibly with zero values.

Restricting groupings (HAVING clause)

The HAVING clause is designed for use with the GROUP BY clause to restrict the **groups** that appear in the final result table. Although similar in syntax, HAVING and WHERE serve different purposes. The WHERE clause filters individual rows going into the final result table, whereas HAVING filters **groups** going into the final result table. The ISO standard requires that column names used in the HAVING clause must also appear in the GROUP BY list or be contained within an aggregate function. In practice, the search condition in the HAVING clause always includes at least one aggregate function; otherwise the search condition could be moved to the WHERE clause and applied to individual rows. (Remember that aggregate functions cannot be used in the WHERE clause.)

The HAVING clause is not a necessary part of SQL—any query expressed using a HAVING clause can always be rewritten without the HAVING clause.

EXAMPLE 6.18 Use of HAVING

For each branch office with more than one member of staff, find the number of staff working in each branch and the sum of their salaries.

```
SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum
FROM Staff
GROUP BY branchNo
HAVING COUNT(staffNo) > 1
ORDER BY branchNo;
```

This is similar to the previous example, with the additional restriction that we want to consider only those groups (that is, branches) with more than one member of staff. This restriction applies to the groups, so the HAVING clause is used. The result table is shown in Table 6.18.

TABLE 6.18 Result table for Example 6.18.

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00

6.3.5 Subqueries

In this section we examine the use of a complete **SELECT** statement embedded within another **SELECT** statement. The results of this **inner SELECT** statement (or **subselect**) are used in the **outer** statement to help determine the contents of the final result. A sub-select can be used in the **WHERE** and **HAVING** clauses of an outer **SELECT** statement, where it is called a **subquery** or **nested query**. Subselects may also appear in **INSERT**, **UPDATE**, and **DELETE** statements (see Section 6.3.10). There are three types of subquery:

- A *scalar subquery* returns a single column and a single row, that is, a single value. In principle, a scalar subquery can be used whenever a single value is needed. Example 6.19 uses a scalar subquery.
- A *row subquery* returns multiple columns, but only a single row. A row subquery can be used whenever a row value constructor is needed, typically in predicates.
- A *table subquery* returns one or more columns and multiple rows. A table subquery can be used whenever a table is needed, for example, as an operand for the **IN** predicate.

EXAMPLE 6.19 Using a subquery with equality

List the staff who work in the branch at '163 Main St'.

```

SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo = (SELECT branchNo
                    FROM Branch
                    WHERE street = '163 Main St');
```

The inner **SELECT** statement (**SELECT** branchNo **FROM** Branch . . .) finds the branch number that corresponds to the branch with street name '163 Main St' (there will be only one such branch number, so this is an example of a scalar subquery). Having obtained this branch number, the outer **SELECT** statement then retrieves the details of all staff who work at this branch. In other words, the inner **SELECT** returns a result table containing a single value 'B003', corresponding to the branch at '163 Main St', and the outer **SELECT** becomes:

```

SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo = 'B003';
```

The result table is shown in Table 6.19.

TABLE 6.19 Result table for Example 6.19.

staffNo	fName	lName	position
SG37	Ann	Beech	Assistant
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

We can think of the subquery as producing a temporary table with results that can be accessed and used by the outer statement. A subquery can be used immediately following a relational operator ($=$, $<$, $>$, $<=$, $>=$, $<>$) in a WHERE clause, or a HAVING clause. The subquery itself is always enclosed in parentheses.

EXAMPLE 6.20 Using a subquery with an aggregate function

List all staff whose salary is greater than the average salary, and show by how much their salary is greater than the average.

```
SELECT staffNo, fName, lName, position,
       salary - (SELECT AVG(salary) FROM Staff) AS salDiff
FROM Staff
WHERE salary > (SELECT AVG(salary) FROM Staff);
```

First, note that we cannot write 'WHERE salary > AVG(salary)', because aggregate functions cannot be used in the WHERE clause. Instead, we use a subquery to find the average salary, and then use the outer SELECT statement to find those staff with a salary greater than this average. In other words, the subquery returns the average salary as £17,000. Note also the use of the scalar subquery in the SELECT list to determine the difference from the average salary. The outer query is reduced then to:

```
SELECT staffNo, fName, lName, position, salary - 17000 AS salDiff
FROM Staff
WHERE salary > 17000;
```

The result table is shown in Table 6.20.

TABLE 6.20 Result table for Example 6.20.

staffNo	fName	lName	position	salDiff
SL21	John	White	Manager	13000.00
SG14	David	Ford	Supervisor	1000.00
SG5	Susan	Brand	Manager	7000.00

The following rules apply to subqueries:

- (1) The ORDER BY clause may not be used in a subquery (although it may be used in the outermost SELECT statement).

- (2) The subquery SELECT list must consist of a single column name or expression, except for subqueries that use the keyword EXISTS (see Section 6.3.8).
- (3) By default, column names in a subquery refer to the table name in the FROM clause of the subquery. It is possible to refer to a table in a FROM clause of an outer query by qualifying the column name (see following).
- (4) When a subquery is one of the two operands involved in a comparison, the subquery must appear on the right-hand side of the comparison. For example, it would be incorrect to express the previous example as:

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE (SELECT AVG(salary) FROM Staff) < salary;
```

because the subquery appears on the left-hand side of the comparison with salary.

EXAMPLE 6.21 Nested subqueries: use of IN

List the properties that are handled by staff who work in the branch at '163 Main St'.

```
SELECT propertyNo, street, city, postcode, type, rooms, rent
FROM PropertyForRent
WHERE staffNo IN (SELECT staffNo
                  FROM Staff
                  WHERE branchNo = (SELECT branchNo
                                    FROM Branch
                                    WHERE street = '163 Main St'));
```

Working from the innermost query outwards, the first query selects the number of the branch at '163 Main St'. The second query then selects those staff who work at this branch number. In this case, there may be more than one such row found, and so we cannot use the equality condition (=) in the outermost query. Instead, we use the IN keyword. The outermost query then retrieves the details of the properties that are managed by each member of staff identified in the middle query. The result table is shown in Table 6.21.

TABLE 6.21 Result table for Example 6.21.

propertyNo	street	city	postcode	type	rooms	rent
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375
PG21	18 Dale Rd	Glasgow	G12	House	5	600

6.3.6 ANY and ALL

The keywords ANY and ALL may be used with subqueries that produce a single column of numbers. If the subquery is preceded by the keyword ALL, the condition will be true only if it is satisfied by all values produced by the subquery. If the subquery is preceded by the keyword ANY, the condition will be true if it is satisfied by any (one or more) values produced by the subquery. If the subquery is

empty, the ALL condition returns true, the ANY condition returns false. The ISO standard also allows the qualifier SOME to be used in place of ANY.

EXAMPLE 6.22 Use of ANY/SOME

Find all staff whose salary is larger than the salary of at least one member of staff at branch B003.

```

SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > SOME (SELECT salary
                     FROM Staff
                     WHERE branchNo = 'B003');
```

Although this query can be expressed using a subquery that finds the minimum salary of the staff at branch B003 and then an outer query that finds all staff whose salary is greater than this number (see Example 6.20), an alternative approach uses the SOME/ANY keyword. The inner query produces the set {12000, 18000, 24000} and the outer query selects those staff whose salaries are greater than any of the values in this set (that is, greater than the minimum value, 12000). This alternative method may seem more natural than finding the minimum salary in a subquery. In either case, the result table is shown in Table 6.22.

TABLE 6.22 Result table for Example 6.22.

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00

EXAMPLE 6.23 Use of ALL

Find all staff whose salary is larger than the salary of every member of staff at branch B003.

```

SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > ALL (SELECT salary
                   FROM Staff
                   WHERE branchNo = 'B003');
```

This example is very similar to the previous example. Again, we could use a subquery to find the maximum salary of staff at branch B003 and then use an outer query to find all staff whose salary is greater than this number. However, in this example we use the ALL keyword. The result table is shown in Table 6.23.

TABLE 6.23 Result table for Example 6.23.

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00

6.3.7 Multi-table Queries

All the examples we have considered so far have a major limitation: the columns that are to appear in the result table must all come from a single table. In many cases, this is insufficient to answer common queries that users will have. To combine columns from several tables into a result table, we need to use a **join** operation. The SQL join operation combines information from two tables by forming pairs of related rows from the two tables. The row pairs that make up the joined table are those where the matching columns in each of the two tables have the same value.

If we need to obtain information from more than one table, the choice is between using a subquery and using a join. If the final result table is to contain columns from different tables, then we must use a join. To perform a join, we simply include more than one table name in the FROM clause, using a comma as a separator, and typically including a WHERE clause to specify the join column(s). It is also possible to use an **alias** for a table named in the FROM clause. In this case, the alias is separated from the table name with a space. An alias can be used to qualify a column name whenever there is ambiguity regarding the source of the column name. It can also be used as a shorthand notation for the table name. If an alias is provided, it can be used anywhere in place of the table name.

EXAMPLE 6.24 Simple join

List the names of all clients who have viewed a property, along with any comments supplied.

```
SELECT c.clientNo, fName, lName, propertyNo, comment
FROM Client c, Viewing v
WHERE c.clientNo = v.clientNo;
```

We want to display the details from both the Client table and the Viewing table, and so we have to use a join. The SELECT clause lists the columns to be displayed. Note that it is necessary to qualify the client number, clientNo, in the SELECT list: clientNo could come from either table, and we have to indicate which one. (We could also have chosen the clientNo column from the Viewing table.) The qualification is achieved by prefixing the column name with the appropriate table name (or its alias). In this case, we have used *c* as the alias for the Client table.

To obtain the required rows, we include those rows from both tables that have identical values in the clientNo columns, using the search condition (c.clientNo = v.clientNo). We call these two columns the **matching columns** for the two tables. This is equivalent to the relational algebra Equijoin operation discussed in Section 5.1.3. The result table is shown in Table 6.24.

TABLE 6.24 Result table for Example 6.24.

clientNo	fName	lName	propertyNo	comment
CR56	Aline	Stewart	PG36	
CR56	Aline	Stewart	PA14	too small
CR56	Aline	Stewart	PG4	
CR62	Mary	Tregear	PA14	no dining room
CR76	John	Kay	PG4	too remote

The most common multi-table queries involve two tables that have a one-to-many (1:*) (or a parent/child) relationship (see Section 12.6.2). The previous query involving clients and viewings is an example of such a query. Each viewing (child) has an associated client (parent), and each client (parent) can have many associated viewings (children). The pairs of rows that generate the query results are parent/child row combinations. In Section 4.2.5 we described how primary key and foreign keys create the parent/child relationship in a relational database: the table containing the primary key is the parent table and the table containing the foreign key is the child table. To use the parent/child relationship in an SQL query, we specify a search condition that compares the primary key and the foreign key. In Example 6.24, we compared the primary key in the Client table, c.clientNo, with the foreign key in the Viewing table, v.clientNo.

The SQL standard provides the following alternative ways to specify this join:

```
FROM Client c JOIN Viewing v ON c.clientNo = v.clientNo
FROM Client JOIN Viewing USING clientNo
FROM Client NATURAL JOIN Viewing
```

In each case, the FROM clause replaces the original FROM and WHERE clauses. However, the first alternative produces a table with two identical clientNo columns; the remaining two produce a table with a single clientNo column.

EXAMPLE 6.25 Sorting a join

For each branch office, list the staff numbers and names of staff who manage properties and the properties that they manage.

```
SELECT s.branchNo, s.staffNo, fName, lName, propertyNo
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
ORDER BY s.branchNo, s.staffNo, propertyNo;
```

In this example, we need to join the Staff and PropertyForRent tables, based on the primary key/foreign key attribute (staffNo). To make the results more readable, we have ordered the output using the branch number as the major sort key and the staff number and property number as the minor keys. The result table is shown in Table 6.25.

TABLE 6.25 Result table for Example 6.25.

branchNo	staffNo	fName	lName	propertyNo
B003	SG14	David	Ford	PG16
B003	SG37	Ann	Beech	PG21
B003	SG37	Ann	Beech	PG36
B005	SL41	Julie	Lee	PL94
B007	SA9	Mary	Howe	PA14

EXAMPLE 6.26 Three-table join

For each branch, list the staff numbers and names of staff who manage properties, including the city in which the branch is located and the properties that the staff manage.

```
SELECT b.branchNo, b.city, s.staffNo, fName, lName, propertyNo
FROM Branch b, Staff s, PropertyForRent p
WHERE b.branchNo = s.branchNo AND s.staffNo = p.staffNo
ORDER BY b.branchNo, s.staffNo, propertyNo;
```

The result table requires columns from three tables: Branch (branchNo and city), Staff (staffNo, fName and lName), and PropertyForRent (propertyNo), so a join must be used. The Branch and Staff details are joined using the condition (b.branchNo = s.branchNo) to link each branch to the staff who work there. The Staff and PropertyForRent details are joined using the condition (s.staffNo = p.staffNo) to link staff to the properties they manage. The result table is shown in Table 6.26.

TABLE 6.26 Result table for Example 6.26.

branchNo	city	staffNo	fName	lName	propertyNo
B003	Glasgow	SG14	David	Ford	PG16
B003	Glasgow	SG37	Ann	Beech	PG21
B003	Glasgow	SG37	Ann	Beech	PG36
B005	London	SL41	Julie	Lee	PL94
B007	Aberdeen	SA9	Mary	Howe	PA14

Note again that the SQL standard provides alternative formulations for the FROM and WHERE clauses, for example:

```
FROM (Branch b JOIN Staff s USING branchNo) AS bs
      JOIN PropertyForRent p USING staffNo
```

EXAMPLE 6.27 Multiple grouping columns

Find the number of properties handled by each staff member, along with the branch number of the member of staff.

```
SELECT s.branchNo, s.staffNo, COUNT(*) AS myCount
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.branchNo, s.staffNo
ORDER BY s.branchNo, s.staffNo;
```

To list the required numbers, we first need to find out which staff actually manage properties. This can be found by joining the Staff and PropertyForRent tables on the staffNo column, using the FROM/WHERE clauses. Next, we need to form groups consisting of the branch number and staff number, using the GROUP BY clause. Finally, we sort the output using the ORDER BY clause. The result table is shown in Table 6.27(a).

TABLE 6.27(a) Result table for Example 6.27.

branchNo	staffNo	myCount
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1

Computing a join

A join is a subset of a more general combination of two tables known as the **Cartesian product** (see Section 5.1.2). The Cartesian product of two tables is another table consisting of all possible pairs of rows from the two tables. The columns of the product table are all the columns of the first table followed by all the columns of the second table. If we specify a two-table query without a WHERE clause, SQL produces the Cartesian product of the two tables as the query result. In fact, the ISO standard provides a special form of the SELECT statement for the Cartesian product:

```
SELECT [DISTINCT | ALL] { * | columnList }
FROM   TableName1 CROSS JOIN TableName2
```

Consider again Example 6.24, where we joined the Client and Viewing tables using the matching column, clientNo. Using the data from Figure 4.3, the Cartesian product of these two tables would contain 20 rows (4 clients * 5 viewings = 20 rows). It is equivalent to the query used in Example 6.24 without the WHERE clause.

Conceptually, the procedure for generating the results of a SELECT with a join is as follows:

- (1) Form the Cartesian product of the tables named in the FROM clause.
- (2) If there is a WHERE clause, apply the search condition to each row of the product table, retaining those rows that satisfy the condition. In terms of the relational algebra, this operation yields a **restriction** of the Cartesian product.
- (3) For each remaining row, determine the value of each item in the SELECT list to produce a single row in the result table.
- (4) If SELECT DISTINCT has been specified, eliminate any duplicate rows from the result table. In the relational algebra, Steps 3 and 4 are equivalent to a **projection** of the restriction over the columns mentioned in the SELECT list.
- (5) If there is an ORDER BY clause, sort the result table as required.

We will discuss query processing in more detail in Chapter 23.

Outer joins

The join operation combines data from two tables by forming pairs of related rows where the matching columns in each table have the same value. If one row of a

table is unmatched, the row is omitted from the result table. This has been the case for the joins we examined earlier. The ISO standard provides another set of join operators called **outer joins** (see Section 5.1.3). The Outer join retains rows that do not satisfy the join condition. To understand the Outer join operators, consider the following two simplified *Branch* and *PropertyForRent* tables, which we refer to as *Branch1* and *PropertyForRent1*, respectively:

Branch1		PropertyForRent1	
branchNo	bCity	propertyNo	pCity
B003	Glasgow	PA14	Aberdeen
B004	Bristol	PL94	London
B002	London	PG4	Glasgow

The (Inner) join of these two tables:

```
SELECT b.*, p.*
FROM Branch1 b, PropertyForRent1 p
WHERE b.bCity = p.pCity;
```

produces the result table shown in Table 6.27(b).

TABLE 6.27(b) Result table for inner join of the *Branch1* and *PropertyForRent1* tables.

branchNo	bCity	propertyNo	pCity
B003	Glasgow	PG4	Glasgow
B002	London	PL94	London

The result table has two rows where the cities are the same. In particular, note that there is no row corresponding to the branch office in Bristol and there is no row corresponding to the property in Aberdeen. If we want to include the unmatched rows in the result table, we can use an Outer join. There are three types of Outer join: **Left**, **Right**, and **Full** Outer joins. We illustrate their functionality in the following examples.

EXAMPLE 6.28 Left Outer join

List all branch offices and any properties that are in the same city.

The Left Outer join of these two tables:

```
SELECT b.*, p.*
FROM Branch1 b LEFT JOIN PropertyForRent1 p ON b.bCity = p.pCity;
```

produces the result table shown in Table 6.28. In this example the Left Outer join includes not only those rows that have the same city, but also those rows of the first (left) table that are unmatched with rows from the second (right) table. The columns from the second table are filled with NULLs.

TABLE 6.28 Result table for Example 6.28.

branchNo	bCity	propertyNo	pCity
B003	Glasgow	PG4	Glasgow
B004	Bristol	NULL	NULL
B002	London	PL94	London

EXAMPLE 6.29 Right Outer join

List all properties and any branch offices that are in the same city.

The Right Outer join of these two tables:

```
SELECT b.*, p.*  
FROM Branch1 b RIGHT JOIN PropertyForRent1 p ON b.bCity = p.pCity;
```

produces the result table shown in Table 6.29. In this example, the Right Outer join includes not only those rows that have the same city, but also those rows of the second (right) table that are unmatched with rows from the first (left) table. The columns from the first table are filled with NULLs.

TABLE 6.29 Result table for Example 6.29.

branchNo	bCity	propertyNo	pCity
NULL	NULL	PA14	Aberdeen
B003	Glasgow	PG4	Glasgow
B002	London	PL94	London

EXAMPLE 6.30 Full Outer join

List the branch offices and properties that are in the same city along with any unmatched branches or properties.

The Full Outer join of these two tables:

```
SELECT b.*, p.*  
FROM Branch1 b FULL JOIN PropertyForRent1 p ON b.bCity = p.pCity;
```

produces the result table shown in Table 6.30. In this case, the Full Outer join includes not only those rows that have the same city, but also those rows that are unmatched in both tables. The unmatched columns are filled with NULLs.

TABLE 6.30 Result table for Example 6.30.

branchNo	bCity	propertyNo	pCity
NULL	NULL	PA14	Aberdeen
B003	Glasgow	PG4	Glasgow
B004	Bristol	NULL	NULL
B002	London	PL94	London

6.3.8 EXISTS and NOT EXISTS

The keywords EXISTS and NOT EXISTS are designed for use only with subqueries. They produce a simple true/false result. EXISTS is true if and only if there exists at least one row in the result table returned by the subquery; it is false if the subquery returns an empty result table. NOT EXISTS is the opposite of EXISTS. Because EXISTS and NOT EXISTS check only for the existence or nonexistence of rows in the subquery result table, the subquery can contain any number of columns. For simplicity, it is common for subqueries following one of these keywords to be of the form:

(SELECT * FROM ...)

EXAMPLE 6.31 Query using EXISTS

Find all staff who work in a London branch office.

```
SELECT staffNo, fName, IName, position
FROM Staff s
WHERE EXISTS (SELECT *
              FROM Branch b
              WHERE s.branchNo = b.branchNo AND city = 'London');
```

This query could be rephrased as ‘Find all staff such that there exists a Branch row containing his/her branch number, branchNo, and the branch city equal to London’. The test for inclusion is the existence of such a row. If it exists, the subquery evaluates to true. The result table is shown in Table 6.31.

TABLE 6.31 Result table for Example 6.31.

staffNo	fName	IName	position
SL21	John	White	Manager
SL41	Julie	Lee	Assistant

Note that the first part of the search condition `s.branchNo = b.branchNo` is necessary to ensure that we consider the correct branch row for each member of staff. If we omitted this part of the query, we would get all staff rows listed out because the subquery (`SELECT * FROM Branch WHERE city = 'London'`) would always be true and the query would be reduced to:

```
SELECT staffNo, fName, lName, position FROM Staff WHERE true;
```

which is equivalent to:

```
SELECT staffNo, fName, lName, position FROM Staff;
```

We could also have written this query using the join construct:

```
SELECT staffNo, fName, lName, position
FROM Staff s, Branch b
WHERE s.branFchNo = b.branchNo AND city = 'London';
```

6.3.9 Combining Result Tables (UNION, INTERSECT, EXCEPT)

In SQL, we can use the normal set operations of *Union*, *Intersection*, and *Difference* to combine the results of two or more queries into a single result table:

- The **Union** of two tables, A and B, is a table containing all rows that are in either the first table A or the second table B or both.
- The **Intersection** of two tables, A and B, is a table containing all rows that are common to both tables A and B.
- The **Difference** of two tables, A and B, is a table containing all rows that are in table A but are not in table B.

The set operations are illustrated in Figure 6.1. There are restrictions on the tables that can be combined using the set operations, the most important one being that the two tables have to be **union-compatible**; that is, they have the same structure. This implies that the two tables must contain the same number of columns, and that their corresponding columns have the same data types and lengths. It is the user’s responsibility to ensure that data values in corresponding columns come from the same *domain*. For example, it would not be sensible to combine a column containing the age of staff with the number of rooms in a property, even though both columns may have the same data type: for example, SMALLINT.

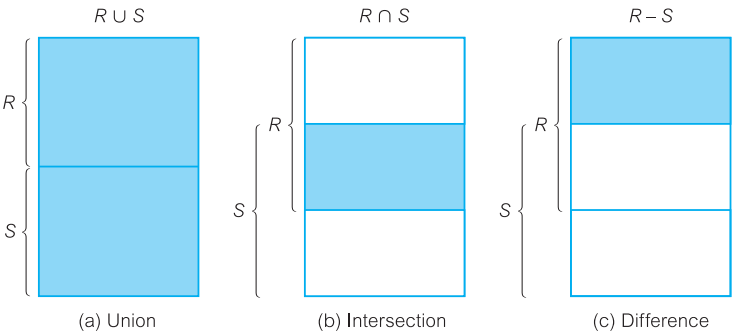


Figure 6.1
Union,
Intersection, and
Difference set
operations.

The three set operators in the ISO standard are called UNION, INTERSECT, and EXCEPT. The format of the set operator clause in each case is:

`operator [ALL] [CORRESPONDING [BY {column1 [, . . .]}]]`

If CORRESPONDING BY is specified, then the set operation is performed on the named column(s); if CORRESPONDING is specified but not the BY clause, the set operation is performed on the columns that are common to both tables. If ALL is specified, the result can include duplicate rows. Some dialects of SQL do not support INTERSECT and EXCEPT; others use MINUS in place of EXCEPT.

TABLE 6.32
Result table for
Example 6.32.

city
London
Glasgow
Aberdeen
Bristol

EXAMPLE 6.32 Use of UNION

Construct a list of all cities where there is either a branch office or a property.

<pre>(SELECT city FROM Branch WHERE city IS NOT NULL) UNION (SELECT city FROM PropertyForRent WHERE city IS NOT NULL);</pre>	<pre>or (SELECT * FROM Branch WHERE city IS NOT NULL) UNION CORRESPONDING BY city (SELECT * FROM PropertyForRent WHERE city IS NOT NULL);</pre>
--	---

This query is executed by producing a result table from the first query and a result table from the second query, and then merging both tables into a single result table consisting of all the rows from both result tables with the duplicate rows removed. The final result table is shown in Table 6.32.

TABLE 6.33
Result table for
Example 6.33.

city
Aberdeen
Glasgow
London

EXAMPLE 6.33 Use of INTERSECT

Construct a list of all cities where there is both a branch office and a property.

<pre>(SELECT city FROM Branch) INTERSECT (SELECT city FROM PropertyForRent);</pre>	<pre>or (SELECT * FROM Branch) INTERSECT CORRESPONDING BY city (SELECT * FROM PropertyForRent);</pre>
--	---

This query is executed by producing a result table from the first query and a result table from the second query, and then creating a single result table consisting of those rows that are common to both result tables. The final result table is shown in Table 6.33.

We could rewrite this query without the INTERSECT operator, for example:

<pre>SELECT DISTINCT b.city FROM Branch b, PropertyForRent p WHERE b.city = p.city;</pre>	<pre>or SELECT DISTINCT city FROM Branch b WHERE EXISTS (SELECT * FROM PropertyForRent p WHERE b.city = p.city);</pre>
---	--

The ability to write a query in several equivalent forms illustrates one of the disadvantages of the SQL language.

EXAMPLE 6.34 Use of EXCEPT

Construct a list of all cities where there is a branch office but no properties.

```
(SELECT city
FROM Branch)
EXCEPT
(SELECT city
FROM PropertyForRent);

or (SELECT *
FROM Branch)
EXCEPT CORRESPONDING BY city
(SELECT *
FROM PropertyForRent);
```

This query is executed by producing a result table from the first query and a result table from the second query, and then creating a single result table consisting of those rows that appear in the first result table but not in the second one. The final result table is shown in Table 6.34.

We could rewrite this query without the EXCEPT operator, for example:

```
SELECT DISTINCT city
FROM Branch
WHERE city NOT IN (SELECT city
FROM PropertyForRent);

or SELECT DISTINCT city
FROM Branch b
WHERE NOT EXISTS
(SELECT *
FROM PropertyForRent p
WHERE b.city = p.city);
```

TABLE 6.34
Result table for
Example 6.34.

city
Bristol

6.3.10 Database Updates

SQL is a complete data manipulation language that can be used for modifying the data in the database as well as querying the database. The commands for modifying the database are not as complex as the SELECT statement. In this section, we describe the three SQL statements that are available to modify the contents of the tables in the database:

- INSERT – adds new rows of data to a table
- UPDATE – modifies existing data in a table
- DELETE – removes rows of data from a table

Adding data to the database (INSERT)

There are two forms of the INSERT statement. The first allows a single row to be inserted into a named table and has the following format:

```
INSERT INTO TableName [(columnList)]
VALUES (dataValueList)
```

TableName may be either a base table or an updatable view (see Section 7.4), and *columnList* represents a list of one or more column names separated by commas. The *columnList* is optional; if omitted, SQL assumes a list of all columns in their original CREATE TABLE order. If specified, then any columns that are omitted from the list must have been declared as NULL columns when the table was created,

unless the `DEFAULT` option was used when creating the column (see Section 7.3.2). The *dataValueList* must match the *columnList* as follows:

- The number of items in each list must be the same.
- There must be a direct correspondence in the position of items in the two lists, so that the first item in *dataValueList* applies to the first item in *columnList*, the second item in *dataValueList* applies to the second item in *columnList*, and so on.
- The data type of each item in *dataValueList* must be compatible with the data type of the corresponding column.

EXAMPLE 6.35 INSERT ... VALUES

Insert a new row into the Staff table supplying data for all columns.

```
INSERT INTO Staff
VALUES ('SG16', 'Alan', 'Brown', 'Assistant', 'M', DATE '1957-05-25', 8300, 'B003');
```

As we are inserting data into each column in the order the table was created, there is no need to specify a column list. Note that character literals such as 'Alan' must be enclosed in single quotes.

EXAMPLE 6.36 INSERT using defaults

Insert a new row into the Staff table supplying data for all mandatory columns: staffNo, fName, lName, position, salary, and branchNo.

```
INSERT INTO Staff (staffNo, fName, lName, position, salary, branchNo)
VALUES ('SG44', 'Anne', 'Jones', 'Assistant', 8100, 'B003');
```

As we are inserting data into only certain columns, we must specify the names of the columns that we are inserting data into. The order for the column names is not significant, but it is more normal to specify them in the order they appear in the table. We could also express the `INSERT` statement as:

```
INSERT INTO Staff
VALUES ('SG44', 'Anne', 'Jones', 'Assistant', NULL, NULL, 8100, 'B003');
```

In this case, we have explicitly specified that the columns `sex` and `DOB` should be set to `NULL`.

The second form of the `INSERT` statement allows multiple rows to be copied from one or more tables to another, and has the following format:

```
INSERT INTO TableName [(columnList)]
SELECT ...
```

TableName and *columnList* are defined as before when inserting a single row. The `SELECT` clause can be any valid `SELECT` statement. The rows inserted into the named table are identical to the result table produced by the subselect. The same restrictions that apply to the first form of the `INSERT` statement also apply here.

EXAMPLE 6.37 INSERT ... SELECT

Assume that there is a table `StaffPropCount` that contains the names of staff and the number of properties they manage:

```
StaffPropCount(staffNo, fName, lName, propCount)
```

Populate the `StaffPropCount` table using details from the `Staff` and `PropertyForRent` tables.

```
INSERT INTO StaffPropCount
(SELECT s.staffNo, fName, lName, COUNT(*)
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.staffNo, fName, lName)
UNION
(SELECT staffNo, fName, lName, 0
FROM Staff s
WHERE NOT EXISTS (SELECT *
                   FROM PropertyForRent p
                   WHERE p.staffNo = s.staffNo));
```

This example is complex, because we want to count the number of properties that staff manage. If we omit the second part of the `UNION`, then we get a list of only those staff who currently manage at least one property; in other words, we exclude those staff who currently do not manage any properties. Therefore, to include the staff who do not manage any properties, we need to use the `UNION` statement and include a second `SELECT` statement to add in such staff, using a 0 value for the count attribute. The `StaffPropCount` table will now be as shown in Table 6.35.

Note that some dialects of SQL may not allow the use of the `UNION` operator within a subselect for an `INSERT`.

TABLE 6.35 Result table for Example 6.37.

staffNo	fName	lName	propCount
SG14	David	Ford	1
SL21	John	White	0
SG37	Ann	Beech	2
SA9	Mary	Howe	1
SG5	Susan	Brand	0
SL41	Julie	Lee	1

Modifying data in the database (UPDATE)

The `UPDATE` statement allows the contents of existing rows in a named table to be changed. The format of the command is:

```
UPDATE TableName
SET columnName1 = dataValue1 [, columnName2 = dataValue2 . . .]
[WHERE searchCondition]
```

TableName can be the name of a base table or an updatable view (see Section 7.4). The SET clause specifies the names of one or more columns that are to be updated. The WHERE clause is optional; if omitted, the named columns are updated for *all* rows in the table. If a WHERE clause is specified, only those rows that satisfy the *searchCondition* are updated. The new *dataValue(s)* must be compatible with the data type(s) for the corresponding column(s).

EXAMPLE 6.38 UPDATE all rows

Give all staff a 3% pay increase.

```
UPDATE Staff
SET salary = salary*1.03;
```

As the update applies to all rows in the Staff table, the WHERE clause is omitted.

EXAMPLE 6.39 UPDATE specific rows

Give all Managers a 5% pay increase.

```
UPDATE Staff
SET salary = salary*1.05
WHERE position = 'Manager';
```

The WHERE clause finds the rows that contain data for Managers and the update salary = salary*1.05 is applied only to these particular rows.

EXAMPLE 6.40 UPDATE multiple columns

Promote David Ford (staffNo = 'SG14') to Manager and change his salary to £18,000.

```
UPDATE Staff
SET position = 'Manager', salary = 18000
WHERE staffNo = 'SG14';
```

Deleting data from the database (DELETE)

The DELETE statement allows rows to be deleted from a named table. The format of the command is:

```
DELETE FROM TableName
[WHERE searchCondition]
```

As with the INSERT and UPDATE statements, *TableName* can be the name of a base table or an updatable view (see Section 7.4). The *searchCondition* is optional; if omitted, *all* rows are deleted from the table. This does not delete the table itself—to delete the table contents and the table definition, the DROP TABLE statement must be used instead (see Section 7.3.3). If a *searchCondition* is specified, only those rows that satisfy the condition are deleted.

EXAMPLE 6.41 DELETE specific rows

Delete all viewings that relate to property PG4.

```
DELETE FROM Viewing
WHERE propertyNo = 'PG4';
```

The WHERE clause finds the rows for property PG4 and the delete operation is applied only to these particular rows.

EXAMPLE 6.42 DELETE all rows

Delete all rows from the Viewing table.

```
DELETE FROM Viewing;
```

No WHERE clause has been specified, so the delete operation applies to all rows in the table. This query removes all rows from the table, leaving only the table definition, so that we are still able to insert data into the table at a later stage.

Chapter Summary

- SQL is a nonprocedural language consisting of standard English words such as SELECT, INSERT, and DELETE that can be used by professionals and non-professionals alike. It is both the formal and de facto standard language for defining and manipulating relational databases.
- The **SELECT** statement is the most important statement in the language and is used to express a query. It combines the three fundamental relational algebra operations of *Selection*, *Projection*, and *Join*. Every SELECT statement produces a query result table consisting of one or more columns and zero or more rows.
- The SELECT clause identifies the columns and/or calculated data to appear in the result table. All column names that appear in the SELECT clause must have their corresponding tables or views listed in the FROM clause.
- The WHERE clause selects rows to be included in the result table by applying a search condition to the rows of the named table(s). The ORDER BY clause allows the result table to be sorted on the values in one or more columns. Each column can be sorted in ascending or descending order. If specified, the ORDER BY clause must be the last clause in the SELECT statement.
- SQL supports five aggregate functions (COUNT, SUM, AVG, MIN, and MAX) that take an entire column as an argument and compute a single value as the result. It is illegal to mix aggregate functions with column names in a SELECT clause, unless the GROUP BY clause is used.
- The GROUP BY clause allows summary information to be included in the result table. Rows that have the same value for one or more columns can be grouped together and treated as a unit for using the aggregate functions. In this case, the aggregate functions take each group as an argument and compute a single value for each group as the result. The HAVING clause acts as a WHERE clause for groups, restricting the groups that appear in the final result table. However, unlike the WHERE clause, the HAVING clause can include aggregate functions.
- A **subselect** is a complete SELECT statement embedded in another query. A subselect may appear within the WHERE or HAVING clauses of an outer SELECT statement, where it is called a **subquery** or **nested query**. Conceptually, a subquery produces a temporary table whose contents can be accessed by the outer query. A subquery can be embedded in another subquery.

- There are three types of subquery: **scalar**, **row**, and **table**. A *scalar subquery* returns a single column and a single row, that is, a single value. In principle, a scalar subquery can be used whenever a single value is needed. A *row subquery* returns multiple columns, but only a single row. A row subquery can be used whenever a row value constructor is needed, typically in predicates. A *table subquery* returns one or more columns and multiple rows. A table subquery can be used whenever a table is needed; for example, as an operand for the IN predicate.
- If the columns of the result table come from more than one table, a **join** must be used, by specifying more than one table in the FROM clause and typically including a WHERE clause to specify the join column(s). The ISO standard allows **Outer joins** to be defined. It also allows the set operations of *Union*, *Intersection*, and *Difference* to be used with the **UNION**, **INTERSECT**, and **EXCEPT** commands.
- As well as SELECT, the SQL DML includes the **INSERT** statement to insert a single row of data into a named table or to insert an arbitrary number of rows from one or more other tables using a **subselect**; the **UPDATE** statement to update one or more values in a specified column or columns of a named table; the **DELETE** statement to delete one or more rows from a named table.

Review Questions

- 6.1 Briefly describe the four basic SQL DML statements and explain their use.
- 6.2 Explain the importance and application of the WHERE clause in the UPDATE and DELETE statements.
- 6.3 Explain the function of each of the clauses in the SELECT statement. What restrictions are imposed on these clauses?
- 6.4 What restrictions apply to the use of the aggregate functions within the SELECT statement? How do nulls affect the aggregate functions?
- 6.5 How can results from two SQL queries be combined? Differentiate how the INTERSECT and EXCEPT commands work.
- 6.6 Differentiate between the three types of subqueries. Why is it important to understand the nature of subquery result before you write an SQL statement?

Exercises

For Exercises 6.7–6.28, use the Hotel schema defined at the start of the Exercises at the end of Chapter 4.

Simple queries

- 6.7 List full details of all hotels.
- 6.8 List full details of all hotels in London.
- 6.9 List the names and addresses of all guests living in London, alphabetically ordered by name.
- 6.10 List all double or family rooms with a price below £40.00 per night, in ascending order of price.
- 6.11 List the bookings for which no dateTo has been specified.

Aggregate functions

- 6.12 How many hotels are there?
- 6.13 What is the average price of a room?
- 6.14 What is the total revenue per night from all double rooms?
- 6.15 How many different guests have made bookings for August?

Subqueries and joins

- 6.16 List the price and type of all rooms at the Grosvenor Hotel.
- 6.17 List all guests currently staying at the Grosvenor Hotel.
- 6.18 List the details of all rooms at the Grosvenor Hotel, including the name of the guest staying in the room, if the room is occupied.
- 6.19 What is the total income from bookings for the Grosvenor Hotel today?
- 6.20 List the rooms that are currently unoccupied at the Grosvenor Hotel.
- 6.21 What is the lost income from unoccupied rooms at the Grosvenor Hotel?

Grouping

- 6.22 List the number of rooms in each hotel.
- 6.23 List the number of rooms in each hotel in London.
- 6.24 What is the average number of bookings for each hotel in August?
- 6.25 What is the most commonly booked room type for each hotel in London?
- 6.26 What is the lost income from unoccupied rooms at each hotel today?

Populating tables

- 6.27 Insert rows into each of these tables.
- 6.28 Update the price of all rooms by 5%.

General

- 6.29 Investigate the SQL dialect on any DBMS that you are currently using. Determine the system's compliance with the DML statements of the ISO standard. Investigate the functionality of any extensions that the DBMS supports. Are there any functions not supported?
- 6.30 Demonstrate that queries written using the UNION operator can be rewritten using the OR operator to produce the same result.
- 6.31 Apply the syntax for inserting data into a table.

Case Study 2

For Exercises 6.32–6.40, use the Projects schema defined in the Exercises at the end of Chapter 5.

- 6.32 List all employees from BRICS countries in alphabetical order of surname.
- 6.33 List all the details of employees born between 1980–90.
- 6.34 List all managers who are female in alphabetical order of surname, and then first name.
- 6.35 Remove all projects that are managed by the planning department.
- 6.36 Assume the planning department is going to be merged with the IT department. Update employee records to reflect the proposed change.
- 6.37 Using the UNION command, list all projects that are managed by the IT and the HR department.
- 6.38 Produce a report of the total hours worked by each female employee, arranged by department number and alphabetically by employee surname within each department.
- 6.39 Remove all project from the database which had no employees worked..
- 6.40 List the total number of employees in each department for those departments with more than 10 employees. Create an appropriate heading for the columns of the results table.

Case Study 3

For Exercises 6.41–6.54, use the Library schema defined in the Exercises at the end of Chapter 5.

- 6.41 List all book titles.
- 6.42 List all borrower details.
- 6.43 List all books titles published between 2010 and 2014.
- 6.44 Remove all books published before 1950 from the database.
- 6.45 List all book titles that have never been borrowed by any borrower.
- 6.46 List all book titles that contain the word 'database' and are available for loan.
- 6.47 List the names of borrowers with overdue books.
- 6.48 How many copies of each book title are there?
- 6.49 How many copies of ISBN "0-321-52306-7" are currently available?
- 6.50 How many times has the book title with ISBN "0-321-52306-7" been borrowed?
- 6.51 Produce a report of book titles that have been borrowed by "Peter Bloomfield."
- 6.52 For each book title with more than three copies, list the names of library members who have borrowed them.
- 6.53 Produce a report with the details of borrowers who currently have books overdue.
- 6.54 Produce a report detailing how many times each book title has been borrowed.

Chapter Objectives

In this chapter you will learn:

- The data types supported by the SQL standard.
- The purpose of the integrity enhancement feature of SQL.
- How to define integrity constraints using SQL, including:
 - required data;
 - domain constraints;
 - entity integrity;
 - referential integrity;
 - general constraints.
- How to use the integrity enhancement feature in the CREATE and ALTER TABLE statements.
- The purpose of views.
- How to create and delete views using SQL.
- How the DBMS performs operations on views.
- Under what conditions views are updatable.
- The advantages and disadvantages of views.
- How the ISO transaction model works.
- How to use the GRANT and REVOKE statements as a level of security.

In the previous chapter we discussed in some detail SQL and, in particular, the SQL data manipulation facilities. In this chapter we continue our presentation of SQL and examine the main SQL data definition facilities.

Structure of this Chapter In Section 7.1 we examine the ISO SQL data types. The 1989 ISO standard introduced an Integrity Enhancement Feature (IEF), which provides facilities for defining referential integrity and other constraints (ISO, 1989). Prior to this standard, it was the responsibility of each application program to ensure compliance with these constraints. The provision of an IEF greatly enhances the functionality of SQL and allows constraint checking to be centralized and standardized. We consider the IEF in Section 7.2 and the main SQL data definition facilities in Section 7.3.

In Section 7.4 we show how views can be created using SQL, and how the DBMS converts operations on views into equivalent operations on the base tables. We also discuss the restrictions that the ISO SQL standard places on views in order for them to be updatable. In Section 7.5, we briefly describe the ISO SQL transaction model.

Views provide a certain degree of database security. SQL also provides a separate access control subsystem, containing facilities to allow users to share database objects or, alternatively, to restrict access to database objects. We discuss the access control subsystem in Section 7.6.

In Chapter 9 we examine in some detail the features that have recently been added to the SQL specification to support object-oriented data management. In Appendix I we discuss how SQL can be embedded in high-level programming languages to access constructs that until recently were not available in SQL. As in the previous chapter, we present the features of SQL using examples drawn from the *DreamHome* case study. We use the same notation for specifying the format of SQL statements as defined in Section 6.2.



7.1 The ISO SQL Data Types

In this section we introduce the data types defined in the SQL standard. We start by defining what constitutes a valid identifier in SQL.

7.1.1 SQL Identifiers

SQL identifiers are used to identify objects in the database, such as table names, view names, and columns. The characters that can be used in a user-defined SQL identifier must appear in a **character set**. The ISO standard provides a default character set, which consists of the uppercase letters A . . . Z, the lowercase letters a . . . z, the digits 0 . . . 9, and the underscore (_) character. It is also possible to specify an alternative character set. The following restrictions are imposed on an identifier:

- an identifier can be no longer than 128 characters (most dialects have a much lower limit than this);
- an identifier must start with a letter;
- an identifier cannot contain spaces.

TABLE 7.1 ISO SQL data types.

DATA TYPE	DECLARATIONS				
boolean	BOOLEAN				
character	CHAR	VARCHAR			
bit [†]	BIT	BIT VARYING			
exact numeric	NUMERIC	DECIMAL	INTEGER	SMALLINT	BIGINT
approximate numeric	FLOAT	REAL	DOUBLE PRECISION		
datetime	DATE	TIME	TIMESTAMP		
interval	INTERVAL				
large objects	CHARACTER LARGE OBJECT		BINARY LARGE OBJECT		

[†]BIT and BIT VARYING have been removed from the SQL:2003 standard.

7.1.2 SQL Scalar Data Types

Table 7.1 shows the SQL scalar data types defined in the ISO standard. Sometimes, for manipulation and conversion purposes, the data types *character* and *bit* are collectively referred to as **string** data types, and *exact numeric* and *approximate numeric* are referred to as **numeric** data types, as they share similar properties. The SQL standard now also defines both character large objects and binary large objects, although we defer discussion of these data types until Chapter 9.

Boolean data

Boolean data consists of the distinct truth values TRUE and FALSE. Unless prohibited by a NOT NULL constraint, boolean data also supports the UNKNOWN truth value as the NULL value. All boolean data type values and SQL truth values are mutually comparable and assignable. The value TRUE is greater than the value FALSE, and any comparison involving the NULL value or an UNKNOWN truth value returns an UNKNOWN result.

Character data

Character data consists of a sequence of characters from an implementation-defined character set, that is, it is defined by the vendor of the particular SQL dialect. Thus, the exact characters that can appear as data values in a character type column will vary. ASCII and EBCDIC are two sets in common use today. The format for specifying a character data type is:

CHARACTER [VARYING] [length]

CHARACTER can be abbreviated to **CHAR** and

CHARACTER VARYING to **VARCHAR**.

When a character string column is defined, a length can be specified to indicate the maximum number of characters that the column can hold (default length is 1). A character string may be defined as having a **fixed** or **varying** length. If the string

is defined to be a fixed length and we enter a string with fewer characters than this length, the string is padded with blanks on the right to make up the required size. If the string is defined to be of a varying length and we enter a string with fewer characters than this length, only those characters entered are stored, thereby using less space. For example, the branch number column `branchNo` of the `Branch` table, which has a fixed length of four characters, is declared as:

```
branchNo CHAR(4)
```

The column address of the `PrivateOwner` table, which has a variable number of characters up to a maximum of 30, is declared as:

```
address VARCHAR(30)
```

Bit data

The bit data type is used to define bit strings, that is, a sequence of binary digits (bits), each having either the value 0 or 1. The format for specifying the bit data type is similar to that of the character data type:

```
BIT [VARYING] [length]
```

For example, to hold the fixed length binary string "0011", we declare a column *bitString*, as:

```
bitString BIT(4)
```

Exact Numeric Data

The exact numeric data type is used to define numbers with an exact representation. The number consists of digits, an optional decimal point, and an optional sign. An exact numeric data type consists of a **precision** and a **scale**. The precision gives the total number of significant decimal digits, that is, the total number of digits, including decimal places but excluding the point itself. The scale gives the total number of decimal places. For example, the exact numeric value `-12.345` has precision 5 and scale 3. A special case of exact numeric occurs with integers. There are several ways of specifying an exact numeric data type:

```
NUMERIC [ precision [, scale] ]
```

```
DECIMAL [ precision [, scale] ]
```

```
INTEGER
```

```
SMALLINT
```

```
BIGINT
```

INTEGER can be abbreviated to **INT** and **DECIMAL** to **DEC**

NUMERIC and **DECIMAL** store numbers in decimal notation. The default scale is always 0; the default precision is implementation-defined. **INTEGER** is used for large positive or negative whole numbers. **SMALLINT** is used for small positive or negative whole numbers and **BIGINT** for very large whole numbers. By specifying this data type, less storage space can be reserved for the data. For example, the maximum absolute value that can be stored with **SMALLINT** might be 32 767. The column `rooms` of the `PropertyForRent` table, which represents the number of rooms in a property, is obviously a small integer and can be declared as:

rooms **SMALLINT**

The column salary of the Staff table can be declared as:

salary **DECIMAL**(7,2)

which can handle a value up to 99,999.99.

Approximate numeric data

The approximate numeric data type is used for defining numbers that do not have an exact representation, such as real numbers. Approximate numeric, or floating point, notation is similar to scientific notation, in which a number is written as a *mantissa* times some power of ten (the *exponent*). For example, 10E3, +5.2E6, −0.2E−4. There are several ways of specifying an approximate numeric data type:

FLOAT [precision]

REAL

DOUBLE PRECISION

The *precision* controls the precision of the mantissa. The precision of REAL and DOUBLE PRECISION is implementation-defined.

Datetime data

The datetime data type is used to define points in time to a certain degree of accuracy. Examples are dates, times, and times of day. The ISO standard subdivides the datetime data type into YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, TIMEZONE_HOUR, and TIMEZONE_MINUTE. The latter two fields specify the hour and minute part of the time zone offset from Universal Coordinated Time (which used to be called Greenwich Mean Time). Three types of datetime data type are supported:

DATE

TIME [timePrecision] [**WITH TIME ZONE**]

TIMESTAMP [timePrecision] [**WITH TIME ZONE**]

DATE is used to store calendar dates using the YEAR, MONTH, and DAY fields. TIME is used to store time using the HOUR, MINUTE, and SECOND fields. TIMESTAMP is used to store date and times. The *timePrecision* is the number of decimal places of accuracy to which the SECOND field is kept. If not specified, TIME defaults to a precision of 0 (that is, whole seconds), and TIMESTAMP defaults to 6 (that is, microseconds). The WITH TIME ZONE keyword controls the presence of the TIMEZONE_HOUR and TIMEZONE_MINUTE fields. For example, the column date of the Viewing table, which represents the date (year, month, day) that a client viewed a property, is declared as:

viewDate **DATE**

Interval data

The interval data type is used to represent periods of time. Every interval data type consists of a contiguous subset of the fields: YEAR, MONTH, DAY, HOUR,

TABLE 7.2 ISO SQL scalar operators.

OPERATOR	MEANING
OCTET_LENGTH	Returns the length of a string in octets (bit length divided by 8). For example, OCTET_LENGTH (x'FFFF') returns 2.
CHAR_LENGTH	Returns the length of a string in characters (or octets, if the string is a bit string). For example, CHAR_LENGTH ('Beech') returns 5.
CAST	Converts a value expression of one data type into a value in another data type. For example, CAST (5.2E6 AS INTEGER).
	Concatenates two character strings or bit strings. For example, fName IName.
CURRENT_USER or USER	Returns a character string representing the current authorization identifier (informally, the current user name).
SESSION_USER	Returns a character string representing the SQL-session authorization identifier.
SYSTEM_USER	Returns a character string representing the identifier of the user who invoked the current module.
LOWER	Converts uppercase letters to lowercase. For example, LOWER (SELECT fName FROM Staff WHERE staffNo = 'SL21') returns 'john'.
UPPER	Converts lower-case letters to upper-case. For example, UPPER (SELECT fName FROM Staff WHERE staffNo = 'SL21') returns 'JOHN'.
TRIM	Removes leading (LEADING), trailing (TRAILING), or both leading and trailing (BOTH) characters from a string. For example, TRIM (BOTH '*' FROM '*** Hello World ***') returns 'Hello World'.
POSITION	Returns the position of one string within another string. For example, POSITION ('ee' IN 'Beech') returns 2.
SUBSTRING	Returns a substring selected from within a string. For example, SUBSTRING ('Beech' FROM 1 TO 3) returns the string 'Bee'.
CASE	Returns one of a specified set of values, based on some condition. For example, CASE type WHEN 'House' WHEN 'Flat' ELSE END THEN 1 THEN 2 0
CURRENT_DATE	Returns the current date in the time zone that is local to the user.
CURRENT_TIME	Returns the current time in the time zone that is the current default for the session. For example, CURRENT_TIME (6) gives time to microseconds precision.
CURRENT_TIMESTAMP	Returns the current date and time in the time zone that is the current default for the session. For example, CURRENT_TIMESTAMP (0) gives time to seconds precision.
EXTRACT	Returns the value of a specified field from a datetime or interval value. For example, EXTRACT (YEAR FROM Registration.dateJoined).
ABS	Operates on a numeric argument and returns its absolute value in the same most specific type. For example, ABS (−17.1) returns 17.1.
MOD	Operates on two exact numeric arguments with scale 0 and returns the modulus (remainder) of the first argument divided by the second argument as an exact numeric with scale 0. For example, MOD (26, 11) returns 4.
LN	Computes the natural logarithm of its argument. For example, LN (65) returns 4.174 (approx).
EXP	Computes the exponential function, that is, e, (the base of natural logarithms) raised to the power equal to its argument. For example, EXP (2) returns 7.389 (approx).

POWER	Raises its first argument to the power of its second argument. For example, POWER (2,3) returns 8.
SQRT	Computes the square root of its argument. For example, SQRT (16) returns 4.
FLOOR	Computes the greatest integer less than or equal to its argument. For example, FLOOR (15.7) returns 15.
CEIL	Computes the least integer greater than or equal to its argument. For example, CEIL (15.7) returns 16.

MINUTE, SECOND. There are two classes of interval data type: **year–month** intervals and **day–time** intervals. The year–month class may contain only the YEAR and/or the MONTH fields; the day–time class may contain only a contiguous selection from DAY, HOUR, MINUTE, SECOND. The format for specifying the interval data type is:

```

INTERVAL {{startField TO endField} singleDatetimeField}
startField = YEAR | MONTH | DAY | HOUR | MINUTE
              [(intervalLeadingFieldPrecision)]
endField = YEAR | MONTH | DAY | HOUR | MINUTE | SECOND
              [(fractionalSecondsPrecision)]
singleDatetimeField = startField | SECOND
              [(intervalLeadingFieldPrecision [, fractionalSecondsPrecision])]

```

In all cases, *startField* has a leading field precision that defaults to 2. For example:

```
INTERVAL YEAR(2) TO MONTH
```

represents an interval of time with a value between 0 years 0 months, and 99 years 11 months. In addition,

```
INTERVAL HOUR TO SECOND(4)
```

represents an interval of time with a value between 0 hours 0 minutes 0 seconds and 99 hours 59 minutes 59.9999 seconds (the fractional precision of second is 4).

Large objects

A **large object** is a data type that holds a large amount of data, such as a long text file or a graphics file. Three different types of large object data types are defined in SQL:

- Binary Large Object (BLOB), a binary string that does not have a character set or collation association;
- Character Large Object (CLOB) and National Character Large Object (NCLOB), both character strings.

We discuss large objects in more detail in Chapter 9.

Scalar operators

SQL provides a number of built-in scalar operators and functions that can be used to construct a scalar expression, that is, an expression that evaluates to a scalar value. Apart from the obvious arithmetic operators (+, −, *, /), the operators shown in Table 7.2 are available.

7.2 Integrity Enhancement Feature

In this section, we examine the facilities provided by the SQL standard for integrity control. Integrity control consists of constraints that we wish to impose in order to protect the database from becoming inconsistent. We consider five types of integrity constraint (see Section 4.3):

- required data;
- domain constraints;
- entity integrity;
- referential integrity;
- general constraints.

These constraints can be defined in the `CREATE` and `ALTER TABLE` statements, as we will explain shortly.

7.2.1 Required Data

Some columns must contain a valid value; they are not allowed to contain nulls. A null is distinct from blank or zero, and is used to represent data that is either not available, missing, or not applicable (see Section 4.3.1). For example, every member of staff must have an associated job position (for example, Manager, Assistant, and so on). The ISO standard provides the **NOT NULL** column specifier in the `CREATE` and `ALTER TABLE` statements to provide this type of constraint. When **NOT NULL** is specified, the system rejects any attempt to insert a null in the column. If **NULL** is specified, the system accepts nulls. The ISO default is **NULL**. For example, to specify that the column position of the `Staff` table cannot be null, we define the column as:

```
position VARCHAR(10) NOT NULL
```

7.2.2 Domain Constraints

Every column has a domain; in other words, a set of legal values (see Section 4.2.1). For example, the sex of a member of staff is either 'M' or 'F', so the domain of the column `sex` of the `Staff` table is a single character string consisting of either 'M' or 'F'. The ISO standard provides two mechanisms for specifying domains in the `CREATE` and `ALTER TABLE` statements. The first is the **CHECK** clause, which allows a constraint to be defined on a column or the entire table. The format of the **CHECK** clause is:

```
CHECK (searchCondition)
```

In a column constraint, the **CHECK** clause can reference only the column being defined. Thus, to ensure that the column `sex` can be specified only as 'M' or 'F', we could define the column as:

```
sex CHAR NOT NULL CHECK (sex IN ('M', 'F'))
```


However, the ISO standard allows domains to be defined more explicitly using the **CREATE DOMAIN** statement:

```
CREATE DOMAIN DomainName [AS] dataType  
[DEFAULT defaultOption]  
[CHECK (searchCondition)]
```

A domain is given a name, *DomainName*, a data type (as described in Section 7.1.2), an optional default value, and an optional CHECK constraint. This is not the complete definition, but it is sufficient to demonstrate the basic concept. Thus, for the previous example, we could define a domain for `sex` as:

```
CREATE DOMAIN SexType AS CHAR  
DEFAULT 'M'  
CHECK (VALUE IN ('M', 'F'));
```

This definition creates a domain `SexType` that consists of a single character with either the value 'M' or 'F'. When defining the column `sex`, we can now use the domain name `SexType` in place of the data type `CHAR`:

```
sex SexType NOT NULL
```

The *searchCondition* can involve a table lookup. For example, we can create a domain `BranchNumber` to ensure that the values entered correspond to an existing branch number in the `Branch` table, using the statement:

```
CREATE DOMAIN BranchNumber AS CHAR(4)  
CHECK (VALUE IN (SELECT branchNo FROM Branch));
```

The preferred method of defining domain constraints is using the **CREATE DOMAIN** statement. Domains can be removed from the database using the **DROP DOMAIN** statement:

```
DROP DOMAIN DomainName [RESTRICT | CASCADE]
```

The drop behavior, **RESTRICT** or **CASCADE**, specifies the action to be taken if the domain is currently being used. If **RESTRICT** is specified and the domain is used in an existing table, view, or assertion definition (see Section 7.2.5), the drop will fail. In the case of **CASCADE**, any table column that is based on the domain is automatically changed to use the domain's underlying data type, and any constraint or default clause for the domain is replaced by a column constraint or column default clause, if appropriate.

7.2.3 Entity Integrity

The primary key of a table must contain a unique, nonnull value for each row (see Section 4.3.2). For example, each row of the `PropertyForRent` table has a unique value for the property number `propertyNo`, which uniquely identifies the property represented by that row. The ISO standard supports entity integrity with the **PRIMARY KEY** clause in the **CREATE** and **ALTER TABLE** statements. For example, to define the primary key of the `PropertyForRent` table, we include the following clause:

```
PRIMARY KEY(propertyNo)
```

To define a composite primary key, we specify multiple column names in the PRIMARY KEY clause, separating each by a comma. For example, to define the primary key of the Viewing table, which consists of the columns clientNo and propertyNo, we include the clause:

```
PRIMARY KEY(clientNo, propertyNo)
```

The PRIMARY KEY clause can be specified only once per table. However, it is still possible to ensure uniqueness for any alternate keys in the table using the keyword **UNIQUE**. Every column that appears in a UNIQUE clause must also be declared as NOT NULL. There may be as many UNIQUE clauses per table as required. SQL rejects any INSERT or UPDATE operation that attempts to create a duplicate value within each candidate key (that is, primary key or alternate key). For example, with the Viewing table we could also have written:

```
clientNo      VARCHAR(5)      NOT NULL,
propertyNo    VARCHAR(5)      NOT NULL,
UNIQUE (clientNo, propertyNo)
```

7.2.4 Referential Integrity

A foreign key is a column, or set of columns, that links each row in the child table containing the foreign key to the row of the parent table containing the matching candidate key value. Referential integrity means that, if the foreign key contains a value, that value must refer to an existing, valid row in the parent table (see Section 4.3.3). For example, the branch number column branchNo in the PropertyForRent table links the property to that row in the Branch table where the property is assigned. If the branch number is not null, it must contain a valid value from the column branchNo of the Branch table, or the property is assigned to an invalid branch office.

The ISO standard supports the definition of foreign keys with the FOREIGN KEY clause in the CREATE and ALTER TABLE statements. For example, to define the foreign key branchNo of the PropertyForRent table, we include the clause:

```
FOREIGN KEY(branchNo) REFERENCES Branch
```

SQL rejects any INSERT or UPDATE operation that attempts to create a foreign key value in a child table without a matching candidate key value in the parent table. The action SQL takes for any UPDATE or DELETE operation that attempts to update or delete a candidate key value in the parent table that has some matching rows in the child table is dependent on the **referential action** specified using the ON UPDATE and ON DELETE subclauses of the FOREIGN KEY clause. When the user attempts to delete a row from a parent table, and there are one or more matching rows in the child table, SQL supports four options regarding the action to be taken:

- **CASCADE**: Delete the row from the parent table and automatically delete the matching rows in the child table. Because these deleted rows may themselves have a candidate key that is used as a foreign key in another table, the foreign key rules for these tables are triggered, and so on in a cascading manner.
- **SET NULL**: Delete the row from the parent table and set the foreign key value(s) in the child table to NULL. This option is valid only if the foreign key columns do not have the NOT NULL qualifier specified.

- **SET DEFAULT:** Delete the row from the parent table and set each component of the foreign key in the child table to the specified default value. This option is valid only if the foreign key columns have a **DEFAULT** value specified (see Section 7.3.2).
- **NO ACTION:** Reject the delete operation from the parent table. This is the default setting if the **ON DELETE** rule is omitted.

SQL supports the same options when the candidate key in the parent table is updated. With **CASCADE**, the foreign key value(s) in the child table are set to the new value(s) of the candidate key in the parent table. In the same way, the updates cascade if the updated column(s) in the child table reference foreign keys in another table.

For example, in the *PropertyForRent* table the staff number *staffNo* is a foreign key referencing the *Staff* table. We can specify a deletion rule such that if a staff record is deleted from the *Staff* table, the values of the corresponding *staffNo* column in the *PropertyForRent* table are set to **NULL**:

FOREIGN KEY (*staffNo*) **REFERENCES** *Staff* **ON DELETE SET NULL**

Similarly, the owner number *ownerNo* in the *PropertyForRent* table is a foreign key referencing the *PrivateOwner* table. We can specify an update rule such that if an owner number is updated in the *PrivateOwner* table, the corresponding column(s) in the *PropertyForRent* table are set to the new value:

FOREIGN KEY (*ownerNo*) **REFERENCES** *PrivateOwner* **ON UPDATE CASCADE**

7.2.5 General Constraints

Updates to tables may be constrained by enterprise rules governing the real-world transactions that are represented by the updates (see Section 4.3.4). For example, *DreamHome* may have a rule that prevents a member of staff from managing more than 100 properties at the same time. The ISO standard allows general constraints to be specified using the **CHECK** and **UNIQUE** clauses of the **CREATE** and **ALTER TABLE** statements and the **CREATE ASSERTION** statement. We discussed the **CHECK** and **UNIQUE** clauses earlier in this section. The **CREATE ASSERTION** statement is an integrity constraint that is not directly linked with a table definition. The format of the statement is:

CREATE ASSERTION *AssertionName*
CHECK (*searchCondition*)

This statement is very similar to the **CHECK** clause discussed earlier. However, when a general constraint involves more than one table, it may be preferable to use an **ASSERTION** rather than duplicate the check in each table or place the constraint in an arbitrary table. For example, to define the general constraint that prevents a member of staff from managing more than 100 properties at the same time, we could write:

```
CREATE ASSERTION StaffNotHandlingTooMuch
CHECK (NOT EXISTS (SELECT staffNo
FROM PropertyForRent
GROUP BY staffNo
HAVING COUNT(*) > 100))
```

We show how to use these integrity features in the following section when we examine the **CREATE** and **ALTER TABLE** statements.



7.3 Data Definition

The SQL DDL allows database objects such as schemas, domains, tables, views, and indexes to be created and destroyed. In this section, we briefly examine how to create and destroy schemas, tables, and indexes. We discuss how to create and destroy views in the next section. The ISO standard also allows the creation of character sets, collations, and translations. However, we will not consider these database objects in this book. The interested reader is referred to Cannan and Otten, 1993.

The main SQL data definition language statements are:

CREATE SCHEMA		DROP SCHEMA
CREATE DOMAIN	ALTER DOMAIN	DROP DOMAIN
CREATE TABLE	ALTER TABLE	DROP TABLE
CREATE VIEW		DROP VIEW

These statements are used to create, change, and destroy the structures that make up the conceptual schema. Although not covered by the SQL standard, the following two statements are provided by many DBMSs:

CREATE INDEX	DROP INDEX
--------------	------------

Additional commands are available to the DBA to specify the physical details of data storage; however, we do not discuss these commands here, as they are system-specific.

7.3.1 Creating a Database

The process of creating a database differs significantly from product to product. In multi-user systems, the authority to create a database is usually reserved for the DBA. In a single-user system, a default database may be established when the system is installed and configured and others can be created by the user as and when required. The ISO standard does not specify how databases are created, and each dialect generally has a different approach.

According to the ISO standard, relations and other database objects exist in an **environment**. Among other things, each environment consists of one or more **catalogs**, and each catalog consists of a set of **schemas**. A schema is a named collection of database objects that are in some way related to one another (all the objects in the database are described in one schema or another). The objects in a schema can be tables, views, domains, assertions, collations, translations, and character sets. All the objects in a schema have the same owner and share a number of defaults.

The standard leaves the mechanism for creating and destroying catalogs as implementation-defined, but provides mechanisms for creating and destroying schemas. The schema definition statement has the following (simplified) form:

```
CREATE SCHEMA [Name | AUTHORIZATION CreatorIdentifier]
```

Therefore, if the creator of a schema `SqlTests` is `Smith`, the SQL statement is:

```
CREATE SCHEMA SqlTests AUTHORIZATION Smith;
```

The ISO standard also indicates that it should be possible to specify within this statement the range of facilities available to the users of the schema, but the details of how these privileges are specified are implementation-dependent.

A schema can be destroyed using the DROP SCHEMA statement, which has the following form:

DROP SCHEMA Name [**RESTRICT** | **CASCADE**]

If **RESTRICT** is specified, which is the default if neither qualifier is specified, the schema must be empty or the operation fails. If **CASCADE** is specified, the operation cascades to drop all objects associated with the schema in the order defined previously. If any of these drop operations fail, the DROP SCHEMA fails. The total effect of a DROP SCHEMA with CASCADE can be very extensive and should be carried out only with extreme caution. It should be noted, however, that the CREATE and DROP SCHEMA statements are not always supported.

7.3.2 Creating a Table (CREATE TABLE)

Having created the database structure, we may now create the table structures for the base relations to be stored in the database. This task is achieved using the CREATE TABLE statement, which has the following basic syntax:

```
CREATE TABLE TableName
    {(columnName dataType [NOT NULL] [UNIQUE]
    [DEFAULT defaultOption] [CHECK (searchCondition)] [, . . .]}
    [PRIMARY KEY (listOfColumns),]
    {[UNIQUE (listOfColumns)] [, . . .]}
    {[FOREIGN KEY (listOfForeignKeyColumns)
REFERENCES ParentTableName [(listOfCandidateKeyColumns)]
    [MATCH {PARTIAL | FULL}
    [ON UPDATE referentialAction]
    [ON DELETE referentialAction]] [, . . .]}
    {[CHECK (searchCondition)] [, . . .]}
```

As we discussed in the previous section, this version of the CREATE TABLE statement incorporates facilities for defining referential integrity and other constraints. There is significant variation in the support provided by different dialects for this version of the statement. However, when it is supported, the facilities should be used.

The CREATE TABLE statement creates a table called TableName consisting of one or more columns of the specified *dataType*. The set of permissible data types is described in Section 7.1.2. The optional **DEFAULT** clause can be specified to provide a default value for a particular column. SQL uses this default value whenever an INSERT statement fails to specify a value for the column. Among other values, the *defaultOption* includes literals. The NOT NULL, UNIQUE, and CHECK clauses were discussed in the previous section. The remaining clauses are known as **table constraints** and can optionally be preceded with the clause:

CONSTRAINT ConstraintName

which allows the constraint to be dropped by name using the ALTER TABLE statement (see following).

The **PRIMARY KEY** clause specifies the column or columns that form the primary key for the table. If this clause is available, it should be specified for every table created. By default, NOT NULL is assumed for each column that comprises the primary

key. Only one PRIMARY KEY clause is allowed per table. SQL rejects any INSERT or UPDATE operation that attempts to create a duplicate row within the PRIMARY KEY column(s). In this way, SQL guarantees the uniqueness of the primary key.

The **FOREIGN KEY** clause specifies a foreign key in the (child) table and the relationship it has to another (parent) table. This clause implements referential integrity constraints. The clause specifies the following:

- A *listOfForeignKeyColumns*, the column or columns from the table being created that form the foreign key.
- A REFERENCES subclause, giving the parent table, that is, the table holding the matching candidate key. If the *listOfCandidateKeyColumns* is omitted, the foreign key is assumed to match the primary key of the parent table. In this case, the parent table must have a PRIMARY KEY clause in its CREATE TABLE statement.
- An optional update rule (ON UPDATE) for the relationship that specifies the action to be taken when a candidate key is updated in the parent table that matches a foreign key in the child table. The **referentialAction** can be CASCADE, SET NULL, SET DEFAULT, or NO ACTION. If the ON UPDATE clause is omitted, the default NO ACTION is assumed (see Section 7.2).
- An optional delete rule (ON DELETE) for the relationship that specifies the action to be taken when a row is deleted from the parent table that has a candidate key that matches a foreign key in the child table. The **referentialAction** is the same as for the ON UPDATE rule.
- By default, the referential constraint is satisfied if any component of the foreign key is null or there is a matching row in the parent table. The MATCH option provides additional constraints relating to nulls within the foreign key. If MATCH FULL is specified, the foreign key components must all be null or must all have values. If MATCH PARTIAL is specified, the foreign key components must all be null, or there must be at least one row in the parent table that could satisfy the constraint if the other nulls were correctly substituted. Some authors argue that referential integrity should imply MATCH FULL.

There can be as many FOREIGN KEY clauses as required. The **CHECK** and **CONSTRAINT** clauses allow additional constraints to be defined. If used as a column constraint, the CHECK clause can reference only the column being defined. Constraints are in effect checked after every SQL statement has been executed, although this check can be deferred until the end of the enclosing transaction (see Section 7.5). Example 7.1 demonstrates the potential of this version of the CREATE TABLE statement.

EXAMPLE 7.1 CREATE TABLE

Create the PropertyForRent table using the available features of the CREATE TABLE statement.

```
CREATE DOMAIN OwnerNumber AS VARCHAR(5)
      CHECK (VALUE IN (SELECT ownerNo FROM PrivateOwner));
CREATE DOMAIN StaffNumber AS VARCHAR(5)
      CHECK (VALUE IN (SELECT staffNo FROM Staff));
CREATE DOMAIN BranchNumber AS CHAR(4)
      CHECK (VALUE IN (SELECT branchNo FROM Branch));
CREATE DOMAIN PropertyNumber AS VARCHAR(5);
```

```

CREATE DOMAIN Street AS VARCHAR(25);
CREATE DOMAIN City AS VARCHAR(15);
CREATE DOMAIN Postcode AS VARCHAR(8);
CREATE DOMAIN PropertyType AS CHAR(1)
    CHECK(VALUE IN ('B', 'C', 'D', 'E', 'F', 'M', 'S'));
CREATE DOMAIN PropertyRooms AS SMALLINT;
    CHECK(VALUE BETWEEN 1 AND 15);
CREATE DOMAIN PropertyRent AS DECIMAL(6,2)
    CHECK(VALUE BETWEEN 0 AND 9999.99);
CREATE TABLE PropertyForRent(
    propertyNo      PropertyNumber      NOT NULL,
    street          Street              NOT NULL,
    city            City                NOT NULL,
    postcode        PostCode,
    type            PropertyType        NOT NULL DEFAULT 'F',
    rooms           PropertyRooms       NOT NULL DEFAULT 4,
    rent            PropertyRent        NOT NULL DEFAULT 600,
    ownerNo         OwnerNumber        NOT NULL,
    staffNo         StaffNumber
    CONSTRAINT StaffNotHandlingTooMuch
        CHECK (NOT EXISTS (SELECT staffNo
                            FROM PropertyForRent
                            GROUP BY staffNo
                            HAVING COUNT(*) > 100)),
    branchNo        BranchNumber
    PRIMARY KEY (propertyNo),
    FOREIGN KEY (staffNo) REFERENCES Staff ON DELETE SET NULL
        ON UPDATE CASCADE,
    FOREIGN KEY (ownerNo) REFERENCES PrivateOwner ON DELETE NO
        ACTION ON UPDATE CASCADE,
    FOREIGN KEY (branchNo) REFERENCES Branch ON DELETE NO
        ACTION ON UPDATE CASCADE);

```

A default value of 'F' for 'Flat' has been assigned to the property type column type. A **CONSTRAINT** for the staff number column has been specified to ensure that a member of staff does not handle too many properties. The constraint checks whether the number of properties the staff member currently handles is more than 100.

The primary key is the property number, `propertyNo`. SQL automatically enforces uniqueness on this column. The staff number, `staffNo`, is a foreign key referencing the `Staff` table. A deletion rule has been specified such that, if a record is deleted from the `Staff` table, the corresponding values of the `staffNo` column in the `PropertyForRent` table are set to `NULL`. Additionally, an update rule has been specified such that if a staff number is updated in the `Staff` table, the corresponding values in the `staffNo` column in the `PropertyForRent` table are updated accordingly. The owner number, `ownerNo`, is a foreign key referencing the `PrivateOwner` table. A deletion rule of **NO ACTION** has been specified to prevent deletions from the `PrivateOwner` table if there are matching `ownerNo` values in the `PropertyForRent` table. An update rule of **CASCADE** has been specified such that, if an owner number is

updated, the corresponding values in the `ownerNo` column in the `PropertyForRent` table are set to the new value. The same rules have been specified for the `branchNo` column. In all FOREIGN KEY constraints, because the *listOfCandidateKeyColumns* has been omitted, SQL assumes that the foreign keys match the primary keys of the respective parent tables.

Note, we have not specified NOT NULL for the staff number column `staffNo`, because there may be periods of time when there is no member of staff allocated to manage the property (for example, when the property is first registered). However, the other foreign key columns—`ownerNo` (the owner number) and `branchNo` (the branch number)—must be specified at all times.

7.3.3 Changing a Table Definition (ALTER TABLE)

The ISO standard provides an ALTER TABLE statement for changing the structure of a table once it has been created. The definition of the ALTER TABLE statement in the ISO standard consists of six options to:

- add a new column to a table;
- drop a column from a table;
- add a new table constraint;
- drop a table constraint;
- set a default for a column;
- drop a default for a column.

The basic format of the statement is:

```
ALTER TABLE TableName
[ADD [COLUMN] columnName dataType [NOT NULL] [UNIQUE]
[DEFAULT defaultOption] [CHECK (searchCondition)]]
[DROP [COLUMN] columnName [RESTRICT | CASCADE]]
[ADD [CONSTRAINT [ConstraintName]] tableConstraintDefinition]
[DROP CONSTRAINT ConstraintName [RESTRICT | CASCADE]]
[ALTER [COLUMN] SET DEFAULT defaultOption]
[ALTER [COLUMN] DROP DEFAULT]
```

where the parameters are as defined for the CREATE TABLE statement in the previous section. A *tableConstraintDefinition* is one of the clauses: PRIMARY KEY, UNIQUE, FOREIGN KEY, or CHECK. The ADD COLUMN clause is similar to the definition of a column in the CREATE TABLE statement. The DROP COLUMN clause specifies the name of the column to be dropped from the table definition, and has an optional qualifier that specifies whether the DROP action is to cascade or not:

- **RESTRICT**: The DROP operation is rejected if the column is referenced by another database object (for example, by a view definition). This is the default setting.
- **CASCADE**: The DROP operation proceeds and automatically drops the column from any database objects it is referenced by. This operation cascades, so that if a column is dropped from a referencing object, SQL checks whether *that* column is referenced by any other object and drops it from there if it is, and so on.

EXAMPLE 7.2 ALTER TABLE

(a) Change the Staff table by removing the default of 'Assistant' for the position column and setting the default for the sex column to female ('F').

```
ALTER TABLE Staff
  ALTER position DROP DEFAULT;
ALTER TABLE Staff
  ALTER sex SET DEFAULT 'F';
```

(b) Change the PropertyForRent table by removing the constraint that staff are not allowed to handle more than 100 properties at a time. Change the Client table by adding a new column representing the preferred number of rooms.

```
ALTER TABLE PropertyForRent
  DROP CONSTRAINT StaffNotHandlingTooMuch;
ALTER TABLE Client
  ADD prefNoRooms PropertyRooms;
```

The ALTER TABLE statement is not available in all dialects of SQL. In some dialects, the ALTER TABLE statement cannot be used to remove an existing column from a table. In such cases, if a column is no longer required, the column could simply be ignored but kept in the table definition. If, however, you wish to remove the column from the table, you must:

- upload all the data from the table;
- remove the table definition using the DROP TABLE statement;
- redefine the new table using the CREATE TABLE statement;
- reload the data back into the new table.

The upload and reload steps are typically performed with special-purpose utility programs supplied with the DBMS. However, it is possible to create a temporary table and use the INSERT . . . SELECT statement to load the data from the old table into the temporary table and then from the temporary table into the new table.

7.3.4 Removing a Table (DROP TABLE)

Over time, the structure of a database will change; new tables will be created and some tables will no longer be needed. We can remove a redundant table from the database using the DROP TABLE statement, which has the format:

```
DROP TABLE TableName [RESTRICT | CASCADE]
```

For example, to remove the PropertyForRent table we use the command:

```
DROP TABLE PropertyForRent;
```

Note, however, that this command removes not only the named table, but also all the rows within it. To simply remove the rows from the table but retain the table structure, use the DELETE statement instead (see Section 6.3.10). The DROP TABLE statement allows you to specify whether the DROP action is to be cascaded:

- **RESTRICT**: The DROP operation is rejected if there are any other objects that depend for their existence upon the continued existence of the table to be dropped.

- **CASCADE**: The **DROP** operation proceeds and SQL automatically drops all dependent objects (and objects dependent on these objects).

The total effect of a **DROP TABLE** with **CASCADE** can be very extensive and should be carried out only with extreme caution. One common use of **DROP TABLE** is to correct mistakes made when creating a table. If a table is created with an incorrect structure, **DROP TABLE** can be used to delete the newly created table and start again.

7.3.5 Creating an Index (**CREATE INDEX**)

An index is a structure that provides accelerated access to the rows of a table based on the values of one or more columns (see Appendix F for a discussion of indexes and how they may be used to improve the efficiency of data retrievals). The presence of an index can significantly improve the performance of a query. However, as indexes may be updated by the system every time the underlying tables are updated, additional overheads may be incurred. Indexes are usually created to satisfy particular search criteria after the table has been in use for some time and has grown in size. The creation of indexes is *not* standard SQL. However, most dialects support at least the following capabilities:

```
CREATE [UNIQUE] INDEX IndexName  
ON TableName (columnName [ASC | DESC] [, . . .])
```

The specified columns constitute the index key and should be listed in major to minor order. Indexes can be created only on base tables *not* on views. If the **UNIQUE** clause is used, uniqueness of the indexed column or combination of columns will be enforced by the DBMS. This is certainly required for the primary key and possibly for other columns as well (for example, for alternate keys). Although indexes can be created at any time, we may have a problem if we try to create a unique index on a table with records in it, because the values stored for the indexed column(s) may already contain duplicates. Therefore, it is good practice to create unique indexes, at least for primary key columns, when the base table is created and the DBMS does not automatically enforce primary key uniqueness.

For the **Staff** and **PropertyForRent** tables, we may want to create at least the following indexes:

```
CREATE UNIQUE INDEX StaffNoInd ON Staff (staffNo);  
CREATE UNIQUE INDEX PropertyNoInd ON PropertyForRent (propertyNo);
```

For each column, we may specify that the order is ascending (**ASC**) or descending (**DESC**), with **ASC** being the default setting. For example, if we create an index on the **PropertyForRent** table as:

```
CREATE INDEX RentInd ON PropertyForRent (city, rent);
```

then an index called **RentInd** is created for the **PropertyForRent** table. Entries will be in alphabetical order by city and then by rent within each city.

7.3.6 Removing an Index (**DROP INDEX**)

If we create an index for a base table and later decide that it is no longer needed, we can use the **DROP INDEX** statement to remove the index from the database. **DROP INDEX** has the following format:

DROP INDEX IndexName

The following statement will remove the index created in the previous example:

DROP INDEX RentInd;

7.4 Views

Recall from Section 4.4 the definition of a view:

View

The dynamic result of one or more relational operations operating on the base relations to produce another relation. A view is a *virtual relation* that does not necessarily exist in the database but can be produced upon request by a particular user, at the time of request.

To the database user, a view appears just like a real table, with a set of named columns and rows of data. However, unlike a base table, a view does not necessarily exist in the database as a stored set of data values. Instead, a view is defined as a query on one or more base tables or views. The DBMS stores the definition of the view in the database. When the DBMS encounters a reference to a view, one approach is to look up this definition and translate the request into an equivalent request against the source tables of the view and then perform the equivalent request. This merging process, called **view resolution**, is discussed in Section 7.4.3. An alternative approach, called **view materialization**, stores the view as a temporary table in the database and maintains the currency of the view as the underlying base tables are updated. We discuss view materialization in Section 7.4.8. First, we examine how to create and use views.

7.4.1 Creating a View (CREATE VIEW)

The format of the CREATE VIEW statement is:

```
CREATE VIEW ViewName [(newColumnName [, . . . ])]
AS subselect [WITH [CASCADED | LOCAL] CHECK OPTION]
```

A view is defined by specifying an SQL SELECT statement. A name may optionally be assigned to each column in the view. If a list of column names is specified, it must have the same number of items as the number of columns produced by the *subselect*. If the list of column names is omitted, each column in the view takes the name of the corresponding column in the *subselect* statement. The list of column names must be specified if there is any ambiguity in the name for a column. This may occur if the *subselect* includes calculated columns, and the AS subclause has not been used to name such columns, or it produces two columns with identical names as the result of a join.

The *subselect* is known as the **defining query**. If WITH CHECK OPTION is specified, SQL ensures that if a row fails to satisfy the WHERE clause of the defining query of the view, it is not added to the underlying base table of the view (see Section 7.4.6). It should be noted that to create a view successfully, you must have SELECT privilege on all the tables referenced in the subselect and USAGE privilege on any domains used in referenced columns. These privileges are discussed further in Section 7.6. Although all views are created in the same way, in practice different types of views are used for different purposes. We illustrate the different types of views with examples.

EXAMPLE 7.3 Create a horizontal view

Create a view so that the manager at branch B003 can see the details only for staff who work in his or her branch office.

A horizontal view restricts a user’s access to selected rows of one or more tables.

```
CREATE VIEW Manager3Staff
AS SELECT *
FROM Staff
WHERE branchNo = 'B003';
```

This creates a view called Manager3Staff with the same column names as the Staff table but containing only those rows where the branch number is B003. (Strictly speaking, the branchNo column is unnecessary and could have been omitted from the definition of the view, as all entries have branchNo = 'B003'.) If we now execute this statement:

```
SELECT * FROM Manager3Staff;
```

we get the result table shown in Table 7.3. To ensure that the branch manager can see only these rows, the manager should not be given access to the base table Staff. Instead, the manager should be given access permission to the view Manager3Staff. This, in effect, gives the branch manager a customized view of the Staff table, showing only the staff at his or her own branch. We discuss access permissions in Section 7.6.

TABLE 7.3 Data for view Manager3Staff.

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003

EXAMPLE 7.4 Create a vertical view

Create a view of the staff details at branch B003 that excludes salary information, so that only managers can access the salary details for staff who work at their branch.

A vertical view restricts a user’s access to selected columns of one or more tables.

```
CREATE VIEW Staff3
AS SELECT staffNo, fName, lName, position, sex
FROM Staff
WHERE branchNo = 'B003';
```

Note that we could rewrite this statement to use the Manager3Staff view instead of the Staff table, thus:

```
CREATE VIEW Staff3
AS SELECT staffNo, fName, lName, position, sex
FROM Manager3Staff;
```

Either way, this creates a view called Staff3 with the same columns as the Staff table, but excluding the salary, DOB, and branchNo columns. If we list this view, we get the result table

shown in Table 7.4. To ensure that only the branch manager can see the salary details, staff at branch B003 should not be given access to the base table `Staff` or the view `Manager3Staff`. Instead, they should be given access permission to the view `Staff3`, thereby denying them access to sensitive salary data.

Vertical views are commonly used where the data stored in a table is used by various users or groups of users. They provide a private table for these users composed only of the columns they need.

TABLE 7.4 Data for view `Staff3`.

staffNo	fName	lName	position	sex
SG37	Ann	Beech	Assistant	F
SG14	David	Ford	Supervisor	M
SG5	Susan	Brand	Manager	F

EXAMPLE 7.5 Grouped and joined views

Create a view of staff who manage properties for rent, which includes the branch number they work at, their staff number, and the number of properties they manage (see Example 6.27).

```
CREATE VIEW StaffPropCnt (branchNo, staffNo, cnt)
AS SELECT s.branchNo, s.staffNo, COUNT(*)
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.branchNo, s.staffNo;
```

This example gives the data shown in Table 7.5. It illustrates the use of a subselect containing a `GROUP BY` clause (giving a view called a **grouped view**), and containing multiple tables (giving a view called a **joined view**). One of the most frequent reasons for using views is to simplify multi-table queries. Once a joined view has been defined, we can often use a simple single-table query against the view for queries that would otherwise require a multi-table join. Note that we have to name the columns in the definition of the view because of the use of the unqualified aggregate function `COUNT` in the subselect.

TABLE 7.5 Data for view `StaffPropCnt`.

branchNo	staffNo	cnt
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1

7.4.2 Removing a View (DROP VIEW)

A view is removed from the database with the `DROP VIEW` statement:

```
DROP VIEW ViewName [RESTRICT | CASCADE]
```

DROP VIEW causes the definition of the view to be deleted from the database. For example, we could remove the Manager3Staff view using the following statement:

```
DROP VIEW Manager3Staff;
```

If CASCADE is specified, DROP VIEW deletes all related dependent objects; in other words, all objects that reference the view. This means that DROP VIEW also deletes any views that are defined on the view being dropped. If RESTRICT is specified and there are any other objects that depend for their existence on the continued existence of the view being dropped, the command is rejected. The default setting is RESTRICT.

7.4.3 View Resolution

Having considered how to create and use views, we now look more closely at how a query on a view is handled. To illustrate the process of **view resolution**, consider the following query, which counts the number of properties managed by each member of staff at branch office B003. This query is based on the StaffPropCnt view of Example 7.5:

```
SELECT staffNo, cnt  
FROM StaffPropCnt  
WHERE branchNo = 'B003'  
ORDER BY staffNo;
```

View resolution merges the example query with the defining query of the StaffPropCnt view as follows:

- (1) The view column names in the SELECT list are translated into their corresponding column names in the defining query. This gives:

```
SELECT s.staffNo AS staffNo, COUNT(*) AS cnt
```

- (2) View names in the FROM clause are replaced with the corresponding FROM lists of the defining query:

```
FROM Staff s, PropertyForRent p
```

- (3) The WHERE clause from the user query is combined with the WHERE clause of the defining query using the logical operator AND, thus:

```
WHERE s.staffNo = p.staffNo AND branchNo = 'B003'
```

- (4) The GROUP BY and HAVING clauses are copied from the defining query. In this example, we have only a GROUP BY clause:

```
GROUP BY s.branchNo, s.staffNo
```

- (5) Finally, the ORDER BY clause is copied from the user query with the view column name translated into the defining query column name:

```
ORDER BY s.staffNo
```

- (6) The final merged query becomes:

```
SELECT s.staffNo AS staffNo, COUNT(*) AS cnt  
FROM Staff s, PropertyForRent p
```

```

WHERE s.staffNo = p.staffNo AND branchNo = 'B003'
GROUP BY s.branchNo, s.staffNo
ORDER BY s.staffNo;

```

This gives the result table shown in Table 7.6.

TABLE 7.6 Result table after view resolution.

staffNo	cnt
SG14	1
SG37	2

7.4.4 Restrictions on Views

The ISO standard imposes several important restrictions on the creation and use of views, although there is considerable variation among dialects.

- If a column in the view is based on an aggregate function, then the column may appear only in **SELECT** and **ORDER BY** clauses of queries that access the view. In particular, such a column may not be used in a **WHERE** clause and may not be an argument to an aggregate function in any query based on the view. For example, consider the view `StaffPropCnt` of Example 7.5, which has a column `cnt` based on the aggregate function `COUNT`. The following query would fail:

```

SELECT COUNT(cnt)
FROM StaffPropCnt;

```

because we are using an aggregate function on the column `cnt`, which is itself based on an aggregate function. Similarly, the following query would also fail:

```

SELECT *
FROM StaffPropCnt
WHERE cnt > 2;

```

because we are using the view column, `cnt`, derived from an aggregate function, on the left-hand side of a **WHERE** clause.

- A grouped view may never be joined with a base table or a view. For example, the `StaffPropCnt` view is a grouped view, so any attempt to join this view with another table or view fails.

7.4.5 View Updatability

All updates to a base table are immediately reflected in all views that encompass that base table. Similarly, we may expect that if a view is updated, the base table(s) will reflect that change. However, consider again the view `StaffPropCnt` of Example 7.5. Consider what would happen if we tried to insert a record that showed that at branch `B003`, staff member `SG5` manages two properties, using the following insert statement:

```

INSERT INTO StaffPropCnt
VALUES ('B003', 'SG5', 2);

```

We have to insert two records into the `PropertyForRent` table showing which properties staff member `SG5` manages. However, we do not know which properties they

are; all we know is that this member of staff manages two properties. In other words, we do not know the corresponding primary key values for the `PropertyForRent` table. If we change the definition of the view and replace the count with the actual property numbers as follows:

```
CREATE VIEW StaffPropList (branchNo, staffNo, propertyNo)
AS SELECT s.branchNo, s.staffNo, p.propertyNo
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo;
```

and we try to insert the record:

```
INSERT INTO StaffPropList
VALUES ('B003', 'SG5', 'PG19');
```

there is still a problem with this insertion, because we specified in the definition of the `PropertyForRent` table that all columns except postcode and `staffNo` were not allowed to have nulls (see Example 7.1). However, as the `StaffPropList` view excludes all columns from the `PropertyForRent` table except the property number, we have no way of providing the remaining nonnull columns with values.

The ISO standard specifies the views that must be updatable in a system that conforms to the standard. The definition given in the ISO standard is that a view is updatable if and only if:

- `DISTINCT` is not specified; that is, duplicate rows must not be eliminated from the query results.
- Every element in the `SELECT` list of the defining query is a column name (rather than a constant, expression, or aggregate function) and no column name appears more than once.
- The `FROM` clause specifies only one table; that is, the view must have a single source table for which the user has the required privileges. If the source table is itself a view, then that view must satisfy these conditions. This, therefore, excludes any views based on a join, union (`UNION`), intersection (`INTERSECT`), or difference (`EXCEPT`).
- The `WHERE` clause does not include any nested `SELECT`s that reference the table in the `FROM` clause.
- There is no `GROUP BY` or `HAVING` clause in the defining query.

In addition, every row that is added through the view must not violate the integrity constraints of the base table. For example, if a new row is added through a view, columns that are not included in the view are set to null, but this must not violate a `NOT NULL` integrity constraint in the base table. The basic concept behind these restrictions is as follows:

Updatable view

For a view to be updatable, the DBMS must be able to trace any row or column back to its row or column in the source table.

7.4.6 WITH CHECK OPTION

Rows exist in a view, because they satisfy the `WHERE` condition of the defining query. If a row is altered such that it no longer satisfies this condition, then it will

disappear from the view. Similarly, new rows will appear within the view when an insert or update on the view causes them to satisfy the WHERE condition. The rows that enter or leave a view are called **migrating rows**.

Generally, the WITH CHECK OPTION clause of the CREATE VIEW statement prohibits a row from migrating out of the view. The optional qualifiers LOCAL/CASCADED are applicable to view hierarchies, that is, a view that is derived from another view. In this case, if WITH LOCAL CHECK OPTION is specified, then any row insert or update on this view, and on any view directly or indirectly defined on this view, must not cause the row to disappear from the view, unless the row also disappears from the underlying derived view/table. If the WITH CASCADED CHECK OPTION is specified (the default setting), then any row insert or update on this view and on any view directly or indirectly defined on this view must not cause the row to disappear from the view.

This feature is so useful that it can make working with views more attractive than working with the base tables. When an INSERT or UPDATE statement on the view violates the WHERE condition of the defining query, the operation is rejected. This behavior enforces constraints on the database and helps preserve database integrity. The WITH CHECK OPTION can be specified only for an updatable view, as defined in the previous section.

EXAMPLE 7.6 WITH CHECK OPTION

Consider again the view created in Example 7.3:

```
CREATE VIEW Manager3Staff
AS SELECT *
FROM Staff
WHERE branchNo = 'B003'
WITH CHECK OPTION;
```

with the virtual table shown in Table 7.3. If we now attempt to update the branch number of one of the rows from B003 to B005, for example:

```
UPDATE Manager3Staff
SET branchNo = 'B005'
WHERE staffNo = 'SG37';
```

then the specification of the WITH CHECK OPTION clause in the definition of the view prevents this from happening, as it would cause the row to migrate from this horizontal view. Similarly, if we attempt to insert the following row through the view:

```
INSERT INTO Manager3Staff
VALUES('SL15', 'Mary', 'Black', 'Assistant', 'F', DATE'1967-06-21', 8000, 'B002');
```

then the specification of WITH CHECK OPTION would prevent the row from being inserted into the underlying Staff table and immediately disappearing from this view (as branch B002 is not part of the view).

Now consider the situation where Manager3Staff is defined not on Staff directly but on another view of Staff:

```
CREATE VIEW LowSalary
AS SELECT *
FROM Staff
WHERE salary > 9000;

CREATE VIEW HighSalary
AS SELECT *
FROM LowSalary
WHERE salary > 10000
WITH LOCAL CHECK OPTION;

CREATE VIEW Manager3Staff
AS SELECT *
FROM HighSalary
WHERE branchNo = 'B003';
```

If we now attempt the following update on Manager3Staff:

```
UPDATE Manager3Staff
SET salary = 9500
WHERE staffNo = 'SG37';
```

then this update would fail: although the update would cause the row to disappear from the view HighSalary, the row would not disappear from the table LowSalary that HighSalary is derived from. However, if instead the update tried to set the salary to 8000, then the update would succeed, as the row would no longer be part of LowSalary. Alternatively, if the view HighSalary had specified WITH CASCADED CHECK OPTION, then setting the salary to either 9500 or 8000 would be rejected, because the row would disappear from HighSalary. Therefore, to ensure that anomalies like this do not arise, each view should normally be created using the WITH CASCADED CHECK OPTION.

7.4.7 Advantages and Disadvantages of Views

Restricting some users' access to views has potential advantages over allowing users direct access to the base tables. Unfortunately, views in SQL also have disadvantages. In this section we briefly review the advantages and disadvantages of views in SQL as summarized in Table 7.7.

TABLE 7.7 Summary of advantages/disadvantages of views in SQL.

ADVANTAGES	DISADVANTAGES
Data independence	Update restriction
Currency	Structure restriction
Improved security	Performance
Reduced complexity	
Convenience	
Customization	
Data integrity	

Advantages

In the case of a DBMS running on a standalone PC, views are usually a convenience, defined to simplify database requests. However, in a multi-user DBMS, views play a central role in defining the structure of the database and enforcing security and integrity. The major advantages of views are described next.

Data independence A view can present a consistent, unchanging picture of the structure of the database, even if the underlying source tables are changed (for example, if columns added or removed, relationships changed, tables split,

restructured, or renamed). If columns are added or removed from a table, and these columns are not required by the view, then the definition of the view need not change. If an existing table is rearranged or split up, a view may be defined so that users can continue to see the old table. In the case of splitting a table, the old table can be recreated by defining a view from the join of the new tables, provided that the split is done in such a way that the original table can be reconstructed. We can ensure that this is possible by placing the primary key in both of the new tables. Thus, if we originally had a `Client` table of the following form:

```
Client (clientNo, fName, lName, telNo, prefType, maxRent, eMail)
```

we could reorganize it into two new tables:

```
ClientDetails (clientNo, fName, lName, telNo, eMail)
```

```
ClientReqs (clientNo, prefType, maxRent)
```

Users and applications could still access the data using the old table structure, which would be recreated by defining a view called `Client` as the natural join of `ClientDetails` and `ClientReqs`, with `clientNo` as the join column:

```
CREATE VIEW Client
AS SELECT cd.clientNo, fName, lName, telNo, prefType, maxRent, eMail
FROM ClientDetails cd, ClientReqs cr
WHERE cd.clientNo = cr.clientNo;
```

Currency Changes to any of the base tables in the defining query are immediately reflected in the view.

Improved security Each user can be given the privilege to access the database only through a small set of views that contain the data appropriate for that user, thus restricting and controlling each user's access to the database.

Reduced complexity A view can simplify queries, by drawing data from several tables into a single table, thereby transforming multi-table queries into single-table queries.

Convenience Views can provide greater convenience to users as users are presented with only that part of the database that they need to see. This also reduces the complexity from the user's point of view.

Customization Views provide a method to customize the appearance of the database, so that the same underlying base tables can be seen by different users in different ways.

Data integrity If the `WITH CHECK OPTION` clause of the `CREATE VIEW` statement is used, then SQL ensures that no row that fails to satisfy the `WHERE` clause of the defining query is ever added to any of the underlying base table(s) through the view, thereby ensuring the integrity of the view.

Disadvantages

Although views provide many significant benefits, there are also some disadvantages with SQL views.

Update restriction In Section 7.4.5 we showed that, in some cases, a view cannot be updated.

Structure restriction The structure of a view is determined at the time of its creation. If the defining query was of the form `SELECT * FROM . . .`, then the `*` refers to the columns of the base table present when the view is created. If columns are subsequently added to the base table, then these columns will not appear in the view, unless the view is dropped and recreated.

Performance There is a performance penalty to be paid when using a view. In some cases, this will be negligible; in other cases, it may be more problematic. For example, a view defined by a complex, multi-table query may take a long time to process, as the view resolution must join the tables together *every time the view is accessed*. View resolution requires additional computer resources. In the next section we briefly discuss an alternative approach to maintaining views that attempts to overcome this disadvantage.

7.4.8 View Materialization

In Section 7.4.3 we discussed one approach to handling queries based on a view, in which the query is modified into a query on the underlying base tables. One disadvantage with this approach is the time taken to perform the view resolution, particularly if the view is accessed frequently. An alternative approach, called **view materialization**, is to store the view as a temporary table in the database when the view is first queried. Thereafter, queries based on the materialized view can be much faster than recomputing the view each time. The speed difference may be critical in applications where the query rate is high and the views are complex, so it is not practical to recompute the view for every query.

Materialized views are useful in new applications such as data warehousing, replication servers, data visualization, and mobile systems. Integrity constraint checking and query optimization can also benefit from materialized views. The difficulty with this approach is maintaining the currency of the view while the base table(s) are being updated. The process of updating a materialized view in response to changes to the underlying data is called **view maintenance**. The basic aim of view maintenance is to apply only those changes necessary to the view to keep it current. As an indication of the issues involved, consider the following view:

```
CREATE VIEW StaffPropRent (staffNo)
AS SELECT DISTINCT staffNo
   FROM PropertyForRent
   WHERE branchNo = 'B003' AND rent > 400;
```

TABLE 7.8
Data for view
StaffPropRent.

staffNo
SG37
SG14

with the data shown in Table 7.8. If we were to insert a row into the `PropertyForRent` table with a `rent` ≤ 400 , then the view would be unchanged. If we were to insert the row ('PG24', . . . , 550, 'CO40', 'SG19', 'B003') into the `PropertyForRent` table, then the row should also appear within the materialized view. However, if we were to insert the row ('PG54', . . . , 450, 'CO89', 'SG37', 'B003') into the `PropertyForRent` table, then no new row need be added to the materialized view, because there is a row for SG37 already. Note that in these three cases the decision whether to insert

the row into the materialized view can be made without access to the underlying `PropertyForRent` table.

If we now wished to delete the new row ('PG24', . . . , 550, 'CO40', 'SG19', 'B003') from the `PropertyForRent` table, then the row should also be deleted from the materialized view. However, if we wished to delete the new row ('PG54', . . . , 450, 'CO89', 'SG37', 'B003') from the `PropertyForRent` table, then the row corresponding to SG37 should not be deleted from the materialized view, owing to the existence of the underlying base row corresponding to property PG21. In these two cases, the decision on whether to delete or retain the row in the materialized view requires access to the underlying base table `PropertyForRent`. For a more complete discussion of materialized views, the interested reader is referred to Gupta and Mumick, 1999.

7.5 Transactions

The ISO standard defines a transaction model based on two SQL statements: `COMMIT` and `ROLLBACK`. Most, but not all, commercial implementations of SQL conform to this model, which is based on IBM's DB2 DBMS. A transaction is a logical unit of work consisting of one or more SQL statements that is guaranteed to be atomic with respect to recovery. The standard specifies that an SQL transaction automatically begins with a **transaction-initiating** SQL statement executed by a user or program (for example, `SELECT`, `INSERT`, `UPDATE`). Changes made by a transaction are not visible to other concurrently executing transactions until the transaction completes. A transaction can complete in one of four ways:

- A `COMMIT` statement ends the transaction successfully, making the database changes permanent. A new transaction starts after `COMMIT` with the next transaction-initiating statement.
- A `ROLLBACK` statement aborts the transaction, backing out any changes made by the transaction. A new transaction starts after `ROLLBACK` with the next transaction-initiating statement.
- For programmatic SQL (see Appendix I), successful program termination ends the final transaction successfully, even if a `COMMIT` statement has not been executed.
- For programmatic SQL, abnormal program termination aborts the transaction.

SQL transactions cannot be nested (see Section 22.4). The `SET TRANSACTION` statement allows the user to configure certain aspects of the transaction. The basic format of the statement is:

```
SET TRANSACTION
[READ ONLY | READ WRITE] |
[ISOLATION LEVEL READ UNCOMMITTED | READ COMMITTED |
REPEATABLE READ | SERIALIZABLE]
```

The `READ ONLY` and `READ WRITE` qualifiers indicate whether the transaction is read-only or involves both read and write operations. The default is `READ WRITE` if neither qualifier is specified (unless the isolation level is `READ UNCOMMITTED`). Perhaps confusingly, `READ ONLY` allows a transaction to issue `INSERT`, `UPDATE`, and `DELETE` statements against temporary tables (but only temporary tables).

TABLE 7.9 Violations of serializability permitted by isolation levels.

ISOLATION LEVEL	DIRTY READ	NONREPEATABLE READ	PHANTOM READ
READ UNCOMMITTED	Y	Y	Y
READ COMMITTED	N	Y	Y
REPEATABLE READ	N	N	Y
SERIALIZABLE	N	N	N

The *isolation level* indicates the degree of interaction that is allowed from other transactions during the execution of the transaction. Table 7.9 shows the violations of serializability allowed by each isolation level against the following three preventable phenomena:

- *Dirty read.* A transaction reads data that has been written by another as yet uncommitted transaction.
- *Nonrepeatable read.* A transaction rereads data that it has previously read, but another committed transaction has modified or deleted the data in the intervening period.
- *Phantom read.* A transaction executes a query that retrieves a set of rows satisfying a certain search condition. When the transaction re-executes the query at a later time, additional rows are returned that have been inserted by another committed transaction in the intervening period.

Only the **SERIALIZABLE** isolation level is safe, that is, generates serializable schedules. The remaining isolation levels require a mechanism to be provided by the DBMS that can be used by the programmer to ensure serializability. Chapter 22 provides additional information on transactions and serializability.

7.5.1 Immediate and Deferred Integrity Constraints

In some situations, we do not want integrity constraints to be checked immediately—that is, after every SQL statement has been executed—but instead at transaction commit. A constraint may be defined as **INITIALLY IMMEDIATE** or **INITIALLY DEFERRED**, indicating which mode the constraint assumes at the start of each transaction. In the former case, it is also possible to specify whether the mode can be changed subsequently using the qualifier **[NOT] DEFERRABLE**. The default mode is **INITIALLY IMMEDIATE**.

The **SET CONSTRAINTS** statement is used to set the mode for specified constraints for the current transaction. The format of this statement is:

```
SET CONSTRAINTS
  {ALL | constraintName [, . . .]} {DEFERRED | IMMEDIATE}
```



7.6 Discretionary Access Control

In Section 2.4 we stated that a DBMS should provide a mechanism to ensure that only authorized users can access the database. Modern DBMSs typically provide one or both of the following authorization mechanisms:

- *Discretionary access control.* Each user is given appropriate access rights (or *privileges*) on specific database objects. Typically users obtain certain privileges when they create an object and can pass some or all of these privileges to other users at their discretion. Although flexible, this type of authorization mechanism can be circumvented by a devious unauthorized user tricking an authorized user into revealing sensitive data.
- *Mandatory access control.* Each database object is assigned a certain *classification level* (for example, Top Secret, Secret, Confidential, Unclassified) and each *subject* (for example, users or programs) is given a designated *clearance level*. The classification levels form a strict ordering (Top Secret > Secret > Confidential > Unclassified) and a subject requires the necessary clearance to read or write a database object. This type of multilevel security mechanism is important for certain government, military, and corporate applications. The most commonly used mandatory access control model is known as Bell–LaPadula (Bell and La Padula, 1974), which we discuss further in Chapter 20.

SQL supports only discretionary access control through the GRANT and REVOKE statements. The mechanism is based on the concepts of **authorization identifiers**, **ownership**, and **privileges**, as we now discuss.

Authorization identifiers and ownership

An authorization identifier is a normal SQL identifier that is used to establish the identity of a user. Each database user is assigned an authorization identifier by the DBA. Usually, the identifier has an associated password, for obvious security reasons. Every SQL statement that is executed by the DBMS is performed on behalf of a specific user. The authorization identifier is used to determine which database objects the user may reference and what operations may be performed on those objects.

Each object that is created in SQL has an owner. The owner is identified by the authorization identifier defined in the AUTHORIZATION clause of the schema to which the object belongs (see Section 7.3.1). The owner is initially the only person who may know of the existence of the object and, consequently, perform any operations on the object.

Privileges

Privileges are the actions that a user is permitted to carry out on a given base table or view. The privileges defined by the ISO standard are:

- SELECT—the privilege to retrieve data from a table;
- INSERT—the privilege to insert new rows into a table;
- UPDATE—the privilege to modify rows of data in a table;
- DELETE—the privilege to delete rows of data from a table;
- REFERENCES—the privilege to reference columns of a named table in integrity constraints;
- USAGE—the privilege to use domains, collations, character sets, and translations. We do not discuss collations, character sets, and translations in this book; the interested reader is referred to Cannan and Otten, 1993.

The INSERT and UPDATE privileges can be restricted to specific columns of the table, allowing changes to these columns but disallowing changes to any other

column. Similarly, the REFERENCES privilege can be restricted to specific columns of the table, allowing these columns to be referenced in constraints, such as check constraints and foreign key constraints, when creating another table, but disallowing others from being referenced.

When a user creates a table using the CREATE TABLE statement, he or she automatically becomes the owner of the table and receives full privileges for the table. Other users initially have no privileges on the newly created table. To give them access to the table, the owner must explicitly grant them the necessary privileges using the GRANT statement.

When a user creates a view with the CREATE VIEW statement, he or she automatically becomes the owner of the view, but does not necessarily receive full privileges on the view. To create the view, a user must have SELECT privilege on all the tables that make up the view and REFERENCES privilege on the named columns of the view. However, the view owner gets INSERT, UPDATE, and DELETE privileges only if he or she holds these privileges for every table in the view.

7.6.1 Granting Privileges to Other Users (GRANT)

The GRANT statement is used to grant privileges on database objects to specific users. Normally the GRANT statement is used by the owner of a table to give other users access to the data. The format of the GRANT statement is:

```
GRANT      {PrivilegeList | ALL PRIVILEGES}
ON         ObjectName
TO         {AuthorizationIdList | PUBLIC}
[WITH GRANT OPTION]
```

PrivilegeList consists of one or more of the following privileges, separated by commas:

```
SELECT
DELETE
INSERT      [(columnName [, . . . ])]
UPDATE      [(columnName [, . . . ])]
REFERENCES  [(columnName [, . . . ])]
USAGE
```

For convenience, the GRANT statement allows the keyword ALL PRIVILEGES to be used to grant all privileges to a user instead of having to specify the six privileges individually. It also provides the keyword PUBLIC to allow access to be granted to all present and future authorized users, not just to the users currently known to the DBMS. *ObjectName* can be the name of a base table, view, domain, character set, collation, or translation.

The WITH GRANT OPTION clause allows the user(s) in *AuthorizationIdList* to pass the privileges they have been given for the named object on to other users. If these users pass a privilege on specifying WITH GRANT OPTION, the users receiving the privilege may in turn grant it to still other users. If this keyword is not specified, the receiving user(s) will not be able to pass the privileges on to other users. In this way, the owner of the object maintains very tight control over who has permission to use the object and what forms of access are allowed.

EXAMPLE 7.7 GRANT all privileges

Give the user with authorization identifier Manager all privileges on the Staff table.

```
GRANT ALL PRIVILEGES  
ON Staff  
TO Manager WITH GRANT OPTION;
```

The user identified as Manager can now retrieve rows from the Staff table, and also insert, update, and delete data from this table. Manager can also reference the Staff table, and all the Staff columns in any table that he or she creates subsequently. We also specified the keyword WITH GRANT OPTION, so that Manager can pass these privileges on to other users.

EXAMPLE 7.8 GRANT specific privileges

Give users Personnel and Director the privileges SELECT and UPDATE on column salary of the Staff table.

```
GRANT SELECT, UPDATE (salary)  
ON Staff  
TO Personnel, Director;
```

We have omitted the keyword WITH GRANT OPTION, so that users Personnel and Director cannot pass either of these privileges on to other users.

EXAMPLE 7.9 GRANT specific privileges to PUBLIC

Give all users the privilege SELECT on the Branch table.

```
GRANT SELECT  
ON Branch  
TO PUBLIC;
```

The use of the keyword PUBLIC means that all users (now and in the future) are able to retrieve all the data in the Branch table. Note that it does not make sense to use WITH GRANT OPTION in this case: as every user has access to the table, there is no need to pass the privilege on to other users.

7.6.2 Revoking Privileges from Users (REVOKE)

The REVOKE statement is used to take away privileges that were granted with the GRANT statement. A REVOKE statement can take away all or some of the privileges that were previously granted to a user. The format of the statement is:

```
REVOKE [GRANT OPTION FOR] {PrivilegeList | ALL PRIVILEGES}  
ON      ObjectName  
FROM    {AuthorizationIdList | PUBLIC} [RESTRICT | CASCADE]
```

The keyword ALL PRIVILEGES refers to all the privileges granted to a user by the user revoking the privileges. The optional GRANT OPTION FOR clause allows privileges passed on via the WITH GRANT OPTION of the GRANT statement to be revoked separately from the privileges themselves.

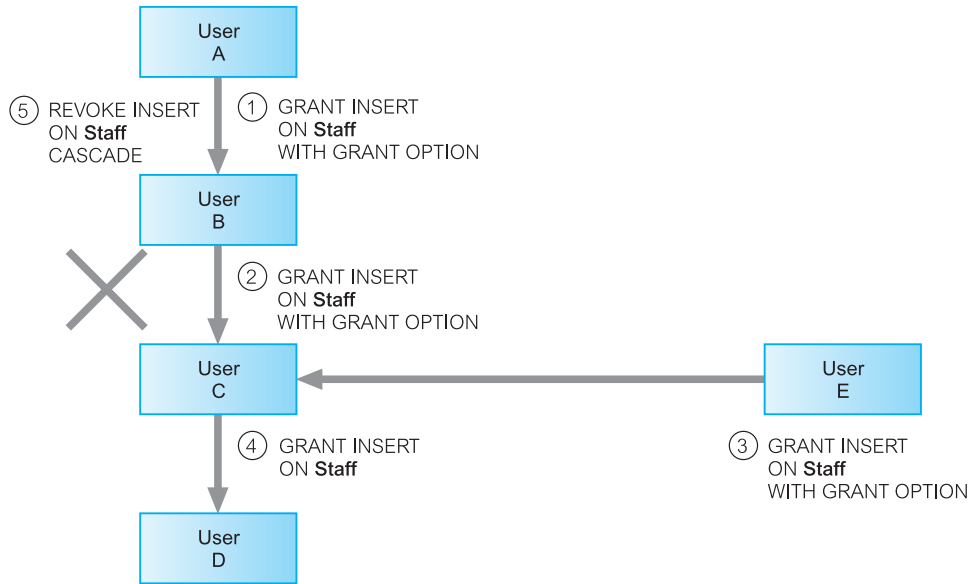


Figure 7.1 Effects of REVOKE.

The **RESTRICT** and **CASCADE** qualifiers operate exactly as in the **DROP TABLE** statement (see Section 7.3.3). Because privileges are required to create certain objects, revoking a privilege can remove the authority that allowed the object to be created (such an object is said to be **abandoned**). The **REVOKE** statement fails if it results in an abandoned object, such as a view, unless the **CASCADE** keyword has been specified. If **CASCADE** is specified, an appropriate **DROP** statement is issued for any abandoned views, domains, constraints, or assertions.

The privileges that were granted to this user by other users are not affected by this **REVOKE** statement. Therefore, if another user has granted the user the privilege being revoked, the other user's grant still allows the user to access the table. For example, in Figure 7.1 User A grants User B **INSERT** privilege on the **Staff** table **WITH GRANT OPTION** (step 1). User B passes this privilege on to User C (step 2). Subsequently, User C gets the same privilege from User E (step 3). User C then passes the privilege on to User D (step 4). When User A revokes the **INSERT** privilege from User B (step 5), the privilege cannot be revoked from User C, because User C has also received the privilege from User E. If User E had not given User C this privilege, the revoke would have cascaded to User C and User D.

EXAMPLE 7.10 REVOKE specific privileges from PUBLIC

*Revoke the privilege **SELECT** on the **Branch** table from all users.*

```

REVOKE SELECT
ON Branch
FROM PUBLIC;

```

EXAMPLE 7.11 REVOKE specific privileges from named user

Revoke all privileges you have given to Director on the Staff table.

```
REVOKE ALL PRIVILEGES
ON Staff
FROM Director;
```

This is equivalent to REVOKE SELECT . . . , as this was the only privilege that has been given to Director.

Chapter Summary

- The ISO standard provides eight base data types: boolean, character, bit, exact numeric, approximate numeric, datetime, interval, and character/binary large objects.
- The SQL DDL statements allow database objects to be defined. The CREATE and DROP SCHEMA statements allow schemas to be created and destroyed; the CREATE, ALTER, and DROP TABLE statements allow tables to be created, modified, and destroyed; the CREATE and DROP INDEX statements allow indexes to be created and destroyed.
- The ISO SQL standard provides clauses in the **CREATE** and **ALTER TABLE** statements to define **integrity constraints** that handle required data, domain constraints, entity integrity, referential integrity, and general constraints. **Required data** can be specified using NOT NULL. **Domain constraints** can be specified using the CHECK clause or by defining domains using the CREATE DOMAIN statement. **Primary keys** should be defined using the PRIMARY KEY clause and **alternate keys** using the combination of NOT NULL and UNIQUE. **Foreign keys** should be defined using the FOREIGN KEY clause and update and delete rules using the subclauses ON UPDATE and ON DELETE. **General constraints** can be defined using the CHECK and UNIQUE clauses. General constraints can also be created using the CREATE ASSERTION statement.
- A **view** is a virtual table representing a subset of columns and/or rows and/or column expressions from one or more base tables or views. A view is created using the CREATE VIEW statement by specifying a **defining query**. It may not necessarily be a physically stored table, but may be recreated each time it is referenced.
- Views can be used to simplify the structure of the database and make queries easier to write. They can also be used to protect certain columns and/or rows from unauthorized access. Not all views are updatable.
- **View resolution** merges the query on a view with the definition of the view producing a query on the underlying base table(s). This process is performed each time the DBMS has to process a query on a view. An alternative approach, called **view materialization**, stores the view as a temporary table in the database when the view is first queried. Thereafter, queries based on the materialized view can be much faster than recomputing the view each time. One disadvantage with materialized views is maintaining the currency of the temporary table.
- The COMMIT statement signals successful completion of a transaction and all changes to the database are made permanent. The ROLLBACK statement signals that the transaction should be aborted and all changes to the database are undone.
- SQL access control is built around the concepts of authorization identifiers, ownership, and privileges. **Authorization identifiers** are assigned to database users by the DBA and identify a user. Each object

that is created in SQL has an **owner**. The owner can pass **privileges** on to other users using the GRANT statement and can revoke the privileges passed on using the REVOKE statement. The privileges that can be passed on are USAGE, SELECT, DELETE, INSERT, UPDATE, and REFERENCES; INSERT, UPDATE, and REFERENCES can be restricted to specific columns. A user can allow a receiving user to pass privileges on using the WITH GRANT OPTION clause and can revoke this privilege using the GRANT OPTION FOR clause.

Review Questions

- 7.1 What are the main SQL DDL statements?
- 7.2 Discuss the functionality and importance of the Integrity Enhancement Feature (IFF).
- 7.3 What are the privileges commonly granted to database users?
- 7.4 Discuss the advantages and disadvantages of views.
- 7.5 Discuss the ways by which a transaction can complete.
- 7.6 What restrictions are necessary to ensure that a view is updatable?
- 7.7 What is a materialized view and what are the advantages of maintaining a materialized view rather than using the view resolution process?
- 7.8 Describe the difference between discretionary and mandatory access control. What type of control mechanism does SQL support?
- 7.9 Describe how the access control mechanisms of SQL work.

Exercises

Answer the following questions using the relational schema from the Exercises at the end of Chapter 4:

- 7.10 Create the **Hotel** table using the integrity enhancement features of SQL.
- 7.11 Now create the **Room**, **Booking**, and **Guest** tables using the integrity enhancement features of SQL with the following constraints:
 - (a) **type** must be one of Single, Double, or Family.
 - (b) **price** must be between £10 and £100.
 - (c) **roomNo** must be between 1 and 100.
 - (d) **dateFrom** and **dateTo** must be greater than today's date.
 - (e) The same room cannot be double-booked.
 - (f) The same guest cannot have overlapping bookings.
- 7.12 Create a separate table with the same structure as the **Booking** table to hold archive records. Using the INSERT statement, copy the records from the **Booking** table to the archive table relating to bookings before 1 January 2013. Delete all bookings before 1 January 2013 from the **Booking** table.
- 7.13 Assume that all hotels are loaded. Create a view containing the cheapest hotels in the world.
- 7.14 Create a view containing the guests who are from BRICS countries.
- 7.15 Give the users **Manager** and **Director** full access to these views, with the privilege to pass the access on to other users.
- 7.16 Give the user **Accounts** SELECT access to these views. Now revoke the access from this user.

7.17 Consider the following view defined on the Hotel schema:

```
CREATE VIEW HotelBookingCount (hotelNo, bookingCount)
AS SELECT h.hotelNo, COUNT(*)
FROM Hotel h, Room r, Booking b
WHERE h.hotelNo = r.hotelNo AND r.roomNo = b.roomNo
GROUP BY h.hotelNo;
```

For each of the following queries, state whether the query is valid, and for the valid ones, show how each of the queries would be mapped on to a query on the underlying base tables.

- (a) **SELECT ***
FROM HotelBookingCount;
- (b) **SELECT** hotelNo
FROM HotelBookingCount
WHERE hotelNo = 'H001';
- (c) **SELECT** MIN(bookingCount)
FROM HotelBookingCount;
- (d) **SELECT COUNT(*)**
FROM HotelBookingCount;
- (e) **SELECT** hotelNo
FROM HotelBookingCount
WHERE bookingCount > 1000;
- (f) **SELECT** hotelNo
FROM HotelBookingCount
ORDER BY bookingCount;

7.19 Assume that we also have a table for suppliers:

Supplier (supplierNo, partNo, price)

and a view **SupplierParts**, which contains the distinct part numbers that are supplied by at least one supplier:

```
CREATE VIEW SupplierParts (partNo)
AS SELECT DISTINCT partNo
FROM Supplier s, Part p
WHERE s.partNo = p.partNo;
```

Discuss how you would maintain this as a materialized view and under what circumstances you would be able to maintain the view without having to access the underlying base tables **Part** and **Supplier**.

7.20 Analyze three different DBMSs of your choice. Identify objects that are available in the system catalog. Compare and contrast the object organization, name scheme, and the ways used to retrieve object description.



7.21 Create the *DreamHome* rental database schema defined in Section 4.2.6 and insert the tuples shown in Figure 4.3.

7.22 Use the view you created in exercise 7.13 to discuss how you would improve the performance of the SQL command.

7.23 You are contracted to investigate queries with degraded performance to improve them. Based on the schemas created in previous exercises, discuss the criteria to decide for or against indexing.

Case Study 2

For Exercises 7.24 to 7.40, use the *Projects* schema defined in the Exercises at the end of Chapter 5.

7.24 Create the *Projects* schema using the integrity enhancement features of SQL with the following constraints:

- (a) **sex** must be one of the single characters 'M' or 'F'.
- (b) **position** must be one of 'Manager', 'Team Leader', 'Analyst', or 'Software Developer'.
- (c) **hoursWorked** must be an integer value between 0 and 40.

7.25 Create a view consisting of projects managed by female managers and ordered by project number.

7.26 Create a view consisting of the attributes `empNo`, `fName`, `lName`, `projName`, and `hoursWorked` attributes.

7.27 Consider the following view defined on the Projects schema:

```
CREATE VIEW EmpProject(empNo, projNo, totalHours)
AS SELECT w.empNo, w.projNo, SUM(hoursWorked)
FROM Employee e, Project p, WorksOn w
WHERE e.empNo = w.empNo AND p.projNo = w.projNo
GROUP BY w.empNo, w.projNo;
```

- (a) **SELECT***
FROM EmpProject;
- (b) **SELECT** projNo
FROM EmpProject
WHERE projNo = 'SCCS';
- (c) **SELECT COUNT**(projNo)
FROM EmpProject
WHERE empNo = 'E1';
- (d) **SELECT** empNo, totalHours
FROM EmpProject
GROUP BY empNo;

General

7.28 Consider the following table:

Part (partNo, contract, partCost)

which represents the cost negotiated under each contract for a part (a part may have a different price under each contract). Now consider the following view **ExpensiveParts**, which contains the distinct part numbers for parts that cost more than £1000:

```
CREATE VIEW ExpensiveParts (partNo)
AS SELECT DISTINCT partNo
FROM Part
WHERE partCost > 1000;
```

Discuss how you would maintain this as a materialized view and under what circumstances you would be able to maintain the view without having to access the underlying base table **Part**.

Chapter Objectives

In this chapter you will learn:

- How to use the SQL programming language.
- How to use SQL cursors.
- How to create stored procedures.
- How to create triggers.
- How to use triggers to enforce integrity constraints.
- The advantages and disadvantages of triggers.
- How to use recursive queries.

The previous two chapters have focused on the main language of relational DBMSs, namely SQL. In Chapter 6 we examined the data manipulation language (DML) statements of SQL: SELECT, INSERT, UPDATE, and DELETE. In Chapter 7 we examined the main data definition language (DDL) statements of SQL, such as the CREATE and ALTER TABLE statements and the CREATE VIEW statement. In this chapter we discuss some other parts of the SQL standard, namely:

- the SQL programming language (SQL/PSM);
- SQL cursors;
- stored procedures;
- triggers;
- recursive queries.

Looking ahead, in Chapter 9 we discuss the features that have been added to the SQL specification to support object-oriented data management, and in Chapter 30 we examine the features that have been added to the specification to support XML (eXtensible Markup Language), called SQL/XML:2011. Finally, in Appendix I we discuss how SQL can be embedded in high-level programming languages. The examples in this chapter use the *DreamHome* rental database shown in Figure 4.3.



8.1 The SQL Programming Language

The initial two versions of the SQL language had no programming constructs; that is, it was not *computationally complete*. To overcome this problem, the more recent versions of the SQL standard allow SQL to be embedded in high-level programming languages to help develop more complex database applications (see Appendix I). However, this approach produces an **impedance mismatch**, because we are mixing different programming paradigms:

- SQL is a declarative language that handles rows of data, whereas a high-level language such as C is a procedural language that can handle only one row of data at a time.
- SQL and 3GLs use different models to represent data. For example, SQL provides the built-in data types Date and Interval, which are not available in traditional programming languages. Thus, it is necessary for the application program to convert between the two representations, which is inefficient both in programming effort and in the use of runtime resources. It has been estimated that as much as 30% of programming effort and code space is expended on this type of conversion (Atkinson *et al.*, 1983). Furthermore, since we are using two different type systems, it is not possible to automatically type check the application as a whole.

It is argued that the solution to these problems is not to replace relational languages by record-level object-oriented languages, but to introduce set-level facilities into programming languages (Date, 1995). However, SQL is now a full programming language and we discuss some of the programming constructs in this section. The extensions are known as **SQL/PSM (Persistent Stored Modules)**; however, to make the discussions more concrete, we base the presentation mainly on the Oracle programming language, PL/SQL.

PL/SQL (Procedural Language/SQL) is Oracle's procedural extension to SQL. There are two versions of PL/SQL: one is part of the Oracle server, and the other is a separate engine embedded in a number of Oracle tools. They are very similar to each other and have the same programming constructs, syntax, and logic mechanisms, although PL/SQL for Oracle tools has some extensions to suit the requirements of the particular tool (for example, PL/SQL has extensions for Oracle Forms).

PL/SQL has concepts similar to modern programming languages, such as variable and constant declarations, control structures, exception handling, and modularization. PL/SQL is a block-structured language: blocks can be entirely separate or nested within one another. The basic units that constitute a PL/SQL program are procedures, functions, and anonymous (*unnamed*) blocks. As illustrated in Figure 8.1, a PL/SQL block has up to three parts:

- an optional declaration part, in which variables, constants, cursors, and exceptions are defined and possibly initialized;
- a mandatory executable part, in which the variables are manipulated;
- an optional exception part, to handle any exceptions raised during execution.

8.1.1 Declarations

Variables and constant variables must be declared before they can be referenced in other statements, including other declarative statements. Examples of declarations are:

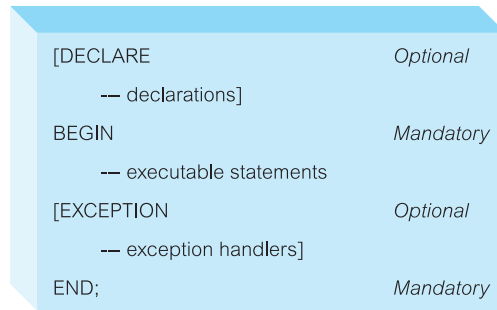


Figure 8.1
General structure
of a PL/SQL
block.

```
vStaffNo VARCHAR2(5);
vRent NUMBER(6, 2) NOT NULL := 600;
MAX_PROPERTIES CONSTANT NUMBER := 100;
```

Note that it is possible to declare a variable as **NOT NULL**, although in this case an initial value must be assigned to the variable. It is also possible to declare a variable to be of the same type as a column in a specified table or another variable using the **%TYPE** attribute. For example, to declare that the `vStaffNo` variable is the same type as the `staffNo` column of the `Staff` table, we could write:

```
vStaffNo Staff.staffNo%TYPE;
vStaffNo1 vStaffNo%TYPE;
```

Similarly, we can declare a variable to be of the same type as an entire row of a table or view using the **%ROWTYPE** attribute. In this case, the fields in the record take their names and data types from the columns in the table or view. For example, to declare a `vStaffRec` variable to be a row from the `Staff` table, we could write:

```
vStaffRec Staff%ROWTYPE;
```

Note that **%TYPE** and **%ROWTYPE** are not standard SQL.

8.1.2 Assignments

In the executable part of a PL/SQL block, variables can be assigned in two ways: using the normal assignment statement (**:=**) or as the result of an SQL **SELECT** or **FETCH** statement. For example:

```
vStaffNo := 'SG14';
vRent := 500;
SELECT COUNT(*) INTO x FROM PropertyForRent WHERE staffNo = vStaffNo;
```

In the third case, the variable `x` is set to the result of the **SELECT** statement (in this case, equal to the number of properties managed by staff member SG14).

Note that in the SQL standard, an assignment uses the **SET** keyword at the start of the line with the **=** symbol, instead of the **:=**. For example:

```
SET vStaffNo = 'SG14'
```

8.1.3 Control Statements

PL/SQL supports the usual conditional, iterative, and sequential flow-of-control mechanisms.

Conditional IF statement The IF statement has the following form:

```
IF (condition) THEN
    <SQL statement list>
[ELSIF (condition) THEN <SQL statement list>]
[ELSE <SQL statement list>]
END IF;
```

Note that the SQL standard specifies **ELSEIF** instead of **ELSIF**.

For example:

```
IF (position = 'Manager') THEN
    salary := salary*1.05;
ELSE
    salary := salary*1.03;
END IF;
```

Conditional CASE statement The CASE statement allows the selection of an execution path based on a set of alternatives and has the following form:

```
CASE (operand)
[WHEN (whenOperandList) | WHEN (searchCondition)
    THEN <SQL statement list>]
[ELSE <SQL statement list>]
END CASE;
```

For example:

<pre>CASE lowercase(x) WHEN 'a' THEN x := 1; WHEN 'b' THEN x := 2; y := 0; WHEN 'default' THEN x := 3; END CASE;</pre>	<pre>UPDATE Staff SET salary = CASE WHEN position = 'Manager' THEN salary * 1.05 ELSE THEN salary * 1.02 END;</pre>
--	---

Iteration statement (LOOP) The LOOP statement has the following form:

```
[labelName:]
LOOP
    <SQL statement list>
    EXIT [labelName] [WHEN (condition)]
END LOOP [labelName];
```

Note that the SQL standard specifies **LEAVE** instead of **EXIT WHEN** (condition).

For example:

```
x:=1;
myLoop:
LOOP
  x := x+1;
  IF (x > 3) THEN
    EXIT myLoop;      --- exit loop immediately
  END LOOP myLoop;
  --- control resumes here
y := 2;
```

In this example, the loop is terminated when x becomes greater than 3 and control resumes immediately after the **END LOOP** keyword.

Iteration statement (WHILE and REPEAT) The **WHILE** and **REPEAT** statements have the following form (note that PL/SQL has no equivalent to the **REPEAT** loop specified in the SQL standard):

PL/SQL	SQL
WHILE (condition) LOOP	WHILE (condition) DO
<SQL statement list>	<SQL statement list>
END LOOP [labelName];	END WHILE [labelName];
	REPEAT
	<SQL statement list>
	UNTIL (condition)
	END REPEAT [labelName];

Iteration statement (FOR) The **FOR** statement has the following form:

PL/SQL	SQL
FOR indexVariable	FOR indexVariable
IN lowerBound .. upperBound LOOP	AS querySpecification DO
<SQL statement list>	<SQL statement list>
END LOOP [labelName];	END FOR [labelName];

The following is an example of a **FOR** loop in PL/SQL:

```
DECLARE
  numberOfStaff NUMBER;
SELECT COUNT(*) INTO numberOfStaff FROM PropertyForRent
      WHERE staffNo = 'SG14';myLoop1:
FOR iStaff IN 1 .. numberOfStaff LOOP
  ....
END LOOP
  myLoop1;
```

The following is an example of a FOR loop in standard SQL:

```
myLoop1:
  FOR iStaff AS SELECT COUNT(*) FROM PropertyForRent
    WHERE staffNo = 'SG14' DO
    ....
  END FOR myLoop1;
```

We present additional examples using some of these structures shortly.

8.1.4 Exceptions in PL/SQL

An **exception** is an identifier in PL/SQL raised during the execution of a block that terminates its main body of actions. A block always terminates when an exception is raised, although the exception handler can perform some final actions. An exception can be raised automatically by Oracle—for example, the exception `NO_DATA_FOUND` is raised whenever no rows are retrieved from the database in a `SELECT` statement. It is also possible for an exception to be raised explicitly using the `RAISE` statement. To handle raised exceptions, separate routines called **exception handlers** are specified.

As mentioned earlier, a user-defined exception is defined in the declarative part of a PL/SQL block. In the executable part, a check is made for the exception condition, and, if found, the exception is raised. The exception handler itself is defined at the end of the PL/SQL block. An example of exception handling is given in Figure 8.2. This example also illustrates the use of the Oracle-supplied package

```
DECLARE
  vpCount    NUMBER;
  vStaffNo PropertyForRent.staffNo%TYPE := 'SG14';
  -- define an exception for the enterprise constraint that prevents a member of staff
  -- managing more than 100 properties
  e_too_many_properties EXCEPTION;
  PRAGMA EXCEPTION_INIT(e_too_many_properties, -20000);
BEGIN
  SELECT COUNT(*) INTO vpCount
  FROM PropertyForRent
  WHERE staffNo = vStaffNo;
  IF vpCount = 100
  -- raise an exception for the general constraint
    RAISE e_too_many_properties;
  END IF;
  UPDATE PropertyForRent SET staffNo = vStaffNo WHERE propertyNo = 'PG4';
EXCEPTION
  -- handle the exception for the general constraint
  WHEN e_too_many_properties THEN
    dbms_output.put_line('Member of staff ' || staffNo || 'already managing 100 properties');
END;
```

Figure 8.2 Example of exception handling in PL/SQL.

DBMS_OUTPUT, which allows output from PL/SQL blocks and subprograms. The procedure `put_line` outputs information to a buffer in the SGA (an area of shared memory that is used to store data and control information for one Oracle instance), which can be displayed by calling the procedure `get_line` or by setting `SERVEROUTPUT ON` in `SQL*Plus`.

Condition handling

The SQL Persistent Stored Modules (SQL/PSM) language includes condition handling to handle exceptions and completion conditions. Condition handling works by first defining a handler by specifying its type, the exception and completion conditions it can resolve, and the action it takes to do so (an SQL procedure statement). Condition handling also provides the ability to explicitly signal exception and completion conditions, using the `SIGNAL/RESIGNAL` statement.

A handler for an associated exception or completion condition can be declared using the `DECLARE . . . HANDLER` statement:

```
DECLARE {CONTINUE | EXIT | UNDO} HANDLER  
FOR SQLSTATE {sqlstateValue | conditionName | SQLEXCEPTION |  
               SQLWARNING | NOT FOUND} handlerAction;
```

A condition name and an optional corresponding `SQLSTATE` value can be declared using:

```
DECLARE conditionName CONDITION  
[FOR SQLSTATE sqlstateValue]
```

and an exception condition can be signaled or resignaled using:

```
SIGNAL sqlstateValue; or RESIGNAL sqlstateValue;
```

When a compound statement containing a handler declaration is executed, a handler is created for the associated conditions. A handler is *activated* when it is the most appropriate handler for the condition that has been raised by the SQL statement. If the handler has specified `CONTINUE`, then on activation it will execute the handler action before returning control to the compound statement. If the handler type is `EXIT`, then after executing the handler action, the handler leaves the compound statement. If the handler type is `UNDO`, then the handler rolls back all changes made within the compound statement, executes the associated handler action, and then returns control to the compound statement. If the handler does not complete with a *successful completion* condition, then an implicit resignal is executed, which determines whether there is another handler that can resolve the condition.

8.1.5 Cursors in PL/SQL

A `SELECT` statement can be used if the query returns *one and only one* row. To handle a query that can return an arbitrary number of rows (that is, zero, one, or more rows) PL/SQL uses **cursors** to allow the rows of a query result to be accessed one at a time. In effect, the cursor acts as a pointer to a particular row of the query result. The cursor can be advanced by 1 to access the next row. A cursor must be *declared* and *opened* before it can be used, and it must be *closed* to deactivate it after it is no longer required. Once the cursor has been opened, the rows of the query

result can be retrieved one at a time using a `FETCH` statement, as opposed to a `SELECT` statement. (In Appendix I we see that SQL can also be embedded in high-level programming languages and that cursors are also used for handling queries that can return an arbitrary number of rows.)

EXAMPLE 8.1 Use of cursors

Figure 8.3 illustrates the use of a cursor to determine the properties managed by staff member SG14. In this case, the query can return an arbitrary number of rows, so a cursor must be used. The important points to note in this example are:

- In the `DECLARE` section, the cursor `propertyCursor` is defined.
- In the statements section, the cursor is first opened. Among others, this has the effect of parsing the `SELECT` statement specified in the `CURSOR` declaration, identifying the rows that satisfy the search criteria (called the *active set*), and positioning the pointer just before the first row in the active set. Note, if the query returns no rows, PL/SQL does not raise an exception when the cursor is open.
- The code then loops over each row in the active set and retrieves the current row values into output variables using the `FETCH INTO` statement. Each `FETCH` statement also advances the pointer to the next row of the active set.
- The code checks whether the cursor did not contain a row (`propertyCursor%NOTFOUND`) and exits the loop if no row was found (`EXIT WHEN`). Otherwise, it displays the property details using the `DBMS_OUTPUT` package and goes around the loop again.
- The cursor is closed on completion of the fetches.
- Finally, the exception block displays any error conditions encountered.

As well as `%NOTFOUND`, which evaluates to true if the most recent fetch does not return a row, there are some other cursor attributes that are useful:

- `%FOUND`: Evaluates to true if the most recent fetch returns a row (complement of `%NOTFOUND`).
- `%ISOPEN`: Evaluates to true if the cursor is open.
- `%ROWCOUNT`: Evaluates to the total number of rows returned so far.

Passing parameters to cursors PL/SQL allows cursors to be parameterized, so that the same cursor definition can be reused with different criteria. For example, we could change the cursor defined in the previous example to:

```
CURSOR propertyCursor (vStaffNo VARCHAR2) IS
    SELECT propertyNo, street, city, postcode
    FROM PropertyForRent
    WHERE staffNo = vStaffNo
    ORDER BY propertyNo;
```

and we could open the cursor using the following example statements:

```
vStaffNo1 PropertyForRent.staffNo%TYPE := 'SG14';
OPEN propertyCursor('SG14');
OPEN propertyCursor('SA9');
OPEN propertyCursor(vStaffNo1);
```

```

DECLARE
    vPropertyNo      PropertyForRent.propertyNo%TYPE;
    vStreet          PropertyForRent.street%TYPE;
    vCity            PropertyForRent.city%TYPE;
    vPostcode        PropertyForRent.postcode%TYPE;
    CURSOR propertyCursor IS
        SELECT propertyNo, street, city, postcode
        FROM PropertyForRent
        WHERE staffNo = 'SG14'
        ORDER by propertyNo;
BEGIN
    -- Open the cursor to start of selection, then loop to fetch each row of the result table
    OPEN propertyCursor;
    LOOP

    -- Fetch next row of the result table
        FETCH propertyCursor
            INTO vPropertyNo, vStreet, vCity, vPostcode;
        EXIT WHEN propertyCursor%NOTFOUND;

    -- Display data
        dbms_output.put_line('Property number: ' || vPropertyNo);
        dbms_output.put_line('Street:          ' || vStreet);
        dbms_output.put_line('City:            ' || vCity);
        IF postcode IS NOT NULL THEN
            dbms_output.put_line('Post Code:        ' || vPostcode);
        ELSE
            dbms_output.put_line('Post Code:        NULL');
        END IF;
    END LOOP;
    IF propertyCursor%ISOPEN THEN CLOSE propertyCursor END IF;

    -- Error condition - print out error
EXCEPTION
    WHEN OTHERS THEN
        dbms_output.put_line('Error detected');
        IF propertyCursor%ISOPEN THEN CLOSE propertyCursor; END IF;
END;

```

Figure 8.3 Using cursors in PL/SQL to process a multirow query.

Updating rows through a cursor It is possible to update and delete a row after it has been fetched through a cursor. In this case, to ensure that rows are not changed between declaring the cursor, opening it, and fetching the rows in the active set, the FOR UPDATE clause is added to the cursor declaration. This has the effect of locking the rows of the active set to prevent any update conflict when the cursor is opened (locking and update conflicts are discussed in Chapter 22).

For example, we may want to reassign the properties that SG14 manages to SG37. The cursor would now be declared as:

```
CURSOR propertyCursor IS
    SELECT propertyNo, street, city, postcode
    FROM PropertyForRent
    WHERE staffNo = 'SG14'
    ORDER BY propertyNo
FOR UPDATE NOWAIT;
```

By default, if the Oracle server cannot acquire the locks on the rows in the active set in a **SELECT FOR UPDATE** cursor, it waits indefinitely. To prevent this, the optional **NOWAIT** keyword can be specified and a test can be made to see if the locking has been successful. When looping over the rows in the active set, the **WHERE CURRENT OF** clause is added to the **SQL UPDATE** or **DELETE** statement to indicate that the update is to be applied to the current row of the active set. For example:

```
UPDATE PropertyForRent
SET staffNo = 'SG37'
WHERE CURRENT OF propertyCursor;
...
COMMIT;
```

Cursors in the SQL standard Cursor handling statements defined in the SQL standard are slightly different from that described previously. The interested reader is referred to Appendix I.

8.2 Subprograms, Stored Procedures, Functions, and Packages

Subprograms are named PL/SQL blocks that can take parameters and be invoked. PL/SQL has two types of subprogram called **(stored) procedures** and **functions**. Procedures and functions can take a set of parameters given to them by the calling program and perform a set of actions. Both can modify and return data passed to them as a parameter. The difference between a procedure and a function is that a function will always return a single value to the caller, whereas a procedure does not. Usually, procedures are used unless only one return value is needed.

Procedures and functions are very similar to those found in most high-level programming languages, and have the same advantages: they provide modularity and extensibility, they promote reusability and maintainability, and they aid abstraction. A parameter has a specified name and data type but can also be designated as:

- **IN** – parameter is used as an input value only.
- **OUT** – parameter is used as an output value only.
- **IN OUT** – parameter is used as both an input and an output value.

For example, we could change the anonymous PL/SQL block given in Figure 8.3 into a procedure by adding the following lines at the start:

```
CREATE OR REPLACE PROCEDURE PropertiesForStaff
  (IN vStaffNo VARCHAR2)
AS . . .
```

The procedure could then be executed in SQL*Plus as:

```
SQL> SET SERVEROUTPUT ON;
SQL> EXECUTE PropertiesForStaff('SG14');
```

We discuss functions and procedures in more detail in Chapter 9.

Packages (PL/SQL)

A **package** is a collection of procedures, functions, variables, and SQL statements that are grouped together and stored as a single program unit. A package has two parts: a specification and a body. A package's *specification* declares all public constructs of the package, and the *body* defines all constructs (public and private) of the package, and so implements the specification. In this way, packages provide a form of encapsulation. Oracle performs the following steps when a procedure or package is created:

- It compiles the procedure or package.
- It stores the compiled code in memory.
- It stores the procedure or package in the database.

For the previous example, we could create a package specification as follows:

```
CREATE OR REPLACE PACKAGE StaffPropertiesPackage AS
  procedure PropertiesForStaff(vStaffNo VARCHAR2);
END StaffPropertiesPackage;
```

and we could create the package body (that is, the implementation of the package) as:

```
CREATE OR REPLACE PACKAGE BODY StaffPropertiesPackage
AS
  . . .
END StaffPropertiesPackage;
```

To reference the items declared within a package specification, we use the dot notation. For example, we could call the PropertiesForStaff procedure as follows:

```
StaffPropertiesPackage.PropertiesForStaff('SG14');
```

8.3 Triggers

A **trigger** defines an action that the database should take when some event occurs in the application. A trigger may be used to enforce some referential integrity constraints, to enforce complex constraints, or to audit changes to data. The general format of a trigger in SQL is:

```
CREATE TRIGGER TriggerName
  BEFORE | AFTER | INSTEAD OF
  INSERT | DELETE | UPDATE [OF TriggerColumnList]
```

```

ON TableName
[REFERENCING {OLD | NEW} AS {OldName | NewName}]
[FOR EACH {ROW | STATEMENT}]
[WHEN Condition]
<trigger action>

```

This is not the complete definition, but it is sufficient to demonstrate the basic concept. The code within a trigger, called the *trigger body* or *trigger action*, is made up of an SQL block. Triggers are based on the Event–Condition–Action (ECA) model:

- The *event* (or *events*) that trigger the rule, which can be an INSERT, UPDATE, or DELETE statement on a specified table (or possibly view). In Oracle, it can also be:
 - a CREATE, ALTER, or DROP statement on any schema object;
 - a database startup or instance shutdown, or a user logon or logoff;
 - a specific error message or any error message.

It is also possible to specify whether the trigger should fire *before* the event or *after* the event.

- The *condition* that determines whether the action should be executed. The condition is optional but, if specified, the action will be executed only if the condition is true.
- The *action* to be taken. This block contains the SQL statements and code to be executed when a triggering statement is issued and the trigger condition evaluates to true.

There are two types of trigger: *row-level* triggers (FOR EACH ROW) that execute for each row of the table that is affected by the triggering event, and *statement-level* triggers (FOR EACH STATEMENT) that execute only once even if multiple rows are affected by the triggering event. SQL also supports INSTEAD OF triggers, which provide a transparent way of modifying views that cannot be modified directly through SQL DML statements (INSERT, UPDATE, and DELETE). These triggers are called INSTEAD OF triggers because, unlike other types of trigger, the trigger is fired *instead of* executing the original SQL statement. Triggers can also activate themselves one after the other. This can happen when the trigger action makes a change to the database that has the effect of causing another event that has a trigger associated with it to fire.

EXAMPLE 8.2 AFTER Row-level trigger

Create an AFTER row-level trigger to keep an audit trail of all rows inserted into the Staff table.

```

CREATE TRIGGER StaffAfterInsert
AFTER INSERT ON Staff
REFERENCING NEW AS new
FOR EACH ROW
BEGIN
    INSERT INTO StaffAudit
    VALUES (:new.staffNo, :new.fName, :new.lName, :new.position,
            :new.sex, :new.DOB, :new.salary, :new.branchNo);
END;

```

Note that the SQL standard uses NEW ROW instead of NEW and OLD ROW instead of OLD.



EXAMPLE 8.3 Using a BEFORE trigger

DreamHome has a rule that prevents a member of staff from managing more than 100 properties at the same time. We could create the trigger shown in Figure 8.4 to enforce this constraint. This trigger is invoked before a row is inserted into the `PropertyForRent`

```
CREATE TRIGGER StaffNotHandlingTooMuch
BEFORE INSERT ON PropertyForRent
REFERENCING NEW AS newrow
FOR EACH ROW
DECLARE
    vpCount    NUMBER;
BEGIN
    SELECT COUNT(*) INTO vpCount
    FROM PropertyForRent
    WHERE staffNo = :newrow.staffNo;
    IF vpCount = 100
        raise_application_error(-20000, ('Member' || :newrow.staffNo || 'already managing 100 properties'));
    END IF;
END;
```

Figure 8.4 Trigger to enforce the constraint that a member of staff cannot manage more than 100 properties at any one time.

table or an existing row is updated. If the member of staff currently manages 100 properties, the system displays a message and aborts the transaction. The following points should be noted:

- The **BEFORE** keyword indicates that the trigger should be executed before an insert is applied to the `PropertyForRent` table.
- The **FOR EACH ROW** keyword indicates that this is a row-level trigger, which executes for each row of the `PropertyForRent` table that is updated in the statement.

EXAMPLE 8.4 Using triggers to enforce referential integrity

By default, Oracle enforces the referential actions `ON DELETE NO ACTION` and `ON UPDATE NO ACTION` on the named foreign keys (see Section 7.2.4). It also allows the additional clause `ON DELETE CASCADE` to be specified to allow deletions from the parent table to cascade to the child table. However, it does not support the `ON UPDATE CASCADE` action, or the `SET DEFAULT` and `SET NULL` actions. If any of these actions are required, they will have to be implemented as triggers or stored procedures, or within the application code. For example, from Example 7.1 the foreign key `staffNo` in the `PropertyForRent` table should have the action `ON UPDATE CASCADE`. This action can be implemented using the triggers shown in Figure 8.5.

Trigger 1 (*PropertyForRent_Check_Before*)

The trigger in Figure 8.5(a) is *fired* whenever the `staffNo` column in the `PropertyForRent` table is updated. The trigger checks *before* the update takes place whether the new value specified exists in the `Staff` table. If an `Invalid_Staff` exception is raised, the trigger issues an error message and prevents the change from occurring.

```

-- Before the staffNo column is updated in the PropertyForRent table, fire this trigger
-- to verify that the new foreign key value is present in the Staff table.
CREATE TRIGGER PropertyForRent_Check_Before
    BEFORE UPDATE OF staffNo ON PropertyForRent
    FOR EACH ROW WHEN (new.staffNo IS NOT NULL)
DECLARE
    dummy CHAR(5);
    invalid_staff EXCEPTION;
    valid_staff EXCEPTION;
    mutating_table EXCEPTION;
    PRAGMA EXCEPTION_INIT (mutating_table, -4091);

-- Use cursor to verify that parent key value exists.
-- Use FOR UPDATE OF to lock parent key's row so that it cannot be deleted
-- by another transaction until this transaction completes.
CURSOR update_cursor (sn CHAR(5)) IS
    SELECT staffNo FROM Staff
    WHERE staffNo = sn
    FOR UPDATE OF staffNo;
BEGIN
    OPEN update_cursor (:new.staffNo);
    FETCH update_cursor INTO dummy;
-- Verify parent key. Raise exceptions as appropriate.
    IF update_cursor%NOTFOUND THEN
        RAISE invalid_staff;
    ELSE
        RAISE valid_staff;
    END IF;
    CLOSE update_cursor;
EXCEPTION
    WHEN invalid_staff THEN
        CLOSE update_cursor;
        raise_application_error(-20000, 'Invalid Staff Number' || :new.staffNo);
    WHEN valid_staff THEN
        CLOSE update_cursor;

-- A mutating table is a table that is currently being modified by an INSERT, UPDATE,
-- or DELETE statement, or one that might need to be updated by the effects of a declarative
-- DELETE CASCADE referential integrity constraint.
-- This error would raise an exception, but in this case the exception is OK, so trap it,
-- but don't do anything.
    WHEN mutating_table THEN
        NULL;
END;

```

Annotations in the diagram:

- before trigger**: points to `BEFORE UPDATE OF staffNo ON PropertyForRent`
- condition for trigger to fire**: points to `FOR EACH ROW WHEN (new.staffNo IS NOT NULL)`
- row-level trigger**: points to `FOR EACH ROW`
- Invalid staffNo specified**: points to `RAISE invalid_staff;` and `WHEN invalid_staff THEN`

Figure 8.5(a) Oracle triggers to enforce ON UPDATE CASCADE on the foreign key staffNo in the PropertyForRent table when the primary key staffNo is updated in the Staff table:
(a) trigger for the PropertyForRent table.

Changes to support triggers on the Staff table

The three triggers shown in Figure 8.5(b) are fired whenever the `staffNo` column in the `Staff` table is updated. Before the definition of the triggers, a sequence number `updateSequence` is created, along with a public variable `updateSeq` (which is accessible to the three triggers through the `seqPackage` package). In addition, the `PropertyForRent` table is modified to add a column called `updateId`, which is used to flag whether a row has been updated, to prevent it from being updated more than once during the cascade operation.

```
-- Create a sequence number and a public variable UPDATESEQ.
CREATE SEQUENCE updateSequence INCREMENT BY 1 MAXVALUE 500 CYCLE;
CREATE PACKAGE seqpackage AS
    updateSeq NUMBER;
END seqpackage;
CREATE OR REPLACE PACKAGE BODY seqpackage AS END seqpackage;

-- Add a new attribute to the PropertyForRent table to flag changed rows.
ALTER TABLE PropertyForRent ADD updateId NUMBER;

-- Before updating the Staff table using this statement trigger, generate a new
-- sequence number and assign it to the public variable UPDATESEQ.
CREATE TRIGGER Cascade_StaffNo_Update1
    BEFORE UPDATE OF staffNo ON Staff
    DECLARE
        dummy NUMBER;
    BEGIN
        SELECT updateSequence.NEXTVAL
        INTO dummy FROM dual;
        seqpackage.updateSeq := dummy;
    END;

-- Create a row after-trigger that cascades the update to the PropertyForRent table.
-- Only cascade the update if the child row has not already been updated by the trigger.
CREATE TRIGGER Cascade_StaffNo_Update2
    AFTER UPDATE OF staffNo ON Staff
    FOR EACH ROW
    BEGIN
        UPDATE PropertyForRent SET staffNo = :new.staffNo,
                                updateId = seqpackage.updateSeq
        WHERE staffNo = :old.staffNo AND updateId IS NULL;

-- Create a final statement after-trigger to reset the updateId flags
CREATE TRIGGER Cascade_StaffNo_Update3
    AFTER UPDATE OF staffNo ON Staff
    BEGIN
        UPDATE PropertyForRent SET updateId = NULL
        WHERE updateId = seqpackage.updateSeq;
```

Package to hold sequence number

add extra column to **PropertyForRent** table

statement-level before trigger (default)

set new sequence number for update

row-level after trigger

update **PropertyForRent** table and set updated flag for these rows

statement-level after trigger; resets flags for updated rows

Figure 8.5(b) Triggers for the Staff table.

Trigger 2 (Cascade_StaffNo_Update1)

This (statement-level) trigger fires before the update to the `staffNo` column in the `Staff` table to set a new sequence number for the update.

Trigger 3 (Cascade_StaffNo_Update2)

This (row-level) trigger fires to update all rows in the `PropertyForRent` table that have the old `staffNo` value (`:old.staffNo`) to the new value (`:new.staffNo`), and to flag the row as having been updated.

Trigger 4 (Cascade_StaffNo_Update3)

The final (statement-level) trigger fires after the update to reset the flagged rows back to unflagged.

Dropping triggers

Triggers can be dropped using the `DROP TRIGGER <TriggerName>` statement.

TRIGGER Privilege

In order to create a trigger on a table, the user either has to be the owner of the table (in which case the user will inherit the `TRIGGER` privilege) or the user will need to have been granted the `TRIGGER` privilege on the table (see Section 7.6).

Advantages and disadvantages of triggers

There are a number of advantages and disadvantages with database triggers. Advantages of triggers include:

- *Elimination of redundant code.* Instead of placing a copy of the functionality of the trigger in every client application that requires it, the trigger is stored only once in the database.
- *Simplifying modifications.* Changing a trigger requires changing it in one place only; all the applications automatically use the updated trigger. Thus, they are only coded once, tested once, and then centrally enforced for all the applications accessing the database. The triggers are usually controlled, or at least audited, by a skilled DBA. The result is that the triggers can be implemented efficiently.
- *Increased security.* Storing the triggers in the database gives them all the benefits of security provided automatically by the DBMS.
- *Improved integrity.* Triggers can be extremely useful for implementing some types of integrity constraints, as we have demonstrated earlier. Storing such triggers in the database means that integrity constraints can be enforced consistently by the DBMS across all applications.
- *Improved processing power.* Triggers add processing power to the DBMS and to the database as a whole.
- *Good fit with the client-server architecture.* The central activation and processing of triggers fits the client-server architecture well (see Chapter 3). A single request from a client can result in the automatic performing of a whole sequence of checks and subsequent operations by the database server. In this way, performance is potentially improved as data and operations are not transferred across the network between the client and the server.

Triggers also have disadvantages, which include:

- *Performance overhead.* The management and execution of triggers have a performance overhead that have to be balanced against the advantages cited previously.

- *Cascading effects.* The action of one trigger can cause another trigger to be fired, and so on, in a cascading manner. Not only can this cause a significant change to the database, but it can also be hard to foresee this effect when designing the trigger.
- *Cannot be scheduled.* Triggers cannot be scheduled; they occur when the event that they are based on happens.
- *Less portable.* Although now covered by the SQL standard, most DBMSs implement their own dialect for triggers, which affects portability.

8.4 Recursion

Atomicity of data means that repeating groups are not allowed in the relational model. As a result, it is extremely difficult to handle recursive queries, that is, queries about relationships that a relation has with itself (directly or indirectly). To illustrate the new operation, we use the example simplified Staff relation shown in Figure 9.16, in the next chapter, which stores staff numbers and the corresponding manager's staff number. To find all the managers of all staff, we can use the following recursive query in SQL:2008:

WITH RECURSIVE

```
AllManagers (staffNo, managerStaffNo) AS
  (SELECT staffNo, managerStaffNo
   FROM Staff
   UNION
   SELECT in.staffNo, out.managerStaffNo
   FROM AllManagers in, Staff out
   WHERE in.managerStaffNo = out.staffNo);
SELECT * FROM AllManagers
ORDER BY staffNo, managerStaffNo;
```

This query creates a result table `AllManagers` with two columns `staffNo` and `managerStaffNo` containing all the managers of all staff. The `UNION` operation is performed by taking the union of all rows produced by the inner block until no new rows are generated. Note, if we had specified `UNION ALL`, any duplicate values would remain in the result table.

In some situations, an application may require the data to be inserted into the result table in a certain order. The recursion statement allows the specification of two orderings:

- depth-first, where each 'parent' or 'containing' item appears in the result before the items that it contains, as well as before its 'siblings' (items with the same parent or container);
- breadth-first, where items follow their 'siblings' without following the siblings' children.

For example, at the end of the `WITH RECURSIVE` statement we could add the following clause:

```
SEARCH BREADTH FIRST BY staffNo, managerStaffNo
SET orderColumn
```

The `SET` clause identifies a new column name (`orderColumn`), which is used by SQL to order the result into the required breadth-first traversal.

If the data can be recursive, not just the data structure, an infinite loop can occur unless the cycle can be detected. The recursive statement has a **CYCLE** clause that instructs SQL to record a specified value to indicate that a new row has already been added to the result table. Whenever a new row is found, SQL checks that the row has not been added previously by determining whether the row has been marked with the specified value. If it has, then SQL assumes a cycle has been encountered and stops searching for further result rows. An example of the **CYCLE** clause is:

```
CYCLE staffNo, managerStaffNo
SET cycleMark TO 'Y' DEFAULT 'N'
USING cyclePath
```

`cycleMark` and `cyclePath` are user-defined column names for SQL to use internally. `cyclePath` is an **ARRAY** with cardinality sufficiently large to accommodate the number of rows in the result and whose element type is a row type with a column for each column in the cycle column list (`staffNo` and `managerStaffNo` in our example). Rows satisfying the query are cached in `cyclePath`. When a row satisfying the query is found for the first time (which can be determined by its absence from `cyclePath`), the value of the `cycleMark` column is set to 'N'. When the same row is found again (which can be determined by its presence in `cyclePath`), the `cycleMark` column of the existing row in the result table is modified to the `cycleMark` value of 'Y' to indicate that the row starts a cycle.

Chapter Summary

- The initial versions of the SQL language had no programming constructs; that is, it was not *computationally complete*. However, with the more recent versions of the standard, SQL is now a full programming language with extensions known as **SQL/PSM (Persistent Stored Modules)**.
- SQL/PSM supports the declaration of variables and has assignment statements, flow of control statements (**IF-THEN-ELSE-END IF**; **LOOP-EXIT WHEN-END LOOP**; **FOR-END LOOP**; **WHILE-END LOOP**), and exceptions.
- A **SELECT** statement can be used if the query returns *one and only one* row. To handle a query that can return an arbitrary number of rows (that is, zero, one, or more rows), SQL uses **cursors** to allow the rows of a query result to be accessed one at a time. In effect, the cursor acts as a pointer to a particular row of the query result. The cursor can be advanced by one to access the next row. A cursor must be *declared* and *opened* before it can be used, and it must be *closed* to deactivate it after it is no longer required. Once the cursor has been opened, the rows of the query result can be retrieved one at a time using a **FETCH** statement, as opposed to a **SELECT** statement.
- **Subprograms** are named PL/SQL blocks that can take parameters and be invoked. PL/SQL has two types of subprograms called (**stored**) **procedures** and **functions**. Procedures and functions can take a set of parameters given to them by the calling program and perform a set of actions. Both can modify and return data passed to them as a parameter. The difference between a procedure and a function is that a function will always return a single value to the caller, whereas a procedure will not. Usually, procedures are used unless only one return value is needed.

- A **trigger** defines an action that the database should take when some event occurs in the application. A trigger may be used to enforce some referential integrity constraints, to enforce complex integrity constraints, or to audit changes to data. Triggers are based on the Event-Condition-Action (ECA) model: the *event* (or *events*) that trigger the rule, the *condition* that determines whether the action should be executed, and the *action* to be taken.
- Advantages of triggers include: eliminates redundant code, simplifies modifications, increases security, improves integrity, improves processing power, and fits well with the client-server architecture. Disadvantages of triggers include: performance overhead, cascading effects, inability to be scheduled, and less portable.

Review Questions

- 8.1 Advanced SQL deals with SQL/PSM and PL/SQL. What led to the introduction of SQL/PSM?
- 8.2 Describe the general structure of a PL/SQL block.
- 8.3 Describe the control statements in PL/SQL. Give examples to illustrate your answers.
- 8.4 Describe how the PL/SQL statements differ from the SQL standard. Give examples to illustrate your answers.
- 8.5 What are SQL cursors? Give an example of the use of an SQL cursor.
- 8.6 How is a procedure different from a function?
- 8.7 Discuss the differences between BEFORE, AFTER, and INSTEAD OF triggers. Give examples to illustrate your answers.
- 8.8 Rows can be changed after they have been fetched through a cursor. How can this event be stopped?
- 8.9 Discuss the advantages and disadvantages of database triggers.

Exercises

For the following questions, use the *Hotel* schema from the Exercises at the end of Chapter 4.

- 8.10 List all hotels in the capital cities of BRICS countries. .
- 8.11 Create a database trigger for the following situations:
 - (a) The price of all double rooms must be greater than £100.
 - (b) The price of double rooms must be greater than the price of the highest single room.
 - (c) A booking cannot be for a hotel room that is already booked for any of the specified dates.
 - (d) A guest cannot make two bookings with overlapping dates.
 - (e) Maintain an audit table with the names and addresses of all guests who make bookings for hotels in London (do not store duplicate guest details).
- 8.12 Create an INSTEAD OF database trigger that will allow data to be inserted into the following view:

```
CREATE VIEW LondonHotelRoom AS
SELECT h.hotelNo, hotelName, city, roomNo, type, price
FROM Hotel h, Room r
WHERE h.hotelNo = r.hotelNo AND city = 'London'
```

- 8.13 Not all modern DBMSs are embedded with SQL/PSM features. Investigate a DBMS of your choice to determine if it is SQL/PSM compliant. Discuss situations in the *DreamHome* project that require trigger creation.

Object-Relational DBMSs

Chapter Objectives

In this chapter you will learn:

- The requirements for advanced database applications.
- Why relational DBMSs currently are not well suited to supporting advanced database applications.
- The problems associated with storing objects in a relational database.
- How the relational model has been extended to support advanced database applications.
- The object-oriented features in the latest SQL standard, SQL:2011, including:
 - row types;
 - user-defined types and user-defined routines;
 - polymorphism;
 - inheritance;
 - reference types and object identity;
 - collection types (ARRAYs, MULTISets, SETs, and LISTs);
 - extensions to the SQL language to make it computationally complete;
 - triggers;
 - support for large objects: Binary Large Objects (BLOBs) and Character Large Objects (CLOBs);
 - recursion.
- Some object-oriented extensions to Oracle.

Object orientation is an approach to software construction that has shown considerable promise for solving some of the classic problems of software development. The underlying concept behind object technology is that all software should be constructed out of standard, reusable components wherever possible. Traditionally, software engineering and database management have existed as separate disciplines. Database technology has concentrated on the static aspects of information storage, while software engineering has modeled the dynamic aspects of software.

With the arrival of the third generation of database management systems, namely **Object-Relational Database Management Systems (ORDBMSs)** and **Object-Oriented Database Management Systems (OODBMSs)**, the two disciplines have been combined to allow the concurrent modeling of both data and the processes acting upon the data.

However, there has been some dispute regarding this next generation of DBMSs. The success of relational systems in the past three decades is evident, and the traditionalists believe that it is sufficient to extend the relational model with additional (object-oriented) capabilities. Others believe that an underlying relational model is inadequate to handle complex applications, such as computer-aided design, computer-aided software engineering, and geographic information systems. In this chapter we consider the ORDBMS, focusing on the extensions that have been made to SQL. Integrating object-oriented concepts into the relational systems is considered a more *evolutionary approach* to supporting advanced database applications. In Chapters 27 and 28 we discuss OODBMSs, which are considered a *revolutionary approach* to integrating object-oriented concepts with database systems. To put the discussion of such systems into context, in this chapter we first consider the characteristics of advanced database applications and why relational DBMSs may not be suitable for handling such applications. Readers unfamiliar with object-oriented concepts are referred to Appendix K.

Structure of this Chapter In Section 9.1 we examine the requirements for the advanced types of database applications that are becoming more commonplace, and in Section 9.2 we discuss why traditional RDBMSs are not well suited to supporting these new applications. In Section 9.3 we examine the problems associated with storing objects in a relational database.

In Section 9.4 we examine the background of the ORDBMS and the types of application that they may be suited to. In Section 9.5 we provide a detailed review of the main features of the SQL:2011 standard released in December 2011. Finally, in Section 9.6 we examine some of the object-oriented extensions that have been added to Oracle, a commercial ORDBMS.

To benefit fully from this chapter, the reader needs to be familiar with object-oriented concepts presented in Appendix K. The examples in this chapter are once again drawn from the *DreamHome* case study documented in Section 11.4 and Appendix A.



9.1 Advanced Database Applications

The computer industry has seen significant changes in the last decade. In database systems, we have seen the widespread acceptance of RDBMSs for traditional business applications such as order processing, inventory control, banking, and airline reservations. However, existing RDBMSs have proven inadequate for applications

whose needs are quite different from those of traditional business database applications. These applications include:

- computer-aided design (CAD);
- computer-aided manufacturing (CAM);
- computer-aided software engineering (CASE);
- network management systems;
- office information systems (OIS) and multimedia systems;
- digital publishing;
- geographic information systems (GIS);
- interactive and dynamic Web sites.

Computer-aided design (CAD)

A CAD database stores data relating to mechanical and electrical design covering, for example, buildings, aircraft, and integrated circuit chips. Designs of this type have some common characteristics:

- 
- Design data is characterized by a large number of types, each with a small number of instances. Conventional databases are typically the opposite. For example, the *DreamHome* database consists of only a dozen or so relations, although relations such as *PropertyForRent*, *Client*, and *Viewing* may contain thousands of tuples.
 - Designs may be very large, perhaps consisting of millions of parts, often with many interdependent subsystem designs.
 - The design is not static but evolves through time. When a design change occurs, its implications must be propagated through all design representations. The dynamic nature of design may mean that some actions cannot be foreseen at the beginning.
 - Updates are far-reaching because of topological or functional relationships, tolerances, and so on. One change is likely to affect a large number of design objects.
 - Often, many design alternatives are being considered for each component, and the correct version for each part must be maintained. This involves some form of version control and configuration management.
 - There may be hundreds of staff members involved with the design, and they may work in parallel on multiple versions of a large design. Even so, the end-product must be consistent and coordinated. This is sometimes referred to as *cooperative engineering*.

Computer-aided manufacturing (CAM)

A CAM database stores similar data to a CAD system, in addition to data relating to discrete production (such as cars on an assembly line) and continuous production (such as chemical synthesis). For example, in chemical manufacturing there are applications that monitor information about the state of the system, such as reactor vessel temperatures, flow rates, and yields. There are also applications that control various physical processes, such as opening valves, applying more heat to reactor vessels, and increasing the flow of cooling systems. These applications are often organized in a hierarchy, with a top-level application monitoring the entire factory

and lower-level applications monitoring individual manufacturing processes. These applications must respond in real time and be capable of adjusting processes to maintain optimum performance within tight tolerances. The applications use a combination of standard algorithms and custom rules to respond to different conditions. Operators may modify these rules occasionally to optimize performance based on complex historical data that the system has to maintain. In this example, the system has to maintain large volumes of data that is hierarchical in nature and maintain complex relationships between the data. It must also be able to rapidly navigate through the data to review and respond to changes.

Computer-aided software engineering (CASE)

A CASE database stores data relating to the stages of the software development lifecycle: planning, requirements collection and analysis, design, implementation, testing, maintenance, and documentation. As with CAD, designs may be extremely large, and cooperative engineering is the norm. For example, software configuration management tools allow concurrent sharing of project design, code, and documentation. They also track the dependencies between these components and assist with change management. Project management tools facilitate the coordination of various project management activities, such as the scheduling of potentially highly complex interdependent tasks, cost estimation, and progress monitoring.

Network management systems

Network management systems coordinate the delivery of communication services across a computer network. These systems perform such tasks as network path management, problem management, and network planning. As with the chemical manufacturing example we discussed earlier, these systems also handle complex data and require real-time performance and continuous operation. For example, a telephone call might involve a chain of network switching devices that route a message from sender to receiver, such as:

Node Link Node Link Node Link Node

where each Node represents a port on a network device and each Link represents a slice of bandwidth reserved for that connection. However, a node may participate in several different connections and any database that is created has to manage a complex graph of relationships. To route connections, diagnose problems, and balance loadings, the network management systems have to be capable of moving through this complex graph in real time.

Office information systems (OIS) and multimedia systems

An OIS database stores data relating to the computer control of information in a business, including electronic mail, documents, invoices, and so on. To provide better support for this area, we need to handle a wider range of data types other than names, addresses, dates, and money. Modern systems now handle free-form text, photographs, diagrams, and audio and video sequences. For example, a multimedia document may handle text, photographs, spreadsheets, and voice commentary. The documents may have a specific structure imposed on them, perhaps described

using a markup language such as SGML (Standardized Generalized Markup Language), HTML (HyperText Markup Language), or XML (extended Markup Language), as we discuss in Chapter 30.

Documents may be shared among many users using systems such as electronic mail and bulletin-boards based on Internet technology.[†] Again, such applications need to store data that has a much richer structure than tuples consisting of numbers and text strings. There is also an increasing need to capture handwritten notes using electronic devices. Although many notes can be transcribed into ASCII text using handwriting analysis techniques, most such data cannot. In addition to words, handwritten data can include sketches, diagrams, and so on.

In the *DreamHome* case study, we may find the following requirements for handling multimedia:



- *Image data.* A client may query an image database of properties for rent. Some queries may simply use a textual description to identify images of desirable properties. In other cases, it may be useful for the client to query using graphical images of the features that may be found in desirable properties (such as bay windows, internal corncicing, or roof gardens).
- *Video data.* A client may query a video database of properties for rent. Again, some queries may simply use a textual description to identify the video images of desirable properties. In other cases, it may be useful for the client to query using video features of the desired properties (such as views of the sea or surrounding hills).
- *Audio data.* A client may query an audio database that describes the features of properties for rent. Once again, some queries may simply use a textual description to identify the desired property. In other cases it may be useful for the client to use audio features of the desired properties (such as the noise level from nearby traffic).
- *Handwritten data.* A member of staff may create notes while carrying out inspections of properties for rent. At a later date, he or she may wish to query such data to find all notes made about a flat in Novar Drive with dry rot.

Digital publishing

The publishing industry is likely to undergo profound changes in business practices over the next decade. It is now possible to store books, journals, papers, and articles electronically and deliver them over high-speed networks to consumers. As with office information systems, digital publishing is being extended to handle multimedia documents consisting of text, audio, image, and video data and animation. In some cases, the amount of information available to be put online is enormous, in the order of petabytes (10^{15} bytes), which would make them the largest databases that a DBMS has ever had to manage.

Geographic information systems (GIS)

A GIS database stores various types of spatial and temporal information, such as that used in land management and underwater exploration. Much of the data in these systems is derived from survey and satellite photographs and tends to be

[†]A potentially damaging criticism of database systems, as noted by a number of observers, is that the largest “database” in the world—the World Wide Web—was initially developed with little or no use of database technology. We discuss the integration of the Web and DBMSs in Chapter 29.

very large. Searches may involve identifying features based, for example, on shape, color, or texture, using advanced pattern-recognition techniques.

For example, EOS (Earth Observing System) is a collection of satellites launched by NASA in the 1990s to gather information that will support scientists concerned with long-term trends regarding the earth's atmosphere, oceans, and land. It is anticipated that these satellites will return over one-third of a petabyte of information per year. This data will be integrated with other data sources and will be stored in EOSDIS (EOS Data and Information System). EOSDIS will supply the information needs of both scientists and nonscientists. For example, students will be able to access EOSDIS to see a simulation of world weather patterns. The immense size of this database and the need to support thousands of users with very heavy volumes of information requests will provide many challenges for DBMSs.

Interactive and dynamic Web sites

Consider a Web site that has an online catalog for selling clothes. The Web site maintains a set of preferences for previous visitors to the site and allows a visitor to:

- browse through thumbnail images of the items in the catalog and select one to obtain a full-size image with supporting details;
- search for items that match a user-defined set of criteria;
- obtain a 3D rendering of any item of clothing based on a customized specification (for example, color, size, fabric);
- modify the rendering to account for movement, illumination, backdrop, occasion, and so on;
- select accessories to go with the outfit, from items presented in a sidebar;
- select a voiceover commentary giving additional details of the item;
- view a running total of the bill, with appropriate discounts;
- conclude the purchase through a secure online transaction.

The requirements for this type of application are not that different from some of the previously mentioned advanced applications: there is a need to handle multimedia content (text, audio, image, video data, and animation) and to modify the display interactively based on user preferences and user selections. As well as handling complex data, the site also has the added complexity of providing 3D rendering. It is argued that in such a situation the database is not just presenting information to the visitor but is *actively* engaged in selling, dynamically providing customized information and atmosphere to the visitor (King, 1997).

As we discuss in Chapters 29 and 30, the Web now provides a relatively new paradigm for data management, and languages such as XML hold significant promise, particularly for the e-Commerce market. According to the Forrester Research Group and eMarketer, U.S. online retail sales for 2011 were approximately US\$225 billion and projected to grow to US\$362 billion by 2016 with computer and consumer electronics accounting for 22% of the market and apparel and accessories for about 20% of the market. The Centre for Retail Research estimated that Europe would exceed the U.S. in 2012 with online retail sales of US\$232 billion. The global business-to-business (B2B) market is expected to be even greater than the business-to-consumer (B2C) market, and B2B revenues are expected to surpass

B2C revenues many times over in the coming years. Companies now can reach almost 2.5 billion online consumers worldwide (35% of the world's population) via their global web presence. As the use of the Internet increases and the technology becomes more sophisticated, then we will see Web sites and business-to-business transactions handle much more complex and interrelated data.

Other advanced database applications include:

- *Scientific and medical applications*, which may store complex data representing systems such as molecular models for synthetic chemical compounds and genetic material.
- *Expert systems*, which may store knowledge and rule bases for AI applications.
- Other applications with complex and interrelated objects and procedural data.

9.2 Weaknesses of RDBMSs

In Chapter 4 we discussed how the relational model has a strong theoretical foundation, based on first-order predicate logic. This theory supported the development of SQL, a declarative language that has now become the standard language for defining and manipulating relational databases. Other strengths of the relational model are its simplicity, its suitability for Online Transaction Processing (OLTP), and its support for data independence. However, the relational data model, and relational DBMSs in particular, are not without their disadvantages. Table 9.1 lists some of the more significant weaknesses often cited by the proponents of the object-oriented approach. We discuss these weaknesses in this section and leave readers to judge for themselves the applicability of these weaknesses.

Poor representation of “real-world” entities

The process of normalization generally leads to the creation of relations that do not correspond to entities in the “real-world.” The fragmentation of a “real-world” entity into many relations, with a physical representation that reflects this structure, is inefficient leading to many joins during query processing. As we will see in Chapter 23, the join is one of the most costly operations to perform.

TABLE 9.1 Summary of weaknesses of relational DBMSs.

WEAKNESS

Poor representation of “real-world” entities

Semantic overloading

Poor support for integrity and enterprise constraints

Homogeneous data structure

Limited operations

Difficulty handling recursive queries

Impedance mismatch

Other problems with RDBMSs associated with concurrency, schema changes, and poor navigational access

Semantic overloading

The relational model has only one construct for representing data and relationships between data: the *relation*. For example, to represent a many-to-many (*:*) relationship between two entities A and B, we create three relations, one to represent each of the entities A and B and one to represent the relationship. There is no mechanism to distinguish between entities and relationships, or to distinguish between different kinds of relationship that exist between entities. For example, a 1:* relationship might be *Has*, *Owns*, *Manages*, and so on. If such distinctions could be made, then it might be possible to build the semantics into the operations. It is said that the relational model is **semantically overloaded**.

There have been many attempts to overcome this problem using **semantic data models**, that is, models that represent more of the meaning of data. The interested reader is referred to the survey papers by Hull and King (1987) and Peckham and Maryanski (1988). However, the relational model is not completely without semantic features. For example, it has domains and keys (see Section 4.2), and functional, multi-valued, and join dependencies (see Chapters 14 and 15).

Poor support for integrity and general constraints

Integrity refers to the validity and consistency of stored data. Integrity is usually expressed in terms of constraints, which are consistency rules that the database is not permitted to violate. In Section 4.3 we introduced the concepts of entity and referential integrity, and in Section 4.2.1 we introduced domains, which are also a type of constraint. Unfortunately, many commercial systems do not fully support these constraints and it is necessary to build them into the applications. This, of course, is dangerous and can lead to duplication of effort and, worse still, inconsistencies. Furthermore, there is no support for general constraints in the relational model, which again means that they have to be built into the DBMS or the application.

As you have seen in Chapters 6 and 7, the SQL standard helps partially resolve this claimed deficiency by allowing some types of constraints to be specified as part of the Data Definition Language (DDL).

Homogeneous data structure

The relational model assumes both horizontal and vertical homogeneity. Horizontal homogeneity means that each tuple of a relation must be composed of the same attributes. Vertical homogeneity means that the values in a particular column of a relation must all come from the same domain. Additionally, the intersection of a row and column must be an atomic value. This fixed structure is too restrictive for many “real-world” objects that have a complex structure, and it leads to unnatural joins, which are inefficient as mentioned previously. In defense of the relational data model, it could equally be argued that its symmetric structure is one of the model’s strengths.

Among the classic examples of complex data and interrelated relationships is a parts explosion in which we wish to represent some object, such as an aircraft, as being composed of parts and composite parts, which in turn are composed of other parts and composite parts, and so on. This weakness has led to research in complex object or nonfirst normal form (NF²) database systems, addressed in the papers by,

for example, Jaeschke and Schek (1982) and Bancilhon and Khoshafian (1989). In the latter paper, objects are defined recursively as follows:

- (1) Every atomic value (such as integer, float, string) is an object.
- (2) If $a_1, a_2, \dots, a_n$ are distinct attribute names and $o_1, o_2, \dots, o_n$ are objects, then $[a_1:o_1, a_2:o_2, \dots, a_n:o_n]$ is a tuple object.
- (3) If $o_1, o_2, \dots, o_n$ are objects, then $S = \{o_1, o_2, \dots, o_n\}$ is a set object.

In this model, the following would be valid objects:

Atomic objects	B003, John, Glasgow
Set	{SG37, SG14, SG5}
Tuple	[branchNo: B003, street: 163 Main St, city: Glasgow]
Hierarchical tuple	[branchNo: B003, street: 163 Main St, city: Glasgow, staff: {SG37, SG14, SG5}]
Set of tuples	{[branchNo: B003, street: 163 Main St, city: Glasgow], [branchNo: B005, street: 22 Deer Rd, city: London]}
Nested relation	{[branchNo: B003, street: 163 Main St, city: Glasgow, staff: {SG37, SG14, SG5}], [branchNo: B005, street: 22 Deer Rd, city: London, staff: {SL21, SL41}]}

Many RDBMSs now allow the storage of **binary large objects (BLOBs)**. A BLOB is a data value that contains binary information representing an image, a digitized video or audio sequence, a procedure, or any large unstructured object. The DBMS does not have any knowledge concerning the content of the BLOB or its internal structure. This prevents the DBMS from performing queries and operations on inherently rich and structured data types. Typically, the database does not manage this information directly, but simply contains a reference to a file. The use of BLOBs is not an elegant solution and storing this information in external files denies it many of the protections naturally afforded by the DBMS. More importantly, BLOBs cannot contain other BLOBs, so they cannot take the form of composite objects. Further, BLOBs generally ignore the behavioral aspects of objects. For example, a picture can be stored as a BLOB in some relational DBMSs. However, the picture can only be stored and displayed. It is not possible to manipulate the internal structure of the picture, or to display or manipulate parts of the picture. An example of the use of BLOBs is given in Figure 19.12.

Limited operations

The relational model has only a fixed set of operations, such as set and tuple-oriented operations, operations that are provided in the SQL specification. However, SQL does not allow new operations to be specified. Again, this is too restrictive to model the behavior of many real-world objects. For example, a GIS application typically uses points, lines, line groups, and polygons, and needs operations for distance, intersection, and containment.

Difficulty handling recursive queries

Atomicity of data means that repeating groups are not allowed in the relational model. As a result, it is extremely difficult to handle recursive queries, that is,

staffNo	managerstaffNo
S005	S004
S004	S003
S003	S002
S002	S001
S001	NULL

(a)

staffNo	managerstaffNo
S005	S004
S004	S003
S003	S002
S002	S001
S001	NULL
S005	S003
S005	S002
S005	S001
S004	S002
S004	S001
S003	S001

(b)

Figure 9.1 (a) Simplified Staff relation; (b) transitive closure of Staff relation.

queries about relationships that a relation has with itself (directly or indirectly). Consider the simplified Staff relation shown in Figure 9.1(a), which stores staff numbers and the corresponding manager’s staff number. How do we find all the managers who directly or indirectly manage staff member S005? To find the first two levels of the hierarchy, we use:

```
SELECT managerStaffNo
FROM Staff
WHERE staffNo = 'S005'
UNION
SELECT managerStaffNo
FROM Staff
WHERE staffNo =
  (SELECT managerStaffNo
   FROM Staff
   WHERE staffNo = 'S005');
```

We can easily extend this approach to find the complete answer to this query. For this particular example, this approach works because we know how many levels in the hierarchy have to be processed. However, if we were to ask a more general query, such as “For each member of staff, find all the managers who directly or indirectly manage the individual,” this approach would be impossible to implement using interactive SQL. To overcome this problem, SQL can be embedded in a high-level programming language, which provides constructs to facilitate iteration (see Appendix I). Additionally, many RDBMSs provide a report writer with similar constructs. In either case, it is the application rather than the inherent capabilities of the system that provides the required functionality.

An extension to relational algebra that has been proposed to handle this type of query is the unary **transitive closure**, or **recursive closure**, operation (Merrett, 1984):

Transitive closure

The transitive closure of a relation R with attributes (A_1, A_2) defined on the same domain is the relation R augmented with all tuples successively deduced by transitivity; that is, if (a, b) and (b, c) are tuples of R , the tuple (a, c) is also added to the result.

This operation cannot be performed with just a fixed number of relational algebra operations, but requires a loop along with the Join, Projection, and Union operations. The result of this operation on our simplified Staff relation is shown in Figure 9.1(b).

Impedance mismatch

In Section 6.1 we noted that SQL-92 lacked *computational completeness*. This is true with most Data Manipulation Languages (DMLs) for RDBMSs. To overcome this problem and to provide additional flexibility, the SQL standard provides embedded SQL to help develop more complex database applications (see Appendix I). However, this approach produces an **impedance mismatch**, because we are mixing different programming paradigms:

- SQL is a declarative language that handles rows of data, whereas a high-level language such as C is a procedural language that can handle only one row of data at a time.
- SQL and 3GLs use different models to represent data. For example, SQL provides the built-in data types Date and Interval, which are not available in traditional programming languages. Thus it is necessary for the application program to convert between the two representations, which is inefficient both in programming effort and in the use of runtime resources. It has been estimated that as much as 30% of programming effort and code space is expended on this type of conversion (Atkinson *et al.*, 1983). Furthermore, as we are using two different type systems, it is not possible to type check the application as a whole automatically.

It is argued that the solution to these problems is not to replace relational languages by row-level object-oriented languages, but to introduce set-level facilities into programming languages (Date, 2000). However, the basis of OODBMSs is to provide a much more seamless integration between the DBMS's data model and the host programming language. We return to this issue in Chapter 27.

Other problems with RDBMSs

- Transactions in business processing are generally short-lived and the concurrency control primitives and protocols such as two-phase locking are not particularly suited for long-duration transactions, which are more common for complex design objects (see Section 22.4, later in this book).
- Schema changes are difficult. Database administrators must intervene to change database structures, and typically programs that access these structures must be modified to adjust to the new structures. These are slow and cumbersome processes even with current technologies. As a result, most organizations are locked into their existing database structures. Even if they are willing and able to change the way they do business to meet new requirements, they are unable to make these changes, because they cannot afford the time and expense required

to modify their information systems (Taylor, 1992). To meet the requirement for increased flexibility, we need a system that caters for natural schema evolution.

- RDBMSs were designed to use content-based *associative access* (that is, declarative statements with selection based on one or more predicates) and are poor at *navigational access* (that is, access based on movement between individual records). Navigational access is important for many of the complex applications we discussed in the previous section.

Of these three problems, the first two are applicable to many DBMSs, not just relational systems. In fact, there is no underlying problem with the relational model that would prevent such mechanisms being implemented.

The latest releases of the SQL standard, SQL:2003, SQL:2006, SQL:2008, and SQL:2011 address some of these deficiencies with the introduction of many new features, such as the ability to define new data types and operations as part of the data definition language, and the addition of new constructs to make the language computationally complete. We discuss SQL:2011 in detail in Section 9.5.

9.3 Storing Objects in a Relational Database

One approach to achieving persistence with an object-oriented programming language, such as C++ or Java, is to use an RDBMS as the underlying storage engine. This requires mapping class instances (that is, objects) to one or more tuples distributed over one or more relations. This can be problematic, as we discuss in this section. For the purposes of discussion, consider the inheritance hierarchy shown in Figure 9.2, which has a *Staff* superclass and three subclasses: *Manager*, *SalesPersonnel*, and *Secretary*.

To handle this type of class hierarchy, we have two basic tasks to perform:

- Design the relations to represent the class hierarchy.

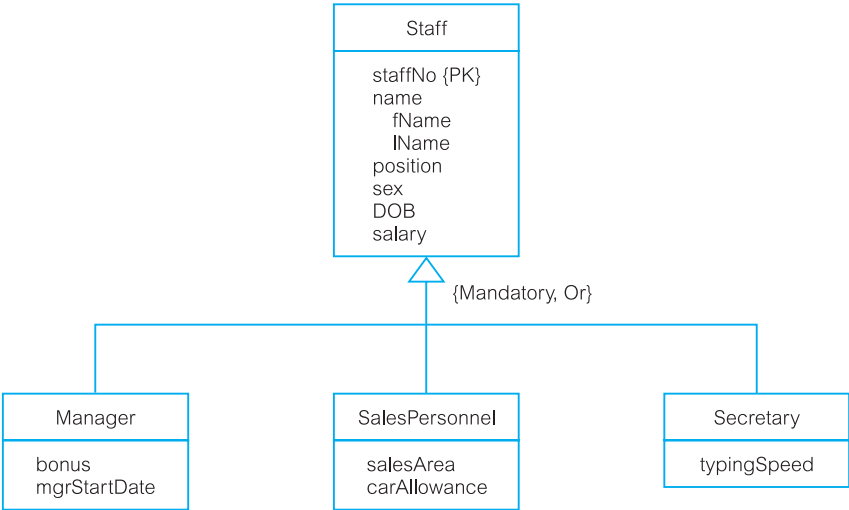


Figure 9.2 Sample inheritance hierarchy for Staff.

- Design how objects will be accessed, which means:
 - writing code to decompose the objects into tuples and store the decomposed objects in relations;
 - writing code to read tuples from the relations and reconstruct the objects.

We now describe these two tasks in more detail.

9.3.1 Mapping Classes to Relations

There are a number of strategies for mapping classes to relations, although each results in a loss of semantic information. The code to make objects persistent and to read the objects back from the database is dependent on the strategy chosen. We consider three alternatives:

- (1) Map each class or subclass to a relation.
- (2) Map each subclass to a relation.
- (3) Map the hierarchy to a single relation.

Map each class or subclass to a relation

One approach is to map each class or subclass to a relation. For the hierarchy given in Figure 9.2, this would give the following four relations (with the primary key underlined):

```
Staff (staffNo, fName, lName, position, sex, DOB, salary)
Manager (staffNo, bonus, mgrStartDate)
SalesPersonnel (staffNo, salesArea, carAllowance)
Secretary (staffNo, typingSpeed)
```

We assume that the underlying data type of each attribute is supported by the RDBMS, although this may not be the case—in which case we would need to write additional code to handle the transformation of one data type to another.

Unfortunately, with this relational schema we have lost semantic information: it is no longer clear which relation represents the superclass and which relations represent the subclasses. We would therefore have to build this knowledge into each application, which as we have said elsewhere can lead to duplication of code and potential for inconsistencies to arise.

Map each subclass to a relation

A second approach is to map each subclass to a relation. For the hierarchy given in Figure 9.2, this would give the following three relations:

```
Manager (staffNo, fName, lName, position, sex, DOB, salary, bonus, mgrStartDate)
SalesPersonnel (staffNo, fName, lName, position, sex, DOB, salary, salesArea, carAllowance)
Secretary (staffNo, fName, lName, position, sex, DOB, salary, typingSpeed)
```

Again, we have lost semantic information in this mapping: it is no longer clear that these relations are subclasses of a single generic class. In this case, to produce a list of all staff we would have to select the tuples from each relation and then union the results together.

Map the hierarchy to a single relation

A third approach is to map the entire inheritance hierarchy to a single relation, giving in this case:

```
Staff (staffNo, fName, lName, position, sex, DOB, salary, bonus, mgrStartDate, salesArea,
      carAllowance, typingSpeed, typeFlag)
```

The attribute typeFlag is a discriminator to distinguish which type each tuple is (for example, it may contain the value 1 for a Manager tuple, 2 for a SalesPersonnel tuple, and 3 for a Secretary tuple). Again, we have lost semantic information in this mapping. Additionally, this mapping will produce an unwanted number of nulls for attributes that do not apply to that tuple. For example, for a Manager tuple, the attributes salesArea, carAllowance, and typingSpeed will be null.

9.3.2 Accessing Objects in the Relational Database

Having designed the structure of the relational database, we now need to insert objects into the database and then provide a mechanism to read, update, and delete the objects. For example, to insert an object into the first relational schema in the previous section (that is, where we have created a relation for each class), the code may look something like the following using programmatic SQL (see Appendix I):

```
Manager* pManager = new Manager;    // create a new Manager object
... code to set up the object ...
EXEC SQL INSERT INTO Staff VALUES (:pManager->staffNo, :pManager->
fName,
    :pManager->lName, :pManager->position, :pManager->sex, :pManager->DOB,
    :pManager->salary);
EXEC SQL INSERT INTO Manager VALUES (:pManager->bonus,
    :pManager->mgrStartDate);
```

On the other hand, if Manager had been declared as a persistent class, then the following (indicative) statement would make the object persistent in an OODBMS:

```
Manager* pManager = new Manager;
```

In Section 27.3.3, we examine different approaches for declaring persistent classes. If we wish to retrieve some data from the relational database, say the details for managers with a bonus in excess of £1000, the code might look something like the following:

```
Manager* pManager = new Manager;    // create a new Manager object
EXEC SQL WHENEVER NOT FOUND GOTO done; // set up error handling
EXEC SQL DECLARE managerCursor      // create cursor for SELECT
CURSOR FOR
    SELECT staffNo, fName, lName, salary, bonus
    FROM Staff s, Manager m          // Need to join Staff and Manager
    WHERE s.staffNo = m.staffNo AND bonus > 1000;
EXEC SQL OPEN managerCursor;
for ( ; ; ) {
```



```

EXEC SQL FETCH managerCursor // fetch the next record in the result
INTO :staffNo, :fName, :lName, :salary, :bonus;
pManager->staffNo = :staffNo; // transfer the data to the Manager object
pManager->fName = :fName;
pManager->lName = :lName;
pManager->salary = :salary;
pManager->bonus = :bonus;
strcpy(pManager->position, "Manager");
}
EXEC SQL CLOSE managerCursor; // close the cursor before completing

```

On the other hand, to retrieve the same set of data in an OODBMS, we may write the following code:

```

os_Set<Manager*> &highBonus
= managerExtent->query("Manager*", "bonus > 1000", db1);

```

This statement queries the extent of the Manager class (managerExtent) to find the required instances (bonus > 1000) from the database (in this example, db1). The commercial OODBMS ObjectStore has a collection template class os_Set, which has been instantiated in this example to contain pointers to Manager objects <Manager*>. In Section 28.3 we provide additional details of object persistence and object retrieval with ObjectStore.

The previous examples have been given to illustrate the complexities involved in mapping an object-oriented language to a relational database. The OODBMS approach that we discuss in Chapters 27 and 28 attempts to provide a more seamless integration of the programming language data model and the database data model, thereby removing the need for complex transformations, which as we discussed earlier could account for as much as 30% of programming effort.

9.4 Introduction to Object-Relational Database Systems

Relational DBMSs are currently the dominant database technology, with an estimated total software revenue worldwide of US\$24 billion in 2011 and estimated to grow to about US\$37 billion by 2016. Until recently, the choice of DBMS seemed to be between the relational DBMS and the object-oriented DBMS. However, many vendors of RDBMS products are conscious of the threat and promise of the OODBMS. They agree that traditional relational DBMSs are not suited to the advanced applications discussed in Section 9.1, and that added functionality is required. However, they reject the claim that extended RDBMSs will not provide sufficient functionality or will be too slow to cope adequately with the new complexity.

If we examine the advanced database applications that are emerging, we find they make extensive use of many object-oriented features such as a user-extensible type system, encapsulation, inheritance, polymorphism, dynamic binding of methods, complex objects including non-first normal form objects, and object identity. The most obvious way to remedy the shortcomings of the relational model is to extend the model with these types of feature. This is the approach that has been

taken by many extended relational DBMSs, although each has implemented different combinations of features. Thus, there is no single extended relational model; rather, there are a variety of these models, whose characteristics depend upon the way and the degree to which extensions were made. However, all the models do share the same basic relational tables and query language, all incorporate some concept of “object,” and some have the ability to store methods (or procedures or triggers) as well as data in the database.

Various terms have been used for systems that have extended the relational data model. The original term that was used to describe such systems was the *Extended Relational DBMS* (ERDBMS) and the term *Universal Server* or *Universal DBMS* (UDBMS) has also been used. However, in recent years the more descriptive term *Object-Relational DBMS* has been used to indicate that the system incorporates some notion of “object.” In this chapter we use the term Object-Relational DBMS (ORDBMS). Three of the leading RDBMS vendors—Oracle, Microsoft, and IBM—have all extended their systems into ORDBMSs, although the functionality provided by each is slightly different. The concept of the ORDBMS, as a hybrid of the RDBMS and the OODBMS, is very appealing, preserving the wealth of knowledge and experience that has been acquired with the RDBMS—so much so that some analysts predict the ORDBMS will have a 50% larger share of the market than the RDBMS.

As might be expected, the standards activity in this area is based on extensions to the SQL standard. The national standards bodies have been working on object extensions to SQL since 1991. These extensions have become part of the SQL standard, with releases in 1999, referred to as SQL:1999, 2003, (SQL:2003), 2006 with extensions for XML (SQL:2006), 2008 (SQL:2008) and 2011 (SQL:2011). These releases of the SQL standard are an ongoing attempt to standardize extensions to the relational model and query language. We discuss the object extensions to SQL in some detail in this chapter. In this book, we generally use the term SQL:2011 to refer to the 1999, 2003, 2006, 2008, and 2011 releases of the standard.

Stonebraker’s view

Stonebraker (1996) proposed a four-quadrant view of the database world, as illustrated in Figure 9.3. In the lower-left quadrant are applications that process simple data and have no requirements for querying the data. These types of application—for example, standard text processing packages such as Microsoft Word—can use the underlying operating system to obtain the essential DBMS functionality of persistence. In the lower-right quadrant are applications that process complex data, but again have no significant requirements for querying the data. For these types of application—for example, computer-aided design packages—an OODBMS may be an appropriate choice of DBMS. In the top-left quadrant are applications that process simple data and also have requirements for complex querying. Many traditional business applications fall into this quadrant and an RDBMS may be the most appropriate DBMS. Finally, in the top-right quadrant are applications that process complex data and have complex querying requirements. This represents many of the advanced database applications that we examined in Section 9.1 and for these applications Stonebraker argues that an ORDBMS may be the most appropriate choice of DBMS.

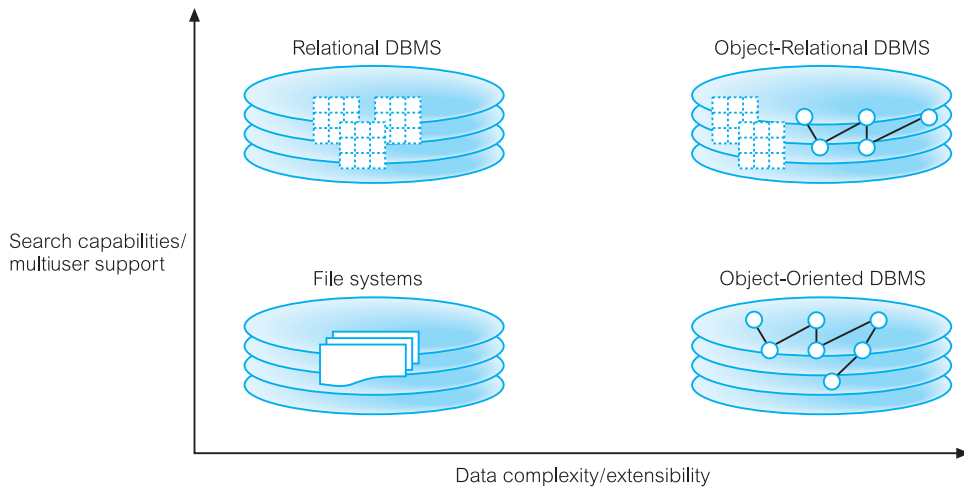


Figure 9.3 Four-quadrant view of the database world.

Although interesting, this is a very simplistic classification and unfortunately many database applications are not so easily compartmentalized. Further, with the introduction of the ODMG data model and query language, which we will discuss in Section 28.2, and the addition of object-oriented data management features to SQL, the distinction between the ORDBMS and OODBMS is less clear.

Advantages of ORDBMSs

Apart from the advantages of resolving many of the weaknesses cited in Section 9.2, the main advantages of extending the relational data model come from *reuse* and *sharing*. Reuse comes from the ability to extend the DBMS server to perform standard functionality centrally, rather than have it coded in each application. For example, applications may require spatial data types that represent points, lines, and polygons, with associated functions that calculate the distance between two points, the distance between a point and a line, whether a point is contained within a polygon, the distance between two polygonal regions overlap, among others. If we can embed this functionality in the server, it allows us to avoid the necessity of defining it in each application that needs it, and consequently allows the functionality to be shared by all applications. These advantages also give rise to increased productivity, both for the developer and for the end-user.

Another obvious advantage is that the extended relational approach preserves the significant body of knowledge and experience that has gone into developing relational applications. This is a significant advantage, as many organizations would find it prohibitively expensive to change. If the new functionality is designed appropriately, this approach should allow organizations to take advantage of the new extensions in an evolutionary way without losing the benefits of current database features and functions. Thus, an ORDBMS could be introduced in an integrative fashion, as proof-of-concept projects. The SQL:2011 standard is designed to

be compatible with the SQL2 standard, and so any ORDBMS that complies with SQL:2011 should provide this capability.

Disadvantages of ORDBMSs

The ORDBMS approach has the obvious disadvantages of complexity and associated increased costs. In addition, there are the proponents of the relational approach who believe the essential simplicity and purity of the relational model are lost with these types of extension. There are also those who believe that the RDBMS is being extended for what will be a minority of applications that do not achieve optimal performance with current relational technology.

In addition, object-oriented purists are not attracted by these extensions, either. They argue that the terminology of object-relational systems is revealing. Instead of discussing object models, terms like “user-defined data types” are used. The terminology of object-orientation abounds with terms like “abstract types,” “class hierarchies,” and “object models.” However, ORDBMS vendors are attempting to portray object models as extensions to the relational model with some additional complexities. This potentially misses the point of object orientation, highlighting the large semantic gap between these two technologies. Object applications are simply not as data-centric as relational-based ones. Object-oriented models and programs deeply combine relationships and encapsulated objects to more closely mirror the real world. This defines broader sets of relationships than those expressed in SQL, and involves functional programs interspersed in the object definitions. In fact, objects are fundamentally not extensions of data, but a completely different concept with far greater power to express real-world relationships and behaviors.

In Chapter 6 we noted that the objectives of a database language included having the capability to be used with minimal user effort, and having a command structure and syntax that must be relatively easy to learn. The initial SQL standard, released in 1989, appeared to satisfy these objectives. The release in 1992 increased in size from 120 pages to approximately 600 pages, and it is more questionable whether it satisfied these objectives. Unfortunately, the size of the SQL:2011 standard is even more daunting, and it would seem that these two objectives are no longer being fulfilled or even being considered by the standards bodies.

9.5 SQL:2011

In Chapters 6 and 7 we provided an extensive tutorial on the features of the ISO SQL standard, concentrating mainly on those features present in the 1992 version of the standard, commonly referred to as SQL2 or SQL-92. ANSI (X3H2) and ISO (ISO/IEC JTC1/SC21/WG3) SQL standardization have added features to the SQL specification to support object-oriented data management, the latest release of which is SQL:2011 (ISO, 2011a). As we mentioned earlier, the SQL:2011 standard is extremely large and comprehensive, and is divided into the following parts:

- (1) *ISO/IEC 9075-1: SQL/Framework.*
- (2) *ISO/IEC 9075-2: SQL/Foundation*, which includes new data types, user-defined types, rules and triggers, transactions, stored routines, and binding methods (embedded SQL, dynamic SQL, and direct invocation of SQL).

- (3) *ISO/IEC 9075-3: SQL/CLI* (Call-Level Interface), which specifies the provision of an API interface to the database, as we discuss in Appendix I, based on the SQL Access Group and X/Open's CLI definitions.
- (4) *ISO/IEC 9075-4: SQL/PSM* (Persistent Stored Modules), which allows procedures and user-defined functions to be written in a 3GL or in SQL and stored in the database, making SQL computationally complete.
- (5) *ISO/IEC 9075-9: SQL/MED* (Management of External Data), which defines extensions to SQL to support management of external data through the use of foreign tables and datalink data types.
- (6) *ISO/IEC 9075-10: SQL/OLB* (Object Language Bindings), which defines facilities for embedding SQL statements in Java programs.
- (7) *ISO/IEC 9075-11: SQL/Schemata* (Information and Definition Schemas), which defines two schemas `INFORMATION_SCHEMA` and `DEFINITION_SCHEMA`. The Information Schema defines views about database objects such as tables, views, and columns. These views are defined in terms of the base tables in the Definition Schema.
- (8) *ISO/IEC 9075-13: SQL/JRT* (Java Routines and Types Using the Java Programming Language), which defines extensions to SQL to allow the invocation of static methods written in Java as SQL-invoked routines, and to use classes defined in Java as SQL structured types.
- (9) *ISO/IEC 9075-14: SQL/XML* (XML-Related Specifications), which defines extensions to SQL to enable creation and manipulation of XML documents.

In this section we examine some of these features, and cover these topics:

- type constructors for row types and reference types;
- user-defined types (distinct types and structured types) that can participate in supertype/subtype relationships;
- user-defined procedures, functions, methods, and operators;
- type constructors for collection types (arrays, sets, lists, and multisets);
- support for large objects—Binary Large Objects (BLOBs) and Character Large Objects (CLOBs);
- recursion.

Many of the object-oriented concepts that we discuss in Appendix K are in the proposal. The definitive release of the SQL:1999 standard became significantly behind schedule and some of the object management features were deferred to a later version of the standard. Many of these are still missing from SQL:2011.

9.5.1 Row Types

A **row type** is a sequence of field name/data type pairs that provides a data type to represent the types of rows in tables, so that complete rows can be stored in variables, passed as arguments to routines, and returned as return values from function calls. A row type can also be used to allow a column of a table to contain row values. In essence, the row is a table nested within a table.

EXAMPLE 9.1 Use of row type

To illustrate the use of row types, we create a simplified Branch table consisting of the branch number and address, and insert a record into the new table:

```
CREATE TABLE Branch (
    branchNo CHAR(4),
    address ROW(street          VARCHAR(25),
               city            VARCHAR(15),
               postcode        ROW(cityIdentifier VARCHAR(4),
                                   subPart      VARCHAR(4))));

INSERT INTO Branch
VALUES ('B005', ROW('23 Deer Rd', 'London', ROW('SW1', '4EH')));
UPDATE Branch
SET address = ROW ('23 Deer Rd', 'London', ROW ('SW1', '4EH'))
WHERE address = ROW ('23 Deer Rd', 'London', ROW ('SW1', '4EH'));
```

9.5.2 User-Defined Types

SQL:2011 allows the definition of **user-defined types** (UDTs), which we have previously referred to as abstract data types (ADTs). They may be used in the same way as the predefined types (for example, CHAR, INT, FLOAT). UDTs are subdivided into two categories: distinct types and structured types. The simpler type of UDT is the **distinct type**, which allows differentiation between the same underlying base types. For example, we could create the following two distinct types:

```
CREATE TYPE OwnerNumberType AS VARCHAR(5) FINAL;
CREATE TYPE StaffNumberType AS VARCHAR(5) FINAL;
```

If we now attempt to treat an instance of one type as an instance of the other type, an error would be generated. Note that although SQL also allows the creation of domains to distinguish between different data types, the purpose of an SQL domain is solely to constrain the set of valid values that can be stored in a column with that domain.

In its more general case, a UDT definition consists of one or more **attribute definitions**, zero or more **routine declarations** (methods) and, in a subsequent release, **operator declarations**. We refer to routines and operators generically as routines. In addition, we can also define the equality and ordering relationships for the UDT using the CREATE ORDERING FOR statement.

The value of an attribute can be accessed using the common dot notation (.). For example, assuming *p* is an instance of the UDT *PersonType*, which has an attribute *fName* of type VARCHAR, we can access the *fName* attribute as:

```
p.fName
p.fName = 'A. Smith'
```

Encapsulation and observer and mutator functions

SQL encapsulates each attribute of structured types by providing a pair of built-in routines that are invoked whenever a user attempts to reference the attribute, an **observer** (get) function and a **mutator** (set) function. The observer function returns

the current value of the attribute; the mutator function sets the value of the attribute to a value specified as a parameter. These functions can be redefined by the user in the definition of the UDT. In this way, attribute values are encapsulated and are accessible to the user only by invoking these functions. For example, the observer function for the fName attribute of `PersonType` would be:

```
FUNCTION fName(p PersonType) RETURNS VARCHAR(15)
    RETURN p.fName;
```

and the corresponding mutator function to set the value to *newValue* would be:

```
FUNCTION fName(p PersonType RESULT, newValue VARCHAR(15))
    RETURNS PersonType
BEGIN
    p.fName = newValue;
    RETURN p;
END;
```

Constructor functions and the NEW expression

A (public) **constructor function** is automatically defined to create new instances of the type. The constructor function has the same name and type as the UDT, takes zero arguments, and returns a new instance of the type with the attributes set to their default value. User-defined **constructor methods** can be provided by the user to initialize a newly created instance of a structured type. Each method must have the same name as the structured type but the parameters must be different from the system-supplied constructor. In addition, each user-defined constructor method must differ in the number of parameters or in the data types of the parameters. For example, we could initialize a constructor for type `PersonType` as follows:

```
CREATE CONSTRUCTOR METHOD PersonType (fN VARCHAR(15),
    IN VARCHAR(15), sx CHAR) RETURNS PersonType SELF AS RESULT
    BEGIN
        SET SELF.fName = fN;
        SET SELF.IName = IN;
        SET SELF.sex = sx;
        RETURN SELF;
    END;
```

The **NEW** expression can be used to invoke the system-supplied constructor function; for example:

```
SET p = NEW PersonType();
```

User-defined constructor methods must be invoked in the context of the **NEW** expression. For example, we can create a new instance of `PersonType` and invoke the previous user-defined constructor method as follows:

```
SET p = NEW PersonType('John', 'White', 'M');
```

This is effectively translated into:

```
SET p = PersonType().PersonType('John', 'White', 'M');
```


Other UDT methods

Instances of UDTs can be constrained to exhibit specified ordering properties. The `EQUALS ONLY BY` and `ORDER FULL BY` clauses may be used to specify type-specific functions for comparing UDT instances. The ordering can be performed using methods that are qualified as:

- **RELATIVE.** The relative method is a function that returns a 0 for equals, a negative value for less than, and a positive value for greater than.
- **MAP.** The map method uses a function that takes a single argument of the UDT type and returns a predefined data type. Comparing two UDTs is achieved by comparing the two map values associated with them.
- **STATE.** The state method compares the attributes of the operands to determine an order.

CAST functions can also be defined to provide user-specified conversion functions between different UDTs. In a subsequent version of the standard, it may also be possible to override some of the built-in operators.

EXAMPLE 9.2 Definition of a new UDT

To illustrate the creation of a new UDT, we create a UDT for a `PersonType`.

```
CREATE TYPE PersonType AS (
    dateOfBirth    DATE,
    fName          VARCHAR(15),
    lName          VARCHAR(15),
    sex            CHAR)
INSTANTIABLE
NOT FINAL
REF IS SYSTEM GENERATED
INSTANCE METHOD age () RETURNS INTEGER,
INSTANCE METHOD age (DOB DATE) RETURNS PersonType;
CREATE INSTANCE METHOD age () RETURNS INTEGER
FOR PersonType
BEGIN
    RETURN /* age calculated from SELF.dateOfBirth */;
END;
CREATE INSTANCE METHOD age (DOB DATE) RETURNS PersonType
FOR PersonType
BEGIN
    SELF.dateOfBirth = /* code to set dateOfBirth from DOB */;
    RETURN SELF;
END;
```

This example also illustrates the use of **stored** and **virtual attributes**. A **stored attribute** is the default type with an attribute name and data type. The data type can be any known data type, including other UDTs. In contrast, **virtual attributes** do not correspond to stored data, but to derived data. There is an implied virtual attribute `age`, which is derived using the (observer) `age` function and assigned using

the (mutator) `age` function.[†] From the user's perspective, there is no distinguishable difference between a stored attribute and a virtual attribute—both are accessed using the corresponding observer and mutator functions. Only the designer of the UDT will know the difference.

The keyword `INSTANTIABLE` indicates that instances can be created for this type. If `NOT INSTANTIABLE` had been specified, we would not be able to create instances of this type, only from one of its subtypes. The keyword `NOT FINAL` indicates that we can create subtypes of this user-defined type. We discuss the clause `REF IS SYSTEM GENERATED` in Section 9.5.6.

9.5.3 Subtypes and Supertypes

SQL:2011 allows UDTs to participate in a subtype/supertype hierarchy using the `UNDER` clause. A type can have more than one subtype but currently only one supertype (that is, multiple inheritance is not supported). A subtype inherits all the attributes and behavior (methods) of its supertype and it can define additional attributes and methods like any other UDT and it can override inherited methods.

EXAMPLE 9.3 Creation of a subtype using the `UNDER` clause

To create a subtype `StaffType` of the supertype `PersonType`, we write:

```
CREATE TYPE StaffType UNDER PersonType AS (
    staffNo      VARCHAR(5),
    position     VARCHAR(10)      DEFAULT 'Assistant',
    salary       DECIMAL(7, 2),
    branchNo     CHAR(4)
INSTANTIABLE
NOT FINAL
INSTANCE METHOD isManager () RETURNS BOOLEAN;
CREATE INSTANCE METHOD isManager() RETURNS BOOLEAN
FOR StaffType
BEGIN
    IF SELF.position = 'Manager' THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF
END)
```

`StaffType` as well as having the attributes defined within the `CREATE TYPE`, also includes the inherited attributes of `PersonType`, along with the associated observer and mutator functions and any specified methods. In particular, the clause `REF IS SYSTEM GENERATED` is also in effect inherited. In addition, we have defined an instance method `isManager` that checks whether the specified member of staff is a Manager. We show how this method can be used in Section 9.5.8.

[†]Note that the function name `age` has been overloaded here. We discuss how SQL distinguishes between these two functions in Section 9.5.5.

An instance of a subtype is considered an instance of all its supertypes. SQL:2011 supports the concept of **substitutability**: that is, whenever an instance of a supertype is expected an instance of the subtype can be used in its place. The type of a UDT can be tested using the TYPE predicate. For example, given a UDT, say Udt1, we can apply the following tests:

```

TYPE Udt1 IS OF (PersonType)           // Check Udt1 is the PersonType or any of
                                           // its subtypes
TYPE Udt1 IS OF (ONLY PersonType)       // Check Udt1 is the PersonType

```

In SQL:2011, as in most programming languages, every instance of a UDT must be associated with exactly *one most specific type*, which corresponds to the lowest subtype assigned to the instance. Thus, if the UDT has more than one direct supertype, then there must be a single type to which the instance belongs, and that single type must be a subtype of all the types to which the instance belongs. In some cases, this can require the creation of a large number of types. For example, a type hierarchy might consist of a maximal supertype *Person*, with *Student* and *Staff* as subtypes; *Student* itself might have three direct subtypes: *Undergraduate*, *Postgraduate*, and *PartTimeStudent*, as illustrated in Figure 9.4(a). If an instance has the type *Person* and *Student*, then the most specific type in this case is *Student*, a nonleaf type, since *Student* is a subtype of *Person*. However, with the current type hierarchy an instance cannot have the type *PartTimeStudent* as well as *Staff*, unless we create a type *PTStudentStaff*, as illustrated in Figure 9.4(b). The new leaf type, *PTStudentStaff*, is then the most specific type of this instance. Similarly, some of the full-time undergraduate and postgraduate students may work part time (as opposed to full-time employees being part-time students), and so we would also have to add subtypes for *FTUGStaff* and *FTPGStaff*. If we generalized this approach, we could potentially create a large number of subtypes. In some cases, a better approach may be to use inheritance at the level of tables as opposed to types, as we discuss shortly.

Privileges

To create a subtype, a user must have the UNDER privilege on the user-defined type specified as a supertype in the subtype definition. In addition, a user must have USAGE privilege on any user-defined type referenced within the new type.

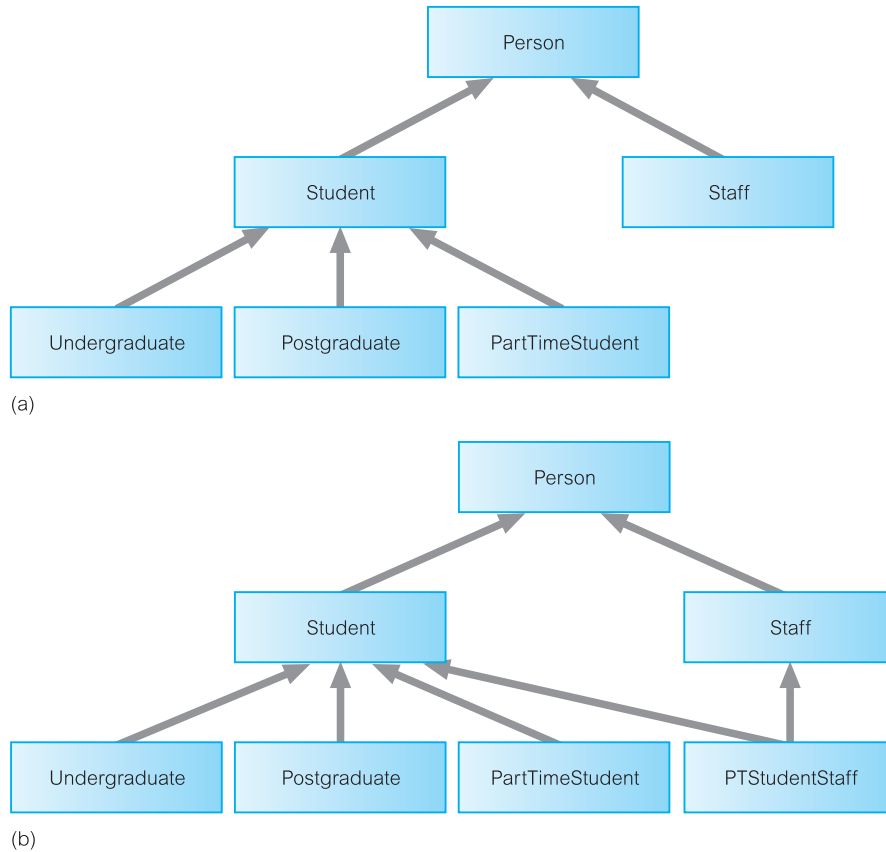
Prior to SQL:1999, the SELECT privilege applied only to columns of tables and views. From SQL:1999, the SELECT privilege also applies to structured types, but only when instances of those types are stored in typed tables and only when the dereference operator is used from a REF value to the referenced row and then invokes a method on that referenced row. When invoking a method on a structured value that is stored in a column of any ordinary SQL table, SELECT privilege is required on that column. If the method is a mutator function, UPDATE privilege is also required on the column. In addition, EXECUTE privilege is required on all methods that are invoked.

9.5.4 User-Defined Routines

User-defined routines (UDRs) define methods for manipulating data and are an important adjunct to UDTs providing the required behavior for the UDTs.

Figure 9.4

(a) Initial Student/Staff hierarchy;
(b) modified Student/Staff hierarchy.



An ORDBMS should provide significant flexibility in this area, such as allowing UDRs to return complex values that can be further manipulated (such as tables), and support for overloading of function names to simplify application development.

In SQL:2011, UDRs may be defined as part of a UDT or separately as part of a schema. An *SQL-invoked routine* may be a procedure, function, or method. It may be externally provided in a standard programming language such as C, C++, or Java, or defined completely in SQL using extensions that make the language computationally complete, as we discuss in Section 9.5.10.

An *SQL-invoked procedure* is invoked from an SQL CALL statement. It may have zero or more parameters, each of which may be an input parameter (IN), an output parameter (OUT), or both an input and output parameter (INOUT), and it has a body if it is defined fully within SQL. An *SQL-invoked function* returns a value; any specified parameters must be input parameters. One input parameter can be designated as the result (using the RESULT keyword), in which case the parameter's data type must match the type of the RETURNS type. Such a function is called *type-preserving*, because it always returns a value whose runtime type is the same as the

most specific type (see Section 9.5.3) of the RETURN parameter (not some subtype of that type). Mutator functions are always type-preserving. An *SQL-invoked method* is similar to a function but has some important differences:

- a method is associated with a single UDT;
- the signature of every method associated with a UDT must be specified in that UDT and the definition of the method must specify that UDT (and must also appear in the same schema as the UDT).

There are three types of methods:

- *constructor methods*, which initialize a newly created instance of a UDT;
- *instance methods*, which operate on specific instances of a UDT;
- *static methods*, which are analogous to class methods in some object-oriented programming languages and operate at the UDT level rather than at the instance level.

In the first two cases, the methods include an additional implicit first parameter called SELF whose data type is that of the associated UDT. We saw an example of the SELF parameter in the user-defined constructor method for `PersonType`. A method can be invoked in one of three ways:

- a constructor method is invoked using the NEW expression, as discussed previously;
- an instance method is invoked using the standard dot notation; for example, `p.fName`, or using the generalized invocation format, for example, `(p AS StaffType).fName()`;
- a static method is invoked using `::`, for example, if `totalStaff` is a static method of `StaffType`, we could invoke it as `StaffType::totalStaff()`.

An *external routine* is defined by specifying an external clause that identifies the corresponding “compiled code” in the operating system’s file storage. For example, we may wish to use a function that creates a thumbnail image for an object stored in the database. The functionality cannot be provided in SQL and so we have to use a function provided externally, using the following CREATE FUNCTION statement with an EXTERNAL clause:

```
CREATE FUNCTION thumbnail(IN myImage ImageType) RETURNS BOOLEAN
EXTERNAL NAME '/usr/dreamhome/bin/images/thumbnail'
LANGUAGE C
PARAMETER STYLE GENERAL
DETERMINISTIC
NO SQL;
```

This SQL statement associates the SQL function named `thumbnail` with an external file, “thumbnail.” It is the user’s responsibility to provide this compiled function. Thereafter, the ORDBMS will provide a method to dynamically link this object file into the database system so that it can be invoked when required. The procedure for achieving this is outside the bounds of the SQL standard and so is left as implementation-defined. A routine is *deterministic* if it always returns the same return value(s) for a given set of inputs. The `NO SQL` indicates that this function contains no SQL statements. The other options are `READS SQL DATA`, `MODIFIES SQL DATA`, and `CONTAINS SQL`.

9.5.5 Polymorphism

In Appendices K.7 and K.8, we discuss the concepts of overriding, overloading, and more generally polymorphism. Different routines may have the same name; that is, routine names may be overloaded, for example to allow a UDT subtype to redefine a method inherited from a supertype, subject to the following constraints:

- No two functions in the same schema are allowed to have the same signature, that is, the same number of arguments, the same data types for each argument, and the same return type.
- No two procedures in the same schema are allowed to have the same name and the same number of parameters.

Overriding applies only to methods and then only based on the runtime value of the implicit SELF argument (note that a method definition has *parameters*, and a method invocation has *arguments*). SQL uses a generalized object model, so that the types of all arguments to a routine are taken into consideration when determining which routine to invoke, in order from left to right. Where there is not an exact match between the data type of an argument and the data type of the parameter specified, type precedence lists are used to determine the closest match. The exact rules for routine determination for a given invocation are quite complex and we do not give the full details here, but illustrate the mechanism for instance methods.

Instance method invocation

The mechanism for determining the appropriate invocation of an instance method is divided into two phases representing static analysis and runtime execution. In this section we provide an overview of these phases. The first phase proceeds as follows:

- All routines with the appropriate name are identified (all remaining routines are eliminated).
- All procedures/functions and all methods for which the user does not have EXECUTE privilege are eliminated.
- All methods that are not associated with the declared type (or subtype) of the implicit SELF argument are eliminated.
- All methods whose parameters are not equal to the number of arguments in the method invocation are eliminated.
- For the methods that remain, the system checks that the data type of each parameter matches the precedence list of the corresponding argument, eliminating those methods that do not match.
- If there are no candidate methods remaining a syntax error occurs.

For the remaining candidate methods the second (runtime) phase proceeds as follows:

- If the most specific type of the runtime value of the implicit argument to the method invocation has a type definition that includes one of the candidate methods, then that method is selected for execution.

- If the most specific type of the runtime value of the implicit argument to the method invocation has a type definition that does not include one of the candidate methods, then the method selected for execution is the candidate method whose associated type is the *nearest* supertype of all supertypes having such a method.

The argument values are converted to the parameter data types, if appropriate, and the body of the method is executed.

9.5.6 Reference Types and Object Identity

Object identity is that aspect of an object that never changes and that distinguishes the object from all other objects (see Appendix K.3). Ideally, an object's identity is independent of its name, structure, and location, and persists even after the object has been deleted, so that it may never be confused with the identity of any other object. Other objects can use an object's identity as a unique way of referencing it.

Before SQL:1999, the only way to define relationships between tables was using the primary key/foreign key mechanism, which in SQL2 could be expressed using the referential table constraint clause REFERENCES, as discussed in Section 7.2.4. Since SQL:1999, **reference types** have been able to be used to define relationships between row types and uniquely identify a row within a table. A reference type value can be stored in one (typed) table and used as a direct reference to a specific row in some base table that has been defined to be of this type (similar to the notion of a pointer type in C or C++). In this respect, a reference type provides a similar functionality as the object identifier (OID) of object-oriented DBMSs (see Appendix K.3). Thus, references allow a row to be shared among multiple tables and enable users to replace complex join definitions in queries with much simpler path expressions. References also give the optimizer an alternative way to navigate data instead of using value-based joins.

REF IS SYSTEM GENERATED in a CREATE TYPE statement indicates that the actual values of the associated REF type are provided by the system, as in the PersonType created in Example 9.2. Other options are available but we omit the details here; the default is REF IS SYSTEM GENERATED. As we see shortly, a base table can be created to be of some structured type. Other columns can be specified for the table but at least one column must be specified, namely a column of the associated REF type, using the clause REF IS <columnName> SYSTEM GENERATED. This column is used to contain unique identifiers for the rows of the associated base table. The identifier for a given row is assigned when the row is inserted into the table and remains associated with that row until it is deleted.

9.5.7 Creating Tables

To maintain upwards compatibility with the SQL2 standard, it is still necessary to use the CREATE TABLE statement to create a table, even if the table consists of a single UDT. In other words, a UDT instance can persist only if it is stored as the column value in a table. There are several variations of the CREATE TABLE statement, as Examples 9.4–9.6 illustrate.

EXAMPLE 9.4 Creation of a table based on a UDT

To create a table using the `PersonType` UDT, we could write:

```
CREATE TABLE Person (
    info PersonType
    CONSTRAINT DOB_Check CHECK(dateOfBirth > DATE '1900-01-01'));
```

or

```
CREATE TABLE Person OF PersonType (
    dateOfBirth WITH OPTIONS
    CONSTRAINT DOB_Check CHECK (dateOfBirth > DATE '1900-01-01')
    REF IS PersonID SYSTEM GENERATED);
```

In the first instance, we would access the columns of the `Person` table using a path expression such as `'Person.info.fName'`; in the second version, we would access the columns using a path expression such as `'Person.fName'`.

More importantly, tables constructed using the second version (`CREATE TABLE . . . OF` statement) are called **typed tables**. The rows of a typed table are considered to be **objects**, and the rows in the first version are not, even though the same UDT has been used in both cases. A typed table has a column for every attribute of the structured type on which it is based. In addition, a typed table has a *self-referencing column* that contains a unique OID (known as a *reference*) for each row of the table. Objects are inserted into a typed table using the normal `INSERT` statement as for any relational table. Apart from the self-referencing column and the attributes of the structured type, no additional columns can be added to the table definition.

The OID is generated automatically when a new row is inserted into the typed table. However, to gain access to the OIDs stored in the self-referencing column, we have to give the column name explicitly using the `REF IS` clause (`PersonID` in our example). Note that a typed table definition must also repeat a reference generation specification that is consistent with the corresponding specification in the UDT: system-generated, user-generated, or derived.

EXAMPLE 9.5 Creation of a subtable using the UNDER clause

We can create a table for staff using table inheritance:

```
CREATE TABLE Staff OF StaffType UNDER Person;
```

When we insert rows into the `Staff` table, the values of the inherited columns are inserted into the `Person` table. Similarly, when we delete rows from the `Staff` table, the rows disappear from both the `Staff` and `Person` tables. As a result, when we access all rows of `Person`, this will also include all `Staff` details.

There are restrictions on the population of a table hierarchy:

- Each row of the supertable `Person` can correspond to at most one row in `Staff`.
- Each row in `Staff` must have exactly one corresponding row in `Person`.

The semantics maintained are those of *containment*: a row in a subtable is in effect “contained” in its supertables. We would expect the SQL `INSERT`, `UPDATE`, and

DELETE statements to maintain this consistency when the rows of subtables and supertables are being modified, as follows (at least conceptually):

- When a row is inserted into a subtable, then the values of any inherited columns of the table are inserted into the corresponding supertables, cascading upwards in the table hierarchy. For example, referring to Figure 9.4(b), if we insert a row into PTStudentStaff, then the values of the inherited columns are inserted into Student and Staff, and then the values of the inherited columns of Student/Staff are inserted into Person.
- When a row is updated in a subtable, a procedure to the previous one is carried out to update the values of inherited columns in the supertables.
- When a row is updated in a supertable, then the values of all inherited columns in all corresponding rows of its direct and indirect subtables are also updated accordingly. As the supertable may itself be a subtable, the previous condition will also have to be applied to ensure consistency.
- When a row is deleted in a subtable/supertable, the corresponding rows in the table hierarchy are deleted. For example, if we deleted a row of Student, the corresponding rows of Person and Undergraduate/Postgraduate/PartTimeStudent/PTStudentStaff are deleted.

SQL:2011 does not provide a mechanism to store all instances of a given UDT unless the user explicitly creates a single table in which all instances are stored. Thus, in SQL:2011 it may not be possible to apply an SQL query to all instances of a given UDT. For example, if we created another table such as:

```
CREATE TABLE Client (
    info      PersonType,
    prefType  CHAR,
    maxRent   DECIMAL(6, 2),
    branchNo  VARCHAR(4) NOT NULL);
```

then the instances of PersonType are now distributed over two tables: Staff and Client. This problem can be overcome in this particular case using the table inheritance mechanism, which allows a table to be created that inherits all the columns of an existing table using the UNDER clause. As would be expected, a subtable inherits every column from its supertable. Note that all the tables in a table hierarchy must have corresponding types that are in the same type hierarchy, and the tables in the table hierarchy must be in the same relative positions as the corresponding types in the type hierarchy. However, not every type in the type hierarchy has to be represented in the table hierarchy, provided the range of types for which tables are defined is contiguous. For example, referring to Figure 9.4(a), it would be legal to create tables for all types except Staff; however, it would be illegal to create tables for Person and Postgraduate without creating one for Student. Note also that additional columns cannot be defined as part of the subtable definition.

EXAMPLE 9.6 Using a reference type to define a relationship

In this example, we model the relationship between PropertyForRent and Staff using a reference type.

```
CREATE TABLE PropertyForRent(
```


propertyNo	PropertyNumber	NOT NULL,	
street	Street	NOT NULL,	
city	City	NOT NULL,	
postcode	PostCode,		
type	PropertyType	NOT NULL	DEFAULT 'F',
rooms	PropertyRooms	NOT NULL	DEFAULT 4,
rent	PropertyRent	NOT NULL	DEFAULT 600,
staffID	REF(StaffType)	SCOPE Staff	

REFERENCES ARE CHECKED ON DELETE CASCADE,
PRIMARY KEY (propertyNo));

In Example 7.1 we modeled the relationship between `PropertyForRent` and `Staff` using the traditional primary key/foreign key mechanism. Here, however, we have used a reference type, `REF(StaffType)`, to model the relationship. The `SCOPE` clause specifies the associated referenced table. `REFERENCES ARE CHECKED` indicates that referential integrity is to be maintained (alternative is `REFERENCES ARE NOT CHECKED`). `ON DELETE CASCADE` corresponds to the normal referential action that existed in SQL2. Note that an `ON UPDATE` clause is not required, as the column `staffID` in the `Staff` table cannot be updated.

Privileges

As with the privileges required to create a new subtype, a user must have the `UNDER` privilege on the referenced supertable. In addition, a user must have `USAGE` privilege on any user-defined type referenced within the new table.

9.5.8 Querying Data

SQL:2011 provides the same syntax as SQL2 for querying and updating tables, with various extensions to handle objects. In this section, we illustrate some of these extensions.

EXAMPLE 9.7 Retrieve a specific column, specific rows

Find the names of all Managers.

```
SELECT s.lName
FROM Staff s
WHERE s.position = 'Manager';
```

This query invokes the implicitly defined observer function `position` in the `WHERE` clause to access the `position` column.

EXAMPLE 9.8 Invoking a user-defined function

Find the names and ages of all Managers.

```
SELECT s.lName, s.age
FROM Staff s
WHERE s.isManager;
```

This alternative method of finding Managers uses the user-defined method `isManager` as a predicate of the `WHERE` clause. This method returns the boolean value `TRUE` if the member of staff is a manager (see Example 9.3). In addition, this query also invokes the inherited virtual (observer) function `age` as an element of the `SELECT` list.

EXAMPLE 9.9 Use of **ONLY** to restrict selection

Find the names of all people in the database over 65 years of age.

```
SELECT p.lName, p.fName
FROM Person p
WHERE p.age > 65;
```

This query lists not only the details of rows that have been explicitly inserted into the `Person` table, but also the names from any rows that have been inserted into any direct or indirect subtables of `Person`, in this case, `Staff` and `Client`.

Suppose, however, that rather than wanting the details of all people, we want only the details of the specific instances of the `Person` table, excluding any subtables. This can be achieved using the `ONLY` keyword:

```
SELECT p.lName, p.fName
FROM ONLY (Person) p
WHERE p.age > 65;
```

EXAMPLE 9.10 Use of the dereference operator

Find the name of the member of staff who manages property 'PG4'.

```
SELECT p.staffID->fName AS fName, p.staffID->lName AS lName
FROM PropertyForRent p
WHERE p.propertyNo = 'PG4';
```

References can be used in path expressions that permit traversal of object references to navigate from one row to another. To traverse a reference, the dereference operator (`->`) is used. In the `SELECT` statement, `p.staffID` is the normal way to access a column of a table. In this particular case though, the column is a reference to a row of the `Staff` table, and so we must use the dereference operator to access the columns of the dereferenced table. In SQL:2011, this query would have required a join or nested subquery.

To retrieve the member of staff for property `PG4`, rather than just the first and last names, we would use the following query instead:

```
SELECT Deref(p.staffID) AS Staff
FROM PropertyForRent p
WHERE p.propertyNo = 'PG4';
```

Although reference types are similar to foreign keys, there are significant differences. In SQL:2011, referential integrity is maintained only by using a referential constraint definition specified as part of the table definition. By themselves,

reference types do not provide referential integrity. Thus, the SQL reference type should not be confused with that provided in the ODMG object model. In the ODMG model, OIDs are used to model relationships between types and referential integrity is automatically defined, will be discussed in Section 28.2.2.

9.5.9 Collection Types

Collections are type constructors that are used to define collections of other types. Collections are used to store multiple values in a single column of a table and can result in nested tables where a column in one table actually contains another table. The result can be a single table that represents multiple master-detail levels. Thus, collections add flexibility to the design of the physical database structure.

SQL:1999 introduced an `ARRAY` collection type and SQL:2003 added the `MULTISET` collection type, and a subsequent version of the standard may introduce parameterized `LIST` and `SET` collection types. In each case, the parameter, called the *element type*, may be a predefined type, a UDT, a row type, or another collection, but cannot be a reference type or a UDT containing a reference type. In addition, each collection must be homogeneous: all elements must be of the same type, or at least from the same type hierarchy. The collection types have the following meaning:

- `ARRAY`—one-dimensional array with a maximum number of elements;
- `MULTISET`—unordered collection that does allow duplicates;
- `LIST`—ordered collection that allows duplicates;
- `SET`—unordered collection that does not allow duplicates.

These types are similar to those defined in the ODMG 3.0 standard that will be discussed in Section 28.2, with the name `Bag` replaced with the SQL `MULTISET`.

ARRAY collection type

An array is an ordered collection of not necessarily distinct values, whose elements are referenced by their ordinal position in the array. An array is declared by a data type and optionally a maximum cardinality; for example:

```
VARCHAR(25) ARRAY[5]
```

The elements of this array can be accessed by an index ranging from 1 to the maximum cardinality (the function `CARDINALITY` returns the number of current elements in the array). Two arrays of comparable types are considered identical if and only if they have the same cardinality and every ordinal pair of elements is identical.

An array type is specified by an array type constructor, which can be defined by enumerating the elements as a comma-separated list enclosed in square brackets or by using a query expression with degree 1; for example:

```
ARRAY ['Mary White', 'Peter Beech', 'Anne Ford', 'John Howe', 'Alan Brand']  
ARRAY (SELECT rooms FROM PropertyForRent)
```

In these cases, the data type of the array is determined by the data types of the various array elements.

EXAMPLE 9.11 Use of a collection ARRAY

To model the requirement that a branch has up to three telephone numbers, we could implement the column as an ARRAY collection type:

```
telNo VARCHAR(13) ARRAY[3]
```

We could now retrieve the first and last telephone numbers at branch B003 using the following query:

```
SELECT telNo[1], telNo[CARDINALITY (telNo)]  
FROM Branch  
WHERE branchNo = 'B003';
```

MULTISET collection type

A multiset is an unordered collection of elements, all of the same type, with duplicates permitted. Because a multiset is unordered there is no ordinal position to reference individual elements of a multiset. Unlike arrays, a multiset is an unbounded collection with no declared maximum cardinality (although there will be an implementation-defined limit). Although multisets are analogous to tables, they are not regarded as the same as tables, and operators are provided to convert a multiset to a table (UNNEST) and a table to a multiset (MULTISET).

There is currently no separate type proposed for sets. Instead, a set is simply a special kind of multiset: one that has no duplicate elements. A predicate is provided to check whether a multiset is a set.

Two multisets of comparable element types, A and B say, are considered identical if and only if they have the same cardinality and for each element x in A, the number of elements of A that are identical to x , including x itself, equals the number of elements of B that are equal to x . Again as with array types, a multiset type constructor can be defined by enumerating their elements as a comma-separated list enclosed in square brackets, or by using a query expression with degree 1, or by using a table value constructor.

Operations on multisets include:

- The SET function, to remove duplicates from a multiset to produce a set.
- The CARDINALITY function, to return the number of current elements.
- The ELEMENT function, to return the element of a multiset if the multiset only has one element (or null if the multiset has no elements). An exception is raised if the multiset has more than one element.
- MULTISET UNION, which computes the union of two multisets; the keywords ALL or DISTINCT can be specified to either retain duplicates or remove them.
- MULTISET INTERSECT, which computes the intersection of two multisets; the keyword DISTINCT can be specified to remove duplicates; the keyword ALL can be specified to place in the result as many instances of each value as the minimum number of instances of that value in either operand.
- MULTISET EXCEPT, which computes the difference of two multisets; again, the keyword DISTINCT can be specified to remove duplicates; the keyword ALL can be specified to place in the result a number of instances of a value, equal to the number of instances of the value in the first operand minus the number of instances of the second operand.

There are three new aggregate functions for multisets:

- **COLLECT**, which creates a multiset from the value of the argument in each row of a group;
- **FUSION**, which creates a multiset union of a multiset value in all rows of a group;
- **INTERSECTION**, which creates the multiset intersection of a multiset value in all rows of a group.

In addition, a number of predicates exist for use with multisets:

- comparison predicate (equality and inequality only);
- **DISTINCT** predicate;
- **MEMBER** predicate;
- **SUBMULTISET** predicate, which tests whether one multiset is a submultiset of another;
- **IS A SET/IS NOT A SET** predicate, which checks whether a multiset is a set.

EXAMPLE 9.12 Use of a collection **MULTISET**

Extend the Staff table to contain the details of a number of next-of-kin and then find the first and last names of John White's next-of-kin.

We include the definition of a `nextOfKin` column in `Staff` as follows (`NameType` contains a `fName` and `lName` attribute):

```
nextOfKin      NameType MULTISET
```

The query becomes:

```
SELECT n.fName, n.lName
FROM Staff s, UNNEST (s.nextOfKin) AS n(fName, lName)
WHERE s.lName = 'White' AND s.fName = 'John';
```

Note that in the `FROM` clause we may use the multiset-valued field `s.nextOfKin` as a table reference.

EXAMPLE 9.13 Use of the **FUSION** and **INTERSECTION** aggregate functions

Consider the following table, `PropertyViewDates`, giving the dates properties have been viewed by potential renters:

propertyNo	viewDates
PA14	MULTISET['14-May-13', '24-May-13']
PG4	MULTISET['20-Apr-13', '14-May-13', '26-May-13']
PG36	MULTISET['28-Apr-13', '14-May-13']
PL94	Null

The following query based on multiset aggregation:

```
SELECT FUSION(viewDates) AS viewDateFusion,  
      INTERSECTION(viewDates) AS viewDateIntersection  
FROM PropertyViewDates;
```

produces the following result set:

viewDateFusion	viewDateIntersection
MULTISET['14-May-13', '14-May-13', '14-May-13', '24-May-13', '20-Apr-13', '26-May-13', '28-Apr-13']	MULTISET['14-May-13']

The fusion is computed by first discarding those rows with a null (in this case, the row for property PL94). Then each member of each of the remaining three multisets is copied to the result set. The intersection is computed by again discarding those rows with a null and then finding the duplicates in the input multisets.

9.5.10 Typed Views

SQL:2011 also supports **typed views**, sometimes called *object views* or *referenceable views*. A typed view is created based on a particular structured type and a subview can be created based on this typed view. The following example illustrates the usage of typed views.

EXAMPLE 9.14 Creation of typed views

The following statements create two views based on the `PersonType` and `StaffType` structured types:

```
CREATE VIEW FemaleView OF PersonType (REF IS personID DERIVED)  
  AS SELECT fName, lName  
  FROM ONLY (Person)  
  WHERE sex = 'F';  
  
CREATE VIEW FemaleStaff3View OF StaffType UNDER FemaleView  
  AS SELECT fName, lName, staffNo, position  
  FROM ONLY (Staff)  
  WHERE branchNo = 'B003';
```

The (`REF IS personID DERIVED`) is the self-referencing column specification discussed previously. When defining a subview, this clause cannot be specified. When defining a maximal superview, this clause can be specified, although the option `SYSTEM GENERATED` cannot be used, only `USER GENERATED` or `DERIVED`. If `USER GENERATED` is specified, then the degree of the view is one more than the number of attributes of the associated structured type; if `DERIVED` is specified then the degree is the same as the number of attributes in the associated structured type and no additional self-referencing column is included.

As with normal views, new column names can be specified as can the `WITH CHECK OPTION` clause.

9.5.11 Persistent Stored Modules

A number of new statement types have been added to SQL to make the language computationally complete, so that object behavior (methods) can be stored and executed from within the database as SQL statements. We discussed these statements in Section 8.1.

9.5.12 Triggers

A **trigger** is an SQL (compound) statement that is executed automatically by the DBMS as a side effect of a modification to a named table. It is similar to an SQL routine, in that it is a named SQL block with declarative, executable, and condition-handling sections. However, unlike a routine, a trigger is executed implicitly whenever the *triggering event* occurs, and a trigger does not have any arguments. The act of executing a trigger is sometimes known as *firing* the trigger. Triggers can be used for a number of purposes including:

- validating input data and maintaining complex integrity constraints that otherwise would be difficult, if not impossible, through table constraints;
- supporting alerts (for example, using electronic mail) that action needs to be taken when a table is updated in some way;
- maintaining audit information, by recording the changes made, and by whom;
- supporting replication, as discussed in Chapter 26.

The basic format of the CREATE TRIGGER statement is as follows:

```
CREATE TRIGGER TriggerName
    BEFORE | AFTER | INSTEAD OF <triggerEvent> ON <TableName>
    [REFERENCING <oldOrNewValuesAliasList>]
    [FOR EACH {ROW | STATEMENT
    [WHEN (triggerCondition)]
    <triggerBody>
```

Triggering events include insertion, deletion, and update of rows in a table. In the latter case only, a triggering event can also be set to cover specific named columns of a table. A trigger has an associated timing of either BEFORE, AFTER, or INSTEAD OF. A BEFORE trigger is fired before the associated event occurs, an AFTER trigger is fired after the associated event occurs, and an INSTEAD OF trigger is fired in place of the trigger event. The triggered action is an SQL procedure statement, which can be executed in one of two ways:

- for each row affected by the event (FOR EACH ROW). This is called a *row-level trigger*;
- only once for the entire event (FOR EACH STATEMENT), which is the default. This is called a *statement-level trigger*.

The <oldOrNewValuesAliasList> can refer to:

- an old or new row (OLD/NEW or OLD ROW/NEW ROW), in the case of a row-level trigger;
- an old or new table (OLD TABLE/NEW TABLE), in the case of an AFTER trigger.

Clearly, old values are not applicable for insert events, and new values are not applicable for delete events. The body of a trigger cannot contain any:

- SQL transaction statements, such as COMMIT or ROLLBACK;
- SQL connection statements, such as CONNECT or DISCONNECT;
- SQL schema definition or manipulation statements, such as the creation or deletion of tables, user-defined types, or other triggers;
- SQL session statements, such as SET SESSION CHARACTERISTICS, SET ROLE, SET TIME ZONE.

Furthermore, SQL does not allow *mutating triggers*, that is, triggers that cause a change resulting in the same trigger to be invoked again, possibly in an endless loop. As more than one trigger can be defined on a table, the order of firing of triggers is important. Triggers are fired as the trigger event (INSERT, UPDATE, DELETE) is executed. The following order is observed:

- (1) Execution of any BEFORE statement-level trigger on the table.
- (2) For each row affected by the statement:
 - (a) execution of any BEFORE row-level trigger;
 - (b) execution of the statement itself;
 - (c) application of any referential constraints;
 - (d) execution of any AFTER row-level trigger.
- (3) Execution of any AFTER statement-level trigger on the table.

Note from this ordering that BEFORE triggers are activated before referential integrity constraints have been checked. Thus, it is possible that the requested change that has caused the trigger to be invoked will violate database integrity constraints and will have to be disallowed. Therefore, BEFORE triggers should not further modify the database.

Should there be more than one trigger on a table with the same trigger event and the same action time (BEFORE or AFTER) then the SQL standard specifies that the triggers are executed in the order they were created. We now illustrate the creation of triggers with some examples.

EXAMPLE 9.15 Use of an AFTER INSERT trigger

Create a set of mailshot records for each new PropertyForRent row. For the purposes of this example, assume that there is a Mailshot table that records prospective renter details and property details.

```
CREATE TRIGGER InsertMailshotTable
  AFTER INSERT ON PropertyForRent
  REFERENCING NEW ROW AS pfr
  BEGIN ATOMIC
    INSERT INTO Mailshot VALUES
      (SELECT c.fName, c.lName, c.maxRent, pfr.propertyNo,
        pfr.street, pfr.city, pfr.postcode, pfr.type, pfr.rooms,
        pfr.rent
      FROM Client c
```



```

WHERE c.branchNo = pfr.branchNo AND
      (c.prefType = pfr.type AND c.maxRent <= pfr.rent))
END;

```

This trigger is executed after the new row has been inserted. The FOR EACH clause has been omitted, defaulting to FOR EACH STATEMENT, as an INSERT statement only inserts one row at a time. The body of the trigger is an INSERT statement based on a subquery that finds all matching client rows.

EXAMPLE 9.16 Use of an AFTER INSERT trigger with condition

Create a trigger that modifies all current mailshot records if the rent for a property changes.

```

CREATE TRIGGER UpdateMailshotTable
AFTER UPDATE OF rent ON PropertyForRent
REFERENCING NEW ROW AS pfr
FOR EACH ROW
BEGIN ATOMIC
    DELETE FROM Mailshot WHERE maxRent > pfr.rent;
    UPDATE Mailshot SET rent = pfr.rent
    WHERE propertyNo = pfr.propertyNo;
END;

```

This trigger is executed after the rent field of a PropertyForRent row has been updated. The FOR EACH ROW clause is specified, as all property rents may have been increased in one UPDATE statement, for example due to a cost of living rise. The body of the trigger has two SQL statements: a DELETE statement to delete those mailshot records where the new rental price is outside the client's price range, and an UPDATE statement to record the new rental price in all rows relating to that property.

Triggers can be a very powerful mechanism if used appropriately. The major advantage is that standard functions can be stored within the database and enforced consistently with each update to the database. This can dramatically reduce the complexity of applications. However, there can be some disadvantages:

- *Complexity.* When functionality is moved from the application to the database, the database design, implementation, and administration tasks become more complex.
- *Hidden functionality.* Moving functionality to the database and storing it as one or more triggers can have the effect of hiding functionality from the user. Although this can simplify things for the user, unfortunately it can also have side effects that may be unplanned, and potentially unwanted and erroneous. The user no longer has control over what happens to the database.
- *Performance overhead.* When the DBMS is about to execute a statement that modifies the database, it now has to evaluate the trigger condition to check whether a trigger should be fired by the statement. This has a performance implication on the DBMS. Clearly, as the number of triggers increases, this overhead also increases. At peak times, this overhead may create performance problems.

Privileges

To create a trigger, a user must have TRIGGER privilege on the specified table, SELECT privilege on any tables referenced in the triggerCondition of the WHEN clause, together with any privileges required to execute the SQL statements in the trigger body.

9.5.13 Large Objects

A **large object** is a data type that holds a large amount of data, such as a long text file or a graphics file. Three different types of large object data types are defined in SQL:2011:

- Binary Large Object (BLOB), a binary string that does not have a character set or collation association;
- Character Large Object (CLOB) and National Character Large Object (NCLOB), both character strings.

The SQL large object is slightly different from the original type of BLOB that appears in some database systems. In such systems, the BLOB is a noninterpreted byte stream, and the DBMS does not have any knowledge concerning the content of the BLOB or its internal structure. This prevents the DBMS from performing queries and operations on inherently rich and structured data types, such as images, video, word processing documents, or Web pages. Generally, this requires that the entire BLOB be transferred across the network from the DBMS server to the client before any processing can be performed. In contrast, the SQL large object does allow some operations to be carried out in the DBMS server. The standard string operators, which operate on character strings and return character strings, also operate on character large object strings, such as:

- The concatenation operator, (string1 || string2), which returns the character string formed by joining the character string operands in the specified order.
- The character substring function, SUBSTRING(string FROM startpos FOR length), which returns a string extracted from a specified string from a start position for a given length.
- The character overlay function, OVERLAY(string1 PLACING string2 FROM startpos FOR length), which replaces a substring of string1, specified as a starting position and a length, with string2. This is equivalent to: SUBSTRING(string1 FROM 1 FOR length - 1) || string2 || SUBSTRING (string1 FROM startpos + length).
- The fold functions, UPPER(string) and LOWER(string), which convert all characters in a string to upper/lower case.
- The trim function, TRIM([LEADING | TRAILING | BOTH string1 FROM] string2), which returns string2 with leading and/or trailing string1 characters removed. If the FROM clause is not specified, all leading and trailing spaces are removed from string2.
- The length function, CHAR_LENGTH(string), which returns the length of the specified string.
- The position function, POSITION(string1 IN string2), which returns the start position of string1 within string2.

However, CLOB strings are not allowed to participate in most comparison operations, although they can participate in a LIKE predicate, and a comparison or quantified comparison predicate that uses the equals (=) or not equals (<>) operators. As a result of these restrictions, a column that has been defined as a CLOB string cannot be referenced in such places as a GROUP BY clause, an ORDER BY clause, a unique or referential constraint definition, a join column, or in one of the set operations (UNION, INTERSECT, and EXCEPT).

A binary large object (BLOB) string is defined as a sequence of octets. All BLOB strings are comparable by comparing octets with the same ordinal position. The following operators operate on BLOB strings and return BLOB strings, and have similar functionality as those defined previously:

- the BLOB concatenation operator (||);
- the BLOB substring function (SUBSTRING);
- the BLOB overlay function (OVERLAY);
- the BLOB trim function (TRIM).

In addition, the BLOB_LENGTH and POSITION functions and the LIKE predicate can also be used with BLOB strings.

EXAMPLE 9.17 Use of Character and Binary Large Objects

Extend the Staff table to hold a resumé and picture for the staff member.

```
ALTER TABLE Staff
  ADD COLUMN resume CLOB(50K);
ALTER TABLE Staff
  ADD COLUMN picture BLOB(12M);
```

Two new columns have been added to the Staff table: `resume`, which has been defined as a CLOB of length 50K, and `picture`, which has been defined as a BLOB of length 12M. The length of a large object is given as a numeric value with an optional specification of K, M, or G, indicating kilobytes, megabytes, or gigabytes, respectively. The default length, if left unspecified, is implementation-defined.

9.5.14 Recursion

In Section 9.2 we discussed the difficulty that RDBMSs have with handling recursive queries. A major new operation in SQL for specifying such queries is **linear recursion**. We discussed recursion in Section 8.4.

9.6 Object-Oriented Extensions in Oracle

In Appendix H we examine some of the standard facilities of Oracle, including the base data types supported by Oracle, the procedural programming language PL/SQL, stored procedures and functions, and triggers. Many of the object-oriented features that appear in the new SQL:2011 standard appear in Oracle in one form or another. In this section we briefly discuss some of the object-oriented features in Oracle.

9.6.1 User-Defined Data Types

As well as supporting the built-in data types that we discuss in Appendix H.2.3, Oracle supports two user-defined data types: object types and collection types.

Object types

An object type is a schema object that has a name, a set of attributes based on the built-in data types or possibly other object types, and a set of methods, similar to what we discussed for an SQL:2011 object type. For example, we could create Address, Staff, and Branch types as follows:

```
CREATE TYPE AddressType AS OBJECT (  
    street      VARCHAR2(25),  
    city        VARCHAR2(15),  
    postcode    VARCHAR2(8));  
  
CREATE TYPE StaffType AS OBJECT (  
    staffNo     VARCHAR2(5),  
    fName       VARCHAR2(15),  
    lName       VARCHAR2(15),  
    position     VARCHAR2(10),  
    sex         CHAR,  
    DOB         DATE,  
    salary      DECIMAL(7, 2),  
    MAP MEMBER FUNCTION age RETURN INTEGER,  
    PRAGMA RESTRICT_REFERENCES(age, WNDS, WNPS, RNPS))  
NOT FINAL;  
CREATE TYPE BranchType AS OBJECT (  
    branchNo    VARCHAR2(4),  
    address     AddressType,  
    MAP MEMBER FUNCTION getbranchNo RETURN VARCHAR2(4),  
    PRAGMA RESTRICT_REFERENCES(getbranchNo, WNDS, WNPS,  
                                RNDS, RNPS));
```

We can then create a Branch (object) table using the following statement:

```
CREATE TABLE Branch OF BranchType (branchNo PRIMARY KEY);
```

This creates a Branch table with columns branchNo and address of type AddressType. Each row in the Branch table is an object of type BranchType. The pragma clause is a compiler directive that denies member functions read/write access to database tables and/or package variables (WNDS means *does not modify database tables*, WNPS means *does not modify packaged variables*, RNDS means *does not query database tables*, and RNPS means *does not reference package variables*). This example also illustrates another object-relational feature in Oracle: the specification of methods.

Methods

The methods of an object type are classified as member, static, or explicit comparison. A **member** method is a function or a procedure that always has an implicit or explicit SELF parameter as its first parameter, whose type is the containing object

type. Such methods are useful as observer and mutator functions and are invoked in the *selfish style*, for example `object.method()`, where the method finds all its arguments among the attributes of the object. We have defined an observer member method `getbranchNo` in the new type `BranchType`; we will show the implementation of this method shortly.

A **static** method is a function or a procedure that does not have an implicit `SELF` parameter. Such methods are useful for specifying user-defined constructors or cast methods and may be invoked by qualifying the method with the type name, as in `typename.method()`.

A **comparison** method is used for comparing instances of object types. Oracle provides two ways to define an order relationship among objects of a given type:

- a **map method** uses Oracle's ability to compare built-in types. In our example, we have defined a map method for the new type `BranchType`, which compares two branch objects based on the values in the `branchNo` attribute. We show an implementation of this method shortly.
- an **order method** uses its own internal logic to compare two objects of a given object type. It returns a value that encodes the order relationship. For example, it may return `-1` if the first is smaller, `0` if they are equal, and `1` if the first is larger.

For an object type, either a map method or an order method can be defined, but not both. If an object type has no comparison method, Oracle cannot determine a greater than or less than relationship between two objects of that type. However, it can attempt to determine whether two objects of the type are equal using the following rules:

- if all the attributes are nonnull and equal, the objects are considered equal;
- if there is an attribute for which the two objects have unequal nonnull values, the objects are considered unequal;
- otherwise, Oracle reports that the comparison is not available (null).

Methods can be implemented in PL/SQL, Java, and C, and overloading is supported provided their formal parameters differ in number, order, or data type. For the previous example, we could create the body for the member functions specified previously for types `BranchType` and `StaffType` as follows:

```
CREATE OR REPLACE TYPE BODY BranchType AS
  MAP MEMBER FUNCTION getbranchNo RETURN VARCHAR2(4) IS
  BEGIN
    RETURN branchNo;
  END;
END;
CREATE OR REPLACE TYPE BODY StaffType AS
  MAP MEMBER FUNCTION age RETURN INTEGER IS
  var NUMBER;
  BEGIN
    var := TRUNC(MONTHS_BETWEEN(SYSDATE, DOB)/12);
    RETURN var;
  END;
END;
```

The member function `getbranchNo` acts not only as an observer method to return the value of the `branchNo` attribute, but also as the comparison (`map`) method for this type. We will provide an example of the use of this method shortly. As in SQL:2011, user-defined functions can also be declared separately from the `CREATE TYPE` statement. In general, user-defined functions can be used in:

- the select list of a `SELECT` statement;
- a condition in the `WHERE` clause;
- the `ORDER BY` or `GROUP BY` clauses;
- the `VALUES` clause of an `INSERT` statement;
- the `SET` clause of an `UPDATE` statement.

Oracle also allows user-defined operators to be created using the `CREATE OPERATOR` statement. Like built-in operators, a user-defined operator takes a set of operands as input and return a result. Once a new operator has been defined, it can be used in SQL statements like any other built-in operator.

Constructor methods Every object type has a system-defined *constructor method* that makes a new object according to the object type's specification. The constructor method has the same name as the object type and has parameters that have the same names and types as the object type's attributes. For example, to create a new instance of `BranchType`, we could use the following expression:

```
BranchType('B003', AddressType('163 Main St', 'Glasgow', 'G11 9QX'));
```

Note that the expression `AddressType('163 Main St', 'Glasgow', 'G11 9QX')` is itself an invocation of the constructor for the type `AddressType`.

Object identifiers

Every row object in an object table has an associated logical object identifier (OID), which by default is a unique system-generated identifier assigned for each row object. The purpose of the OID is to uniquely identify each row object in an object table. To do this, Oracle implicitly creates and maintains an index on the OID column of the object table. The OID column is hidden from users and there is no access to its internal structure. Although OID values in themselves are not very meaningful, the OIDs can be used to fetch and navigate objects. (Note, objects that appear in object tables are called *row objects* and objects that occupy columns of relational tables or as attributes of other objects are called *column objects*.)

Oracle requires every row object to have a unique OID. The unique OID value may be specified to come from the row object's primary key or to be system-generated, using either the clause `OBJECT IDENTIFIER IS PRIMARY KEY` or `OBJECT IDENTIFIER IS SYSTEM GENERATED` (the default) in the `CREATE TABLE` statement. For example, we could restate the creation of the `Branch` table as:

```
CREATE TABLE Branch OF BranchType (branchNo PRIMARY KEY)
OBJECT IDENTIFIER IS PRIMARY KEY;
```

REF data type

Oracle provides a built-in data type called `REF` to encapsulate references to row objects of a specified object type. In effect, a `REF` is used to model an association

between two row objects. A REF can be used to examine or update the object it refers to and to obtain a copy of the object it refers to. The only changes that can be made to a REF are to replace its contents with a reference to a different object of the same object type or to assign it a null value. At an implementation level, Oracle uses object identifiers to construct REFs.

As in SQL:2011, a REF can be constrained to contain only references to a specified object table, using a SCOPE clause. As it is possible for the object identified by a REF to become unavailable, for example through deletion of the object, Oracle SQL has a predicate IS DANGLING to test REFs for this condition. Oracle also provides a dereferencing operator, Deref, to access the object referred to by a REF. For example, to model the manager of a branch we could change the definition of type BranchType to:

```
CREATE TYPE BranchType AS OBJECT (
    branchNo    VARCHAR2(4),
    address     AddressType,
    manager     REF StaffType,
    MAP MEMBER FUNCTION getbranchNo RETURN VARCHAR2(4),
    PRAGMA RESTRICT_REFERENCES(getbranchNo, WNDS, WNPS,
                                RNDS, RNPS));
```

In this case, we have modeled the manager through the reference type, REF StaffType. We see an example of how to access this column shortly.

Type inheritance

Oracle supports single inheritance allowing a subtype to be derived from a single parent type. The subtype inherits all the attributes and methods of the supertype and additionally can add new attributes and methods, and it can override any of the inherited methods. As with SQL:2011, the UNDER clause is used to specify the supertype.

Collection types

For modeling multi-valued attributes and many-to-many relationships, Oracle currently supports two collection types: array types and nested tables.

Array types An **array** is an ordered set of data elements that are all of the same data type. Each element has an **index**, which is a number corresponding to the element's position in the array. An array can have a fixed or variable size, although in the latter case a maximum size must be specified when the array type is declared. For example, a branch office can have up to three telephone numbers, which we could model in Oracle by declaring the following new type:

```
CREATE TYPE TelNoArrayType AS VARRAY(3) OF VARCHAR2(13);
```

The creation of an array type does not allocate space but instead defines a data type that can be used as:

- the data type of a column of a relational table;
- an object type attribute;
- a PL/SQL variable, parameter, or function return type.

For example, we could modify the type `BranchType` to include an attribute of this new type:

```
phoneList      TelNoArrayType,
```

An array is normally stored inline, that is, in the same tablespace as the other data in its row. If it is sufficiently large, however, Oracle stores it as a BLOB.

Nested tables A **nested table** is an unordered set of data elements that are all of the same data type. It has a single column of a built-in type or an object type. If the column is an object type, the table can also be viewed as a multicolumn table, with a column for each attribute of the object type. For example, to model next-of-kin for members of staff, we may define a new type as follows:

```
CREATE TYPE NextOfKinType AS OBJECT (
    fName    VARCHAR2(15),
    lName    VARCHAR2(15),
    telNo    VARCHAR2(13));
CREATE TYPE NextOfKinNestedType AS TABLE OF NextOfKinType;
```

We can now modify the type `StaffType` to include this new type as a nested table:

```
nextOfKin      NextOfKinNestedType,
```

and create a table for staff using the following statement:

```
CREATE TABLE Staff OF StaffType (
    PRIMARY KEY staffNo)
    OBJECT IDENTIFIER IS PRIMARY KEY
    NESTED TABLE nextOfKin STORE AS NextOfKinStorageTable (
        (PRIMARY KEY(Nested_Table_Id, lName, telNo))
        ORGANIZATION INDEX COMPRESS)
    RETURN AS LOCATOR;
```

The rows of a nested table are stored in a separate storage table that cannot be directly queried by the user but can be referenced in DDL statements for maintenance purposes. A hidden column in this storage table, `Nested_Table_Id`, matches the rows with their corresponding parent row. All the elements of the nested table of a given row of `Staff` have the same value of `Nested_Table_Id` and elements that belong to a different row of `Staff` have a different value of `Nested_Table_Id`.

We have indicated that the rows of the `nextOfKin` nested table are to be stored in a separate storage table called `NextOfKinStorageTable`. In the `STORE AS` clause, we have also specified that the storage table is index-organized (`ORGANIZATION INDEX`), to cluster rows belonging to the same parent. We have specified `COMPRESS` so that the `Nested_Table_Id` part of the index key is stored only once for each row of a parent row rather than being repeated for every row of a parent row object.

The specification of `Nested_Table_Id` and the given attributes as the primary key for the storage table serves two purposes: it serves as the key for the index and it enforces uniqueness of the columns (`lName`, `telNo`) of a nested table within each row of the parent table. By including these columns in the key, the statement ensures that the columns contain distinct values within each member of staff.

In Oracle, the collection-typed value is encapsulated. Consequently, a user must access the contents of a collection via interfaces provided by Oracle. Generally, when the user accesses a nested table, Oracle returns the entire collection value to the user's client process. This may have performance implications, and so Oracle supports the ability to return a nested table value as a locator, which is like a handle to the collection value. The `RETURN AS LOCATOR` clause indicates that the nested table is to be returned in the locator form when retrieved. If this is not specified, the default is `VALUE`, which indicates that the entire nested table is to be returned instead of just a locator to the nested table.

Nested tables differ from arrays in the following ways:

- Arrays have a maximum size, but nested tables do not.
- Arrays are always dense, but nested tables can be sparse, and so individual elements can be deleted from a nested table but not from an array.
- Oracle stores array data inline (in the same tablespace) but stores nested table data out of line in a *store table*, which is a system-generated database table associated with the nested table.
- When stored in the database, arrays retain their ordering and subscripts, but nested tables do not.

9.6.2 Manipulating Object Tables

In this section we briefly discuss how to manipulate object tables using the sample objects created previously for illustration. For example, we can insert objects into the `Staff` table as follows:

```
INSERT INTO Staff VALUES ('SG37', 'Ann', 'Beech', 'Assistant', 'F', '10-Nov-1960', 12000, NextOfKinNestedType());
INSERT INTO Staff VALUES ('SG5', 'Susan', 'Brand', 'Manager', 'F', '3-Jun-1940', 24000, NextOfKinNestedType());
```

The expression `NextOfKinNestedType()` invokes the constructor method for this type to create an empty `nextOfKin` attribute. We can insert data into the nested table using the following statement:

```
INSERT INTO TABLE (SELECT s.nextOfKin
                     FROM Staff s
                     WHERE s.staffNo = 'SG5')
VALUES ('John', 'Brand', '0141-848-2000');
```

This statement uses a `TABLE` expression to identify the nested table as the target for the insertion, namely the nested table in the `nextOfKin` column of the row object in the `Staff` table that has a `staffNo` of 'SG5'. Finally, we can insert an object into the `Branch` table:

```
INSERT INTO Branch
SELECT 'B003', AddressType('163 Main St', 'Glasgow', 'G11 9QX'), REF(s),
        TelNoArrayType('0141-339-2178', '0141-339-4439')
FROM Staff s
WHERE s.staffNo = 'SG5';
```

or alternatively:

```
INSERT INTO Branch VALUES ('B003', AddressType('163 Main St', 'Glasgow',
    'G11 9QX'), (SELECT REF(s) FROM Staff s WHERE s.staffNo = 'SG5'),
    TelNoArrayType('0141-339-2178', '0141-339-4439'));
```

Querying object tables In Oracle, we can return an ordered list of branch numbers using the following query:

```
SELECT b.branchNo
FROM Branch b
ORDER BY VALUE(b);
```

This query implicitly invokes the comparison method `getbranchNo` that we defined as a map method for the type `BranchType` to order the data in ascending order of `branchNo`. We can return all the data for each branch using the following query:

```
SELECT b.branchNo, b.address, DEREF(b.manager), b.phoneList
FROM Branch b
WHERE b.address.city = 'Glasgow'
ORDER BY VALUE(b);
```

Note the use of the `DEREF` operator to access the manager object. This query writes out the values for the `branchNo` column, all columns of an address, all columns of the manager object (of type `StaffType`), and all relevant telephone numbers.

We can retrieve next of kin data for all staff at a specified branch using the following query:

```
SELECT b.branchNo, b.manager.staffNo, n.*
FROM Branch b, TABLE(b.manager.nextOfKin) n
WHERE b.branchNo = 'B003';
```

Many applications are unable to handle collection types and instead require a flattened view of the data. In this example, we have flattened (or *unnested*) the nested set using the `TABLE` keyword. Note also that the expression `b.manager.staffNo` is a shorthand notation for `y.staffNo` where `y = DEREF(b.manager)`.

9.6.3 Object Views

In Sections 4.4 and 7.4 we examined the concept of views. In much the same way that a view is a virtual table, an **object view** is a virtual object table. Object views allow the data to be customized for different users. For example, we may create a view of the `Staff` table to prevent some users from seeing sensitive personal or salary-related information. In Oracle, we may now create an object view that not only restricts access to some data but also prevents some methods from being invoked, such as a delete method. It has also been argued that object views provide a simple migration path from a purely relational-based application to an object-oriented one, thereby allowing companies to experiment with this new technology.

For example, assume that we have created the object types defined in Section 9.6.1 and assume that we have created and populated the following relational schema for *DreamHome* with associated structured types *BranchType* and *StaffType*:



```
Branch      (branchNo, street, city, postcode, mgrStaffNo)
Telephone   (telNo, branchNo)
Staff        (staffNo, fName, lName, position, sex, DOB, salary, branchNo)
NextOfKin    (staffNo, fName, lName, telNo)
```

We could create an object-relational schema using the object view mechanism as follows:

```
CREATE VIEW StaffView OF StaffType WITH OBJECT IDENTIFIER (staffNo) AS
  SELECT s.staffNo, s.fName, s.lName, s.sex, s.position, s.DOB, s.salary,
        CAST (MULTISET (SELECT n.fName, n.lName, n.telNo
        FROM NextOfKin n WHERE n.staffNo = s.staffNo)
        AS NextOfKinNestedType) AS nextOfKin
FROM Staff s;
CREATE VIEW BranchView OF BranchType WITH OBJECT IDENTIFIER
(branchNo) AS
  SELECT b.branchNo, AddressType(b.street, b.city, b.postcode) AS address,
        MAKE_REF(StaffView, b.mgrStaffNo) AS manager,
        CAST (MULTISET (SELECT telNo FROM Telephone t
        WHERE t.branchNo = b.branchNo) AS TelNoArrayType) AS phoneList
FROM Branch b;
```

In each case, the **SELECT** subquery inside the **CAST/MULTISET** expression selects the data we require (in the first case, a list of next of kin for the member of staff and in the second case, a list of telephone numbers for the branch). The **MULTISET** keyword indicates that this is a list rather than a singleton value, and the **CAST** operator then casts this list to the required type. Note also the use of the **MAKE_REF** operator, which creates a **REF** to a row of an object view or a row in an object table whose object identifier is primary-key based.

The **WITH OBJECT IDENTIFIER** specifies the attributes of the object type that will be used as a key to identify each row in the object view. In most cases, these attributes correspond to the primary key columns of the base table. The specified attributes must be unique and identify exactly one row in the view. If the object view is defined on an object table or an object view, this clause can be omitted or **WITH OBJECT IDENTIFIER DEFAULT** can be specified. In each case, we have specified the primary key of the corresponding base table to provide uniqueness.

9.6.4 Privileges

Oracle defines the following system privileges for user-defined types:

- **CREATE TYPE** – to create user-defined types in the user's schema;
- **CREATE ANY TYPE** – to create user-defined types in any schema;
- **ALTER ANY TYPE** – to alter user-defined types in any schema;

- DROP ANY TYPE – to drop named types in any schema;
- EXECUTE ANY TYPE – to use and reference named types in any schema;
- UNDER ANY TYPE – to create subtypes under any non-final object types;
- UNDER ANY VIEW – to create subviews under any object view.

In addition, the EXECUTE schema object privilege allows a user to use the type to define a table, define a column in a relational table, declare a variable or parameter of the named type, and to invoke the type's methods.

Chapter Summary

- Advanced database applications include computer-aided design (CAD), computer-aided manufacturing (CAM), computer-aided software engineering (CASE), network management systems, office information systems (OIS) and multimedia systems, digital publishing, geographic information systems (GIS), and interactive and dynamic Web sites, as well as applications with complex and interrelated objects and procedural data.
- The relational model, and relational systems in particular, have weaknesses such as poor representation of “real-world” entities, semantic overloading, poor support for integrity and enterprise constraints, limited operations, and impedance mismatch. The limited modeling capabilities of relational DBMSs have made them unsuitable for advanced database applications.
- There is no single extended relational data model; rather, there are a variety of these models, whose characteristics depend upon the way and the degree to which extensions were made. However, all the models do share the same basic relational tables and query language, all incorporate some concept of “object,” and some have the ability to store methods or procedures/triggers as well as data in the database.
- Various terms have been used for systems that have extended the relational data model. The original term used to describe such systems was the *Extended Relational DBMS* (ERDBMS), and the term *Universal Server* or *Universal DBMS* (UDBMS) has also been used. However, in recent years, the more descriptive term **Object-Relational DBMS (ORDBMS)** has been used to indicate that the system incorporates some notion of “object.”
- Since 1999, the SQL standard includes object management extensions for: row types, user-defined types (UDTs) and user-defined routines (UDRs), polymorphism, inheritance, reference types and object identity, collection types (ARRAYs), new language constructs that make SQL computationally complete, triggers, and support for large objects—Binary Large Objects (BLOBs) and Character Large Objects (CLOBs)—and recursion.

Review Questions

- 9.1 Discuss the general characteristics of advanced database applications.
- 9.2 Discuss why the weaknesses of the relational data model and relational DBMSs may make them unsuitable for advanced database applications.
- 9.3 Describe strategies used in ORDB designs to map classes to relations.
- 9.4 What functionality would typically be provided by an ORDBMS?
- 9.5 Compare and contrast the different collection types.
- 9.6 What is a typed table? How is it different from other table types?
- 9.7 Discuss how references types and object identity can be used.

- 9.8 Compare and contrast procedures, functions, and methods.
- 9.9 Discuss the extensions required for query processing and query optimization to fully support the ORDBMS.
- 9.10 Discuss the collection types available in SQL:2011.
- 9.11 What are the security problems associated with the introduction of user-defined methods? Suggest some solutions to these problems.

Exercises

- 9.12 Investigate one of the advanced database applications discussed in Section 9.1, or a similar one that handles complex, interrelated data. In particular, examine its functionality and the data types and operations it uses. Map the data types and operations to the object-oriented concepts discussed in Appendix K.
- 9.13 Analyze one of the RDBMSs that you currently use. Discuss the object-oriented features provided by the system. What additional functionality do these features provide?
- 9.14 Analyze the RDBMSs that you are currently using. Discuss the object-oriented facilities provided by the system. What additional functionality do these facilities provide?
- 9.15 Discuss key issues to consider when converting a relational database to an object relational database.
- 9.16 *DreamHome* has contracted you to help them with changing their system from a relational database to an object relational database. Produce a presentation highlighting the approaches and key processes that will be involved.
- 9.17 Create an insert trigger that sets up a mailshot table recording the names and addresses of all guests who have stayed at the hotel during the days before and after New Year for the past two years.
- 9.18 Investigate the relational schema and object relational schema used in the *EasyDrive School of Motoring* case study. What are the key differences between the two?
- 9.19 Create an object-relational schema for the *DreamHome* case study documented in Appendix A. Add user-defined functions that you consider appropriate. Implement the queries listed in Appendix A using SQL:2011.
- 9.20 Create an object-relational schema for the *University Accommodation Office* case study documented in Appendix B.1. Add user-defined functions that you consider appropriate.
- 9.21 Create an object-relational schema for the *EasyDrive School of Motoring* case study documented in Appendix B.2. Add user-defined functions that you consider appropriate.
- 9.22 Create an object-relational schema for the *Wellmeadows* case study documented in Appendix B.3. Add user-defined functions that you consider appropriate.
- 9.23 You have been asked by the Managing Director of *DreamHome* to investigate and prepare a report on the applicability of an object-relational DBMS for the organization. The report should compare the technology of the RDBMS with that of the ORDBMS, and should address the advantages and disadvantages of implementing an ORDBMS within the organization, and any perceived problem areas. The report should also consider the applicability of an object-oriented DBMS, and a comparison of the two types of system for *DreamHome* should be included. Finally, the report should contain a fully justified set of conclusions on the applicability of the ORDBMS for *DreamHome*.



Database Analysis and Design

Chapter	10	Database System Development Lifecycle	345
Chapter	11	Database Analysis and the <i>DreamHome</i> Case Study	375
Chapter	12	Entity–Relationship Modeling	405
Chapter	13	Enhanced Entity–Relationship Modeling	433
Chapter	14	Normalization	451
Chapter	15	Advanced Normalization	481

Database System Development Lifecycle

Chapter Objectives

In this chapter you will learn:

- The main components of an information system.
- The main stages of the database system development lifecycle (DSDLC).
- The main phases of database design: conceptual, logical, and physical design.
- The types of criteria used to evaluate a DBMS.
- How to evaluate and select a DBMS.
- The benefits of Computer-Aided Software Engineering (CASE) tools.

Software has now surpassed hardware as the key to the success of many computer-based systems. Unfortunately, the track record for developing software is not particularly impressive. The last few decades have seen the proliferation of software applications ranging from small, relatively simple applications consisting of a few lines of code to large, complex applications consisting of millions of lines of code. Many of these applications have required constant maintenance. This maintenance involved correcting faults that had been detected, implementing new user requirements, and modifying the software to run on new or upgraded platforms. The effort spent on maintenance began to absorb resources at an alarming rate. As a result, many major software projects were late, over budget, unreliable, and difficult to maintain, and performed poorly. These issues led to what has become known as the **software crisis**. Although this term was first used in the late 1960s, the crisis is still with us. As a result, some authors now refer to the software crisis as the **software depression**. As an indication of the crisis, a study carried out in the UK by OASIG, a Special Interest Group concerned with the Organizational Aspects of IT, reached the following conclusions about software projects (OASIG, 1996):

- 80–90% do not meet their performance goals;
- about 80% are delivered late and over budget;
- around 40% fail or are abandoned;
- under 40% fully address training and skills requirements;

- less than 25% properly integrate enterprise and technology objectives;
- just 10–20% meet all their success criteria.

There are several major reasons for the failure of software projects, including:

- lack of a complete requirements specification;
- lack of an appropriate development methodology;
- poor decomposition of design into manageable components.

As a solution to these problems, a structured approach to the development of software was proposed called the **Information Systems Lifecycle (ISLC)** or the **Software Development Lifecycle (SDLC)**. However, when the software being developed is a database system the lifecycle is more specifically referred to as the **Database System Development Lifecycle (DSDLC)**.

Structure of this Chapter In Section 10.1 we briefly describe the information systems lifecycle and discuss how this lifecycle relates to the database system development lifecycle. In Section 10.2 we present an overview of the stages of the database system development lifecycle. In Sections 10.3 to 10.13 we describe each stage of the lifecycle in more detail. In Section 10.14 we discuss how Computer-Aided Software Engineering (CASE) tools can provide support for the database system development lifecycle.

10.1 The Information Systems Lifecycle

Information system

The resources that enable the collection, management, control, and dissemination of information throughout an organization.

Since the 1970s, database systems have been gradually replacing file-based systems as part of an organization's Information Systems (IS) infrastructure. At the same time, there has been a growing recognition that data is an important corporate resource that should be treated with respect, like all other organizational resources. This resulted in many organizations establishing whole departments or functional areas called Data Administration (DA) and Database Administration (DBA) that are responsible for the management and control of the corporate data and the corporate database, respectively.

A computer-based information system includes a database, database software, application software, computer hardware, and personnel using and developing the system.

The database is a fundamental component of an information system, and its development and usage should be viewed from the perspective of the wider requirements of the organization. Therefore, the lifecycle of an organization's information system is inherently linked to the lifecycle of the database system that supports it. Typically, the stages in the lifecycle of an information system include: planning, requirements collection and analysis, design, prototyping, implementation, testing, conversion, and operational maintenance. In this chapter we review these stages

from the perspective of developing a database system. However, it is important to note that the development of a database system should also be viewed from the broader perspective of developing a component part of the larger organization-wide information system.

Throughout this chapter we use the terms “functional area” and “application area” to refer to particular enterprise activities within an organization such as marketing, personnel, and stock control.

10.2 The Database System Development Lifecycle

As a database system is a fundamental component of the larger organization-wide information system, the database system development lifecycle is inherently associated with the lifecycle of the information system. The stages of the database system development lifecycle are shown in Figure 10.1. Below the name of each stage is the section in this chapter that describes that stage.

It is important to recognize that the stages of the database system development lifecycle are not strictly sequential, but involve some amount of repetition of previous stages through *feedback loops*. For example, problems encountered during database design may necessitate additional requirements collection and analysis. As there are feedback loops between most stages, we show only some of the more obvious ones in Figure 10.1. A summary of the main activities associated with each stage of the database system development lifecycle is described in Table 10.1.

For small database systems, with a small number of users, the lifecycle need not be very complex. However, when designing a medium to large database systems with tens to thousands of users, using hundreds of queries and application programs, the lifecycle can become extremely complex. Throughout this chapter, we concentrate on activities associated with the development of medium to large database systems. In the following sections we describe the main activities associated with each stage of the database system development lifecycle in more detail.

10.3 Database Planning

Database planning

The management activities that allow the stages of the database system development lifecycle to be realized as efficiently and effectively as possible.

Database planning must be integrated with the overall IS strategy of the organization. There are three main issues involved in formulating an IS strategy, which are:

- identification of enterprise plans and goals with subsequent determination of information systems needs;
- evaluation of current information systems to determine existing strengths and weaknesses;
- appraisal of IT opportunities that might yield competitive advantage.

The methodologies used to resolve these issues are outside the scope of this book; however, the interested reader is referred to Robson (1997) and Cadle & Yeates (2007) for a fuller discussion.

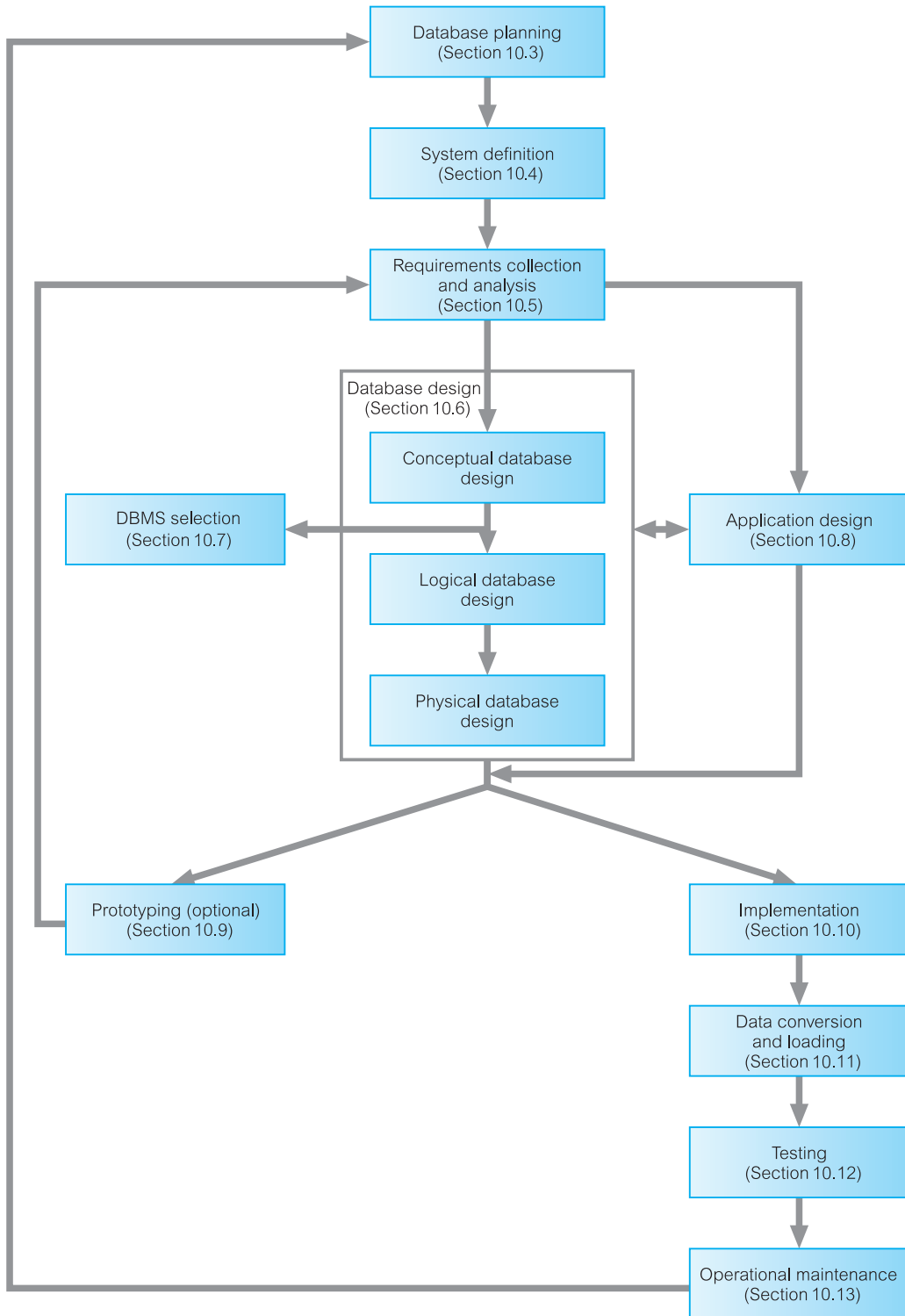


Figure 10.1 The stages of the database system development lifecycle.

TABLE 10.1 Summary of the main activities associated with each stage of the database system development lifecycle.

STAGE	MAIN ACTIVITIES
<i>Database planning</i>	Planning how the stages of the lifecycle can be realized most efficiently and effectively.
<i>System definition</i>	Specifying the scope and boundaries of the database system, including the major user views, its users, and application areas.
<i>Requirements collection and analysis</i>	Collection and analysis of the requirements for the new database system.
<i>Database design</i>	Conceptual, logical, and physical design of the database.
<i>DBMS selection</i>	Selecting a suitable DBMS for the database system.
<i>Application design</i>	Designing the user interface and the application programs that use and process the database.
<i>Prototyping (optional)</i>	Building a working model of the database system, which allows the designers or users to visualize and evaluate how the final system will look and function.
<i>Implementation</i>	Creating the physical database definitions and the application programs.
<i>Data conversion and loading</i>	Loading data from the old system to the new system and, where possible, converting any existing applications to run on the new database.
<i>Testing</i>	Database system is tested for errors and validated against the requirements specified by the users.
<i>Operational maintenance</i>	Database system is fully implemented. The system is continuously monitored and maintained. When necessary, new requirements are incorporated into the database system through the preceding stages of the lifecycle.

An important first step in database planning is to clearly define the **mission statement** for the database system. The mission statement defines the major aims of the database system. Those driving the database project within the organization (such as the Director and/or owner) normally define the mission statement. A mission statement helps to clarify the purpose of the database system and provide a clearer path towards the efficient and effective creation of the required database system. Once the mission statement is defined, the next activity involves identifying the **mission objectives**. Each mission objective should identify a particular task that the database system must support. The assumption is that if the database system supports the mission objectives, then the mission statement should be met. The mission statement and objectives may be accompanied by some additional information that specifies in general terms the work to be done, the resources with which to do it, and the money to pay for it all. We demonstrate the creation of a mission statement and mission objectives for the database system of *DreamHome* in Section 11.4.2.

Database planning should also include the development of standards that govern how data will be collected, how the format should be specified, what documentation will be needed, and how design and implementation should proceed. Standards can be very time-consuming to develop and maintain, requiring resources to set them up initially, and to continue maintaining them. However, a well-designed set of standards provides a basis for training staff and measuring



quality control, and can ensure that work conforms to a pattern, irrespective of staff skills and experience. For example, specific rules may govern how data items can be named in the data dictionary, which in turn may prevent both redundancy and inconsistency. Any legal or enterprise requirements concerning the data should be documented, such as the stipulation that some types of data must be treated confidentially.

10.4 System Definition

System definition

Describes the scope and boundaries of the database system and the major user views.



Before attempting to design a database system, it is essential that we first identify the boundaries of the system that we are investigating and how it interfaces with other parts of the organization’s information system. It is important that we include within our system boundaries not only the current users and application areas, but also future users and applications. We present a diagram that represents the scope and boundaries of the *DreamHome* database system in Figure 11.10. Included within the scope and boundary of the database system are the major user views that are to be supported by the database.

10.4.1 User Views

User view

Defines what is required of a database system from the perspective of a particular job role (such as Manager or Supervisor) or enterprise application area (such as marketing, personnel, or stock control).

A database system may have one or more user views. Identifying user views is an important aspect of developing a database system because it helps to ensure that no major users of the database are forgotten when developing the requirements for the new database system. User views are also particularly helpful in the development of a relatively complex database system by allowing the requirements to be broken down into manageable pieces.

A user view defines what is required of a database system in terms of the data to be held and the transactions to be performed on the data (in other words, what the users will do with the data). The requirements of a user view may be distinct to that view or overlap with other views. Figure 10.2 is a diagrammatic representation of a database system with multiple user views (denoted user view 1 to 6). Note that whereas user views (1, 2, and 3) and (5 and 6) have overlapping requirements (shown as hatched areas), user view 4 has distinct requirements.

10.5 Requirements Collection and Analysis

Requirements collection and analysis

The process of collecting and analyzing information about the part of the organization that is to be supported by the database system, and using this information to identify the requirements for the new system.

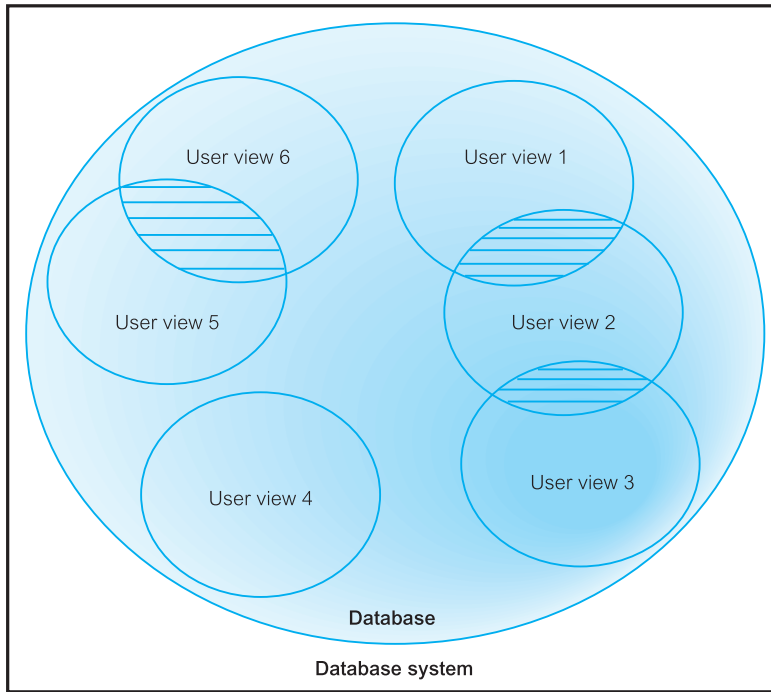


Figure 10.2 Representation of a database system with multiple user views: user views (1, 2, and 3) and (5 and 6) have overlapping requirements (shown as hatched areas), whereas user view 4 has distinct requirements.

This stage involves the collection and analysis of information about the part of the enterprise to be served by the database. There are many techniques for gathering this information, called **fact-finding techniques**, which we discuss in detail in Chapter 11. Information is gathered for each major user view (that is, job role or enterprise application area), including:

- a description of the data used or generated;
- the details of how data is to be used or generated;
- any additional requirements for the new database system.

This information is then analyzed to identify the requirements (or features) to be included in the new database system. These requirements are described in documents collectively referred to as **requirements specifications** for the new database system.

Requirements collection and analysis is a preliminary stage to database design. The amount of data gathered depends on the nature of the problem and the policies of the enterprise. Too much study too soon leads to *paralysis by analysis*. Too little thought can result in an unnecessary waste of both time and money due to working on the wrong solution to the wrong problem.

The information collected at this stage may be poorly structured and include some informal requests, which must be converted into a more structured statement of

requirements. This is achieved using **requirements specification techniques**, which include, for example, Structured Analysis and Design (SAD) techniques, Data Flow Diagrams (DFD), and Hierarchical Input Process Output (HIPO) charts supported by documentation. As you will see shortly, Computer-Aided Software Engineering (CASE) tools may provide automated assistance to ensure that the requirements are complete and consistent. In Section 27.8 we will discuss how the Unified Modeling Language (UML) supports requirements analysis and design.

Identifying the required functionality for a database system is a critical activity, as systems with inadequate or incomplete functionality will annoy the users, which may lead to rejection or underutilization of the system. However, excessive functionality can also be problematic, as it can overcomplicate a system, making it difficult to implement, maintain, use, or learn.

Another important activity associated with this stage is deciding how to deal with the situation in which there is more than one user view for the database system. There are three main approaches to managing the requirements of a database system with multiple user views:

- the **centralized** approach;
- the **view integration** approach;
- a combination of both approaches.

10.5.1 Centralized Approach

Centralized approach

Requirements for each user view are merged into a single set of requirements for the new database system. A data model representing all user views is created during the database design stage.

The centralized (or one-shot) approach involves collating the requirements for different user views into a single list of requirements. The collection of user views is given a name that provides some indication of the application area covered by all the merged user views. In the database design stage (see Section 10.6), a global data model is created, which represents all user views. The global data model is composed of diagrams and documentation that formally describe the data requirements of the users. A diagram representing the management of user views 1 to 3 using the centralized approach is shown in Figure 10.3. Generally, this approach is preferred when there is a significant overlap in requirements for each user view and the database system is not overly complex.

10.5.2 View Integration Approach

View integration approach

Requirements for each user view remain as separate lists. Data models representing each user view are created and then merged later during the database design stage.

The view integration approach involves leaving the requirements for each user view as separate lists of requirements. In the database design stage (see Section 10.6), we

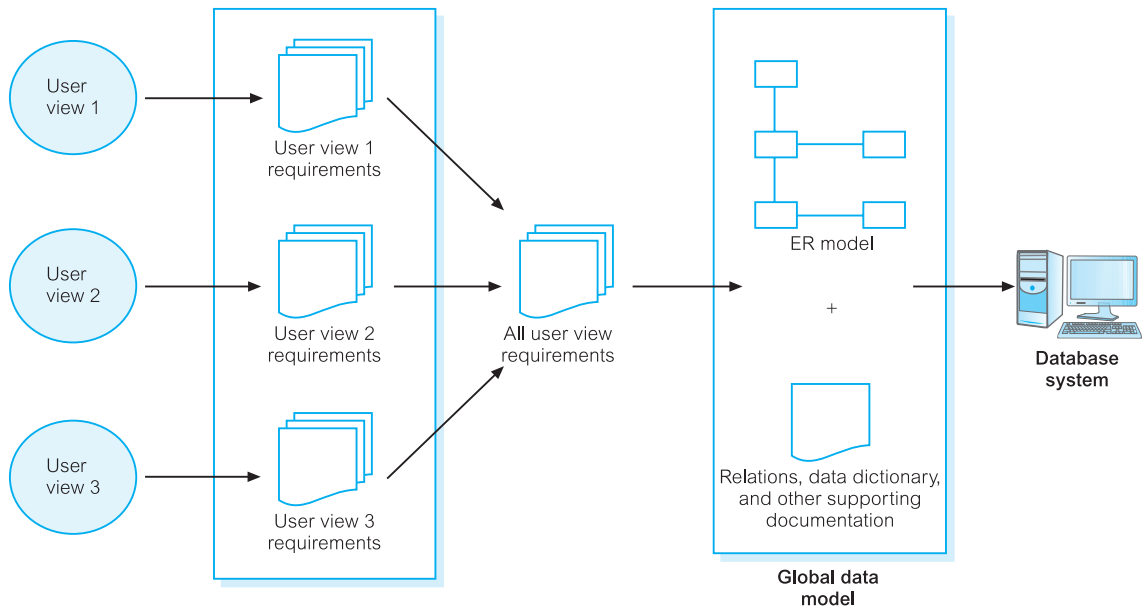


Figure 10.3 The centralized approach to managing multiple user views 1 to 3.

first create a data model for each user view. A data model that represents a single user view (or a subset of all user views) is called a **local data model**. Each model is composed of diagrams and documentation that formally describes the requirements of one or more—but not all—user views of the database. The local data models are then merged at a later stage of database design to produce a **global data model**, which represents *all* user requirements for the database. A diagram representing the management of user views 1 to 3 using the view integration approach is shown in Figure 10.4. Generally, this approach is preferred when there are significant differences between user views and the database system is sufficiently complex to justify dividing the work into more manageable parts. We demonstrate how to use the view integration approach in Chapter 17, Step 2.6.

For some complex database systems, it may be appropriate to use a combination of both the centralized and view integration approaches to manage multiple user views. For example, the requirements for two or more user views may be first merged using the centralized approach, which is used to build a local logical data model. This model can then be merged with other local logical data models using the view integration approach to produce a global logical data model. In this case, each local logical data model represents the requirements of two or more user views and the final global logical data model represents the requirements of all user views of the database system.

We discuss how to manage multiple user views in more detail in Section 11.4.4, and using the methodology described in this book, we demonstrate how to build a database for the *DreamHome* property rental case study using a combination of the centralized and view integration approaches.



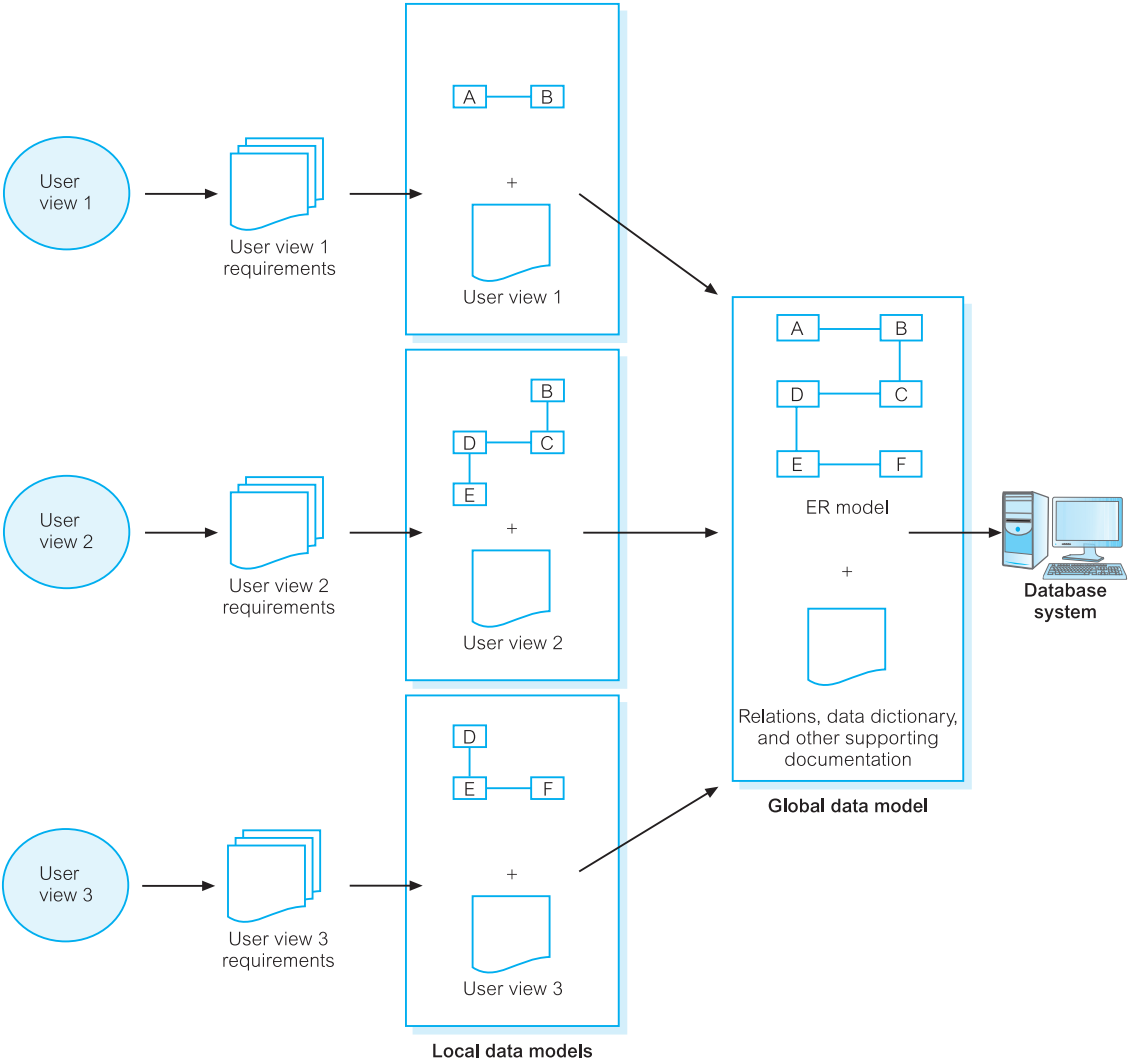


Figure 10.4 The view integration approach to managing multiple user views 1 to 3.

10.6 Database Design

Database design

The process of creating a design that will support the enterprise’s mission statement and mission objectives for the required database system.

In this section we present an overview of the main approaches to database design. We also discuss the purpose and use of data modeling in database design. We then describe the three phases of database design: conceptual, logical, and physical design.

10.6.1 Approaches to Database Design

The two main approaches to the design of a database are referred to as “bottom-up” and “top-down.” The **bottom-up** approach begins at the fundamental level of attributes (that is, properties of entities and relationships), which through analysis of the associations between attributes are grouped into relations that represent types of entities and relationships between entities. In Chapters 14 and 15 we discuss the process of normalization, which represents a bottom-up approach to database design. Normalization involves the identification of the required attributes and their subsequent aggregation into normalized relations based on functional dependencies between the attributes.

The bottom-up approach is appropriate for the design of simple databases with a relatively small number of attributes. However, this approach becomes difficult when applied to the design of more complex databases with a larger number of attributes, where it is difficult to establish all the functional dependencies between the attributes. As the conceptual and logical data models for complex databases may contain hundreds to thousands of attributes, it is essential to establish an approach that will simplify the design process. Also, in the initial stages of establishing the data requirements for a complex database, it may be difficult to establish all the attributes to be included in the data models.

A more appropriate strategy for the design of complex databases is to use the **top-down** approach. This approach starts with the development of data models that contain a few high-level entities and relationships and then applies successive top-down refinements to identify lower-level entities, relationships, and the associated attributes. The top-down approach is illustrated using the concepts of the Entity-Relationship (ER) model, beginning with the identification of entities and relationships between the entities, which are of interest to the organization. For example, we may begin by identifying the entities *PrivateOwner* and *PropertyForRent*, and then the relationship between these entities, *PrivateOwner Owns PropertyForRent*, and finally the associated attributes such as *PrivateOwner* (ownerNo, name, and address) and *PropertyForRent* (propertyNo and address). Building a high-level data model using the concepts of the ER model is discussed in Chapters 12 and 13.

There are other approaches to database design, such as the inside-out approach and the mixed strategy approach. The **inside-out** approach is related to the bottom-up approach, but differs by first identifying a set of major entities and then spreading out to consider other entities, relationships, and attributes associated with those first identified. The **mixed strategy** approach uses both the bottom-up and top-down approach for various parts of the model before finally combining all parts together.

10.6.2 Data Modeling

The two main purposes of data modeling are to assist in the understanding of the meaning (semantics) of the data and to facilitate communication about the information requirements. Building a data model requires answering questions about entities, relationships, and attributes. In doing so, the designers discover the semantics of the enterprise’s data, which exist whether or not they happen to be recorded in a formal data model. Entities, relationships, and attributes are fundamental to

all enterprises. However, their meaning may remain poorly understood until they have been correctly documented. A data model makes it easier to understand the meaning of the data, and thus we model data to ensure that we understand:

- each user’s perspective of the data;
- the nature of the data itself, independent of its physical representations;
- the use of data across user views.

Data models can be used to convey the designer’s understanding of the information requirements of the enterprise. Provided both parties are familiar with the notation used in the model, it will support communication between the users and designers. Increasingly, enterprises are standardizing the way that they model data by selecting a particular approach to data modeling and using it throughout their database development projects. The most popular high-level data model used in database design, and the one we use in this book, is based on the concepts of the ER model. We describe ER modeling in detail in Chapters 12 and 13.

Criteria for data models

An *optimal* data model should satisfy the criteria listed in Table 10.2 (Fleming and Von Halle, 1989). However, sometimes these criteria are incompatible with each other and trade-offs are sometimes necessary. For example, in attempting to achieve greater *expressibility* in a data model, we may lose *simplicity*.

10.6.3 Phases of Database Design

Database design is made up of three main phases: conceptual, logical, and physical design.

Conceptual database design

Conceptual database design	The process of constructing a model of the data used in an enterprise, independent of <i>all</i> physical considerations.
----------------------------	---

TABLE 10.2 The criteria to produce an optimal data model.

Structural validity	Consistency with the way the enterprise defines and organizes information.
Simplicity	Ease of understanding by IS professionals and nontechnical users.
Expressibility	Ability to distinguish between different data, relationships between data, and constraints.
Nonredundancy	Exclusion of extraneous information; in particular, the representation of any one piece of information exactly once.
Shareability	Not specific to any particular application or technology and thereby usable by many.
Extensibility	Ability to evolve to support new requirements with minimal effect on existing users.
Integrity	Consistency with the way the enterprise uses and manages information.
Diagrammatic representation	Ability to represent a model using an easily understood diagrammatic notation.

The first phase of database design is called **conceptual database design** and involves the creation of a conceptual data model of the part of the enterprise that we are interested in modeling. The data model is built using the information documented in the users' requirements specification. Conceptual database design is entirely independent of implementation details such as the target DBMS software, application programs, programming languages, hardware platform, or any other physical considerations. In Chapter 16, we present a practical step-by-step guide on how to perform conceptual database design.

Throughout the process of developing a conceptual data model, the model is tested and validated against the users' requirements. The conceptual data model of the enterprise is a source of information for the next phase, namely logical database design.

Logical database design

Logical database design

The process of constructing a model of the data used in an enterprise based on a specific data model, but independent of a particular DBMS and other physical considerations.

The second phase of database design is called **logical database design**, which results in the creation of a logical data model of the part of the enterprise that we are interested in modeling. The conceptual data model created in the previous phase is refined and mapped onto a logical data model. The logical data model is based on the target data model for the database (for example, the relational data model).

Whereas a conceptual data model is independent of all physical considerations, a logical model is derived knowing the underlying data model of the target DBMS. In other words, we know that the DBMS is, for example, relational, network, hierarchical, or object-oriented. However, we ignore any other aspects of the chosen DBMS and, in particular, any physical details, such as storage structures or indexes.

Throughout the process of developing a logical data model, the model is tested and validated against the users' requirements. The technique of **normalization** is used to test the correctness of a logical data model. Normalization ensures that the relations derived from the data model do not display data redundancy, which can cause update anomalies when implemented. In Chapter 14 we illustrate the problems associated with data redundancy and describe the process of normalization in detail. The logical data model should also be examined to ensure that it supports the transactions specified by the users.

The logical data model is a source of information for the next phase, namely physical database design, providing the physical database designer with a vehicle for making trade-offs that are very important to efficient database design. The logical model also serves an important role during the operational maintenance stage of the database system development lifecycle. Properly maintained and kept up to date, the data model allows future changes to application programs or data to be accurately and efficiently represented by the database.

In Chapter 17 we present a practical step-by-step guide for logical database design.

Physical database design

Physical database design

The process of producing a description of the implementation of the database on secondary storage; it describes the base relations, file organizations, and indexes used to achieve efficient access to the data, and any associated integrity constraints and security measures.

Physical database design is the third and final phase of the database design process, during which the designer decides how the database is to be implemented. The previous phase of database design involved the development of a logical structure for the database, which describes relations and enterprise constraints. Although this structure is DBMS-independent, it is developed in accordance with a particular data model, such as the relational, network, or hierarchic. However, in developing the physical database design, we must first identify the target DBMS. Therefore, physical design is tailored to a specific DBMS system. There is feedback between physical and logical design, because decisions are taken during physical design for improving performance that may affect the structure of the logical data model.

In general, the main aim of physical database design is to describe how we intend to physically implement the logical database design. For the relational model, this involves:

- creating a set of relational tables and the constraints on these tables from the information presented in the logical data model;
- identifying the specific storage structures and access methods for the data to achieve an optimum performance for the database system;
- designing security protection for the system.

Ideally, conceptual and logical database design for larger systems should be separated from physical design for three main reasons:

- it deals with a different subject matter—the *what*, not the *how*;
- it is performed at a different time—the *what* must be understood before the *how* can be determined;
- it requires different skills, which are often found in different people.

Database design is an iterative process that has a starting point and an almost endless procession of refinements. They should be viewed as learning processes. As the designers come to understand the workings of the enterprise and the meanings of its data, and express that understanding in the selected data models, the information gained may well necessitate changes to other parts of the design. In particular, conceptual and logical database designs are critical to the overall success of the system. If the designs are not a true representation of the enterprise, it will be difficult, if not impossible, to define all the required user views or to maintain database integrity. It may even prove difficult to define the physical implementation or to maintain acceptable system performance. On the other hand, the ability to adjust to change is one hallmark of good database design. Therefore, it is worthwhile spending the time and energy necessary to produce the best possible design.

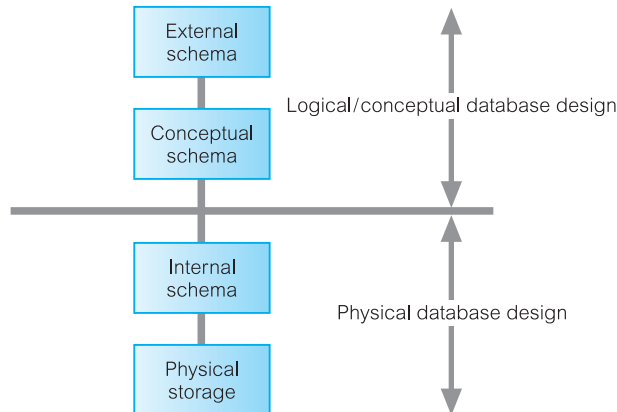


Figure 10.5
Data modeling
and the ANSI-
SPARC
architecture.

In Chapter 2, we discussed the three-level ANSI-SPARC architecture for a database system, consisting of external, conceptual, and internal schemas. Figure 10.5 illustrates the correspondence between this architecture and conceptual, logical, and physical database design. In Chapters 18 and 19 we present a step-by-step methodology for the physical database design phase.

10.7 DBMS Selection

DBMS selection

The selection of an appropriate DBMS to support the database system.

If no DBMS exists, an appropriate part of the lifecycle in which to make a selection is between the conceptual and logical database design phases (see Figure 10.1). However, selection can be done at any time prior to logical design provided sufficient information is available regarding system requirements such as performance, ease of restructuring, security, and integrity constraints.

Although DBMS selection may be infrequent, as enterprise needs expand or existing systems are replaced, it may become necessary at times to evaluate new DBMS products. In such cases, the aim is to select a system that meets the current and future requirements of the enterprise, balanced against costs that include the purchase of the DBMS product, any additional software/hardware required to support the database system, and the costs associated with changeover and staff training.

A simple approach to selection is to check off DBMS features against requirements. In selecting a new DBMS product, there is an opportunity to ensure that the selection process is well planned, and the system delivers real benefits to the enterprise. In the following section we describe a typical approach to selecting the “best” DBMS.

10.7.1 Selecting the DBMS

The main steps to selecting a DBMS are listed in Table 10.3.

TABLE 10.3 Main steps to selecting a DBMS.

Define Terms of Reference of study
Shortlist two or three products
Evaluate products
Recommend selection and produce report

Define Terms of Reference of study

The Terms of Reference for the DBMS selection is established, stating the objectives and scope of the study and the tasks that need to be undertaken. This document may also include a description of the criteria (based on the users’ requirements specification) to be used to evaluate the DBMS products, a preliminary list of possible products, and all necessary constraints and timescales for the study.

Shortlist two or three products

Criteria considered to be “critical” to a successful implementation can be used to produce a preliminary list of DBMS products for evaluation. For example, the decision to include a DBMS product may depend on the budget available, level of vendor support, compatibility with other software, and whether the product runs on particular hardware. Additional useful information on a product can be gathered by contacting existing users, who may provide specific details on how good the vendor support actually is, on how the product supports particular applications, and whether certain hardware platforms are more problematic than others. There may also be benchmarks available that compare the performance of DBMS products. Following an initial study of the functionality and features of DBMS products, a shortlist of two or three products is identified.

The World Wide Web is an excellent source of information and can be used to identify potential candidate DBMSs. For example, InfoWorld’s online technology test center (available at www.infoworld/test-center.com) provides a comprehensive review of DBMS products. Vendors’ Web sites can also provide valuable information on DBMS products.

Evaluate products

There are various features that can be used to evaluate a DBMS product. For the purposes of the evaluation, these features can be assessed as groups (for example, data definition) or individually (for example, data types available). Table 10.4 lists possible features for DBMS product evaluation grouped by data definition, physical definition, accessibility, transaction handling, utilities, development, and other features.

If features are checked off simply with an indication of how good or bad each is, it may be difficult to make comparisons between DBMS products. A more useful approach is to weight features and/or groups of features with respect to their importance to the organization, and to obtain an overall weighted value that can

TABLE 10.4 Features for DBMS evaluation.

DATA DEFINITION	PHYSICAL DEFINITION
Primary key enforcement	File structures available
Foreign key specification	File structure maintenance
Data types available	Ease of reorganization
Data type extensibility	Indexing
Domain specification	Variable length fields/records
Ease of restructuring	Data compression
Integrity controls	Encryption routines
View mechanism	Memory requirements
Data dictionary	Storage requirements
Data independence	
Underlying data model	
Schema evolution	
ACCESSIBILITY	TRANSACTION HANDLING
Query language: SQL2/SQL:2011/ODMG compliant	Backup and recovery routines Checkpointing facility
Interfacing to 3GLs	Logging facility
Multi-user	Granularity of concurrency
Security	Deadlock resolution strategy
• Access controls • Authorization mechanism	Advanced transaction models Parallel query processing
UTILITIES	DEVELOPMENT
Performance measuring	4GL/5GL tools
Tuning	CASE tools
Load/unload facilities	Windows capabilities
User usage monitoring	Stored procedures, triggers, and rules
Database administration support	Web development tools
OTHER FEATURES	
Upgradability	Interoperability with other DBMSs and other systems
Vendor stability	Web integration
User base	Replication utilities
Training and user support	Distributed capabilities

(Continued)

TABLE 10.4 (Continued)

OTHER FEATURES	
Documentation	Portability
Operating system required	Hardware required
Cost	Network support
Online help	Object-oriented capabilities
Standards used	Architecture (2- or 3-tier client/server)
Version management	Performance
Extensible query optimization	Transaction throughput
Scalability	Maximum number of concurrent users
Support for reporting and analytical tools	XML and Web services support

be used to compare products. Table 10.5 illustrates this type of analysis for the “Physical definition” group for a sample DBMS product. Each selected feature is given a rating out of 10, a weighting out of 1 to indicate its importance relative to other features in the group, and a calculated score based on the rating times the weighting. For example, in Table 10.5 the feature “Ease of reorganization” is given a rating of 4, and a weighting of 0.25, producing a score of 1.0. This feature is given the highest weighting in this table, indicating its importance in this part of the evaluation. Additionally, the “Ease of reorganization” feature is weighted, for example, five times higher than the feature “Data compression” with the lowest weighting of 0.05, whereas the two features “Memory requirements” and “Storage

TABLE 10.5 Analysis of features for DBMS product evaluation.

DBMS: Sample Product		Vendor: Sample Vendor		
Physical Definition Group				
FEATURES	COMMENTS	RATING	WEIGHTING	SCORE
File structures available	Choice of 4	8	0.15	1.2
File structure maintenance	Not self-regulating	6	0.2	1.2
Ease of reorganization		4	0.25	1.0
Indexing		6	0.15	0.9
Variable length fields/records		6	0.15	0.9
Data compression	Specify with file structure	7	0.05	0.35
Encryption routines	Choice of 2	4	0.05	0.2
Memory requirements		0	0.00	0
Storage requirements		0	0.00	0
Totals		41	1.0	5.75
Physical definition group		5.75	0.25	1.44

requirements” are given a weighting of 0.00 and are therefore not included in this evaluation.

We next sum all the scores for each evaluated feature to produce a total score for the group. The score for the group is then itself subject to a weighting, to indicate its importance relative to other groups of features included in the evaluation. For example, in Table 10.5, the total score for the “Physical definition” group is 5.75; however, this score has a weighting of 0.25.

Finally, all the weighted scores for each assessed group of features are summed to produce a single score for the DBMS product, which is compared with the scores for the other products. The product with the highest score is the “winner”.

In addition to this type of analysis, we can also evaluate products by allowing vendors to demonstrate their product or by testing the products in-house. In-house evaluation involves creating a pilot testbed using the candidate products. Each product is tested against its ability to meet the users’ requirements for the database system. Benchmarking reports published by the Transaction Processing Council can be found at www.tpc.org.

Recommend selection and produce report

The final step of the DBMS selection is to document the process and to provide a statement of the findings and recommendations for a particular DBMS product.

10.8 Application Design

Application design

The design of the user interface and the application programs that use and process the database.

In Figure 10.1, observe that database and application design are parallel activities of the database system development lifecycle. In most cases, it is not possible to complete the application design until the design of the database itself has taken place. On the other hand, the database exists to support the applications, and so there must be a flow of information between application design and database design.

We must ensure that all the functionality stated in the users’ requirements specification is present in the application design for the database system. This involves designing the application programs that access the database and designing the transactions, (that is, the database access methods). In addition to designing how the required functionality is to be achieved, we have to design an appropriate user interface to the database system. This interface should present the required information in a user-friendly way. The importance of user interface design is sometimes ignored or left until late in the design stages. However, it should be recognized that the interface may be one of the most important components of the system. If it is easy to learn, simple to use, straightforward and forgiving, the users will be inclined to make good use of what information is presented. On the other hand, if the interface has none of these characteristics, the system will undoubtedly cause problems.

In the following sections, we briefly examine two aspects of application design: transaction design and user interface design.

10.8.1 Transaction Design

Before discussing transaction design, we first describe what a transaction represents.

Transaction

An action, or series of actions, carried out by a single user or application program, that accesses or changes the content of the database.

Transactions represent “real-world” events such as the registering of a property for rent, the addition of a new member of staff, the registration of a new client, and the renting out of a property. These transactions have to be applied to the database to ensure that data held by the database remains current with the “real-world” situation and to support the information needs of the users.

A transaction may be composed of several operations, such as the transfer of money from one account to another. However, from the user’s perspective, these operations still accomplish a single task. From the DBMS’s perspective, a transaction transfers the database from one consistent state to another. The DBMS ensures the consistency of the database even in the presence of a failure. The DBMS also ensures that once a transaction has completed, the changes made are permanently stored in the database and cannot be lost or undone (without running another transaction to compensate for the effect of the first transaction). If the transaction cannot complete for any reason, the DBMS should ensure that the changes made by that transaction are undone. In the example of the bank transfer, if money is debited from one account and the transaction fails before crediting the other account, the DBMS should undo the debit. If we were to define the debit and credit operations as separate transactions, then once we had debited the first account and completed the transaction, we are not allowed to undo that change (without running another transaction to credit the debited account with the required amount).

The purpose of transaction design is to define and document the high-level characteristics of the transactions required on the database, including:

- data to be used by the transaction;
- functional characteristics of the transaction;
- output of the transaction;
- importance to the users;
- expected rate of usage.

This activity should be carried out early in the design process to ensure that the implemented database is capable of supporting all the required transactions. There are three main types of transactions: retrieval transactions, update transactions, and mixed transactions:

- **Retrieval transactions** are used to retrieve data for display on the screen or in the production of a report. For example, the operation to search for and display

- the details of a property (given the property number) is an example of a retrieval transaction.
- **Update transactions** are used to insert new records, delete old records, or modify existing records in the database. For example, the operation to insert the details of a new property into the database is an example of an update transaction.
 - **Mixed transactions** involve both the retrieval and updating of data. For example, the operation to search for and display the details of a property (given the property number) and then update the value of the monthly rent is an example of a mixed transaction.

10.8.2 User Interface Design Guidelines

Before implementing a form or report, it is essential that we first design the layout. Useful guidelines to follow when designing forms or reports are listed in Table 10.6 (Shneiderman et al., 2009).

Meaningful title

The information conveyed by the title should clearly and unambiguously identify the purpose of the form/report.

Comprehensible instructions

Familiar terminology should be used to convey instructions to the user. The instructions should be brief, and, when more information is required, help screens should

TABLE 10.6 Guidelines for form/report design.

Meaningful title
Comprehensible instructions
Logical grouping and sequencing of fields
Visually appealing layout of the form/report
Familiar field labels
Consistent terminology and abbreviations
Consistent use of color
Visible space and boundaries for data entry fields
Convenient cursor movement
Error correction for individual characters and entire fields
Error messages for unacceptable values
Optional fields marked clearly
Explanatory messages for fields
Completion signal

be made available. Instructions should be written in a consistent grammatical style, using a standard format.

Logical grouping and sequencing of fields

Related fields should be positioned together on the form/report. The sequencing of fields should be logical and consistent.

Visually appealing layout of the form/report

The form/report should present an attractive interface to the user. The form/report should appear balanced with fields or groups of fields evenly positioned throughout the form/report. There should not be areas of the form/report that have too few or too many fields. Fields or groups of fields should be separated by a regular amount of space. Where appropriate, fields should be vertically or horizontally aligned. In cases where a form on screen has a hardcopy equivalent, the appearance of both should be consistent.

Familiar field labels

Field labels should be familiar. For example, if Sex were replaced by Gender, it is possible that some users would be confused.

Consistent terminology and abbreviations

An agreed list of familiar terms and abbreviations should be used consistently.

Consistent use of color

Color should be used to improve the appearance of a form/report and to highlight important fields or important messages. To achieve this, color should be used in a consistent and meaningful way. For example, fields on a form with a white background may indicate data entry fields and those with a blue background may indicate display-only fields.

Visible space and boundaries for data-entry fields

A user should be visually aware of the total amount of space available for each field. This allows a user to consider the appropriate format for the data before entering the values into a field.

Convenient cursor movement

A user should easily identify the operation required to move a cursor throughout the form/report. Simple mechanisms such as using the Tab key, arrows, or the mouse pointer should be used.

Error correction for individual characters and entire fields

A user should easily identify the operation required to make alterations to field values. Simple mechanisms should be available, such as using the Backspace key or overtyping.

Error messages for unacceptable values

If a user attempts to enter incorrect data into a field, an error message should be displayed. The message should inform the user of the error and indicate permissible values.

Optional fields marked clearly

Optional fields should be clearly identified for the user. This can be achieved using an appropriate field label or by displaying the field using a color that indicates the type of the field. Optional fields should be placed after required fields.

Explanatory messages for fields

When a user places a cursor on a field, information about the field should appear in a regular position on the screen, such as a window status bar.

Completion signal

It should be clear to a user when the process of filling in fields on a form is complete. However, the option to complete the process should not be automatic, as the user may wish to review the data entered.

10.9 Prototyping

At various points throughout the design process, we have the option to either fully implement the database system or build a prototype.

Prototyping

Building a working model of a database system.

A prototype is a working model that does not normally have all the required features or provide all the functionality of the final system. The main purpose of developing a prototype database system is to allow users to use the prototype to identify the features of the system that work well or are inadequate, and—if possible—to suggest improvements or even new features to the database system. In this way, we can greatly clarify the users' requirements for both the users and developers of the system and evaluate the feasibility of a particular system design. Prototypes should have the major advantage of being relatively inexpensive and quick to build.

There are two prototyping strategies in common use today: requirements prototyping and evolutionary prototyping. **Requirements prototyping** uses a prototype to determine the requirements of a proposed database system, and once the requirements are complete, the prototype is discarded. Although **evolutionary prototyping** is used for the same purposes, the important difference is that the prototype is not discarded, but with further development becomes the working database system.

10.10 Implementation

Implementation

The physical realization of the database and application designs.

On completion of the design stages (which may or may not have involved prototyping), we are now in a position to implement the database and the application programs. The database implementation is achieved using the DDL of the selected DBMS or a GUI, which provides the same functionality while hiding the low-level DDL statements. The DDL statements are used to create the database structures and empty database files. Any specified user views are also implemented at this stage.

The application programs are implemented using the preferred third- or fourth-generation language (3GL or 4GL). Parts of these application programs are the database transactions, which are implemented using the DML of the target DBMS, possibly embedded within a host programming language, such as Visual Basic (VB), VB.net, Python, Delphi, C, C++, C#, Java, COBOL, Fortran, Ada, or Pascal. We also implement the other components of the application design such as menu screens, data entry forms, and reports. Again, the target DBMS may have its own fourth-generation tools that allow rapid development of applications through the provision of nonprocedural query languages, reports generators, forms generators, and application generators.

Security and integrity controls for the system are also implemented. Some of these controls are implemented using the DDL, but others may need to be defined outside the DDL, using, for example, the supplied DBMS utilities or operating system controls. Note that SQL is both a DML and a DDL, as described in Chapters 6, 7, and 8.

10.11 Data Conversion and Loading

Data conversion and loading

Transferring any existing data into the new database and converting any existing applications to run on the new database.

This stage is required only when a new database system is replacing an old system. Nowadays, it is common for a DBMS to have a utility that loads existing files into the new database. The utility usually requires the specification of the source file and the target database, and then automatically converts the data to the required format of the new database files. Where applicable, it may be possible for the developer to convert and use application programs from the old system for use by the new system. Whenever conversion and loading are required, the process should be properly planned to ensure a smooth transition to full operation.

10.12 Testing

Testing

The process of running the database system with the intent of finding errors.

Before going live, the newly developed database system should be thoroughly tested. This is achieved using carefully planned test strategies and realistic data, so that the entire testing process is methodically and rigorously carried out. Note that in our definition of testing we have not used the commonly held view that testing is the process of demonstrating that faults are not present. In fact, testing cannot

show the absence of faults; it can show only that software faults are present. If testing is conducted successfully, it will uncover errors with the application programs and possibly the database structure. As a secondary benefit, testing demonstrates that the database and the application programs *appear* to be working according to their specification and that performance requirements *appear* to be satisfied. In addition, metrics collected from the testing stage provide a measure of software reliability and software quality.

As with database design, the users of the new system should be involved in the testing process. The ideal situation for system testing is to have a test database on a separate hardware system, but often this is not possible. If real data is to be used, it is essential to have backups made in case of error.

Testing should also cover usability of the database system. Ideally, an evaluation should be conducted against a usability specification. Examples of criteria that can be used to conduct the evaluation include the following (Sommerville, 2010):

- Learnability: How long does it take a new user to become productive with the system?
- Performance: How well does the system response match the user's work practice?
- Robustness: How tolerant is the system of user error?
- Recoverability: How good is the system at recovering from user errors?
- Adapatability: How closely is the system tied to a single model of work?

Some of these criteria may be evaluated in other stages of the lifecycle. After testing is complete, the database system is ready to be "signed off" and handed over to the users.

10.13 Operational Maintenance

Operational maintenance

The process of monitoring and maintaining the database system following installation.

In the previous stages, the database system has been fully implemented and tested. The system now moves into a maintenance stage, which involves the following activities:

- Monitoring the performance of the system. If the performance falls below an acceptable level, tuning or reorganization of the database may be required.
- Maintaining and upgrading the database system (when required). New requirements are incorporated into the database system through the preceding stages of the lifecycle.

Once the database system is fully operational, close monitoring takes place to ensure that performance remains within acceptable levels. A DBMS normally provides various utilities to aid database administration, including utilities to load data into a database and to monitor the system. The utilities that allow system monitoring give information on, for example, database usage, locking efficiency (including number of deadlocks that have occurred, and so on), and query execution strategy. The DBA can use this information to tune the system to give better

performance; for example, by creating additional indexes to speed up queries, by altering storage structures, or by combining or splitting tables.

The monitoring process continues throughout the life of a database system and in time may lead to reorganization of the database to satisfy the changing requirements. These changes in turn provide information on the likely evolution of the system and the future resources that may be needed. This, together with knowledge of proposed new applications, enables the DBA to engage in capacity planning and to notify or alert senior staff to adjust plans accordingly. If the DBMS lacks certain utilities, the DBA can either develop the required utilities in-house or purchase additional vendor tools, if available. We discuss database administration in more detail in Chapter 20.

When a new database system is brought online, the users should operate it in parallel with the old system for a period of time. This approach safeguards current operations in case of unanticipated problems with the new system. Periodic checks on data consistency between the two systems need to be made, and only when both systems appear to be producing the same results consistently should the old system be dropped. If the changeover is too hasty, the end-result could be disastrous. Despite the foregoing assumption that the old system may be dropped, there may be situations in which both systems are maintained.

10.14 CASE Tools

The first stage of the database system development lifecycle—database planning—may also involve the selection of suitable Computer-Aided Software Engineering (CASE) tools. In its widest sense, CASE can be applied to any tool that supports software development. Appropriate productivity tools are needed by data administration and database administration staff to permit the database development activities to be carried out as efficiently and effectively as possible. CASE support may include:

- a data dictionary to store information about the database system's data;
- design tools to support data analysis;
- tools to permit development of the corporate data model, and the conceptual and logical data models;
- tools to enable the prototyping of applications.

CASE tools may be divided into three categories: upper-CASE, lower-CASE, and integrated-CASE, as illustrated in Figure 10.6. **Upper-CASE** tools support the initial stages of the database system development lifecycle, from planning through to database design. **Lower-CASE** tools support the later stages of the lifecycle, from implementation through testing, to operational maintenance. **Integrated-CASE** tools support all stages of the lifecycle and thus provide the functionality of both upper- and lower-CASE in one tool.

Benefits of CASE

The use of appropriate CASE tools should improve the productivity of developing a database system. We use the term “productivity” to relate both to the efficiency of the development process and to the effectiveness of the developed system.

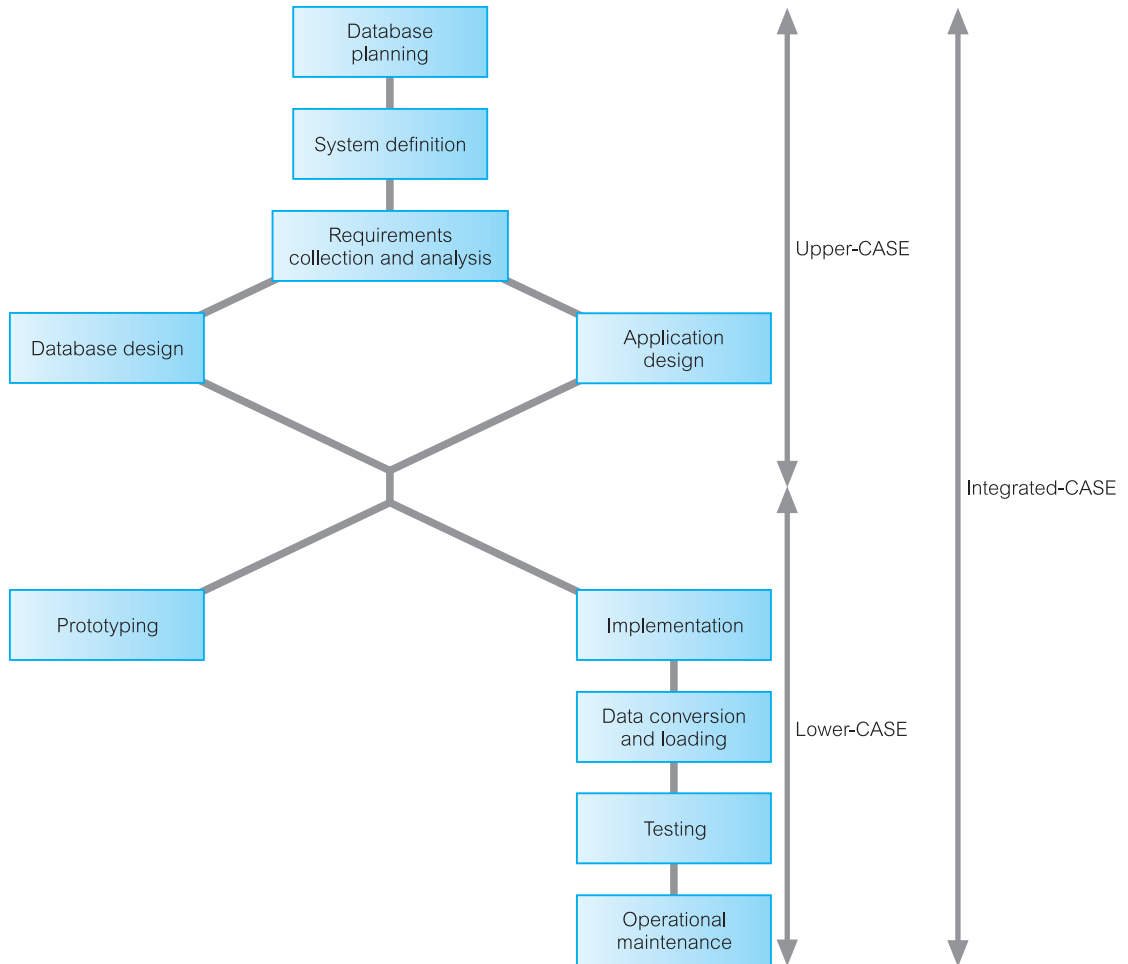


Figure 10.6 Application of CASE tools.

Efficiency refers to the cost, in terms of time and money, of realizing the database system. CASE tools aim to support and automate the development tasks and thus improve efficiency. *Effectiveness* refers to the extent to which the system satisfies the information needs of its users. In the pursuit of greater productivity, raising the effectiveness of the development process may be even more important than increasing its efficiency. For example, it would not be sensible to develop a database system extremely efficiently when the end-product is not what the users want. In this way, effectiveness is related to the quality of the final product. Because computers are better than humans at certain tasks—for example, consistency checking—CASE tools can be used to increase the effectiveness of some tasks in the development process.

CASE tools provide the following benefits that improve productivity:

- *Standards:* CASE tools help to enforce standards on a software project or across the organization. They encourage the production of standard test

components that can be reused, thus simplifying maintenance and increasing productivity.

- *Integration:* CASE tools store all the information generated in a repository, or data dictionary. Thus, it should be possible to store the data gathered during all stages of the database system development lifecycle. The data then can be linked together to ensure that all parts of the system are integrated. In this way, an organization's information system no longer has to consist of independent, unconnected components.
- *Support for standard methods:* Structured techniques make significant use of diagrams, which are difficult to draw and maintain manually. CASE tools simplify this process, resulting in documentation that is correct and more current.
- *Consistency:* Because all the information in the data dictionary is interrelated, CASE tools can check its consistency.
- *Automation:* Some CASE tools can automatically transform parts of a design specification into executable code. This feature reduces the work required to produce the implemented system, and may eliminate errors that arise during the coding process.

Chapter Summary

- An **information system** is the resources that enable the collection, management, control, and dissemination of information throughout an organization.
- A computer-based information system includes the following components: database, database software, application software, computer hardware (including storage media), and personnel using and developing the system.
- The database is a fundamental component of an information system, and its development and usage should be viewed from the perspective of the wider requirements of the organization. Therefore, the lifecycle of an organizational information system is inherently linked to the lifecycle of the database that supports it.
- The main stages of the **database system development lifecycle** include: database planning, system definition, requirements collection and analysis, database design, DBMS selection (optional), application design, prototyping (optional), implementation, data conversion and loading, testing, and operational maintenance.
- **Database planning** involves the management activities that allow the stages of the database system development lifecycle to be realized as efficiently and effectively as possible.
- **System definition** involves identifying the scope and boundaries of the database system and user views. A **user view** defines what is required of a database system from the perspective of a particular job role (such as Manager or Supervisor) or enterprise application (such as marketing, personnel, or stock control).
- **Requirements collection and analysis** is the process of collecting and analyzing information about the part of the organization that is to be supported by the database system, and using this information to identify the requirements for the new system. There are three main approaches to managing the requirements for a database system that has multiple user views: the **centralized** approach, the **view integration** approach, and a combination of both approaches.
- The **centralized** approach involves merging the requirements for each user view into a single set of requirements for the new database system. A data model representing all user views is created during the database design stage. In the **view integration** approach, requirements for each user view remain as separate lists. Data models representing each user view are created then merged later during the database design stage.

- **Database design** is the process of creating a design that will support the enterprise's mission statement and mission objectives for the required database system. There are three phases of database design: conceptual, logical, and physical database design.
- **Conceptual database design** is the process of constructing a model of the data used in an enterprise, independent of *all* physical considerations.
- **Logical database design** is the process of constructing a model of the data used in an enterprise based on a specific data model, but independent of a particular DBMS and other physical considerations.
- **Physical database design** is the process of producing a description of the implementation of the database on secondary storage; it describes the base relations, file organizations, and indexes used to achieve efficient access to the data, and any associated integrity constraints and security measures.
- **DBMS selection** involves selecting a suitable DBMS for the database system.
- **Application design** involves user interface design and transaction design, which describes the application programs that use and process the database. A database **transaction** is an action or series of actions carried out by a single user or application program, which accesses or changes the content of the database.
- **Prototyping** involves building a working model of the database system, which allows the designers or users to visualize and evaluate the system.
- **Implementation** is the physical realization of the database and application designs.
- **Data conversion and loading** involves transferring any existing data into the new database and converting any existing applications to run on the new database.
- **Testing** is the process of running the database system with the intent of finding errors.
- **Operational maintenance** is the process of monitoring and maintaining the system following installation.
- **Computer-Aided Software Engineering (CASE)** applies to any tool that supports software development and permits the database system development activities to be carried out as efficiently and effectively as possible. CASE tools may be divided into three categories: upper-CASE, lower-CASE, and integrated-CASE.



Review Questions

- 10.1 Discuss the interdependence that exists between DSDLC stages.
- 10.2 What do you understand by the term "system mission"? Why is it important during system development?
- 10.3 Describe the main purpose(s) and activities associated with each stage of the database system development lifecycle.
- 10.4 Discuss what a user view represents in the context of a database system.
- 10.5 Discuss the main approaches to database design. Discuss the contexts where each is appropriate.
- 10.6 Compare and contrast the three phases of database design.
- 10.7 What are the main purposes of data modeling and identify the criteria for an optimal data model?
- 10.8 Identify the stage(s) in which it is appropriate to select a DBMS and describe an approach to selecting the "best" DBMS.
- 10.9 Application design involves transaction design and user interface design. Describe the purpose and main activities associated with each.
- 10.10 Discuss why testing cannot show the absence of faults, only that software faults are present.
- 10.11 What is the difference between the prototyping approach and the database systems development lifecycle?

 Exercises

- 10.12 Assume you have been contracted to develop a database system for a university library. You are required to use a systems development lifecycle approach. Discuss how you are going to approach the project. Describe user groups that will be involved during the requirement analysis. What are the key issues that need to be answered during fact finding?
- 10.13 Describe the process of evaluating and selecting a DBMS product for each of the case studies described in Appendix B.
- 10.14 Assume that you are an employee of a consultancy company that specializes in the analysis, design, and implementation of database systems. A client has recently approached your company with a view to implementing a database system but they are not familiar with the development process. You have been assigned the task to present an overview of the Database System Development Lifecycle (DSDL) to them, identifying the main stages of this lifecycle. With this task in mind, create a slide presentation and/or short report for the client. (The client for this exercise can be any one of the fictitious case studies given in Appendix B or some real company identified by you or your professor).
- 10.15 This exercise requires you to first gain permission to interview one or more people responsible for the development and/or administration of a real database system. During the interview(s), find out the following information:
- (a) The approach taken to develop the database system.
 - (b) How the approach taken differs or is similar to the DSDL approach described in this chapter.
 - (c) How the requirements for different users (user views) of the database systems were managed.
 - (d) Whether a CASE tool was used to support the development of the database system.
 - (e) How the DBMS product was evaluated and then selected.
 - (f) How the database system is monitored and maintained.

Database Analysis and the *DreamHome* Case Study

Chapter Objectives

In this chapter you will learn:

- When fact-finding techniques are used in the database system development lifecycle.
- The types of facts collected in each stage of the database system development lifecycle.
- The types of documentation produced in each stage of the database system development lifecycle.
- The most commonly used fact-finding techniques.
- How to use each fact-finding technique and the advantages and disadvantages of each.
- About a property rental company called *DreamHome*.
- How to apply fact-finding techniques to the early stages of the database system development lifecycle.

In Chapter 10 we introduced the stages of the database system development lifecycle. There are many occasions during these when it is critical that the database developer captures the necessary facts to build the required database system. The necessary facts include, for example, the terminology used within the enterprise, problems encountered using the current system, opportunities sought from the new system, necessary constraints on the data and users of the new system, and a prioritized set of requirements for the new system. These facts are captured using fact-finding techniques.

Fact-finding

The formal process of using techniques such as interviews and questionnaires to collect facts about systems, requirements, and preferences.

In this chapter we discuss when a database developer might use fact-finding techniques and what types of facts should be captured. We present an overview of how these facts are used to generate the main types of documentation used throughout the database system development lifecycle. We describe the most commonly used fact-finding techniques and identify the advantages and disadvantages



of each. We finally demonstrate how some of these techniques may be used during the earlier stages of the database system development lifecycle using a property management company called *DreamHome*. The *DreamHome* case study is used throughout this book.



Structure of this Chapter In Section 11.1 we discuss when a database developer might use fact-finding techniques. (Throughout this book we use the term “database developer” to refer to a person or group of people responsible for the analysis, design, and implementation of a database system.) In Section 11.2 we illustrate the types of facts that should be collected and the documentation that should be produced at each stage of the database system development lifecycle. In Section 11.3 we describe the five most commonly used fact-finding techniques and identify the advantages and disadvantages of each. In Section 11.4 we demonstrate how fact-finding techniques can be used to develop a database system for a case study called *DreamHome*, a property management company. We begin this section by providing an overview of the *DreamHome* case study. We then examine the first three stages of the database system development lifecycle, namely database planning, system definition, and requirements collection and analysis. For each stage we demonstrate the process of collecting data using fact-finding techniques and describe the documentation produced.

11.1 When Are Fact-Finding Techniques Used?

There are many occasions for fact-finding during the database system development life cycle. However, fact-finding is particularly crucial to the early stages of the lifecycle, including the database planning, system definition, and requirements collection and analysis stages. It is during these early stages that the database developer captures the essential facts necessary to build the required database. Fact-finding is also used during database design and the later stages of the lifecycle, but to a lesser extent. For example, during physical database design, fact-finding becomes technical as the database developer attempts to learn more about the DBMS selected for the database system. Also, during the final stage, operational maintenance, fact-finding is used to determine whether a system requires tuning to improve performance or further development to include new requirements.

Note that it is important to have a rough estimate of how much time and effort is to be spent on fact-finding for a database project. As we mentioned in Chapter 10, too much study too soon leads to *paralysis by analysis*. However, too little thought can result in an unnecessary waste of both time and money, due to working on the wrong solution to the wrong problem.

11.2 What Facts Are Collected?

Throughout the database system development lifecycle, the database developer needs to capture facts about the current and/or future system. Table 11.1 provides examples of the sorts of data captured and the documentation produced for each stage of the lifecycle. As we mentioned in Chapter 10, the stages of the database system development lifecycle are not strictly sequential, but involve some amount of repetition of previous stages through feedback loops. This is also true for the data captured and the documentation produced at each stage. For example, problems encountered during database design may necessitate additional data capture on the requirements for the new system.

TABLE 11.1 Examples of the data captured and the documentation produced for each stage of the database system development lifecycle.

STAGE OF DATABASE SYSTEM DEVELOPMENT LIFECYCLE	EXAMPLES OF DATA CAPTURED	EXAMPLES OF DOCUMENTATION PRODUCED
Database planning	Aims and objectives of database project	Mission statement and objectives of database system
System definition	Description of major user views (includes job roles or business application areas)	Definition of scope and boundary of database system; definition of user views to be supported
Requirements collection and analysis	Requirements for user views; systems specifications, including performance and security requirements	Users' and system requirements specifications
Database design	Users' responses to checking the conceptual/logical database design; functionality provided by target DBMS	Conceptual/logical database design (includes ER model(s), data dictionary, and relational schema); physical database design
Application design	Users' responses to checking interface design	Application design (includes description of programs and user interface)
DBMS selection	Functionality provided by target DBMS	DBMS evaluation and recommendations
Prototyping	Users' responses to prototype	Modified users' requirements and systems specifications
Implementation	Functionality provided by target DBMS	
Data conversion and loading	Format of current data; data import capabilities of target DBMS	
Testing	Test results	Testing strategies used; analysis of test results
Operational maintenance	Performance testing results; new or changing user and system requirements	User manual; analysis of performance results; modified users' requirements and systems specifications

11.3

Fact-Finding Techniques

A database developer normally uses several fact-finding techniques during a single database project. There are five commonly used fact-finding techniques:

- examining documentation;
- interviewing;
- observing the enterprise in operation;
- research;
- questionnaires.

In the following sections we describe these fact-finding techniques and identify the advantages and disadvantages of each.

11.3.1

Examining Documentation

Examining documentation can be useful when we are trying to gain some insight as to how the need for a database arose. We may also find that documentation can help to provide information on the part of the enterprise associated with the problem. If the problem relates to the current system, there should be documentation associated with that system. By examining documents, forms, reports, and files associated with the current system, we can quickly gain some understanding of the system. Examples of the types of documentation that should be examined are listed in Table 11.2.

11.3.2

Interviewing

Interviewing is the most commonly used and normally the most useful fact-finding technique. We can interview to collect information from individuals face-to-face. There can be several objectives to using interviewing, such as finding out

TABLE 11.2 Examples of types of documentation that should be examined.

PURPOSE OF DOCUMENTATION	EXAMPLES OF USEFUL SOURCES
Describes problem and need for database	Internal memos, emails, and minutes of meetings Employee complaints and documents that describe the problem Social media such as blogs and tweets Performance reviews/reports
Describes the part of the enterprise affected by problem	Organizational chart, mission statement, and strategic plan of the enterprise Objectives for the part of the enterprise being studied Task/job descriptions Samples of completed manual forms and reports Samples of completed computerized forms and reports
Describes current system	Various types of flowcharts and diagrams Data dictionary Database system design Program documentation User/training manuals

TABLE 11.3 Advantages and disadvantages of using interviewing as a fact-finding technique.

ADVANTAGES	DISADVANTAGES
Allows interviewee to respond freely and openly to questions	Very time-consuming and costly, and therefore may be impractical
Allows interviewee to feel part of project	Success is dependent on communication skills of interviewer
Allows interviewer to follow up on interesting comments made by interviewee	Success can be dependent on willingness of interviewees to participate in interviews
Allows interviewer to adapt or reword questions during interview	
Allows interviewer to observe interviewee's body language	

facts, verifying facts, clarifying facts, generating enthusiasm, getting the end-user involved, identifying requirements, and gathering ideas and opinions. However, using the interviewing technique requires good communication skills for dealing effectively with people who have different values, priorities, opinions, motivations, and personalities. As with other fact-finding techniques, interviewing is not always the best method for all situations. The advantages and disadvantages of using interviewing as a fact-finding technique are listed in Table 11.3.

There are two types of interview: unstructured and structured. **Unstructured interviews** are conducted with only a general objective in mind and with few, if any, specific questions. The interviewer counts on the interviewee to provide a framework and direction to the interview. This type of interview frequently loses focus and, for this reason, it often does not work well for database analysis and design.

In **structured interviews**, the interviewer has a specific set of questions to ask the interviewee. Depending on the interviewee's responses, the interviewer will direct additional questions to obtain clarification or expansion. **Open-ended questions** allow the interviewee to respond in any way that seems appropriate. An example of an open-ended question is: "Why are you dissatisfied with the report on client registration?" **Closed-ended questions** restrict answers to either specific choices or short, direct responses. An example of such a question might be: "Are you receiving the report on client registration on time?" or "Does the report on client registration contain accurate information?" Both questions require only a "Yes" or "No" response.

To ensure a successful interview includes selecting appropriate individuals to interview, preparing extensively for the interview, and conducting the interview in an efficient and effective manner.

11.3.3 Observing the Enterprise in Operation

Observation is one of the most effective fact-finding techniques for understanding a system. With this technique, it is possible to either participate in or watch a person perform activities to learn about the system. This technique is particularly useful when the validity of data collected through other methods is in question or when the complexity of certain aspects of the system prevents a clear explanation by the end-users.

TABLE 11.4 Advantages and disadvantages of using observation as a fact-finding technique.

ADVANTAGES	DISADVANTAGES
Allows the validity of facts and data to be checked	People may knowingly or unknowingly perform differently when being observed
Observer can see exactly what is being done	May miss observing tasks involving different levels of difficulty or volume normally experienced during that time period
Observer can also obtain data describing the physical environment of the task	Some tasks may not always be performed in the manner in which they are observed
Relatively inexpensive	May be impractical
Observer can do work measurements	

As with the other fact-finding techniques, successful observation requires preparation. To ensure that the observation is successful, it is important to know as much about the individuals and the activity to be observed as possible. For example, “When are the low, normal, and peak periods for the activity being observed?” and “Will the individuals be upset by having someone watch and record their actions?” The advantages and disadvantages of using observation as a fact-finding technique are listed in Table 11.4.

11.3.4 Research

A useful fact-finding technique is to research the application and problem. Computer trade journals, reference books, and the Internet (including user groups and bulletin boards) are good sources of information. They can provide information on how others have solved similar problems, plus on whether software packages exist to solve or even partially solve the problem. The advantages and disadvantages of using research as a fact-finding technique are listed in Table 11.5.

11.3.5 Questionnaires

Another fact-finding technique is to conduct surveys through questionnaires. Questionnaires are special-purpose documents that allow facts to be gathered from a large number of people while maintaining some control over their responses.

TABLE 11.5 Advantages and disadvantages of using research as a fact-finding technique.

ADVANTAGES	DISADVANTAGES
Can save time if solution already exists	Requires access to appropriate sources of information
Researcher can see how others have solved similar problems or met similar requirements	May ultimately not help in solving problem because problem is not documented elsewhere
Keeps researcher up to date with current developments	

TABLE 11.6 Advantages and disadvantages of using questionnaires as a fact-finding technique.

ADVANTAGES	DISADVANTAGES
People can complete and return questionnaires at their convenience	Number of respondents can be low, possibly only 5% to 10%
Relatively inexpensive way to gather data from a large number of people	Questionnaires may be returned incomplete
People more likely to provide the real facts as responses can be kept confidential	May not provide an opportunity to adapt or reword questions that have been misinterpreted
Responses can be tabulated and analyzed quickly	Cannot observe and analyze the respondent's body language

When dealing with a large audience, no other fact-finding technique can tabulate the same facts as efficiently. The advantages and disadvantages of using questionnaires as a fact-finding technique are listed in Table 11.6.

There are two types of questions that can be asked in a questionnaire: free-format and fixed-format. **Free-format questions** offer the respondent greater freedom in providing answers. A question is asked and the respondent records the answer in the space provided after the question. Examples of free-format questions are: “What reports do you currently receive and how are they used?” and “Are there any problems with these reports? If so, please explain.” The problems with free-format questions are that the respondent’s answers may prove difficult to tabulate, and in some cases, may not match the questions asked.

Fixed-format questions require specific responses from individuals. Given any question, the respondent must choose from the available answers. This makes the results much easier to tabulate. On the other hand, the respondent cannot provide additional information that might prove valuable. An example of a fixed-format question is: “The current format of the report on property rentals is ideal and should not be changed.” The respondent may be given the option to answer “Yes” or “No” to this question, or be given the option to answer from a range of responses including “Strongly agree,” “Agree,” “No opinion,” “Disagree,” and “Strongly disagree.”

11.4 Using Fact-Finding Techniques: A Worked Example

In this section we first present an overview of the *DreamHome* case study and then use this case study to illustrate how to establish a database project. In particular, we illustrate how fact-finding techniques can be used and the documentation produced in the early stages of the database system development lifecycle—namely, the database planning, system definition, and requirements collection and analysis stages.





11.4.1 The *DreamHome* Case Study—An Overview of the Current System

The first branch office of *DreamHome* was opened in 1992 in Glasgow in the UK. Since then, the Company has grown steadily and now has several offices in most of the main cities of the UK. However, the Company is now so large that more and more administrative staff are being employed to cope with the ever-increasing amount of paperwork. Furthermore, the communication and sharing of information between offices, even in the same city, is poor. The Director of the Company, Sally Mellweadows, feels that too many mistakes are being made and that the success of the Company will be short-lived if she does not do something to remedy the situation. She knows that a database could help in part to solve the problem and has requested that a database system be developed to support the running of *DreamHome*. The Director has provided the following brief description of how *DreamHome* currently operates.

DreamHome specializes in property management, taking an intermediate role between owners who wish to rent out their furnished property and clients of *DreamHome* who require to rent furnished property for a fixed period. *DreamHome* currently has about 2000 staff working in 100 branches. When a member of staff joins the Company, the *DreamHome* staff registration form is used. The staff registration form for Susan Brand is shown in Figure 11.1.

Each branch has an appropriate number and type of staff including a Manager, Supervisors, and Assistants. The Manager is responsible for the day-to-day running of a branch and each Supervisor is responsible for supervising a group of staff called Assistants. An example of the first page of a report listing the details of staff working at a branch office in Glasgow is shown in Figure 11.2.

DreamHome Staff Registration Form	
Staff Number SG5 Full Name Susan Brand Sex F DOB 3-Jun-70	Branch Number B003 Branch Address 163 Main St, Glasgow Telephone Number(s) 0141-339-2178 / 0141-339-4439
Position Manager Salary 24000	
Enter details where applicable Supervisor Name	Manager Start Date 01-Jun-99 Manager Bonus 2350

Figure 11.1 The *DreamHome* staff registration form for Susan Brand.

DreamHome Staff Listing																							
Branch Number	B003	Branch Address																					
Telephone Number(s)	0141-339-2178 / 0141-339-4439	163 Main St, Glasgow G11 9QX																					
<table border="1"> <thead> <tr> <th>Staff Number</th> <th>Name</th> <th>Position</th> </tr> </thead> <tbody> <tr> <td>SG5</td> <td>Susan Brand</td> <td>Manager</td> </tr> <tr> <td>SG14</td> <td>David Ford</td> <td>Supervisor</td> </tr> <tr> <td>SG37</td> <td>Ann Beech</td> <td>Assistant</td> </tr> <tr> <td>SG112</td> <td>Annet Longhorn</td> <td>Supervisor</td> </tr> <tr> <td>SG126</td> <td>Chris Lawrence</td> <td>Assistant</td> </tr> <tr> <td>SG132</td> <td>Sofie Walters</td> <td>Assistant</td> </tr> </tbody> </table>			Staff Number	Name	Position	SG5	Susan Brand	Manager	SG14	David Ford	Supervisor	SG37	Ann Beech	Assistant	SG112	Annet Longhorn	Supervisor	SG126	Chris Lawrence	Assistant	SG132	Sofie Walters	Assistant
Staff Number	Name	Position																					
SG5	Susan Brand	Manager																					
SG14	David Ford	Supervisor																					
SG37	Ann Beech	Assistant																					
SG112	Annet Longhorn	Supervisor																					
SG126	Chris Lawrence	Assistant																					
SG132	Sofie Walters	Assistant																					
Page 1																							

Figure 11.2 Example of the first page of a report listing the details of staff working at a *DreamHome* branch office in Glasgow.

Each branch office offers a range of properties for rent. To offer property through *DreamHome*, a property owner normally contacts the *DreamHome* branch office nearest to the property for rent. The owner provides the details of the property and agrees an appropriate rent for the property with the branch Manager. The registration form for a property in Glasgow is shown in Figure 11.3.

Once a property is registered, *DreamHome* provides services to ensure that the property is rented out for maximum return for both the property owner and, of course, *DreamHome*. These services include interviewing prospective renters (called clients), organizing viewings of the property by clients, advertising the property in local or national newspapers (when necessary), and negotiating the lease. Once rented, *DreamHome* assumes responsibility for the property including the collection of rent.

Members of the public interested in renting out property must first contact their nearest *DreamHome* branch office to register as clients of *DreamHome*. However, before registration is accepted, a prospective client is normally interviewed to record personal details and preferences of the client in terms of property requirements. The registration form for a client called Mike Ritchie is shown in Figure 11.4.

Once registration is complete, clients are provided with weekly reports that list properties currently available for rent. An example of the first page of a report listing the properties available for rent at a branch office in Glasgow is shown in Figure 11.5.

Clients may request to view one or more properties from the list and after viewing will normally provide a comment on the suitability of the property. The first page of a report describing the comments made by clients on a property in Glasgow is shown in Figure 11.6. Properties that prove difficult to rent out are normally advertised in local and national newspapers.

DreamHome Property Registration Form	
Property Number PG16 Type Flat Rooms 4 Rent 450 Address 5 Novar Drive, Glasgow, G12 9AX	Owner Number C093 (If known) Person/Business Name Tony Shaw Address 12 Park Pl, Glasgow G4 0QR Tel No 0141-225-7025
	Enter details where applicable Type of business Contact Name
Managed by staff David Ford	Registered at branch 163 Main St, Glasgow

Figure 11.3 The DreamHome property registration form for a property in Glasgow.

DreamHome Client Registration Form	
Client Number CR74 (Enter if known) Full Name Mike Ritchie	Branch Number B003 Branch Address 163 Main St, Glasgow
Enter property requirements Type Flat Max Rent 750	Registered By Ann Beech Date Registered 16-Nov-11

Figure 11.4 The DreamHome client registration form for Mike Ritchie.

DreamHome Property Listing for Week beginning 01/06/13																																							
If you are interested in viewing or renting any of the properties in this list, please contact our branch office as soon as possible.																																							
Branch Address 163 Main St, Glasgow G11 9QX		Telephone Number(s) 0141-339-2178 / 0141-339-4439																																					
<table border="1"> <thead> <tr> <th>Property No</th> <th>Address</th> <th>Type</th> <th>Rooms</th> <th>Rent</th> </tr> </thead> <tbody> <tr> <td>PG4</td> <td>6 Lawrence St, Glasgow</td> <td>Flat</td> <td>3</td> <td>350</td> </tr> <tr> <td>PG36</td> <td>2 Manor Rd, Glasgow</td> <td>Flat</td> <td>3</td> <td>375</td> </tr> <tr> <td>PG21</td> <td>18 Dale Road, Glasgow</td> <td>House</td> <td>5</td> <td>600</td> </tr> <tr> <td>PG16</td> <td>5 Novar Drive, Glasgow</td> <td>Flat</td> <td>4</td> <td>450</td> </tr> <tr> <td>PG77</td> <td>100A Apple Lane, Glasgow</td> <td>House</td> <td>6</td> <td>560</td> </tr> <tr> <td>PG81</td> <td>781 Greentree Dr, Glasgow</td> <td>Flat</td> <td>4</td> <td>440</td> </tr> </tbody> </table>					Property No	Address	Type	Rooms	Rent	PG4	6 Lawrence St, Glasgow	Flat	3	350	PG36	2 Manor Rd, Glasgow	Flat	3	375	PG21	18 Dale Road, Glasgow	House	5	600	PG16	5 Novar Drive, Glasgow	Flat	4	450	PG77	100A Apple Lane, Glasgow	House	6	560	PG81	781 Greentree Dr, Glasgow	Flat	4	440
Property No	Address	Type	Rooms	Rent																																			
PG4	6 Lawrence St, Glasgow	Flat	3	350																																			
PG36	2 Manor Rd, Glasgow	Flat	3	375																																			
PG21	18 Dale Road, Glasgow	House	5	600																																			
PG16	5 Novar Drive, Glasgow	Flat	4	450																																			
PG77	100A Apple Lane, Glasgow	House	6	560																																			
PG81	781 Greentree Dr, Glasgow	Flat	4	440																																			
Page 1																																							

Figure 11.5 The first page of the *DreamHome* property for rent report listing property available at a branch in Glasgow.

DreamHome Property Viewing Report																							
Property Nummer PG4		Property Address 6 Lawrence St, Glasgow																					
Type Flat																							
Rent 350																							
<table border="1"> <thead> <tr> <th>Client No</th> <th>Name</th> <th>Date</th> <th>Comments</th> </tr> </thead> <tbody> <tr> <td>CR76</td> <td>John Kay</td> <td>20/04/13</td> <td>Too remote.</td> </tr> <tr> <td>CR56</td> <td>Aline Stewart</td> <td>26/05/13</td> <td></td> </tr> <tr> <td>CR74</td> <td>Mike Ritchie</td> <td>11/11/13</td> <td></td> </tr> <tr> <td>CR62</td> <td>Mary Tregear</td> <td>11/11/13</td> <td>OK, but needs redecoration throughout.</td> </tr> </tbody> </table>				Client No	Name	Date	Comments	CR76	John Kay	20/04/13	Too remote.	CR56	Aline Stewart	26/05/13		CR74	Mike Ritchie	11/11/13		CR62	Mary Tregear	11/11/13	OK, but needs redecoration throughout.
Client No	Name	Date	Comments																				
CR76	John Kay	20/04/13	Too remote.																				
CR56	Aline Stewart	26/05/13																					
CR74	Mike Ritchie	11/11/13																					
CR62	Mary Tregear	11/11/13	OK, but needs redecoration throughout.																				
Page 1																							

Figure 11.6 The first page of the *DreamHome* property viewing report for a property in Glasgow.

DreamHome Lease Number 00345810	
Client Number <u>CR74</u> (Enter if known) Full Name <u>Mike Ritchie</u> (Please print) Client Signature _____	Property Number <u>PG16</u> Property Address <u>5 Novar Dr, Glasgow</u>
Enter payment details Monthly Rent <u>450</u> Payment Method <u>Cheque</u> Deposit Paid (Y or N) <u>Yes</u>	Rent Start <u>01/06/12</u> Rent Finish <u>31/05/13</u> Duration <u>1 year</u>

Figure 11.7 The *DreamHome* lease form for a client called Mike Ritchie renting a property in Glasgow.

Once a client has identified a suitable property, a member of staff draws up a lease. The lease between a client called Mike Ritchie and a property in Glasgow is shown in Figure 11.7.

At the end of a rental period a client may request that the rental be continued; however, this requires that a new lease be drawn up. Alternatively, a client may request to view alternative properties for the purposes of renting.



11.4.2 The *DreamHome* Case Study—Database Planning

The first step in developing a database system is to clearly define the **mission statement** for the database project, which defines the major aims of the database system. Once the mission statement is defined, the next activity involves identifying the **mission objectives**, which should identify the particular tasks that the database must support (see Section 10.3).

Creating the mission statement for the *DreamHome* database system

We begin the process of creating a mission statement for the *DreamHome* database system by conducting interviews with the Director and any other appropriate staff, as indicated by the Director. Open-ended questions are normally the most useful at this stage of the process. Examples of typical questions we might ask include:

- “What is the purpose of your company?”
- “Why do you feel that you need a database?”
- “How do you know that a database will solve your problem?”

For example, the database developer may start the interview by asking the Director of *DreamHome* the following questions:

Database Developer: *What is the purpose of your company?*

Director: We offer a wide range of high-quality properties for rent to clients registered at our branches throughout the UK. Our ability to offer quality properties, of course, depends upon the services we provide to property owners. We provide a highly professional service to property owners to ensure that properties are rented out for maximum return.

Database Developer: *Why do you feel that you need a database?*

Director: To be honest, we can't cope with our own success. Over the past few years we've opened several branches in most of the main cities of the UK, and at each branch we now offer a larger selection of properties to a growing number of clients. However, this success has been accompanied with increasing data management problems, which means that the level of service we provide is falling. Also, there's a lack of cooperation and sharing of information between branches, which is a very worrying development.

Database Developer: *How do you know that a database will solve your problem?*

Director: All I know is that we are drowning in paperwork. We need something that will speed up the way we work by automating a lot of the day-to-day tasks that seem to take forever these days. Also, I want the branches to start working together. Databases will help to achieve this, won't they?

Responses to these types of questions should help formulate the mission statement. An example mission statement for the *DreamHome* database system is shown in Figure 11.8. When we have a clear and unambiguous mission statement that the staff of *DreamHome* agree with, we move on to define the mission objectives.

Creating the mission objectives for the *DreamHome* database system

The process of creating mission objectives involves conducting interviews with appropriate members of staff. Again, open-ended questions are normally the most useful at this stage of the process. To obtain the complete range of mission

"The purpose of the *DreamHome* database system is to maintain the data that is used and generated to support the property rentals business for our clients and property owners and to facilitate the cooperation and sharing of information between branches."

Figure 11.8 Mission statement for the *DreamHome* database system.

objectives, we interview various members of staff with different roles in *DreamHome*. Examples of typical questions that we might ask include:

- “What is your job description?”
- “What kinds of tasks do you perform in a typical day?”
- “What kinds of data do you work with?”
- “What types of reports do you use?”
- “What types of things do you need to keep track of?”
- “What service does your company provide to your customers?”

These questions (or similar) are put to the Director of *DreamHome* and members of staff in the role of Manager, Supervisor, and Assistant. It may be necessary to adapt the questions as required, depending on whom is being interviewed.

Director

Database Developer: *What role do you play for the company?*

Director: I oversee the running of the company to ensure that we continue to provide the best possible property rental service to our clients and property owners.

Database Developer: *What kinds of tasks do you perform in a typical day?*

Director: I monitor the running of each branch by our Managers. I try to ensure that the branches work well together and share important information about properties and clients. I normally try to keep a high profile with my branch Managers by calling into each branch at least once or twice a month.

Database Developer: *What kinds of data do you work with?*

Director: I need to see everything, well at least a summary of the data used or generated by *DreamHome*. That includes data about staff at all branches, all properties and their owners, all clients, and all leases. I also like to keep an eye on the extent to which branches advertise properties in newspapers.

Database Developer: *What types of reports do you use?*

Director: I need to know what’s going on at all the branches and there are lots of them. I spend a lot of my working day going over long reports on all aspects of *DreamHome*. I need reports that are easy to access and that let me get a good overview of what’s happening at a given branch and across all branches.

Database Developer: *What types of things do you need to keep track of?*

Director: As I said before, I need to have an overview of everything; I need to see the whole picture.

Database Developer: *What service does your company provide to your customers?*

Director: We aim to provide the best property rental service in the UK. I believe that this will be achieved with the support of the new database system, which will allow

my staff to deal more efficiently with our customers and clients and better marketing of our properties through the development of a new *DreamHome* Web site. This site will allow our current and new renting clients to view our properties on the Web.

Manager

Database Developer: *What is your job description?*

Manager: My job title is Manager. I oversee the day-to-day running of my branch to provide the best property rental service to our clients and property owners.

Database Developer: *What kinds of tasks do you perform in a typical day?*

Manager: I ensure that the branch has the appropriate number and type of staff on duty at all times. I monitor the registering of new properties and new clients, and the renting activity of our currently active clients. It's my responsibility to ensure that we have the right number and type of properties available to offer our clients. I sometimes get involved in negotiating leases for our top-of-the-range properties, although due to my workload, I often have to delegate this task to Supervisors.

Database Developer: *What kinds of data do you work with?*

Manager: I mostly work with data on the properties offered at my branch and the owners, clients, and leases. I also need to know when properties are proving difficult to rent out so that I can arrange for them to be advertised in newspapers. I need to keep an eye on this aspect of the business, because advertising can get costly. I also need access to data about staff working at my branch and staff at other local branches. This is because I sometimes need to contact other branches to arrange management meetings or to borrow staff from other branches on a temporary basis to cover staff shortages due to sickness or during holiday periods. This borrowing of staff between local branches is informal and thankfully doesn't happen very often. Besides data on staff, it would be helpful to see other types of data at the other branches such as data on property, property owners, clients, and leases, you know, to compare notes. Actually, I think the Director hopes that this database project is going to help promote cooperation and sharing of information between branches. However, some of the Managers I know are not going to be too keen on this, because they think we're in competition with each other. Part of the problem is that a percentage of a Manager's salary is made up of a bonus, which is related to the number of properties we rent out.

Database Developer: *What types of reports do you use?*

Manager: I need various reports on staff, property, owners, clients, and leases. I need to know at a glance which properties we need to lease out and what clients are looking for.

Database Developer: *What types of things do you need to keep track of?*

Manager: I need to keep track of staff salaries. I need to know how well the properties on our books are being rented out and when leases are coming up for renewal. I also need to keep eye on our expenditure on advertising in newspapers.

Database Developer: *What service does your company provide to your customers?*

Manager: Remember that we have two types of customers; that is, clients wanting to rent property and property owners. We need to make sure that our clients find the property they're looking for quickly without too much legwork and at a reasonable rent, and, of course, that our property owners see good returns from renting out their properties with minimal hassle. As you may already know from speaking to our Director, as well as from developing a new database system, we also intend to develop a new *DreamHome* Web site. This Web site will help our clients view our properties at home before coming into our branches to arrange a viewing. I need to ensure that no matter how clients contacts us—either by email through using our Web site, by phone, or in person—that they receive the same efficient service to help them find the properties that they seek.

Supervisor

Database Developer: *What is your job description?*

Supervisor: My job title is Supervisor. I spend most of my time in the office dealing directly with our customers; that is, clients wanting to rent property and property owners. I'm also responsible for a small group of staff called Assistants and making sure that they are kept busy, but that's not a problem, as there's always plenty to do—it's never-ending actually.

Database Developer: *What kinds of tasks do you perform in a typical day?*

Supervisor: I normally start the day by allocating staff to particular duties, such as dealing with clients or property owners, organizing for clients to view properties, and filing paperwork. When a client finds a suitable property, I process the drawing up of a lease, although the Manager must see the documentation before any signatures are requested. I keep client details up to date and register new clients when they want to join the Company. When a new property is registered, the

Manager allocates responsibility for managing that property to me or one of the other Supervisors or Assistants.

Database Developer: *What kinds of data do you work with?*

Supervisor: I work with data about staff at my branch, property, property owners, clients, property viewings, and leases.

Database Developer: *What types of reports do you use?*

Supervisor: Reports on staff and properties for rent.

Database Developer: *What types of things do you need to keep track of?*

Supervisor: I need to know what properties are available for rent and when currently active leases are due to expire. I also need to know what clients are looking for. I need to keep our Manager up to date with any properties that are proving difficult to rent out. I need to ensure that clients who contact us by email requesting to view properties are given a quick response from us inviting them to call into their nearest *DreamHome* branch office. As part of the service we provide to our property owners, we need to interview all clients first before they are allowed to view our properties. There is nothing unusual about this, as we have always interviewed our clients on their first visit to a *DreamHome* branch, and it's during this time that we note their details and their property requirements.

Assistant

Database Developer: *What is your job description?*

Assistant: My job title is Assistant. I deal directly with our clients.

Database Developer: *What kinds of tasks do you perform in a typical day?*

Assistant: I answer general queries from clients about properties for rent. You know what I mean: "Do you have such and such type of property in a particular area of Glasgow?" I also register new clients and arrange for clients to view properties. When we're not too busy, I file paperwork, but I hate this part of the job—it's so boring.

Database Developer: *What kinds of data do you work with?*

Assistant: I work with data on property and property viewings by clients and sometimes leases.

Database Developer: *What types of reports do you use?*

Assistant: Lists of properties available for rent. These lists are updated every week.

Database Developer: *What types of things do you need to keep track of?*

Assistant: Whether certain properties are available for renting out and which clients are still actively looking for property.

To maintain (enter, update, and delete) data on branches.
 To maintain (enter, update, and delete) data on staff.
 To maintain (enter, update, and delete) data on properties for rent.
 To maintain (enter, update, and delete) data on property owners.
 To maintain (enter, update, and delete) data on clients.
 To maintain (enter, update, and delete) data on property viewings.
 To maintain (enter, update, and delete) data on leases.
 To maintain (enter, update, and delete) data on newspaper adverts.

To perform searches on branches.
 To perform searches on staff.
 To perform searches on properties for rent.
 To perform searches on property owners.
 To perform searches on clients.
 To perform searches on property viewings.
 To perform searches on leases.
 To perform searches on newspaper adverts.

To track the status of property for rent.
 To track the status of clients wishing to rent.
 To track the status of leases.

To report on branches.
 To report on staff.
 To report on properties for rent.
 To report on property owners.
 To report on clients.
 To report on property viewings.
 To report on leases.
 To report on newspaper adverts.

Figure 11.9 Mission objectives for the *DreamHome* database system.

Database Developer: *What service does your company provide to your customers?*

Assistant: We try to answer questions about properties available for rent such as: “Do you have a two-bedroom flat in Hyndland, Glasgow?” and “What should I expect to pay for a one-bedroom flat in the city center?”

Responses to these types of questions should help to formulate the mission objectives. An example of the mission objectives for the *DreamHome* database system is shown in Figure 11.9.



11.4.3 The *DreamHome* Case Study—System Definition

The purpose of the system definition stage is to define the scope and boundary of the database system and its major user views. In Section 10.4.1 we described how a user view represents the requirements that should be supported by a database system as defined by a particular job role (such as Director or Supervisor) or business application area (such as property rentals or property sales).

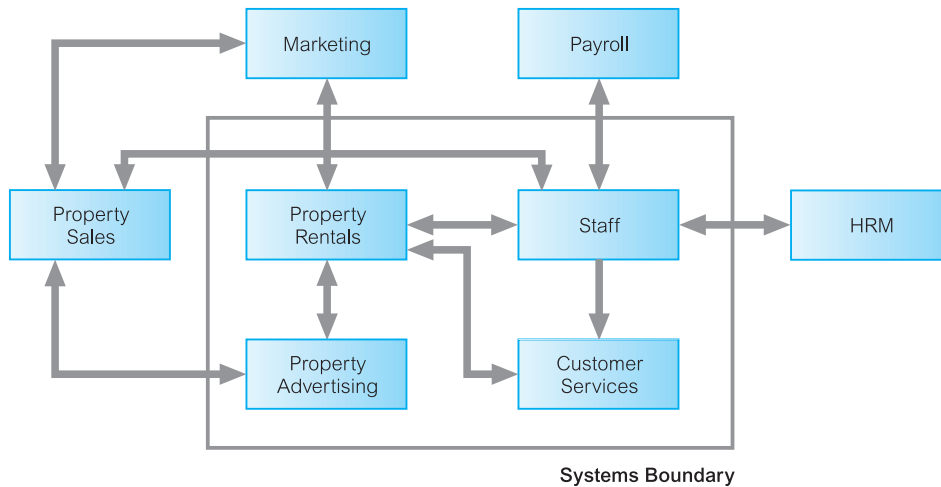


Figure 11.10 Systems boundary for the *DreamHome* database system.

Defining the systems boundary for the *DreamHome* database system

During this stage of the database system development lifecycle, further interviews with users can be used to clarify or expand on data captured in the previous stage. However, additional fact-finding techniques can also be used, including examining the sample documentation shown in Section 11.4.1. The data collected so far is analyzed to define the boundary of the database system. The systems boundary for the *DreamHome* database system is shown in Figure 11.10.

Identifying the major user views for the *DreamHome* database system

We now analyze the data collected so far to define the main user views of the database system. The majority of data about the user views was collected during interviews with the Director and members of staff in the role of Manager, Supervisor, and Assistant. The main user views for the *DreamHome* database system are shown in Figure 11.11.

11.4.4 The *DreamHome* Case Study—Requirements Collection and Analysis



During this stage, we continue to gather more details on the user views identified in the previous stage, to create a **users' requirements specification** that describes in detail the data to be held in the database and how the data is to be used. While gathering more information on the user views, we also collect any general requirements for the system. The purpose of gathering this information is to create a **systems specification**, which describes any features to be included in the new database system, such as networking and shared access requirements, performance requirements, and the levels of security required.

As we collect and analyze the requirements for the new system, we also learn about the most useful and most troublesome features of the current system. When

Data	Access Type	Director	Manager	Supervisor	Assistant	Client
All Branches	Maintain					
	Query	X	X			
	Report	X	X			
Single Branch	Maintain		X			
	Query		X			
	Report		X			
All Staff	Maintain					
	Query	X	X			
	Report	X	X			
Branch Staff	Maintain		X			
	Query		X	X		
	Report		X	X		
All Property	Maintain					
	Query	X				X
	Report	X	X			X
Branch Property	Maintain		X	X		
	Query		X	X	X	
	Report		X	X	X	
All Owners	Maintain					
	Query	X				
	Report	X	X			
Branch Owners	Maintain		X	X		
	Query		X	X	X	
	Report		X			
All Clients	Maintain					X
	Query	X				X
	Report	X	X			
Branch Clients	Maintain		X	X		
	Query		X	X	X	
	Report		X			
All Viewings	Maintain					
	Query					
	Report					
Branch Viewings	Maintain			X	X	
	Query			X	X	
	Report			X	X	
All Leases	Maintain					
	Query	X				
	Report	X	X			
Branch Leases	Maintain		X	X		
	Query		X	X	X	
	Report		X	X		
All Newspapers	Maintain					
	Query	X				
	Report	X	X			
Branch Newspapers	Maintain		X			
	Query		X			
	Report		X			

Figure 11.11 Major user views for the *DreamHome* database system.

building a new database system, it is sensible to try to retain the good things about the old system while introducing the benefits that will be part of using the new system.

An important activity associated with this stage is deciding how to deal with situations in which there are more than one user view. As we discussed in Section 10.6, there are three major approaches to dealing with multiple user views: the **centralized** approach, the **view integration** approach, and a combination of both approaches. We discuss how these approaches can be used shortly.

Gathering more information on the user views of the *DreamHome* database system

To find out more about the requirements for each user view, we may again use a selection of fact-finding techniques, including interviews and observing the business in operation. Examples of the types of questions that we may ask about the data (represented as X) required by a user view include:

“What type of data do you need to hold on X?”

“What sorts of things do you do with the data on X?”

For example, we might ask a Manager the following questions:

Database Developer: *What type of data do you need to hold on staff?*

Manager: The types of data held on a member of staff is his or her full name, position, gender, date of birth, and salary.

Database Developer: *What sorts of things do you do with the data on staff?*

Manager: I need to be able to enter the details of new members of staff and delete their details when they leave. I need to keep the details of staff up to date and print reports that list the full name, position, and salary of each member of staff at my branch. I need to be able to allocate staff to Supervisors. Sometimes when I need to communicate with other branches, I need to find out the names and telephone numbers of Managers at other branches.

We need to ask similar questions about all the important data to be stored in the database. Responses to these questions will help identify the necessary details for the users' requirements specification.

Gathering information on the system requirements of the *DreamHome* database system

While conducting interviews about user views, we should also collect more general information on the system requirements. Examples of the types of questions that we might ask about the system include:

“What transactions run frequently on the database?”

“What transactions are critical to the operation of the organization?”

“When do the critical transactions run?”

“When are the low, normal, and high workload periods for the critical transactions?”

“What type of security do you want for the database system?”

“Is there any highly sensitive data that should be accessed only by certain members of staff?”

“What historical data do you want to hold?”

“What are the networking and shared access requirements for the database system?”

“What type of protection from failures or data loss do you want for the database system?”

For example, we might ask a Manager the following questions:

Database Developer: *What transactions run frequently on the database?*

Manager: We frequently get requests either by phone or by clients who call into our branch to search for a particular type of property in a particular area of the city and for a rent no higher than a particular amount. We hope that clients using the new *DreamHome* Web site will be able to view our properties at any time of the day or night. We also need up-to-date information on properties and clients so that reports can be run off that show properties currently available for rent and clients currently seeking property.

Database Developer: *What transactions are critical to the operation of the business?*

Manager: Again, critical transactions include being able to search for particular properties and to print out reports with up-to-date lists of properties available for rent. Our clients would go elsewhere if we couldn't provide this basic service.

Database Developer: *When do the critical transactions run?*

Manager: Every day.

Database Developer: *When are the low, normal, and high workload periods for the critical transactions?*

Manager: We're open six days a week. In general, we tend to be quiet in the mornings and get busier as the day progresses. However, the busiest time-slots each day for dealing with customers are between 12 and 2pm and 5 and 7pm. We hope that clients using the new *DreamHome* Web site will be able to search through our properties on their own PCs; this should cut down on the number of property queries that staff have to deal with.

We might ask the Director the following questions:

Database Developer: *What type of security do you want for the database system?*

Director: I don't suppose a database holding information for a property rental company holds very sensitive data, but I wouldn't want any of our competitors to see the data

on properties, owners, clients, and leases. Staff should see only the data necessary to do their job in a form that suits what they're doing. For example, although it's necessary for Supervisors and Assistants to see client details, client records should be displayed only one at a time and not as a report. As far as clients using the new *DreamHome* Web site are concerned, we want them to have access to our properties and their own details—but nothing else.

Database Developer: *Is there any highly sensitive data that should be accessed only by certain members of staff?*

Director: As I said before, staff should see only the data necessary to do their jobs. For example, although Supervisors need to see data on staff, salary details should not be included.

Database Developer: *What historical data do you want to hold?*

Director: I want to keep the details of clients and owners for a couple of years after their last dealings with us, so that we can mail them our latest offers, and generally try to attract them back. I also want to be able to keep lease information for a couple of years, so that we can analyze it to find out which types of properties and areas of each city are the most popular for the property rental market, and so on.

Database Developer: *What are the networking and shared access requirements for the database system?*

Director: I want all the branches networked to our main branch office here in Glasgow so that staff can access the system from wherever and whenever they need to. At most branches, I would expect about two or three staff to be accessing the system at any one time, but remember that we have about 100 branches. Most of the time the staff should be just accessing local branch data. However, I don't really want there to be any restrictions about how often or when the system can be accessed, unless it's got real financial implications. As I said earlier, clients using the new *DreamHome* Web site should have access to our properties and their own details, but nothing else.

Database Developer: *What type of protection from failures or data loss do you want for the database system?*

Director: The best, of course. All our business is going to be conducted using the database, so if it goes down, we're sunk. To be serious for a minute, I think we probably have to back up our data every evening when the branch closes. What do you think?

We need to ask similar questions about all the important aspects of the system. Responses to these questions should help identify the necessary details for the system requirements specification.

Managing the user views of the DreamHome database system

How do we decide whether to use the centralized or view integration approach, or a combination of both to manage multiple user views? One way to help make a decision is to examine the overlap in the data used between the user views identified during the system definition stage. Table 11.7 cross-references the Director, Manager, Supervisor, Assistant, and Client user views with the main types of data used by each user view.

We see from Table 11.7 that there is overlap in the data used by all user views. However, the Director and Manager user views and the Supervisor and Assistant user views show more similarities in terms of data requirements. For example, only the Director and Manager user views require data on branches and newspapers, whereas only the Supervisor and Assistant user views require data on property viewings. The Client user view requires access to the least amount of data, and that is only the property and client data. Based on this analysis, we use the *centralized* approach to first merge the requirements for the Director and Manager user views (given the collective name of **Branch** user views) and the requirements for the Supervisor, Assistant, and Client user views (given the collective name of **StaffClient** user views). We then develop data models representing the Branch and StaffClient user views and then use the *view integration* approach to merge the two data models.

Of course, for a simple case study like *DreamHome*, we could easily use the centralized approach for all user views, but we will stay with our decision to create two collective user views so that we can describe and demonstrate how the view integration approach works in practice in Chapter 17.

It is difficult to give precise rules as to when it is appropriate to use the centralized or view integration approaches. The decision should be based on an assessment of the complexity of the database system and the degree of overlap between the various user views. However, whether we use the centralized or view integration approach or a mixture of both to build the underlying database, ultimately we need to re-establish

TABLE 11.7 Cross-reference of user views with the main types of data used by each.

	DIRECTOR	MANAGER	SUPERVISOR	ASSISTANT	CLIENT
branch	X	X			
staff	X	X	X		
property for rent	X	X	X	X	X
owner	X	X	X	X	
client	X	X	X	X	X
property viewing			X	X	
lease	X	X	X	X	
newspaper	X	X			

the original user views (namely Director, Manager, Supervisor, Assistant, and Client) for the working database system. We describe and demonstrate the establishment of the user views for the database system in Chapter 18.

All of the information gathered so far on each user view of the database system is described in a document called a **users' requirements specification**. The users' requirements specification describes the data requirements for each user view and examples of how the data is used by the user view. For ease of reference, the users' requirements specifications for the Branch and StaffClient user views of the *DreamHome* database system are given in Appendix A. In the remainder of this chapter, we present the general systems requirements for the *DreamHome* database system.

The systems specification for the *DreamHome* database system

The systems specification should list all the important features for the *DreamHome* database system. The types of features that should be described in the systems specification include:

- initial database size;
- database rate of growth;
- the types and average number of record searches;
- networking and shared access requirements;
- performance;
- security;
- backup and recovery;
- legal issues.

Systems Requirements for *DreamHome* Database System

Initial database size

- (1) There are approximately 2000 members of staff working at over 100 branches. There is an average of 20 and a maximum of 40 members of staff at each branch.
- (2) There are approximately 100,000 properties available at all branches. There is an average of 1000 and a maximum of 3000 properties at each branch.
- (3) There are approximately 60,000 property owners. There is an average of 600 and a maximum of 1000 property owners at each branch.
- (4) There are approximately 100,000 clients registered across all branches. There is an average of 1000 and a maximum of 1500 clients registered at each branch.
- (5) There are approximately 4,000,000 viewings across all branches. There is an average of 40,000 and a maximum of 100,000 viewings at each branch.
- (6) There are approximately 400,000 leases across all branches. There are an average of 4000 and a maximum of 10,000 leases at each branch.
- (7) There are approximately 50,000 newspaper ads in 100 newspapers across all branches.

Database rate of growth

- (1) Approximately 500 new properties and 200 new property owners will be added to the database each month.

- (2) Once a property is no longer available for rent, the corresponding record will be deleted from the database. Approximately 100 records of properties will be deleted each month.
- (3) If a property owner does not provide properties for rent at any time within a period of two years, his or her record will be deleted. Approximately 100 property owner records will be deleted each month.
- (4) Approximately 20 members of staff join and leave the company each month. The records of staff who have left the company will be deleted after one year. Approximately 20 staff records will be deleted each month.
- (5) Approximately 1000 new clients register at branches each month. If a client does not view or rent out a property at any time within a period of two years, his or her record will be deleted. Approximately 100 client records will be deleted each month.
- (6) Approximately 5000 new viewings are recorded across all branches each day. The details of property viewings will be deleted one year after the creation of the record.
- (7) Approximately 1000 new leases will be recorded across all branches each month. The details of property leases will be deleted two years after the creation of the record.
- (8) Approximately 1000 newspaper adverts are placed each week. The details of newspaper adverts will be deleted one year after the creation of the record.

The types and average number of record searches

- (1) Searching for the details of a branch—approximately 10 per day.
- (2) Searching for the details of a member of staff at a branch—approximately 20 per day.
- (3) Searching for the details of a given property—approximately 5000 per day (Monday to Thursday), and approximately 10,000 per day (Friday and Saturday). Peak workloads are 12.00–14.00 and 17.00–19.00 daily. (The workloads for property searches should be reassessed after the *DreamHome* Web site is launched.)
- (4) Searching for the details of a property owner—approximately 100 per day.
- (5) Searching for the details of a client—approximately 1000 per day (Monday to Thursday), and approximately 2000 per day (Friday and Saturday). Peak workloads are 12.00–14.00 and 17.00–19.00 daily.
- (6) Searching for the details of a property viewing—approximately 2000 per day (Monday to Thursday), and approximately 5000 per day (Friday and Saturday). Peak workloads are 12.00–14.00 and 17.00–19.00 daily.
- (7) Searching for the details of a lease—approximately 1000 per day (Monday to Thursday), and approximately 2000 per day (Friday and Saturday). Peak workloads are 12.00–14.00 and 17.00–19.00 daily.

Networking and shared access requirements

All branches should be securely networked to a centralized database located at *DreamHome*'s main office in Glasgow. The system should allow for at least two to

three people concurrently accessing the system from each branch. Consideration needs to be given to the licensing requirements for this number of concurrent accesses.

Performance

- (1) During opening hours, but not during peak periods, expect less than a 1-second response for all single record searches. During peak periods, expect less than a 5-second response for each search.
- (2) During opening hours, but not during peak periods, expect less than a 5-second response for each multiple record search. During peak periods, expect less than a 10-second response for each multiple record search.
- (3) During opening hours, but not during peak periods, expect less than a 1-second response for each update/save. During peak periods, expect less than a 5-second response for each update/save.

Security

- (1) The database should be password-protected.
- (2) Each member of staff should be assigned database access privileges appropriate to a particular user view, namely Director, Manager, Supervisor, or Assistant.
- (3) A member of staff should see only the data necessary to do his or her job in a form that suits what he or she is doing.
- (4) A client should see only property data and their own personal details using the *DreamHome* Web site.

Backup and Recovery

The database should be backed up daily at 12 midnight.

Legal Issues

Each country has laws that govern the way that the computerized storage of personal data is handled. As the *DreamHome* database holds data on staff, clients, and property owners, any legal issues that must be complied with should be investigated and implemented. The professional, legal, and ethical issues associated with data management are discussed in Chapter 21.

11.4.5 The *DreamHome* Case Study—Database Design

In this chapter we demonstrated the creation of the users' requirements specification for the Branch and Staff user views and the systems specification for the *DreamHome* database system. These documents are the sources of information for the next stage of the lifecycle called **database design**. In Chapters 16 to 19 we provide a step-by-step methodology for database design and use the *DreamHome* case study and the documents created for the *DreamHome* database system in this chapter to demonstrate the methodology in practice.



Chapter Summary

- **Fact-finding** is the formal process of using techniques such as interviews and questionnaires to collect facts about systems, requirements, and preferences.
- Fact-finding is particularly crucial to the early stages of the database system development lifecycle, including the database planning, system definition, and requirements collection and analysis stages.
- The five most common fact-finding techniques are examining documentation, interviewing, observing the enterprise in operation, conducting research, and using questionnaires.
- There are two main documents created during the requirements collection and analysis stage: the **users' requirements specification** and the **systems specification**.
- The **users' requirements specification** describes in detail the data to be held in the database and how the data is to be used.
- The **systems specification** describes any features to be included in the database system, such as the performance and security requirements.

Review Questions

- 11.1 Briefly discuss the objectives of interviews in a database development project.
- 11.2 Describe how fact-finding is used throughout the stages of the database system development lifecycle.
- 11.3 For each stage of the database system development lifecycle identify examples of the facts captured and the documentation produced.
- 11.4 A database developer normally uses several fact-finding techniques during a single database project. The five most commonly used techniques are examining documentation, interviewing, observing the business in operation, conducting research, and using questionnaires. Describe each fact-finding technique and identify the advantages and disadvantages of each.
- 11.5 What are the dangers of not defining mission objectives and a mission statement for a database system?
- 11.6 What is the purpose of identifying the systems boundary for a database system?
- 11.7 How do the contents of a users' requirements specification differ from a systems specification?
- 11.8 Database development and fact finding are inseparable. One factor for the success of the development processes is user involvement. What is the role played by users in the database development process?

Exercises

- 11.9 Assume that your friend is currently employed by a multinational consultancy company that deals with database analysis and development in Tanzania. His first assignment is to carry out a fact-finding mission for a client that intends to develop a database system for their company to control their daily business transactions. Your friend has decided to ask you for advice.

Task: Prepare a brief note that would help your friend successfully choose the best fact-finding technique. For each technique, outline the factors crucial for realizing quality facts. The note should also detail issues to be avoided or taken care of for each tool to succeed.

The client for this exercise and those that follow can be any one of the fictitious case studies given in Appendix B or some real company identified by you or your professor.

- 11.10** Assume that you are an employee of a consultancy company that specializes in the analysis, design, and implementation of database systems. A client has recently approached your company with a view to implementing a database system.

Task: You are required to establish the database project through the early stages of the project. With this task in mind, create a mission statement and mission objectives and high-level systems diagram for the client's database system.

- 11.11** Assume that you are an employee of a consultancy company that specializes in the analysis, design, and implementation of database systems. A client has recently approached your company with a view to implementing a database system. It has already been established that the client's database system will support many different groups of users (user views).

Task: You are required to identify how to best manage the requirements for these user views. With this task in mind, create a report that identifies high-level requirements for each user view and shows the relationship between the user views. Conclude the report by identifying and justifying the best approach to managing the multi-user view requirements.

Chapter Objectives

In this chapter you will learn:

- How to use Entity–Relationship (ER) modeling in database design.
- The basic concepts associated with the ER model: entities, relationships, and attributes.
- A diagrammatic technique for displaying an ER model using the Unified Modeling Language (UML).
- How to identify and resolve problems with ER models called connection traps.

In Chapter 11 we described the main techniques for gathering and capturing information about what the users require of a database system. Once the requirements collection and analysis stage of the database system development lifecycle is complete and we have documented the requirements for the database system, we are ready to begin the database design stage.

One of the most difficult aspects of database design is the fact that designers, programmers, and end-users tend to view data and its use in different ways. Unfortunately, unless we gain a common understanding that reflects how the enterprise operates, the design we produce will fail to meet the users' requirements. To ensure that we get a precise understanding of the nature of the data and how it is used by the enterprise, we need a model for communication that is nontechnical and free of ambiguities. The Entity–Relationship (ER) model is one such example. ER modeling is a top-down approach to database design that begins by identifying the important data called *entities* and *relationships* between the data that must be represented in the model. We then add more details, such as the information we want to hold about the entities and relationships called *attributes* and any *constraints* on the entities, relationships, and attributes. ER modeling is an important technique for any database designer to master and forms the basis of the methodology presented in this book.

In this chapter we introduce the basic concepts of the ER model. Although there is general agreement about what each concept means, there are a number of different notations that can be used to represent each concept diagrammatically. We have chosen a diagrammatic notation that uses an increasingly popular object-oriented modeling language called the **Unified Modeling Language (UML)** (Booch *et al.*,

1999). UML is the successor to a number of object-oriented analysis and design methods introduced in the 1980s and 1990s. The Object Management Group (OMG) is responsible for the creation and management of UML (available at www.uml.org). UML is currently recognized as the de facto industry standard modeling language for object-oriented software engineering projects. Although we use the UML notation for drawing ER models, we continue to describe the concepts of ER models using traditional database terminology. In Section 27.8 we will provide a fuller discussion on UML. We also include a summary of two alternative diagrammatic notations for ER models in Appendix C.

In the next chapter we discuss the inherent problems associated with representing complex database applications using the basic concepts of the ER model. To overcome these problems, additional “semantic” concepts were added to the original ER model, resulting in the development of the Enhanced Entity–Relationship (EER) model. In Chapter 13 we describe the main concepts associated with the EER model, called specialization/generalization, aggregation, and composition. We also demonstrate how to convert the ER model shown in Figure 12.1 into the EER model shown in Figure 13.8.

Structure of this Chapter In Sections 12.1, 12.2, and 12.3 we introduce the basic concepts of the Entity–Relationship model: entities, relationships, and attributes. In each section we illustrate how the basic ER concepts are represented pictorially in an ER diagram using UML. In Section 12.4 we differentiate between weak and strong entities and in Section 12.5 we discuss how attributes normally associated with entities can be assigned to relationships. In Section 12.6 we describe the structural constraints associated with relationships. Finally, in Section 12.7 we identify potential problems associated with the design of an ER model called connection traps and demonstrate how these problems can be resolved.

The ER diagram shown in Figure 12.1 is an example of one of the possible end-products of ER modeling. This model represents the relationships between data described in the requirements specification for the Branch view of the *DreamHome* case study given in Appendix A. This figure is presented at the start of this chapter to show the reader an example of the type of model that we can build using ER modeling. At this stage, the reader should not be concerned about fully understanding this diagram, as the concepts and notation used in this figure are discussed in detail throughout this chapter.



12.1 Entity Types

Entity type

A group of objects with the same properties, which are identified by the enterprise as having an independent existence.

The basic concept of the ER model is the **entity type**, which represents a group of “objects” in the “real world” with the same properties. An entity type has an

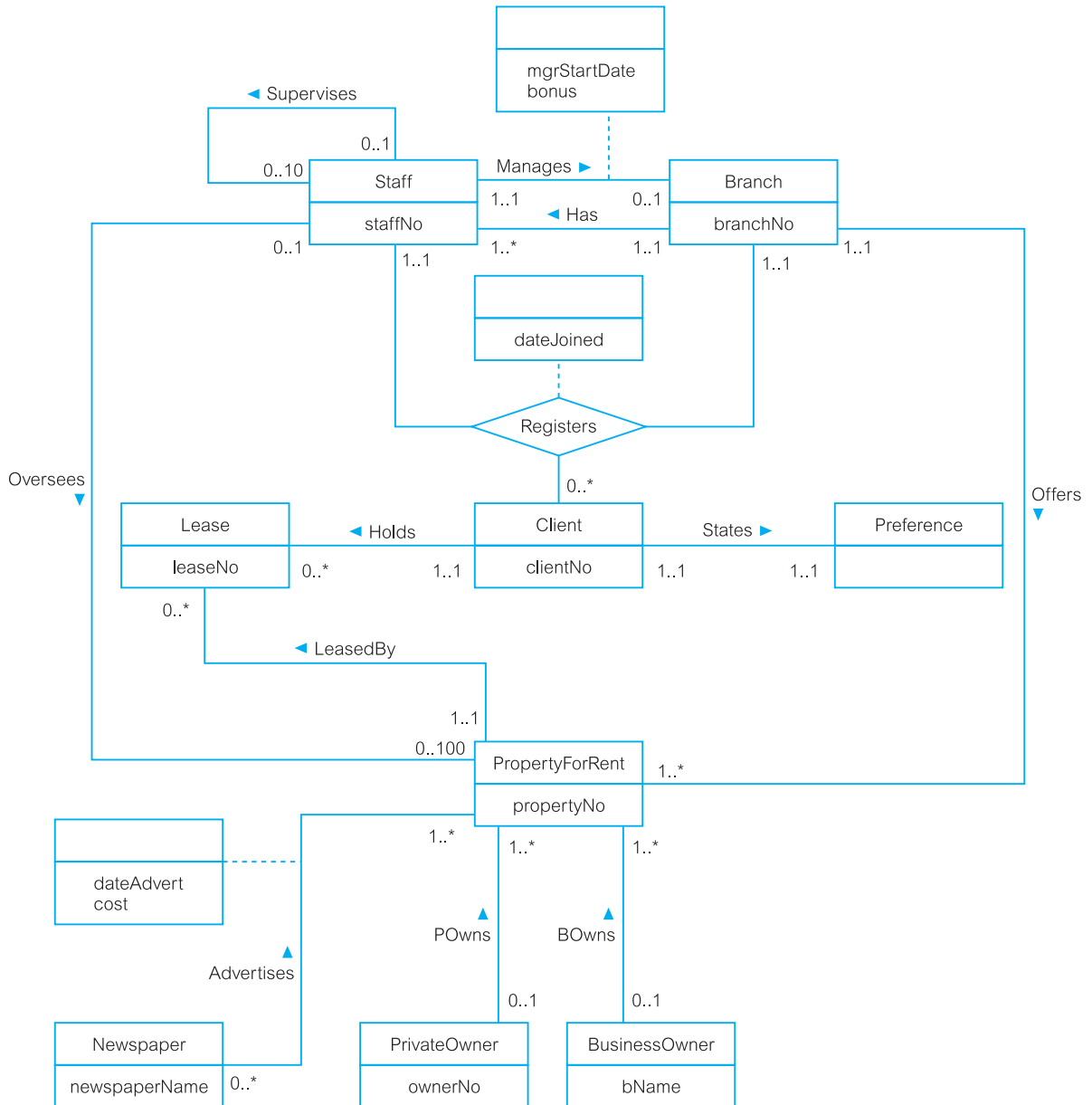


Figure 12.1 An Entity–Relationship (ER) diagram of the Branch view of *DreamHome*.

independent existence and can be objects with a physical (or “real”) existence or objects with a conceptual (or “abstract”) existence, as listed in Figure 12.2. Note that we are able to give only a working definition of an entity type, as no strict formal definition exists. This means that different designers may identify different entities.

Entity occurrence

A uniquely identifiable object of an entity type.

As with entity types and entities, it is necessary to distinguish between the terms “relationship type” and “relationship occurrence.”

Relationship occurrence

A uniquely identifiable association that includes one occurrence from each participating entity type.

A **relationship occurrence** indicates the particular entity occurrences that are related. Throughout this book, we use the terms “relationship type” or “relationship occurrence.” However, as with the term “entity,” we use the more general term “relationship” when the meaning is obvious.

Consider a relationship type called *Has*, which represents an association between Branch and Staff entities, that is Branch *Has* Staff. Each occurrence of the *Has* relationship associates one Branch entity occurrence with one Staff entity occurrence. We can examine examples of individual occurrences of the *Has* relationship using a *semantic net*. A semantic net is an object-level model, which uses the symbol • to represent entities and the symbol ♦ to represent relationships. The semantic net in Figure 12.4 shows three examples of the *Has* relationships (denoted r1, r2, and r3). Each relationship describes an association of a single Branch entity occurrence with a single Staff entity occurrence. Relationships are represented by lines that join each participating Branch entity with the associated Staff entity. For example, relationship r1 represents the association between Branch entity B003 and Staff entity SG37.

Note that we represent each Branch and Staff entity occurrences using values for the primary key attributes, namely *branchNo* and *staffNo*. Primary key attributes uniquely identify each entity occurrence and are discussed in detail in the following section.

If we represented an enterprise using semantic nets, it would be difficult to understand, due to the level of detail. We can more easily represent the relationships between entities in an enterprise using the concepts of the ER model. The ER model uses a higher level of abstraction than the semantic net by combining sets of entity occurrences into entity types and sets of relationship occurrences into relationship types.

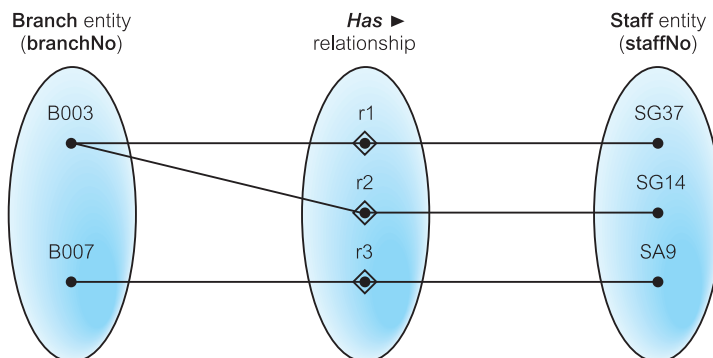
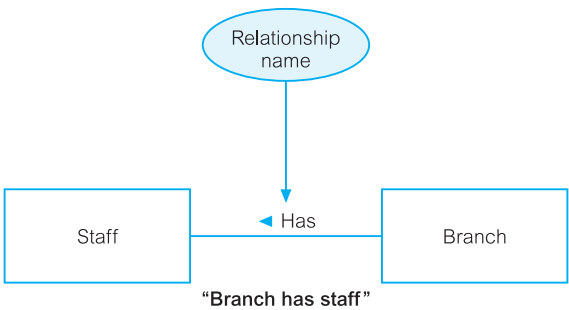


Figure 12.4 A semantic net showing individual occurrences of the *Has* relationship type.

Figure 12.5
A diagrammatic
representation of
Branch *Has* Staff
relationship type.



Diagrammatic representation of relationship types

Each relationship type is shown as a line connecting the associated entity types and labeled with the name of the relationship. Normally, a relationship is named using a verb (for example, *Supervises* or *Manages*) or a short phrase including a verb (for example, *LeasedBy*). Again, the first letter of each word in the relationship name is shown in uppercase. Whenever possible, a relationship name should be unique for a given ER model.

A relationship is only labeled in one direction, which normally means that the name of the relationship only makes sense in one direction (for example, *Branch Has Staff* makes more sense than *Staff Has Branch*). So once the relationship name is chosen, an arrow symbol is placed beside the name indicating the correct direction for a reader to interpret the relationship name (for example, *Branch Has* ► *Staff*) as shown in Figure 12.5.

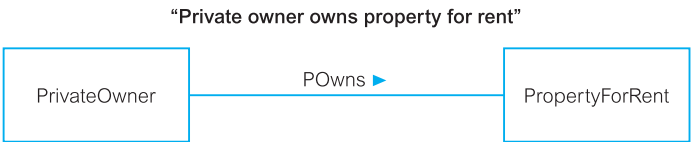
12.2.1 Degree of Relationship Type

Degree of a relationship type

The number of participating entity types in a relationship.

The entities involved in a particular relationship type are referred to as **participants** in that relationship. The number of participants in a relationship type is called the **degree** of that relationship. Therefore, the degree of a relationship indicates the number of entity types involved in a relationship. A relationship of degree two is called **binary**. An example of a binary relationship is the *Has* relationship shown in Figure 12.5 with two participating entity types; namely, *Staff* and *Branch*. A second example of a binary relationship is the *POwns* relationship shown in Figure 12.6 with two participating entity types; namely, *PrivateOwner* and *PropertyForRent*. The *Has* and *POwns* relationships are also shown in Figure 12.1 as well as other examples of

Figure 12.6
An example
of a binary
relationship called
POwns.



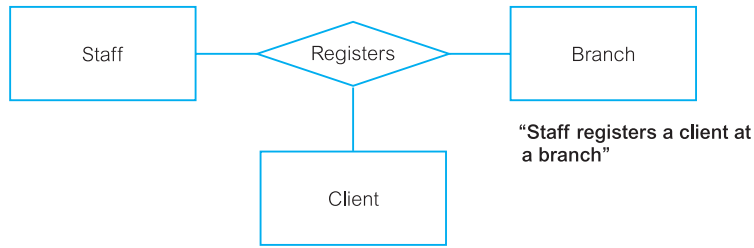


Figure 12.7 An example of a ternary relationship called *Registers*.

binary relationships. In fact, the most common degree for a relationship is binary, as demonstrated in this figure.

A relationship of degree three is called **ternary**. An example of a ternary relationship is *Registers* with three participating entity types: *Staff*, *Branch*, and *Client*. This relationship represents the registration of a client by a member of staff at a branch. The term “complex relationship” is used to describe relationships with degrees higher than binary.

Diagrammatic representation of complex relationships

The UML notation uses a diamond to represent relationships with degrees higher than binary. The name of the relationship is displayed inside the diamond, and in this case, the directional arrow normally associated with the name is omitted. For example, the ternary relationship called *Registers* is shown in Figure 12.7. This relationship is also shown in Figure 12.1.

A relationship of degree four is called **quaternary**. As we do not have an example of such a relationship in Figure 12.1, we describe a quaternary relationship called *Arranges* with four participating entity types—namely, *Buyer*, *Solicitor*, *FinancialInstitution*, and *Bid*—in Figure 12.8. This relationship represents the situation where a buyer, advised by a solicitor and supported by a financial institution, places a bid.

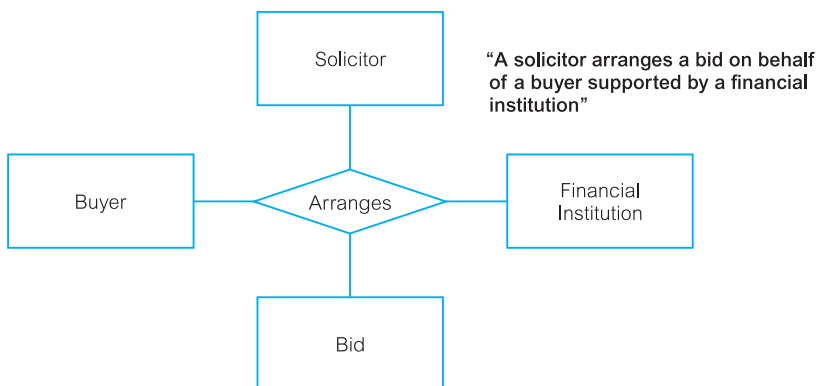
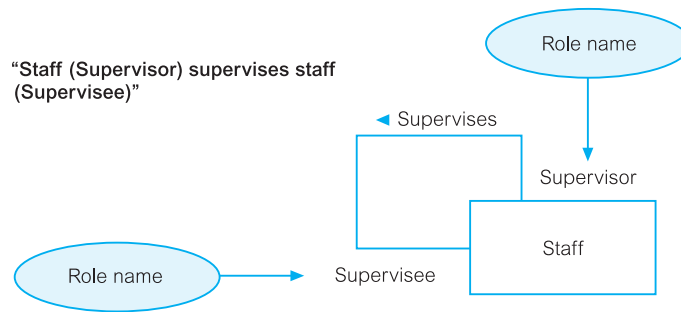


Figure 12.8 An example of a quaternary relationship called *Arranges*.

Figure 12.9

An example of a recursive relationship called *Supervises* with role names Supervisor and Supervisee.



12.2.2 Recursive Relationship

Recursive relationship

A relationship type in which the *same* entity type participates more than once in *different roles*.

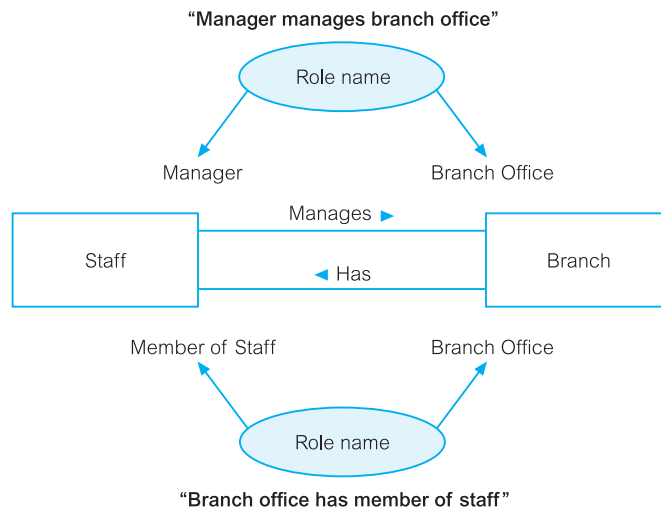
Consider a recursive relationship called *Supervises*, which represents an association of staff with a Supervisor where the Supervisor is also a member of staff. In other words, the *Staff* entity type participates twice in the *Supervises* relationship; the first participation as a Supervisor, and the second participation as a member of staff who is supervised (Supervisee). Recursive relationships are sometimes called *unary* relationships.

Relationships may be given **role names** to indicate the purpose that each participating entity type plays in a relationship. Role names can be important for recursive relationships to determine the function of each participant. The use of role names to describe the *Supervises* recursive relationship is shown in Figure 12.9. The first participation of the *Staff* entity type in the *Supervises* relationship is given the role name “Supervisor” and the second participation is given the role name “Supervisee.”

Role names may also be used when two entities are associated through more than one relationship. For example, the *Staff* and *Branch* entity types are associated through two distinct relationships called *Manages* and *Has*. As shown in Figure 12.10,

Figure 12.10

An example of entities associated through two distinct relationships called *Manages* and *Has* with role names.



the use of role names clarifies the purpose of each relationship. For example, in the case of *Staff Manages Branch*, a member of staff (Staff entity) given the role name “Manager” manages a branch (Branch entity) given the role name “Branch Office.” Similarly, for *Branch Has Staff*, a branch, given the role name “Branch Office” has staff given the role name “Member of Staff.”

Role names are usually not required if the function of the participating entities in a relationship is unambiguous.

12.3 Attributes

Attribute

A property of an entity or a relationship type.

The particular properties of entity types are called attributes. For example, a Staff entity type may be described by the *staffNo*, *name*, *position*, and *salary* attributes. The attributes hold values that describe each entity occurrence and represent the main part of the data stored in the database.

A relationship type that associates entities can also have attributes similar to those of an entity type, but we defer discussion of this until Section 12.5. In this section, we concentrate on the general characteristics of attributes.

Attribute domain

The set of allowable values for one or more attributes.

Each attribute is associated with a set of values called a **domain**. The domain defines the potential values that an attribute may hold and is similar to the domain concept in the relational model (see Section 4.2). For example, the number of rooms associated with a property is between 1 and 15 for each entity occurrence. We therefore define the set of values for the number of rooms (*rooms*) attribute of the *PropertyForRent* entity type as the set of integers between 1 and 15.

Attributes may share a domain. For example, the address attributes of the *Branch*, *PrivateOwner*, and *BusinessOwner* entity types share the same domain of all possible addresses. Domains can also be composed of domains. For example, the domain for the address attribute of the *Branch* entity is made up of subdomains: *street*, *city*, and *postcode*.

The domain of the name attribute is more difficult to define, as it consists of all possible names. It is certainly a character string, but it might consist not only of letters but also hyphens or other special characters. A fully developed data model includes the domains of each attribute in the ER model.

As we now explain, attributes can be classified as being: *simple* or *composite*, *single-valued* or *multi-valued*, or *derived*.

12.3.1 Simple and Composite Attributes

Simple attribute

An attribute composed of a single component with an independent existence.

Simple attributes cannot be further subdivided into smaller components. Examples of simple attributes include `position` and `salary` of the `Staff` entity. Simple attributes are sometimes called *atomic* attributes.

Composite attribute

An attribute composed of multiple components, each with an independent existence.

Some attributes can be further divided to yield smaller components with an independent existence of their own. For example, the `address` attribute of the `Branch` entity with the value (163 Main St, Glasgow, G11 9QX) can be subdivided into `street` (163 Main St), `city` (Glasgow), and `postcode` (G11 9QX) attributes.

The decision to model the `address` attribute as a simple attribute or to subdivide the attribute into `street`, `city`, and `postcode` is dependent on whether the user view of the data refers to the `address` attribute as a single unit or as individual components.

12.3.2 Single-valued and Multi-valued Attributes

Single-valued attribute

An attribute that holds a single value for each occurrence of an entity type.

The majority of attributes are single-valued. For example, each occurrence of the `Branch` entity type has a single value for the branch number (`branchNo`) attribute (for example, B003), and therefore the `branchNo` attribute is referred to as being single-valued.

Multi-valued attribute

An attribute that holds multiple values for each occurrence of an entity type.

Some attributes have multiple values for each entity occurrence. For example, each occurrence of the `Branch` entity type can have multiple values for the `telNo` attribute (for example, branch number B003 has telephone numbers 0141-339-2178 and 0141-339-4439) and therefore the `telNo` attribute in this case is multi-valued. A multi-valued attribute may have a set of numbers with upper and lower limits. For example, the `telNo` attribute of the `Branch` entity type has between one and three values. In other words, a branch may have a minimum of a single telephone number to a maximum of three telephone numbers.

12.3.3 Derived Attributes

Derived attribute

An attribute that represents a value that is derivable from the value of a related attribute or set of attributes, not necessarily in the same entity type.

The values held by some attributes may be derived. For example, the value for the duration attribute of the **Lease** entity is calculated from the **rentStart** and **rentFinish** attributes, also of the **Lease** entity type. We refer to the duration attribute as a derived attribute, the value of which is derived from the **rentStart** and **rentFinish** attributes.

In some cases, the value of an attribute is derived from the entity occurrences in the same entity type. For example, the total number of staff (**totalStaff**) attribute of the **Staff** entity type can be calculated by counting the total number of **Staff** entity occurrences.

Derived attributes may also involve the association of attributes of different entity types. For example, consider an attribute called **deposit** of the **Lease** entity type. The value of the **deposit** attribute is calculated as twice the monthly rent for a property. Therefore, the value of the **deposit** attribute of the **Lease** entity type is derived from the **rent** attribute of the **PropertyForRent** entity type.

12.3.4 Keys

Candidate key

The minimal set of attributes that uniquely identifies each occurrence of an entity type.

A candidate key is the minimal number of attributes, whose value(s) uniquely identify each entity occurrence. For example, the branch number (**branchNo**) attribute is the candidate key for the **Branch** entity type, and has a distinct value for each branch entity occurrence. The candidate key must hold values that are unique for every occurrence of an entity type. This implies that a candidate key cannot contain a null (see Section 4.2). For example, each branch has a unique branch number (for example, B003), and there will never be more than one branch with the same branch number.

Primary key

The candidate key that is selected to uniquely identify each occurrence of an entity type.

An entity type may have more than one candidate key. For the purposes of discussion, consider that a member of staff has a unique company-defined staff number (**staffNo**) and also a unique National Insurance Number (NIN) that is used by the government. We therefore have two candidate keys for the **Staff** entity, one of which must be selected as the primary key.

The choice of primary key for an entity is based on considerations of attribute length, the minimal number of attributes required, and the future certainty of uniqueness. For example, the company-defined staff number contains a maximum of five characters (for example, SG14), and the NIN contains a maximum of nine characters (for example, WL220658D). Therefore, we select **staffNo** as the primary key of the **Staff** entity type and **NIN** is then referred to as the **alternate key**.

Composite key

A candidate key that consists of two or more attributes.

In some cases, the key of an entity type is composed of several attributes whose values together are unique for each entity occurrence but not separately. For example, consider an entity called *Advert* with *propertyNo* (property number), *newspaperName*, *dateAdvert*, and *cost* attributes. Many properties are advertised in many newspapers on a given date. To uniquely identify each occurrence of the *Advert* entity type requires values for the *propertyNo*, *newspaperName*, and *dateAdvert* attributes. Thus, the *Advert* entity type has a composite primary key made up of the *propertyNo*, *newspaperName*, and *dateAdvert* attributes.

Diagrammatic representation of attributes

If an entity type is to be displayed with its attributes, we divide the rectangle representing the entity in two. The upper part of the rectangle displays the name of the entity and the lower part lists the names of the attributes. For example, Figure 12.11 shows the ER diagram for the *Staff* and *Branch* entity types and their associated attributes.

The first attribute(s) to be listed is the primary key for the entity type, if known. The name(s) of the primary key attribute(s) can be labeled with the tag {PK}. In UML, the name of an attribute is displayed with the first letter in lowercase and, if the name has more than one word, with the first letter of each subsequent word in uppercase (for example, *address* and *telNo*). Additional tags that can be used include partial primary key {PPK} when an attribute forms part of a composite primary key, and alternate key {AK}. As shown in Figure 12.11, the primary key of the *Staff* entity type is the *staffNo* attribute and the primary key of the *Branch* entity type is the *branchNo* attribute.

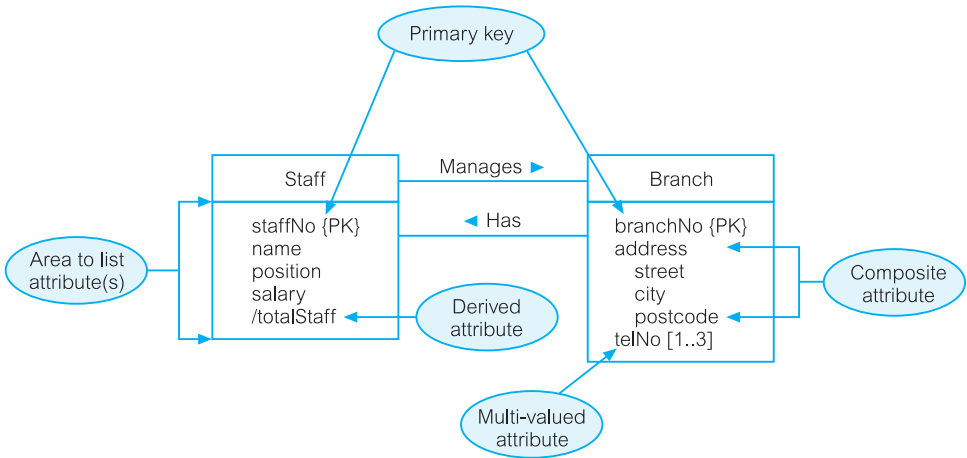


Figure 12.11 Diagrammatic representation of Staff and Branch entities and their attributes.

For some simpler database systems, it is possible to show all the attributes for each entity type in the ER diagram. However, for more complex database systems, we just display the attribute, or attributes, that form the primary key of each entity type. When only the primary key attributes are shown in the ER diagram, we can omit the {PK} tag.

For simple, single-valued attributes, there is no need to use tags, so we simply display the attribute names in a list below the entity name. For composite attributes, we list the name of the composite attribute followed below and indented to the right by the names of its simple component attributes. For example, in Figure 12.11 the composite attribute `address` of the `Branch` entity is shown, followed below by the names of its component attributes, `street`, `city`, and `postcode`. For multivalued attributes, we label the attribute name with an indication of the range of values available for the attribute. For example, if we label the `telNo` attribute with the range `[1..*]`, this means that the values for the `telNo` attribute is one or more. If we know the precise maximum number of values, we can label the attribute with an exact range. For example, if the `telNo` attribute holds one to a maximum of three values, we can label the attribute with `[1..3]`.

For derived attributes, we prefix the attribute name with a `/`. For example, the derived attribute of the `Staff` entity type is shown in Figure 12.11 as `/totalStaff`.

12.4 Strong and Weak Entity Types

We can classify entity types as being strong or weak.

Strong entity type

An entity type that is *not* existence-dependent on some other entity type.

An entity type is referred to as being strong if its existence does not depend upon the existence of another entity type. Examples of strong entities are shown in Figure 12.1 and include the `Staff`, `Branch`, `PropertyForRent`, and `Client` entities. A characteristic of a strong entity type is that each entity occurrence is uniquely identifiable using the primary key attribute(s) of that entity type. For example, we can uniquely identify each member of staff using the `staffNo` attribute, which is the primary key for the `Staff` entity type.

Weak entity type

An entity type that is existence-dependent on some other entity type.

A weak entity type is dependent on the existence of another entity type. An example of a weak entity type called `Preference` is shown in Figure 12.12. A characteristic of a weak entity is that each entity occurrence cannot be uniquely identified using only the attributes associated with that entity type. For example, note that there is no primary key for the `Preference` entity. This means that we cannot identify each occurrence of the `Preference` entity type using only the attributes of this

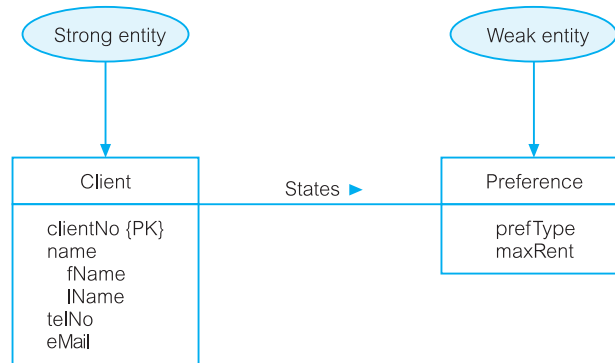


Figure 12.12 A strong entity type called Client and a weak entity type called Preference.

entity. We can uniquely identify each preference only through the relationship that a preference has with a client who is uniquely identifiable using the primary key for the Client entity type, namely clientNo. In this example, the Preference entity is described as having existence dependency for the Client entity, which is referred to as being the owner entity.

Weak entity types are sometimes referred to as *child*, *dependent*, or *subordinate* entities and strong entity types as *parent*, *owner*, or *dominant* entities.

12.5 Attributes on Relationships

As we mentioned in Section 12.3, attributes can also be assigned to relationships. For example, consider the relationship *Advertises*, which associates the *Newspaper* and *PropertyForRent* entity types as shown in Figure 12.1. To record the date the property was advertised and the cost, we associate this information with the *Advertises* relationship as attributes called *dateAdvert* and *cost*, rather than with the *Newspaper* or the *PropertyForRent* entities.

Diagrammatic representation of attributes on relationships

We represent attributes associated with a relationship type using the same symbol as an entity type. However, to distinguish between a relationship with an attribute and an entity, the rectangle representing the attribute(s) is associated with the relationship using a dashed line. For example, Figure 12.13 shows the *Advertises* relationship with the attributes *dateAdvert* and *cost*. A second example shown in Figure 12.1 is the *Manages* relationship with the *mgrStartDate* and *bonus* attributes.

The presence of one or more attributes assigned to a relationship may indicate that the relationship conceals an unidentified entity type. For example, the presence of the `dateAdvert` and `cost` attributes on the *Advertises* relationship indicates the presence of an entity called `Advert`.

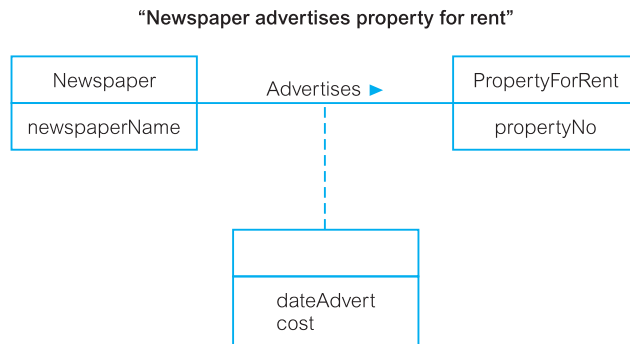


Figure 12.13
An example of a relationship called *Advertises* with attributes *dateAdvert* and *cost*.

12.6 Structural Constraints

We now examine the constraints that may be placed on entity types that participate in a relationship. The constraints should reflect the restrictions on the relationships as perceived in the “real world.” Examples of such constraints include the requirements that a property for rent must have an owner and each branch must have staff. The main type of constraint on relationships is called **multiplicity**.

Multiplicity

The number (or range) of possible occurrences of an entity type that may relate to a single occurrence of an associated entity type through a particular relationship.

Multiplicity constrains the way that entities are related. It is a representation of the policies (or business rules) established by the user or enterprise. Ensuring that all appropriate **constraints** are identified and represented is an important part of modeling an enterprise.

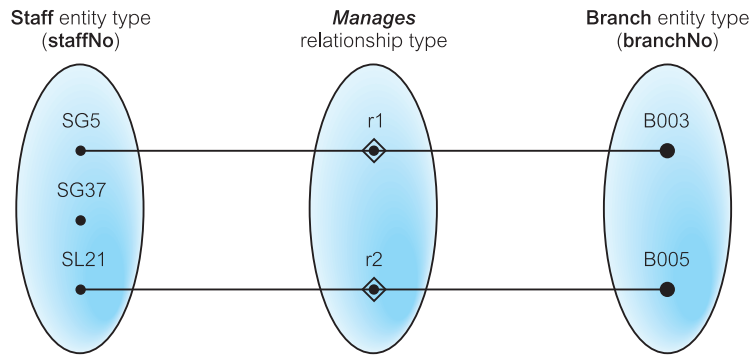
As we mentioned earlier, the most common degree for relationships is binary. Binary relationships are generally referred to as being one-to-one (1:1), one-to-many (1:*), or many-to-many (*:*). We examine these three types of relationships using the following integrity constraints:

- a member of staff manages a branch (1:1);
- a member of staff oversees properties for rent (1:*);
- newspapers advertise properties for rent (*:*)

In Sections 12.6.1, 12.6.2, and 12.6.3 we demonstrate how to determine the multiplicity for each of these constraints and show how to represent each in an ER diagram. In Section 12.6.4 we examine multiplicity for relationships of degrees higher than binary.

It is important to note that not all integrity constraints can be easily represented in an ER model. For example, the requirement that a member of staff receives an additional day's holiday for every year of employment with the enterprise may be difficult to represent in an ER model.

Figure 12.14(a)
Semantic net showing two occurrences of the Staff *Manages* Branch relationship type.



12.6.1 One-to-One (1:1) Relationships

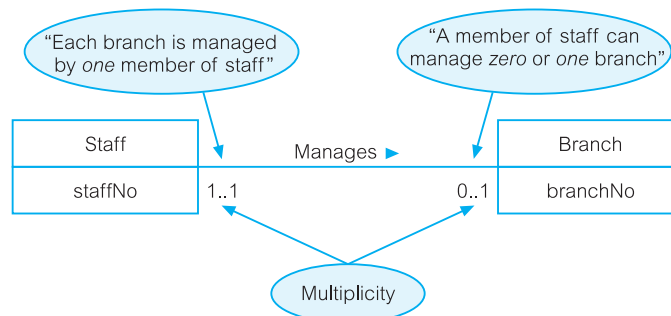
Consider the relationship *Manages*, which relates the Staff and Branch entity types. Figure 12.14(a) displays two occurrences of the *Manages* relationship type (denoted *r1* and *r2*) using a semantic net. Each relationship (*rn*) represents the association between a single Staff entity occurrence and a single Branch entity occurrence. We represent each entity occurrence using the values for the primary key attributes of the Staff and Branch entities, namely *staffNo* and *branchNo*.

Determining the multiplicity

Determining the multiplicity normally requires examining the precise relationships between the data given in a enterprise constraint using sample data. The sample data may be obtained by examining filled-in forms or reports and, if possible, from discussion with users. However, it is important to stress that to reach the right conclusions about a constraint requires that the sample data examined or discussed is a true representation of all the data being modeled.

In Figure 12.14(a) we see that *staffNo* SG5 manages *branchNo* B003 and *staffNo* SL21 manages *branchNo* B005, but *staffNo* SG37 does not manage any branch. In other words, a member of staff can manage *zero or one* branch and each branch is managed by *one* member of staff. As there is a maximum of one branch for each member of staff involved in this relationship and a maximum of one member of staff for each branch, we refer to this type of relationship as *one-to-one*, which we usually abbreviate as (1:1).

Figure 12.14(b)
The multiplicity of the Staff *Manages* Branch one-to-one (1:1) relationship.



Diagrammatic representation of 1:1 relationships

An ER diagram of the *Staff Manages Branch* relationship is shown in Figure 12.14(b). To represent that a member of staff can manage *zero or one* branch, we place a “0..1” beside the Branch entity. To represent that a branch always has *one* manager, we place a “1..1” beside the Staff entity. (Note that for a 1:1 relationship, we may choose a relationship name that makes sense in either direction.)

12.6.2 One-to-Many (1:*) Relationships

Consider the relationship *Oversees*, which relates the Staff and PropertyForRent entity types. Figure 12.15(a) displays three occurrences of the Staff *Oversees* PropertyForRent relationship type (denoted *r1*, *r2*, and *r3*) using a semantic net. Each relationship (*rn*) represents the association between a single Staff entity occurrence and a single PropertyForRent entity occurrence. We represent each entity occurrence using the values for the primary key attributes of the Staff and PropertyForRent entities, namely *staffNo* and *propertyNo*.

Determining the multiplicity

In Figure 12.15(a) we see that *staffNo* SG37 oversees *propertyNos* PG21 and PG36, and *staffNo* SA9 oversees *propertyNo* PA14 but *staffNo* SG5 does not oversee any properties for rent and *propertyNo* PG4 is not overseen by any member of staff. In summary, a member of staff can oversee *zero or more* properties for rent and a property for rent is overseen by *zero or one* member of staff. Therefore, for members of staff participating in this relationship there are *many* properties for rent, and for properties participating in this relationship there is a maximum of *one* member of staff. We refer to this type of relationship as *one-to-many*, which we usually abbreviate as (1:*).

Diagrammatic representation of 1:* relationships An ER diagram of the Staff *Oversees* PropertyForRent relationship is shown in Figure 12.15(b). To represent that a member of staff can oversee *zero or more* properties for rent, we place a “0..*” beside the PropertyForRent entity. To represent that each property for rent is overseen by *zero or one* member of staff, we place a “0..1” beside the Staff entity. (Note that with 1:* relationships, we choose a relationship name that makes sense in the 1:* direction.)

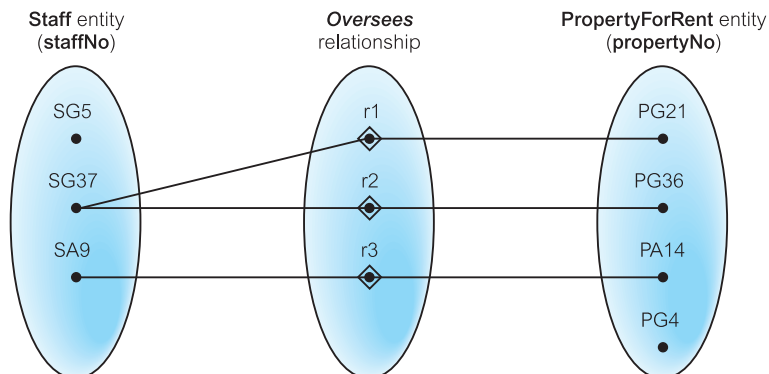
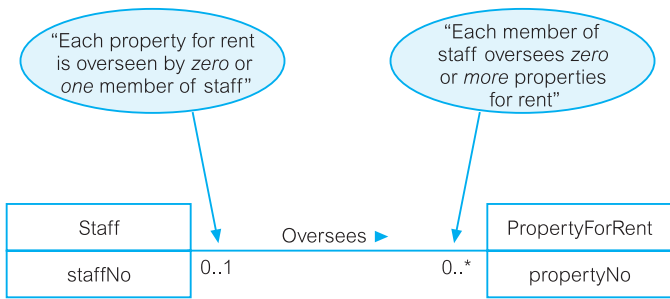


Figure 12.15(a)
Semantic net
showing three
occurrences of
the Staff Oversees
PropertyForRent
relationship type.

Figure 12.15(b)
The multiplicity of the Staff Oversees PropertyForRent one-to-many (1:*) relationship type.



If we know the actual minimum and maximum values for the multiplicity, we can display these instead. For example, if a member of staff oversees a minimum of zero and a maximum of 100 properties for rent, we can replace the “0..*” with “0..100.”

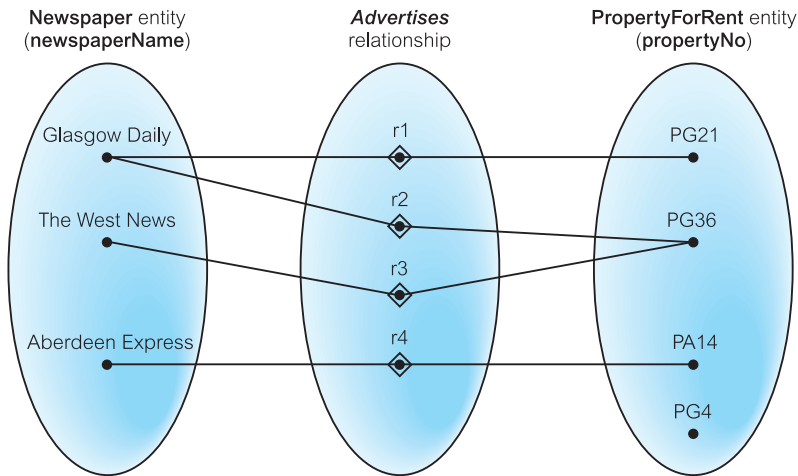
12.6.3 Many-to-Many (*:*) Relationships

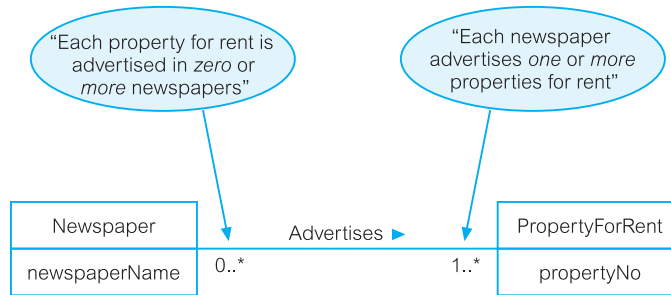
Consider the relationship *Advertises*, which relates the Newspaper and PropertyForRent entity types. Figure 12.16(a) displays four occurrences of the *Advertises* relationship (denoted r1, r2, r3, and r4) using a semantic net. Each relationship (*rn*) represents the association between a single Newspaper entity occurrence and a single PropertyForRent entity occurrence. We represent each entity occurrence using the values for the primary key attributes of the Newspaper and PropertyForRent entity types, namely newspaperName and propertyNo.

Determining the multiplicity

In Figure 12.16(a) we see that the *Glasgow Daily* advertises propertyNos PG21 and PG36, *The West News* also advertises propertyNo PG36 and the *Aberdeen Express* advertises propertyNo PA14. However, propertyNo PG4 is not advertised in any newspaper. In other words, *one* newspaper advertises *one or more* properties for rent and *one* property for rent is advertised in *zero or more* newspapers. Therefore, for

Figure 12.16(a)
Semantic net showing four occurrences of the Newspaper Advertises PropertyForRent relationship type.



**Figure 12.16(b)**

The multiplicity of the Newspaper *Advertises* PropertyForRent many-to-many (*:*) relationship.

newspapers there are *many* properties for rent, and for each property for rent participating in this relationship there are *many* newspapers. We refer to this type of relationship as many-to-many, which we usually abbreviate as (*:*) .

Diagrammatic representation of *: * relationships

An ER diagram of the Newspaper *Advertises* PropertyForRent relationship is shown in Figure 12.16(b). To represent that each newspaper can advertise *one or more* properties for rent, we place a "1..*" beside the PropertyForRent entity type. To represent that each property for rent can be advertised by *zero or more* newspapers, we place a "0..*" beside the Newspaper entity. (Note that for a *: * relationship, we may choose a relationship name that makes sense in either direction.)

12.6.4 Multiplicity for Complex Relationships

Multiplicity for complex relationships—that is, those higher than binary—is slightly more complex.

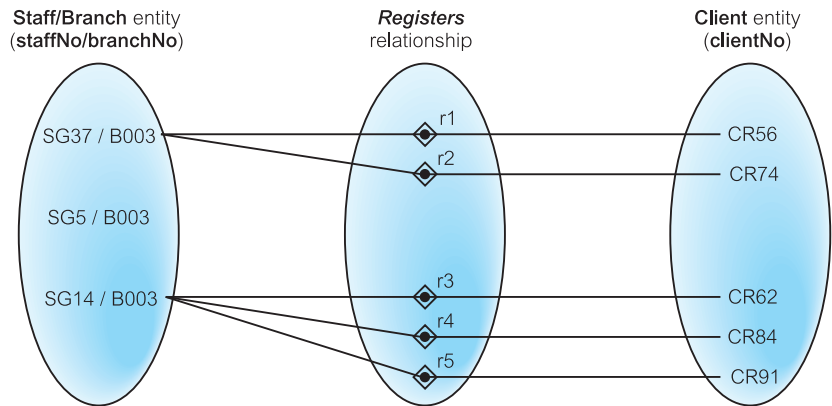
Multiplicity (complex relationship)

The number (or range) of possible occurrences of an entity type in an n -ary relationship when the other $(n-1)$ values are fixed.

In general, the multiplicity for n -ary relationships represents the potential number of entity occurrences in the relationship when $(n-1)$ values are fixed for the other participating entity types. For example, the multiplicity for a ternary relationship represents the potential range of entity occurrences of a particular entity in the relationship when the other two values representing the other two entities are fixed. Consider the ternary *Registers* relationship between Staff, Branch, and Client shown in Figure 12.7. Figure 12.17(a) displays five occurrences of the *Registers* relationship (denoted r1 to r5) using a semantic net. Each relationship (rn) represents the association of a single Staff entity occurrence, a single Branch entity occurrence, and a single Client entity occurrence. We represent each entity occurrence using the values for the primary key attributes of the Staff, Branch, and Client entities, namely staffNo, branchNo, and clientNo. In Figure 12.17(a) we examine the *Registers* relationship when the values for the Staff and Branch entities are fixed.

Figure 12.17(a)

Semantic net showing five occurrences of the ternary *Registers* relationship with values for Staff and Branch entity types fixed.



Determining the multiplicity

In Figure 12.17(a) with the staffNo/branchNo values fixed there are *zero or more* clientNo values. For example, staffNo SG37 at branchNo B003 registers clientNo CR56 and CR74, and staffNo SG14 at branchNo B003 registers clientNo CR62, CR84, and CR91. However, SG5 at branchNo B003 registers no clients. In other words, when the staffNo and branchNo values are fixed the corresponding clientNo values are *zero or more*. Therefore, the multiplicity of the *Registers* relationship from the perspective of the Staff and Branch entities is 0..*, which is represented in the ER diagram by placing the 0..* beside the Client entity.

If we repeat this test we find that the multiplicity when Staff/Client values are fixed is 1..1, which is placed beside the Branch entity, and the Client/Branch values are fixed is 1..1, which is placed beside the Staff entity. An ER diagram of the ternary *Registers* relationship showing multiplicity is in Figure 12.17(b).

A summary of the possible ways that multiplicity constraints can be represented along with a description of the meaning is shown in Table 12.1.

12.6.5 Cardinality and Participation Constraints

Multiplicity actually consists of two separate constraints known as cardinality and participation.

Cardinality	Describes the maximum number of possible relationship occurrences for an entity participating in a given relationship type.
--------------------	---

Figure 12.17(b)

The multiplicity of the ternary *Registers* relationship.

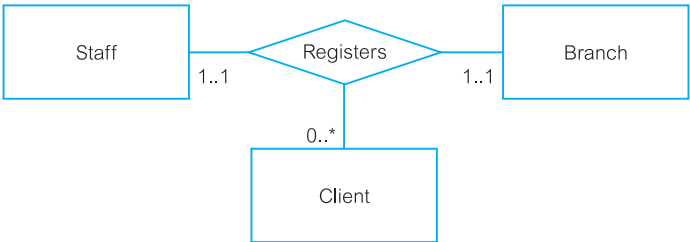


TABLE 12.1 A summary of ways to represent multiplicity constraints.

ALTERNATIVE WAYS TO REPRESENT MULTIPLICITY CONSTRAINTS	MEANING
0..1	Zero or one entity occurrence
1..1 (or just 1)	Exactly one entity occurrence
0..* (or just *)	Zero or many entity occurrences
1..*	One or many entity occurrences
5..10	Minimum of 5 up to a maximum of 10 entity occurrences
0, 3, 6–8	Zero or three or six, seven, or eight entity occurrences

The cardinality of a binary relationship is what we previously referred to as a one-to-one (1:1), one-to-many (1:*), and many-to-many (*:*). The cardinality of a relationship appears as the *maximum* values for the multiplicity ranges on either side of the relationship. For example, the *Manages* relationship shown in Figure 12.18 has a one-to-one (1:1) cardinality, and this is represented by multiplicity ranges with a maximum value of 1 on both sides of the relationship.

Participation

Determines whether all or only some entity occurrences participate in a relationship.

The participation constraint represents whether all entity occurrences are involved in a particular relationship (referred to as **mandatory** participation) or only some (referred to as **optional** participation). The participation of entities in a

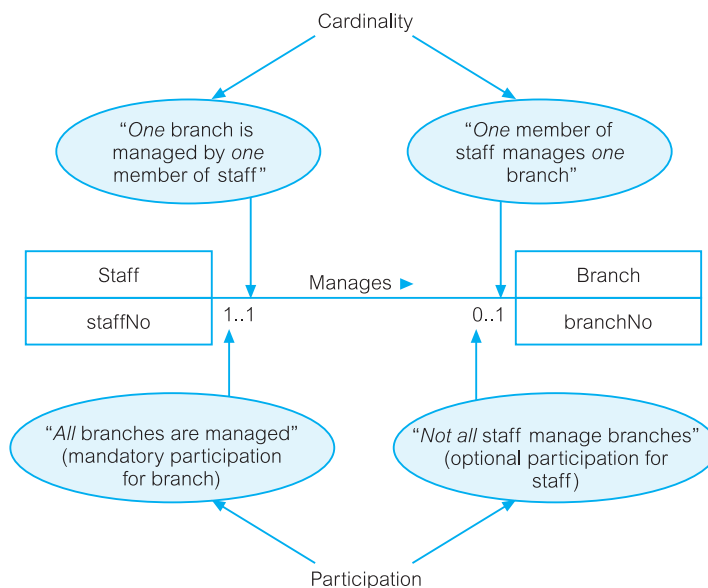


Figure 12.18
Multiplicity described as cardinality and participation constraints for the Staff *Manages* Branch (1:1) relationship.

relationship appears as the *minimum* values for the multiplicity ranges on either side of the relationship. Optional participation is represented as a minimum value of 0, and mandatory participation is shown as a minimum value of 1. It is important to note that the participation for a given entity in a relationship is represented by the minimum value on the *opposite* side of the relationship; that is, the minimum value for the multiplicity beside the related entity. For example, in Figure 12.18, the optional participation for the Staff entity in the *Manages* relationship is shown as a minimum value of 0 for the multiplicity beside the Branch entity and the mandatory participation for the Branch entity in the *Manages* relationship is shown as a minimum value of 1 for the multiplicity beside the Staff entity.

A summary of the conventions introduced in this section to represent the basic concepts of the ER model is shown on the inside front cover of this book.

12.7 Problems with ER Models

In this section we examine problems that may arise when creating an ER model. These problems are referred to as **connection traps**, and normally occur due to a misinterpretation of the meaning of certain relationships (Howe, 1989). We examine two main types of connection traps, called **fan traps** and **chasm traps**, and illustrate how to identify and resolve such problems in ER models.

In general, to identify connection traps we must ensure that the meaning of a relationship is fully understood and clearly defined. If we do not understand the relationships we may create a model that is not a true representation of the “real world.”

12.7.1 Fan Traps

Fan trap

Where a model represents a relationship between entity types, but the pathway between certain entity occurrences is ambiguous.

A fan trap may exist where two or more 1:* relationships fan out from the same entity. A potential fan trap is illustrated in Figure 12.19(a), which shows two 1:* relationships (*Has* and *Operates*) emanating from the same entity called Division.

This model represents the facts that a single division operates *one or more* branches and has *one or more* staff. However, a problem arises when we want to know which members of staff work at a particular branch. To appreciate the problem, we examine some occurrences of the *Has* and *Operates* relationships using values for the primary key attributes of the Staff, Division, and Branch entity types, as shown in Figure 12.19(b).

If we attempt to answer the question: “At which branch does staff number SG37 work?” we are unable to give a specific answer based on the current structure. We can determine only that staff number SG37 works at Branch B003 *or* B007. The



Figure 12.19(a) An example of a fan trap.

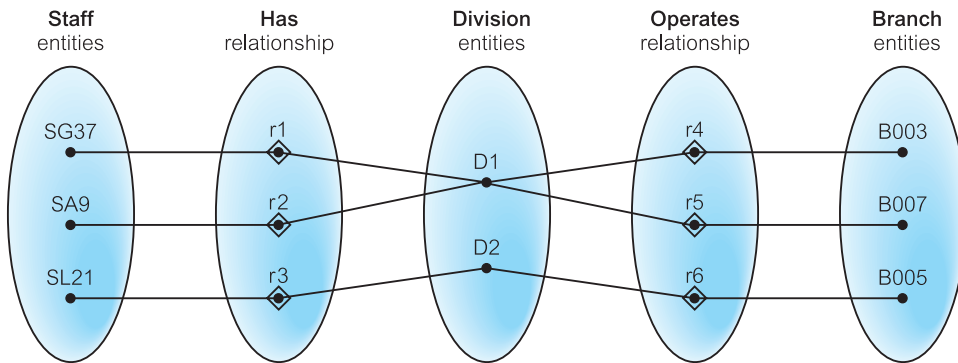


Figure 12.19(b) The semantic net of the ER model shown in Figure 12.19(a).

inability to answer this question specifically is the result of a fan trap associated with the misrepresentation of the correct relationships between the Staff, Division, and Branch entities. We resolve this fan trap by restructuring the original ER model to represent the correct association between these entities, as shown in Figure 12.20(a).

If we now examine occurrences of the *Operates* and *Has* relationships, as shown in Figure 12.20(b), we are now in a position to answer the type of question posed earlier. From this semantic net model, we can determine that staff number SG37 works at branch number B003, which is part of division D1.

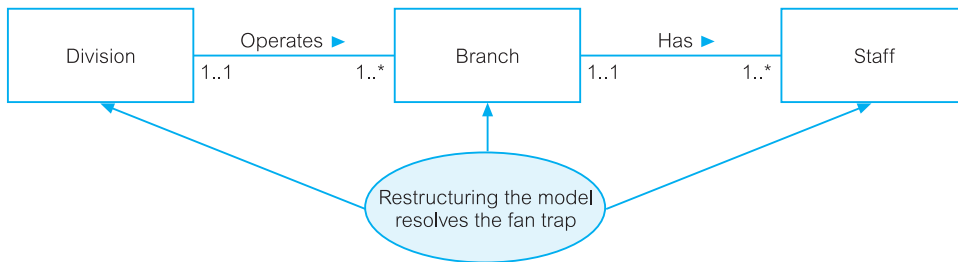


Figure 12.20(a) The ER model shown in Figure 12.19(a) restructured to remove the fan trap.

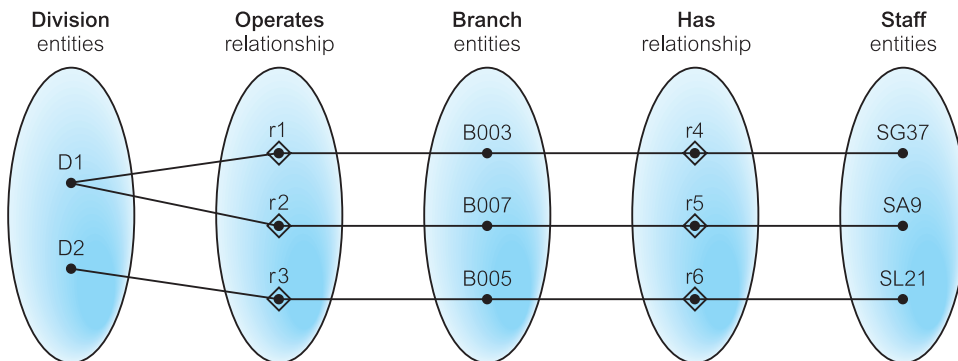


Figure 12.20(b) The semantic net of the ER model shown in Figure 12.20(a).

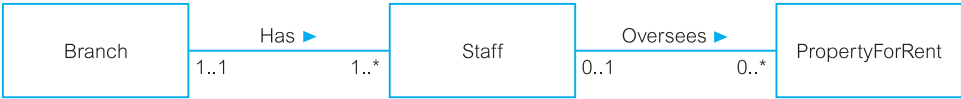


Figure 12.21(a) An example of a chasm trap.

12.7.2 Chasm Traps

Chasm trap

Where a model suggests the existence of a relationship between entity types, but the pathway does not exist between certain entity occurrences.

A chasm trap may occur where there are one or more relationships with a minimum multiplicity of zero (that is, optional participation) forming part of the pathway between related entities. A potential chasm trap is illustrated in Figure 12.21(a), which shows relationships between the Branch, Staff, and PropertyForRent entities.

This model represents the facts that a single branch has *one or more* staff who oversee *zero or more* properties for rent. We also note that not all staff oversee property, and not all properties are overseen by a member of staff. A problem arises when we want to know which properties are available at each branch. To appreciate the problem, we examine some occurrences of the *Has* and *Oversees* relationships using values for the primary key attributes of the Branch, Staff, and PropertyForRent entity types, as shown in Figure 12.21(b).

If we attempt to answer the question: “At which branch is property number PA14 available?” we are unable to answer this question, as this property is not yet allocated to a member of staff working at a branch. The inability to answer this question is considered to be a loss of information (as we know a property must be available at a branch), and is the result of a chasm trap. The multiplicity of both the Staff and PropertyForRent entities in the *Oversees* relationship has a minimum value of zero, which means that some properties cannot be associated with a branch through a member of staff. Therefore, to solve this problem, we need to identify the missing relationship, which in this case is the *Offers* relationship

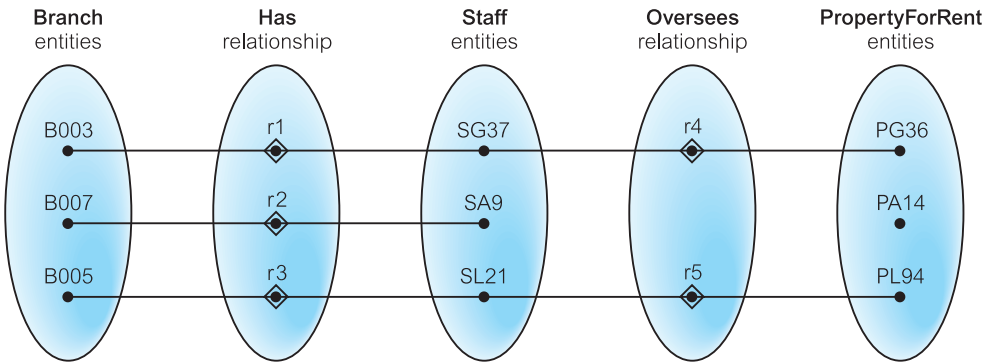


Figure 12.21(b) The semantic net of the ER model shown in Figure 12.21(a).

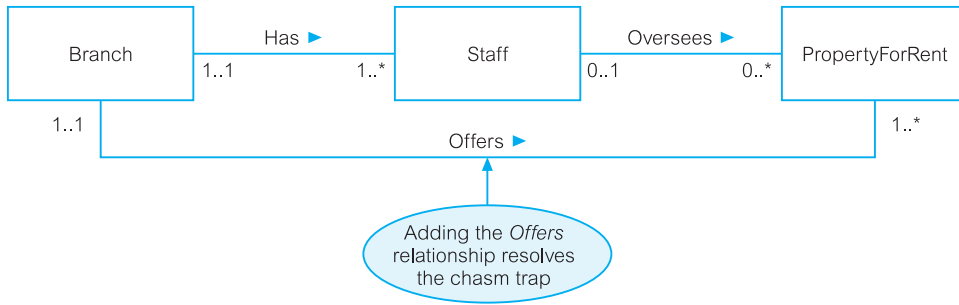


Figure 12.22(a) The ER model shown in Figure 12.21(a) restructured to remove the chasm trap.

between the Branch and PropertyForRent entities. The ER model shown in Figure 12.22(a) represents the true association between these entities. This model ensures that at all times, the properties associated with each branch are known, including properties that are not yet allocated to a member of staff.

If we now examine occurrences of the *Has*, *Oversees*, and *Offers* relationship types, as shown in Figure 12.22(b), we are now able to determine that property number PA14 is available at branch number B007.

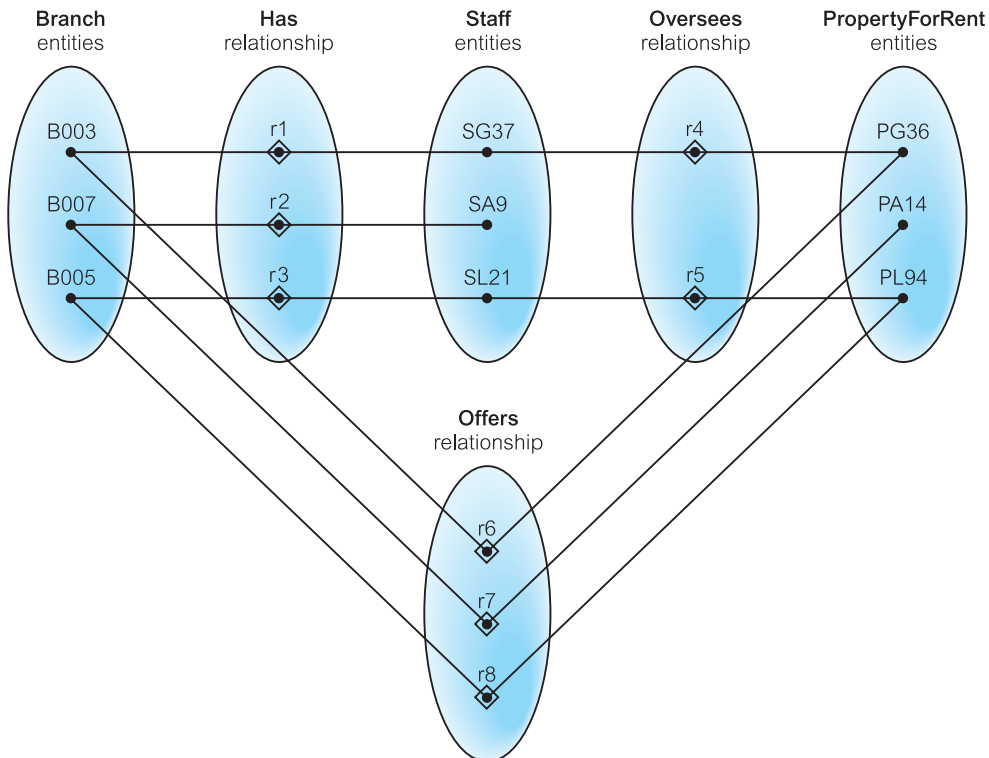


Figure 12.22(b) The semantic net of the ER model shown in Figure 12.22(a).

Chapter Summary

- An **entity type** is a group of objects with the same properties, which are identified by the enterprise as having an independent existence. An **entity occurrence** is a uniquely identifiable object of an entity type.
- A **relationship type** is a set of meaningful associations among entity types. A **relationship occurrence** is a uniquely identifiable association, which includes one occurrence from each participating entity type.
- The **degree of a relationship type** is the number of participating entity types in a relationship.
- A **recursive relationship** is a relationship type where the *same* entity type participates more than once in *different* roles.
- An **attribute** is a property of an entity or a relationship type.
- An **attribute domain** is the set of allowable values for one or more attributes.
- A **simple attribute** is composed of a single component with an independent existence.
- A **composite attribute** is composed of multiple components each with an independent existence.
- A **single-valued attribute** holds a single value for each occurrence of an entity type.
- A **multi-valued attribute** holds multiple values for each occurrence of an entity type.
- A **derived attribute** represents a value that is derivable from the value of a related attribute or set of attributes, not necessarily in the same entity.
- A **candidate key** is the minimal set of attributes that uniquely identifies each occurrence of an entity type.
- A **primary key** is the candidate key that is selected to uniquely identify each occurrence of an entity type.
- A **composite key** is a candidate key that consists of two or more attributes.
- A **strong entity type** is *not* existence-dependent on some other entity type. A **weak entity type** is existence-dependent on some other entity type.
- **Multiplicity** is the number (or range) of possible occurrences of an entity type that may relate to a single occurrence of an associated entity type through a particular relationship.
- **Multiplicity for a complex relationship** is the number (or range) of possible occurrences of an entity type in an n -ary relationship when the other $(n-1)$ values are fixed.
- **Cardinality** describes the maximum number of possible relationship occurrences for an entity participating in a given relationship type.
- **Participation** determines whether all or only some entity occurrences participate in a given relationship.
- A **fan trap** exists where a model represents a relationship between entity types, but the pathway between certain entity occurrences is ambiguous.
- A **chasm trap** exists where a model suggests the existence of a relationship between entity types, but the pathway does not exist between certain entity occurrences.

Review Questions

- 12.1 Why is the ER model considered a top-down approach? Describe the four basic components of the ER model.
- 12.2 Describe what relationship types represent in an ER model and provide examples of unary, binary, ternary, and quaternary relationships.

- 12.3 The ER model uses a number of notations and tags to represent different concepts. Outline how the basic ER components are represented in an ER diagram.
- 12.4 Describe what the multiplicity constraint represents for a relationship type.
- 12.5 What are integrity constraints and how does multiplicity model these constraints?
- 12.6 How does multiplicity represent both the cardinality and the participation constraints on a relationship type?
- 12.7 Provide an example of a relationship type with attributes.
- 12.8 Distinguish between the Entity—Relationship model and the Entity—Relationship diagram.
- 12.9 Describe how fan and chasm traps can occur in an ER model and how they can be resolved.

Exercises

- 12.10 Create an ER model for each of the following descriptions:
 - (a) Each company operates four departments, and each department belongs to one company.
 - (b) Each department in part (a) employs one or more employees, and each employee works for one department.
 - (c) Each of the employees in part (b) may or may not have one or more dependents, and each dependent belongs to one employee.
 - (d) Each employee in part (c) may or may not have an employment history.
 - (e) Represent all the ER models described in (a), (b), (c), and (d) as a single ER model.
- 12.11 Assume you have been contracted by a university to develop a database system to keep track of student registration and accommodation records. The university courses are offered by faculties. Depending on the student's IQ, there are no limitations to how many courses a student can enroll in. The faculties are not responsible for student accommodation. The university owns a number of hostels and each student is given a shared room key after enrollment. Each room has furniture attached to it.
 - (a) Identify the main entity types for the project.
 - (b) Identify the main relationship types and specify the multiplicity for each relationship. State any assumptions that you make about the data.
 - (c) Using your answers for (a) and (b), draw a single ER diagram to represent the data requirements for the project.
- 12.12 Read the following case study, which describes the data requirements for a DVD rental company. The DVD rental company has several branches throughout the United States. The data held on each branch is the branch address made up of street, city, state, and zip code, and the telephone number. Each branch is given a branch number, which is unique throughout the company. Each branch is allocated staff, which includes a Manager. The Manager is responsible for the day-to-day running of a given branch. The data held on a member of staff is his or her name, position, and salary. Each member of staff is given a staff number, which is unique throughout the company. Each branch has a stock of DVDs. The data held on a DVD is the catalog number, DVD number, title, category, daily rental, cost, status, and the names of the main actors and the director. The catalog number uniquely identifies each DVD. However, in most cases, there are several copies of each DVD at a branch, and the individual copies are identified using the DVD number. A DVD is given a category such as Action, Adult, Children, Drama, Horror, or Sci-Fi. The status indicates whether a specific copy of a DVD is available for rent. Before borrowing a DVD from the company, a customer must first register as a member of a local branch. The data held on a member is the first and last name, address, and the date that the member registered at a branch. Each member is given a member number, which is unique throughout all branches of the company. Once registered, a member is free to rent DVDs, up to a maximum of ten at any one time. The data held on each DVD rented is the rental number, the full name and number of the member, the DVD number, title, and daily rental, and the dates the DVD is rented out and returned. The DVD number is unique throughout the company.
 - (a) Identify the main entity types of the DVD rental company.
 - (b) Identify the main relationship types between the entity types described in part (a) and represent each relationship as an ER diagram.

- (c) Determine the multiplicity constraints for each relationships described in part (b). Represent the multiplicity for each relationship in the ER diagrams created in part (b).
- (d) Identify attributes and associate them with entity or relationship types. Represent each attribute in the ER diagrams created in (c).
- (e) Determine candidate and primary key attributes for each (strong) entity type.
- (f) Using your answers to parts (a) to (e), attempt to represent the data requirements of the DVD rental company as a single ER diagram. State any assumptions necessary to support your design.

12.13 Create an ER model for each of the following descriptions:

- (a) A large organization has several parking lots, which are used by staff.
- (b) Each parking lot has a unique name, location, capacity, and number of floors (where appropriate).
- (c) Each parking lot has parking spaces, which are uniquely identified using a space number.
- (d) Members of staff can request the sole use of a single parking space. Each member of staff has a unique number, name, telephone extension number, and vehicle license number.
- (e) Represent all the ER models described in parts (a), (b), (c), and (d) as a single ER model. Provide any assumptions necessary to support your model.

The final answer to this exercise is shown as Figure 13.11.

12.14 Create an ER model to represent the data used by the library.

The library provides books to borrowers. Each book is described by title, edition, and year of publication, and is uniquely identified using the ISBN. Each borrower is described by his or her name and address and is uniquely identified using a borrower number. The library provides one or more copies of each book and each copy is uniquely identified using a copy number, status indicating if the book is available for loan, and the allowable loan period for a given copy. A borrower may loan one or many books, and the date each book is loaned out and is returned is recorded. Loan number uniquely identifies each book loan.

The answer to this exercise is shown as Figure 13.12.

Enhanced Entity–Relationship Modeling

Chapter Objectives

In this chapter you will learn:

- The limitations of the basic concepts of the Entity–Relationship (ER) model and the requirements to represent more complex applications using additional data modeling concepts.
- The most useful additional data modeling concepts of the Enhanced Entity–Relationship (EER) model called specialization/generalization, aggregation, and composition.
- A diagrammatic technique for displaying specialization/generalization, aggregation, and composition in an EER diagram using the Unified Modeling Language (UML).

In Chapter 12 we discussed the basic concepts of the ER model. These basic concepts are normally adequate for building data models of traditional, administrative-based database systems such as stock control, product ordering, and customer invoicing. However, since the 1980s there has been a rapid increase in the development of many new database systems that have more demanding database requirements than those of the traditional applications. Examples of such database applications include Computer-Aided Design (CAD), Computer-Aided Manufacturing (CAM), Computer-Aided Software Engineering (CASE) tools, Office Information Systems (OIS) and Multimedia Systems, Digital Publishing, and Geographical Information Systems (GIS). The main features of these applications are described in Chapter 27. As the basic concepts of ER modeling are often insufficient to represent the requirements of the newer, more complex applications, this stimulated the need to develop additional “semantic” modeling concepts. Many different semantic data models have been proposed and some of the most important semantic concepts have been successfully incorporated into the original ER model. The ER model supported with additional semantic concepts is called the Enhanced Entity–Relationship (EER) model. In this chapter we describe three of the most important and useful additional concepts of the EER model, namely specialization/generalization, aggregation, and composition. We also illustrate how specialization/generalization, aggregation, and composition are represented in an EER diagram using UML (Booch *et al.*, 1998). In Chapter 12 we introduced UML and demonstrated how UML could be used to diagrammatically represent the basic concepts of the ER model.

Structure of this Chapter In Section 13.1 we discuss the main concepts associated with specialization/generalization and illustrate how these concepts are represented in an EER diagram using UML. We conclude this section with a worked example that demonstrates how to introduce specialization/generalization into an ER model using UML. In Section 13.2 we describe the concept of aggregation and in Section 13.3 the related concept of composition. We provide examples of aggregation and composition and show how these concepts can be represented in an EER diagram using UML.

13.1 Specialization/Generalization

The concept of specialization/generalization is associated with special types of entities known as **superclasses** and **subclasses**, and the process of **attribute inheritance**. We begin this section by defining superclasses and subclasses and by examining superclass/subclass relationships. We describe the process of attribute inheritance and contrast the process of specialization with the process of generalization. We then describe the two main types of constraints on superclass/subclass relationships called participation and disjoint constraints. We show how to represent specialization/generalization in an EER diagram using UML. We conclude this section with a worked example of how specialization/generalization may be introduced into the ER model of the Branch user views of the *DreamHome* case study described in Appendix A and shown in Figure 12.1.



13.1.1 Superclasses and Subclasses

As we discussed in Chapter 12, an entity type represents a set of entities of the same type such as *Staff*, *Branch*, and *PropertyForRent*. We can also form entity types into a hierarchy containing superclasses and subclasses.

Superclass

An entity type that includes one or more distinct subgroupings of its occurrences, which must be represented in a data model.

Subclass

A distinct subgrouping of occurrences of an entity type, which must be represented in a data model.

Entity types that have distinct subclasses are called superclasses. For example, the entities that are members of the *Staff* entity type may be classified as *Manager*, *SalesPersonnel*, and *Secretary*. In other words, the *Staff* entity is referred to as the **superclass** of the *Manager*, *SalesPersonnel*, and *Secretary* **subclasses**. The relationship between a superclass and any one of its subclasses is called a superclass/subclass relationship. For example, *Staff/Manager* has a superclass/subclass relationship.

13.1.2 Superclass/Subclass Relationships

Each member of a subclass is also a member of the superclass. In other words, the entity in the subclass is the same entity in the superclass, but has a distinct role. The relationship between a superclass and a subclass is one-to-one (1:1) and is called a superclass/subclass relationship (see Section 12.6.1). Some superclasses may contain overlapping subclasses, as illustrated by a member of staff who is both a Manager and a member of Sales Personnel. In this example, Manager and SalesPersonnel are overlapping subclasses of the Staff superclass. On the other hand, not every member of a superclass is necessarily a member of a subclass; for example, members of staff without a distinct job role such as a Manager or a member of Sales Personnel.

We can use superclasses and subclasses to avoid describing different types of staff with possibly different attributes within a single entity. For example, Sales Personnel may have special attributes such as salesArea and carAllowance. If all staff attributes and those specific to particular jobs are described by a single Staff entity, this may result in a lot of nulls for the job-specific attributes. Clearly, Sales Personnel have common attributes with other staff, such as staffNo, name, position, and salary. However, it is the unshared attributes that cause problems when we try to represent all members of staff within a single entity. We can also show relationships that are associated with only particular types of staff (sub-classes) and not with staff, in general. For example, Sales Personnel may have distinct relationships that are not appropriate for all staff, such as SalesPersonnel Uses Car.

To illustrate these points, consider the relation called AllStaff shown in Figure 13.1. This relation holds the details of all members of staff, no matter what position they hold. A consequence of holding all staff details in one relation is that while the attributes appropriate to all staff are filled (namely, staffNo, name, position, and salary), those that are only applicable to particular job roles are only partially filled. For example, the attributes associated with the Manager (mgrStartDate and bonus),

staffNo	name	position	salary	mgrStartDate	bonus	sales Area	car Allowance	typing Speed
SL21	John White	Manager	30000	01/02/95	2000			
SG37	Ann Beech	Assistant	12000					
SG66	Mary Martinez	Sales Manager	27000			SA1A	5000	
SA9	Mary Howe	Assistant	9000					
SL89	Stuart Stern	Secretary	8500					100
SL31	Robert Chin	Snr Sales Asst	17000			SA2B	3700	
SG5	Susan Brand	Manager	24000	01/06/91	2350			

Figure 13.1 The AllStaff relation holding details of all staff.

SalesPersonnel (salesArea and carAllowance), and Secretary (typingSpeed) subclasses have values for those members in these subclasses. In other words, the attributes associated with the Manager, SalesPersonnel, and Secretary subclasses are empty for those members of staff not in these subclasses.

There are two important reasons for introducing the concepts of superclasses and subclasses into an ER model. First, it avoids describing similar concepts more than once, thereby saving time for the designer and making the ER diagram more readable. Second, it adds more semantic information to the design in a form that is familiar to many people. For example, the assertions that “Manager IS-A member of staff” and “flat IS-A type of property,” communicates significant semantic content in a concise form.

13.1.3 Attribute Inheritance

As mentioned earlier, an entity in a subclass represents the same “real world” object as in the superclass, and may possess subclass-specific attributes, as well as those associated with the superclass. For example, a member of the SalesPersonnel subclass *inherits* all the attributes of the Staff superclass, such as staffNo, name, position, and salary together with those specifically associated with the SalesPersonnel subclass, such as salesArea and carAllowance.

A subclass is an entity in its own right and so it may also have one or more subclasses. An entity and its subclasses and their subclasses, and so on, is called a **type hierarchy**. Type hierarchies are known by a variety of names, including **specialization hierarchy** (for example, Manager is a specialization of Staff), **generalization hierarchy** (for example, Staff is a generalization of Manager), and **IS-A hierarchy** (for example, Manager IS-A (member of) Staff). We describe the process of specialization and generalization in the following sections.

A subclass with more than one superclass is called a **shared subclass**. In other words, a member of a shared subclass must be a member of the associated superclasses. As a consequence, the attributes of the superclasses are inherited by the shared subclass, which may also have its own additional attributes. This process is referred to as **multiple inheritance**.

13.1.4 Specialization Process

Specialization

The process of maximizing the differences between members of an entity by identifying their distinguishing characteristics.

Specialization is a top-down approach to defining a set of superclasses and their related subclasses. The set of subclasses is defined on the basis of some distinguishing characteristics of the entities in the superclass. When we identify a set of subclasses of an entity type, we then associate attributes specific to each subclass (where necessary), and also identify any relationships between each subclass and other entity types or subclasses (where necessary). For example, consider a model where all members of staff are represented as an entity called Staff. If we apply the process of specialization on the Staff entity, we attempt to identify differences between members of this entity, such as members with distinctive attributes

and/or relationships. As described earlier, staff with the job roles of Manager, Sales Personnel, and Secretary have distinctive attributes and therefore we identify Manager, SalesPersonnel, and Secretary as subclasses of a specialized Staff superclass.

13.1.5 Generalization Process

Generalization

The process of minimizing the differences between entities by identifying their common characteristics.

The process of generalization is a bottom-up approach, that results in the identification of a generalized superclass from the original entity types. For example, consider a model where Manager, SalesPersonnel, and Secretary are represented as distinct entity types. If we apply the process of generalization on these entities, we attempt to identify similarities between them, such as common attributes and relationships. As stated earlier, these entities share attributes common to all staff, and therefore we identify Manager, SalesPersonnel, and Secretary as subclasses of a generalized Staff superclass.

As the process of generalization can be viewed as the reverse of the specialization process, we refer to this modeling concept as “specialization/generalization.”

Diagrammatic representation of specialization/generalization

UML has a special notation for representing specialization/generalization. For example, consider the specialization/generalization of the Staff entity into subclasses that represent job roles. The Staff superclass and the Manager, SalesPersonnel, and Secretary subclasses can be represented in an EER diagram, as illustrated in Figure 13.2. Note that the Staff superclass and the subclasses, being entities, are represented as rectangles. The subclasses are attached by lines to a triangle that points toward the superclass. The label below the specialization/generalization triangle, shown as {Optional, And}, describes the constraints on the relationship between the superclass and its subclasses. These constraints are discussed in more detail in Section 13.1.6.

Attributes that are specific to a given subclass are listed in the lower section of the rectangle representing that subclass. For example, salesArea and carAllowance attributes are associated only with the SalesPersonnel subclass, and are not applicable to the Manager or Secretary subclasses. Similarly, we show attributes that are specific to the Manager (mgrStartDate and bonus) and Secretary (typingSpeed) subclasses.

Attributes that are common to all subclasses are listed in the lower section of the rectangle representing the superclass. For example, staffNo, name, position, and salary attributes are common to all members of staff and are associated with the Staff superclass. Note that we can also show relationships that are only applicable to specific subclasses. For example, in Figure 13.2, the Manager subclass is related to the Branch entity through the *Manages* relationship, whereas the Staff superclass is related to the Branch entity through the *Has* relationship.

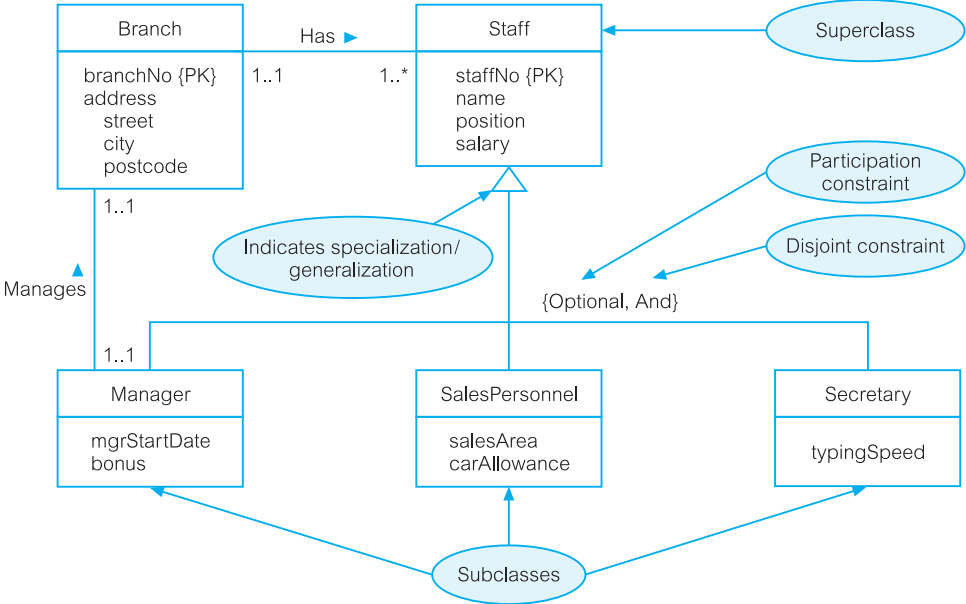


Figure 13.2 Specialization/generalization of the Staff entity into subclasses representing job roles.

We may have several specializations of the same entity based on different distinguishing characteristics. For example, another specialization of the Staff entity may produce the subclasses FullTimePermanent and PartTimeTemporary, which distinguishes between the types of employment contract for members of staff. The specialization of the Staff entity type into job role and contract of employment subclasses is shown in Figure 13.3. In this figure, we show attributes that are specific to the FullTimePermanent (salaryScale and holidayAllowance) and PartTimeTemporary (hourlyRate) subclasses.

As described earlier, a superclass and its subclasses and their subclasses, and so on, is called a type hierarchy. An example of a type hierarchy is shown in Figure 13.4, where the job roles specialization/generalization shown in Figure 13.2 are expanded to show a shared subclass called SalesManager and the subclass called Secretary with its own subclass called AssistantSecretary. In other words, a member of the SalesManager shared subclass must be a member of the SalesPersonnel and Manager subclasses as well as the Staff superclass. As a consequence, the attributes of the Staff superclass (staffNo, name, position, and salary), and the attributes of the subclasses SalesPersonnel (salesArea and carAllowance) and Manager (mgrStartDate and bonus) are inherited by the SalesManager subclass, which also has its own additional attribute called salesTarget.

AssistantSecretary is a subclass of Secretary, which is a subclass of Staff. This means that a member of the AssistantSecretary subclass must be a member of the Secretary subclass and the Staff superclass. As a consequence, the attributes of the Staff superclass (staffNo, name, position, and salary) and the attribute of the Secretary subclass (typingSpeed) are inherited by the AssistantSecretary subclass, which also has its own additional attribute called startDate.

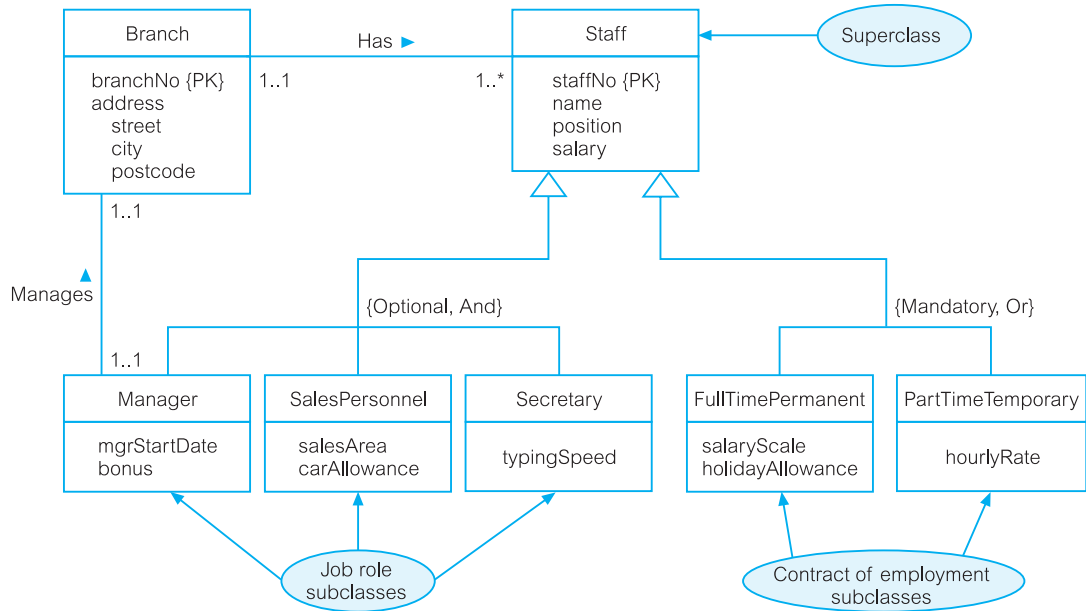


Figure 13.3 Specialization/generalization of the Staff entity into subclasses representing job roles and contracts of employment.

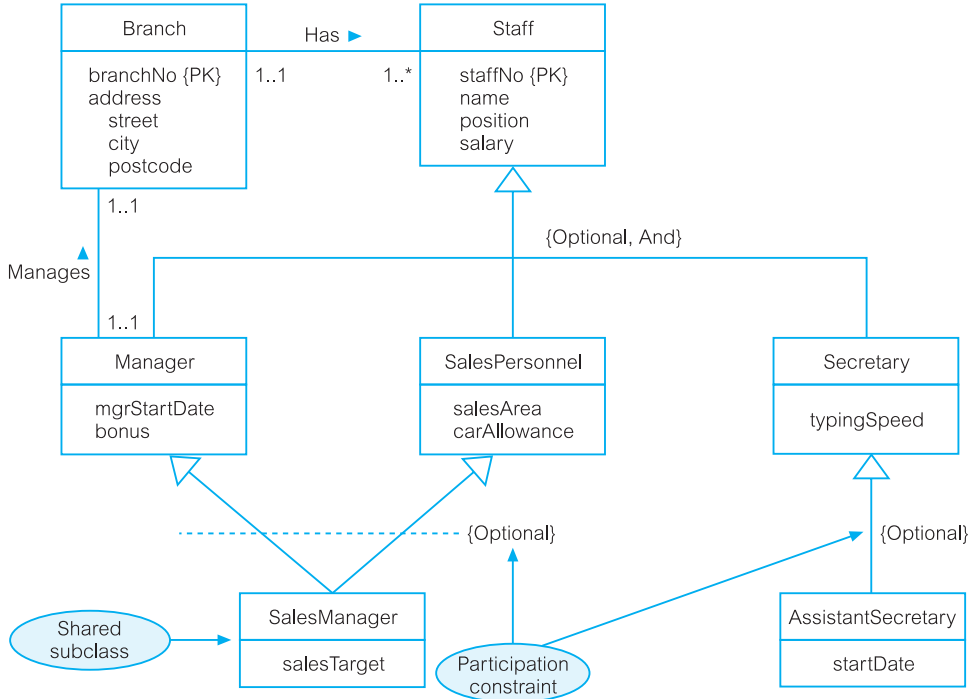


Figure 13.4 Specialization/generalization of the Staff entity into job roles including a shared subclass called SalesManager and a subclass called Secretary with its own subclass called AssistantSecretary.

13.1.6 Constraints on Specialization/Generalization

There are two constraints that may apply to a specialization/generalization called **participation constraints** and **disjoint constraints**.

Participation constraints

Participation constraint

Determines whether every member in the superclass must participate as a member of a subclass.

A participation constraint may be **mandatory** or **optional**. A superclass/subclass relationship with mandatory participation specifies that every member in the superclass must also be a member of a subclass. To represent mandatory participation, “Mandatory” is placed in curly brackets below the triangle that points towards the superclass. For example, in Figure 13.3 the contract of employment specialization/generalization is mandatory participation, which means that every member of staff must have a contract of employment.

A superclass/subclass relationship with optional participation specifies that a member of a superclass need not belong to any of its subclasses. To represent optional participation, “Optional” is placed in curly brackets below the triangle that points towards the superclass. For example, in Figure 13.3 the job role specialization/generalization has optional participation, which means that a member of staff need not have an additional job role such as a Manager, Sales Personnel, or Secretary.

Disjoint constraints

Disjoint constraint

Describes the relationship between members of the subclasses and indicates whether it is possible for a member of a superclass to be a member of one, or more than one, subclass.

The disjoint constraint only applies when a superclass has more than one subclass. If the subclasses are **disjoint**, then an entity occurrence can be a member of only one of the subclasses. To represent a disjoint superclass/subclass relationship, “Or” is placed next to the participation constraint within the curly brackets. For example, in Figure 12.3 the subclasses of the contract of employment specialization/generalization is disjoint, which means that a member of staff must have a full-time permanent *or* a part-time temporary contract, but not both.

If subclasses of a specialization/generalization are not disjoint (called **nondisjoint**), then an entity occurrence may be a member of more than one subclass. To represent a nondisjoint superclass/subclass relationship, “And” is placed next to the participation constraint within the curly brackets. For example, in Figure 13.3 the job role specialization/generalization is nondisjoint, which means that an entity occurrence can be a member of both the Manager, SalesPersonnel, and Secretary subclasses. This is confirmed by the presence of the shared subclass called SalesManager shown in Figure 13.4. Note that it is not necessary to include the disjoint constraint for hierarchies that have a single subclass at a given level and for this reason only the participation constraint is shown for the SalesManager and AssistantSecretary subclasses of Figure 13.4.

The disjoint and participation constraints of specialization and generalization are distinct, giving rise to four categories: “mandatory and disjoint,” “optional and disjoint,” “mandatory and nondisjoint,” and “optional and nondisjoint.”

13.1.7 Worked Example of using Specialization/Generalization to Model the Branch View of the *DreamHome* Case Study



The database design methodology described in this book includes the use of specialization/generalization as an optional step (Step 1.6) in building an EER model. The choice to use this step is dependent on the complexity of the enterprise (or part of the enterprise) being modeled and whether using the additional concepts of the EER model will help the process of database design.

In Chapter 12 we described the basic concepts necessary to build an ER model to represent the Branch user views of the *DreamHome* case study. This model was shown as an ER diagram in Figure 12.1. In this section, we show how specialization/generalization may be used to convert the ER model of the Branch user views into an EER model.

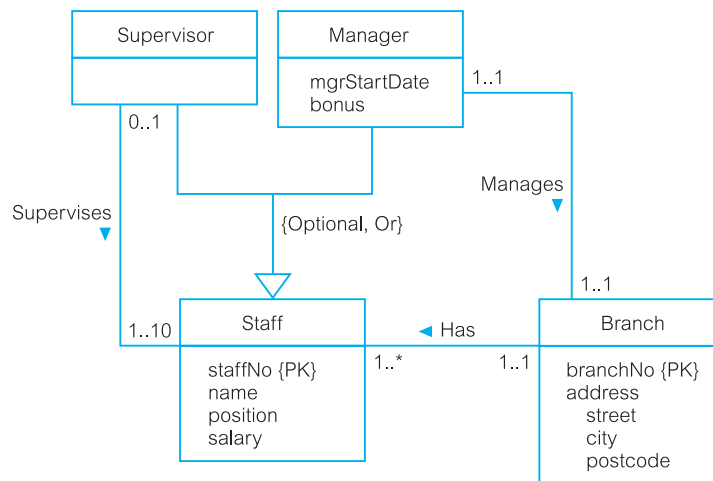
As a starting point, we first consider the entities shown in Figure 12.1. We examine the attributes and relationships associated with each entity to identify any similarities or differences between the entities. In the Branch user views' requirements specification there are several instances where there is the potential to use specialization/generalization as discussed shortly.

- (a) For example, consider the *Staff* entity in Figure 12.1, which represents all members of staff. However, in the data requirements specification for the Branch user views of the *DreamHome* case study given in Appendix A, there are two key job roles mentioned, namely *Manager* and *Supervisor*. We have three options as to how we may best model members of staff. The first option is to represent all members of staff as a generalized *Staff* entity (as in Figure 12.1), the second option is to create three distinct entities *Staff*, *Manager*, and *Supervisor*, and the third option is to represent the *Manager* and *Supervisor* entities as subclasses of a *Staff* superclass. The option we select is based on the commonality of attributes and relationships associated with each entity. For example, all attributes of the *Staff* entity are represented in the *Manager* and *Supervisor* entities, including the same primary key, namely *staffNo*. Furthermore, the *Supervisor* entity does not have any additional attributes representing this job role. On the other hand, the *Manager* entity has two additional attributes: *mgrStartDate* and *bonus*. In addition, both the *Manager* and *Supervisor* entities are associated with distinct relationships, namely *Manager* *Manages* *Branch* and *Supervisor* *Supervises* *Staff*. Based on this information, we select the third option and create *Manager* and *Supervisor* subclasses of the *Staff* superclass, as shown in Figure 13.5. Note that in this EER diagram, the subclasses are shown above the superclass. The relative positioning of the subclasses and superclass is not significant, however; what is important is that the specialization/generalization triangle points toward the superclass.

The specialization/generalization of the *Staff* entity is optional and disjoint (shown as {Optional, Or}), as not all members of staff are *Managers* or *Supervisors*, and in addition a single member of staff cannot be both a *Manager*

Figure 13.5

Staff superclass with Supervisor and Manager subclasses.



and a Supervisor. This representation is particularly useful for displaying the shared attributes associated with these subclasses and the Staff superclass and also the distinct relationships associated with each subclass, namely Manager *Manages* Branch and Supervisor *Supervises* Staff.

- (b) Consider for specialization/generalization the relationship between owners of property. The data requirements specification for the Branch user views describes two types of owner, namely PrivateOwner and BusinessOwner as shown in Figure 12.1. Again, we have three options as to how we may best model owners of property. The first option is to leave PrivateOwner and BusinessOwner as two distinct entities (as shown in Figure 12.1), the second option is to represent both types of owner as a generalized Owner entity, and the third option is to represent the PrivateOwner and BusinessOwner entities as subclasses of an Owner superclass. Before we are able to reach a decision we first examine the attributes and relationships associated with these entities. PrivateOwner and BusinessOwner entities share common attributes, namely address and telNo and have a similar relationship with property for rent (namely PrivateOwner *POwns* PropertyForRent and BusinessOwner *BOwns* PropertyForRent). However, both types of owner also have different attributes; for example, PrivateOwner has distinct attributes ownerNo and name, and BusinessOwner has distinct attributes bName, bType, and contactName. In this case, we create a superclass called Owner, with PrivateOwner and BusinessOwner as subclasses, as shown in Figure 13.6.

The specialization/generalization of the Owner entity is mandatory and disjoint (shown as {Mandatory, Or}), as an owner must be either a private owner *or* a business owner, but cannot be both. Note that we choose to relate the Owner superclass to the PropertyForRent entity using the relationship called Owns.

The examples of specialization/generalization described previously are relatively straightforward. However, the specialization/generalization process can be taken further as illustrated in the following example.

- (c) There are several persons with common characteristics described in the data requirements specification for the Branch user views of the *DreamHome* case

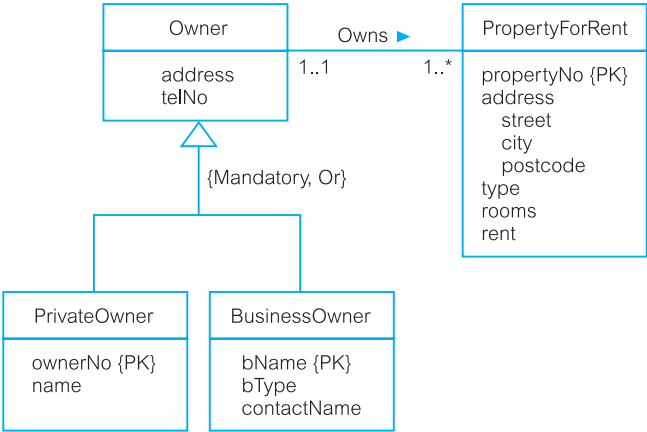


Figure 13.6
Owner superclass
with PrivateOwner
and BusinessOwner
subclasses.

study. For example, members of staff, private property owners, and clients all have number and name attributes. We could create a *Person* superclass with *Staff* (including *Manager* and *Supervisor* subclasses), *PrivateOwner*, and *Client* as subclasses, as shown in Figure 13.7.

We now consider to what extent we wish to use specialization/generalization to represent the Branch user views of the *DreamHome* case study. We decide to use the specialization/generalization examples described in (a) and (b) above but not (c), as shown in Figure 13.8. To simplify the EER diagram only attributes

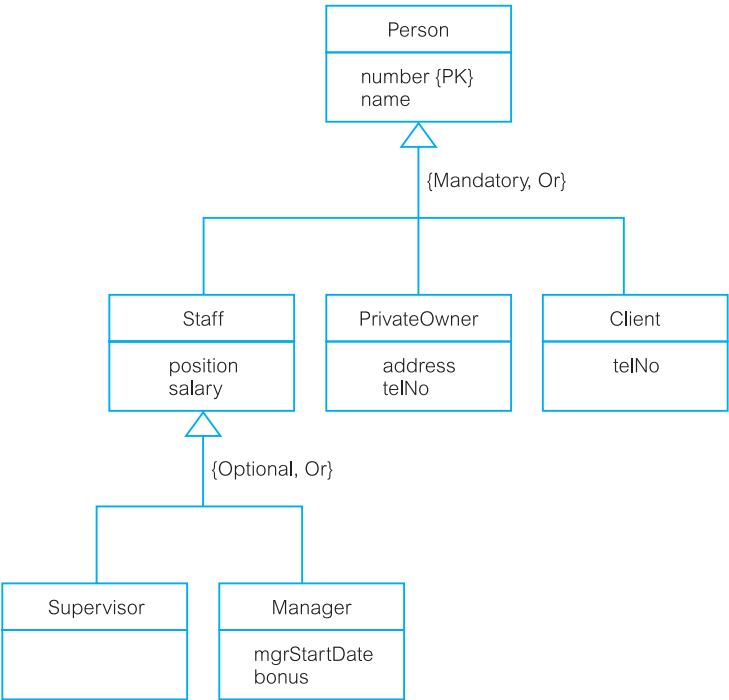


Figure 13.7
Person superclass
with Staff
(including
Supervisor and
Manager
subclasses),
PrivateOwner,
and Client
subclasses.

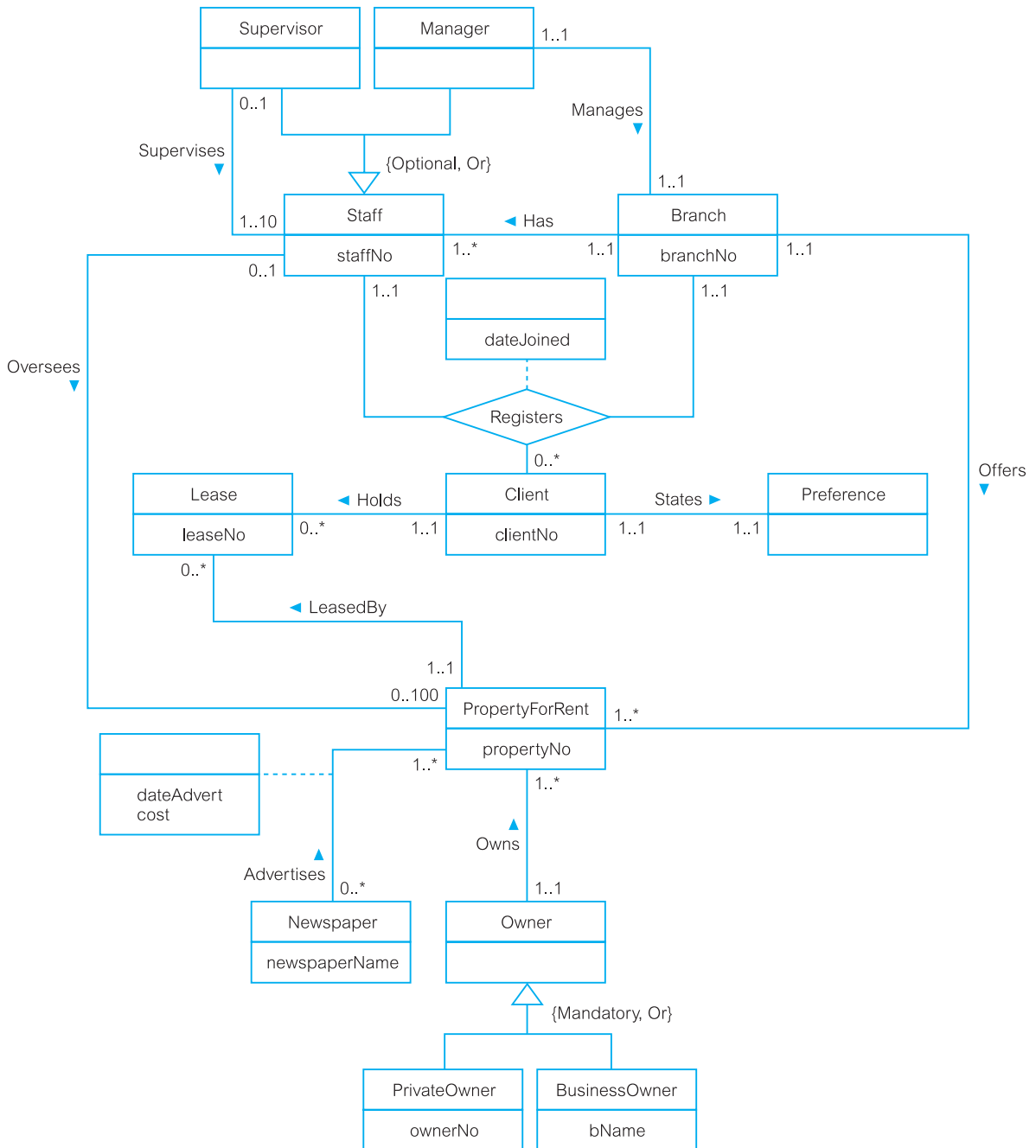


Figure 13.8 An EER model of the Branch user views of *DreamHome* with specialization/generalization.

associated with primary keys or relationships are shown. We leave out the representation shown in Figure 13.7 from the final EER model, because the use of specialization/generalization in this case places too much emphasis on the relationship between entities that are persons rather than emphasizing the relationship between these entities and some of the core entities such as Branch and PropertyForRent.

The option to use specialization/generalization, and to what extent, is a subjective decision. In fact, the use of specialization/generalization is presented as an optional step in our methodology for conceptual database design discussed in Chapter 16, Step 1.6.

As described in Section 2.3, the purpose of a data model is to provide the concepts and notations that allow database designers and end-users to unambiguously and accurately communicate their understanding of the enterprise data. Therefore, if we keep these goals in mind, we should use the additional concepts of specialization/generalization only when the enterprise data is too complex to easily represent using only the basic concepts of the ER model.

At this stage, we may consider whether the introduction of specialization/generalization to represent the Branch user views of *DreamHome* is a good idea. In other words, is the requirement specification for the Branch user views better represented as the ER model shown in Figure 12.1 or as the EER model shown in Figure 13.8? We leave this for the reader to consider.

13.2 Aggregation

Aggregation

Represents a “has-a” or “is-part-of” relationship between entity types, where one represents the “whole” and the other the “part.”

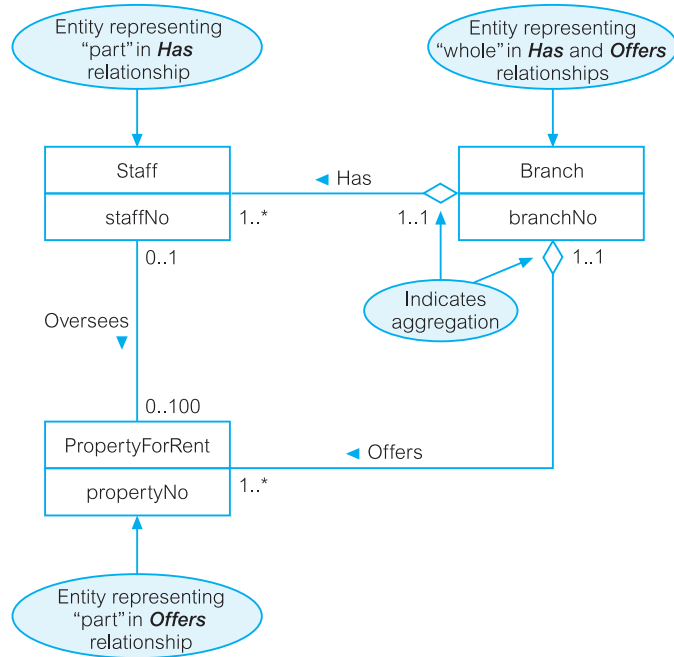
A relationship represents an association between two entity types that are conceptually at the same level. Sometimes we want to model a “has-a” or “is-part-of” relationship, in which one entity represents a larger entity (the “whole”), consisting of smaller entities (the “parts”). This special kind of relationship is called an **aggregation** (Booch *et al.*, 1998). Aggregation does not change the meaning of navigation across the relationship between the whole and its parts, or link the lifetimes of the whole and its parts. An example of an aggregation is the *Has* relationship, which relates the Branch entity (the “whole”) to the Staff entity (the “part”).

Diagrammatic representation of aggregation

UML represents aggregation by placing an open diamond shape at one end of the relationship line, next to the entity that represents the “whole.” In Figure 13.9, we redraw part of the EER diagram shown in Figure 13.8 to demonstrate aggregation. This EER diagram displays two examples of aggregation, namely Branch *Has* Staff and Branch *Offers* PropertyForRent. In both relationships, the Branch entity represents the “whole” and therefore the open diamond shape is placed beside this entity.

Figure 13.9

Examples of aggregation:
Branch *Has* Staff
and Branch *Offers*
PropertyForRent.



13.3 Composition

Composition

A specific form of aggregation that represents an association between entities, where there is a strong ownership and coincidental lifetime between the “whole” and the “part.”

Aggregation is entirely conceptual and does nothing more than distinguish a “whole” from a “part.” However, there is a variation of aggregation called **composition** that represents a strong ownership and coincidental lifetime between the “whole” and the “part” (Booch *et al.*, 1998). In a composite, the “whole” is responsible for the disposition of the “parts,” which means that the composition must manage the creation and destruction of its “parts.” In other words, an object may be part of only one composite at a time. There are no examples of composition in Figure 13.8. For the purposes of discussion, consider an example of a composition, namely the *Displays* relationship, which relates the Newspaper entity to the Advert entity. As a composition, this emphasizes the fact that an Advert entity (the “part”) belongs to exactly one Newspaper entity (the “whole”). This is in contrast to aggregation, in which a part may be shared by many wholes. For example, a Staff entity may be “a part of” one or more Branches entities.

Diagrammatic representation of composition

UML represents composition by placing a filled-in diamond shape at one end of the relationship line next to the entity that represents the “whole” in the relationship. For example, to represent the Newspaper *Displays* Advert composition, the filled-in diamond shape is placed next to the Newspaper entity, which is the “whole” in this relationship, as shown in Figure 13.10.

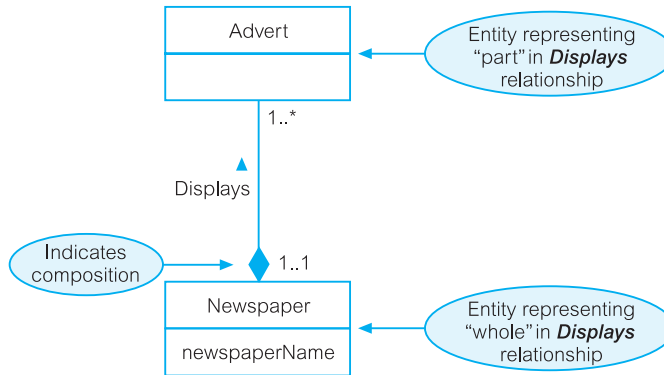


Figure 13.10
 An example of composition:
 Newspaper
Displays Advert.

As discussed with specialization/generalization, the options to use aggregation and composition, and to what extent, are again subjective decisions. Aggregation and composition should be used only when there is a requirement to emphasize special relationships between entity types such as “has-a” or “is-part-of,” which has implications on the creation, update, and deletion of these closely related entities. We discuss how to represent such constraints between entity types in our methodology for logical database design in Chapter 17, Step 2.4.

If we remember that the major aim of a data model is to unambiguously and accurately communicate an understanding of the enterprise data. We should only use the additional concepts of aggregation and composition when the enterprise data is too complex to easily represent using only the basic concepts of the ER model.

Chapter Summary

- A **superclass** is an entity type that includes one or more distinct subgroupings of its occurrences, which require to be represented in a data model. A **subclass** is a distinct subgrouping of occurrences of an entity type, which require to be represented in a data model.
- **Specialization** is the process of maximizing the differences between members of an entity by identifying their distinguishing features.
- **Generalization** is the process of minimizing the differences between entities by identifying their common features.
- There are two constraints that may apply to a specialization/generalization called **participation constraints** and **disjoint constraints**.
- A **participation constraint** determines whether every member in the superclass must participate as a member of a subclass.
- A **disjoint constraint** describes the relationship between members of the subclasses and indicates whether it is possible for a member of a superclass to be a member of one, or more than one, subclass.
- **Aggregation** represents a “has-a” or “is-part-of” relationship between entity types, where one represents the “whole” and the other the “part.”

- **Composition** is a specific form of aggregation that represents an association between entities, where there is a strong ownership and coincidental lifetime between the “whole” and the “part.”

Review Questions

- 13.1 What are the key differences between the ER and EER models?
- 13.2 Describe situations that would call for an enhanced entity–relationship in data modeling.
- 13.3 Describe and illustrate using an example the process of attribute inheritance.
- 13.4 What are the main reasons for introducing the concepts of superclasses and subclasses into an ER model?
- 13.5 Describe what a shared subclass represents and how this concept relates to multiple inheritance.
- 13.6 Describe and contrast the process of specialization with the process of generalization.
- 13.7 Describe the UML notation used to represent superclass/subclass relationships.
- 13.8 Describe and contrast the concepts of aggregation and composition and provide an example of each.

Exercises

- 13.9 Consider whether it is appropriate to introduce the enhanced concepts of specialization/generalization, aggregation, and/or composition for the case studies described in Appendix B.
- 13.10 The features of EER and ER models can co-exist in the same diagram. What situations lead to this co-existence? Analyze the case presented in question 12.11 and redraw the ER diagram as an EER diagram with the additional enhanced concepts.
- 13.11 Introduce specialization/generalization concepts into the ER model shown in Figure 13.11 and described in Exercise 12.13 to show the following:

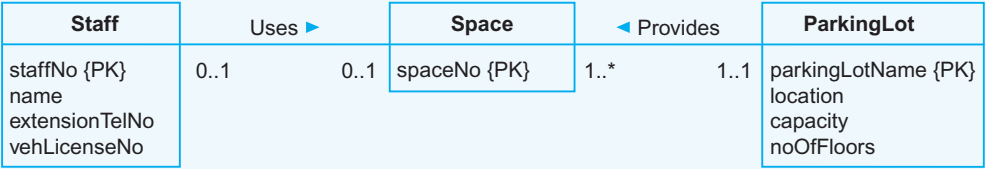


Figure 13.11 Parking lot ER model was described in Exercise 12.13.

- (a) The majority of parking spaces are under cover and each can be allocated for use by a member of staff for a monthly rate.
- (b) Parking spaces that are not under cover are free to use and each can be allocated for use by a member of staff.
- (c) Up to twenty covered parking spaces are available for use by visitors to the company. However, only members of staff are able to book out a space for the day of the visit. There is no charge for this type of booking, but the member of staff must provide the visitor's vehicle license number.

The final answer to this exercise is shown as Figure 17.11.

13.12 The library case study described in Exercise 12.14 is extended to include the fact that the library has a significant stock of books that are no longer suitable for loaning out. These books can be sold for a fraction of the original cost. However, not all library books are eventually sold as many are considered too damaged to sell on, or are simply lost or stolen. Each book copy that is suitable for selling has a price and the date that the book is no longer to be loaned out. Introduce enhanced concepts into the ER model shown in Figure 13.12 and described in Exercise 12.14 to accommodate this extension to the original case study.

The answer to this exercise is shown as Figure 17.12.

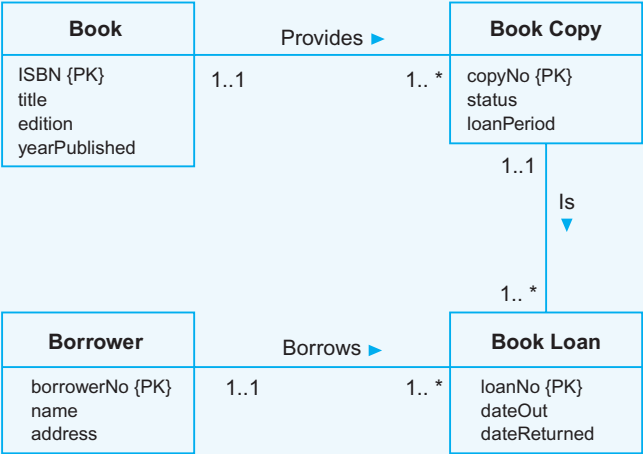


Figure 13.12 Library ER model was described in Exercise 12.14.

4 Normalization

Chapter Objectives

In this chapter you will learn:

- The purpose of normalization.
- How normalization can be used when designing a relational database.
- The potential problems associated with redundant data in base relations.
- The concept of functional dependency, which describes the relationship between attributes.
- The characteristics of functional dependencies used in normalization.
- How to identify functional dependencies for a given relation.
- How functional dependencies identify the primary key for a relation.
- How to undertake the process of normalization.
- How normalization uses functional dependencies to group attributes into relations that are in a known normal form.
- How to identify the most commonly used normal forms: First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).
- The problems associated with relations that break the rules of 1NF, 2NF, or 3NF.
- How to represent attributes shown on a form as 3NF relations using normalization.

When we design a database for an enterprise, the main objective is to create an accurate representation of the data, relationships between the data, and constraints on the data that is pertinent to the enterprise. To help achieve this objective, we can use one or more database design techniques. In Chapters 12 and 13 we described a technique called ER modeling. In this chapter and the next we describe another database design technique called **normalization**.

Normalization is a database design technique that begins by examining the relationships (called functional dependencies) between attributes. Attributes describe some property of the data or of the relationships between the data that is important to the enterprise. Normalization uses a series of tests (described as normal forms) to help identify the optimal grouping for these attributes to ultimately identify a set of suitable relations that supports the data requirements of the enterprise.

Although the main purpose of this chapter is to introduce the concept of functional dependencies and describe normalization up to Third Normal Form (3NF), in Chapter 15 we take a more formal look at functional dependencies and also consider later normal forms that go beyond 3NF.

Structure of this Chapter In Section 14.1 we describe the purpose of normalization. In Section 14.2 we discuss how normalization can be used to support relational database design. In Section 14.3 we identify and illustrate the potential problems associated with data redundancy in a base relation that is not normalized. In Section 14.4 we describe the main concept associated with normalization, called functional dependency, which describes the relationship between attributes. We also describe the characteristics of the functional dependencies that are used in normalization. In Section 14.5 we present an overview of normalization and then proceed in the following sections to describe the process involving the three most commonly used normal forms, namely First Normal Form (1NF) in Section 14.6, Second Normal Form (2NF) in Section 14.7, and Third Normal Form (3NF) in Section 14.8. The 2NF and 3NF described in these sections are based on the *primary key* of a relation. In Section 14.9 we present general definitions for 2NF and 3NF based on all *candidate keys* of a relation.

Throughout this chapter we use examples taken from the *DreamHome* case study described in Section 11.4 and documented in Appendix A.



14.1 The Purpose of Normalization

Normalization

A technique for producing a set of relations with desirable properties, given the data requirements of an enterprise.

The purpose of normalization is to identify a suitable set of relations that support the data requirements of an enterprise. The characteristics of a suitable set of relations include the following:

- the *minimal* number of attributes necessary to support the data requirements of the enterprise;
- attributes with a close logical relationship (described as functional dependency) are found in the same relation;
- *minimal* redundancy, with each attribute represented only once, with the important exception of attributes that form all or part of foreign keys (see Section 4.2.5), which are essential for the joining of related relations.

The benefits of using a database that has a suitable set of relations is that the database will be easier for the user to access and maintain the data, and take up

minimal storage space on the computer. The problems associated with using a relation that is not appropriately normalized is described later in Section 14.3.

14.2 How Normalization Supports Database Design

Normalization is a formal technique that can be used at any stage of database design. However, in this section we highlight two main approaches for using normalization, as illustrated in Figure 14.1. Approach 1 shows how normalization can be used as a bottom-up standalone database design technique, and Approach 2 shows how normalization can be used as a validation technique to check the structure of relations, which may have been created using a top-down approach such as ER modeling. No matter which approach is used, the goal is the same; creating a set of well-designed relations that meet the data requirements of the enterprise.

Figure 14.1 shows examples of data sources that can be used for database design. Although the users' requirements specification (see Section 10.5) is the preferred data source, it is possible to design a database based on the information taken directly from other data sources, such as forms and reports, as illustrated in this chapter and the next. Figure 14.1 also shows that the same data source can be used for both approaches; however, although this is true in principle, in practice the approach taken is likely to be determined by the size, extent, and complexity of the database being described by the data sources and by the preference and expertise of the database designer. The opportunity to use

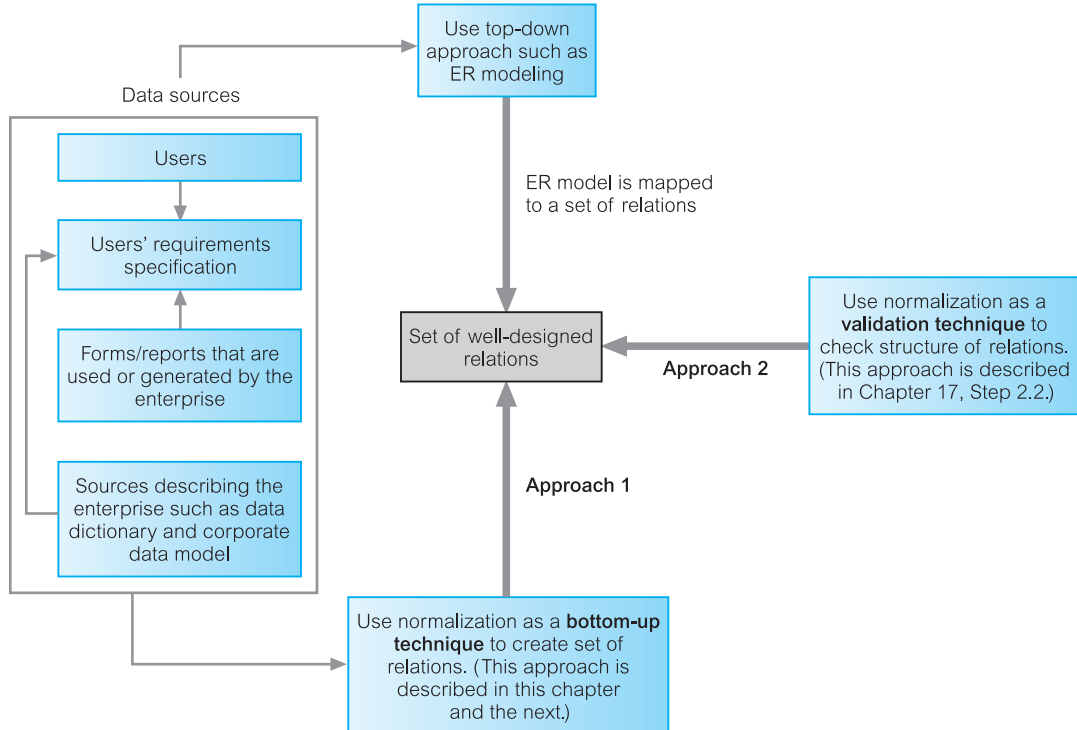


Figure 14.1 How normalization can be used to support database design.

normalization as a bottom-up standalone technique (Approach 1) is often limited by the level of detail that the database designer is reasonably expected to manage. However, this limitation is not applicable when normalization is used as a validation technique (Approach 2), as the database designer focuses on only part of the database, such as a single relation, at any one time. Therefore, no matter what the size or complexity of the database, normalization can be usefully applied.

I4.3 Data Redundancy and Update Anomalies

As stated in Section 14.1, a major aim of relational database design is to group attributes into relations to minimize data redundancy. If this aim is achieved, the potential benefits for the implemented database include the following:

- updates to the data stored in the database are achieved with a minimal number of operations, thus reducing the opportunities for data inconsistencies occurring in the database;
- reduction in the file storage space required by the base relations thus minimizing costs.

Of course, relational databases also rely on the existence of a certain amount of data redundancy. This redundancy is in the form of copies of primary keys (or candidate keys) acting as foreign keys in related relations to enable the modeling of relationships between data.

In this section we illustrate the problems associated with unwanted data redundancy by comparing the Staff and Branch relations shown in Figure 14.2 with the StaffBranch relation shown in Figure 14.3. The StaffBranch relation is an alternative format of the Staff and Branch relations. The relations have the following form:

Staff (staffNo, sName, position, salary, branchNo)

Branch (branchNo, bAddress)

StaffBranch (staffNo, sName, position, salary, branchNo, bAddress)

Figure 14.2
Staff and Branch
relations.

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

StaffBranch

<u>staffNo</u>	<u>sName</u>	<u>position</u>	<u>salary</u>	<u>branchNo</u>	<u>bAddress</u>
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Figure 14.3

StaffBranch relation.

Note that the primary key for each relation is underlined.

In the StaffBranch relation there is redundant data; the details of a branch are repeated for every member of staff located at that branch. In contrast, the branch details appear only once for each branch in the Branch relation, and only the branch number (branchNo) is repeated in the Staff relation to represent where each member of staff is located. Relations that have redundant data may have problems called **update anomalies**, which are classified as insertion, deletion, or modification anomalies.

14.3.1 Insertion Anomalies

There are two main types of insertion anomaly, which we illustrate using the StaffBranch relation shown in Figure 14.3:

- To insert the details of new members of staff into the StaffBranch relation, we must include the details of the branch at which the staff are to be located. For example, to insert the details of new staff located at branch number B007, we must enter the correct details of branch number B007 so that the branch details are consistent with values for branch B007 in other tuples of the StaffBranch relation. The relations shown in Figure 14.2 do not suffer from this potential inconsistency, because we enter only the appropriate branch number for each staff member in the Staff relation. Instead, the details of branch number B007 are recorded in the database as a single tuple in the Branch relation.
- To insert details of a new branch that currently has no members of staff into the StaffBranch relation, it is necessary to enter nulls into the attributes for staff, such as staffNo. However, as staffNo is the primary key for the StaffBranch relation, attempting to enter nulls for staffNo violates entity integrity (see Section 4.3), and is not allowed. We therefore cannot enter a tuple for a new branch into the StaffBranch relation with a null for the staffNo. The design of the relations shown in Figure 14.2 avoids this problem, because branch details are entered in the Branch relation separately from the staff details. The details of staff ultimately located at that branch are entered at a later date into the Staff relation.

14.3.2 Deletion Anomalies

If we delete a tuple from the StaffBranch relation that represents the last member of staff located at a branch, the details about that branch are also lost from the database. For example, if we delete the tuple for staff number SA9 (Mary Howe)

from the `StaffBranch` relation, the details relating to branch number B007 are lost from the database. The design of the relations in Figure 14.2 avoids this problem, because branch tuples are stored separately from staff tuples and only the attribute `branchNo` relates the two relations. If we delete the tuple for staff number SA9 from the `Staff` relation, the details on branch number B007 remain unaffected in the `Branch` relation.

14.3.3 Modification Anomalies

If we want to change the value of one of the attributes of a particular branch in the `StaffBranch` relation—for example, the address for branch number B003—we must update the tuples of all staff located at that branch. If this modification is not carried out on all the appropriate tuples of the `StaffBranch` relation, the database will become inconsistent. In this example, branch number B003 may appear to have different addresses in different staff tuples.

The previous examples illustrate that the `Staff` and `Branch` relations of Figure 14.2 have more desirable properties than the `StaffBranch` relation of Figure 14.3. This demonstrates that although the `StaffBranch` relation is subject to update anomalies, we can avoid these anomalies by decomposing the original relation into the `Staff` and `Branch` relations. There are two important properties associated with decomposition of a larger relation into smaller relations:

- The **lossless-join** property ensures that any instance of the original relation can be identified from corresponding instances in the smaller relations.
- The **dependency preservation** property ensures that a constraint on the original relation can be maintained by simply enforcing some constraint on each of the smaller relations. In other words, we do not need to perform joins on the smaller relations to check whether a constraint on the original relation is violated.

Later in this chapter, we discuss how the process of normalization can be used to derive well-formed relations. However, we first introduce functional dependencies, which are fundamental to the process of normalization.

14.4 Functional Dependencies

An important concept associated with normalization is **functional dependency**, which describes the relationship between attributes (Maier, 1983). In this section we describe functional dependencies and then focus on the particular characteristics of functional dependencies that are useful for normalization. We then discuss how functional dependencies can be identified and used to identify the primary key for a relation.

14.4.1 Characteristics of Functional Dependencies

For the discussion on functional dependencies, assume that a relational schema has attributes $(A, B, C, \dots, Z)$ and that the database is described by a single **universal relation** called $R = (A, B, C, \dots, Z)$. This assumption means that every attribute in the database has a unique name.

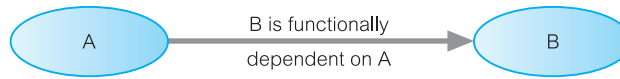


Figure 14.4
A functional
dependency
diagram.

Functional dependency

Describes the relationship between attributes in a relation. For example, if A and B are attributes of relation R, B is functionally dependent on A (denoted $A \twoheadrightarrow B$), if each value of A is associated with exactly one value of B. (A and B may each consist of one or more attributes.)

Functional dependency is a property of the meaning or semantics of the attributes in a relation. The semantics indicate how attributes relate to one another, and specify the functional dependencies between attributes. When a functional dependency is present, the dependency is specified as a **constraint** between the attributes.

Consider a relation with attributes A and B, where attribute B is functionally dependent on attribute A. If we know the value of A and we examine the relation that holds this dependency, we find only one value of B in all the tuples that have a given value of A, at any moment in time. Thus, when two tuples have the same value of A, they also have the same value of B. However, for a given value of B, there may be several different values of A. The dependency between attributes A and B can be represented diagrammatically, as shown Figure 14.4.

An alternative way to describe the relationship between attributes A and B is to say that “A functionally determines B.” Some readers may prefer this description, as it more naturally follows the direction of the functional dependency arrow between the attributes.

Determinant

Refers to the attribute, or group of attributes, on the left-hand side of the arrow of a functional dependency.

When a functional dependency exists, the attribute or group of attributes on the left-hand side of the arrow is called the **determinant**. For example, in Figure 14.4, A is the determinant of B. We demonstrate the identification of a functional dependency in the following example.

EXAMPLE 14.1 An example of a functional dependency

Consider the attributes `staffNo` and `position` of the `Staff` relation in Figure 14.2. For a specific `staffNo`—for example, SL21—we can determine the position of that member of staff as Manager. In other words, `staffNo` functionally determines `position`, as shown in Figure 14.5(a). However, Figure 14.5(b) illustrates that the opposite is not true, as `position` does not functionally determine `staffNo`. A member of staff holds one position; however, there may be several members of staff with the same position.

The relationship between `staffNo` and `position` is one-to-one (1:1): for each staff number there is only one position. On the other hand, the relationship between `position` and `staffNo` is one-to-many (1:*) : there are several staff numbers associated with a given position. In this example, `staffNo` is the determinant of this functional dependency. For the purposes of normalization, we are interested in identifying functional dependencies between attributes of a relation that have a one-to-one relationship between the attribute(s) that

makes up the determinant on the left-hand side and the attribute(s) on the right-hand side of a dependency.

When identifying functional dependencies between attributes in a relation, it is important to distinguish clearly between the values held by an attribute at a given point in time and the *set of all possible values* that an attribute may hold at different times. In other words, a functional dependency is a property of a relational schema (intension) and not a property of a particular instance of the schema (extension) (see Section 4.2.1). This point is illustrated in the following example.

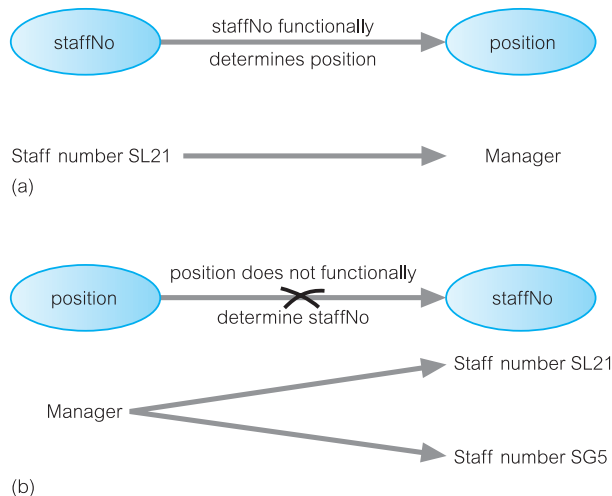


Figure 14.5 (a) staffNo functionally determines position (staffNo @ position); (b) position does not functionally determine staffNo (position @ staffNo).

EXAMPLE 14.2 Example of a functional dependency that holds for all time

Consider the values shown in **staffNo** and **sName** attributes of the **Staff** relation in Figure 14.2. We see that for a specific **staffNo**—SL21—for example, we can determine the name of that member of staff as John White. Furthermore, it appears that for a specific **sName**—for example, John White—we can determine the staff number for that member of staff as SL21. Can we therefore conclude that the **staffNo** attribute functionally determines the **sName** attribute and/or that the **sName** attribute functionally determines the **staffNo** attribute? If the values shown in the **Staff** relation of Figure 14.2 represent the *set of all possible values* for **staffNo** and **sName** attributes, then the following functional dependencies hold:

staffNo @ sName
sName @ staffNo

However, if the values shown in the **Staff** relation of Figure 14.2 simply represent a *set of values* for **staffNo** and **sName** attributes at a given moment in time, then we are not so interested in such relationships between attributes. The reason is that we want to identify functional dependencies that hold for all possible values for attributes of a relation as these represent the types of integrity constraints that we need to identify. Such constraints indicate the limitations on the values that a relation can legitimately assume.

One approach to identifying the set of all possible values for attributes in a relation is to more clearly understand the purpose of each attribute in that relation. For example,

the purpose of the values held in the **staffNo** attribute is to uniquely identify each member of staff, whereas the purpose of the values held in the **sName** attribute is to hold the names of members of staff. Clearly, the statement that if we know the staff number (**staffNo**) of a member of staff, we can determine the name of the member of staff (**sName**) remains true. However, as it is possible for the **sName** attribute to hold duplicate values for members of staff with the same name, then we would not be able to determine the staff number (**staffNo**) of some members of staff in this category. The relationship between **staffNo** and **sName** is one-to-one (1:1): for each staff number there is only one name. On the other hand, the relationship between **sName** and **staffNo** is one-to-many (1:*) : there can be several staff numbers associated with a given name. The functional dependency that remains true after consideration of all possible values for the **staffNo** and **sName** attributes of the **Staff** relation is:

staffNo $\twoheadrightarrow$ **sName**

An additional characteristic of functional dependencies that is useful for normalization is that their determinants should have the minimal number of attributes necessary to maintain the functional dependency with the attribute(s) on the righthand side. This requirement is called **full functional dependency**.

**Full
functional
dependency**

Indicates that if A and B are attributes of a relation, B is fully functionally dependent on A if B is functionally dependent on A, but not on any proper subset of A.

A functional dependency $A \twoheadrightarrow B$ is a *full* functional dependency if removal of any attribute from A results in the dependency no longer existing. A functional dependency $A \twoheadrightarrow B$ is a **partial dependency** if there is some attribute that can be removed from A and yet the dependency still holds. An example of how a full functional dependency is derived from a partial functional dependency is presented in Example 14.3.

EXAMPLE 14.3 Example of a full functional dependency

Consider the following functional dependency that exists in the **Staff** relation of Figure 14.2:

staffNo, sName $\twoheadrightarrow$ **branchNo**

It is correct to say that each value of (**staffNo, sName**) is associated with a single value of **branchNo**. However, it is not a full functional dependency, because **branchNo** is also functionally dependent on a subset of (**staffNo, sName**), namely **staffNo**. In other words, the functional dependency shown in the example is an example of a partial dependency. The type of functional dependency that we are interested in identifying is a full functional dependency as shown here:

staffNo $\twoheadrightarrow$ **branchNo**

Additional examples of partial and full functional dependencies are discussed in Section 14.7.

In summary, the functional dependencies that we use in normalization have the following characteristics:

- There is a *one-to-one* relationship between the attribute(s) on the left-hand side (determinant) and those on the right-hand side of a functional dependency. (Note that the relationship in the opposite direction—that is, from the right-hand to the left-hand side attributes—can be a one-to-one relationship or one-to-many relationship.)
- They hold for *all* time.
- The determinant has the *minimal* number of attributes necessary to maintain the dependency with the attribute(s) on the right-hand side. In other words, there must be a full functional dependency between the attribute(s) on the left-hand and right-hand sides of the dependency.

So far we have discussed functional dependencies that we are interested in for the purposes of normalization. However, there is an additional type of functional dependency called a **transitive dependency** that we need to recognize, because its existence in a relation can potentially cause the types of update anomaly discussed in Section 14.3. In this section we simply describe these dependencies so that we can identify them when necessary.

Transitive dependency

A condition where A, B, and C are attributes of a relation such that if $A \twoheadrightarrow B$ and $B \twoheadrightarrow C$, then C is transitively dependent on A via B (provided that A is not functionally dependent on B or C).

An example of a transitive dependency is provided in Example 14.4.

EXAMPLE 14.4 Example of a transitive functional dependency

Consider the following functional dependencies within the `StaffBranch` relation shown in Figure 14.3:

`staffNo` $\twoheadrightarrow$ `sName`, `position`, `salary`, `branchNo`, `bAddress`
`branchNo` $\twoheadrightarrow$ `bAddress`

The transitive dependency `branchNo` $\twoheadrightarrow$ `bAddress` exists on `staffNo` via `branchNo`. In other words, the `staffNo` attribute functionally determines the `bAddress` via the `branchNo` attribute and neither `branchNo` nor `bAddress` functionally determines `staffNo`. An additional example of a transitive dependency is discussed in Section 14.8.

In the following sections we demonstrate approaches to identifying a set of functional dependencies and then discuss how these dependencies can be used to identify a primary key for the example relations.

14.4.2 Identifying Functional Dependencies

Identifying all functional dependencies between a set of attributes should be quite simple if the meaning of each attribute and the relationships between the attributes are well understood. This type of information may be provided by the enterprise in

the form of discussions with users and/or appropriate documentation, such as the users' requirements specification. However, if the users are unavailable for consultation and/or the documentation is incomplete, then—depending on the database application—it may be necessary for the database designer to use their common sense and/or experience to provide the missing information. Example 14.5 illustrates how easy it is to identify functional dependencies between attributes of a relation when the purpose of each attribute and the attributes' relationships are well understood.

EXAMPLE 14.5 Identifying a set of functional dependencies for the **StaffBranch** relation

We begin by examining the semantics of the attributes in the **StaffBranch** relation shown in Figure 14.3. For the purposes of discussion, we assume that the position held and the branch determine a member of staff's salary. We identify the functional dependencies based on our understanding of the attributes in the relation as:

staffNo $\twoheadrightarrow$ sName, position, salary, branchNo, bAddress
branchNo $\twoheadrightarrow$ bAddress
bAddress $\twoheadrightarrow$ branchNo
branchNo, position $\twoheadrightarrow$ salary
bAddress, position $\twoheadrightarrow$ salary

We identify five functional dependencies in the **StaffBranch** relation with staffNo, branchNo, bAddress, (branchNo, position), and (bAddress, position) as determinants. For each functional dependency, we ensure that *all* the attributes on the right-hand side are functionally dependent on the determinant on the left-hand side.

As a contrast to this example, we now consider the situation where functional dependencies are to be identified in the absence of appropriate information about the meaning of attributes and their relationships. In this case, it may be possible to identify functional dependencies if sample data is available that is a true representation of *all* possible data values that the database may hold. We demonstrate this approach in Example 14.6.

EXAMPLE 14.6 Using sample data to identify functional dependencies

Consider the data for attributes denoted A, B, C, D, and E in the **Sample** relation of Figure 13.6. It is important first to establish that the data values shown in this relation are representative of all possible values that can be held by attributes A, B, C, D, and E. For the purposes of this example, let us assume that this is true despite the relatively small amount of data shown in this relation. The process of identifying the functional dependencies (denoted fd1 to fd5) that exist between the attributes of the **Sample** relation shown in Figure 14.6 is described next.

To identify the functional dependencies that exist between attributes A, B, C, D, and E, we examine the **Sample** relation shown in Figure 14.6 and identify when values in one column are consistent with the presence of particular values in other columns. We begin with the first column on the left-hand side and work our way over to the right-hand side of the relation and then we look at combinations of columns; in other words, where values in two or more columns are consistent with the appearance of values in other columns.

Sample Relation

A	B	C	D	E
a	b	z	w	q
e	b	r	w	p
a	d	z	w	t
e	d	r	w	q
a	f	z	s	t
e	f	r	s	t

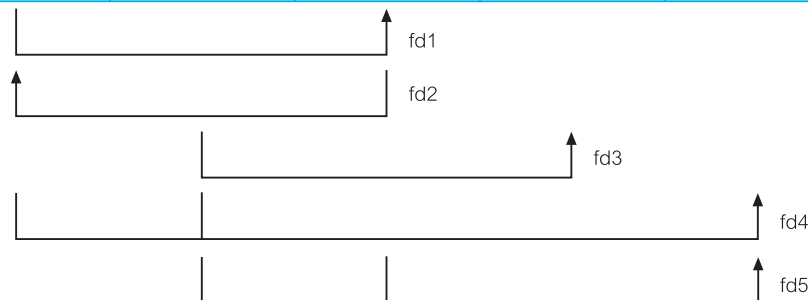


Figure 14.6 The Sample relation displaying data for attributes A, B, C, D, and E and the functional dependencies (fd1 to fd5) that exist between these attributes.

For example, when the value “a” appears in column A, the value “z” appears in column C, and when “e” appears in column A, the value “r” appears in column C. We can therefore conclude that there is a one-to-one (1:1) relationship between attributes A and C. In other words, attribute A functionally determines attribute C and this is shown as functional dependency 1 (fd1) in Figure 14.6. Furthermore, as the values in column C are consistent with the appearance of particular values in column A, we can also conclude that there is a (1:1) relationship between attributes C and A. In other words, C functionally determines A, and this is shown as fd2 in Figure 14.6. If we now consider attribute B, we can see that when “b” or “d” appears in column B, then “w” appears in column D and when “f” appears in column B, then “s” appears in column D. We can therefore conclude that there is a (1:1) relationship between attributes B and D. In other words, B functionally determines D, and this is shown as fd3 in Figure 14.6. However, attribute D does not functionally determine attribute B as a single unique value in column D, such as “w” is not associated with a single consistent value in column B. In other words, when “w” appears in column D, the values “b” or “d” appears in column B. Hence, there is a one-to-many relationship between attributes D and B. The final single attribute to consider is E, and we find that the values in this column are not associated with the consistent appearance of particular values in the other columns. In other words, attribute E does not functionally determine attributes A, B, C, or D.

We now consider combinations of attributes and the appearance of consistent values in other columns. We conclude that unique combination of values in columns A and B such as (a, b) is associated with a single value in column E, which in this example is “q.” In other words attributes (A, B) functionally determines attribute E, and this is shown as fd4 in Figure 14.6. However, the reverse is not true, as we have already stated that attribute E, does not functionally determine any other attribute in the relation. We also conclude that attributes (B, C) functionally determine attribute E using the same reasoning described earlier, and this functional dependency is shown as fd5 in

Figure 14.6. We complete the examination of the relation shown in Figure 14.6 by considering all the remaining combinations of columns.

In summary, we describe the function dependencies between attributes A to E in the Sample relation shown in Figure 14.6 as follows:

A $\twoheadrightarrow$ C (fd1)
 C $\twoheadrightarrow$ A (fd2)
 B $\twoheadrightarrow$ D (fd3)
 A, B $\twoheadrightarrow$ E (fd4)
 B, C $\twoheadrightarrow$ E (fd5)

14.4.3 Identifying the Primary Key for a Relation Using Functional Dependencies

The main purpose of identifying a set of functional dependencies for a relation is to specify the set of integrity constraints that must hold on a relation. An important integrity constraint to consider first is the identification of candidate keys, one of which is selected to be the primary key for the relation. We demonstrate the identification of a primary key for a given relation in the following two examples.

EXAMPLE 14.7 Identifying the primary key for the StaffBranch relation

In Example 14.5 we describe the identification of five functional dependencies for the StaffBranch relation shown in Figure 14.3. The determinants for these functional dependencies are staffNo, branchNo, bAddress, (branchNo, position), and (bAddress, position).

To identify the candidate key(s) for the StaffBranch relation, we must identify the attribute (or group of attributes) that uniquely identifies each tuple in this relation. If a relation has more than one candidate key, we identify the candidate key that is to act as the primary key for the relation (see Section 4.2.5). All attributes that are not part of the primary key (non-primary-key attributes) should be functionally dependent on the key.

The only candidate key of the StaffBranch relation, and therefore the primary key, is staffNo, as *all* other attributes of the relation are functionally dependent on staffNo. Although branchNo, bAddress, (branchNo, position), and (bAddress, position) are determinants in this relation, they are not candidate keys for the relation.

EXAMPLE 14.8 Identifying the primary key for the Sample relation

In Example 14.6 we identified five functional dependencies for the Sample relation. We examine the determinant for each functional dependency to identify the candidate key(s) for the relation. A suitable determinant must functionally determine the other attributes in the relation. The determinants in the Sample relation are A, B, C, (A, B), and (B, C). However, the only determinants that determine all the other attributes of the relation are (A, B) and (B, C). In the case of (A, B), A functionally determines C, B functionally determines D, and (A, B) functionally determines E. In other words, the attributes that make up the determinant (A, B) can determine all the other attributes in the relation either separately as A or B or together as (A, B). Hence, we see that an essential characteristic for a candidate key of a relation is that the attributes of a determinant

either individually or working together must be able to functionally determine *all* the other attributes in the relation. This characteristic is also true for the determinant (B, C), but is not a characteristic of the other determinants in the **Sample** relation (namely A, B, or C), as in each case they can determine only one other attribute in the relation. In conclusion, there are two candidate keys for the **Sample** relation; namely, (A, B) and (B, C), and as each has similar characteristics (such as number of attributes), the selection of primary key for the **Sample** relation is arbitrary. The candidate key not selected to be the primary key is referred to as the alternate key for the **Sample** relation.

So far in this section we have discussed the types of functional dependency that are most useful in identifying important constraints on a relation and how these dependencies can be used to identify a primary key (or candidate keys) for a given relation. The concepts of functional dependencies and keys are central to the process of normalization. We continue the discussion on functional dependencies in the next chapter for readers interested in a more formal coverage of this topic. However, in this chapter, we continue by describing the process of normalization.

14.5 The Process of Normalization

Normalization is a formal technique for analyzing relations based on their primary key (or candidate keys) and functional dependencies (Codd, 1972b). The technique involves a series of rules that can be used to test individual relations so that a database can be normalized to any degree. When a requirement is not met, the relation violating the requirement must be decomposed into relations that individually meet the requirements of normalization.

Three normal forms were initially proposed called First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF). Subsequently, R. Boyce and E. F. Codd introduced a stronger definition of third normal form called Boyce–Codd Normal Form (BCNF) (Codd, 1974). With the exception of 1NF, all these normal forms are based on functional dependencies among the attributes of a relation (Maier, 1983). Higher normal forms that go beyond BCNF were introduced later such as Fourth Normal Form (4NF) and Fifth Normal Form (5NF) (Fagin, 1977, 1979). However, these later normal forms deal with situations that are very rare. In this chapter we describe only the first three normal forms and leave discussion of BCNF, 4NF, and 5NF to the next chapter.

Normalization is often executed as a series of steps. Each step corresponds to a specific normal form that has known properties. As normalization proceeds, the relations become progressively more restricted (stronger) in format and also less vulnerable to update anomalies. For the relational data model, it is important to recognize that it is only First Normal Form (1NF) that is critical in creating update anomalies; all subsequent normal forms are optional. However, to avoid the update anomalies discussed in Section 14.3, it is generally recommended that we proceed to at least Third Normal Form (3NF). Figure 14.7 illustrates the relationship between the various normal forms. It shows that some 1NF relations are also in 2NF, and that some 2NF relations are also in 3NF, and so on.

In the following sections we describe the process of normalization in detail. Figure 14.8 provides an overview of the process and highlights the main actions

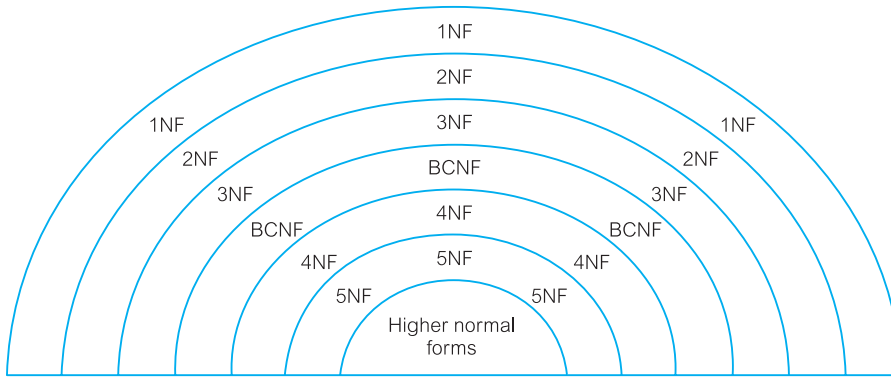


Figure 14.7 Diagrammatic illustration of the relationship between the normal forms.

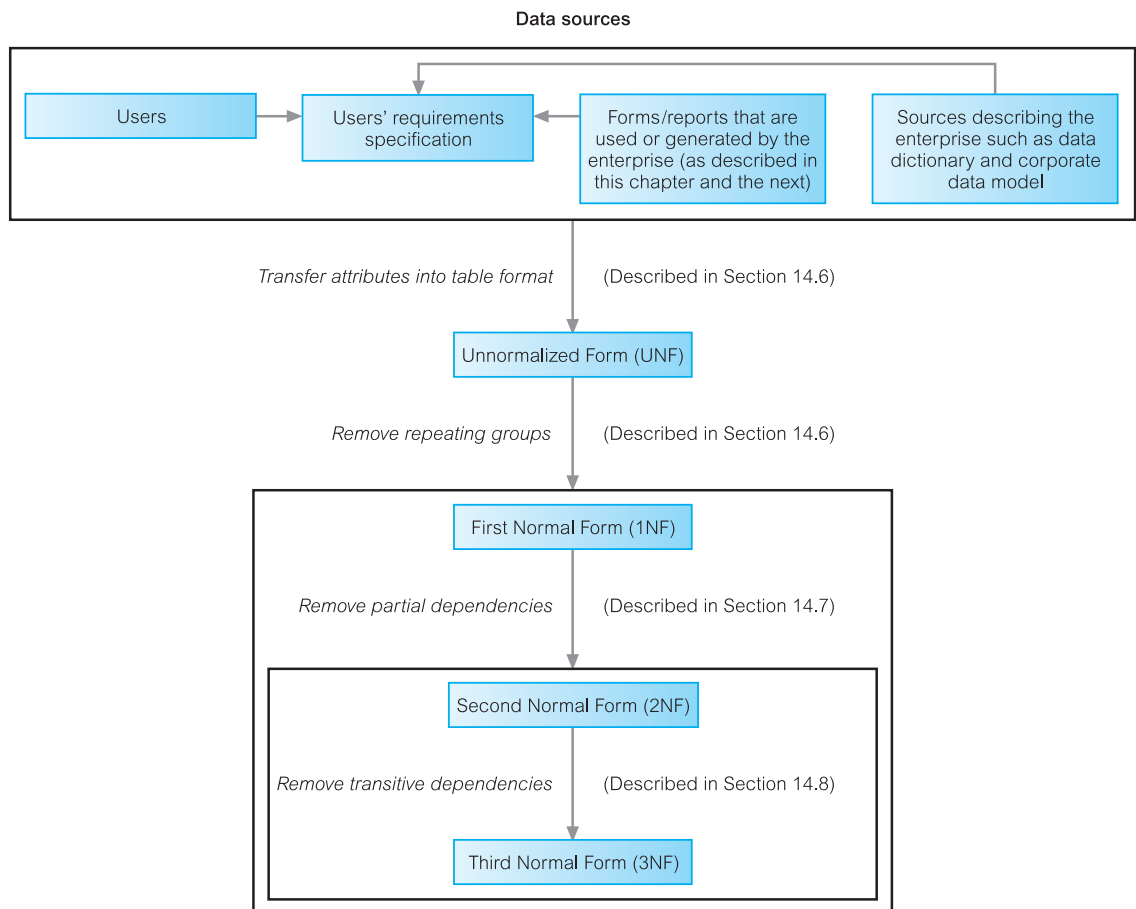


Figure 14.8 Diagrammatic illustration of the process of normalization.

taken in each step of the process. The number of the section that covers each step of the process is also shown in this figure.

In this chapter, we describe normalization as a bottom-up technique extracting information about attributes from sample forms that are first transformed into table format, which is described as being in Unnormalized Form (UNF). This table is then subjected progressively to the different requirements associated with each normal form until ultimately the attributes shown in the original sample forms are represented as a set of 3NF relations. Although the example used in this chapter proceeds from a given normal form to the one above, this is not necessarily the case with other examples. As shown in Figure 13.8, the resolution of a particular problem with, say, a 1NF relation may result in the relation being transformed to 2NF relations, or in some cases directly into 3NF relations in one step.

To simplify the description of normalization we assume that a set of functional dependencies is given for each relation in the worked examples and that each relation has a designated primary key. In other words, it is essential that the meaning of the attributes and their relationships is well understood before beginning the process of normalization. This information is fundamental to normalization and is used to test whether a relation is in a particular normal form. In Section 14.6 we begin by describing First Normal Form (1NF). In Sections 14.7 and 14.8 we describe Second Normal Form (2NF) and Third Normal Forms (3NF) based on the *primary key* of a relation and then present a more general definition of each in Section 14.9. The more general definitions of 2NF and 3NF take into account all *candidate keys* of a relation, rather than just the primary key.



14.6 First Normal Form (1NF)

Before discussing First Normal Form, we provide a definition of the state prior to First Normal Form.

Unnormalized Form (UNF)	A table that contains one or more repeating groups.
First Normal Form (1NF)	A relation in which the intersection of each row and column contains one and only one value.

In this chapter, we begin the process of normalization by first transferring the data from the source (for example, a standard data entry form) into table format with rows and columns. In this format, the table is in unnormalized Form and is referred to as an **unnormalized table**. To transform the unnormalized table to First Normal Form, we identify and remove repeating groups within the table. A repeating group is an attribute, or group of attributes, within a table that occurs with multiple values for a single occurrence of the nominated key attribute(s) for that table. Note that in this context, the term “key” refers to the attribute(s) that uniquely identify each row within the Unnormalized table. There are two common approaches to removing repeating groups from unnormalized tables:

- 1) *By entering appropriate data in the empty columns of rows containing the repeating data.* In other words, we fill in the blanks by duplicating the nonrepeating data, where required. This approach is commonly referred to as “flattening” the table.

(2) By placing the repeating data, along with a copy of the original key attribute(s), in a separate relation. Sometimes the unnormalized table may contain more than one repeating group, or repeating groups within repeating groups. In such cases, this approach is applied repeatedly until no repeating groups remain. A set of relations is in 1NF if it contains no repeating groups.

For both approaches, the resulting tables are now referred to as 1NF relations containing atomic (or single) values at the intersection of each row and column. Although both approaches are correct, approach 1 introduces more redundancy into the original UNF table as part of the “flattening” process, whereas approach 2 creates two or more relations with less redundancy than in the original UNF table. In other words, approach 2 moves the original UNF table further along the normalization process than approach 1. However, no matter which initial approach is taken, the original UNF table will be normalized into the same set of 3NF relations.

We demonstrate both approaches in the following worked example using the *DreamHome* case study.

EXAMPLE 14.9 First Normal Form (1NF)

A collection of (simplified) *DreamHome* leases is shown in Figure 14.9. The lease on top is for a client called John Kay who is leasing a property in Glasgow, which is owned by Tina Murphy. For this worked example, we assume that a client rents a given property only once and cannot rent more than one property at any one time.



DreamHome Lease	
DreamHome Lease	
DreamHome Lease	
DreamHome Lease	
Client Number <u>CR76</u> (Enter if known) Full Name <u>John Kay</u> (Please print)	Property Number <u>PG4</u> Property Address <u>6 Lawrence St, Glasgow</u>
Monthly Rent <u>350</u> Rent Start <u>01/07/12</u> Rent Finish <u>31/08/13</u>	Owner Number <u>CO40</u> (Enter if known) Full Name <u>Tina Murphy</u> (Please print)

Figure 14.9 Collection of (simplified) *DreamHome* leases.

Sample data is taken from two leases for two different clients called John Kay and Aline Stewart and is transformed into table format with rows and columns, as shown in Figure 14.10. This is an example of an unnormalized table.

ClientRental								
clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-12	31-Aug-13	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-13	1-Sep-14	50	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-11	10-June-12	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-12	1-Dec-13	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-14	10-Aug-15	450	CO93	Tony Shaw

Figure 14.10 ClientRental unnormalized table.

We identify the key attribute for the ClientRental unnormalized table as clientNo. Next, we identify the repeating group in the unnormalized table as the property rented details, which repeats for each client. The structure of the repeating group is:

Repeating Group = (propertyNo, pAddress, rentStart, rentFinish, rent, ownerNo, oName)

As a consequence, there are multiple values at the intersection of certain rows and columns. For example, there are two values for propertyNo (PG4 and PG16) for the client named John Kay. To transform an unnormalized table into 1NF, we ensure that there is a single value at the intersection of each row and column. This is achieved by removing the repeating group.

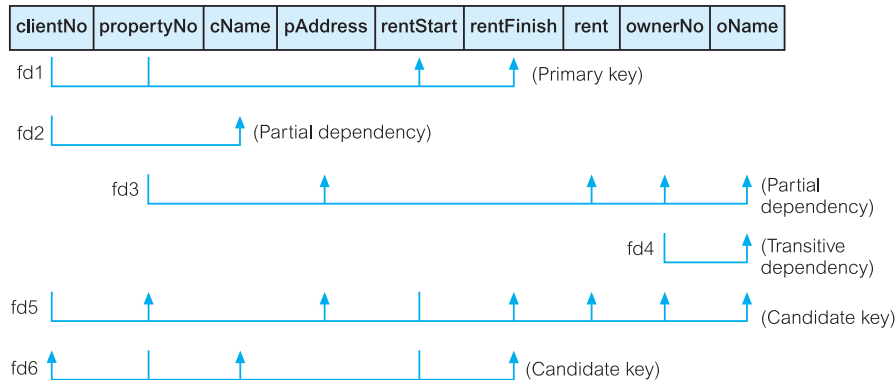
With the first approach, we remove the repeating group (property rented details) by entering the appropriate client data into each row. The resulting first normal form ClientRental relation is shown in Figure 14.11.

ClientRental								
clientNo	propertyNo	cName	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	PG4	John Kay	6 Lawrence St, Glasgow	1-Jul-12	31-Aug-13	350	CO40	Tina Murphy
CR76	PG16	John Kay	5 Novar Dr, Glasgow	1-Sep-13	1-Sep-14	450	CO93	Tony Shaw
CR56	PG4	Aline Stewart	6 Lawrence St, Glasgow	1-Sep-11	10-Jun-12	350	CO40	Tina Murphy
CR56	PG36	Aline Stewart	2 Manor Rd, Glasgow	10-Oct-12	1-Dec-13	375	CO93	Tony Shaw
CR56	PG16	Aline Stewart	5 Novar Dr, Glasgow	1-Nov-14	10-Aug-15	450	CO93	Tony Shaw

Figure 14.11 First Normal Form ClientRental relation.

In Figure 14.12, we present the functional dependencies (fd1 to fd6) for the ClientRental relation. We use the functional dependencies (as discussed in Section 14.4.3) to identify candidate keys for the ClientRental relation as being composite keys comprising (clientNo, propertyNo), (clientNo, rentStart), and (propertyNo, rentStart). We select (clientNo, propertyNo) as the primary key for the relation, and for clarity we place the attributes that make up the primary key together at the left-hand side of the relation. In this example, we assume that the rentFinish attribute is not appropriate as a component of a candidate key as it may contain nulls (see Section 4.3.1).

ClientRental

**Figure 14.12** Functional dependencies of the ClientRental relation.

The ClientRental relation is defined as follows:

ClientRental (clientNo, propertyNo, cName, pAddress, rentStart, rentFinish, rent, ownerNo, oName)

The ClientRental relation is in 1NF, as there is a single value at the intersection of each row and column. The relation contains data describing clients, property rented, and property owners, which is repeated several times. As a result, the ClientRental relation contains significant data redundancy. If implemented, the 1NF relation would be subject to the update anomalies described in Section 14.3. To remove some of these, we must transform the relation into second normal form, which we discuss shortly.

With the second approach, we remove the repeating group (property rented details) by placing the repeating data along with a copy of the original key attribute (clientNo) in a separate relation, as shown in Figure 14.13.

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

PropertyRentalOwner

clientNo	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	PG4	6 Lawrence St, Glasgow	1-Jul-12	31-Aug-13	350	CO40	Tina Murphy
CR76	PG16	5 Novar Dr, Glasgow	1-Sep-13	1-Sep-14	450	CO93	Tony Shaw
CR56	PG4	6 Lawrence St, Glasgow	1-Sep-11	10-Jun-12	350	CO40	Tina Murphy
CR56	PG36	2 Manor Rd, Glasgow	10-Oct-12	1-Dec-13	375	CO93	Tony Shaw
CR56	PG16	5 Novar Dr, Glasgow	1-Nov-14	10-Aug-15	450	CO93	Tony Shaw

Figure 14.13 Alternative 1NF Client and PropertyRental-Owner relations.

With the help of the functional dependencies identified in Figure 14.12 we identify a primary key for the relations. The format of the resulting 1NF relations are as follows:

Client (clientNo, cName)
 PropertyRentalOwner (clientNo, propertyNo, pAddress, rentStart, rentFinish, rent, ownerNo, oName)

The Client and PropertyRentalOwner relations are both in 1NF, as there is a single value at the intersection of each row and column. The Client relation contains data describing clients and the PropertyRentalOwner relation contains data describing property rented by clients and property owners. However, as we see from Figure 14.13, this relation also contains some redundancy and as a result may suffer from similar update anomalies to those described in Section 14.3.

To demonstrate the process of normalizing relations from 1NF to 2NF, we use only the ClientRental relation shown in Figure 14.11. However, recall that both approaches are correct, and will ultimately result in the production of the same relations as we continue the process of normalization. We leave the process of completing the normalization of the Client and PropertyRentalOwner relations as an exercise for the reader, which is given at the end of this chapter.

14.7 Second Normal Form (2NF)

Second Normal Form (2NF) is based on the concept of full functional dependency, which we described in Section 14.4. Second normal form applies to relations with composite keys, that is, relations with a primary key composed of two or more attributes. A relation with a single-attribute primary key is automatically in at least 2NF. A relation that is not in 2NF may suffer from the update anomalies discussed in Section 14.3. For example, suppose we wish to change the rent of property number PG4. We have to update two tuples in the ClientRental relation in Figure 14.11. If only one tuple is updated with the new rent, this results in an inconsistency in the database.

Second Normal Form (2NF)

A relation that is in first normal form and every non-primary-key attribute is fully functionally dependent on the primary key.

The normalization of 1NF relations to 2NF involves the removal of partial dependencies. If a partial dependency exists, we remove the partially dependent attribute(s) from the relation by placing them in a new relation along with a copy of their determinant. We demonstrate the process of converting 1NF relations to 2NF relations in the following example.

EXAMPLE 14.10 Second Normal Form (2NF)

As shown in Figure 14.12, the ClientRental relation has the following functional dependencies:

- | | | |
|-----|--|-------------------------|
| fd1 | clientNo, propertyNo ® rentStart, rentFinish | (Primary key) |
| fd2 | clientNo ® cName | (Partial dependency) |
| fd3 | propertyNo ® pAddress, rent, ownerNo, oName | (Partial dependency) |
| fd4 | ownerNo ® oName | (Transitive dependency) |
| fd5 | clientNo, rentStart ® propertyNo, pAddress, rentFinish, rent, ownerNo, oName | (Candidate key) |
| fd6 | propertyNo, rentStart ® clientNo, cName, rentFinish | (Candidate key) |

Client		Rental			
clientNo	cName	clientNo	propertyNo	rentStart	rentFinish
CR76	John Kay	CR76	PG4	1-Jul-12	31-Aug-13
CR56	Aline Stewart	CR76	PG16	1-Sep-13	1-Sep-14
		CR56	PG4	1-Sep-11	10-Jun-12
		CR56	PG36	10-Oct-12	1-Dec-13
		CR56	PG16	1-Nov-14	10-Aug-15

PropertyOwner				
propertyNo	pAddress	rent	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93	Tony Shaw

Figure 14.14 Second normal form relations derived from the ClientRental relation.

Using these functional dependencies, we continue the process of normalizing the ClientRental relation. We begin by testing whether the ClientRental relation is in 2NF by identifying the presence of any partial dependencies on the primary key. We note that the client attribute (cName) is partially dependent on the primary key, in other words, on only the clientNo attribute (represented as fd2). The property attributes (pAddress, rent, ownerNo, oName) are partially dependent on the primary key, that is, on only the propertyNo attribute (represented as fd3). The property rented attributes (rentStart and rentFinish) are fully dependent on the whole primary key; that is the clientNo and propertyNo attributes (represented as fd1).

The identification of partial dependencies within the ClientRental relation indicates that the relation is not in 2NF. To transform the ClientRental relation into 2NF requires the creation of new relations so that the non-primary-key attributes are removed along with a copy of the part of the primary key on which they are fully functionally dependent. This results in the creation of three new relations called Client, Rental, and PropertyOwner, as shown in Figure 14.14. These three relations are in second normal form, as every non-primary-key attribute is fully functionally dependent on the primary key of the relation. The relations have the following form:

Client	(<u>clientNo</u> , cName)
Rental	(<u>clientNo</u> , <u>propertyNo</u> , rentStart, rentFinish)
PropertyOwner	(<u>propertyNo</u> , pAddress, rent, ownerNo, oName)

14.8 Third Normal Form (3NF)

Although 2NF relations have less redundancy than those in 1NF, they may still suffer from update anomalies. For example, if we want to update the name of an owner, such as Tony Shaw (ownerNo CO93), we have to update two tuples in the PropertyOwner relation of Figure 14.14. If we update only one tuple and not the other, the database would be in an inconsistent state. This update anomaly is caused by a transitive dependency, which we described in Section 14.4. We need to remove such dependencies by progressing to third normal form.

Third Normal Form (3NF)

A relation that is in first and second normal form and in which no non-primary-key attribute is transitively dependent on the primary key.

The normalization of 2NF relations to 3NF involves the removal of transitive dependencies. If a transitive dependency exists, we remove the transitively dependent attribute(s) from the relation by placing the attribute(s) in a new relation along with a copy of the determinant. We demonstrate the process of converting 2NF relations to 3NF relations in the following example.

EXAMPLE 14.11 Third Normal Form (3NF)

The functional dependencies for the Client, Rental, and PropertyOwner relations, derived in Example 14.10, are as follows:

<u>Client</u>		
fd2	clientNo ® cName	(Primary key)
<u>Rental</u>		
fd1	clientNo, propertyNo ® rentStart, rentFinish	(Primary key)
fd5'	clientNo, rentStart ® propertyNo, rentFinish	(Candidate key)
fd6'	propertyNo, rentStart ® clientNo, rentFinish	(Candidate key)
<u>PropertyOwner</u>		
fd3	propertyNo ® pAddress, rent, ownerNo, oName	(Primary key)
fd4	ownerNo ® oName	(Transitive dependency)

All the non-primary-key attributes within the Client and Rental relations are functionally dependent on only their primary keys. The Client and Rental relations have no transitive dependencies and are therefore already in 3NF. Note that where a functional dependency (fd) is labeled with a prime (such as fd5'), this indicates that the dependency has altered compared with the original functional dependency shown in Figure 14.12.

All the non-primary-key attributes within the PropertyOwner relation are functionally dependent on the primary key, with the exception of oName, which is transitively dependent on ownerNo (represented as fd4). This transitive dependency was previously identified in Figure 14.12. To transform the PropertyOwner relation into 3NF, we must first remove this transitive dependency by creating two new relations called PropertyForRent and Owner, as shown in Figure 14.15. The new relations have the following form:

PropertyForRent	(propertyNo, pAddress, rent, ownerNo)
Owner	(ownerNo, oName)

The PropertyForRent and Owner relations are in 3NF, as there are no further transitive dependencies on the primary key.

PropertyForRent				Owner	
propertyNo	pAddress	rent	ownerNo	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93		

Figure 14.15 Third normal form relations derived from the PropertyOwner relation.

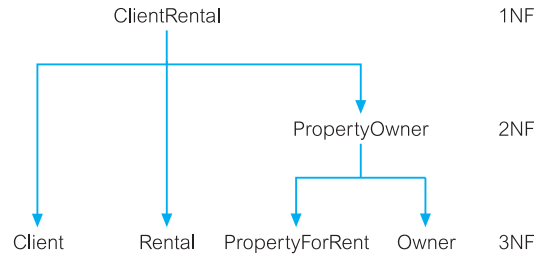


Figure 14.16
The decomposition
of the ClientRental
1NF relation into
3NF relations.

The ClientRental relation shown in Figure 14.11 has been transformed by the process of normalization into four relations in 3NF. Figure 14.16 illustrates the process by which the original 1NF relation is decomposed into the 3NF relations. The resulting 3NF relations have the form:

Client	(<u>clientNo</u> , cName)
Rental	(<u>clientNo</u> , <u>propertyNo</u> , rentStart, rentFinish)
PropertyForRent	(<u>propertyNo</u> , pAddress, rent, ownerNo)
Owner	(<u>ownerNo</u> , oName)

The original ClientRental relation shown in Figure 14.11 can be recreated by joining the Client, Rental, PropertyForRent, and Owner relations through the primary key/foreign key mechanism. For example, the ownerNo attribute is a primary key within the Owner relation and is also present within the PropertyForRent relation as a foreign key. The ownerNo attribute acting as a primary key/foreign key allows the association of the PropertyForRent and Owner relations to identify the name of property owners.

The clientNo attribute is a primary key of the Client relation and is also present within the Rental relation as a foreign key. Note that in this case the clientNo attribute in the Rental relation acts both as a foreign key and as part of the primary key of this relation. Similarly, the propertyNo attribute is the primary key of the PropertyForRent relation and is also present within the Rental relation acting both as a foreign key and as part of the primary key for this relation.

In other words, the normalization process has decomposed the original ClientRental relation using a series of relational algebra projections (see Section 5.1). This results in a **lossless-join** (also called *nonloss-* or *nonadditive-join*) decomposition, which is reversible using the natural join operation. The Client, Rental, PropertyForRent, and Owner relations are shown in Figure 14.17.

14.9 General Definitions of 2NF and 3NF

The definitions for 2NF and 3NF given in Sections 14.7 and 14.8 disallow partial or transitive dependencies on the *primary key* of relations to avoid the update anomalies described in Section 14.3. However, these definitions do not take into account other candidate keys of a relation, if any exist. In this section, we present more general definitions for 2NF and 3NF that take into account candidate keys of a relation. Note that this requirement does not alter the definition for 1NF as this normal form is independent of keys and functional dependencies. For the general definitions, we state that a candidate-key attribute is part of any candidate key and

Figure 14.17

A summary of the 3NF relations derived from the ClientRental relation.

Client		Rental			
clientNo	cName	clientNo	propertyNo	rentStart	rentFinish
CR76	John Kay	CR76	PG4	1-Jul-12	31-Aug-13
CR56	Aline Stewart	CR76	PG16	1-Sep-13	1-Sep-14
		CR56	PG4	1-Sep-11	10-Jun-12
		CR56	PG36	10-Oct-12	1-Dec-13
		CR56	PG16	1-Nov-14	10-Aug-15

PropertyForRent				Owner	
propertyNo	pAddress	rent	ownerNo	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93		

that partial, full, and transitive dependencies are with respect to all candidate keys of a relation.

Second Normal Form (2NF)

A relation that is in first normal form and every non-candidate-key attribute is fully functionally dependent on *any candidate key*.

Third Normal Form (3NF)

A relation that is in first and second normal form and in which no non-candidate-key attribute is transitively dependent on *any candidate key*.

When using the general definitions of 2NF and 3NF, we must be aware of partial and transitive dependencies on all candidate keys and not just the primary key. This can make the process of normalization more complex; however, the general definitions place additional constraints on the relations and may identify hidden redundancy in relations that could be missed.

The trade-off is whether it is better to keep the process of normalization simpler by examining dependencies on primary keys only, which allows the identification of the most problematic and obvious redundancy in relations, or to use the general definitions and increase the opportunity to identify missed redundancy. In fact, it is often the case that whether we use the definitions based on primary keys or the general definitions of 2NF and 3NF, the decomposition of relations is the same. For example, if we apply the general definitions of 2NF and 3NF to Examples 14.10 and 14.11 described in Sections 14.7 and Section 14.8, the same decomposition of the larger relations into smaller relations results. The reader may wish to verify this fact.

In the following chapter we re-examine the process of identifying functional dependencies that are useful for normalization and take the process of normalization further by discussing normal forms that go beyond 3NF such as BCNF. Also in this chapter we present a second worked example taken from the *DreamHome* case study that reviews the process of normalization from UNF through to BCNF.



Chapter Summary

- **Normalization** is a technique for producing a set of relations with desirable properties, given the data requirements of an enterprise. Normalization is a formal method that can be used to identify relations based on their keys and the functional dependencies among their attributes.
- Relations with data redundancy suffer from **update anomalies**, which can be classified as insertion, deletion, and modification anomalies.
- One of the main concepts associated with normalization is **functional dependency**, which describes the relationship between attributes in a relation. For example, if A and B are attributes of relation R, B is functionally dependent on A (denoted $A \twoheadrightarrow B$), if each value of A is associated with exactly one value of B. (A and B may each consist of one or more attributes.)
- The **determinant** of a functional dependency refers to the attribute, or group of attributes, on the left-hand side of the arrow.
- The main characteristics of functional dependencies that we use for normalization have a one-to-one relationship between attribute(s) on the left-hand and right-hand sides of the dependency, hold for all time, and are fully functionally dependent.
- **Unnormalized Form (UNF)** is a table that contains one or more repeating groups.
- **First Normal Form (1NF)** is a relation in which the intersection of each row and column contains one and only one value.
- **Second Normal Form (2NF)** is a relation that is in first normal form and every non-primary-key attribute is fully functionally dependent on the *primary key*. **Full functional dependency** indicates that if A and B are attributes of a relation, B is fully functionally dependent on A if B is functionally dependent on A but not on any proper subset of A.
- **Third Normal Form (3NF)** is a relation that is in first and second normal form in which no non-primary-key attribute is transitively dependent on the *primary key*. **Transitive dependency** is a condition where A, B, and C are attributes of a relation such that if $A \twoheadrightarrow B$ and $B \twoheadrightarrow C$, then C is transitively dependent on A via B (provided that A is not functionally dependent on B or C).
- **General definition for Second Normal Form (2NF)** is a relation that is in first normal form and every non-candidate-key attribute is fully functionally dependent on *any candidate key*. In this definition, a candidate-key attribute is part of any candidate key.
- **General definition for Third Normal Form (3NF)** is a relation that is in first and second normal form in which no non-candidate-key attribute is transitively dependent on *any candidate key*. In this definition, a candidate-key attribute is part of any candidate key.

Review Questions

- 14.1 Describe the purpose of normalizing data.
- 14.2 Discuss the alternative ways that normalization can be used to support database design.
- 14.3 How does normalization eradicate update anomalies from a relation?
- 14.4 Describe the concept of functional dependency.
- 14.5 What are the main characteristics of functional dependencies that are used for normalization?

- 14.6 Describe how a database designer typically identifies the set of functional dependencies associated with a relation.
- 14.7 Describe factors that would influence the choice of normalization or ER modeling when designing a database.
- 14.8 Why is normalization regarded as a bottom-up design approach? How does it differ from ER modeling?
- 14.9 Describe the two approaches to converting an UNF table to 1NF relation(s).
- 14.10 The second normal form (2NF) is realized by removing partial dependencies from 1NF relations. Briefly describe the term “partial dependency.”
- 14.11 Describe the concept of transitive dependency and describe how this concept relates to 3NF. Provide an example to illustrate your answer.
- 14.12 Discuss how the definitions of 2NF and 3NF based on primary keys differ from the general definitions of 2NF and 3NF. Provide an example to illustrate your answer.

Exercises

- 14.13 Normalization is an important concept for database professionals. Whether you are the designer, database analyst or administrator, it is useful for designing, situation verification as well as performance tuning. What are the basic issues to be aware of before carrying out the normalization process?.
- 14.14 Examine the Patient Medication Form for the *Wellmeadows Hospital* case study (see Appendix B) shown in Figure 14.18.
- (a) Identify the functional dependencies represented by the attributes shown in the form in Figure 14.18. State any assumptions that you make about the data and the attributes shown in this form.
- (b) Describe and illustrate the process of normalizing the attributes shown in Figure 14.18 to produce a set of well-designed 3NF relations.
- (c) Identify the primary, alternate, and foreign keys in your 3NF relations.

Wellmeadows Hospital
Patient Medication Form

Patient Number: P10034

Full Name: Robert MacDonald

Ward Number: Ward 11

Bed Number: 84

Ward Name: Orthopaedic

Drug Number	Name	Description	Dosage	Method of Admin	Units per Day	Start Date	Finish Date
10223	Morphine	Pain Killer	10mg/ml	Oral	50	24/03/13	24/04/14
10334	Tetracycline	Antibiotic	0.5mg/ml	IV	10	24/03/13	17/04/13
10223	Morphine	Pain Killer	10mg/ml	Oral	10	25/04/14	02/05/15

Figure 14.18 The *Wellmeadows Hospital* Patient Medication Form.

- 14.15 The table shown in Figure 14.19 lists sample dentist/patient appointment data. A patient is given an appointment at a specific time and date with a dentist located at a particular surgery. On each day of patient appointments, a dentist is allocated to a specific surgery for that day.
- The table shown in Figure 14.19 is susceptible to update anomalies. Provide examples of insertion, deletion, and update anomalies.
 - Identify the functional dependencies represented by the attributes shown in the table of Figure 14.19. State any assumptions you make about the data and the attributes shown in this table.
 - Describe and illustrate the process of normalizing the table shown in Figure 14.19 to 3NF relations. Identify the primary, alternate, and foreign keys in your 3NF relations.

staffNo	dentistName	patNo	patName	appointment date	time	surgeryNo
S1011	Tony Smith	P100	Gillian White	12-Sep-13	10.00	S15
S1011	Tony Smith	P105	Jill Bell	12-Sep-13	12.00	S15
S1024	Helen Pearson	P108	Ian MacKay	12-Sep-13	10.00	S10
S1024	Helen Pearson	P108	Ian MacKay	14-Sep-13	14.00	S10
S1032	Robin Plevin	P105	Jill Bell	14-Sep-13	16.30	S15
S1032	Robin Plevin	P110	John Walker	15-Sep-13	18.00	S13

Figure 14.19 Table displaying sample dentist/patient appointment data.

- 14.16 An agency called *Instant Cover* supplies part-time/temporary staff to hotels within Scotland. The table shown in Figure 14.20 displays sample data, which lists the time spent by agency staff working at various hotels. The National Insurance Number (NIN) is unique for every member of staff.
- The table shown in Figure 14.20 is susceptible to update anomalies. Provide examples of insertion, deletion, and update anomalies.
 - Identify the functional dependencies represented by the attributes shown in the table of Figure 14.20. State any assumptions that you make about the data and the attributes shown in this table.
 - Describe and illustrate the process of normalizing the table shown in Figure 14.20 to 3NF. Identify primary, alternate, and foreign keys in your relations.

NIN	contractNo	hours	eName	hNo	hLoc
1135	C1024	16	Smith J	H25	East Kilbride
1057	C1024	24	Hocine D	H25	East Kilbride
1068	C1025	28	White T	H4	Glasgow
1135	C1025	15	Smith J	H4	Glasgow

Figure 14.20 Table displaying sample data for the *Instant Cover* agency.

- 14.17 A company called *FastCabs* provides a taxi service to clients. The table shown in Figure 14.21 displays some details of client bookings for taxis. Assume that a taxi driver is assigned to a single taxi, but a taxi can be assigned to one or more drivers.
- Identify the functional dependencies that exist between the columns of the table in Figure 14.21 and identify the primary key and any alternate key(s) (if present) for the table.
 - Describe why the table in Figure 14.21 is not in 3NF.
 - The table shown in Figure 14.21 is susceptible to update anomalies. Provide examples of how insertion, deletion, and modification anomalies could occur on this table.

JobID	JobDate Time	driverID	driver Name	taxiID	clientID	clientName	jobPickUpAddress
1	25/07/14 10.00	D1	Joe Bull	T1	C1	Anne Woo	1 Storrie Rd, Paisley
2	29/07/14 10.00	D1	Joe Bull	T1	C1	Anne Woo	1 Storrie Rd, Paisley
3	30/07/14 11.00	D2	Tom Win	T2	C1	Anne Woo	3 High Street, Paisley
4	2/08/14 13.00	D3	Jim Jones	T3	C2	Mark Tin	1A Lady Lane, Paisley
5	2/08/14 13.00	D4	Steven Win	T1	C3	John Seal	22 Red Road, Paisley
6	25/08/14 10.00	D2	Tom Win	T2	C4	Karen Bow	17 High Street, Paisley

Figure 14.21 Table displaying sample data for *FastCabs*.

14.18 Applying normalisation to 3NF on the table shown in Figure 14.21 results in the formation of the three 3NF tables shown in Figure 14.22.

- Identify the functional dependencies that exist between the columns of each table in Figure 14.22 and identify the primary key and any alternate and foreign key(s) (if present) for each table.
- Describe why storing the *FastCabs* data across three 3NF tables avoids the update anomalies described in Exercise 14.17(b).
- Describe how the original table shown in Figure 14.21 can be re-created through relational joins between primary key and foreign keys columns of the tables in Figure 14.22.

JobID	JobDateTime	driverID	clientID	jobPickUpAddress
1	25/07/14 10.00	D1	C1	1 Storrier Rd, Paisley
2	29/07/14 10.00	D1	C1	1 Storrier Rd, Paisley
3	30/07/14 11.00	D2	C1	3 High Street, Paisley
4	2/08/14 13.00	D3	C2	1A Lady Lane, Paisley
5	2/08/14 13.00	D4	C3	22 Red Road, Paisley
6	25/08/14 10.00	D2	C4	17 High Street, Paisley

driverID	driverName	taxiID
D1	Joe Bull	T1
D2	Tom Win	T2
D3	Jim Jones	T3
D4	Steven Win	T1

clientID	clientName
C1	Anne Woo
C2	Mark Tin
C3	John Seal
C4	Karen Bow

Figure 14.22 Tables (in 3NF) displaying sample data for *FastCabs*.

14.19 Students can lease university flats and some of the details of leases held by students for places in university flats are shown in Figure 14.23. A place number (placeNo) uniquely identifies each single room in all flats and is used when leasing a room to a student.

- Identify the functional dependencies that exist between the columns of the table in Figure 14.23 and identify the primary key and any alternate key(s) (if present) for the table.
- Describe why the table in Figure 14.23 is not in 3NF.
- The table shown in Figure 14.23 is susceptible to update anomalies. Provide examples of how insertion, deletion, and modification anomalies could occur on this table.

leaseNo	bannerID	placeNo	fName	lName	startDate	finishDate	flatNo	flatAddress
10003	B017706	78	Jane	Watt	01/09/2010	30/06/2011	F56	34 High Street, Paisley
10259	B017706	88	Jane	Watt	01/09/2011	30/06/2012	F78	111 Storrie Road, Paisley
10364	B013399	89	Tom	Jones	01/09/2011	30/06/2012	F78	111 Storrie Road, Paisley
10566	B012124	102	Karen	Black	01/09/2011	30/06/2012	F79	120 Lady Lane, Paisley
11067	B034511	88	Steven	Smith	01/09/2012	30/06/2013	F78	111 Storrie Road, Paisley
11169	B013399	78	Tom	Jones	01/09/2012	30/06/2013	F56	34 High Street, Paisley

Figure 14.23 Table displaying sample data for university accommodation.

14.20 Applying normalisation to 3NF on the table shown in Figure 14.23 results in the formation of the four tables shown in Figure 14.24.

- Identify the functional dependencies that exist between the columns of each table in Figure 14.24 and identify the primary key and any alternate and foreign key(s) (if present) for each table.
- Describe why storing the university accommodation data across four 3NF tables avoids the update anomalies described in Exercise 14.19(b).
- Describe how the original table shown in Figure 14.23 can be re-created through relational joins between primary key and foreign keys columns of the tables in Figure 14.24.

leaseNo	bannerID	placeNo	startDate	finishDate
10003	B017706	78	01/09/2010	30/06/2011
10259	B017706	88	01/09/2011	30/06/2012
10364	B013399	89	01/09/2011	30/06/2012
10566	B012124	102	01/09/2011	30/06/2012
11067	B034511	88	01/09/2012	30/06/2013
11169	B013399	78	01/09/2012	30/06/2013

flatNo	flat Address
F56	34 High Street, Paisley
F78	111 Storrie Road, Paisley
F79	120 Lady Lane, Paisley

bannerID	fName	lName
B017706	Jane	Watt
B013399	Tom	Jones
B012124	Karen	Black
B034511	Steven	Smith

placeNo	flatNo
78	F56
88	F78
89	F78
102	F79

Figure 14.24 Tables (in 3NF) displaying sample data of university accommodation.

Chapter Objectives

In this chapter you will learn:

- How inference rules can identify a set of *all* functional dependencies for a relation.
- How inference rules called Armstrong's axioms can identify a *minimal* set of useful functional dependencies from the set of all functional dependencies for a relation.
- Normal forms that go beyond Third Normal Form (3NF), which includes Boyce–Codd Normal Form (BCNF), Fourth Normal Form (4NF), and Fifth Normal Form (5NF).
- How to identify BCNF.
- How to represent attributes shown on a report as BCNF relations using normalization.
- The concept of multi-valued dependencies and 4NF.
- The problems associated with relations that break the rules of 4NF.
- How to create 4NF relations from a relation that breaks the rules of 4NF.
- The concept of join dependency and 5NF.
- The problems associated with relations that break the rules of 5NF.
- How to create 5NF relations from a relation that breaks the rules of 5NF.

In the previous chapter we introduced the technique of normalization and the concept of functional dependencies between attributes. We described the benefits of using normalization to support database design and demonstrated how attributes shown on sample forms are transformed into 1NF, 2NF, and 3NF relations. In this chapter, we return to consider functional dependencies and describe normal forms that go beyond 3NF such as BCNF, 4NF, and 5NF. Relations in 3NF are normally sufficiently well structured to prevent the problems associated with data redundancy, which was described in Section 14.3. However, later normal forms were created to identify relatively rare problems with relations that, if not corrected, might result in undesirable data redundancy.

Structure of this Chapter With the exception of 1NF, all normal forms discussed in the previous chapter and in this chapter are based on functional dependencies among the attributes of a relation. In Section 15.1 we continue the discussion on the concept of functional dependency, which was introduced in the previous chapter. We present a more formal and theoretical aspect of functional dependencies by discussing inference rules for functional dependencies.

In the previous chapter we described the three most commonly used normal forms: 1NF, 2NF, and 3NF. However, R. Boyce and E.F. Codd identified a weakness with 3NF and introduced a stronger definition of 3NF Boyce-Codd Normal Form (BCNF) (Codd, 1974), which we describe in Section 15.2. In Section 15.3 we present a worked example to demonstrate the process of normalizing attributes originally shown on a report into a set of BCNF relations.

Higher normal forms that go beyond BCNF were introduced later, such as Fourth (4NF) and Fifth (5NF) Normal Forms (Fagin, 1977, 1979). However, these later normal forms deal with situations that are very rare. We describe 4NF and 5NF in Sections 15.4 and 15.5.

To illustrate the process of normalization, examples are drawn from the *DreamHome* case study described in Section 11.4 and documented in Appendix A.



15.1 More on Functional Dependencies

One of the main concepts associated with normalization is **functional dependency**, which describes the relationship between attributes (Maier, 1983). In the previous chapter we introduced this concept. In this section we describe this concept in a more formal and theoretical way by discussing inference rules for functional dependencies.

15.1.1 Inference Rules for Functional Dependencies

In Section 14.4 we identified the characteristics of the functional dependencies that are most useful in normalization. However, even if we restrict our attention to functional dependencies with a one-to-one (1:1) relationship between attributes on the left-hand and right-hand sides of the dependency that hold for all time and are fully functionally dependent, then the complete set of functional dependencies for a given relation can still be very large. It is important to find an approach that can reduce that set to a manageable size. Ideally, we want to identify a set of functional dependencies (represented as X) for a relation that is smaller than the complete set of functional dependencies (represented as Y) for that relation and has the property that every functional dependency in Y is implied by the functional dependencies in X . Hence, if we enforce the integrity constraints defined by the functional dependencies in X , we automatically enforce the integrity constraints defined in

the larger set of functional dependencies in Y . This requirement suggests that there must be functional dependencies that can be inferred from other functional dependencies. For example, functional dependencies $A \rightarrow B$ and $B \rightarrow C$ in a relation implies that the functional dependency $A \rightarrow C$ also holds in that relation. $A \rightarrow C$ is an example of a **transitive** functional dependency and was discussed previously in Sections 14.4 and 14.7.

How do we begin to identify useful functional dependencies on a relation? Normally, the database designer starts by specifying functional dependencies that are semantically obvious; however, there are usually numerous other functional dependencies. In fact, the task of specifying all possible functional dependencies for “real” database projects is more often than not impractical. However, in this section we do consider an approach that helps identify the complete set of functional dependencies for a relation and then discuss how to achieve a minimal set of functional dependencies that can represent the complete set.

The set of all functional dependencies that are implied by a given set of functional dependencies X is called the **closure** of X , written X^+ . We clearly need a set of rules to help compute X^+ from X . A set of inference rules, called **Armstrong’s axioms**, specifies how new functional dependencies can be inferred from given ones (Armstrong, 1974). For our discussion, let A , B , and C be subsets of the attributes of the relation R . Armstrong’s axioms are as follows:

- (1) **Reflexivity:** If B is a subset of A , then $A \rightarrow B$
- (2) **Augmentation:** If $A \rightarrow B$, then $A, C \rightarrow B, C$
- (3) **Transitivity:** If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$

Note that each of these three rules can be directly proved from the definition of functional dependency. The rules are **complete** in that given a set X of functional dependencies, all functional dependencies implied by X can be derived from X using these rules. The rules are also **sound** in that no additional functional dependencies can be derived that are not implied by X . In other words, the rules can be used to derive the closure of X^+ .

Several further rules can be derived from the three given previously that simplify the practical task of computing X^+ . In the following rules, let D be another subset of the attributes of relation R . Then:

- (4) **Self-determination:** $A \rightarrow A$
- (5) **Decomposition:** If $A \rightarrow B, C$, then $A \rightarrow B$ and $A \rightarrow C$
- (6) **Union:** If $A \rightarrow B$ and $A \rightarrow C$, then $A \rightarrow B, C$
- (7) **Composition:** If $A \rightarrow B$ and $C \rightarrow D$ then $A, C \rightarrow B, D$

Rule 1 (Reflexivity) and Rule 4 (Self-determination) state that a set of attributes always determines any of its subsets or itself. Because these rules generate functional dependencies that are always true, such dependencies are trivial and, as stated earlier, are generally not interesting or useful. Rule 2 (Augmentation) states that adding the same set of attributes to both the left-hand and right-hand sides of a dependency results in another valid dependency. Rule 3 (Transitivity) states that functional dependencies are transitive. Rule 5 (Decomposition) states that we can remove attributes from the right-hand side of a dependency. Applying this rule repeatedly, we can decompose $A \rightarrow B, C, D$ functional dependency into the set of

dependencies $A \rightarrow B$, $A \rightarrow C$, and $A \rightarrow D$. Rule 6 (Union) states that we can do the opposite: we can combine a set of dependencies $A \rightarrow B$, $A \rightarrow C$, and $A \rightarrow D$ into a single functional dependency $A \rightarrow B, C, D$. Rule 7 (Composition) is more general than Rule 6 and states that we can combine a set of nonoverlapping dependencies to form another valid dependency.

To begin to identify the set of functional dependencies F for a relation, we typically first identify the dependencies that are determined from the semantics of the attributes of the relation. Then we apply Armstrong's axioms (Rules 1 to 3) to infer additional functional dependencies that are also true for that relation. A systematic way to determine these additional functional dependencies is to first determine each set of attributes A that appears on the left-hand side of some functional dependencies and then to determine the set of *all* attributes that are dependent on A . Thus, for each set of attributes A , we can determine the set A^+ of attributes that are functionally determined by A based on F (A^+ is called the **closure of A under F**).

15.1.2 Minimal Sets of Functional Dependencies

In this section we introduce what is referred to as **equivalence** of sets of functional dependencies. A set of functional dependencies Y is **covered by** a set of functional dependencies X , if every functional dependency in Y is also in X^+ ; that is, every dependency in Y can be inferred from X . A set of functional dependencies X is minimal if it satisfies the following conditions:

- Every dependency in X has a single attribute on its right-hand side.
- We cannot replace any dependency $A \rightarrow B$ in X with dependency $C \rightarrow B$, where C is a proper subset of A , and still have a set of dependencies that is equivalent to X .
- We cannot remove any dependency from X and still have a set of dependencies that is equivalent to X .

A minimal set of dependencies should be in a standard form with no redundancies. A minimal cover of a set of functional dependencies X is a minimal set of dependencies $X_{\min}$ that is equivalent to X . Unfortunately, there can be several minimal covers for a set of functional dependencies. We demonstrate the identification of the minimal cover for the StaffBranch relation in the following example.

EXAMPLE 15.1 Identifying the minimal set of functional dependencies of the Staff-Branch relation

We apply the three conditions described previously on the set of functional dependencies for the StaffBranch relation listed in Example 14.5 to produce the following functional dependencies:

```

staffNo → sName
staffNo → position
staffNo → salary
staffNo → branchNo
branchNo → bAddress
bAddress → branchNo
branchNo, position → salary
bAddress, position → salary

```

These functional dependencies satisfy the three conditions for producing a minimal set of functional dependencies for the `StaffBranch` relation. Condition 1 ensures that every dependency is in a standard form with a single attribute on the right-hand side. Conditions 2 and 3 ensure that there are no redundancies in the dependencies, either by having redundant attributes on the left-hand side of a dependency (Condition 2) or by having a dependency that can be inferred from the remaining functional dependencies in $\times$ (Condition 3).

In the following section we return to consider normalization. We begin by discussing BCNF, a stronger normal form than 3NF.

15.2 Boyce–Codd Normal Form (BCNF)

In the previous chapter we demonstrated how 2NF and 3NF disallow partial and transitive dependencies on the *primary key* of a relation, respectively. Relations that have these types of dependencies may suffer from the update anomalies discussed in Section 14.3. However, the definitions of 2NF and 3NF discussed in Sections 14.7 and 14.8, respectively, do not consider whether such dependencies remain on other candidate keys of a relation, if any exist. In Section 14.9 we presented general definitions for 2NF and 3NF that disallow partial and transitive dependencies on any *candidate key* of a relation, respectively. Application of the general definitions of 2NF and 3NF may identify additional redundancy caused by dependencies that violate one or more candidate keys. However, despite these additional constraints, dependencies can still exist that will cause redundancy to be present in 3NF relations. This weakness in 3NF resulted in the presentation of a stronger normal form called Boyce–Codd Normal Form (BCNF: Codd, 1974).

15.2.1 Definition of BCNF

BCNF is based on functional dependencies that take into account all candidate keys in a relation; however, BCNF also has additional constraints compared with the general definition of 3NF given in Section 14.9.

Boyce–Codd Normal Form (BCNF)

A relation is in BCNF if and only if every determinant is a candidate key.

To test whether a relation is in BCNF, we identify all the determinants and make sure that they are candidate keys. Recall that a determinant is an attribute, or a group of attributes, on which some other attribute is fully functionally dependent.

The difference between 3NF and BCNF is that for a functional dependency $A \rightarrow B$, 3NF allows this dependency in a relation if B is a primary-key attribute and A is not a candidate key, whereas BCNF insists that for this dependency to remain in a relation, A must be a candidate key. Therefore, BCNF is a stronger form of 3NF, such that every relation in BCNF is also in 3NF. However, a relation in 3NF is not necessarily in BCNF.

Before considering the next example, we re-examine the `Client`, `Rental`, `PropertyForRent`, and `Owner` relations shown in Figure 14.17. The `Client`, `PropertyForRent`, and `Owner` relations are all in BCNF, as each relation only has a single determinant, which is the candidate key. However, recall that the `Rental` relation contains the

three determinants (clientNo, propertyNo), (clientNo, rentStart), and (propertyNo, rentStart), originally identified in Example 14.11, as shown below:

fd1 clientNo, propertyNo $\rightarrow$ rentStart, rentFinish
 fd5' clientNo, rentStart $\rightarrow$ propertyNo, rentFinish
 fd6' propertyNo, rentStart $\rightarrow$ clientNo, rentFinish

As the three determinants of the Rental relation are also candidate keys, the Rental relation is also already in BCNF. Violation of BCNF is quite rare, as it may happen only under specific conditions. The potential to violate BCNF may occur when:

- the relation contains two (or more) composite candidate keys *or*
- the candidate keys overlap, that is have at least one attribute in common.

In the following example, we present a situation in which a relation violates BCNF and demonstrate the transformation of this relation to BCNF. This example demonstrates the process of converting a 1NF relation to BCNF relations.



EXAMPLE 15.2 Boyce–Codd Normal Form (BCNF)

In this example, we extend the *DreamHome* case study to include a description of client interviews by members of staff. The information relating to these interviews is in the ClientInterview relation shown in Figure 15.1. The members of staff involved in interviewing clients are allocated to a specific room on the day of interview. However, a room may be allocated to several members of staff as required throughout a working day. A client is interviewed only once on a given date, but may be requested to attend further interviews at later dates.

ClientInterview

clientNo	interviewDate	interviewTime	staffNo	roomNo
CR76	13-May-14	10.30	SG5	G101
CR56	13-May-14	12.00	SG5	G101
CR74	13-May-14	12.00	SG37	G102
CR56	1-Jul-14	10.30	SG5	G102

Figure 15.1 ClientInterview relation.

The ClientInterview relation has three candidate keys: (clientNo, interviewDate), (staffNo, interviewDate, interviewTime), and (roomNo, interviewDate, interviewTime). Therefore the ClientInterview relation has three composite candidate keys, which overlap by sharing the common attribute interviewDate. We select (clientNo, interviewDate) to act as the primary key for this relation. The ClientInterview relation has the following form:

ClientInterview (clientNo, interviewDate, interviewTime, staffNo, roomNo)

The ClientInterview relation has the following functional dependencies:

fd1 clientNo, interviewDate $\rightarrow$ interviewTime, staffNo, roomNo (Primary key)
 fd2 staffNo, interviewDate, interviewTime $\rightarrow$ clientNo (Candidate key)
 fd3 roomNo, interviewDate, interviewTime $\rightarrow$ staffNo, clientNo (Candidate key)
 fd4 staffNo, interviewDate $\rightarrow$ roomNo

We examine the functional dependencies to determine the normal form of the `ClientInterview` relation. As functional dependencies `fd1`, `fd2`, and `fd3` are all candidate keys for this relation, none of these dependencies will cause problems for the relation. The only functional dependency that requires discussion is $(\text{staffNo}, \text{interviewDate}) \rightarrow \text{roomNo}$ (represented as `fd4`). Even though $(\text{staffNo}, \text{interviewDate})$ is not a candidate key for the `ClientInterview` relation this functional dependency is allowed in 3NF because `roomNo` is a primary-key attribute being part of the candidate key $(\text{roomNo}, \text{interviewDate}, \text{interviewTime})$. As there are no partial or transitive dependencies on the primary key $(\text{clientNo}, \text{interviewDate})$, and functional dependency `fd4` is allowed, the `ClientInterview` relation is in 3NF.

However, this relation is not in BCNF (a stronger normal form of 3NF) due to the presence of the $(\text{staffNo}, \text{interviewDate})$ determinant, which is not a candidate key for the relation. BCNF requires that all determinants in a relation must be a candidate key for the relation. As a consequence, the `ClientInterview` relation may suffer from update anomalies. For example, to change the room number for staff number `SG5` on the 13-May-14 we must update two tuples. If only one tuple is updated with the new room number, this results in an inconsistent state for the database.

To transform the `ClientInterview` relation to BCNF, we must remove the violating functional dependency by creating two new relations called `Interview` and `StaffRoom`, as shown in Figure 15.2. The `Interview` and `StaffRoom` relations have the following form:

`Interview` (`clientNo`, `interviewDate`, `interviewTime`, `staffNo`)

`StaffRoom` (`staffNo`, `interviewDate`, `roomNo`)

We can decompose any relation that is not in BCNF into BCNF as illustrated. However, it may not always be desirable to transform a relation into BCNF; for example, if there is a functional dependency that is not preserved when we perform the decomposition (that is, the determinant and the attributes that it determines are placed in different relations). In this situation, it is difficult to enforce the functional dependency in the relation, and an important constraint is lost. When this occurs, it may be better to stop at 3NF, which always preserves

Interview

<code>clientNo</code>	<code>interviewDate</code>	<code>interviewTime</code>	<code>staffNo</code>
CR76	13-May-14	10.30	SG5
CR56	13-May-14	12.00	SG5
CR74	13-May-14	12.00	SG37
CR56	1-Jul-14	10.30	SG5

StaffRoom

<code>staffNo</code>	<code>interviewDate</code>	<code>roomNo</code>
SG5	13-May-14	G101
SG37	13-May-14	G102
SG5	1-Jul-14	G102

Figure 15.2 The `Interview` and `StaffRoom` BCNF relations.

dependencies. Note that in Example 15.2, in creating the two BCNF relations from the original *ClientInterview* relation, we have “lost” the functional dependency, $\text{roomNo}, \text{interviewDate}, \text{interviewTime} \rightarrow \text{staffNo}, \text{clientNo}$ (represented as fd3), as the determinant for this dependency is no longer in the same relation. However, we must recognize that if the functional dependency, $\text{staffNo}, \text{interviewDate} \rightarrow \text{roomNo}$ (represented as fd4) is not removed, the *ClientInterview* relation will have data redundancy.

The decision as to whether it is better to stop the normalization at 3NF or progress to BCNF is dependent on the amount of redundancy resulting from the presence of fd4 and the significance of the “loss” of fd3. For example, if it is the case that members of staff conduct only one interview per day, then the presence of fd4 in the *ClientInterview* relation will not cause redundancy and therefore the decomposition of this relation into two BCNF relations is not helpful or necessary. On the other hand, if members of staff conduct numerous interviews per day, then the presence of fd4 in the *ClientInterview* relation will cause redundancy and normalization of this relation to BCNF is recommended. However, we should also consider the significance of losing fd3; in other words, does fd3 convey important information about client interviews that must be represented in one of the resulting relations? The answer to this question will help to determine whether it is better to retain all functional dependencies or remove data redundancy.

15.3 Review of Normalization Up to BCNF

The purpose of this section is to review the process of normalization described in the previous chapter and in Section 15.2. We demonstrate the process of transforming attributes displayed on a sample report from the *DreamHome* case study into a set of BCNF relations. In this worked example we use the definitions of 2NF and 3NF that are based on the primary key of a relation. We leave the normalization of this worked example using the general definitions of 2NF and 3NF as an exercise for the reader.



EXAMPLE 15.3 First normal form (1NF) to Boyce–Codd Normal Form (BCNF)

In this example we extend the *DreamHome* case study to include property inspection by members of staff. When staff are required to undertake these inspections, they are allocated a company car for use on the day of the inspections. However, a car may be allocated to several members of staff as required throughout the working day. A member of staff may inspect several properties on a given date, but a property is inspected only once on a given date. Examples of the *DreamHome* Property Inspection Report are presented in Figure 15.3. The report on top describes staff inspections of property PG4 in Glasgow.

First Normal Form (1NF)

We first transfer sample data held on two property inspection reports into table format with rows and columns. This is referred to as the *StaffPropertyInspection* unnormalized table and is shown in Figure 15.4. We identify the key attribute for this unnormalized table as *propertyNo*.

We identify the repeating group in the unnormalized table as the property inspection and staff details, which repeats for each property. The structure of the repeating group is:

Repeating Group = (iDate, iTime, comments, staffNo, sName, carReg)

DreamHome
Property Inspection Report

DreamHome
Property Inspection Report

Property Number PG4

 Property Address 6 Lawrence St, Glasgow

Inspection Date	Inspection Time	Comments	Staff no	Staff Name	Car Registration
18-Oct-12	10.00	Need to replace crockery	SG37	Ann Beech	M231 JGR
22-Apr-13	09.00	In good order	SG14	David Ford	M533 HDR
1-Oct-13	12.00	Damp rot in bathroom	SG14	David Ford	N721 HFR

Page 1

Figure 15.3 DreamHome Property Inspection reports.

StaffPropertyInspection

propertyNo	pAddress	iDate	iTime	comments	staffNo	sName	carReg
PG4	6 Lawrence St, Glasgow	18-Oct-12	10.00	Need to replace crockery	SG37	Ann Beech	M231 JGR
		22-Apr-13	09.00	In good order	SG14	David Ford	M533 HDR
		1-Oct-13	12.00	Damp rot in bathroom	SG14	David Ford	N721 HFR
PG16	5 Novar Dr, Glasgow	22-Apr-13	13.00	Replace living room carpet	SG14	David Ford	M533 HDR
		24-Oct-13	14.00	Good condition	SG37	Ann Beech	N721 HFR

Figure 15.4 StaffPropertyInspection unnormalized table.

As a consequence, there are multiple values at the intersection of certain rows and columns. For example, for `propertyNo` PG4 there are three values for `iDate` (18-Oct-12, 22-Apr-13, 1-Oct-13). We transform the unnormalized form to first normal form using the first approach described in Section 14.6. With this approach, we remove the repeating group (property inspection and staff details) by entering the appropriate property details (nonrepeating data) into each row. The resulting first normal form `StaffPropertyInspection` relation is shown in Figure 15.5.

In Figure 15.6, we present the functional dependencies (fd1 to fd6) for the `StaffPropertyInspection` relation. We use the functional dependencies (as discussed in Section 14.4.3) to identify candidate keys for the `StaffPropertyInspection` relation as being composite keys comprising (`propertyNo`, `iDate`), (`staffNo`, `iDate`, `iTime`), and (`carReg`, `iDate`, `iTime`). We select (`propertyNo`, `iDate`) as the primary key for this relation. For clarity, we

StaffPropertyInspection							
propertyNo	iDate	iTime	pAddress	comments	staffNo	sName	carReg
PG4	18-Oct-12	10.00	6 Lawrence St, Glasgow	Need to replace crockery	SG37	Ann Beech	M231 JGR
PG4	22-Apr-13	09.00	6 Lawrence St, Glasgow	In good order	SG14	David Ford	M533 HDR
PG4	1-Oct-13	12.00	6 Lawrence St, Glasgow	Damp rot in bathroom	SG14	David Ford	N721 HFR
PG16	22-Apr-13	13.00	5 Novar Dr, Glasgow	Replace living room carpet	SG14	David Ford	M533 HDR
PG16	24-Oct-13	14.00	5 Novar Dr, Glasgow	Good condition	SG37	Ann Beech	N721 HFR

Figure 15.5 The First Normal Form(1NF) StaffPropertyInspection relation.

place the attributes that make up the primary key together, at the left-hand side of the relation. The StaffPropertyInspection relation is defined as follows:

StaffPropertyInspection (propertyNo, iDate, iTime, pAddress, comments, staffNo, sName, carReg)

The StaffPropertyInspection relation is in 1NF, as there is a single value at the intersection of each row and column. The relation contains data describing the inspection of property by members of staff, with the property and staff details repeated several times. As a result, the StaffPropertyInspection relation contains significant redundancy. If implemented, this 1NF relation would be subject to update anomalies. To remove some of these, we must transform the relation into second normal form.

Second Normal Form (2NF)

The normalization of 1NF relations to 2NF involves the removal of partial dependencies on the primary key. If a partial dependency exists, we remove the functionally dependent attributes from the relation by placing them in a new relation with a copy of their determinant.

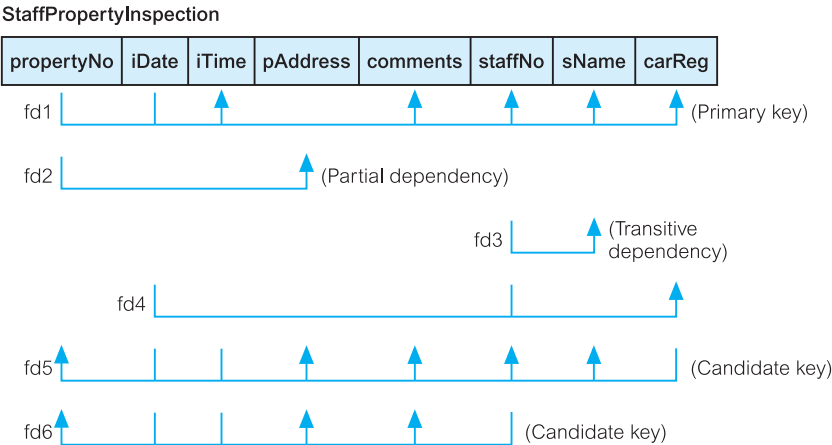


Figure 15.6 Functional dependencies of the StaffPropertyInspection relation.

As shown in Figure 15.6, the functional dependencies (fd1 to fd6) of the `StaffPropertyInspection` relation are as follows:

fd1	<code>propertyNo, iDate → iTime, comments, staffNo, sName, carReg</code>	(Primary key)
fd2	<code>propertyNo → pAddress</code>	(Partial dependency)
fd3	<code>staffNo → sName</code>	(Transitive dependency)
fd4	<code>staffNo, iDate → carReg</code>	
fd5	<code>carReg, iDate, iTime → propertyNo, pAddress, comments, staffNo, sName</code>	(Candidate key)
fd6	<code>staffNo, iDate, iTime → propertyNo, pAddress, comments</code>	(Candidate key)

Using the functional dependencies, we continue the process of normalizing the `StaffPropertyInspection` relation. We begin by testing whether the relation is in 2NF by identifying the presence of any partial dependencies on the primary key. We note that the property attribute (`pAddress`) is partially dependent on part of the primary key—namely, the `propertyNo` (represented as fd2)—whereas the remaining attributes (`iTime`, `comments`, `staffNo`, `sName`, and `carReg`) are fully dependent on the whole primary key (`propertyNo` and `iDate`), (represented as fd1). Note that although the determinant of the functional dependency `staffNo, iDate → carReg` (represented as fd4) requires only the `iDate` attribute of the primary key, we do not remove this dependency at this stage, as the determinant also includes another non-primary-key attribute, namely `staffNo`. In other words, this dependency is *not* wholly dependent on part of the primary key and therefore does not violate 2NF.

The identification of the partial dependency (`propertyNo → pAddress`) indicates that the `StaffPropertyInspection` relation is not in 2NF. To transform the relation into 2NF requires the creation of new relations so that the attributes that are not fully dependent on the primary key are associated with only the appropriate part of the key.

The `StaffPropertyInspection` relation is transformed into second normal form by removing the partial dependency from the relation and creating two new relations called `Property` and `PropertyInspection` with the following form:

<code>Property</code>	(<u><code>propertyNo</code></u> , <code>pAddress</code>)
<code>PropertyInspection</code>	(<u><code>propertyNo</code></u> , <u><code>iDate</code></u> , <code>iTime</code> , <code>comments</code> , <code>staffNo</code> , <code>sName</code> , <code>carReg</code>)

These relations are in 2NF, as every non-primary-key attribute is functionally dependent on the primary key of the relation.

Third Normal Form (3NF)

The normalization of 2NF relations to 3NF involves the removal of transitive dependencies. If a transitive dependency exists, we remove the transitively dependent attributes from the relation by placing them in a new relation along with a copy of their determinant. The functional dependencies within the `Property` and `PropertyInspection` relations are as follows:

Property Relation

fd2 `propertyNo → pAddress`

PropertyInspection Relation

fd1 `propertyNo, iDate → iTime, comments, staffNo, sName, carReg`

fd3 `staffNo → sName`

fd4 `staffNo, iDate → carReg`

fd5' `carReg, iDate, iTime → propertyNo, comments, staffNo, sName`

fd6' `staffNo, iDate, iTime → propertyNo, comments`

As the Property relation does not have transitive dependencies on the primary key, it is therefore already in 3NF. However, although all the non-primary-key attributes within the PropertyInspection relation are functionally dependent on the primary key, sName is also transitively dependent on staffNo (represented as fd3). We also note the functional dependency staffNo, iDate $\rightarrow$ carReg (represented as fd4) has a non-primary-key attribute carReg partially dependent on a non-primary-key attribute, staffNo. We do not remove this dependency at this stage as part of the determinant for this dependency includes a primary-key attribute: iDate. In other words, this dependency is *not* wholly transitively dependent on non-primary-key attributes and therefore does not violate 3NF. (In other words, as described in Section 14.9, when considering all candidate keys of a relation, the staffNo, iDate $\rightarrow$ carReg dependency is allowed in 3NF, because carReg is a primary-key attribute, as it is part of the candidate key (carReg, iDate, iTime) of the original PropertyInspection relation.)

To transform the PropertyInspection relation into 3NF, we remove the transitive dependency (staffNo $\rightarrow$ sName) by creating two new relations called Staff and PropertyInspect with the form:

```
Staff          (staffNo, sName)
PropertyInspect (propertyNo, iDate, iTime, comments, staffNo, carReg)
```

The Staff and PropertyInspect relations are in 3NF as no non-primary-key attribute is wholly functionally dependent on another non-primary-key attribute. Thus, the StaffPropertyInspection relation shown in Figure 15.5 has been transformed by the process of normalization into three relations in 3NF with the following form:

```
Property       (propertyNo, pAddress)
Staff          (staffNo, sName)
PropertyInspect (propertyNo, iDate, iTime, comments, staffNo, carReg)
```

Boyce–Codd Normal Form (BCNF)

We now examine the Property, Staff, and PropertyInspect relations to determine whether they are in BCNF. Recall that a relation is in BCNF if every determinant of a relation is a candidate key. Therefore, to test for BCNF, we simply identify all the determinants and make sure they are candidate keys.

The functional dependencies for the Property, Staff, and PropertyInspect relations are as follows:

Property Relation

fd2 propertyNo $\rightarrow$ pAddress

Staff Relation

fd3 staffNo $\rightarrow$ sName

PropertyInspect Relation

fd1' propertyNo, iDate $\rightarrow$ iTime, comments, staffNo, carReg

fd4 staffNo, iDate $\rightarrow$ carReg

fd5' carReg, iDate, iTime $\rightarrow$ propertyNo, comments, staffNo

fd6' staffNo, iDate, iTime $\rightarrow$ propertyNo, comments

We can see that the Property and Staff relations are already in BCNF, as the determinant in each of these relations is also the candidate key. The only 3NF relation that is not in BCNF is PropertyInspect, because of the presence of the determinant (staffNo, iDate), which is not a candidate key (represented as fd4). As a consequence the PropertyInspect relation may suffer from update anomalies. For example, to change the car allocated to staff number SG14 on the 22-Apr-12, we must update two tuples. If only one tuple is updated with the new car registration number, this results in an inconsistent state for the database.

To transform the `PropertyInspect` relation into BCNF, we must remove the dependency that violates BCNF by creating two new relations called `StaffCar` and `Inspection` with the form:

```
StaffCar  (staffNo, iDate, carReg)
Inspection (propertyNo, iDate, iTime, comments, staffNo)
```

The `StaffCar` and `Inspection` relations are in BCNF, as the determinant in each of these relations is also a candidate key.

In summary, the decomposition of the `StaffPropertyInspection` relation shown in Figure 15.5 into BCNF relations is shown in Figure 15.7. In this example, the decomposition of the original `StaffPropertyInspection` relation to BCNF relations has resulted in the “loss” of the functional dependency: `carReg, iDate, iTime → propertyNo, pAddress, comments, staffNo, sName`, as parts of the determinant are in different relations (represented as fd5). However, we recognize that if the functional dependency, `staffNo, iDate → carReg` (represented as fd4) is not removed, the `PropertyInspect` relation will have data redundancy.

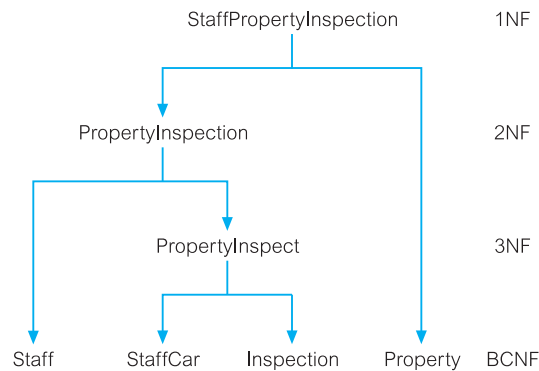


Figure 15.7 Decomposition of the `StaffPropertyInspection` relation into BCNF relations.

The resulting BCNF relations have the following form:

```
Property (propertyNo, pAddress)
Staff (staffNo, sName)
Inspection (propertyNo, iDate, iTime, comments, staffNo)
StaffCar (staffNo, iDate, carReg)
```

The original `StaffPropertyInspection` relation shown in Figure 15.5 can be recreated from the `Property`, `Staff`, `Inspection`, and `StaffCar` relations using the primary key/foreign key mechanism. For example, the attribute `staffNo` is a primary key within the `Staff` relation and is also present within the `Inspection` relation as a foreign key. The foreign key allows the association of the `Staff` and `Inspection` relations to identify the name of the member of staff undertaking the property inspection.

15.4 Fourth Normal Form (4NF)

Although BCNF removes any anomalies due to functional dependencies, further research led to the identification of another type of dependency called a **Multi-Valued Dependency (MVD)**, which can also cause data redundancy (Fagin, 1977). In this section, we briefly describe a multi-valued dependency and the association of this type of dependency with Fourth Normal Form (4NF).

BranchStaffOwner		
branchNo	sName	oName
B003	Ann Beech	Carol Farrel
B003	David Ford	Carol Farrel
B003	Ann Beech	Tina Murphy
B003	David Ford	Tina Murphy

Figure 15.8(a) The BranchStaffOwner relation.

15.4.1 Multi-Valued Dependency

The possible existence of multi-valued dependencies in a relation is due to 1NF, which disallows an attribute in a tuple from having a set of values. For example, if we have two multi-valued attributes in a relation, we have to repeat each value of one of the attributes with every value of the other attribute, to ensure that tuples of the relation are consistent. This type of constraint is referred to as a multi-valued dependency and results in data redundancy. Consider the BranchStaffOwner relation shown in Figure 15.8, which displays the names of members of staff (sName) and property owners (oName) at each branch office (branchNo). In this example, assume that staff name (sName) uniquely identifies each member of staff and that the owner name (oName) uniquely identifies each owner.

In this example, members of staff called Ann Beech and David Ford work at branch B003, and property owners called Carol Parrel and Tina Murphy are registered at branch B003. However, as there is no direct relationship between members of staff and property owners at a given branch office, we must create a tuple for every combination of member of staff and owner to ensure that the relation is consistent.

**Multi-Valued
Dependency
(MVD)**

Represents a dependency between attributes (for example, A, B, and C) in a relation, such that for each value of A there is a set of values for B and a set of values for C. However, the set of values for B and C are independent of each other.

We represent a MVD between attributes A, B, and C in a relation using the following notation:

$A \twoheadrightarrow B$
 $A \twoheadrightarrow C$

A multi-valued dependency constraint potentially exists in the BranchStaffOwner relation, because two independent 1:* relationships are represented in the same relation. We specify the MVD constraint in the BranchStaffOwner relation shown in Figure 15.8(a) as follows:

$\text{branchNo} \twoheadrightarrow \text{sName}$
 $\text{branchNo} \twoheadrightarrow \text{oName}$

A multi-valued dependency can be further defined as being **trivial** or **nontrivial**. A MVD $A \twoheadrightarrow B$ in relation R is defined as being trivial if (a) B is a subset of A or (b) $A \cup B = R$. A MVD is defined as being nontrivial if neither (a) nor (b) are satisfied. A trivial MVD does not specify a constraint on a relation; a nontrivial MVD does specify a constraint.

BranchStaff		BranchOwner	
branchNo	sName	branchNo	oName
B003	Ann Beech	B003	Carol Farrel
B003	David Ford	B003	Tina Murphy

Figure 15.8(b) The BranchStaff and BranchOwner 4NF relations.

The BranchStaffOwner relation shown in Figure 15.8(a) contains two nontrivial dependencies that is $\text{branchNo} \twoheadrightarrow \text{sName}$ and $\text{branchNo} \twoheadrightarrow \text{oName}$ with branchNo not a candidate key of the relation. The BranchStaffOwner relation is therefore constrained by the nontrivial MVDs to repeat rows to ensure that the relation remains consistent in terms of the relationship between the sName and oName attributes. For example, if we wanted to add a new property owner for branch B003, we would have to create two new tuples, one for each member of staff, to ensure that the relation remains consistent. This is an example of an update anomaly caused by the presence of the nontrivial MVDs. We therefore clearly require a normal form that prevents relational structures such as the BranchStaffOwner relation.

15.4.2 Definition of Fourth Normal Form

Fourth Normal Form (4NF)

A relation is in 4NF if and only if for every nontrivial multi-valued dependency $A \twoheadrightarrow B$, A is a candidate key of the relation.

Fourth normal form (4NF) prevents a relation from containing a nontrivial MVD without the associated determinant being a candidate key for the relation (Fagin, 1977). When the 4NF rule is violated, the potential for data redundancy exists, as shown previously in Figure 15.8(a). The normalization of a relation breaking the 4NF rule requires the removal of the offending MVD from the relation by placing the multi-valued attribute(s) in a new relation along with a copy of the determinant.

For example, the BranchStaffOwner relation in Figure 15.8(a) is not in 4NF, because of the presence of two nontrivial MVDs. We decompose the BranchStaffOwner relation into the BranchStaff and BranchOwner relations, as shown in Figure 15.8(b). Both new relations are in 4NF, because the BranchStaff relation contains the trivial MVD $\text{branchNo} \twoheadrightarrow \text{sName}$, and the BranchOwner relation contains the trivial MVD $\text{branchNo} \twoheadrightarrow \text{oName}$. Note that the 4NF relations do not display data redundancy, and the potential for update anomalies is removed. For example, to add a new property owner for branch B003, we simply create a single tuple in the BranchOwner relation. For a detailed discussion on 4NF, the interested reader is referred to Date (2012) and Elmasri and Navathe (2010).

15.5 Fifth Normal Form (5NF)

Whenever we decompose a relation into two relations, the resulting relations have the lossless-join property. This property refers to the ability to rejoin the resulting

relations to produce the original relation. However, there are cases in which there is a requirement to decompose a relation into more than two relations. Although rare, these cases are managed by join dependency and Fifth Normal Form (5NF). In this section we briefly describe the lossless-join dependency and the association with 5NF.

15.5.1 Lossless-Join Dependency

Lossless-join dependency

A property of decomposition that ensures that no spurious tuples are generated when relations are reunited through a natural join operation.

In splitting relations by projection, we are very explicit about the method of decomposition. In particular, we are careful to use projections that can be reversed by joining the resulting relations, so that the original relation is reconstructed. Such a decomposition is called a **lossless-join** (also called a *nonloss-* or *nonadditive-join*) decomposition, because it preserves all the data in the original relation and does not result in the creation of additional spurious tuples. For example, Figures 15.8(a) and 15.8(b) show that the decomposition of the BranchStaffOwner relation into the BranchStaff and BranchOwner relations has the lossless-join property. In other words, the original BranchStaffOwner relation can be reconstructed by performing a natural join operation on the BranchStaff and BranchOwner relations. In this example, the original relation is decomposed into two relations. However, there are cases where we require to perform a lossless-join decompose of a relation into more than two relations (Aho *et al.*, 1979). These cases are the focus of the lossless-join dependency and 5NF.

15.5.2 Definition of Fifth Normal Form

Fifth Normal Form (5NF)

A relation is in 5NF if and only if for every join dependency ($R_1, R_2, \dots, R_n$) in a relation R , each projection includes a candidate key of the original relation.

Fifth normal form (5NF) prevents a relation from containing a nontrivial join dependency (JD) without the associated projection including a candidate key of the original relation (Fagin, 1977). Nontrivial JDs that are not associated with candidate keys are very rare, so 4NF relations are normally also in 5NF.

Although rare, let us examine what potential problems exist for a relation that breaks the rules of 5NF. The PropertyItemSupplier relation shown in Figure 15.9(a) is not in 5NF, as it contains a nontrivial join dependency constraint. This relation describes properties (propertyNo) that require certain items (itemDescription), which are supplied by suppliers (supplierNo) to the properties (propertyNo). Furthermore, whenever a property (p) requires a certain item (i), and a supplier (s) supplies that item (i), and the supplier (s) already supplies *at least one* item to that property (p), then the supplier (s) will also supply the required item (i) to property (p). In this example, assume that a description of an item (itemDescription) uniquely identifies each type of item.

PropertyItem		ItemSupplier		PropertySupplier	
propertyNo	itemDescription	itemDescription	supplierNo	propertyNo	supplierNo
PG4	Bed	Bed	S1	PG4	S1
PG4	Chair	Chair	S2	PG4	S2
PG16	Bed	Bed	S2	PG16	S2

Figure 15.10 PropertyItem, ItemSupplier, and PropertySupplier 5NF relations.

Chapter Summary

- **Inference rules** can be used to identify the set of *all* functional dependencies associated with a relation. This set of dependencies can be very large for a given relation.
- Inference rules called **Armstrong’s axioms** can be used to identify a *minimal* set of functional dependencies from the set of all functional dependencies for a relation.
- A relation is in **Boyce-Codd Normal Form (BCNF)** if and only if every determinant is a candidate key.
- A relation is in **Fourth Normal Form (4NF)** if and only if for every nontrivial multi-valued dependency $A \twoheadrightarrow B$, A is a candidate key of the relation.
- A **multi-valued dependency (MVD)** represents a dependency between attributes (A , B , and C) in a relation such that for each value of A there is a set of values of B and a set of values for C . However, the set of values for B and C are independent of each other.
- A **lossless-join dependency** is a property of decomposition, which means that no spurious tuples are generated when relations are combined through a natural join operation.
- A relation is in **Fifth Normal Form (5NF)** if and only if for every join dependency $(R_1, R_2, \dots, R_n)$ in a relation R , each projection includes a candidate key of the original relation.

Review Questions

- 15.1 What is the overall purpose of using inference rules and Armstrong’s axioms while developing a database?
- 15.2 What conditions would make a 3NF relation violate Boyce-Codd Normal Form (BCNF)?
- 15.3 Discuss the purpose of Boyce–Codd Normal Form (BCNF) and discuss how BCNF differs from 3NF. Provide an example to illustrate your answer.
- 15.4 Describe the concept of multi-valued dependency and discuss how this concept relates to 4NF. Provide an example to illustrate your answer.
- 15.5 Describe the concept of join dependency and discuss how this concept relates to 5NF. Provide an example to illustrate your answer.

Exercises

- 15.6 On completion of Exercise 14.14, examine the 3NF relations created to represent the attributes shown in the *Wellmeadows Hospital* form shown in Figure 14.18. Determine whether these relations are also in BCNF. If not, transform the relations that do not conform into BCNF.
- 15.7 On completion of Exercise 14.15, examine the 3NF relations created to represent the attributes shown in the relation that displays dentist/patient appointment data in Figure 14.19. Determine whether these relations are also in BCNF. If not, transform the relations that do not conform into BCNF.
- 15.8 BCNF is a stronger form of 3NF. Thus, every relation in BCNF is also in 3NF. However, a relation in 3NF is not necessarily in BCNF. Using example to emphasize your case, describe the situations when a 3NF relation is in BCNF and when it is not.
- 15.9 The relation shown in Figure 15.11 lists the students (**studentName**) enrolled in a postgraduate program at Mzumbe University. Every student is required to provide information concerning names of sponsor(s) (**sponsorName**), at least two referees (**refereeName**), and three supervisors (**supervisorName**). Assume that sponsors, referees and supervisors are not allowed to be related to more than one student.

studentName	refereeName	sponsorName	supervisorName

Figure 15.11 Student Registrations

- (a) Describe why the relation shown in Figure 15.11 is not in 4NF.
- (b) The relation shown in Figure 15.11 violates normalization principles. What are the problems users are likely to face?
- (c) Describe how you would normalize the relation shown in Figure 15.11 to 4NF.
- 15.10 The relation shown in Figure 15.12 describes hospitals (**hospitalName**) that require certain items (**itemDescription**), which are supplied by suppliers (**supplierNo**) to the hospitals (**hospitalName**). Furthermore, whenever a hospital (h) requires a certain item (i), and a supplier (s) supplies that item (i), and the supplier (s) already supplies *at least one* item to that hospital (h), then the supplier (s) will also supply the required item (i) to the hospital (h). In this example, assume that a description of an item (**itemDescription**) uniquely identifies each type of item.
- (a) Describe why the relation shown in Figure 15.12 is not in 5NF.
- (b) Describe and illustrate the process of normalizing the relation shown in Figure 15.12 to 5NF.

hospitalName	itemDescription	supplierNo
Western General	Antiseptic Wipes	S1
Western General	Paper Towels	S2
Yorkhill	Antiseptic Wipes	S2
Western General	Antiseptic Wipes	S2

Figure 15.12 The HospitalItemSupplier relation.

Methodology

Chapter	16	Methodology—Conceptual Database Design	455
Chapter	17	Methodology—Logical Database Design for the Relational Model	479
Chapter	18	Methodology—Physical Database Design for Relational Databases	513
Chapter	19	Methodology—Monitoring and Tuning the Operational System	537

Methodology—Conceptual Database Design

Chapter Objectives

In this chapter you will learn:

- The purpose of a design methodology.
- The three main phases of database design: conceptual, logical, and physical design.
- How to decompose the scope of the design into specific views of the enterprise.
- How to use ER modeling to build a local conceptual data model based on the information given in a view of the enterprise.
- How to validate the resultant conceptual data model to ensure that it is a true and accurate representation of a view of the enterprise.
- How to document the process of conceptual database design.
- End-users play an integral role throughout the process of conceptual database design.

In Chapter 10 we described the main stages of the database system development lifecycle, one of which is **database design**. This stage starts only after a complete analysis of the enterprise's requirements has been undertaken.

In this chapter, and Chapters 17–19, we describe a methodology for the database design stage of the database system development lifecycle for relational databases. The methodology is presented as a step-by-step guide to the three main phases of database design, namely: conceptual, logical, and physical design (see Figure 10.1). The main aim of each phase is as follows:

- **Conceptual database design**—to build the conceptual representation of the database, which includes identification of the important entities, relationships, and attributes.
- **Logical database design**—to translate the conceptual representation to the logical structure of the database, which includes designing the relations.
- **Physical database design**—to decide how the logical structure is to be physically implemented (as base relations) in the target DBMS.

Structure of this Chapter In Section 16.1 we define what a database design methodology is and review the three phases of database design. In Section 16.2 we provide an overview of the methodology and briefly describe the main activities associated with each design phase. In Section 16.3 we focus on the methodology for conceptual database design and present a detailed description of the steps required to build a conceptual data model. We use the ER modeling technique described in Chapters 12 and 13 to create the *conceptual data model*. The conceptual data model described in this chapter is the starting point for the next phase of database design described in the following chapter.

In Chapter 17 we focus on the methodology for logical database design for the relational model and present a detailed description of the steps required to convert a conceptual data model into a *logical data model*. This chapter also includes an optional step that describes how to merge two or more logical data models into a single logical data model, for those using the view integration approach (see Section 10.5) to manage the design of a database with multiple user views. The logical data model described in Chapter 17 is the starting point for the final phase of database design described in the following two chapters.

In Chapters 18 and 19 we complete the database design methodology by presenting a detailed description of the steps associated with the production of the physical database design for relational DBMSs. This part of the methodology illustrates that the development of the logical data model alone is insufficient to guarantee the optimum implementation of a database system. For example, we may have to consider modifying the logical model to achieve acceptable levels of performance.

Appendix D presents a summary of the database design methodology for those readers who are already familiar with database design and simply require an overview of the main steps. Throughout the methodology the terms “entity” and “relationship” are used in place of “entity type” and “relationship type” where the meaning is obvious; “type” is generally added only to avoid ambiguity. In this chapter we mostly use examples from the StaffClient user views of the *DreamHome* case study documented in Section 11.4 and Appendix A.



16.1 Introduction to the Database Design Methodology

Before presenting the methodology, we discuss what a design methodology represents and describe the three phases of database design. Finally, we present guidelines for achieving success in database design.

16.1.1 What Is a Design Methodology?

Design methodology

A structured approach that uses procedures, techniques, tools, and documentation aids to support and facilitate the process of design.

A design methodology consists of phases each containing a number of steps that guide the designer in the techniques appropriate at each stage of the project. A design methodology also helps the designer to plan, manage, control, and evaluate database development projects. Furthermore, it is a structured approach for analyzing and modeling a set of requirements for a database in a standardized and organized manner.

16.1.2 Conceptual, Logical, and Physical Database Design

In presenting this database design methodology, the design process is divided into three main phases: conceptual, logical, and physical database design.

Conceptual database design

The process of constructing a model of the data used in an enterprise, independent of *all* physical considerations.

The conceptual database design phase begins with the creation of a conceptual data model of the enterprise that is entirely independent of implementation details such as the target DBMS, application programs, programming languages, hardware platform, performance issues, or any other physical considerations.

Logical database design

The process of constructing a model of the data used in an enterprise based on a specific data model, but independent of a particular DBMS and other physical considerations.

The logical database design phase maps the conceptual data model on to a logical model, which is influenced by the data model for the target database (for example, the relational model). The logical data model is a source of information for the physical design phase, providing the physical database designer with a vehicle for making trade-offs that are very important to the design of an efficient database.

Physical database design

The process of producing a description of the implementation of the database on secondary storage; it describes the base relations, file organizations, and indexes used to achieve efficient access to the data, and any associated integrity constraints and security measures.

The physical database design phase allows the designer to make decisions on how the database is to be implemented. Therefore, physical design is tailored to a specific DBMS. There is feedback between physical and logical design, because decisions taken during physical design for improving performance may affect the logical data model.

16.1.3 Critical Success Factors in Database Design

The following guidelines are often critical to the success of database design:

- Work interactively with the users as much as possible.
- Follow a structured methodology throughout the data modeling process.

- Employ a data-driven approach.
- Incorporate structural and integrity considerations into the data models.
- Combine conceptualization, normalization, and transaction validation techniques into the data modeling methodology.
- Use diagrams to represent as much of the data models as possible.
- Use a Database Design Language (DBDL) to represent additional data semantics that cannot easily be represented in a diagram.
- Build a data dictionary to supplement the data model diagrams and the DBDL.
- Be willing to repeat steps.

These factors are built into the methodology we present for database design.

16.2 Overview of the Database Design Methodology

In this section, we present an overview of the database design methodology. The steps in the methodology are as follows.

Conceptual database design

- Step 1 Build conceptual data model
 - Step 1.1 Identify entity types
 - Step 1.2 Identify relationship types
 - Step 1.3 Identify and associate attributes with entity or relationship types
 - Step 1.4 Determine attribute domains
 - Step 1.5 Determine candidate, primary, and alternate key attributes
 - Step 1.6 Consider use of enhanced modeling concepts (optional step)
 - Step 1.7 Check model for redundancy
 - Step 1.8 Validate conceptual data model against user transactions
 - Step 1.9 Review conceptual data model with user

Logical database design for the relational model

- Step 2 Build logical data model
 - Step 2.1 Derive relations for logical data model
 - Step 2.2 Validate relations using normalization
 - Step 2.3 Validate relations against user transactions
 - Step 2.4 Check integrity constraints
 - Step 2.5 Review logical data model with user
 - Step 2.6 Merge logical data models into global model (optional step)
 - Step 2.7 Check for future growth

Physical database design for relational databases

- Step 3 Translate logical data model for target DBMS
 - Step 3.1 Design base relations

- Step 3.2 Design representation of derived data
- Step 3.3 Design general constraints
- Step 4 Design file organizations and indexes
 - Step 4.1 Analyze transactions
 - Step 4.2 Choose file organizations
 - Step 4.3 Choose indexes
 - Step 4.4 Estimate disk space requirements
- Step 5 Design user views
- Step 6 Design security mechanisms
- Step 7 Consider the introduction of controlled redundancy
- Step 8 Monitor and tune the operational system

This methodology can be used to design relatively simple to highly complex database systems. Just as the database design stage of the database systems development lifecycle (see Section 10.6) has three phases—conceptual, logical, and physical design—so too has the methodology. Step 1 creates a conceptual database design, Step 2 creates a logical database design, and Steps 3 to 8 create a physical database design. Depending on the complexity of the database system being built, some of the steps may be omitted. For example, Step 2.6 of the methodology is not required for database systems with a single user view or database systems with multiple user views being managed using the centralized approach (see Section 10.5). For this reason, we refer to the creation of a single conceptual data model only in Step 1 and single logical data model only in Step 2. However, if the database designer is using the view integration approach (see Section 10.5) to manage user views for a database system, then Steps 1 and 2 may be repeated as necessary to create the required number of models, which are then merged in Step 2.6.

In Chapter 10, we introduced the term “local conceptual data model” or “local logical data model” to refer to the modeling of one or more, but not all, user views of a database system and the term “global logical data model” to refer to the modeling of all user views of a database system. However, the methodology is presented using the more general terms “conceptual data model” and “logical data model” with the exception of the optional Step 2.6, which necessitates the use of the terms *local* logical data model and *global* logical data model, as it is this step that describes the tasks necessary to merge separate local logical data models to produce a global logical data model.

An important aspect of any design methodology is to ensure that the models produced are repeatedly validated so that they continue to be an accurate representation of the part of the enterprise being modeled. In this methodology the data models are validated in various ways such as by using normalization (Step 2.2), by ensuring the critical transactions are supported (Steps 1.8 and 2.3), and by involving the users as much as possible (Steps 1.9 and 2.5).

The logical model created at the end of Step 2 is then used as the source of information for physical database design described in Steps 3 to 8. Again, depending on the complexity of the database systems being designed and/or the functionality of the target DBMS, some of the steps of physical database design may be omitted. For example, Step 4.2 may not be applicable for certain PC-based

DBMSs. The steps of physical database design are described in detail in Chapters 18 and 19.

Database design is an iterative process that has a starting point and an almost endless procession of refinements. Although the steps of the methodology are presented here as a procedural process, it must be emphasized that this does not imply that it should be performed in this manner. It is likely that knowledge gained in one step may alter decisions made in a previous step. Similarly, it may be useful to look briefly at a later step to help with an earlier step. Therefore, the methodology should act as a framework to help guide the designer through database design effectively.



To illustrate the database design methodology we use the *DreamHome* case study. The *DreamHome* database has several user views (Director, Manager, Supervisor, Assistant, and Client) that are managed using a combination of the centralized and view integration approaches (see Section 11.4). Applying the centralized approach resulted in the identification of two collections of user views called StaffClient user views and Branch user views. The user views represented by each collection are as follows:

- **StaffClient user views**—representing Supervisor, Assistant, and Client user views;
- **Branch user views**—representing Director and Manager user views.

In this chapter, which describes Step 1 of the methodology, we use the StaffClient user views to illustrate the building of a conceptual data model, and then in the following chapter, which describes Step 2, we describe how this model is translated into a logical data model. As the StaffClient user views represent only a subset of all the user views of the *DreamHome* database, it is more correct to refer to the data models as local data models. However, as stated earlier when we described the methodology and the worked examples, for simplicity we use the terms conceptual data model and logical data model until the optional Step 2.6, which describes the integration of the local logical data models for the StaffClient user views and the Branch user views.

16.3 Conceptual Database Design Methodology

This section provides a step-by-step guide for conceptual database design.

Step 1: Build Conceptual Data Model

Objective

To build a conceptual data model of the data requirements of the enterprise.

The first step in conceptual database design is to build one (or more) conceptual data models of the data requirements of the enterprise. A conceptual data model comprises:

- entity types;
- relationship types;
- attributes and attribute domains;

- primary keys and alternate keys;
- integrity constraints.

The conceptual data model is supported by documentation, including ER diagrams and a data dictionary, which is produced throughout the development of the model. We detail the types of supporting documentation that may be produced as we go through the various steps. The tasks involved in Step 1 are:

- Step 1.1 Identify entity types
- Step 1.2 Identify relationship types
- Step 1.3 Identify and associate attributes with entity or relationship types
- Step 1.4 Determine attribute domains
- Step 1.5 Determine candidate, primary, and alternate key attributes
- Step 1.6 Consider use of enhanced modeling concepts (optional step)
- Step 1.7 Check model for redundancy
- Step 1.8 Validate conceptual data model against user transactions
- Step 1.9 Review conceptual data model with user

Step 1.1: Identify entity types

Objective To identify the required entity types.

The first step in building a conceptual data model is to determine and define the main objects that the users are interested in. These objects are the entity types for the model (see Section 12.1). One method of identifying entities is to examine the users' requirements specification. From this specification, we identify nouns or noun phrases that are mentioned (for example, staff number, staff name, property number, property address, rent, number of rooms). We also look for major objects, such as people, places, or concepts of interest, excluding those nouns that are merely qualities of other objects. For example, we could group staff number and staff name with an object or entity called **Staff** and group property number, property address, rent, and number of rooms with an entity called **PropertyForRent**.

An alternative way of identifying entities is to look for objects that have an existence in their own right. For example, **Staff** is an entity because staff exist whether or not we know their names, positions, and dates of birth. If possible, the users should assist with this activity.

It is sometimes difficult to identify entities because of the way they are presented in the users' requirements specification. Users often talk in terms of examples or analogies. Instead of talking about staff in general, users may mention people's names. In some cases, users talk in terms of job roles, particularly when people or organizations are involved. These roles may be job titles or responsibilities, such as Director, Manager, Supervisor, or Assistant.

To confuse matters further, users frequently use synonyms and homonyms. Two words are *synonyms* when they have the same meaning, for example, "branch" and "office." *Homonyms* occur when the same word can have different meanings depending on the context. For example, the word "program" has several alternative meanings such as a course of study, a series of events, a plan of work, and an item on the television.

It is not always obvious whether a particular object is an entity, a relationship, or an attribute. For example, how would we classify marriage? In fact, depending on the actual requirements, we could classify marriage as any or all of these. Design is subjective, and different designers may produce different, but equally valid, interpretations. The activity therefore relies, to a certain extent, on judgement and experience. Database designers must take a very selective view of the world and categorize the things that they observe within the context of the enterprise. Thus, there may be no unique set of entity types deducible from a given requirements specification. However, successive iterations of the design process should lead to the choice of entities that are at least adequate for the system required. For the StaffClient user views of *DreamHome*, we identify the following entities:



- Staff
- PropertyForRent
- PrivateOwner
- BusinessOwner
- Client
- Preference
- Lease

Document entity types

As entity types are identified, assign them names that are meaningful and obvious to the user. Record the names and descriptions of entities in a data dictionary. If possible, document the expected number of occurrences of each entity. If an entity is known by different names, the names are referred to as synonyms or *aliases*, which are also recorded in the data dictionary. Figure 16.1 shows an extract from the data dictionary that documents the entities for the StaffClient user views of *DreamHome*.

Step 1.2: Identify relationship types

Objective

To identify the important relationships that exist between the entity types.

Having identified the entities, the next step is to identify all the relationships that exist between these entities (see Section 12.2). When we identify entities, one method

Entity name	Description	Aliases	Occurrence
Staff	General term describing all staff employed by <i>DreamHome</i> .	Employee	Each member of staff works at one particular branch.
PropertyForRent	General term describing all property for rent.	Property	Each property has a single owner and is available at one specific branch, where the property is managed by one member of staff. A property is viewed by many clients and rented by a single client, at any one time.

Figure 16.1 Extract from the data dictionary for the StaffClient user views of *DreamHome* showing a description of entities.

is to look for nouns in the users' requirements specification. Again, we can use the grammar of the requirements specification to identify relationships. Typically, relationships are indicated by verbs or verbal expressions. For example:

- Staff Manages PropertyForRent
- PrivateOwner Owns PropertyForRent
- PropertyForRent AssociatedWith Lease

The fact that the requirements specification records these relationships suggests that they are important to the enterprise and should be included in the model.

We are interested only in required relationships between entities. In the previous examples, we identified the `Staff Manages PropertyForRent` and the `PrivateOwner Owns PropertyForRent` relationships. We may also be inclined to include a relationship between `Staff` and `PrivateOwner` (for example, `Staff Assists PrivateOwner`). However, although this is a possible relationship, from the requirements specification, it is not a relationship that we are interested in modeling.

In most instances, the relationships are binary; in other words, the relationships exist between exactly two entity types. However, we should be careful to look out for complex relationships that may involve more than two entity types (see Section 12.2.1) and recursive relationships that involve only one entity type (see Section 12.2.2).

Great care must be taken to ensure that all the relationships that are either explicit or implicit in the users' requirements specification are detected. In principle, it should be possible to check each pair of entity types for a potential relationship between them, but this would be a daunting task for a large system comprising hundreds of entity types. On the other hand, it is unwise not to perform some such check, and the responsibility is often left to the analyst/designer. However, missing relationships should become apparent when we validate the model against the transactions that are to be supported (Step 1.8).

Use Entity–Relationship (ER) diagrams

It is often easier to visualize a complex system rather than decipher long textual descriptions of a users' requirements specification. We use ER diagrams to represent entities and how they relate to one another more easily. Throughout the database design phase, we recommend that ER diagrams be used whenever necessary to help build up a picture of the part of the enterprise that we are modeling. In this book, we use UML, but other notations perform a similar function (see Appendix C).

Determine the multiplicity constraints of relationship types

Having identified the relationships to model, we next determine the multiplicity of each relationship (see Section 12.6). If specific values for the multiplicity are known, or even upper or lower limits, document these values as well.

Multiplicity constraints are used to check and maintain data quality. These constraints are assertions about entity occurrences that can be applied when the database is updated to determine whether the updates violate the stated rules of the enterprise. A model that includes multiplicity constraints more explicitly represents

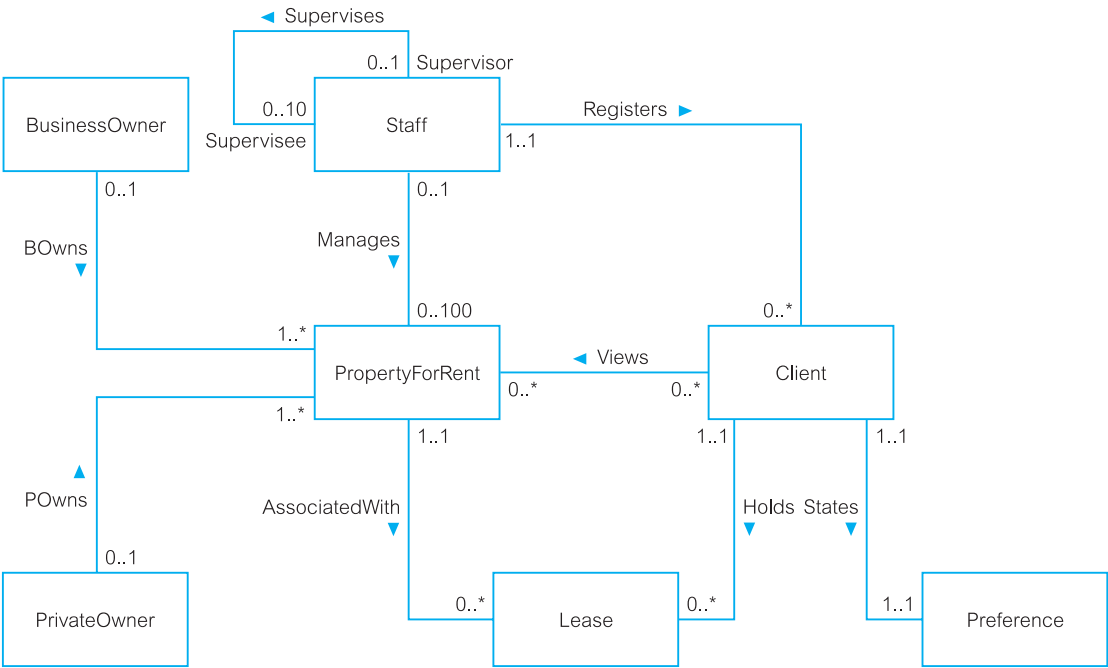


Figure 16.2 First-cut ER diagram showing entity and relationship types for the StaffClient user views of *DreamHome*.

the semantics of the relationships and results in a better representation of the data requirements of the enterprise.

Check for fan and chasm traps

Having identified the necessary relationships, check that each relationship in the ER model is a true representation of the “real world,” and that fan or chasm traps have not been created inadvertently (see Section 12.7).

Figure 16.2 shows the first-cut ER diagram for the StaffClient user views of the *DreamHome* case study.



Document relationship types

As relationship types are identified, assign them names that are meaningful and obvious to the user. Also record relationship descriptions and the multiplicity constraints in the data dictionary. Figure 16.3 shows an extract from the data dictionary that documents the relationships for the StaffClient user views of *DreamHome*.

Step 1.3: Identify and associate attributes with entity or relationship types

Objective

To associate attributes with appropriate entity or relationship types.

The next step in the methodology is to identify the types of facts about the entities and relationships that we have chosen to be represented in the database. In a

Entity name	Multiplicity	Relationship	Multiplicity	Entity name
Staff	0..1	Manages	0..100	PropertyForRent Staff
	0..1	Supervises	0..10	
PropertyForRent	1..1	AssociatedWith	0..*	Lease

Figure 16.3
Extract from the data dictionary for the StaffClient user views of *DreamHome*, showing a description of relationships.

similar way to identifying entities, we look for nouns or noun phrases in the users' requirements specification. The attributes can be identified where the noun or noun phrase is a property, quality, identifier, or characteristic of one of these entities or relationships (see Section 12.3).

By far the easiest thing to do when we have identified an entity (x) or a relationship (y) in the requirements specification is to ask "*What information are we required to hold on x or y ?*" The answer to this question should be described in the specification. However, in some cases it may be necessary to ask the users to clarify the requirements. Unfortunately, they may give answers to this question that also contain other concepts, so the users' responses must be carefully considered.

Simple/composite attributes

It is important to note whether an attribute is simple or composite (see Section 12.3.1). Composite attributes are made up of simple attributes. For example, the address attribute can be simple and hold all the details of an address as a single value, such as "115 Dumbarton Road, Glasgow, G11 6YG." However, the address attribute may also represent a composite attribute, made up of simple attributes that hold the address details as separate values in the attributes street ("115 Dumbarton Road"), city ("Glasgow"), and postcode ("G11 6YG"). The option to represent address details as a simple or composite attribute is determined by the users' requirements. If the user does not need to access the separate components of an address, we represent the address attribute as a simple attribute. On the other hand, if the user does need to access the individual components of an address, we represent the address attribute as being composite, made up of the required simple attributes.

In this step, it is important that we identify all simple attributes to be represented in the conceptual data model including those attributes that make up a composite attribute.

Single/multi-valued attributes

In addition to being simple or composite, an attribute can also be single-valued or multi-valued (see Section 12.3.2). Most attributes encountered will be single-valued, but occasionally a multi-valued attribute may be encountered; that is, an attribute that holds multiple values for a single entity occurrence. For example, we may identify the attribute telNo (the telephone number) of the Client entity as a multi-valued attribute.

On the other hand, client telephone numbers may have been identified as a separate entity from Client. This is an alternative, and equally valid, way to model this. As you will see in Step 2.1, multi-valued attributes are mapped to relations anyway, so both approaches produce the same end result.

Derived attributes

Attributes whose values are based on the values of other attributes are known as *derived attributes* (see Section 12.3.3). Examples of derived attributes include:

- the age of a member of staff;
- the number of properties that a member of staff manages;
- the rental deposit (calculated as twice the monthly rent).

Often, these attributes are not represented in the conceptual data model. However, sometimes the value of the attribute or attributes on which the derived attribute is based may be deleted or modified. In this case, the derived attribute must be shown in the data model to avoid this potential loss of information. However, if a derived attribute is shown in the model, we must indicate that it is derived. The representation of derived attributes will be considered during physical database design. Depending on how an attribute is used, new values for a derived attribute may be calculated each time it is accessed or when the value(s) it is derived from changes. However, this issue is not the concern of conceptual database design, and is discussed in more detail in Step 3.2 in Chapter 18.

Potential problems

When identifying the entities, relationships, and attributes for the user views, it is not uncommon for it to become apparent that one or more entities, relationships, or attributes have been omitted from the original selection. In this case, return to the previous steps, document the new entities, relationships, or attributes, and re-examine any associated relationships.

As there are generally many more attributes than entities and relationships, it may be useful to first produce a list of all attributes given in the users' requirements specification. As an attribute is associated with a particular entity or relationship, remove the attribute from the list. In this way, we ensure that an attribute is associated with only one entity or relationship type and, when the list is empty, that all attributes are associated with some entity or relationship type.

We must also be aware of cases where attributes appear to be associated with more than one entity or relationship type, as this can indicate the following:

- (1) We have identified several entities that can be represented as a single entity. For example, we may have identified entities Assistant and Supervisor both with the attributes staffNo (the staff number), name, sex, and DOB (date of birth), which can be represented as a single entity called Staff with the attributes staffNo (the staff number), name, sex, DOB, and position (with values Assistant or Supervisor). On the other hand, it may be that these entities share many attributes but there are also attributes or relationships that are unique to each entity. In this case, we must decide whether we want to generalize the entities into a single entity such as Staff, or leave them as specialized entities representing distinct staff

roles. The consideration of whether to specialize or generalize entities was discussed in Chapter 13 and is addressed in more detail in Step 1.6.

- (2) We have identified a relationship between entity types. In this case, we must associate the attribute with only *one* entity, the parent entity, and ensure that the relationship was previously identified in Step 1.2. If this is not the case, the documentation should be updated with details of the newly identified relationship. For example, we may have identified the entities *Staff* and *PropertyForRent* with the following attributes:

Staff	staffNo, name, position, sex, DOB
PropertyForRent	propertyNo, street, city, postcode, type, rooms, rent, managerName

The presence of the *managerName* attribute in *PropertyForRent* is intended to represent the relationship *Staff Manages PropertyForRent*. In this case, the *managerName* attribute should be omitted from *PropertyForRent* and the relationship *Manages* should be added to the model.

DreamHome attributes for entities

For the *StaffClient* user views of *DreamHome*, we identify and associate attributes with entities as follows:



Staff	staffNo, name (composite: fName, lName), position, sex, DOB
PropertyForRent	propertyNo, address (composite: street, city, postcode), type, rooms, rent
PrivateOwner	ownerNo, name (composite: fName, lName), address, telNo
BusinessOwner	ownerNo, bName, bType, address, telNo, contactName
Client	clientNo, name (composite: fName, lName), telNo, eMail
Preference	prefType, maxRent
Lease	leaseNo, paymentMethod, deposit (derived as PropertyForRent.rent*2), depositPaid, rentStart, rentFinish, duration (derived as rentFinish – rentStart)

DreamHome attributes for relationships

Some attributes should not be associated with entities, but instead should be associated with relationships. For the *StaffClient* user views of *DreamHome*, we identify and associate attributes with relationships, as follows:



Views	viewDate, comment
-------	-------------------

Document attributes

As attributes are identified, assign them names that are meaningful to the user. Record the following information for each attribute:

- attribute name and description;
- data type and length;
- any aliases that the attribute is known by;
- whether the attribute is composite and, if so, the simple attributes that make up the composite attribute;
- whether the attribute is multi-valued;

Entity name	Attributes	Description	Data Type & Length	Nulls	Multi-valued	...
Staff	staffNo	Uniquely identifies a member of staff	5 variable characters	No	No	
	name					
	fName	First name of staff	15 variable characters	No	No	
	lName	Last name of staff	15 variable characters	No	No	
	position	Job title of member of staff	10 variable characters	No	No	
	sex	Gender of member of staff	1 character (M or F)	Yes	No	
	DOB	Date of birth of member of staff	Date	Yes	No	
PropertyForRent	propertyNo	Uniquely identifies a property for rent	5 variable characters	No	No	

Figure 16.4 Extract from the data dictionary for the StaffClient user views of *DreamHome* showing a description of attributes.

- whether the attribute is derived and, if so, how it is to be computed;
- any default value for the attribute.



Figure 16.4 shows an extract from the data dictionary that documents the attributes for the StaffClient user views of *DreamHome*.

Step 1.4: Determine attribute domains

Objective

To determine domains for the attributes in the conceptual data model.

The objective of this step is to determine domains for all the attributes in the model (see Section 12.3). A **domain** is a pool of values from which one or more attributes draw their values. For example, we may define:

- the attribute domain of valid staff numbers (staffNo) as being a five-character variable-length string, with the first two characters as letters and the next one to three characters as digits in the range 1–999;
- the possible values for the sex attribute of the Staff entity as being either “M” or “F.” The domain of this attribute is a single character string consisting of the values “M” or “F.”

A fully developed data model specifies the domains for each attribute and includes:

- allowable set of values for the attribute;
- sizes and formats of the attribute.

Further information can be specified for a domain, such as the allowable operations on an attribute, and which attributes can be compared with other attributes or used in combination with other attributes. However, implementing these characteristics of attribute domains in a DBMS is still the subject of research.

Document attribute domains

As attribute domains are identified, record their names and characteristics in the data dictionary. Update the data dictionary entries for attributes to record their domain in place of the data type and length information.

Step 1.5: Determine candidate, primary, and alternate key attributes

Objective

To identify the candidate key(s) for each entity type and, if there is more than one candidate key, to choose one to be the primary key and the others as alternate keys.

This step is concerned with identifying the candidate key(s) for an entity and then selecting one to be the primary key (see Section 12.3.4). A **candidate key** is a minimal set of attributes of an entity that uniquely identifies each occurrence of that entity. We may identify more than one candidate key, in which case we must choose one to be the **primary key**; the remaining candidate keys are called **alternate keys**.

People's names generally do not make good candidate keys. For example, we may think that a suitable candidate key for the *Staff* entity would be the composite attribute name, the member of staff's name. However, it is possible for two people with the same name to join *DreamHome*, which would clearly invalidate the choice of name as a candidate key. We could make a similar argument for the names of property owners. In such cases, rather than coming up with combinations of attributes that may provide uniqueness, it may be better to use an existing attribute that would always ensure uniqueness, such as the *staffNo* attribute for the *Staff* entity and the *ownerNo* attribute for the *PrivateOwner* entity, or define a new attribute that would provide uniqueness.

When choosing a primary key from among the candidate keys, use the following guidelines to help make the selection:

- the candidate key with the minimal set of attributes;
- the candidate key that is least likely to have its values changed;
- the candidate key with fewest characters (for those with textual attribute(s));
- the candidate key with smallest maximum value (for those with numerical attribute(s));
- the candidate key that is easiest to use from the users' point of view.

In the process of identifying primary keys, note whether an entity is strong or weak. If we are able to assign a primary key to an entity, the entity is referred to as being *strong*. On the other hand, if we are unable to identify a primary key for an entity, the entity is referred to as being *weak* (see Section 12.4). The primary key of a weak entity can be identified only when we map the weak entity and its relationship with its owner entity to a relation through the placement of a foreign key in that relation. The process of mapping entities and their relationships to relations is described in Step 2.1, and therefore the identification of primary keys for weak entities cannot take place until that step.

DreamHome primary keys

The primary keys for the *StaffClient* user views of *DreamHome* are shown in Figure 16.5. Note that the *Preference* entity is a weak entity and, as identified previously, the *Views* relationship has two attributes, *viewDate* and *comment*.



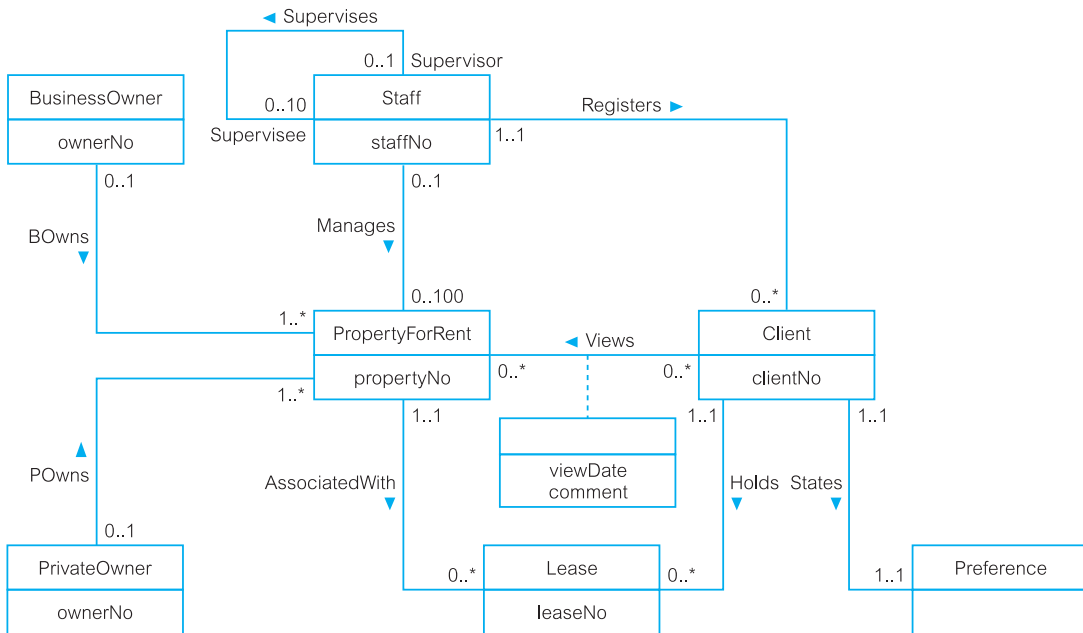


Figure 16.5 ER diagram for the StaffClient user views of *DreamHome* with primary keys added.

Document primary and alternate keys

Record the identification of primary and any alternate keys in the data dictionary.

Step 1.6: Consider use of enhanced modeling concepts (optional step)

Objective

To consider the use of enhanced modeling concepts, such as specialization/generalization, aggregation, and composition.

In this step, we have the option to continue the development of the ER model using the enhanced modeling concepts discussed in Chapter 13, namely specialization/generalization, aggregation, and composition. If we select the specialization approach, we attempt to highlight differences between entities by defining one or more **subclasses** of a **superclass** entity. If we select the generalization approach, we attempt to identify common features between entities to define a generalizing superclass entity. We may use aggregation to represent a ‘has-a’ or ‘is-part-of’ relationship between entity types, where one represents the ‘whole’ and the other the ‘part’. We may use composition (a special type of aggregation) to represent an association between entity types where there is a strong ownership and coincidental lifetime between the ‘whole’ and the ‘part’.

For the StaffClient user views of *DreamHome*, we choose to generalize the two entities PrivateOwner and BusinessOwner to create a superclass Owner that contains the common attributes ownerNo, address, and telNo. The relationship that the Owner superclass has with its subclasses is *mandatory* and *disjoint*, denoted as {Mandatory, Or}; each member of the Owner superclass must be a member of one of the subclasses, but cannot belong to both.



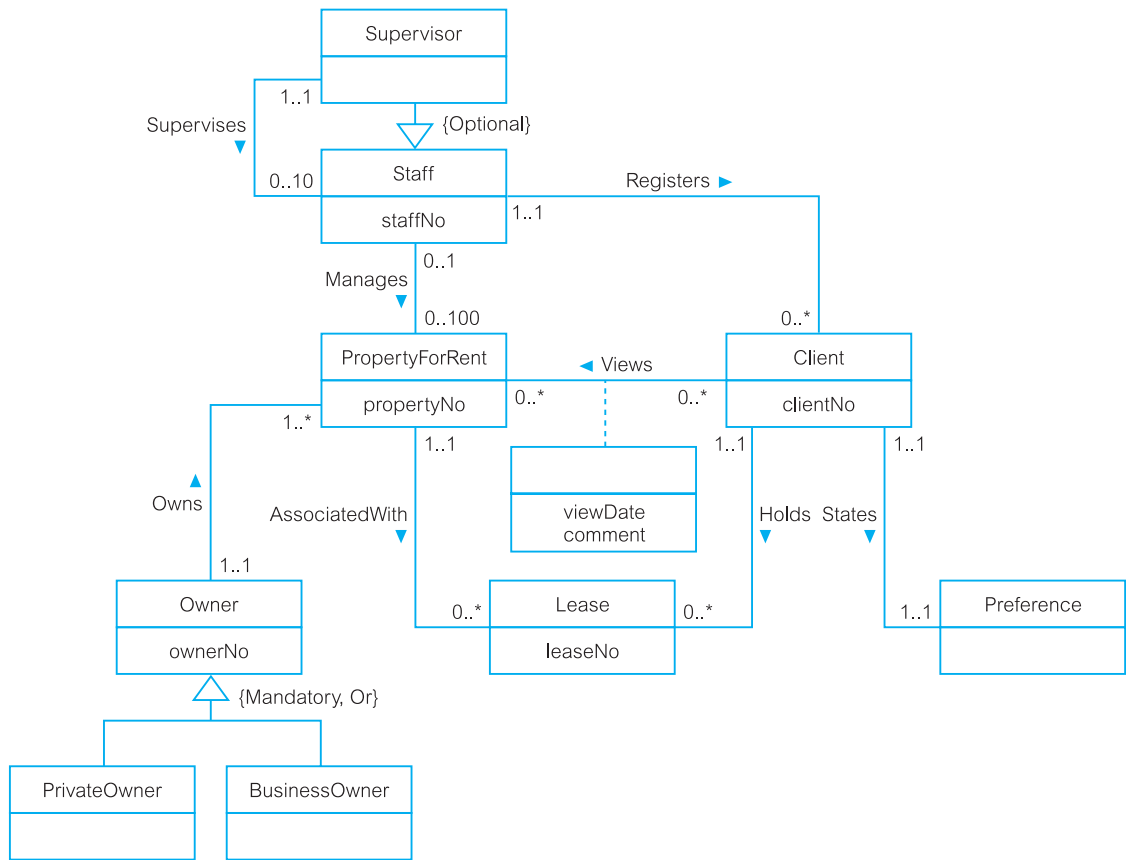


Figure 16.6 Revised ER diagram for the StaffClient user views of *DreamHome* with specialization/generalization added.

In addition, we identify one specialization subclass of *Staff*, namely *Supervisor*, specifically to model the *Supervises* relationship. The relationship that the *Staff* superclass has with the *Supervisor* subclass is *optional*: a member of the *Staff* superclass does not necessarily have to be a member of the *Supervisor* subclass. To keep the design simple, we decide not to use aggregation or composition. The revised ER diagram for the StaffClient user views of *DreamHome* is shown in Figure 16.6.

There are no strict guidelines on when to develop the ER model using enhanced modeling concepts, as the choice is often subjective and dependent on the particular characteristics of the situation that is being modeled. As a useful “rule of thumb” when considering the use of these concepts, always attempt to represent the important entities and their relationships as clearly as possible in the ER diagram. Therefore, the use of enhanced modeling concepts should be guided by the readability of the ER diagram and the clarity by which it models the important entities and relationships.

These concepts are associated with enhanced ER modeling. However, as this step is optional, we simply use the term “ER diagram” when referring to the diagrammatic representation of data models throughout the methodology.



Step 1.7: Check model for redundancy

Objective To check for the presence of any redundancy in the model.

In this step, we examine the conceptual data model with the specific objective of identifying whether there is any redundancy present and removing any that does exist. The three activities in this step are:

- (1) re-examine one-to-one (1:1) relationships;
- (2) remove redundant relationships;
- (3) consider time dimension.

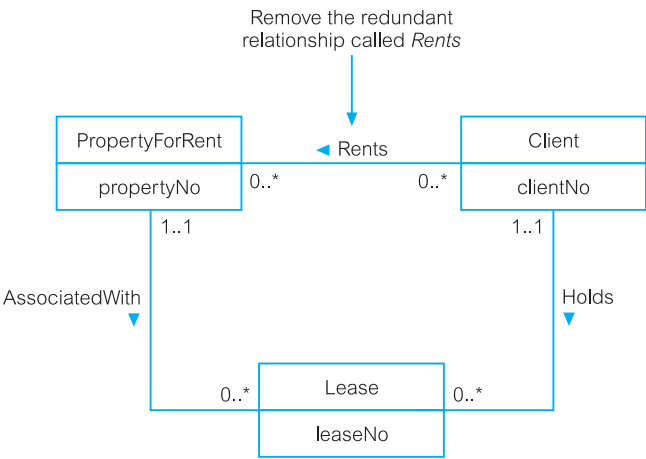
(1) Re-examine one-to-one (1:1) relationships

In the identification of entities, we may have identified two entities that represent the same object in the enterprise. For example, we may have identified the two entities *Client* and *Renter* that are actually the same; in other words, *Client* is a synonym for *Renter*. In this case, the two entities should be merged together. If the primary keys are different, choose one of them to be the primary key and leave the other as an alternate key.

(2) Remove redundant relationships

A relationship is redundant if the same information can be obtained via other relationships. We are trying to develop a minimal data model and, as redundant relationships are unnecessary, they should be removed. It is relatively easy to identify whether there is more than one path between two entities. However, this does not necessarily imply that one of the relationships is redundant, as they may represent different associations between the entities. For example, consider the relationships between the *PropertyForRent*, *Lease*, and *Client* entities shown in Figure 16.7. There are two ways to find out which clients rent which properties. There is the direct route using the *Rents* relationship between the *Client* and *PropertyForRent* entities, and there is the indirect route, using the *Holds* and *AssociatedWith* relationships via the *Lease* entity. Before we can assess whether both routes are required, we need

Figure 16.7
Remove the redundant relationship called *Rents*.



to establish the purpose of each relationship. The *Rents* relationship indicates which client rents which property. On the other hand, the *Holds* relationship indicates which client holds which lease, and the *AssociatedWith* relationship indicates which properties are associated with which leases. Although it is true that there is a relationship between clients and the properties they rent, this is not a direct relationship and the association is more accurately represented through a lease. The *Rents* relationship is therefore redundant and does not convey any additional information about the relationship between *PropertyForRent* and *Client* that cannot more correctly be found through the *Lease* entity. To ensure that we create a minimal model, the redundant *Rents* relationship must be removed.

(3) Consider time dimension

The time dimension of relationships is important when assessing redundancy. For example, consider the situation in which we wish to model the relationships between the entities *Man*, *Woman*, and *Child*, as illustrated in Figure 16.8. Clearly, there are two paths between *Man* and *Child*: one via the direct relationship *FatherOf* and the other via the relationships *MarriedTo* and *MotherOf*. Consequently, we may think that the relationship *FatherOf* is unnecessary. However, this would be incorrect for two reasons:

- (1) The father may have children from a previous marriage, and we are modeling only the father's current marriage through a 1:1 relationship.
- (2) The father and mother may not be married, or the father may be married to someone other than the mother (or the mother may be married to someone who is not the father).

In either case, the required relationship could not be modeled without the *FatherOf* relationship. The message is that it is important to examine the meaning of each relationship between entities when assessing redundancy. At the end of this step, we have simplified the local conceptual data model by removing any inherent redundancy.

Step 1.8: Validate conceptual data model against user transactions

Objective

To ensure that the conceptual data model supports the required transactions.

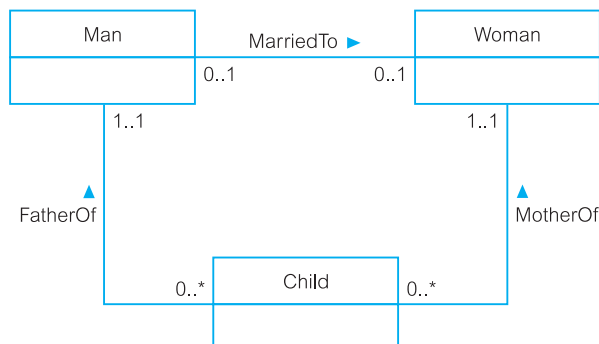


Figure 16.8
Example of a nonredundant relationship *FatherOf*.

We now have a conceptual data model that represents the data requirements of the enterprise. The objective of this step is to check the model to ensure that the model supports the required transactions. Using the model, we attempt to perform the operations manually. If we can resolve all transactions in this way, we have checked that the conceptual data model supports the required transactions. However, if we are unable to perform a transaction manually, there must be a problem with the data model, which must be resolved. In this case, it is likely that we have omitted an entity, a relationship, or an attribute from the data model.

We examine two possible approaches to ensuring that the conceptual data model supports the required transactions:

- (1) describing the transactions;
- (2) using transaction pathways.

Describing the transaction



Using the first approach, we check that all the information (entities, relationships, and their attributes) required by each transaction is provided by the model, by documenting a description of each transaction's requirements. We illustrate this approach for an example *DreamHome* transaction listed in Appendix A from the StaffClient user views:

Transaction (d): List the details of properties managed by a named member of staff at the branch

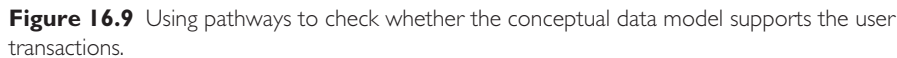
The details of properties are held in the *PropertyForRent* entity and the details of staff who manage properties are held in the *Staff* entity. In this case, we can use the *Staff Manages PropertyForRent* relationship to produce the required list.

Using transaction pathways

The second approach to validating the data model against the required transactions involves diagrammatically representing the pathway taken by each transaction directly on the ER diagram. An example of this approach for the query transactions for the StaffClient user views listed in Appendix A is shown in Figure 16.9. Clearly, the more transactions that exist, the more complex this diagram would become, so for readability we may need several such diagrams to cover all the transactions.

This approach allows the designer to visualize areas of the model that are not required by transactions and those areas that are critical to transactions. We are therefore in a position to directly review the support provided by the data model for the transactions required. If there are areas of the model that do not appear to be used by any transactions, we may question the purpose of representing this information in the data model. On the other hand, if there are areas of the model that are inadequate in providing the correct pathway for a transaction, we may need to investigate the possibility that critical entities, relationships, or attributes have been missed.

It may look like a lot of hard work to check every transaction that the model has to support in this way, and it certainly can be. As a result, it may be tempting to omit this step. However, it is very important that these checks are performed now



Step 1.9: Review conceptual data model with user

To review the conceptual data model with the users to ensure that they consider the model to be a “true” representation of the data requirements of the enterprise.

The steps in this methodology are summarized in Appendix D. The next chapter describes the steps of the logical database design methodology.

Chapter Summary

- A **design methodology** is a structured approach that uses procedures, techniques, tools, and documentation aids to support and facilitate the process of design.
- Database design includes three main phases: **conceptual**, **logical**, and **physical** database design.
- **Conceptual database design** is the process of constructing a model of the data used in an enterprise, independent of *all* physical considerations.
- Conceptual database design begins with the creation of a **conceptual data model** of the enterprise, which is entirely independent of implementation details such as the target DBMS, application programs, programming languages, hardware platform, performance issues, or any other physical considerations.
- **Logical database design** is the process of constructing a model of the data used in an enterprise based on a specific data model (such as the relational model), but independent of a particular DBMS and other physical considerations. Logical database design translates the conceptual data model into a **logical data model** of the enterprise.
- **Physical database design** is the process of producing a description of the implementation of the database on secondary storage; it describes the base relations, file organizations, and indexes used to achieve efficient access to the data, and any associated integrity constraints and security measures.
- The physical database design phase allows the designer to make decisions on how the database is to be implemented. Therefore, **physical design** is tailored to a specific DBMS. There is feedback between physical and conceptual/logical design, because decisions taken during physical design to improve performance may affect the structure of the conceptual/logical data model.
- There are several critical factors for the success of the database design stage, including, for example, working interactively with users and being willing to repeat steps.
- The main objective of Step 1 of the methodology is to build a conceptual data model of the data requirements of the enterprise. A conceptual data model comprises: entity types, relationship types, attributes, attribute domains, primary keys, and alternate keys.
- A conceptual data model is supported by documentation, such as ER diagrams and a data dictionary, which is produced throughout the development of the model.
- The conceptual data model is validated to ensure it supports the required transactions. Two possible approaches to ensure that the conceptual data model supports the required transactions are: (1) checking that all the information (entities, relationships, and their attributes) required by each transaction is provided by the model by documenting a description of each transaction's requirements; (2) diagrammatically representing the pathway taken by each transaction directly on the ER diagram.

Review Questions

- 16.1 Describe the purpose of a design methodology.
- 16.2 Describe the main phases involved in database design.
- 16.3 Identify important factors in the success of database design.
- 16.4 Discuss the important role played by users in the process of database design.
- 16.5 Describe the main objective of conceptual database design.
- 16.6 What is the purpose of conceptual database design? How is it utilized while designing a database?

- 16.7 How would you identify entity and relationship types from a user's requirements specification?
- 16.8 How would you identify attributes from a user's requirements specification and then associate the attributes with entity or relationship types?
- 16.9 Using enhanced modeling concepts in the conceptual design is optional. Describe the possible EER concepts that can be presented in a conceptual model.
- 16.10 How would you check a data model for redundancy? Give an example to illustrate your answer.
- 16.11 Discuss why you would want to validate a conceptual data model and describe two approaches to validating a conceptual data model.
- 16.12 Identify and describe the purpose of the documentation generated during conceptual database design.



Exercises



The *DreamHome* case study

- 16.13 Create a conceptual data model for the Branch user views of *DreamHome* documented in Appendix A. Compare your ER diagram with Figure 13.8 and justify any differences found.
- 16.14 Analyze the *DreamHome* case study and examine if there are situations that call for enhanced modeling. Present the enhanced data model of the case.

The *University Accommodation Office* case study

- 16.15 Describe how you will approach the task of creating the conceptual model for university accommodation documented in Appendix B.1.
- 16.16 Create a conceptual data model for the case study. State any assumptions necessary to support your design. Check that the conceptual data model supports the required transactions.

The *EasyDrive School of Motoring* case study

- 16.17 Analyze the *EasyDrive School of Motoring* case study documented in Appendix B.2 and prepare a data dictionary detailing all the key conceptual issues.
- 16.18 Create a conceptual data model for the case study. State any assumptions necessary to support your design. Check that the conceptual data model supports the required transactions.

The *Wellmeadows Hospital* case study

- 16.19 Identify user views for the Medical Director and Charge Nurse in the *Wellmeadows Hospital* case study described in Appendix B.3.
- 16.20 Provide a users' requirements specification for each of these user views.
- 16.21 Create conceptual data models for each of the user views. State any assumptions necessary to support your design.

Methodology—Logical Database Design for the Relational Model

Chapter Objectives

In this chapter you will learn:

- How to derive a set of relations from a conceptual data model.
- How to validate these relations using the technique of normalization.
- How to validate a logical data model to ensure it supports the required transactions.
- How to merge local logical data models based on one or more user views into a global logical data model that represents all user views.
- How to ensure that the final logical data model is a true and accurate representation of the data requirements of the enterprise.

In Chapter 10, we described the main stages of the database system development lifecycle, one of which is database design. This stage is made up of three phases: conceptual, logical, and physical database design. In the previous chapter we introduced a methodology that describes the steps that make up the three phases of database design and then presented Step 1 of this methodology for conceptual database design.

In this chapter we describe Step 2 of the methodology, which translates the conceptual model produced in Step 1 into a logical data model.

The methodology for logical database design described in this book also includes an optional Step 2.6, which is required when the database has multiple user views that are managed using the view integration approach (see Section 10.5). In this case, we repeat Steps 1 to 2.5 as necessary to create the required number of **local logical data models**, which are then finally merged in Step 2.6 to form a **global logical data model**. A local logical data model represents the data requirements of one or more but not all user views of a database and a global logical data model represents the data requirements for all user views (see Section 10.5). However, after concluding Step 2.6 we cease to use the term “global logical data model” and simply refer to the final model as being a “logical data model.” The final step of the logical database design phase is to consider how well the model is able to support possible future developments for the database system.

It is the logical data model created in Step 2 that forms the starting point for physical database design, which is described as Steps 3 to 8 in Chapters 18 and 19. Throughout the methodology, the terms “entity” and “relationship” are used in place of “entity type” and “relationship type” where the meaning is obvious; “type” is generally only added to avoid ambiguity.

17.1 Logical Database Design Methodology for the Relational Model

This section describes the steps of the logical database design methodology for the relational model.

Step 2: Build Logical Data Model

Objective

To translate the conceptual data model into a logical data model and then to validate this model to check that it is structurally correct and able to support the required transactions.

In this step, the main objective is to translate the conceptual data model created in Step 1 into a **logical data model** of the data requirements of the enterprise. This objective is achieved by following these activities:

- Step 2.1 Derive relations for logical data model
- Step 2.2 Validate relations using normalization
- Step 2.3 Validate relations against user transactions
- Step 2.4 Check integrity constraints
- Step 2.5 Review logical data model with user
- Step 2.6 Merge logical data models into global data model (optional step)
- Step 2.7 Check for future growth

We begin by deriving a set of relations (relational schema) from the conceptual data model created in Step 1. The structure of the relational schema is validated using normalization and then checked to ensure that the relations are capable of supporting the transactions given in the users' requirements specification. We next check whether all important integrity constraints are represented by the logical data model. At this stage, the logical data model is validated by the users to ensure that they consider the model to be a true representation of the data requirements of the enterprise.

The methodology for Step 2 is presented so that it is applicable for the design of simple to complex database systems. For example, to create a database with a single user view or with multiple user views that are managed using the centralized approach (see Section 10.5) then Step 2.6 is omitted. If, however, the database has multiple user views that are being managed using the view integration approach (see Section 10.5), then Steps 2.1 to 2.5 are repeated for the required number of data models, each of which represents different user views of the database system. In Step 2.6, these data models are merged.

Step 2 concludes with an assessment of the logical data model, which may or may not have involved Step 2.6, to ensure that the final model is able to support possible future developments. On completion of Step 2, we should have a single

logical data model that is a correct, comprehensive, and unambiguous representation of the data requirements of the enterprise.

We demonstrate Step 2 using the conceptual data model created in the previous chapter for the StaffClient user views of the *DreamHome* case study and represented in Figure 17.1 as an ER diagram. We also use the Branch user views of *DreamHome*, which is represented in Figure 13.8 as an ER diagram to illustrate some concepts that are not present in the StaffClient user views and to demonstrate the merging of data models in Step 2.6.

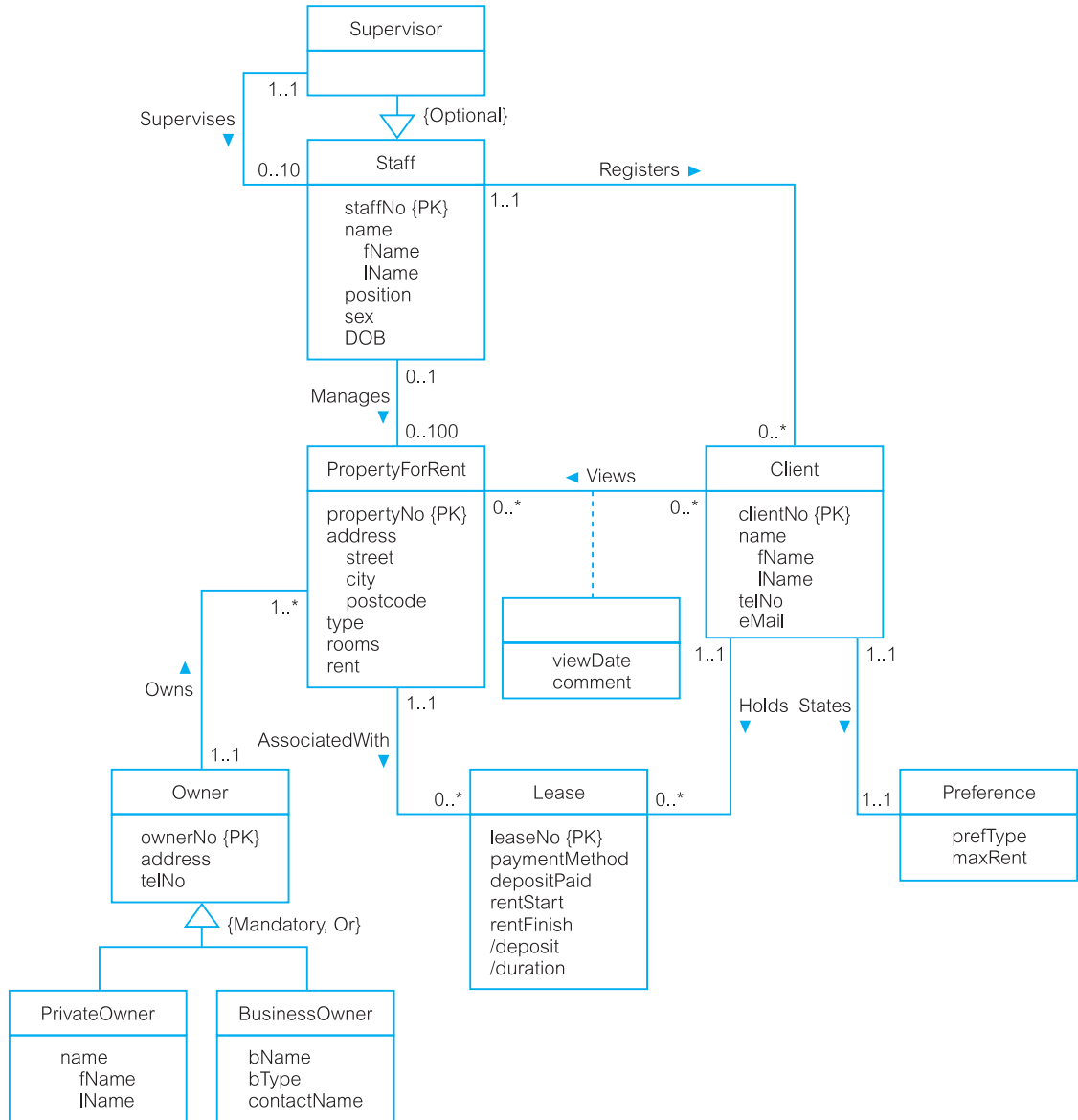


Figure 17.1 Conceptual data model for the StaffClient user views showing all attributes.

Step 2.1: Derive relations for logical data model**Objective**

To create relations for the logical data model to represent the entities, relationships, and attributes that have been identified.

In this step, we derive relations for the logical data model to represent the entities, relationships, and attributes. We describe the composition of each relation using a DBDL for relational databases. Using the DBDL, we first specify the name of the relation, followed by a list of the relation's simple attributes enclosed in brackets. We then identify the primary key and any alternate and/or foreign key(s) of the relation. Following the identification of a foreign key, the relation containing the referenced primary key is given. Any derived attributes are also listed, along with how each one is calculated.

The relationship that an entity has with another entity is represented by the primary key/ foreign key mechanism. In deciding where to post (or place) the foreign key attribute(s), we must first identify the “parent” and “child” entities involved in the relationship. The parent entity refers to the entity that posts a copy of its primary key into the relation that represents the child entity, to act as the foreign key.

We describe how relations are derived for the following structures that may occur in a conceptual data model:

- (1) strong entity types;
- (2) weak entity types;
- (3) one-to-many (1:*) binary relationship types;
- (4) one-to-one (1:1) binary relationship types;
- (5) one-to-one (1:1) recursive relationship types;
- (6) superclass/subclass relationship types;
- (7) many-to-many (*:*) binary relationship types;
- (8) complex relationship types;
- (9) multi-valued attributes.



For most of the examples discussed in the following sections, we use the conceptual data model for the StaffClient user views of *DreamHome*, which is represented as an ER diagram in Figure 17.1.

(1) Strong entity types

For each strong entity in the data model, create a relation that includes all the simple attributes of that entity. For composite attributes, such as name, include only the constituent simple attributes: fName and lName in the relation. For example, the composition of the Staff relation shown in Figure 17.1 is:

Staff (staffNo, fName, lName, position, sex, DOB)

Primary Key staffNo

(2) Weak entity types

For each weak entity in the data model, create a relation that includes all the simple attributes of that entity. The primary key of a weak entity is partially or fully derived from each owner entity and so the identification of the primary key of a weak entity

cannot be made until after all the relationships with the owner entities have been mapped. For example, the weak entity *Preference* in Figure 17.1 is initially mapped to the following relation:

Preference (prefType, maxRent)

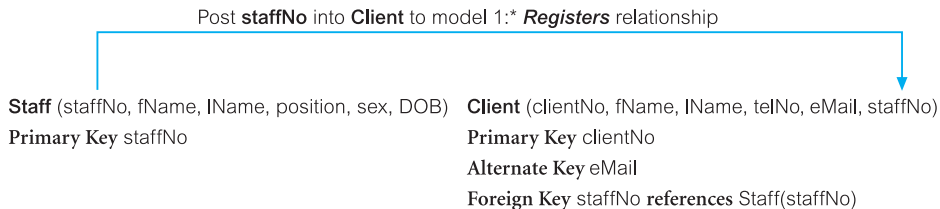
Primary Key None (at present)

In this situation, the primary key for the *Preference* relation cannot be identified until after the *States* relationship has been appropriately mapped.

(3) One-to-many (1:*) binary relationship types

For each 1:* binary relationship, the entity on the “one side” of the relationship is designated as the parent entity and the entity on the “many side” is designated as the child entity. To represent this relationship, we post a copy of the primary key attribute(s) of the parent entity into the relation representing the child entity, to act as a foreign key.

For example, the *Staff Registers Client* relationship shown in Figure 17.1 is a 1:* relationship, as a single member of staff can register many clients. In this example *Staff* is on the “one side” and represents the parent entity, and *Client* is on the “many side” and represents the child entity. The relationship between these entities is established by placing a copy of the primary key of the *Staff* (parent) entity, *staffNo*, into the *Client* (child) relation. The composition of the *Staff* and *Client* relations is:



In the case where a 1:* relationship has one or more attributes, these attributes should follow the posting of the primary key to the child relation. For example, if the *Staff Registers Client* relationship had an attribute called *dateRegister* representing when a member of staff registered the client, this attribute should also be posted to the *Client* relation, along with the copy of the primary key of the *Staff* relation, namely *staffNo*.

(4) One-to-one (1:1) binary relationship types

Creating relations to represent a 1:1 relationship is slightly more complex, as the cardinality cannot be used to help identify the parent and child entities in a relationship. Instead, the participation constraints (see Section 12.6.5) are used to help decide whether it is best to represent the relationship by combining the entities involved into one relation or by creating two relations and posting a copy of the primary key from one relation to the other. We consider how to create relations to represent the following participation constraints:

- (a) *mandatory* participation on *both* sides of 1:1 relationship;
- (b) *mandatory* participation on *one* side of 1:1 relationship;
- (c) *optional* participation on *both* sides of 1:1 relationship.

(a) Mandatory participation on both sides of 1:1 relationship In this case we should combine the entities involved into one relation and choose one of the primary keys of the original entities to be the primary key of the new relation, while the other (if one exists) is used as an alternate key.

The Client *States* Preference relationship is an example of a 1:1 relationship with mandatory participation on both sides. In this case, we choose to merge the two relations together to give the following Client relation:

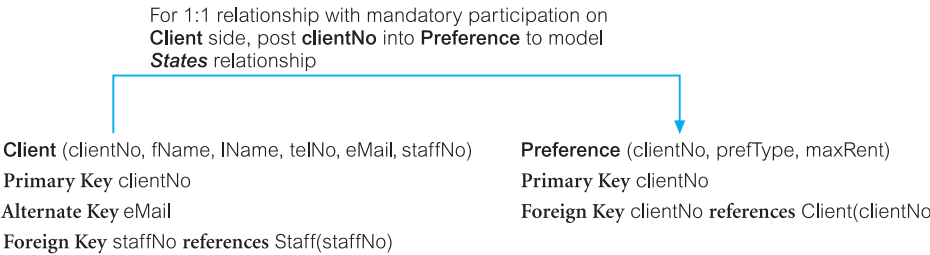
Client (clientNo, fName, lName, telNo, eMail, prefType, maxRent, staffNo)
Primary Key clientNo
Alternate Key eMail
Foreign Key staffNo **references** Staff(staffNo)

In the case where a 1:1 relationship with mandatory participation on both sides has one or more attributes, these attributes should also be included in the merged relation. For example, if the *States* relationship had an attribute called *dateStated* recording the date the preferences were stated, this attribute would also appear as an attribute in the merged Client relation.

Note that it is possible to merge two entities into one relation only when there are no other direct relationships between these two entities that would prevent this, such as a 1:* relationship. If this were the case, we would need to represent the *States* relationship using the primary key/foreign key mechanism. We discuss how to designate the parent and child entities in this type of situation in part (c) shortly.

(b) Mandatory participation on one side of a 1:1 relationship In this case we are able to identify the parent and child entities for the 1:1 relationship using the participation constraints. The entity that has optional participation in the relationship is designated as the parent entity, and the entity that has mandatory participation in the relationship is designated as the child entity. As described previously, a copy of the primary key of the parent entity is placed in the relation representing the child entity. If the relationship has one or more attributes, these attributes should follow the posting of the primary key to the child relation.

For example, if the 1:1 Client *States* Preference relationship had partial participation on the Client side (in other words, not every client specifies preferences), then the Client entity would be designated as the parent entity and the Preference entity would be designated as the child entity. Therefore, a copy of the primary key of the Client (parent) entity, clientNo, would be placed in the Preference (child) relation, giving:



Note that the foreign key attribute of the *Preference* relation also forms the relation's primary key. In this situation, the primary key for the *Preference* relation could not have been identified until after the foreign key had been posted from the *Client* relation to the *Preference* relation. Therefore, at the end of this step we should identify any new primary key or candidate keys that have been formed in the process, and update the data dictionary accordingly.

(c) Optional participation on both sides of a 1:1 relationship In this case the designation of the parent and child entities is arbitrary unless we can find out more about the relationship that can help us make a decision one way or the other.

For example, consider how to represent a 1:1 *Staff Uses Car* relationship with optional participation on both sides of the relationship. (Note that the discussion that follows is also relevant for 1:1 relationships with mandatory participation for both entities where we cannot select the option to combine the entities into a single relation.) If there is no additional information to help select the parent and child entities, the choice is arbitrary. In other words, we have the choice to post a copy of the primary key of the *Staff* entity to the *Car* entity, or vice versa.

However, assume that the majority of cars, but not all, are used by staff, and that only a minority of staff use cars. The *Car* entity, although optional, is closer to being mandatory than the *Staff* entity. We therefore designate *Staff* as the parent entity and *Car* as the child entity, and post a copy of the primary key of the *Staff* entity (*staffNo*) into the *Car* relation.

(5) One-to-one (1:1) recursive relationships

For a 1:1 recursive relationship, follow the rules for participation as described previously for a 1:1 relationship. However, in this special case of a 1:1 relationship, the entity on both sides of the relationship is the same. For a 1:1 recursive relationship with mandatory participation on both sides, represent the recursive relationship as a single relation with two copies of the primary key. As before, one copy of the primary key represents a foreign key and should be renamed to indicate the relationship it represents.

For a 1:1 recursive relationship with mandatory participation on only one side, we have the option to create a single relation with two copies of the primary key as described previously, or to create a new relation to represent the relationship. The new relation would have only two attributes, both copies of the primary key. As before, the copies of the primary keys act as foreign keys and have to be renamed to indicate the purpose of each in the relation.

For a 1:1 recursive relationship with optional participation on both sides, again create a new relation as described earlier.

(6) Superclass/subclass relationship types

For each superclass/subclass relationship in the conceptual data model, we identify the superclass entity as the parent entity and the subclass entity as the child entity. There are various options on how to represent such a relationship as one or more relations. The selection of the most appropriate option is dependent on a number of factors, such as the disjointness and participation constraints on the

TABLE 17.1 Guidelines for the representation of a superclass/subclass relationship based on the participation and disjoint constraints.

PARTICIPATION CONSTRAINT	DISJOINT CONSTRAINT	RELATIONS REQUIRED
Mandatory	Nondisjoint {And}	Single relation (with one or more discriminators to distinguish the type of each tuple)
Optional	Nondisjoint {And}	Two relations: one relation for superclass and one relation for all subclasses (with one or more discriminators to distinguish the type of each tuple)
Mandatory	Disjoint {Or}	Many relations: one relation for each combined superclass/subclass
Optional	Disjoint {Or}	Many relations: one relation for superclass and one for each subclass

superclass/subclass relationship (see Section 13.1.6), whether the subclasses are involved in distinct relationships, and the number of participants in the superclass/subclass relationship. Guidelines for the representation of a superclass/subclass relationship based only on the participation and disjoint constraints are shown in Table 17.1.

For example, consider the Owner superclass/subclass relationship shown in Figure 17.1. From Table 17.1, there are various ways to represent this relationship as one or more relations, as shown in Figure 17.2. The options range from placing all the attributes into one relation with two discriminators pOwnerFlag and bOwnerFlag indicating whether a tuple belongs to a particular subclass (Option 1) to dividing the attributes into three relations (Option 4). In this case the most appropriate representation of the superclass/subclass relationship is determined by the constraints on this relationship. From Figure 17.1, the relationship that the Owner superclass has with its subclasses is *mandatory* and *disjoint*, as each member of the Owner superclass must be a member of one of the subclasses (PrivateOwner or BusinessOwner) but cannot belong to both. We therefore select Option 3 as the best representation of this relationship and create a separate relation to represent each subclass, and include a copy of the primary key attribute(s) of the superclass in each.

It must be stressed that Table 17.1 is for guidance only as there may be other factors that influence the final choice. For example, with Option 1 (mandatory, nondisjoint) we have chosen to use two discriminators to distinguish whether the tuple is a member of a particular subclass. An equally valid way to represent this would be to have one discriminator that distinguishes whether the tuple is a member of PrivateOwner, BusinessOwner, or both. Alternatively, we could dispense with discriminators all together and simply test whether one of the attributes unique to a particular subclass has a value present to determine whether the tuple is a member of that subclass. In this case, we would have to ensure that the attributes examined allowed nulls to indicate nonmembership of a particular subclass.

In Figure 17.1, there is another superclass/subclass relationship between Staff and Supervisor with optional participation. However, as the Staff superclass only has

Option 1 – Mandatory, nondisjoint

Allowner (ownerNo, address, telNo, fName, lName, bName, bType, contactName, pOwnerFlag, bOwnerFlag)

Primary Key ownerNo

Option 2 – Optional, nondisjoint

Owner (ownerNo, address, telNo)

Primary Key ownerNo

OwnerDetails (ownerNo, fName lName, bName, bType, contactName, pOwnerFlag, bOwnerFlag)

Primary Key ownerNo

Foreign Key ownerNo **references** Owner(ownerNo)

Option 3 – Mandatory, disjoint

PrivateOwner (ownerNo, fName, lName, address, telNo)

Primary Key ownerNo

BusinessOwner (ownerNo, bName, bType, contactName, address, telNo)

Primary Key ownerNo

Option 4 – Optional, disjoint

Owner (ownerNo, address, telNo)

Primary Key ownerNo

PrivateOwner (ownerNo, fName, lName)

Primary Key ownerNo

Foreign Key ownerNo **references** Owner(ownerNo)

BusinessOwner (ownerNo, bName, bType, contactName)

Primary Key ownerNo

Foreign Key ownerNo **references** Owner(ownerNo)

Figure 17.2

Various representations of the Owner superclass/subclass relationship based on the participation and disjointness constraints shown in Table 17.1.

one subclass (Supervisor), there is no disjoint constraint. In this case, as there are many more “supervised staff” than supervisors, we choose to represent this relationship as a single relation:

Staff (staffNo, fName, lName, position, sex, DOB, supervisorStaffNo)

Primary Key staffNo

Foreign Key supervisorStaffNo **references** Staff(staffNo)

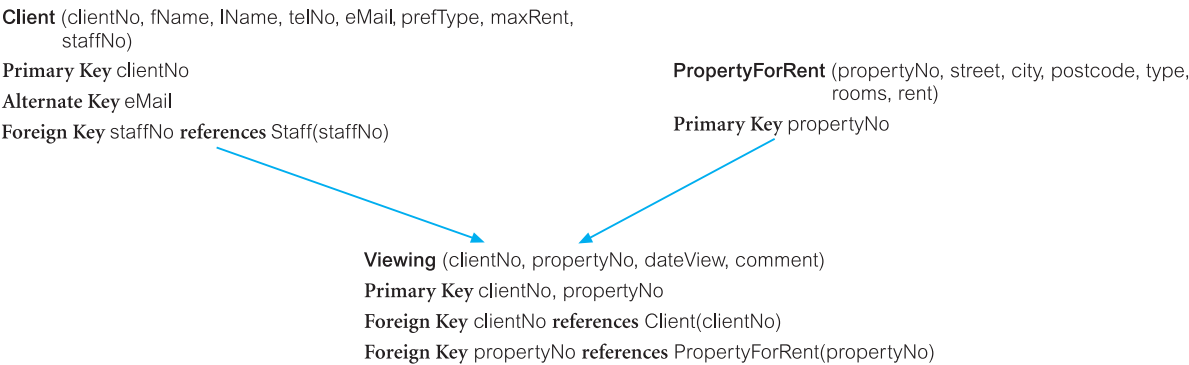
If we had left the superclass/subclass relationship as a 1:* recursive relationship as we had it originally in Figure 17.5 with optional participation on both sides this would have resulted in the same representation as previously.

(7) Many-to-many (*:*) binary relationship types

For each *:~ binary relationship, create a relation to represent the relationship and include any attributes that are part of the relationship. We post a copy of the primary key attribute(s) of the entities that participate in the relationship into the new relation, to act as foreign keys. One or both of these foreign keys will also form the primary key of the new relation, possibly in combination with one or more of the attributes of the relationship. (If one or more of the attributes that

form the relationship provide uniqueness, then an entity has been omitted from the conceptual data model, although this mapping process resolves this issue.)

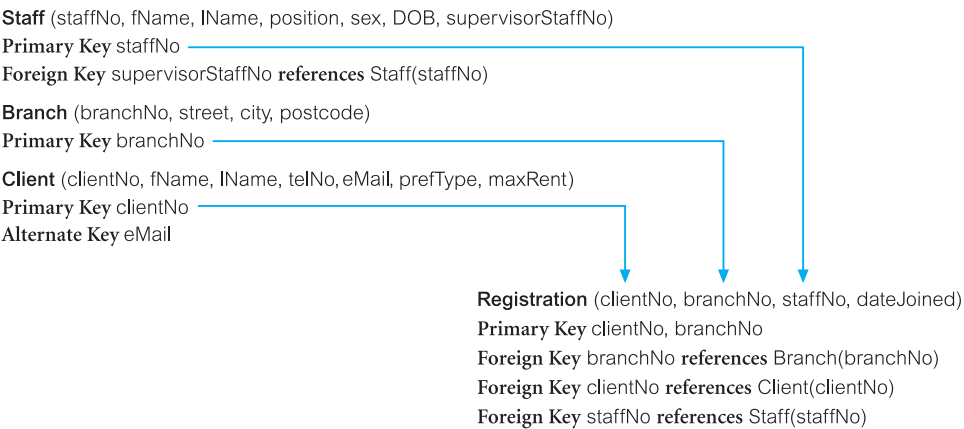
For example, consider the *Client Views PropertyForRent* relationship shown in Figure 17.1. In this example, the *Views* relationship has two attributes called *dateView* and *comments*. To represent this, we create relations for the strong entities *Client* and *PropertyForRent* and we create a relation *Viewing* to represent the relationship *Views*, to give:



(8) Complex relationship types

For each complex relationship, create a relation to represent the relationship and include any attributes that are part of the relationship. We post a copy of the primary key attribute(s) of the entities that participate in the complex relationship into the new relation, to act as foreign keys. Any foreign keys that represent a “many” relationship (for example, 1..*, 0..*) generally will also form the primary key of this new relation, possibly in combination with some of the attributes of the relationship.

For example, the ternary *Registers* relationship in the Branch user views represents the association between the member of staff who registers a new client at a branch, as shown in Figure 13.8. To represent this, we create relations for the strong entities *Branch*, *Staff*, and *Client*, and we create a relation *Registration* to represent the relationship *Registers*, to give:



Staff (staffNo, fName, lName, position, sex, DOB, supervisorStaffNo) Primary Key staffNo Foreign Key supervisorStaffNo references Staff(staffNo)	PrivateOwner (ownerNo, fName, lName, address, telNo) Primary Key ownerNo
BusinessOwner (ownerNo, bName, bType, contactName, address, telNo) Primary Key ownerNo Alternate Key bName Alternate Key telNo	Client (clientNo, fName, lName, telNo, eMail, prefType, maxRent, staffNo) Primary Key clientNo Alternate Key eMail Foreign Key staffNo references Staff(staffNo)
PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo, staffNo) Primary Key propertyNo Foreign Key ownerNo references PrivateOwner(ownerNo) and BusinessOwner(ownerNo) Foreign Key staffNo references Staff(staffNo)	Viewing (clientNo, propertyNo, dateView, comment) Primary Key clientNo, propertyNo Foreign Key clientNo references Client(clientNo) Foreign Key propertyNo references PropertyForRent(propertyNo)
Lease (leaseNo, paymentMethod, depositPaid, rentStart, rentFinish, clientNo, propertyNo) Primary Key leaseNo Alternate Key propertyNo, rentStart Alternate Key clientNo, rentStart Foreign Key clientNo references Client(clientNo) Foreign Key propertyNo references PropertyForRent(propertyNo) Derived deposit (PropertyForRent.rent*2) Derived duration (rentFinish – rentStart)	

Figure 17.3 Relations for the StaffClient user views of *DreamHome*.

Note that the *Registers* relationship is shown as a binary relationship in Figure 17.1 and this is consistent with its composition in Figure 17.3. The discrepancy between how *Registers* is modeled in the StaffClient (as a binary relationship) and Branch (as a complex [ternary] relationship) user views of *DreamHome* is discussed and resolved in Step 2.6.



(9) Multi-valued attributes

For each multi-valued attribute in an entity, create a new relation to represent the multi-valued attribute, and include the primary key of the entity in the new relation to act as a foreign key. Unless the multi-valued attribute, is itself an alternate key of the entity, the primary key of the new relation is the combination of the multi-valued attribute and the primary key of the entity.

For example, in the Branch user views to represent the situation where a single branch has up to three telephone numbers, the telNo attribute of the Branch entity has been defined as being a multi-valued attribute, as shown in Figure 13.8. To represent this, we create a relation for the Branch entity and we create a new relation called Telephone to represent the multi-valued attribute telNo, to give:



TABLE 17.2 Summary of how to map entities and relationships to relations.

ENTITY/RELATIONSHIP	MAPPING
Strong entity	Create relation that includes all simple attributes.
Weak entity	Create relation that includes all simple attributes (primary key still has to be identified after the relationship with each owner entity has been mapped).
1:* binary relationship	Post primary key of entity on the “one” side to act as foreign key in relation representing entity on the “many” side. Any attributes of relationship are also posted to the “many” side.
1:1 binary relationship: (a) Mandatory participation on both sides (b) Mandatory participation on one side	Combine entities into one relation. Post primary key of entity on the “optional” side to act as foreign key in relation representing entity on the “mandatory” side.
(c) Optional participation on both sides	Arbitrary without further information.
Superclass/subclass relationship	See Table 17.1.
*:~ binary relationship, complex relationship	Create a relation to represent the relationship and include any attributes of the relationship. Post a copy of the primary keys from each of the owner entities into the new relation to act as foreign keys.
Multi-valued attribute	Create a relation to represent the multi-valued attribute and post a copy of the primary key of the owner entity into the new relation to act as a foreign key.

Table 17.2 summarizes how to map entities and relationships to relations.

Document relations and foreign key attributes



At the end of Step 2.1, document the composition of the relations derived for the logical data model using the DBDL. The relations for the StaffClient user views of *DreamHome* are shown in Figure 17.3.

Now that each relation has its full set of attributes, we are in a position to identify any new primary and/or alternate keys. This is particularly important for weak entities that rely on the posting of the primary key from the parent entity (or entities) to form a primary key of their own. For example, the weak entity Viewing now has a composite primary key, made up of a copy of the primary key of the PropertyForRent entity (propertyNo) and a copy of the primary key of the Client entity (clientNo).

The DBDL syntax can be extended to show integrity constraints on the foreign keys (Step 2.5). The data dictionary should also be updated to reflect any new primary and alternate keys identified in this step. For example, following the posting of primary keys, the Lease relation has gained new alternate keys formed from the attributes (propertyNo, rentStart) and (clientNo, rentStart).

Step 2.2: Validate relations using normalization**Objective**

To validate the relations in the logical data model using normalization.

In the previous step we derived a set of relations to represent the conceptual data model created in Step 1. In this step we validate the groupings of attributes in each relation using the rules of normalization. The purpose of normalization is to ensure that the set of relations has a minimal yet sufficient number of attributes necessary to support the data requirements of the enterprise. Also, the relations should have minimal data redundancy to avoid the problems of update anomalies discussed in Section 14.3. However, some redundancy is essential to allow the joining of related relations.

The use of normalization requires that we first identify the functional dependencies that hold between the attributes in each relation. The characteristics of functional dependencies that are used for normalization were discussed in Section 14.4 and can be identified only if the meaning of each attribute is well understood. The functional dependencies indicate important relationships between the attributes of a relation. It is those functional dependencies and the primary key for each relation that are used in the process of normalization.

The process of normalization takes a relation through a series of steps to check whether the composition of attributes in a relation conforms or otherwise with the rules for a given normal form such as 1NF, 2NF, and 3NF. The rules for each normal form were discussed in detail in Sections 14.6 to 14.8. To avoid the problems associated with data redundancy, it is recommended that each relation be in at least 3NF.

The process of deriving relations from a conceptual data model should produce relations that are already in 3NF. If, however, we identify relations that are not in 3NF, this may indicate that part of the logical data model and/or conceptual data model is incorrect, or that we have introduced an error when deriving the relations from the conceptual data model. If necessary, we must restructure the problem relation(s) and/or data model(s) to ensure a true representation of the data requirements of the enterprise.

It is sometimes argued that a normalized database design does not provide maximum processing efficiency. However, the following points can be argued:

- A normalized design organizes the data according to its functional dependencies. Consequently, the process lies somewhere between conceptual and physical design.
- The logical design may not be the final design. It should represent the database designer's best understanding of the nature and meaning of the data required by the enterprise. If there are specific performance criteria, the physical design may be different. One possibility is that some normalized relations are denormalized, and this approach is discussed in detail in Step 7 of the physical database design methodology (see Chapter 19).
- A normalized design is robust and free of the update anomalies discussed in Section 14.3.
- Modern computers are much more powerful than those that were available a few years ago. It is sometimes reasonable to implement a design that gains ease of use at the expense of additional processing.

- To use normalization, a database designer must understand completely each attribute that is to be represented in the database. This benefit may be the most important.
- Normalization produces a flexible database design that can be extended easily.

Step 2.3: Validate relations against user transactions

Objective

To ensure that the relations in the logical data model support the required transactions.

The objective of this step is to validate the logical data model to ensure that the model supports the required transactions, as detailed in the users' requirements specification. This type of check was carried out in Step 1.8 to ensure that the conceptual data model supported the required transactions. In this step, we check whether the relations created in the previous step also support these transactions, and thereby ensure that no error has been introduced while creating relations.

Using the relations, the primary key/foreign key links shown in the relations, the ER diagram, and the data dictionary, we attempt to perform the operations manually. If we can resolve all transactions in this way, we have validated the logical data model against the transactions. However, if we are unable to perform a transaction manually, there must be a problem with the data model that must be resolved. In this case, it is likely that an error has been introduced while creating the relations and we should go back and check the areas of the data model that the transaction is accessing to identify and resolve the problem.

Step 2.4: Check integrity constraints

Objective

To check whether integrity constraints are represented in the logical data model.

Integrity constraints are the constraints that we wish to impose in order to protect the database from becoming incomplete, inaccurate, or inconsistent. Although DBMS controls for integrity constraints may or may not exist, this is not the question here. At this stage we are concerned only with high-level design, that is, specifying *what* integrity constraints are required, irrespective of *how* this might be achieved. A logical data model that includes all important integrity constraints is a “true” representation of the data requirements for the enterprise. We consider the following types of integrity constraint:

- required data;
- attribute domain constraints;
- multiplicity;
- entity integrity;
- referential integrity;
- general constraints.

Required data

Some attributes must always contain a valid value; in other words, they are not allowed to hold nulls. For example, every member of staff must have an associated

job position (such as Supervisor or Assistant). These constraints should have been identified when we documented the attributes in the data dictionary (Step 1.3).

Attribute domain constraints

Every attribute has a domain, that is, a set of values that are legal. For example, the sex of a member of staff is either “M” or “F,” so the domain of the sex attribute is a single character string consisting of “M” or “F.” These constraints should have been identified when we chose the attribute domains for the data model (Step 1.4).

Multiplicity

Multiplicity represents the constraints that are placed on relationships between data in the database. Examples of such constraints include the requirements that a branch has many staff and a member of staff works at a single branch. Ensuring that all appropriate integrity constraints are identified and represented is an important part of modeling the data requirements of an enterprise. In Step 1.2 we defined the relationships between entities, and all integrity constraints that can be represented in this way were defined and documented in this step.

Entity integrity

The primary key of an entity cannot hold nulls. For example, each tuple of the Staff relation must have a value for the primary key attribute, *staffNo*. These constraints should have been considered when we identified the primary keys for each entity type (Step 1.5).

Referential integrity

A foreign key links each tuple in the child relation to the tuple in the parent relation containing the matching candidate key value. Referential integrity means that if the foreign key contains a value, that value must refer to an existing tuple in the parent relation. For example, consider the Staff *Manages* PropertyForRent relationship. The *staffNo* attribute in the PropertyForRent relation links the property for rent to the tuple in the Staff relation containing the member of staff who manages that property. If *staffNo* is not null, it must contain a valid value that exists in the *staffNo* attribute of the Staff relation, or the property will be assigned to a nonexistent member of staff.

There are two issues regarding foreign keys that must be addressed. The first considers whether nulls are allowed for the foreign key. For example, can we store the details of a property for rent without having a member of staff specified to manage it—that is, can we specify a null *staffNo*? The issue is not whether the staff number exists, but whether a staff number must be specified. In general, if the participation of the child relation in the relationship is:

- mandatory, then nulls are not allowed;
- optional, then nulls are allowed.

The second issue we must address is how to ensure referential integrity. To do this, we specify **existence constraints** that define conditions under which a candidate key or foreign key may be inserted, updated, or deleted. For the 1:* Staff *Manages* PropertyForRent relationship, consider the following cases.

Case 1: Insert tuple into child relation (PropertyForRent) To ensure referential integrity, check that the foreign key attribute, `staffNo`, of the new `PropertyForRent` tuple is set to null or to a value of an existing `Staff` tuple.

Case 2: Delete tuple from child relation (PropertyForRent) If a tuple of a child relation is deleted referential integrity is unaffected.

Case 3: Update foreign key of child tuple (PropertyForRent) This case is similar to Case 1. To ensure referential integrity, check whether the `staffNo` of the updated `PropertyForRent` tuple is set to null or to a value of an existing `Staff` tuple.

Case 4: Insert tuple into parent relation (Staff) Inserting a tuple into the parent relation (`Staff`) does not affect referential integrity; it simply becomes a parent without any children: in other words, a member of staff without properties to manage.

Case 5: Delete tuple from parent relation (Staff) If a tuple of a parent relation is deleted, referential integrity is lost if there exists a child tuple referencing the deleted parent tuple; in other words, if the deleted member of staff currently manages one or more properties. There are several strategies we can consider:

- **NO ACTION**—Prevent a deletion from the parent relation if there are any referenced child tuples. In our example, “You cannot delete a member of staff if he or she currently manages any properties.”
- **CASCADE**—When the parent tuple is deleted, automatically delete any referenced child tuples. If any deleted child tuple acts as the parent in another relationship, then the delete operation should be applied to the tuples in this child relation, and so on in a cascading manner. In other words, deletions from the parent relation cascade to the child relation. In our example, “Deleting a member of staff automatically deletes all properties he or she manages.” Clearly, in this situation, this strategy would not be wise. If we have used the enhanced modeling technique of *composition* to relate the parent and child entities, **CASCADE** should be specified (see Section 13.3).
- **SET NULL**—When a parent tuple is deleted, the foreign key values in all corresponding child tuples are automatically set to null. In our example, “If a member of staff is deleted, indicate that the current assignment of those properties previously managed by that employee is unknown.” We can consider this strategy only if the attributes constituting the foreign key accept nulls.
- **SET DEFAULT**—When a parent tuple is deleted, the foreign key values in all corresponding child tuples should automatically be set to their default values. In our example, “If a member of staff is deleted, indicate that the current assignment of some properties is being handled by another (default) member of staff such as the Manager.” We can consider this strategy only if the attributes constituting the foreign key have default values defined.
- **NO CHECK**—When a parent tuple is deleted, do nothing to ensure that referential integrity is maintained.

Case 6: Update primary key of parent tuple (Staff) If the primary key value of a parent relation tuple is updated, referential integrity is lost if there exists a child tuple referencing the old primary key value; that is, if the updated member of staff

```

Staff (staffNo, fName, lName, position, sex, DOB, supervisorStaffNo)
Primary Key staffNo
Foreign Key supervisorStaffNo references Staff(staffNo) ON UPDATE CASCADE ON DELETE SET NULL

Client (clientNo, fName, lName, telNo, eMail, prefType, maxRent, staffNo)
Primary Key clientNo
Alternate Key eMail
Foreign Key staffNo references Staff(staffNo) ON UPDATE CASCADE ON DELETE NO ACTION

PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo, staffNo)
Primary Key propertyNo
Foreign Key ownerNo references PrivateOwner(ownerNo) and BusinessOwner(ownerNo)
ON UPDATE CASCADE ON DELETE NO ACTION

Foreign Key staffNo references Staff(staffNo) ON UPDATE CASCADE ON DELETE SET NULL

Viewing (clientNo, propertyNo, dateView, comment)
Primary Key clientNo, propertyNo
Foreign Key clientNo references Client(clientNo) ON UPDATE CASCADE ON DELETE NO ACTION
Foreign Key propertyNo references PropertyForRent(propertyNo)
ON UPDATE CASCADE ON DELETE CASCADE

Lease (leaseNo, paymentMethod, depositPaid, rentStart, rentFinish, clientNo, propertyNo)
Primary Key leaseNo
Alternate Key propertyNo, rentStart
Alternate Key clientNo, rentStart
Foreign Key clientNo references Client(clientNo) ON UPDATE CASCADE ON DELETE NO ACTION
Foreign Key propertyNo references PropertyForRent(propertyNo)
ON UPDATE CASCADE ON DELETE NO ACTION

```

Figure 17.4 Referential integrity constraints for the relations in the StaffClient user views of *DreamHome*.

currently manages one or more properties. To ensure referential integrity, the strategies described earlier can be used. In the case of CASCADE, the updates to the primary key of the parent tuple are reflected in any referencing child tuples, and if a referencing child tuple is itself a primary key of a parent tuple, this update will also cascade to its referencing child tuples, and so on in a cascading manner. It is normal for updates to be specified as CASCADE.

The referential integrity constraints for the relations that have been created for the StaffClient user views of *DreamHome* are shown in Figure 17.4.



General constraints

Finally, we consider constraints known as general constraints. Updates to entities may be controlled by constraints governing the “real-world” transactions that are represented by the updates. For example, *DreamHome* has a rule that prevents a member of staff from managing more than 100 properties at the same time.

Document all integrity constraints

Document all integrity constraints in the data dictionary for consideration during physical design.

Step 2.5: Review logical data model with user**Objective**

To review the logical data model with the users to ensure that they consider the model to be a true representation of the data requirements of the enterprise.

The logical data model should now be complete and fully documented. However, to confirm that this is the case, users are requested to review the logical data model to ensure that they consider the model to be a true representation of the data requirements of the enterprise. If the users are dissatisfied with the model, then some repetition of earlier steps in the methodology may be required.

If the users are satisfied with the model, then the next step taken depends on the number of user views associated with the database and, more importantly, how they are being managed. If the database system has a single user view or multiple user views that are being managed using the centralization approach (see Section 10.5), then we proceed directly to the final step of Step 2: Step 2.7. If the database has multiple user views that are being managed using the view integration approach (see Section 10.5), then we proceed to Step 2.6. The view integration approach results in the creation of several logical data models, each of which represents one or more, but not all, user views of a database. The purpose of Step 2.6 is to merge these data models to create a single logical data model that represents all user views of a database. However, before we consider this step, we discuss briefly the relationship between logical data models and data flow diagrams.

Relationship between logical data model and data flow diagrams

A logical data model reflects the structure of stored data for an enterprise. A Data Flow Diagram (DFD) shows data moving about the enterprise and being stored in datastores. All attributes should appear within an entity type if they are held within the enterprise, and will probably be seen flowing around the enterprise as a data flow. When these two techniques are being used to model the users' requirements specification, we can use each one to check the consistency and completeness of the other. The rules that control the relationship between the two techniques are:

- each datastore should represent a whole number of entity types;
- attributes on data flows should belong to entity types.

Step 2.6: Merge logical data models into global model (optional step)**Objective**

To merge local logical data models into a single global logical data model that represents all user views of a database.

This step is necessary only for the design of a database with multiple user views that are being managed using the view integration approach. To facilitate the description of the merging process, we use the terms “local logical data model” and “global logical data model.” A **local logical data model** represents one or more but not all user views of a database whereas **global logical data model** represents *all* user views of a database. In this step we merge two or more local logical data models into a single global logical data model.

The source of information for this step is the local data models created through Step 1 and Steps 2.1 to 2.5 of the methodology. Although each local logical data model should be correct, comprehensive, and unambiguous, each model is a representation only of one or more but not all user views of a database. In other words, each model represents only part of the complete database. This may mean that there are inconsistencies as well as overlaps when we look at the complete set of user views. Thus, when we merge the local logical data models into a single global model, we must endeavor to resolve conflicts between the user views and any overlaps that exist.

Therefore, on completion of the merging process, the resulting global logical data model is subjected to validations similar to those performed on the local data models. The validations are particularly necessary and should be focused on areas of the model that are subjected to most change during the merging process.

The activities in this step include:

Step 2.6.1 Merge local logical data models into global logical data model

Step 2.6.2 Validate global logical data model

Step 2.6.3 Review global logical data model with users

We demonstrate this step using the local logical data model developed previously for the StaffClient user views of the *DreamHome* case study and using the model developed in Chapters 12 and 13 for the Branch user views of *DreamHome*. Figure 17.5 shows the relations created from the ER model for the Branch user views given in Figure 13.8. We leave it as an exercise for the reader to show that this mapping is correct (see Exercise 17.6).



Step 2.6.1: Merge local logical data models into global logical data model

Objective

To merge local logical data models into a single global logical data model.

Up to this point, for each local logical data model we have produced an ER diagram, a relational schema, a data dictionary, and supporting documentation that describes the constraints on the data. In this step, we use these components to identify the similarities and differences between the models and thereby help merge the models together.

For a simple database system with a small number of user views, each with a small number of entity and relationship types, it is a relatively easy task to compare the local models, merge them together, and resolve any differences that exist. However, in a large system, a more systematic approach must be taken. We present one approach that may be used to merge the local models together and resolve any inconsistencies found. For a discussion on other approaches, the interested reader is referred to the papers by Batini and Lanzerini (1986), Biskup and Convent (1986), Spaccapietra *et al.* (1992) and Bouguettaya *et al.* (1998).

Some typical tasks in this approach are:

- (1) Review the names and contents of entities/relations and their candidate keys.
- (2) Review the names and contents of relationships/foreign keys.
- (3) Merge entities/relations from the local data models.

Branch (branchNo, street, city, postcode, mgrStaffNo) Primary Key branchNo Alternate Key postcode Foreign Key mgrStaffNo references Manager(staffNo)	Telephone (telNo, branchNo) Primary Key telNo Foreign Key branchNo references Branch(branchNo)
Staff (staffNo, name, position, salary, supervisorStaffNo, branchNo) Primary Key staffNo Foreign Key supervisorStaffNo references Staff(staffNo) Foreign Key branchNo references Branch(branchNo)	Manager (staffNo, mgrStartDate, bonus) Primary Key staffNo Foreign Key staffNo references Staff(staffNo)
PrivateOwner (ownerNo, name, address, telNo) Primary Key ownerNo	BusinessOwner (bName, bType, contactName, address, telNo) Primary Key bName Alternate Key telNo
PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo, staffNo, bName, branchNo) Primary Key propertyNo Foreign Key ownerNo references PrivateOwner(ownerNo) Foreign Key bName references BusinessOwner(bName) Foreign Key staffNo references Staff(staffNo) Foreign Key branchNo references Branch(branchNo)	Client (clientNo, name, telNo, eMail, prefType, maxRent) Primary Key clientNo Alternate Key eMail
Lease (leaseNo, paymentMethod, depositPaid, rentStart, rentFinish, clientNo, propertyNo) Primary Key leaseNo Alternate Key propertyNo, rentStart Alternate Key clientNo, rentStart Foreign Key clientNo references Client(clientNo) Foreign Key propertyNo references PropertyForRent(propertyNo) Derived deposit (PropertyForRent.rent*2) Derived duration (rentFinish – rentStart)	Registration (clientNo, branchNo, staffNo, dateJoined) Primary Key clientNo, branchNo Foreign Key clientNo references Client(clientNo) Foreign Key branchNo references Branch(branchNo) Foreign Key staffNo references Staff(staffNo)
Advert (propertyNo, newspaperName, dateAdvert, cost) Primary Key propertyNo, newspaperName, dateAdvert Foreign Key propertyNo references PropertyForRent(propertyNo) Foreign Key newspaperName references Newspaper(newspaperName)	Newspaper (newspaperName, address, telNo, contactName) Primary Key newspaperName Alternate Key telNo

Figure 17.5 Relations for the Branch user views of *DreamHome*.

- (4) Include (without merging) entities/relations unique to each local data model.
- (5) Merge relationships/foreign keys from the local data models.
- (6) Include (without merging) relationships/foreign keys unique to each local data model.
- (7) Check for missing entities/relations and relationships/foreign keys.
- (8) Check foreign keys.
- (9) Check integrity constraints.
- (10) Draw the global ER/relation diagram.
- (11) Update the documentation.

In some of these tasks, we have used the term “entities/relations” and “relationships/foreign keys.” This allows the designer to choose whether to examine the ER

models or the relations that have been derived from the ER models in conjunction with their supporting documentation, or even to use a combination of both approaches. It may be easier to base the examination on the composition of relations, as this removes many syntactic and semantic differences that may exist between different ER models possibly produced by different designers.

Perhaps the easiest way to merge several local data models together is first to merge two of the data models to produce a new model, and then successively to merge the remaining local data models until all the local models are represented in the final global data model. This may prove a simpler approach than trying to merge all the local data models at the same time.

(1) Review the names and contents of entities/relations and their candidate keys

It may be worthwhile to review the names and descriptions of entities/relations that appear in the local data models by inspecting the data dictionary. Problems can arise when two or more entities/relations:

- have the same name but are, in fact, different (homonyms);
- are the same but have different names (synonyms).

It may be necessary to compare the data content of each entity/relation to resolve these problems. In particular, use the candidate keys to help identify equivalent entities/relations that may be named differently across user views. A comparison of the relations in the Branch and StaffClient user views of *DreamHome* is shown in Table 17.3. The relations that are common to each user views are highlighted.



(2) Review the names and contents of relationships/foreign keys

This activity is the same as described for entities/relations. A comparison of the foreign keys in the Branch and StaffClient user views of *DreamHome* is shown in Table 17.4. The foreign keys that are common to each user view are highlighted. Note, in particular, that of the relations that are common to both user views, the *Staff* and *PropertyForRent* relations have an extra foreign key, *branchNo*.



This initial comparison of the relationship names/foreign keys in each view again gives some indication of the extent to which the user views overlap. However, it is important to recognize that we should not rely too heavily on the fact that entities or relationships with the same name play the same role in both user views. However, comparing the names of entities/relations and relationships/foreign keys is a good starting point when searching for overlap between the user views, as long as we are aware of the pitfalls.

We must be careful of entities or relationships that have the same name but in fact represent different concepts (also called homonyms). An example of this occurrence is the *Staff Manages PropertyForRent* (StaffClient user views) and *Manager Manages Branch* (Branch user views). Obviously, the *Manages* relationship in this case means something different in each user view.

We must therefore ensure that entities or relationships that have the same name represent the same concept in the “real world,” and that the names that differ in each user view represent different concepts. To achieve this, we compare the attributes (and, in particular, the keys) associated with each entity and also their associated

TABLE 17.3 A comparison of the names of entities/relations and their candidate keys in the Branch and StaffClient user views.

BRANCH USER VIEWS		STAFFCLIENT USER VIEWS	
ENTITY/RELATION	CANDIDATE KEYS	ENTITY/RELATION	CANDIDATE KEYS
Branch	branchNo postcode		
Telephone	telNo		
Staff	staffNo	Staff	staffNo
Manager	staffNo		
PrivateOwner	ownerNo	PrivateOwner	ownerNo
BusinessOwner	bName telNo	BusinessOwner	bName telNo ownerNo
Client	clientNo eMail	Client	clientNo eMail
PropertyForRent	propertyNo	PropertyForRent Viewing	propertyNo clientNo, propertyNo
Lease	leaseNo propertyNo, rentStart clientNo, rentStart	Lease	leaseNo propertyNo, rentStart clientNo, rentStart
Registration	(clientNo, branchNo)		
Newspaper	newspaperName telNo		
Advert	(propertyNo, newspaperName, dateAdvert)		

relationships with other entities. We should also be aware that entities or relationships in one user view may be represented simply as attributes in another user view. For example, consider the scenario where the Branch entity has an attribute called manager Name in one user view, which is represented as an entity called Manager in another user view.

(3) Merge entities/relations from the local data models

Examine the name and content of each entity/relation in the models to be merged to determine whether entities/relations represent the same thing and can therefore be merged. Typical activities involved in this task include:

- merging entities/relations with the same name and the same primary key;
- merging entities/relations with the same name but different primary keys;
- merging entities/relations with different names using the same or different primary keys.

TABLE 17.4 A comparison of the foreign keys in the Branch and StaffClient user views.

BRANCH USER VIEWS			STAFFCLIENT USER VIEWS		
CHILD RELATION	FOREIGN KEYS	PARENT RELATION	CHILD RELATION	FOREIGN KEYS	PARENT RELATION
Branch	mgrStaffNo →	Manager(staffNo)			
Telephone ^a	branchNo →	Branch(branchNo)			
Staff	supervisorStaffNo → branchNo →	Staff(staffNo) Branch(branchNo)	Staff	supervisorStaffNo →	Staff(staffNo)
Manager	staffNo →	Staff(staffNo)			
PrivateOwner			PrivateOwner		
Business Owner			Business Owner		
Client			Client	StaffNo →	Staff(staffNo)
PropertyForRent	ownerNo → bName → staffNo → branchNo →	PrivateOwner(ownerNo) BusinessOwner(ownerNo) Staff(staffNo) Branch(branchNo)	PropertyForRent	ownerNo → ownerNo → staffNo →	PrivateOwner(ownerNo) PrivateOwner(ownerNo) Staff(staffNo)
			Viewing	clientNo → propertyNo →	Client(clientNo) PropertyForRent(propertyNo)
Lease	clientNo → propertyNo →	Client(clientNo) PropertyForRent(propertyNo)	Lease	clientNo → propertyNo →	Client(clientNo) PropertyForRent(propertyNo)
Registration ^b	clientNo → branchNo → staffNo →	Client(clientNo) Branch(branchNo) Staff(staffNo)			
Newspaper					
Advert ^c	propertyNo → newspaperName →	PropertyForRent(propertyNo) Newspaper(newspaperName)			

^aThe Telephone relation is created from the multi-valued attribute telNo.
^bThe Registration relation is created from the ternary relationship Registers.
^cThe Advert relation is created from the many-to-many (*,*) relationship Advertises.

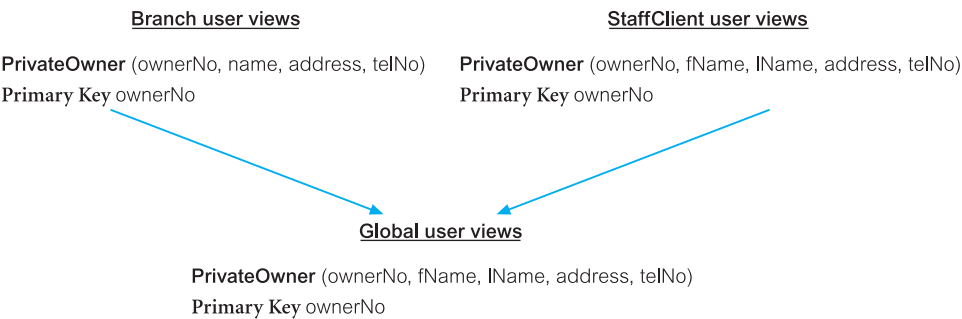


Figure 17.6 Merging the PrivateOwner relations from the Branch and StaffClient user views.

Merging entities/relations with the same name and the same primary key Generally, entities/relations with the same primary key represent the same “real-world” object and should be merged. The merged entity/relation includes the attributes from the original entities/relations with duplicates removed. For example, Figure 17.6 lists the attributes associated with the relation PrivateOwner defined in the Branch and StaffClient user views. The primary key of both relations is ownerNo. We merge these two relations together by combining their attributes, so that the merged PrivateOwner relation now has all the original attributes associated with both PrivateOwner relations. Note that there is conflict between the user views on how we should represent the name of an owner. In this situation, we should (if possible) consult the users of each user view to determine the final representation. Note that in this example, we use the decomposed version of the owner’s name, represented by the fName and lName attributes, in the merged global view.

In a similar way, from Table 17.2 the Staff, Client, PropertyForRent, and Lease relations have the same primary keys in both user views and the relations can be merged as discussed earlier.

Merging entities/relations with the same name but different primary keys In some situations, we may find two entities/relations with the same name and similar candidate keys, but with different primary keys. In this case, the entities/relations should be merged together as described previously. However, it is necessary to choose one key to be the primary key, the others becoming alternate keys. For example, Figure 17.7 lists the attributes associated with the two relations BusinessOwner defined in the two user views. The primary key of the BusinessOwner relation in the Branch user views is bName and the primary key of the BusinessOwner relation in the StaffClient user views is ownerNo. However, the alternate key for BusinessOwner in the StaffClient user views is bName. Although the primary keys are different, the primary key of BusinessOwner in the Branch user views is the alternate key of BusinessOwner in the StaffClient user views. We merge these two relations together as shown in Figure 17.7 and include bName as an alternate key.

Merging entities/relations with different names using the same or different primary keys In some cases, we may identify entities/relations that have different

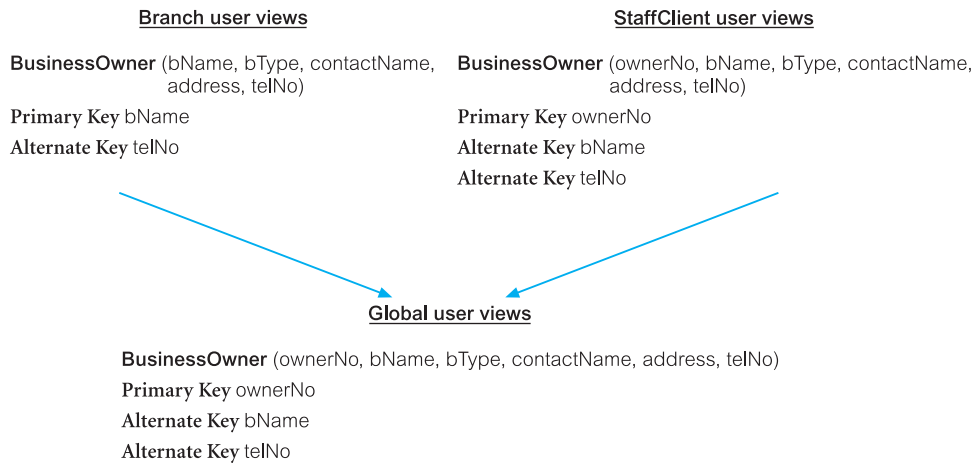


Figure 17.7 Merging the BusinessOwner relations with different primary keys.

names but appear to have the same purpose. These equivalent entities/relations may be recognized simply by:

- their name, which indicates their similar purpose;
- their content and, in particular, their primary key;
- their association with particular relationships.

An obvious example of this occurrence would be entities called *Staff* and *Employee*, which if found to be equivalent should be merged.

(4) Include (without merging) entities/relations unique to each local data model

The previous tasks should identify all entities/relations that are the same. All remaining entities/relations are included in the global model without change. From Table 17.2, the *Branch*, *Telephone*, *Manager*, *Registration*, *Newspaper*, and *Advert* relations are unique to the Branch user views, and the *Viewing* relation is unique to the StaffClient user views.

(5) Merge relationships/foreign keys from the local data models

In this step we examine the name and purpose of each relationship/foreign key in the data models. Before merging relationships/foreign keys, it is important to resolve any conflicts between the relationships, such as differences in multiplicity constraints. The activities in this step include:

- merging relationships/foreign keys with the same name and the same purpose;
- merging relationships/foreign keys with different names but the same purpose.

Using Table 17.3 and the data dictionary, we can identify foreign keys with the same name and the same purpose which can be merged into the global model.

Note that the *Registers* relationship in the two user views essentially represents the same ‘event’: in the StaffClient user views, the *Registers* relationship models a member of staff registering a client; and this is represented using *staffNo* as a foreign Key in Client: in the Branch user views, the situation is slightly more complex, due to the additional modeling of branches, and this requires a new relation called *Registration* to model a member of staff registering a client at a branch. In this case, we ignore the *Registers* relationship in the StaffClient user views and include the equivalent relationships/foreign keys from the Branch user views in the next step.

(6) Include (without merging) relationships/foreign keys unique to each local data model

Again, the previous task should identify relationships/foreign keys that are the same (by definition, they must be between the same entities/relations, which would have been merged together earlier). All remaining relationships/foreign keys are included in the global model without change.

(7) Check for missing entities/relations and relationships/foreign keys

Perhaps one of the most difficult tasks in producing the global model is identifying missing entities/relations and relationships/foreign keys between different local data models. If a corporate data model exists for the enterprise, this may reveal entities and relationships that do not appear in any local data model. Alternatively, as a preventative measure, when interviewing the users of a specific user views, ask them to pay particular attention to the entities and relationships that exist in other user views. Otherwise, examine the attributes of each entity/relation and look for references to entities/relations in other local data models. We may find that we have an attribute associated with an entity/relation in one local data model that corresponds to a primary key, alternate key, or even a non-key attribute of an entity/relation in another local data model.

(8) Check foreign keys

During this step, entities/relations and relationships/foreign keys may have been merged, primary keys changed, and new relationships identified. Confirm that the foreign keys in child relations are still correct, and make any necessary modifications. The relations that represent the global logical data model for *DreamHome* are shown in Figure 17.8.



(9) Check integrity constraints

Confirm that the integrity constraints for the global logical data model do not conflict with those originally specified for each user view. For example, if any new relationships have been identified and new foreign keys have been created, ensure that appropriate referential integrity constraints are specified. Any conflicts must be resolved in consultation with the users.

Branch (branchNo, street, city, postcode, mgrStaffNo) Primary Key branchNo Alternate Key postcode Foreign Key mgrStaffNo references Manager(staffNo)	Telephone (telNo, branchNo) Primary Key telNo Foreign Key branchNo references Branch(branchNo)
Staff (staffNo, fName, lName, position, sex, DOB, salary, supervisorStaffNo, branchNo) Primary Key staffNo Foreign Key supervisorStaffNo references Staff(staffNo) Foreign Key branchNo references Branch(branchNo)	Manager (staffNo, mgrStartDate, bonus) Primary Key staffNo Foreign Key staffNo references Staff(staffNo)
PrivateOwner (ownerNo, fName, lName, address, telNo) Primary Key ownerNo	BusinessOwner (ownerNo, bName, bType, contactName, address, telNo) Primary Key ownerNo Alternate Key bName Alternate Key telNo
PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo, staffNo, branchNo) Primary Key propertyNo Foreign Key ownerNo references PrivateOwner(ownerNo) and BusinessOwner(ownerNo) Foreign Key staffNo references Staff(staffNo) Foreign Key branchNo references Branch(branchNo)	Viewing (clientNo, propertyNo, dateView, comment) Primary Key clientNo, propertyNo Foreign Key clientNo references Client(clientNo) Foreign Key propertyNo references PropertyForRent(propertyNo)
Client (clientNo, fName, lName, telNo, eMail, prefType, maxRent) Primary Key clientNo Alternate Key eMail	Registration (clientNo, branchNo, staffNo, dateJoined) Primary Key clientNo, branchNo Foreign Key clientNo references Client(clientNo) Foreign Key branchNo references Branch(branchNo) Foreign Key staffNo references Staff(staffNo)
Lease (leaseNo, paymentMethod, depositPaid, rentStart, rentFinish, clientNo, propertyNo) Primary Key leaseNo Alternate Key propertyNo, rentStart Alternate Key clientNo, rentStart Foreign Key clientNo references Client(clientNo) Foreign Key propertyNo references PropertyForRent(propertyNo) Derived deposit (PropertyForRent.rent*2) Derived duration (rentFinish – rentStart)	Newspaper (newspaperName, address, telNo, contactName) Primary Key newspaperName Alternate Key telNo
Advert (propertyNo, newspaperName, dateAdvert, cost) Primary Key propertyNo, newspaperName, dateAdvert Foreign Key propertyNo references PropertyForRent(propertyNo) Foreign Key newspaperName references Newspaper(newspaperName)	

Figure 17.8 Relations that represent the global logical data model for *DreamHome*.

(10) Draw the global ER/relation diagram

We now draw a final diagram that represents all the merged local logical data models. If relations have been used as the basis for merging, we call the resulting diagram a **global relation diagram**, which shows primary keys and foreign keys. If local ER diagrams have been used, the resulting diagram is simply a global ER diagram. The global relation diagram for *DreamHome* is shown in Figure 17.9.



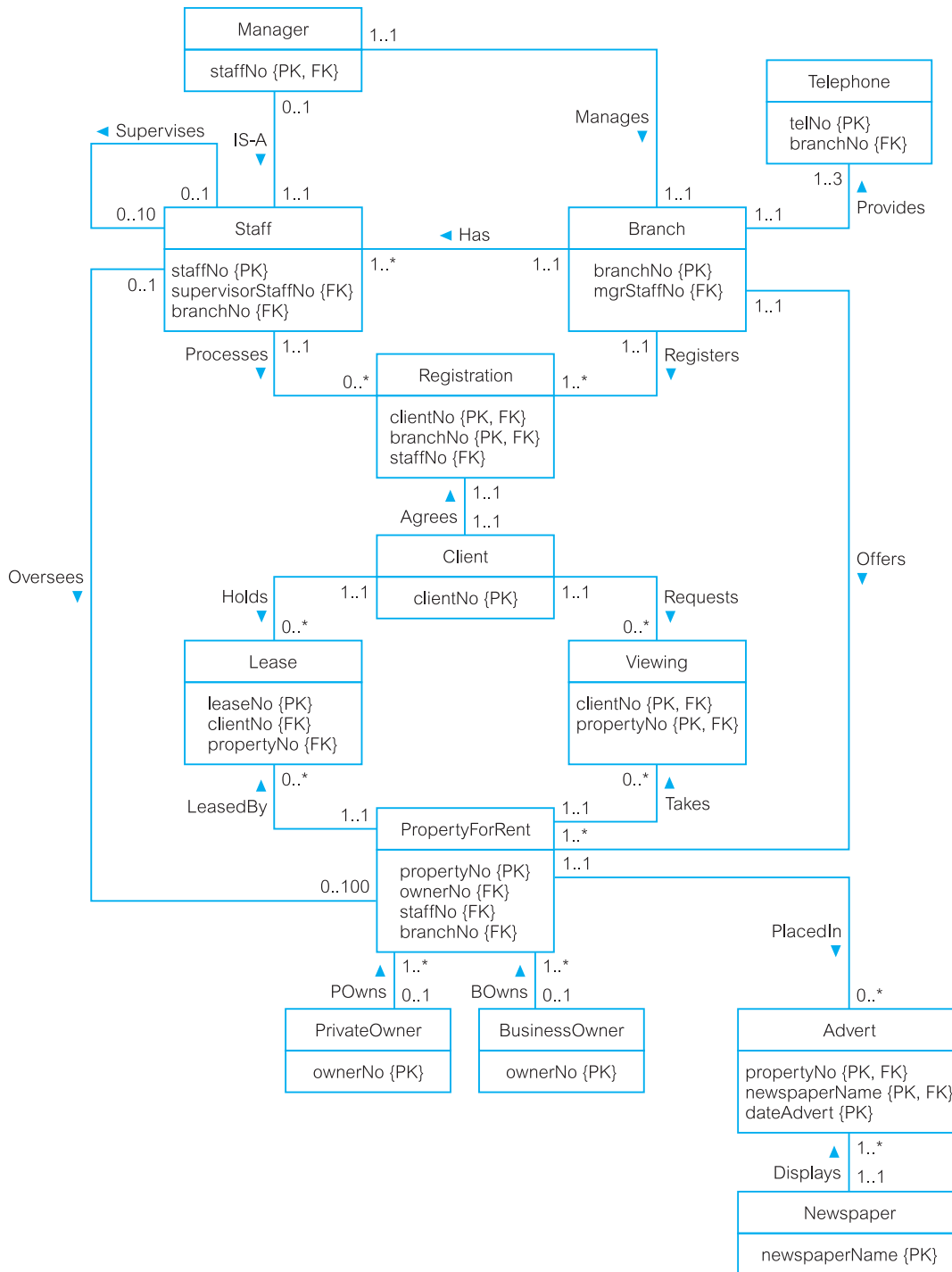


Figure 17.9 Global relation diagram for *DreamHome*.

(11) Update the documentation

Update the documentation to reflect any changes made during the development of the global data model. It is very important that the documentation is up to date and reflects the current data model. If changes are made to the model subsequently, either during database implementation or during maintenance, then the documentation should be updated at the same time. Out-of-date information will cause considerable confusion at a later time.

Step 2.6.2: Validate global logical data model

Objective

To validate the relations created from the global logical data model using the technique of normalization and to ensure that they support the required transactions, if necessary.

This step is equivalent to Steps 2.2 and 2.3, in which we validated each local logical data model. However, it is necessary to check only those areas of the model that resulted in any change during the merging process. In a large system, this will significantly reduce the amount of rechecking that needs to be performed.

Step 2.6.3: Review global logical data model with users

Objective

To review the global logical data model with the users to ensure that they consider the model to be a true representation of the data requirements of an enterprise.

The global logical data model for the enterprise should now be complete and accurate. The model and the documentation that describes the model should be reviewed with the users to ensure that it is a true representation of the enterprise.

To facilitate the description of the tasks associated with Step 2.6, it is necessary to use the terms “*local* logical data model” and “*global* logical data model.” However, at the end of this step when the local data models have been merged into a *single* global data model, the distinction between the data models that refer to some or all user views of a database is no longer necessary. Therefore, after completing this step we refer to the single global data model using the simpler term “logical data model” for the remaining steps of the methodology.

Step 2.7: Check for future growth

Objective

To determine whether there are any significant changes likely in the foreseeable future and to assess whether the logical data model can accommodate these changes.

Logical database design concludes by considering whether the logical data model (which may or may not have been developed using Step 2.6) is capable of being extended to support possible future developments. If the model can sustain current requirements only, then the life of the model may be relatively short and significant reworking may be necessary to accommodate new requirements. It is important to develop a model that is *extensible* and has the ability to evolve to

support new requirements with minimal effect on existing users. Of course, this may be very difficult to achieve, as the enterprise may not know what it wants to do in the future. Even if it does, it may be prohibitively expensive both in time and money to accommodate possible future enhancements now. Therefore, it may be necessary to be selective in what is accommodated. Consequently, it is worth examining the model to check its ability to be extended with minimal impact. However, it is not necessary to incorporate any changes into the data model unless requested by the user.

At the end of Step 2 the logical data model is used as the source of information for physical database design, which is described in the following two chapters as Steps 3 to 8 of the methodology.

For readers familiar with database design, a summary of the steps of the methodology is presented in Appendix D.

Chapter Summary

- The database design methodology includes three main phases: conceptual, logical, and physical database design.
- **Logical database design** is the process of constructing a model of the data used in an enterprise based on a specific data model but independent of a particular DBMS and other physical considerations.
- A **logical data model** includes ER diagram(s), relational schema, and supporting documentation such as the data dictionary, which is produced throughout the development of the model.
- The purpose of Step 2.1 of the methodology for logical database design is to derive a **relational schema** from the conceptual data model created in Step 1.
- In Step 2.2 the relational schema is validated using the rules of normalization to ensure that each relation is structurally correct. **Normalization** is used to improve the model so that it satisfies various constraints that avoids unnecessary duplication of data. In Step 2.3 the relational schema is also validated to ensure that it supports the transactions given in the users' requirements specification.
- In Step 2.4 the integrity constraints of the logical data model are checked. **Integrity constraints** are the constraints that are to be imposed on the database to protect the database from becoming incomplete, inaccurate, or inconsistent. The main types of integrity constraints include: required data, attribute domain constraints, multiplicity, entity integrity, referential integrity, and general constraints.
- In Step 2.5 the logical data model is validated by the users.
- Step 2.6 of logical database design is an optional step and is required only if the database has multiple user views that are being managed using the view integration approach (see Section 10.5), which results in the creation of two or more local logical data models. A **local logical data model** represents the data requirements of one or more, but not all, user views of a database. In Step 2.6 these data models are merged into a **global logical data model**, which represents the requirements of all user views. This logical data model is again validated using normalization, against the required transaction, and by users.
- Logical database design concludes with Step 2.7, which includes consideration of whether the model is capable of being extended to support possible future developments. At the end of Step 2, the logical data model, which may or may not have been developed using Step 2.6, is the source of information for physical database design described as Steps 3 to 8 in Chapters 18 and 19.

Review Questions

- 17.1 Describe the steps used to build a logical data model.
- 17.2 Describe the rules for deriving relations that represent:
- (a) strong entity types;
 - (b) weak entity types;
 - (c) one-to-many (1:*) binary relationship types;
 - (d) one-to-one (1:1) binary relationship types;
 - (e) one-to-one (1:1) recursive relationship types;
 - (f) superclass/subclass relationship types;
 - (g) many-to-many (*:*) binary relationship types;
 - (h) complex relationship types;
 - (i) multi-valued attributes.
- Give examples to illustrate your answers.
- 17.3 Discuss how the technique of normalization can be used to validate the relations derived from the conceptual data model.
- 17.4 Database design is quite complex and important. Discuss the role played by users during the design process.
- 17.5 Describe database design language (DBDL). Discuss how it is used to derive relations during the logical database design phase.
- 17.6 Describe the purpose of merging data models. Discuss the difference between local logical data model and global logical data model.
- 17.7 Why is it important to check the logical data model for future growth?

Exercises



- 17.8 Derive relations from the following conceptual data model shown in Figure 17.10.

The *DreamHome* case study

- 17.9 Create a relational schema for the Branch user view of *DreamHome* based on the conceptual data model produced in Exercise 16.13 and compare your schema with the relations listed in Figure 17.5. Justify any differences found.

The *University Accommodation Office* case study

- 17.10 Create and validate a logical data model from the conceptual data model for the *University Accommodation Office* case study created in Exercise 16.16.

The *EasyDrive School of Motoring* case study

- 17.11 Create and validate a logical data model from the conceptual data model for the *EasyDrive School of Motoring* case study created in Exercise 16.18.

The *Wellmeadows Hospital* case study

- 17.12 Create and validate the local logical data models for each of the local conceptual data models of the *Wellmeadows Hospital* case study identified in Exercise 16.21.

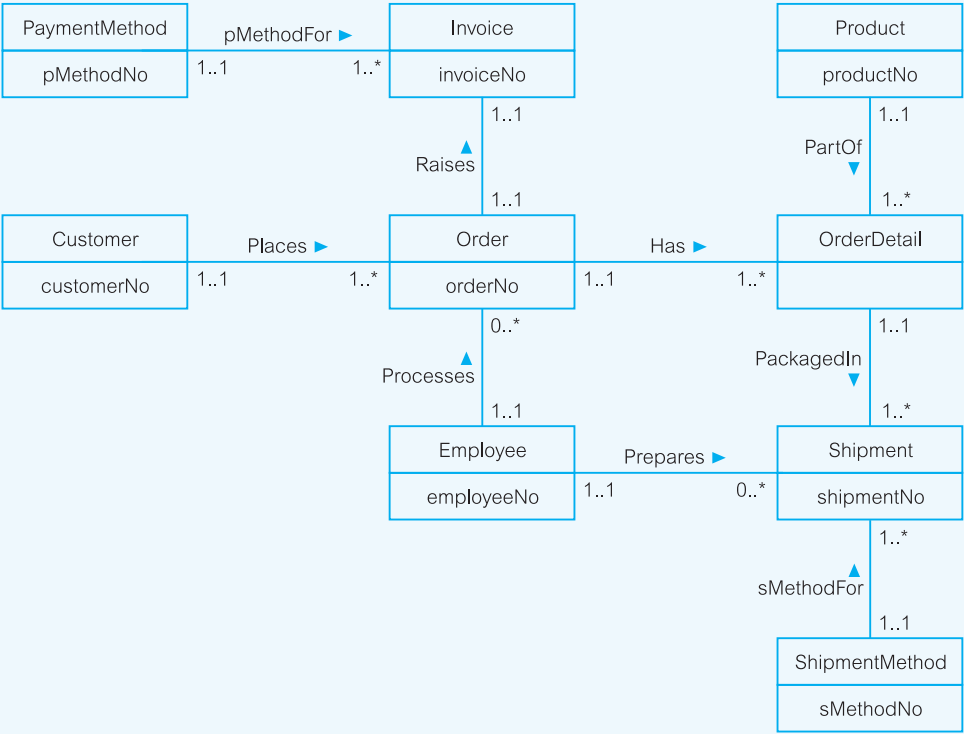


Figure 17.10 An example conceptual data model.

17.13 Merge the local data models to create a global logical data model of the *Wellmeadows Hospital* case study. State any assumptions necessary to support your design.

The **Parking Lot** case study

17.14 Present the relational schema mapped from the *Parking Lot* EER model shown in Figure 17.11 and described in Exercises 12.13 and 13.11.

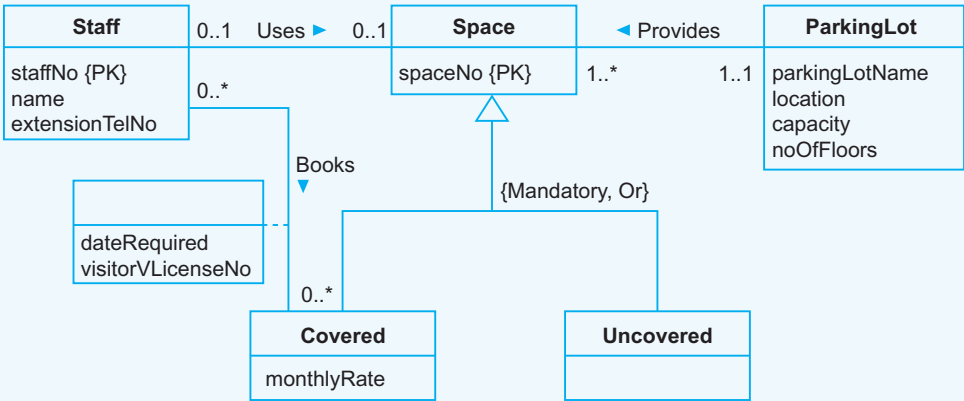


Figure 17.11 An EER model of the *Parking Lot* case study.

The *Library* case study

17.15 Describe the relational schema mapped from the Library EER model shown in Figure 17.12 and described in Exercises 12.14 and 13.12.

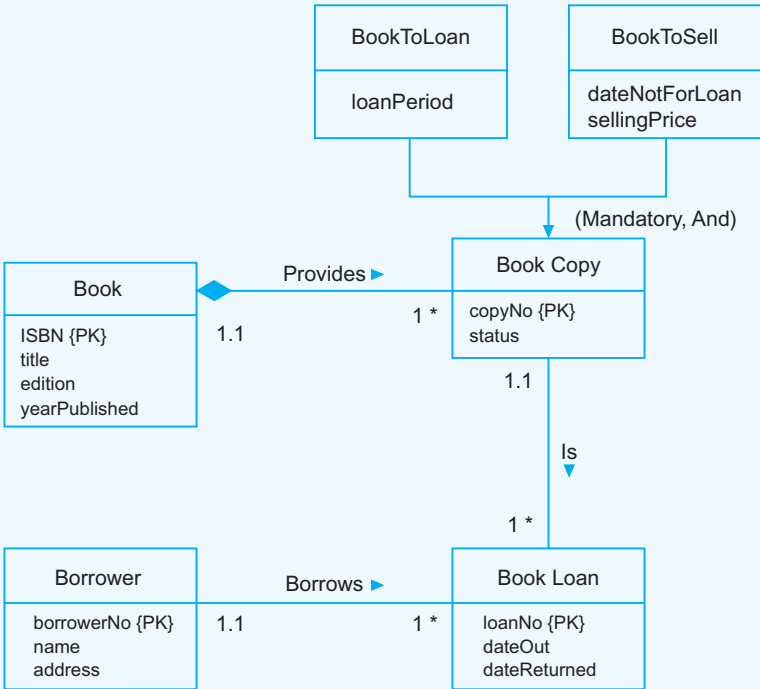


Figure 17.12 An EER model of the *Library* case study.

17.16 The ER diagram in Figure 17.13 shows only entities and primary key attributes. The absence of recognizable named entities or relationships is to emphasize the rule-based nature of the mapping rules described previously in Step 2.1 of logical database design.

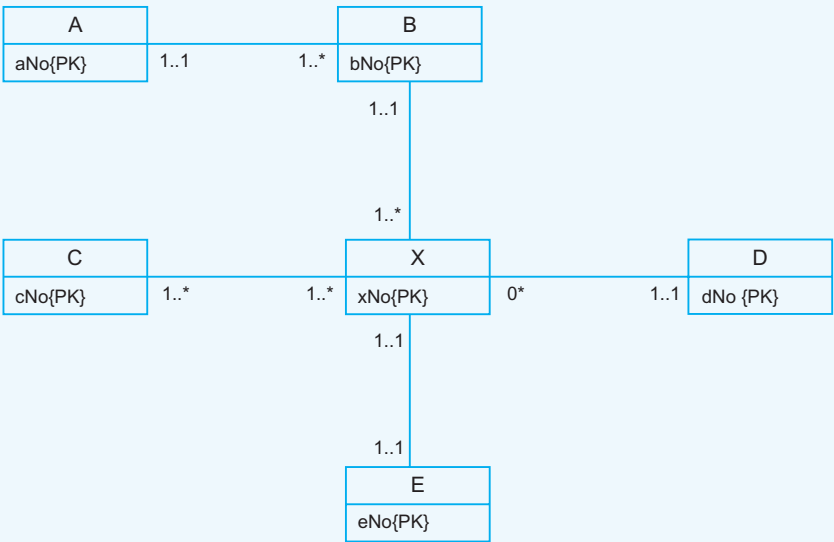


Figure 17.13 An example ER model.

Answer the following questions with reference to how the ER model in Figure 17.13 maps to relational tables.

- (a) How many relations will represent the ER model?
- (b) How many foreign keys are mapped to the relation representing X?
- (c) Which relation(s) will have no foreign key?
- (d) Using only the letter identifier for each entity, provide appropriate names for the relations mapped from the ER model.
- (e) If the cardinality for each relationship is changed to *one-to-one* with total participation for all entities, how many relations would be derived from this version of the ER model?

Methodology—Physical Database Design for Relational Databases

Chapter Objectives

In this chapter you will learn:

- The purpose of physical database design.
- How to map the logical database design to a physical database design.
- How to design base relations for the target DBMS.
- How to design general constraints for the target DBMS.
- How to select appropriate file organizations based on analysis of transactions.
- When to use secondary indexes to improve performance.
- How to estimate the size of the database.
- How to design user views.
- How to design security mechanisms to satisfy user requirements.

In this chapter and the next we describe and illustrate by example a physical database design methodology for relational databases.

The starting point for this chapter is the logical data model and the documentation that describes the model created in the conceptual/logical database design methodology described in Chapters 16 and 17. The methodology started by producing a conceptual data model in Step 1 and then derived a set of relations to produce a logical data model in Step 2. The derived relations were validated to ensure they were correctly structured using the technique of normalization described in Chapters 14 and 15, and to ensure that they supported the transactions the users require.

In the third and final phase of the database design methodology, the designer must decide how to translate the logical database design (that is, the entities, attributes, relationships, and constraints) into a physical database design that can be implemented using the target DBMS. As many parts of physical database design are highly dependent on the target DBMS, there may be more than one way of implementing any given part of the database. Consequently, to do this work properly, the designer must be fully aware of the functionality of the target DBMS, and must understand the advantages and disadvantages of each alternative approach for a particular implementation. For some systems the designer

may also need to select a suitable storage strategy that takes account of intended database usage.

Structure of this Chapter In Section 18.1 we provide a comparison of logical and physical database design. In Section 18.2 we provide an overview of the physical database design methodology and briefly describe the main activities associated with each design phase. In Section 18.3 we focus on the methodology for physical database design and present a detailed description of the first four steps required to build a physical data model. In these steps, we show how to convert the relations derived for the logical data model into a specific database implementation. We provide guidelines for choosing storage structures for the base relations and deciding when to create indexes. In places, we show physical implementation details to clarify the discussion.

In Chapter 19 we complete our presentation of the physical database design methodology and discuss how to monitor and tune the operational system. In particular, we consider when it is appropriate to denormalize the logical data model and introduce redundancy. Appendix D presents a summary of the database design methodology for those readers who are already familiar with database design and require merely an overview of the main steps.

18.1 Comparison of Logical and Physical Database Design

In presenting a database design methodology we divide the design process into three main phases: conceptual, logical, and physical database design. The phase prior to physical design—logical database design—is largely independent of implementation details, such as the specific functionality of the target DBMS and application programs, but is dependent on the target data model. The output of this process is a logical data model consisting of an ER/relation diagram, relational schema, and supporting documentation that describes this model, such as a data dictionary. Together, these represent the sources of information for the physical design process and provide the physical database designer with a vehicle for making tradeoffs that are so important to an efficient database design.

Whereas logical database design is concerned with the *what*, physical database design is concerned with the *how*. It requires different skills that are often found in different people. In particular, the physical database designer must know how the computer system hosting the DBMS operates and must be fully aware of the functionality of the target DBMS. As the functionality provided by current systems varies widely, physical design must be tailored to a specific DBMS. However, physical database design is not an isolated activity—there is often feedback between physical, logical, and application design. For example, decisions taken during physical design for improving performance, such as merging relations together, might affect the structure of the logical data model, which will have an associated effect on the application design.

18.2 Overview of the Physical Database Design Methodology

Physical database design

The process of producing a description of the implementation of the database on secondary storage; it describes the base relations, file organizations, and indexes used to achieve efficient access to the data, and any associated integrity constraints and security measures.

The steps of the physical database design methodology are as follows:

- Step 3 Translate logical data model for target DBMS
 - Step 3.1 Design base relations
 - Step 3.2 Design representation of derived data
 - Step 3.3 Design general constraints
- Step 4 Design file organizations and indexes
 - Step 4.1 Analyze transactions
 - Step 4.2 Choose file organizations
 - Step 4.3 Choose indexes
 - Step 4.4 Estimate disk space requirements
- Step 5 Design user views
- Step 6 Design security mechanisms
- Step 7 Consider the introduction of controlled redundancy
- Step 8 Monitor and tune the operational system

The physical database design methodology presented in this book is divided into six main steps, numbered consecutively from 3 to follow the three steps of the conceptual and logical database design methodology. Step 3 of physical database design involves the design of the base relations and general constraints using the available functionality of the target DBMS. This step also considers how we should represent any derived data present in the data model.

Step 4 involves choosing the file organizations and indexes for the base relations. Typically, PC DBMSs have a fixed storage structure, but other DBMSs tend to provide a number of alternative file organizations for data. From the user's viewpoint, the internal storage representation for relations should be transparent—the user should be able to access relations and tuples without having to specify where or how the tuples are stored. This requires that the DBMS provides *physical data independence*, so that users are unaffected by changes to the physical structure of the database, as discussed in Section 2.1.5. The mapping between the logical data model and physical data model is defined in the internal schema, as shown in Figure 2.1. The designer may have to provide the physical design details to both the DBMS and the operating system. For the DBMS, the designer may have to specify the file organizations that are to be used to represent each relation; for the operating system, the designer must specify details such as the location and protection for each file. We recommend that the reader reviews Appendix F on file organization and storage structures before reading Step 4 of the methodology.

Step 5 involves deciding how each user view should be implemented. Step 6 involves designing the security measures necessary to protect the data from

unauthorized access, including the access controls that are required on the base relations.

Step 7 (described in Chapter 19) considers relaxing the normalization constraints imposed on the logical data model to improve the overall performance of the system. This step should be undertaken only if necessary, because of the inherent problems involved in introducing redundancy while still maintaining consistency. Step 8 (Chapter 19) is an ongoing process of monitoring the operational system to identify and resolve any performance problems resulting from the design and to implement new or changing requirements.

Appendix D presents a summary of the methodology for those readers who are already familiar with database design and require merely an overview of the main steps.

18.3 The Physical Database Design Methodology for Relational Databases

This section provides a step-by-step guide to the first four steps of the physical database design methodology for relational databases. In places, we demonstrate the close association between physical database design and implementation by describing how alternative designs can be implemented using various target DBMSs. The remaining two steps are covered in the next chapter.

Step 3: Translate Logical Data Model for Target DBMS

Objective

To produce a relational database schema from the logical data model that can be implemented in the target DBMS.

The first activity of physical database design involves the translation of the relations in the logical data model into a form that can be implemented in the target relational DBMS. The first part of this process entails collating the information gathered during logical database design and documented in the data dictionary, along with the information gathered during the requirements collection and analysis stage and documented in the systems specification. The second part of the process uses this information to produce the design of the base relations. This process requires intimate knowledge of the functionality offered by the target DBMS. For example, the designer will need to know:

- how to create base relations;
- whether the system supports the definition of primary keys, foreign keys, and alternate keys;
- whether the system supports the definition of required data (that is, whether the system allows attributes to be defined as NOT NULL);
- whether the system supports the definition of domains;
- whether the system supports relational integrity constraints;
- whether the system supports the definition of general constraints.

The three activities of Step 3 are:

- Step 3.1 Design base relations
- Step 3.2 Design representation of derived data
- Step 3.3 Design general constraints

Step 3.1: Design base relations

Objective

To decide how to represent the base relations identified in the logical data model in the target DBMS.

To start the physical design process, we first collate and assimilate the information about the relations produced during logical database design. The necessary information can be obtained from the data dictionary and the definition of the relations described using the DBDL. For each relation identified in the logical data model, we have a definition consisting of:

- the name of the relation;
- a list of simple attributes in brackets;
- the primary key and, where appropriate, alternate keys (AK) and foreign keys (FK);
- referential integrity constraints for any foreign keys identified.

From the data dictionary, we also have for each attribute:

- its domain, consisting of a data type, length, and any constraints on the domain;
- an optional default value for the attribute;
- whether the attribute can hold nulls;
- whether the attribute is derived and, if so, how it should be computed.

To represent the design of the base relations, we use an extended form of the DBDL to define domains, default values, and null indicators. For example, for the *PropertyForRent* relation of the *DreamHome* case study, we may produce the design shown in Figure 18.1.



Implementing base relations

The next step is to decide how to implement the base relations. This decision is dependent on the target DBMS; some systems provide more facilities than others for defining base relations. We have previously demonstrated how to implement base relations using the ISO SQL standard (Section 7.1). We also show how to implement base relations using Microsoft Office Access (Appendix H.1.3), and Oracle (Appendix H.2.3).

Document design of base relations

The design of the base relations should be fully documented, along with the reasons for selecting the proposed design. In particular, document the reasons for selecting one approach when many alternatives exist.

Domain PropertyNumber:	variable length character string, length 5
Domain Street:	variable length character string, length 25
Domain City:	variable length character string, length 15
Domain Postcode:	variable length character string, length 8
Domain PropertyType:	single character, must be one of 'B', 'C', 'D', 'E', 'F', 'H', 'M', 'S'
Domain PropertyRooms:	integer, in the range 1–15
Domain PropertyRent:	monetary value, in the range 0.00–9999.99
Domain OwnerNumber:	variable length character string, length 5
Domain StaffNumber:	variable length character string, length 5
Domain BranchNumber:	fixed length character string, length 4
PropertyForRent(propertyNo PropertyNumber NOT NULL, street Street NOT NULL, city City NOT NULL, postcode Postcode, type PropertyType NOT NULL DEFAULT 'F', rooms PropertyRooms NOT NULL DEFAULT 4, rent PropertyRent NOT NULL DEFAULT 600, ownerNo OwnerNumber NOT NULL, staffNo StaffNumber, branchNo BranchNumber NOT NULL, PRIMARY KEY (propertyNo), FOREIGN KEY (staffNo) REFERENCES Staff(staffNo) ON UPDATE CASCADE ON DELETE SET NULL, FOREIGN KEY (ownerNo) REFERENCES PrivateOwner(ownerNo) and BusinessOwner(ownerNo) ON UPDATE CASCADE ON DELETE NO ACTION, FOREIGN KEY (branchNo) REFERENCES Branch(branchNo) ON UPDATE CASCADE ON DELETE NO ACTION);	

Figure 18.1 DBDL for the PropertyForRent relation.

Step 3.2: Design representation of derived data

Objective

To decide how to represent any derived data present in the logical data model in the target DBMS.

Attributes whose value can be found by examining the values of other attributes are known as **derived** or **calculated attributes**. For example, the following are all derived attributes:

- the number of staff who work in a particular branch;
- the total monthly salaries of all staff;
- the number of properties that a member of staff handles.

Often, derived attributes do not appear in the logical data model but are documented in the data dictionary. If a derived attribute is displayed in the model, a “/” is used to indicate that it is derived (see Section 12.1.2). The first step is to examine the logical data model and the data dictionary, and produce a list of all derived attributes. From a physical database design perspective, whether a derived

PropertyForRent

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46	SA9	B007
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87	SL41	B005
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40		B003
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93	SG37	B003
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87	SG37	B003
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93	SG14	B003

Staff

staffNo	fName	lName	branchNo	noOfProperties
SL21	John	White	B005	0
SG37	Ann	Beech	B003	2
SG14	David	Ford	B003	1
SA9	Mary	Howe	B007	1
SG5	Susan	Brand	B003	0
SL41	Julie	Lee	B005	1

Figure 18.2

The PropertyForRent relation and a simplified Staff relation with the derived attribute noOfProperties.

attribute is stored in the database or calculated every time it is needed is a tradeoff. The designer should calculate:

- the additional cost to store the derived data and keep it consistent with operational data from which it is derived;
- the cost to calculate it each time it is required.

The less expensive option is chosen subject to performance constraints. For the previous example, we could store an additional attribute in the Staff relation representing the number of properties that each member of staff currently manages. A simplified Staff relation based on the sample instance of the *DreamHome* database shown in Figure 4.3 with the new derived attribute noOfProperties is shown in Figure 18.2.

The additional storage overhead for this new derived attribute would not be particularly significant. The attribute would need to be updated every time a member of staff were assigned to or deassigned from managing a property or the property was removed from the list of available properties. In each case, the noOfProperties attribute for the appropriate member of staff would be incremented or decremented by 1. It would be necessary to ensure that this change is made consistently to maintain the correct count and thereby ensure the integrity of the database. When a query accesses this attribute, the value would be immediately available and would not have to be calculated. On the other hand, if the attribute is not stored directly in the Staff relation, it must be calculated each time it is required. This involves a join of the Staff and PropertyForRent relations. Thus, if this type of query is frequent or is considered to be critical for performance purposes, it may be more appropriate to store the derived attribute rather than calculate it each time.

It may also be more appropriate to store derived attributes whenever the DBMS's query language cannot easily cope with the algorithm to calculate the



derived attribute. For example, SQL has a limited set of aggregate functions, as we discussed in Chapter 6.

Document design of derived data

The design of derived data should be fully documented, along with the reasons for selecting the proposed design. In particular, document the reasons for selecting one approach where many alternatives exist.

Step 3.3: Design general constraints

Objective To design the general constraints for the target DBMS.

Updates to relations may be constrained by integrity constraints governing the “real-world” transactions that are represented by the updates. In Step 3.1 we designed a number of integrity constraints: required data, domain constraints, and entity and referential integrity. In this step we have to consider the remaining *general constraints*. The design of such constraints is again dependent on the choice of DBMS; some systems provide more facilities than others for defining general constraints. As in the previous step, if the system is compliant with the SQL standard, some constraints may be easy to implement. For example, *DreamHome* has a rule that prevents a member of staff from managing more than 100 properties at the same time. We could design this constraint into the SQL CREATE TABLE statement for *PropertyForRent* using the following clause:



```
CONSTRAINT StaffNotHandlingTooMuch
CHECK (NOT EXISTS (SELECT staffNo
                    FROM PropertyForRent
                    GROUP BY staffNo
                    HAVING COUNT(*) > 100))
```

Alternatively, a *trigger* could be used to enforce some constraints as we illustrated in Section 8.3. In some systems there will be no support for some or all of the general constraints and it will be necessary to design the constraints into the application. For example, there are very few relational DBMSs (if any) that would be able to handle a time constraint such as “at 17.30 on the last working day of each year, archive the records for all properties sold that year and delete the associated records.”

Document design of general constraints

The design of general constraints should be fully documented. In particular, document the reasons for selecting one approach where many alternatives exist.

Step 3.4: Design File Organizations and Indexes

Objective To determine the optimal file organizations to store the base relations and the indexes that are required to achieve acceptable performance, that is, the way in which relations and tuples will be held on secondary storage.

One of the main objectives of physical database design is to store and access data in an efficient way (see Appendix F). Although some storage structures are efficient for bulk loading data into the database, they may be inefficient after that. Thus, we may have to choose to use an efficient storage structure to set up the database and then choose another for operational use.

Again, the types of file organization available are dependent on the target DBMS; some systems provide more choice of storage structures than others. It is extremely important that the physical database designer fully understands the storage structures that are available, and how the target system uses these structures. This may require the designer to know how the system's query optimizer functions. For example, there may be circumstances under which the query optimizer would not use a secondary index, even if one were available. Thus, adding a secondary index would not improve the performance of the query, and the resultant overhead would be unjustified. We discuss query processing and optimization in Chapter 23.

As with logical database design, physical database design must be guided by the nature of the data and its intended use. In particular, the database designer must understand the typical *workload* that the database must support. During the requirements collection and analysis stage, there may have been requirements specified about how fast certain transactions must run or how many transactions must be processed per second. This information forms the basis for a number of decisions that will be made during this step.

With these objectives in mind, we now discuss the activities in Step 4:

- Step 4.1 Analyze transactions
- Step 4.2 Choose file organizations
- Step 4.3 Choose indexes
- Step 4.4 Estimate disk space requirements

Step 4: Design File Organizations and Indexes

Step 4.1: Analyze transactions

Objective

To understand the functionality of the transactions that will run on the database and to analyze the important transactions.

To carry out physical database design effectively, it is necessary to have knowledge of the transactions or queries that will run on the database. This includes both qualitative and quantitative information. In analyzing the transactions, we attempt to identify performance criteria, such as:

- the transactions that run frequently and will have a significant impact on performance;
- the transactions that are critical to the operation of the business;
- the times during the day/week when there will be a high demand made on the database (called the *peak load*).

We use this information to identify the parts of the database that may cause performance problems. At the same time, we need to identify the high-level functionality of the transactions, such as the attributes that are updated in an update transaction or the criteria used to restrict the tuples that are retrieved in a query. We use this information to select appropriate file organizations and indexes.

In many situations, it is not possible to analyze all the expected transactions, so we should at least investigate the most “important” ones. It has been suggested that the most active 20% of user queries account for 80% of the total data access (Wiederhold, 1983). This 80/20 rule may be used as a guideline in carrying out the analysis. To help identify which transactions to investigate, we can use a *transaction relation cross-reference matrix*, which shows the relations that each transaction accesses, and/or a *transaction usage map*, which diagrammatically indicates which relations are potentially heavily used. To focus on areas that may be problematic, one way to proceed is to:

- (1) map all transaction paths to relations;
- (2) determine which relations are most frequently accessed by transactions;
- (3) analyze the data usage of selected transactions that involve these relations.

Map all transaction paths to relations

In Steps 1.8, 2.3, and 2.6.2 of the conceptual/logical database design methodology, we validated the data models to ensure they supported the transactions that the users require by mapping the transaction paths to entities/relations. If a transaction pathway diagram was used similar to the one shown in Figure 16.9, we may be able to use this diagram to determine the relations that are most frequently accessed. On the other hand, if the transactions were validated in some other way, it may be useful to create a transaction/relation cross-reference matrix. The matrix shows, in a visual way, the transactions that are required and the relations they access. For example, Table 18.1 shows a **transaction/relation cross-reference matrix** for the following selection of typical entry, update/delete, and query transactions for *DreamHome* (see Appendix A):



- | | |
|--|--------------------|
| (A) Enter the details for a new property and the owner (such as details of property number PG4 in Glasgow owned by Tina Murphy). | } StaffClient view |
| (B) Update/delete the details of a property. | |
| (C) Identify the total number of staff in each position at branches in Glasgow. | |
| (D) List the property number, address, type, and rent of all properties in Glasgow, ordered by rent. | } Branch view |
| (E) List the details of properties for rent managed by a named member of staff. | |
| (F) Identify the total number of properties assigned to each member of staff at a given branch | |

The matrix indicates, for example, that transaction (A) reads the Staff table and also inserts tuples into the PropertyForRent and PrivateOwner/BusinessOwner relations. To be more useful, the matrix should indicate in each cell the number of accesses over some time interval (for example, hourly, daily, or weekly). However, to keep the matrix simple, we do not show this information. This matrix shows that both the Staff and PropertyForRent relations are accessed by five of the six transactions, and so

TABLE 18.1 Cross-referencing transactions and relations.

TRANSACTION/ RELATION	(A)				(B)				(C)				(D)				(E)				(F)			
	I	R	U	D	I	R	U	D	I	R	U	D	I	R	U	D	I	R	U	D	I	R	U	D
Branch									X				X								X			
Telephone																								
Staff		X				X			X								X				X			
Manager																								
PrivateOwner	X																							
BusinessOwner	X																							
PropertyForRent	X					X	X	X					X				X				X			
Viewing																								
Client																								
Registration																								
Lease																								
Newspaper																								
Advert																								

I = Insert; R = Read; U = Update; D = Delete

efficient access to these relations may be important to avoid performance problems. We therefore conclude that a closer inspection of these transactions and relations are necessary.

Determine frequency information

In the requirements specification for *DreamHome* given in Section 11.4.4, it was estimated that there are about 100,000 properties for rent and 2000 staff distributed over 100 branch offices, with an average of 1000 and a maximum of 3000 properties at each branch. Figure 18.3 shows the **transaction usage map** for transactions (C), (D), (E), and (F), which all access at least one of the *Staff* and *PropertyForRent* relations, with these numbers added. Due to the size of the *PropertyForRent* relation, it will be important that access to this relation is as efficient as possible. We may now decide that a closer analysis of transactions involving this particular relation would be useful.

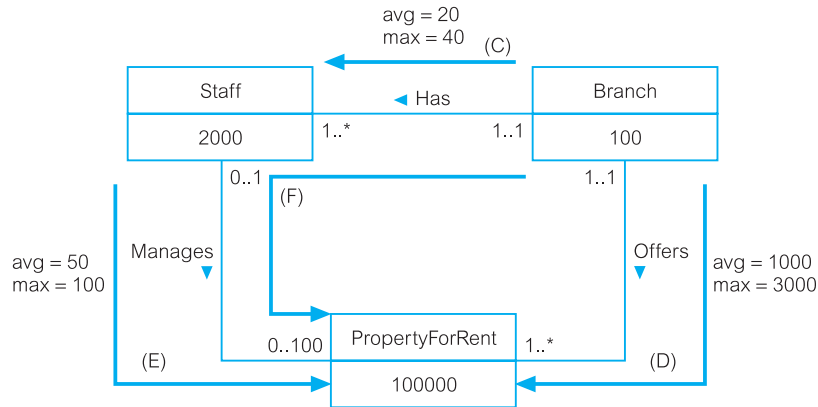


In considering each transaction, it is important to know not only the average and maximum number of times that it runs per hour, but also the day and time that the transaction is run, including when the peak load is likely. For example, some transactions may run at the average rate for most of the time, but have a peak loading between 14.00 and 16.00 on a Thursday prior to a meeting on Friday morning. Other transactions may run only at specific times—for example, 17.00–19.00 on Fridays/Saturdays, which is also their peak loading.

When transactions require frequent access to particular relations, then their pattern of operation is very important. If these transactions operate in a mutually

Figure 18.3

Transaction usage map for some sample transactions showing expected occurrences.



exclusive manner, the risk of likely performance problems is reduced. However, if their operating patterns conflict, potential problems may be alleviated by examining the transactions more closely to determine whether changes can be made to the structure of the relations to improve performance, as we discuss in Step 7 in the next chapter. Alternatively, it may be possible to reschedule some transactions so that their operating patterns do not conflict (for example, it may be possible to leave some summary transactions until a quieter time in the evening or overnight).

Analyze data usage

Having identified the important transactions, we now analyze each one in more detail. For each transaction, we should determine:

- The relations and attributes accessed by the transaction and the type of access; that is, whether it is an insert, update, delete, or retrieval (also known as a query) transaction.

For an update transaction, note the attributes that are updated, as these attributes may be candidates for avoiding an access structure (such as a secondary index).

- The attributes used in any predicates (in SQL, the predicates are the conditions specified in the WHERE clause). Check to see whether the predicates involve:
 - pattern matching; for example: (name LIKE '%Smith%');
 - range searches; for example: (salary BETWEEN 10000 AND 20000);
 - exact-match key retrieval; for example: (salary = 30000).

This applies not only to queries, but also to update and delete transactions, which can restrict the tuples to be updated/deleted in a relation.

These attributes may be candidates for access structures.

- For a query, the attributes that are involved in the join of two or more relations.

Again, these attributes may be candidates for access structures.

- The expected frequency at which the transaction will run; for example, the transaction will run approximately 50 times per day.

- The performance goals for the transaction; for example, the transaction must complete within 1 second.

The attributes used in any predicates for very frequent or critical transactions should have a higher priority for access structures.

Figure 18.4 shows an example of a **transaction analysis form** for transaction (D). This form shows that the average frequency of this transaction is 50 times per hour, with a peak loading of 100 times per hour daily between 17.00 and 19.00.

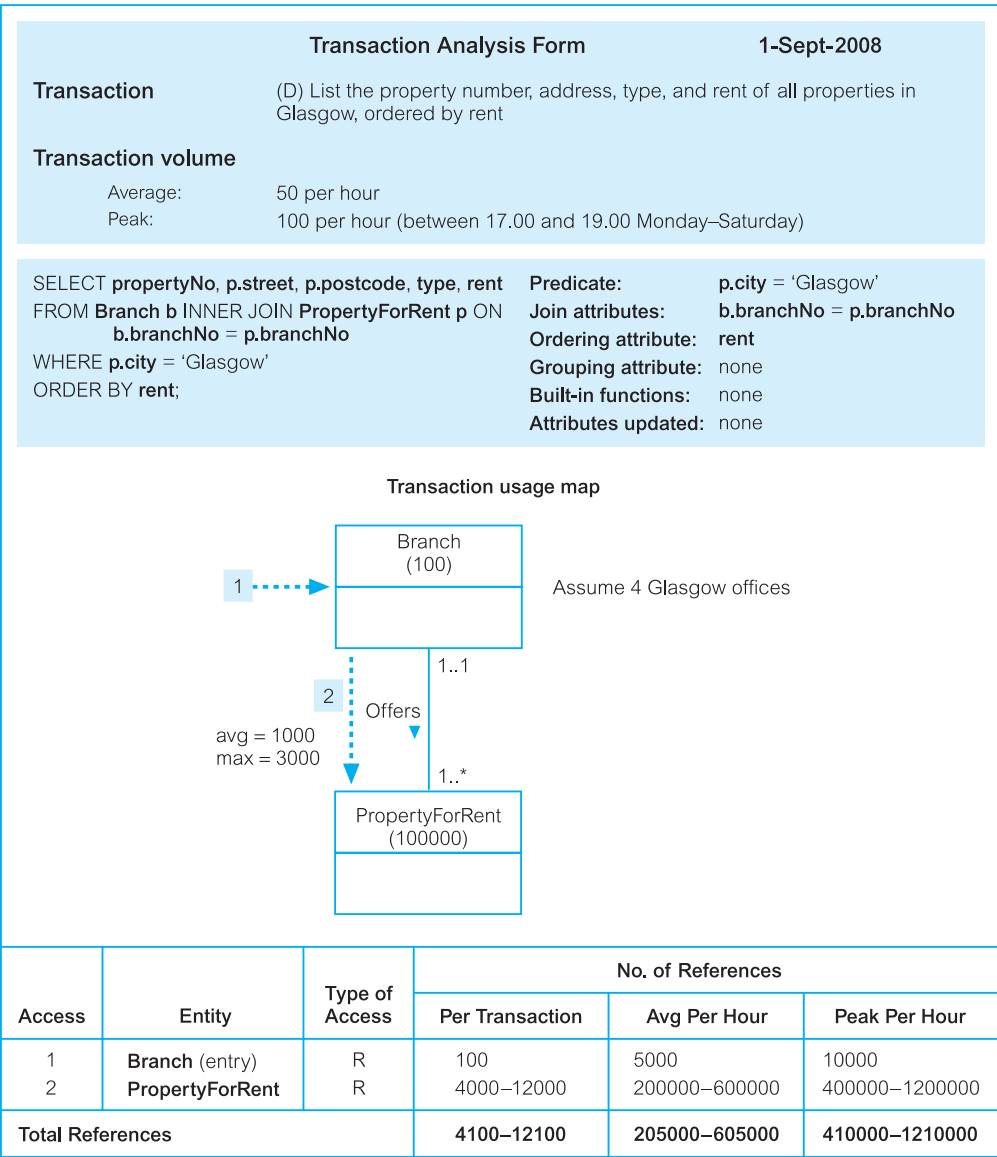


Figure 18.4 Example transaction analysis form.

In other words, typically half the branches will run this transaction per hour, and at peak time all branches will run this transaction once per hour.

The form also shows the required SQL statement and the transaction usage map. At this stage, the full SQL statement may be too detailed, but the types of details that are shown adjacent to the SQL statement should be identified, namely:

- any predicates that will be used;
- any attributes that will be required to join relations together (for a query transaction);
- attributes used to order results (for a query transaction);
- attributes used to group data together (for a query transaction);
- any built-in functions that may be used (such as AVG, SUM);
- any attributes that will be updated by the transaction.

This information will be used to determine the indexes that are required, as we discuss next. Below the transaction usage map, there is a detailed breakdown documenting:

- how each relation is accessed (reads in this case);
- how many tuples will be accessed each time the transaction is run;
- how many tuples will be accessed per hour on average and at peak loading times.

The frequency information will identify the relations that will need careful consideration to ensure that appropriate access structures are used. As mentioned previously, the search conditions used by transactions that have time constraints become higher priority for access structures.

Step 4.2: Choose file organizations

Objective

To determine an efficient file organization for each base relation.

One of the main objectives of physical database design is to store and access data in an efficient way. For example, if we want to retrieve staff tuples in alphabetical order of name, sorting the file by staff name is a good file organization. However, if we want to retrieve all staff whose salary is in a certain range, searching a file ordered by staff name would not be particularly efficient. To complicate matters, some file organizations are efficient for bulk loading data into the database but inefficient after that. In other words, we may want to use an efficient storage structure to set up the database and then change it for normal operational use.

The objective of this step therefore is to choose an optimal file organization for each relation, if the target DBMS allows this. In many cases, a relational DBMS may give little or no choice for choosing file organizations, although some may be established as indexes are specified. However, as an aid to understanding file organizations and indexes more fully, we provide guidelines in Appendix F.7 for selecting a file organization based on the following types of file:

- Heap
- Hash
- Indexed Sequential Access Method (ISAM)

- B⁺-tree
- Clusters

If the target DBMS does not allow the choice of file organizations, this step can be omitted.

Document choice of file organizations

The choice of file organizations should be fully documented, along with the reasons for the choice. In particular, document the reasons for selecting one approach where many alternatives exist.

Step 4.3: Choose indexes

Objective

To determine whether adding indexes will improve the performance of the system.

One approach to selecting an appropriate file organization for a relation is to keep the tuples unordered and create as many **secondary indexes** as necessary. Another approach is to order the tuples in the relation by specifying a *primary* or *clustering index* (see Appendix F.5). In this case, choose the attribute for ordering or clustering the tuples as:

- the attribute that is used most often for join operations, as this makes the join operation more efficient, or
- the attribute that is used most often to access the tuples in a relation in order of that attribute.

If the ordering attribute chosen is a key of the relation, the index will be a *primary index*; if the ordering attribute is not a key, the index will be a *clustering index*. Remember that each relation can have only either a primary index or a clustering index.

Specifying indexes

We saw in Section 7.3.5 that an index can usually be created in SQL using the CREATE INDEX statement. For example, to create a primary index on the PropertyForRent relation based on the propertyNo attribute, we might use the following SQL statement:

```
CREATE UNIQUE INDEX PropertyNoInd ON PropertyForRent(propertyNo);
```

To create a clustering index on the PropertyForRent relation based on the staffNo attribute, we might use the following SQL statement:

```
CREATE INDEX StaffNoInd ON PropertyForRent(staffNo) CLUSTER;
```

As we have already mentioned, in some systems the file organization is fixed. For example, until recently Oracle has supported only B⁺-trees, but has now added support for clusters. On the other hand, INGRES offers a wide set of different index structures that can be chosen using the following optional clause in the CREATE INDEX statement:

```
[STRUCTURE = BTREE | ISAM | HASH | HEAP]
```

Choosing secondary indexes

Secondary indexes provide a mechanism for specifying an additional key for a base relation that can be used to retrieve data more efficiently. For example, the *PropertyForRent* relation may be hashed on the property number, *propertyNo*, the *primary index*. However, there may be frequent access to this relation based on the rent attribute. In this case, we may decide to add rent as a *secondary index*.

There is an overhead involved in the maintenance and use of secondary indexes that has to be balanced against the performance improvement gained when retrieving data. This overhead includes:

- adding an index record to every secondary index whenever a tuple is inserted into the relation;
- updating a secondary index when the corresponding tuple in the relation is updated;
- the increase in disk space needed to store the secondary index;
- possible performance degradation during query optimization, as the query optimizer may consider all secondary indexes before selecting an optimal execution strategy.

Guidelines for choosing a “wish-list” of indexes

One approach to determining which secondary indexes are needed is to produce a “*wish-list*” of attributes that we consider to be candidates for indexing, and then to examine the impact of maintaining each of these indexes. We provide the following guidelines to help produce such a wish-list:

- (1) Do not index small relations. It may be more efficient to search the relation in memory than to store an additional index structure.
- (2) In general, index the primary key of a relation if it is not a key of the file organization. Although the SQL standard provides a clause for the specification of primary keys, as discussed in Section 7.2.3, it should be noted that this does not guarantee that the primary key will be indexed.
- (3) Add a secondary index to a foreign key, if it is frequently accessed. For example, we may frequently join the *PropertyForRent* relation and the *PrivateOwner/Business Owner* relations on the attribute *ownerNo*, the owner number. Therefore, it may be more efficient to add a secondary index to the *PropertyForRent* relation based on the attribute *ownerNo*. Note that some DBMSs may automatically index foreign keys.
- (4) Add a secondary index to any attribute that is heavily used as a secondary key (for example, add a secondary index to the *PropertyForRent* relation based on the attribute *rent*, as discussed previously).
- (5) Add a secondary index on attributes that are frequently involved in:
 - (a) selection or join criteria;
 - (b) ORDER BY;
 - (c) GROUP BY;
 - (d) other operations involving sorting (such as UNION or DISTINCT).
- (6) Add a secondary index on attributes involved in built-in aggregate functions, along with any attributes used for the built-in functions. For example,

to find the average staff salary at each branch, we could use the following SQL query:

```
SELECT branchNo, AVG(salary)
FROM Staff
GROUP BY branchNo;
```

From the previous guideline, we could consider adding an index to the `branchNo` attribute by virtue of the `GROUP BY` clause. However, it may be more efficient to consider an index on both the `branchNo` attribute and the `salary` attribute. This may allow the DBMS to perform the entire query from data in the index alone, without having to access the data file. This is sometimes called an **index-only plan**, as the required response can be produced using only data in the index.

- (7) As a more general case of the previous guideline, add a secondary index on attributes that could result in an index-only plan.
- (8) Avoid indexing an attribute or relation that is frequently updated.
- (9) Avoid indexing an attribute if the query will retrieve a significant proportion (for example 25%) of the tuples in the relation. In this case, it may be more efficient to search the entire relation than to search using an index.
- (10) Avoid indexing attributes that consist of long character strings.

If the search criteria involve more than one predicate, and one of the terms contains an `OR` clause, and the term has no index/sort order, then adding indexes for the other attributes is not going to help improve the speed of the query, because a linear search of the relation will still be required. For example, assume that only the `type` and `rent` attributes of the `PropertyForRent` relation are indexed, and we need to use the following query:

```
SELECT *
FROM PropertyForRent
WHERE (type = 'Flat' OR rent > 500 OR rooms > 5);
```

Although the two indexes could be used to find the tuples where (`type = 'Flat'` or `rent > 500`), the fact that the `rooms` attribute is not indexed will mean that these indexes cannot be used for the full `WHERE` clause. Thus, unless there are other queries that would benefit from having the `type` and `rent` attributes indexed, there would be no benefit gained from indexing them for this query.

On the other hand, if the predicates in the `WHERE` clause were `AND`'ed together, the two indexes on the `type` and `rent` attributes could be used to optimize the query.

Removing indexes from the wish-list

Having drawn up the wish-list of potential indexes, we should now consider the impact of each of these on update transactions. If the maintenance of the index is likely to slow down important update transactions, then consider dropping the index from the list. Note, however, that a particular index may also make update operations more efficient. For example, if we want to update a member of staff's salary, given the member's staff number, `staffNo`, and we have an index on `staffNo`, then the tuple to be updated can be found more quickly.

It is a good idea to experiment when possible to determine whether an index is improving performance, providing very little improvement, or adversely impacting performance. In the last case, clearly we should remove this index from the wish-list. If there is little observed improvement with the addition of the index, further examination may be necessary to determine under what circumstances the index will be useful, and whether these circumstances are sufficiently important to warrant the implementation of the index.

Some systems allow users to inspect the optimizer's strategy for executing a particular query or update, sometimes called the **Query Execution Plan** (QEP). For example, Microsoft Office Access has a Performance Analyzer, Oracle has an EXPLAIN PLAN diagnostic utility (see Section 23.6.3), DB2 has an EXPLAIN utility, and INGRES has an online QEP-viewing facility. When a query runs slower than expected, it is worth using such a facility to determine the reason for the slowness and to find an alternative strategy that may improve the performance of the query.

If a large number of tuples are being inserted into a relation with one or more indexes, it may be more efficient to drop the indexes first, perform the inserts, and then recreate the indexes afterwards. As a rule of thumb, if the insert will increase the size of the relation by at least 10%, drop the indexes temporarily.

Updating the database statistics

The query optimizer relies on database statistics held in the system catalog to select the optimal strategy. Whenever we create an index, the DBMS automatically adds the presence of the index to the system catalog. However, we may find that the DBMS requires a utility to be run to update the statistics in the system catalog relating to the relation and the index.

Document choice of indexes

The choice of indexes should be fully documented, along with the reasons for the choice. In particular, if there are performance reasons why some attributes should not be indexed, these should also be documented.



File organizations and indexes for *DreamHome* with Microsoft Office Access

Like most, if not all, PC DBMSs, Microsoft Office Access uses a fixed file organization, so if the target DBMS is Microsoft Office Access, Step 4.2 can be omitted. Microsoft Office Access does, however, support indexes as we will now briefly discuss. In this section we use the terminology of Office Access, which refers to a relation as a *table* with *fields* and *records*.

Guidelines for indexes In Office Access, the primary key of a table is automatically indexed, but a field whose data type is Memo, Hyperlink, or OLE Object cannot be indexed. For other fields, Microsoft advises indexing a field if all the following apply:

- the field's data type is Text, Number, Currency, or Date/Time;
- the user anticipates searching for values stored in the field;

- the user anticipates sorting values in the field;
- the user anticipates storing many different values in the field. If many of the values in the field are the same, the index may not significantly speed up queries.

In addition, Microsoft advises:

- indexing fields on both sides of a join or creating a relationship between these fields, in which case Office Access will automatically create an index on the foreign key field, if one does not exist already;
- when grouping records by the values in a joined field, specifying GROUP BY for the field that is in the same table as the field the aggregate is being calculated on.

Microsoft Office Access can optimize simple and complex predicates (which are called *expressions* in Office Access). For certain types of complex expressions, Microsoft Office Access uses a data access technology called *Rushmore* to achieve a greater level of optimization. A complex expression is formed by combining two simple expressions with the AND or OR operator, such as:

```
branchNo = 'BOOI' AND rooms > 5
type = 'Flat' OR rent > 300
```

In Office Access, a complex expression is fully or partially optimizable depending on whether one or both simple expressions are optimizable, and which operator was used to combine them. A complex expression is *Rushmore-optimizable* if all three of the following conditions are true:

- the expression uses AND or OR to join two conditions;
- both conditions are made up of simple optimizable expressions;
- both expressions contain indexed fields. The fields can be indexed individually or they can be part of a multiple-field index.

Indexes for DreamHome Before creating the wish-list, we ignore small tables from further consideration, as small tables can usually be processed in memory without requiring additional indexes. For *DreamHome* we ignore the Branch, Telephone, Manager, and Newspaper tables from further consideration. Based on the guidelines provided earlier:

- (1) Create the primary key for each table, which will cause Office Access to automatically index this field.
- (2) Ensure all relationships are created in the Relationships window, which will cause Office Access to automatically index the foreign key fields.

As an illustration of which other indexes to create, we consider the query transactions listed in Appendix A for the StaffClient user views of *DreamHome*. We can produce a summary of interactions between the base tables and these transactions shown in Table 18.2. This figure shows for each table: the transaction(s) that operate on the table, the type of access (a search based on a *predicate*, a join together with the *join field*, any *ordering field*, and any *grouping field*), and the frequency with which the transaction runs.



TABLE 18.2 Interactions between base tables and query transactions for the StaffClient user views of *DreamHome*.

TABLE	TRANSACTION	FIELD	FREQUENCY PER DAY)
Staff	(a), (d)	predicate: fName, lName	20
	(a)	join: Staff on supervisorStaffNo	20
	(b)	ordering: fName, lName	20
	(b)	predicate: position	20
Client	(e)	join: Staff on staff No	1 000–000
	(j)	predicate: fName, lName	1 000
PropertyForRent	(c)	predicate: rentFinish	5 000–10,000
	(k), (l)	predicate: rentFinish	100
	(c)	join: PrivateOwner/BusinessOwner on ownerNo	5 000–10,000
	(d)	join: Staff on staff No	20
	(f)	predicate: city	50
	(f)	predicate: rent	50
	(g)	join: Client on clientNo	100
Viewing	(i)	join: Client on clientNo	100
Lease	(c)	join: PropertyForRent on propertyNo	5 000–10,000
	(l)	join: PropertyForRent on propertyNo	100
	(j)	join: Client on clientNo	1 000

Based on this information, we choose to create the additional indexes shown in Table 18.3. We leave it as an exercise for the reader to choose additional indexes to create in Microsoft Office Access for the transactions listed in Appendix A for the Branch view of *DreamHome* (see Exercise 18.5).



File organizations and indexes for *DreamHome* with Oracle

In this section we repeat the previous exercise of determining appropriate file organizations and indexes for the StaffClient user views of *DreamHome*. Once again,

TABLE 18.3 Additional indexes to be created in Microsoft Office Access based on the query transactions for the StaffClient user views of *DreamHome*.

TABLE	INDEX
Staff	fName, lName
	position
Client	fName, lName
PropertyForRent	rentFinish
	city
	rent

we use the terminology of the DBMS—Oracle refers to a relation as a *table* with *columns* and *rows*.

Oracle automatically adds an index for each primary key. In addition, Oracle recommends that UNIQUE indexes not be explicitly defined on tables but instead UNIQUE integrity constraints be defined on the desired columns. Oracle enforces UNIQUE integrity constraints by automatically defining a unique index on the unique key. Exceptions to this recommendation are usually performance-related. For example, using a CREATE TABLE . . . AS SELECT with a UNIQUE constraint is slower than creating the table without the constraint and then manually creating a UNIQUE index.

Assume that the tables are created with the identified primary, alternate, and foreign keys specified. We now identify whether any clusters are required and whether any additional indexes are required. To keep the design simple, we will assume that clusters are not appropriate. Again, considering just the query transactions listed in Appendix A for the StaffClient user views of *DreamHome*, there may be performance benefits in adding the indexes shown in Table 18.4. Again, we leave it as an exercise for the reader to choose additional indexes to create in Oracle for the transactions listed in Appendix A for the Branch view of *DreamHome* (see Exercise 18.6).

Step 4.4: Estimate disk space requirements

Objective

To estimate the amount of disk space that will be required by the database.

It may be a requirement that the physical database implementation can be handled by the current hardware configuration. Even if this is not the case, the designer still

TABLE 18.4 Additional indexes to be created in Oracle based on the query transactions for the StaffClient user views of *DreamHome*.

TABLE	INDEX
Staff	fName, lName supervisorStaffNo position
Client	staffNo fName, lName
PropertyForRent	ownerNo staffNo clientNo rentFinish city rent
Viewing	clientNo
Lease	propertyNo clientNo

has to estimate the amount of disk space that is required to store the database, in case new hardware has to be procured. The objective of this step is to estimate the amount of disk space that is required to support the database implementation on secondary storage. As with the previous steps, estimating the disk usage is highly dependent on the target DBMS and the hardware used to support the database. In general, the estimate is based on the size of each tuple and the number of tuples in the relation. The latter estimate should be a maximum number, but it may also be worth considering how the relation will grow and modifying the resulting disk size by this growth factor to determine the potential size of the database in the future. In Appendix J (see companion Web site), we illustrate the process for estimating the size of relations created in Oracle.

■ Step 5: Design User Views

Objective

To design the user views that were identified during the requirements collection and analysis stage of the database system development lifecycle.



The first phase of the database design methodology presented in Chapter 16 involved the production of a conceptual data model for either the single user view or a number of combined user views identified during the requirements collection and analysis stage. In Section 11.4.4 we identified four user views for *DreamHome* named Director, Manager, Supervisor, Assistant, and Client. Following an analysis of the data requirements for these user views, we used the centralized approach to merge the requirements for the user views as follows:

- **Branch**, consisting of the Director and Manager user views;
- **StaffClient**, consisting of the Supervisor, Assistant, and Client user views.

In Step 2 the conceptual data model was mapped to a logical data model based on the relational model. The objective of this step is to design the user views identified previously. In a standalone DBMS on a PC, user views are usually a convenience to simplify database requests. However, in a multi-user DBMS, user views play a central role in defining the structure of the database and enforcing security. In Section 7.4.7, we discussed the major advantages of views, such as data independence, reduced complexity, and customization. We previously discussed how to create views using the ISO SQL standard (Section 7.4.1), and how to create views (stored queries) in Microsoft Office Access (Appendix M; see companion Web site).

Document design of user views

The design of the individual user views should be fully documented.

■ Step 6: Design Security Mechanisms

Objective

To design the security mechanisms for the database as specified by the users during the requirements and collection stage of the database system development lifecycle.

A database represents an essential corporate resource and so security of this resource is extremely important. During the requirements collection and analysis stage of the database system development lifecycle, specific security requirements should have been documented in the system requirements specification (see Section 11.4.4). The objective of this step is to decide how these security requirements will be realized. Some systems offer different security facilities than others. Again, the database designer must be aware of the facilities offered by the target DBMS. As we discuss in Chapter 20, relational DBMSs generally provide two types of database security:

- system security;
- data security.

System security covers access and use of the database at the system level, such as a user name and password. **Data security** covers access and use of database objects (such as relations and views) and the actions that users can have on the objects. Again, the design of access rules is dependent on the target DBMS; some systems provide more facilities than others for designing access rules. We have previously discussed how to create access rules using the discretionary GRANT and REVOKE statements of the ISO SQL standard (Section 7.6). We also show how to create access rules using Microsoft Office Access (Appendix H.1.9), and Oracle (Appendix H.2.5). We discuss security more fully in Chapter 20.

Document design of security measures

The design of the security measures should be fully documented. If the physical design affects the logical data model, this model should also be updated.

Chapter Summary

- **Physical database design** is the process of producing a description of the implementation of the database on secondary storage. It describes the base relations and the storage structures and access methods used to access the data effectively, along with any associated integrity constraints and security measures. The design of the base relations can be undertaken only once the designer is fully aware of the facilities offered by the target DBMS.
- The initial step (Step 3) of physical database design is the translation of the logical data model into a form that can be implemented in the target relational DBMS.
- The next step (Step 4) designs the file organizations and access methods that will be used to store the base relations. This involves analyzing the transactions that will run on the database, choosing suitable file organizations based on this analysis, choosing indexes and, finally, estimating the disk space that will be required by the implementation.
- **Secondary indexes** provide a mechanism for specifying an additional key for a base relation that can be used to retrieve data more efficiently. However, there is an overhead involved in the maintenance and use of secondary indexes that has to be balanced against the performance improvement gained when retrieving data.
- One approach to selecting an appropriate file organization for a relation is to keep the tuples unordered and create as many secondary indexes as necessary. Another approach is to order the tuples in the relation by specifying a primary or clustering index. One approach to determining which secondary indexes are needed is to produce a “wish-list” of attributes that we consider are candidates for indexing, and then to examine the impact of maintaining each of these indexes.

- The objective of Step 5 is to design an implementation of the user views identified during the requirements collection and analysis stage, such as using the mechanisms provided by SQL.
- A database represents an essential corporate resource, so security of this resource is extremely important. The objective of Step 6 is to design the realization of the security mechanisms identified during the requirements collection and analysis stage.

Review Questions

- 18.1 Physical database design depends much on the logical and conceptual design. Discuss the validity of this statement.
- 18.2 Discuss why it is important to analyze transactions before implementing the index.
- 18.3 Describe the purpose of the main steps in the physical design methodology presented in this chapter.
- 18.4 Describe the 80/20 rule and how it is used in the physical database model.

Exercises

The *DreamHome* case study



- 18.5 In Step 4.3 we chose the indexes to create in Microsoft Office Access for the query transactions listed in Appendix A for the StaffClient user views of *DreamHome*. Choose indexes to create in Microsoft Office Access for the query transactions listed in Appendix A for the Branch user views of *DreamHome*.
- 18.6 Repeat Exercise 18.5 using Oracle as the target DBMS.
- 18.7 Create a physical database design for the logical design of the *DreamHome* case study (described in Chapter 17) based on the DBMS that you have access to.
- 18.8 Implement this physical design for *DreamHome* created in Exercise 18.7.

The *University Accommodation Office* case study

- 18.9 Based on the logical data model developed in Exercise 17.10, create a physical database design for the *University Accommodation Office* case study (described in Appendix B.1) based on the DBMS that you have access to.
- 18.10 Implement the *University Accommodation Office* database using the physical design created in Exercise 18.9.

The *EasyDrive School of Motoring* case study

- 18.11 Based on the logical data model developed in Exercise 17.11, create a physical database design for the *EasyDrive School of Motoring* case study (described in Appendix B.2) based on the DBMS that you have access to.
- 18.12 Based on the physical design you just created for *EasyDrive School of Motoring*, describe critical performance challenges that can be foreseen.

The *Wellmeadows Hospital* case study

- 18.13 Based on the logical data model developed in Exercise 17.13, create a physical database design for the *Wellmeadows Hospital* case study (described in Appendix B.3) based on the DBMS that you have access to.
- 18.14 Implement the *Wellmeadows Hospital* database using the physical design created in Exercise 18.13.

Methodology—Monitoring and Tuning the Operational System

Chapter Objectives

In this chapter you will learn:

- The meaning of denormalization.
- When to denormalize to improve performance.
- The importance of monitoring and tuning the operational system.
- How to measure efficiency.
- How system resources affect performance.

In the previous chapter we presented the first five steps of the physical database design methodology for relational databases. In this chapter we describe and illustrate by example the final two steps of the physical database design methodology. We provide guidelines for determining when to denormalize the logical data model and introduce redundancy, and then discuss the importance of monitoring the operational system and continuing to tune it. In places, we show physical implementation details to clarify the discussion.

19.1 Denormalizing and Introducing Controlled Redundancy

Step 7: Consider the Introduction of Controlled Redundancy

Objective

To determine whether introducing redundancy in a controlled manner by relaxing the normalization rules will improve the performance of the system.

As we discussed in Chapter 14 and 15, normalization is a technique for deciding which attributes belong together in a relation. One of the basic aims of relational database design is to group attributes together in a relation because there is a functional dependency between them. The result of normalization is a logical database design that is structurally consistent and has minimal redundancy. However, it is

sometimes argued that a normalized database design does not provide maximum processing efficiency. Consequently, there may be circumstances in which it may be necessary to accept the loss of some of the benefits of a fully normalized design in favor of performance. This should be considered only when it is estimated that the system will not be able to meet its performance requirements. We are not advocating that normalization should be omitted from logical database design: normalization forces us to understand completely each attribute that has to be represented in the database. This may be the most important factor that contributes to the overall success of the system. In addition, the following factors have to be considered:

- denormalization makes implementation more complex;
- denormalization often sacrifices flexibility;
- denormalization may speed up retrievals but slows down updates.

Formally, the term **denormalization** refers to a refinement to the relational schema such that the degree of normalization for a modified relation is less than the degree of at least one of the original relations. We also use the term more loosely to refer to situations in which we combine two relations into one new relation, and the new relation is still normalized but contains more nulls than the original relations. Some authors refer to denormalization as **usage refinement**.

As a general rule of thumb, if performance is unsatisfactory and a relation has a low update rate and a very high query rate, denormalization may be a viable option. The transaction/relation cross-reference matrix that may have been produced in Step 4.1 provides useful information for this step. The matrix summarizes in a visual way the access patterns of the transactions that will run on the database. It can be used to highlight possible candidates for denormalization and to assess the effects this would have on the rest of the model.

More specifically, in this step we consider duplicating certain attributes or joining relations together to reduce the number of joins required to perform a query. Indirectly, we have encountered an implicit example of denormalization when dealing with address attributes. For example, consider the definition of the Branch relation:

Branch (branchNo, street, city, postcode, mgrStaffNo)

Strictly speaking, this relation is not in third normal form: postcode (the post or zip code) functionally determines city. In other words, we can determine the value of the city attribute given a value for the postcode attribute. Hence, the Branch relation is in 2NF. To normalize the relation to 3NF, it would be necessary to split the relation into two, as follows:

Branch (branchNo, street, postcode, mgrStaffNo)

Postcode (postcode, city)

However, we would rarely wish to access the branch address without the city attribute. This would mean that we would have to perform a join whenever we want a complete address for a branch. As a result, we settle for 2NF and implement the original Branch relation.

Unfortunately, there are no fixed rules for determining when to denormalize relations. In this step we discuss some of the more common situations for considering denormalization. For additional information, the interested reader is referred

to Rogers (1989) and Fleming and Von Halle (1989). In particular, we consider denormalization in the following situations, specifically to speed up frequent or critical transactions:

- Step 7.1 Combining one-to-one (1:1) relationships
- Step 7.2 Duplicating non-key attributes in one-to-many (1:*) relationships to reduce joins
- Step 7.3 Duplicating foreign key attributes in one-to-many (1:*) relationships to reduce joins
- Step 7.4 Duplicating attributes in many-to-many (*:*) relationships to reduce joins
- Step 7.5 Introducing repeating groups
- Step 7.6 Creating extract tables
- Step 7.7 Partitioning relations

To illustrate these steps, we use the relation diagram shown in Figure 19.1(a) and the sample data shown in Figure 19.1(b).

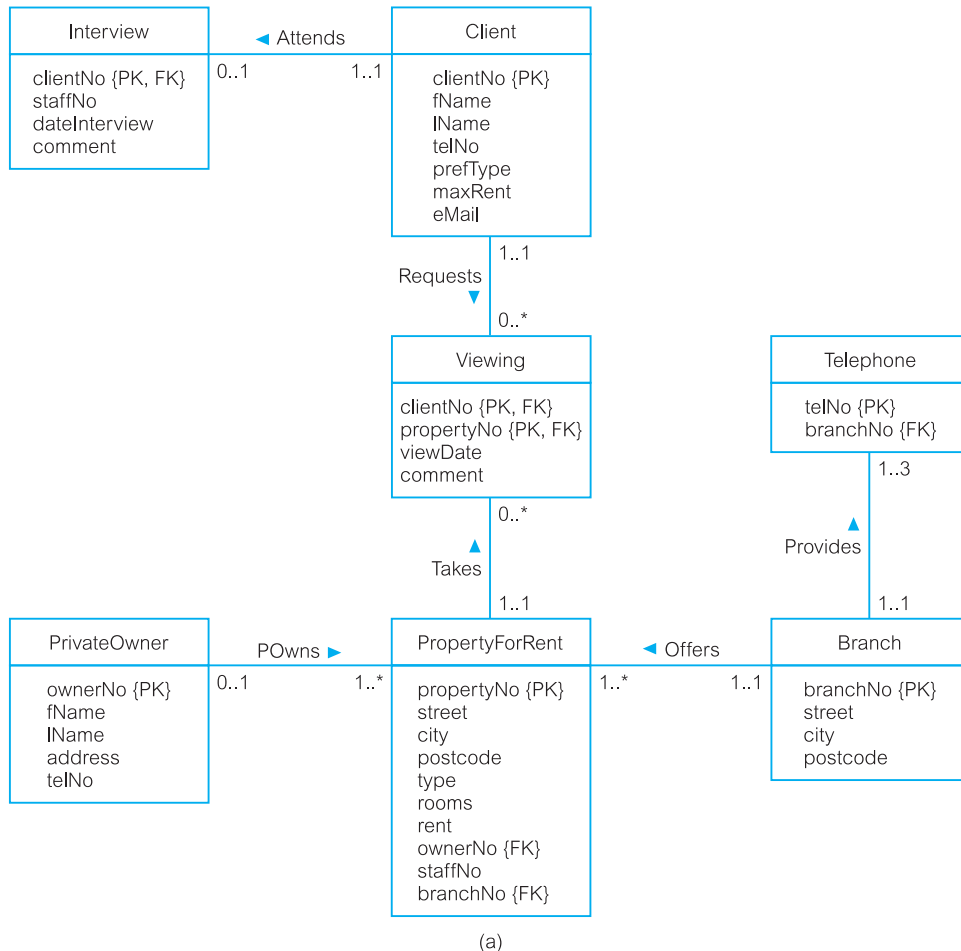


Figure 19.1(a) Sample relation diagram.

Figure 19.1(b)
Sample relations.

Branch				Telephone	
branchNo	street	city	postcode	telNo	branchNo
B005	22 Deer Rd	London	SW1 4EH	0207-886-1212	B005
B007	16 Argyll St	Aberdeen	AB2 3SU	0207-886-1300	B005
B003	163 Main St	Glasgow	G11 9QX	0207-886-4100	B005
B004	32 Manse Rd	Bristol	BS99 1NZ	01224-67125	B007
B002	56 Clover Dr	London	NW10 6EU	0141-339-2178	B003
				0141-339-4439	B003
				0117-916-1170	B004
				0208-963-1030	B002

PropertyForRent									
propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46	SA9	B007
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87	SL41	B005
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40		B003
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93	SG37	B003
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87	SG37	B003
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93	SG14	B003

Client						
clientNo	fName	lName	telNo	prefType	maxRent	eMail
CR76	John	Kay	0207-774-5632	Flat	425	john.kay@gmail.com
CR56	Aline	Stewart	0141-848-1825	Flat	350	astewart@hotmail.com
CR74	Mike	Ritchie	01475-392178	House	750	mritchie01@yahoo.co.uk
CR62	Mary	Tregear	01224-196720	Flat	600	maryt@hotmail.co.uk

Interview			
clientNo	staffNo	dateInterview	comment
CR56	SG37	11-Apr-12	current lease ends in June
CR62	SA9	7-Mar-12	needs property urgently

PrivateOwner				
ownerNo	fName	lName	address	telNo
CO46	Joe	Keogh	2 Fergus Dr, Aberdeen AB2 7SX	01224-861212
CO87	Carol	Farrel	6 Achray St, Glasgow G32 9DX	0141-357-7419
CO40	Tina	Murphy	63 Well St, Glasgow G42	0141-943-1728
CO93	Tony	Shaw	12 Park Pl, Glasgow G4 0QR	0141-225-7025

Viewing			
clientNo	propertyNo	viewDate	comment
CR56	PA14	24-May-13	too small
CR76	PG4	20-Apr-13	too remote
CR56	PG4	26-May-13	
CR62	PA14	14-May-13	
CR56	PG36	28-Apr-13	no dining room

(b)

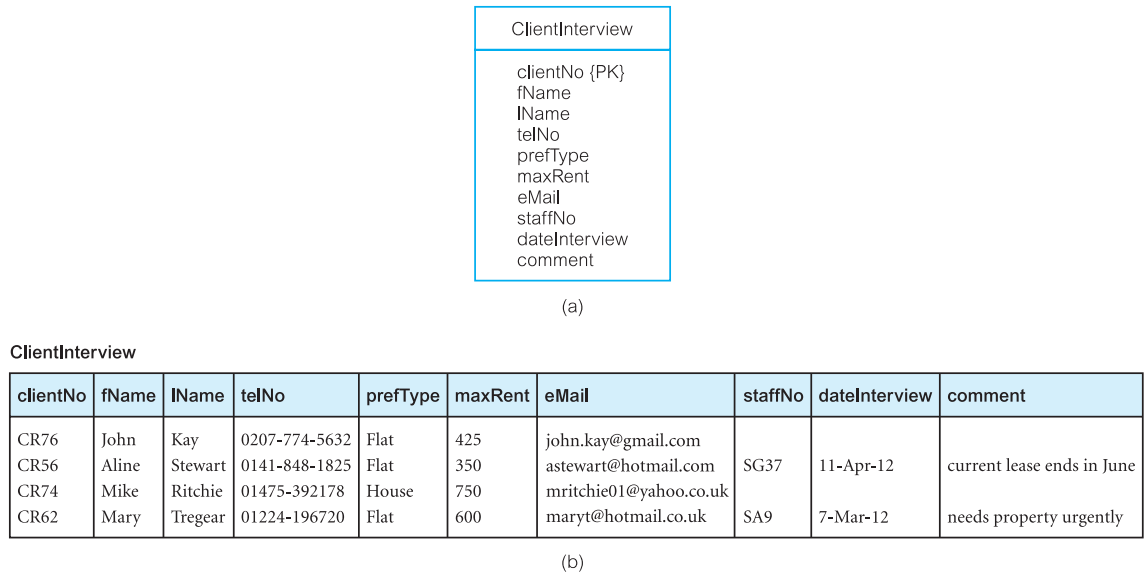


Figure 19.2 Combined Client and Interview: (a) revised extract from the relation diagram; (b) combined relation.

Step 7.1: Combining one-to-one (1:1) relationships

Re-examine one-to-one (1:1) relationships to determine the effects of combining the relations into a single relation. Combination should be considered only for relations that are frequently referenced together and infrequently referenced separately. Consider, for example, the 1:1 relationship between *Client* and *Interview*, as shown in Figure 19.1. The *Client* relation contains information on potential renters of property; the *Interview* relation contains the date of the interview and comments made by a member of staff about a *Client*.

We could combine these two relations together to form a new relation *ClientInterview*, as shown in Figure 19.2. Because the relationship between *Client* and *Interview* is 1:1 and the participation is optional, there may be a significant number of nulls in the combined relation *ClientInterview*, depending on the proportion of tuples involved in the participation, as shown in Figure 19.2(b). If the original *Client* relation is large and the proportion of tuples involved in the participation is small, there will be a significant amount of wasted space.

Step 7.2: Duplicating non-key attributes in one-to-many (1:*) relationships to reduce joins

With the specific aim of reducing or removing joins from frequent or critical queries, consider the benefits that may result in duplicating one or more non-key attributes of the parent relation in the child relation in a 1:* relationship. For example, whenever the *PropertyForRent* relation is accessed, it is very common for the owner's name to be accessed at the same time. A typical SQL query would be:

```
SELECT p.*, o.lName
FROM PropertyForRent p, PrivateOwner o
WHERE p.ownerNo = o.ownerNo AND branchNo = 'B003';
```

PropertyForRent

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	IName	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46	Keogh	SA9	B007
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87	Farrel	SL41	B005
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40	Murphy		B003
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93	Shaw	SG37	B003
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87	Farrel	SG37	B003
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93	Shaw	SG14	B003

Figure 19.3 Revised PropertyForRent relation with duplicated IName attribute from the PrivateOwner relation.

based on the original relation diagram and sample relations shown in Figure 19.1. If we duplicate the IName attribute in the PropertyForRent relation, we can remove the PrivateOwner relation from the query, which in SQL becomes:

```
SELECT p.*
FROM PropertyForRent p
WHERE branchNo = 'B003';
```

based on the revised relation shown in Figure 19.3.

The benefits that result from this change have to be balanced against the problems that may arise. For example, if the duplicated data is changed in the parent relation, it must be updated in the child relation. Further, for a 1:* relationship there may be multiple occurrences of each data item in the child relation (for example, the names Farrel and Shaw both appear twice in the revised PropertyForRent relation), in which case it becomes necessary to maintain consistency of multiple copies. If the update of the IName attribute in the PrivateOwner and PropertyForRent relation cannot be automated, the potential for loss of integrity is considerable. An associated problem with duplication is the additional time that is required to maintain consistency automatically every time a tuple is inserted, updated, or deleted. In our case, it is unlikely that the name of the owner of a property will change, so the duplication may be warranted.

Another problem to consider is the increase in storage space resulting from the duplication. Again, with the relatively low cost of secondary storage nowadays, this may not be so much of a problem. However, this is not a justification for arbitrary duplication.

A special case of a one-to-many (1:*) relationship is a **lookup table**, sometimes called a *reference table* or *pick list*. Typically, a lookup table contains a code and a description. For example, we may define a lookup (parent) table for property type and modify the PropertyForRent (child) table, as shown in Figure 19.4. The advantages of using a lookup table are:

- reduction in the size of the child relation; the type code occupies 1 byte as opposed to 5 bytes for the type description;
- if the description can change (which is not the case in this particular example), it is easier changing it once it's in the lookup table as opposed to changing it many times in the child relation;
- the lookup table can be used to validate user input.

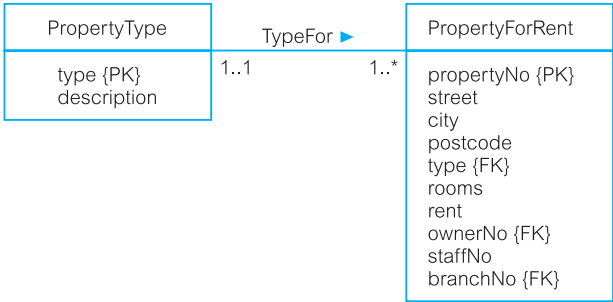


Figure 19.4
Lookup table for
property type:
(a) relation
diagram;
(b) sample
relations.

PropertyType

type	description
H	House
F	Flat

PropertyForRent

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	H	6	650	CO46	SA9	B007
PL94	6 Argyll St	London	NW2	F	4	400	CO87	SL41	B005
PG4	6 Lawrence St	Glasgow	G11 9QX	F	3	350	CO40		B003
PG36	2 Manor Rd	Glasgow	G32 4QX	F	3	375	CO93	SG37	B003
PG21	18 Dale Rd	Glasgow	G12	H	5	600	CO87	SG37	B003
PG16	5 Novar Dr	Glasgow	G12 9AX	F	4	450	CO93	SG14	B003

If the lookup table is used in frequent or critical queries, and the description is unlikely to change, consideration should be given to duplicating the description attribute in the child relation, as shown in Figure 19.5. The original lookup table is not redundant—it can still be used to validate user input. However, by duplicating the description in the child relation, we have eliminated the need to join the child relation to the lookup table.

PropertyForRent

propertyNo	street	city	postcode	type	description	rooms	rent	ownerNo	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	H	House	6	650	CO46	SA9	B007
PL94	6 Argyll St	London	NW2	F	Flat	4	400	CO87	SL41	B005
PG4	6 Lawrence St	Glasgow	G11 9QX	F	Flat	3	350	CO40		B003
PG36	2 Manor Rd	Glasgow	G32 4QX	F	Flat	3	375	CO93	SG37	B003
PG21	18 Dale Rd	Glasgow	G12	H	House	5	600	CO87	SG37	B003
PG16	5 Novar Dr	Glasgow	G12 9AX	F	Flat	4	450	CO93	SG14	B003

Figure 19.5 Modified PropertyForRent relation with duplicated description attribute.

Step 7.3: Duplicating foreign key attributes in one-to-many (1:*) relationships to reduce joins



Again, with the specific aim of reducing or removing joins from frequent or critical queries, consider the benefits that may result in duplicating one or more of the foreign key attributes in a relationship. For example, a frequent query for *DreamHome* is to list all the private property owners at a branch, using an SQL query of the following form:

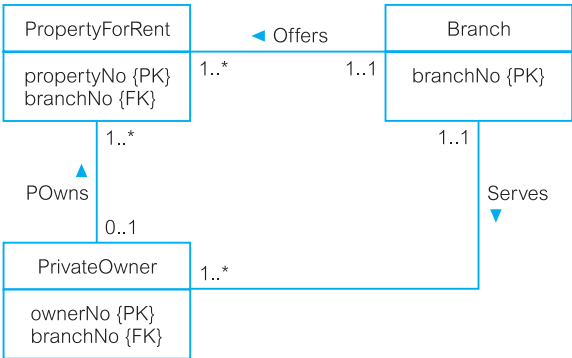
```
SELECT o.IName
FROM PropertyForRent p, PrivateOwner o
WHERE p.ownerNo = o.ownerNo AND branchNo = 'B003';
```

based on the original data shown in Figure 19.1. In other words, because there is no direct relationship between *PrivateOwner* and *Branch*, then to get the list of owners we have to use the *PropertyForRent* relation to gain access to the branch number, *branchNo*. We can remove the need for this join by duplicating the foreign key *branchNo* in the *PrivateOwner* relation; that is, we introduce a direct relationship between the *Branch* and *PrivateOwner* relations. In this case, we can simplify the SQL query to:

```
SELECT o.IName
FROM PrivateOwner o
WHERE branchNo = 'B003';
```

based on the revised relation diagram and *PrivateOwner* relation shown in Figure 19.6. If this change is made, it will be necessary to introduce additional foreign key constraints, as discussed in Step 2.2.

Figure 19.6
Duplicating the foreign key *branchNo* in the *PrivateOwner* relation:
(a) revised (simplified) relation diagram with *branchNo* included as a foreign key;
(b) revised *PrivateOwner* relation.



(a)

PrivateOwner					
ownerNo	fName	IName	address	telNo	branchNo
CO46	Joe	Keogh	2 Fergus Dr, Aberdeen AB2 7SX	01224-861212	B007
CO87	Carol	Farrel	6 Achray St, Glasgow G32 9DX	0141-357-7419	B003
CO40	Tina	Murphy	63 Well St, Glasgow G42	0141-943-1728	B003
CO93	Tony	Shaw	12 Park Pl, Glasgow G4 0QR	0141-225-7025	B003

(b)

If an owner could rent properties through many branches, the previous change would not work. In this case, it would be necessary to model a many-to-many (**:*) relationship between Branch and PrivateOwner. Note also that the PropertyForRent relation has the branchNo attribute because it is possible for a property not to have a member of staff allocated to it, particularly at the start when the property is first taken on by the agency. If the PropertyForRent relation did not have the branch number, it would be necessary to join the PropertyForRent relation to the Staff relation based on the staffNo attribute to get the required branch number. The original SQL query would then become:

```
SELECT o.IName
FROM Staff s, PropertyForRent p, PrivateOwner o
WHERE s.staffNo = p.staffNo AND p.ownerNo = o.ownerNo AND s.branchNo = 'B003';
```

Removing two joins from the query may provide greater justification for creating a direct relationship between PrivateOwner and Branch and thereby duplicating the foreign key branchNo in the PrivateOwner relation.

Step 7.4: Duplicating attributes in many-to-many (:*) relationships to reduce joins**

During logical database design, we mapped each **:*) relationship into three relations: the two relations derived from the original entities and a new relation representing the relationship between the two entities. Now, if we wish to produce information from the **:*) relationship, we have to join these three relations. In some circumstances, it may be possible to reduce the number of relations to be joined by duplicating attributes from one of the original entities in the intermediate relation.

For example, the **:*) relationship between Client and PropertyForRent has been decomposed by introducing the intermediate Viewing relation. Consider the requirement that the *DreamHome* sales staff should contact clients who have still to make a comment on the properties they have viewed. However, the sales staff need only the street attribute of the property when talking to the clients. The required SQL query is:

```
SELECT p.street, c.*, v.viewDate
FROM Client c, Viewing v, PropertyForRent p
WHERE v.propertyNo = p.propertyNo AND c.clientNo = v.clientNo AND comment
IS NULL;
```

based on the relation model and sample data shown in Figure 19.1. If we duplicate the street attribute in the intermediate Viewing relation, we can remove the PropertyForRent relation from the query, giving the SQL query:

```
SELECT c.*, v.street, v.viewDate
FROM Client c, Viewing v
WHERE c.clientNo = v.clientNo AND comment IS NULL;
```

based on the revised Viewing relation shown in Figure 19.7.

Step 7.5: Introducing repeating groups

Repeating groups were eliminated from the logical data model as a result of the requirement that all entities be in first normal form. Repeating groups were



Figure 19.7

Duplicating the street attribute from the PropertyForRent relation in the Viewing relation.

Viewing				
clientNo	propertyNo	street	viewDate	comment
CR56	PA14	16 Holhead	24-May-13	too small
CR76	PG4	6 Lawrence St	20-Apr-13	too remote
CR56	PG4	6 Lawrence St	26-May-13	
CR62	PA14	16 Holhead	14-May-13	no dining room
CR56	PG36	2 Manor Rd	28-Apr-13	



separated out into a new relation, forming a 1:* relationship with the original (parent) relation. Occasionally, reintroducing repeating groups is an effective way to improve system performance. For example, each *DreamHome* branch office has a maximum of three telephone numbers, although all offices do not necessarily have the same number of lines. In the logical data model, we created a Telephone entity with a three-to-one (3:1) relationship with Branch, resulting in two relations, as shown in Figure 19.1.

If access to this information is important or frequent, it may be more efficient to combine the relations and store the telephone details in the original Branch relation with one attribute for each telephone, as shown in Figure 19.8.

In general, this type of denormalization should be considered only in the following circumstances:

- the absolute number of items in the repeating group is known (in this example there is a maximum of three telephone numbers);
- the number is static and will not change over time (the maximum number of telephone lines is fixed and is not expected to change);
- the number is not very large, typically not greater than 10, although this is not as important as the first two conditions.

Figure 19.8

Branch incorporating repeating group: (a) revised relation diagram; (b) revised relation.



(a)

branchNo	street	city	postcode	telNo1	telNo2	telNo3
B005	22 Deer Rd	London	SW1 4EH	0207-886-1212	0207-886-1300	0207-886-4100
B007	16 Argyll St	Aberdeen	AB2 3SU	01224-67125		
B003	163 Main St	Glasgow	G11 9QX	0141-339-2178	0141-339-4439	
B004	32 Manse Rd	Bristol	BS99 1NZ	0117-916-1170		
B002	56 Clover Dr	London	NW10 6EU	0208-963-1030		

(b)

Sometimes it may be only the most recent or current value in a repeating group, or just the fact that there is a repeating group, that is needed most frequently. In the previous example we may choose to store one telephone number in the Branch relation and leave the remaining numbers for the Telephone relation. This would remove the presence of nulls from the Branch relation, as each branch must have at least one telephone number.

Step 7.6: Creating extract tables

There may be situations where reports have to be run at peak times during the day. These reports access derived data and perform multirelation joins on the same set of base relations. However, the data the report is based on may be relatively static or, in some cases, may not have to be current (that is, if the data is a few hours old, the report would be perfectly acceptable). In this case, it may be possible to create a single, highly denormalized extract table based on the relations required by the reports, and allow the users to access the extract table directly instead of the base relations. The most common technique for producing extract tables is to create and populate the tables in an overnight batch run when the system is lightly loaded.

Step 7.7: Partitioning relations

Rather than combining relations together, an alternative approach that addresses the key problem with supporting very large relations (and indexes) is to decompose them into a number of smaller and more manageable pieces called **partitions**. As illustrated in Figure 19.9, there are two main types of partitioning: horizontal partitioning and vertical partitioning.

Horizontal partitioning

Distributes the **tuples** of a relation across a number of (smaller) relations.

Vertical partitioning

Distributes the **attributes** of a relation across a number of (smaller) relations (the primary key is duplicated to allow the original relation to be reconstructed).

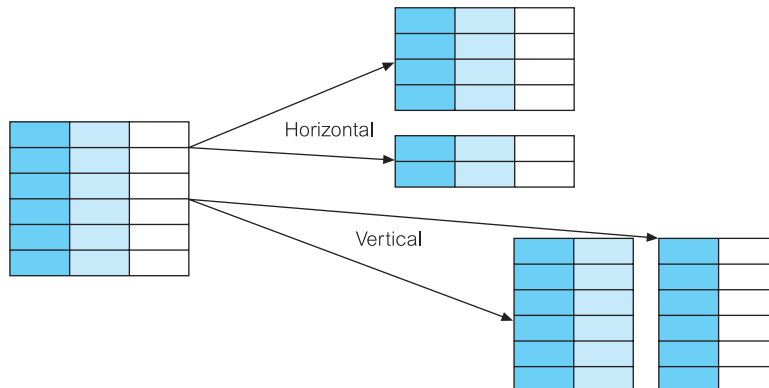


Figure 19.9

Horizontal and vertical partitioning.

Figure 19.10

Oracle SQL statement to create a hash partition.

```
CREATE TABLE ArchivedPropertyForRentPartition(
  propertyNo VARHAR2(5) NOT NULL,
  street VARCHAR2(25) NOT NULL,
  city VARCHAR2(15) NOT NULL,
  postcode VARCHAR2(8),
  type CHAR NOT NULL,
  rooms SMALLINT NOT NULL,
  rent NUMBER(6, 2) NOT NULL,
  ownerNo VARCHAR2(5) NOT NULL,
  staffNo VARCHAR2(5),
  branchNo CHAR(4) NOT NULL,
  PRIMARY KEY (propertyNo),
  FOREIGN KEY (ownerNo) REFERENCES PrivateOwner(ownerNo),
  FOREIGN KEY (staffNo) REFERENCES Staff(staffNo),
  FOREIGN KEY (branchNo) REFERENCES Branch(branchNo))
PARTITION BY HASH (branchNo)
(PARTITION b1 TABLESPACE TB01,
PARTITION b2 TABLESPACE TB02,
PARTITION b3 TABLESPACE TB03,
PARTITION b4 TABLESPACE TB04);
```



Partitions are particularly useful in applications that store and analyze large amounts of data. For example, *DreamHome* maintains an *ArchivedPropertyForRent* relation with several hundreds of thousands of tuples that are held indefinitely for analysis purposes. Searching for a particular tuple at a branch could be quite time-consuming; however, we could reduce this time by horizontally partitioning the relation, with one partition for each branch. We can create a (*hash*) partition for this scenario in Oracle using the SQL statement shown in Figure 19.10.

As well as hash partitioning, other common types of partitioning are **range** (each partition is defined by a range of values for one or more attributes) and **list** (each partition is defined by a list of values for an attribute). There are also composite partitions, such as **range-hash** and **list-hash** (each partition is defined by a range or a list of values and then each partition is further subdivided based on a hash function).

There may also be circumstances in which we frequently examine particular attributes of a very large relation and it may be appropriate to vertically partition the relation into those attributes that are frequently accessed together and another vertical partition for the remaining attributes (with the primary key replicated in each partition to allow the original relation to be reconstructed using a join).

Partitioning has a number of advantages:

- *Improved load balancing.* Partitions can be allocated to different areas of secondary storage thereby permitting parallel access while at the same time minimizing the contention for access to the same storage area if the relation was not partitioned.
- *Improved performance.* By limiting the amount of data to be examined or processed, and by enabling parallel execution, performance can be enhanced.

- *Increased availability.* If partitions are allocated to different storage areas and one storage area were to become unavailable, the other partitions would still be available.
- *Improved recovery.* Smaller partitions can be recovered more efficiently (equally well, the DBA may find backing up smaller partitions easier than backing up very large relations).
- *Security.* Data in a partition can be restricted to those users who require access to it, with different partitions having different access restrictions.

Partitioning can also have a number of disadvantages:

- *Complexity.* Partitioning is not usually transparent to end-users and queries that utilize more than one partition become more complex to write.
- *Reduced performance.* Queries that combine data from more than one partition may be slower than a nonpartitioned approach.
- *Duplication.* Vertical partitioning involves duplication of the primary key. This leads not only to increased storage requirements but also to potential inconsistencies arising.

Consider implications of denormalization

Consider the implications of denormalization on the previous steps in the methodology. For example, it may be necessary to reconsider the choice of indexes on the relations that have been denormalized to establish whether existing indexes should be removed or additional indexes added. In addition it will be necessary to consider how data integrity will be maintained. Common solutions are:

- *Triggers.* Triggers can be used to automate the updating of derived or duplicated data (see Section 8.3).
- *Transactions.* Build into each application transactions that make the updates to denormalized data as a single (*atomic*) action.
- *Batch reconciliation.* Run batch programs at appropriate times to make the denormalized data consistent.

In terms of maintaining integrity, triggers provide the best solution, although they can cause performance problems. The advantages and disadvantages of denormalization are summarized in Table 19.1.

TABLE 19.1 Advantages and disadvantages of denormalization.

ADVANTAGES	DISADVANTAGES
Can improve performance by:	May speed up retrievals but can slow down updates.
• precomputing derived data;	Always application-specific and needs to be re-evaluated if the application changes.
• minimizing the need for joins;	Can increase the size of relations.
• reducing the number of foreign keys in relations;	May simplify implementation in some cases but may make it more complex in others.
• reducing the number indexes (thereby saving storage space);	Sacrifices flexibility.
• reducing the number of relations.	

Document introduction of redundancy

The introduction of redundancy should be fully documented, along with the reasons for introducing it. In particular, document the reasons for selecting one approach where many alternatives exist. Update the logical data model to reflect any changes made as a result of denormalization.

19.2 Monitoring the System to Improve Performance

Step 8: Monitor and Tune the Operational System

Objective

To monitor the operational system and improve the performance of the system to correct inappropriate design decisions or reflect changing requirements.

For this activity, we should remember that one of the main objectives of physical database design is to store and access data in an efficient way (see Appendix F). There are a number of factors that we may use to measure efficiency:

- *Transaction throughput.* This is the number of transactions that can be processed in a given time interval. In some systems, such as airline reservations, high transaction throughput is critical to the overall success of the system.
- *Response time.* This is the elapsed time for the completion of a single transaction. From a user's point of view, we want to minimize response time as much as possible. However, there are some factors that influence response time that the designer may have no control over, such as system loading or communication times. Response time can be shortened by:
 - reducing contention and wait times, particularly disk I/O wait times;
 - reducing the amount of time for which resources are required;
 - using faster components.
- *Disk storage.* This is the amount of disk space required to store the database files. The designer may wish to minimize the amount of disk storage used.

However, there is no one factor that is always correct. Typically, the designer must trade one factor off against another to achieve a reasonable balance. For example, increasing the amount of data stored may decrease the response time or transaction throughput. The initial physical database design should not be regarded as static, but should be considered to be an estimate of how the operational system might perform. Once the initial design has been implemented, it will be necessary to monitor the system and tune it as a result of observed performance and changing requirements (see Step 8). Many DBMSs provide the DBA with utilities to monitor and tune the operation of the system.

There are many benefits to be gained from tuning the database:

- Tuning can avoid the procurement of additional hardware.
- It may be possible to downsize the hardware configuration. This results in less and cheaper hardware and consequently less expensive maintenance.
- A well-tuned system produces faster response times and better throughput, which in turn makes the users—and hence the organization—more productive.
- Improved response times can improve staff morale.
- Improved response times can increase customer satisfaction.

These last two benefits are less tangible than the others. However, we can certainly state that slow response times may demoralize staff and potentially lose customers. To tune an operational system, the physical database designer must be aware of how the various hardware components interact and affect database performance, as we now discuss.

Understanding system resources

Main memory Main memory accesses are significantly faster than secondary storage accesses—sometimes tens or even hundreds of thousands of times faster. In general, the more main memory available to the DBMS and the database applications, the faster the applications will run. However, it is sensible always to have a minimum of 5% of main memory available. It is also advisable not to have any more than 10% available; otherwise, main memory is not being used optimally. When there is insufficient memory to accommodate all processes, the operating system transfers pages of processes to disk to free up memory. When one of these pages is next required, the operating system has to transfer it back from disk. Sometimes it is necessary to swap entire processes from memory to disk and back again to free up memory. Problems occur with main memory when paging or swapping becomes excessive.

To ensure efficient usage of main memory, it is necessary to understand how the target DBMS uses main memory, what buffers it keeps in main memory, what parameters exist to allow the size of the buffers to be adjusted, and so on. For example, Oracle keeps a data dictionary cache in main memory that ideally should be large enough to handle 90% of data dictionary accesses without having to retrieve the information from disk. It is also necessary to understand the access patterns of users: an increase in the number of concurrent users accessing the database will result in an increase in the amount of memory being utilized.

CPU The CPU controls the tasks of the other system resources and executes user processes, and is the most costly resource in the system, so it needs to be correctly utilized. The main objective for this component is to prevent CPU contention in which processes are waiting for the CPU. CPU bottlenecks occur when either the operating system or user processes make too many demands on the CPU. This is often a result of excessive paging.

It is necessary to understand the typical workload through a 24-hour period and ensure that sufficient resources are available for not only the normal workload but also the peak workload (for example, if the system has 90% CPU utilization and 10% idle during the normal workload then there may not be sufficient scope to handle the peak workload). One option is to ensure that during peak load no unnecessary jobs are being run and that such jobs are instead run in off-hours. Another option may be to consider multiple CPUs, which allows the processing to be distributed and operations to be performed in parallel.

CPU MIPS (millions of instructions per second) can be used as a guide in comparing platforms and determining their ability to meet the enterprise's throughput requirements.

Disk I/O With any large DBMS, there is a significant amount of disk I/O involved in storing and retrieving data. Disks usually have a recommended I/O rate and

Figure 19.11

Typical disk configuration.



when this rate is exceeded, I/O bottlenecks occur. Although CPU clock speeds have increased dramatically in recent years, I/O speeds have not increased proportionately. The way in which data is organized on disk can have a major impact on the overall disk performance. One problem that can arise is **disk contention**. This occurs when multiple processes try to access the same disk simultaneously. Most disks have limits on both the number of accesses and the amount of data they can transfer per second and when these limits are reached, processes may have to wait to access the disk. To avoid this, it is recommended that storage should be evenly distributed across available drives to reduce the likelihood of performance problems occurring. Figure 19.11 illustrates the basic principles of distributing the data across disks:

- the operating system files should be separated from the database files;
- the main database files should be separated from the index files;
- the recovery log file (see Section 22.3.3) should be separated from the rest of the database.

If a disk still appears to be overloaded, one or more of its heavily accessed files can be moved to a less active disk (this is known as *distributing I/O*). **Load balancing** can be achieved by applying this principle to each of the disks until they all have approximately the same amount of I/O. Once again, the physical database designer needs to understand how the DBMS operates, the characteristics of the hardware, and the access patterns of the users.

Disk I/O has been revolutionized with the introduction of RAID (Redundant Array of Independent Disks) technology. RAID works on having a large disk array comprising an arrangement of several independent disks that are organized to increase performance and at the same time improve reliability. We discuss RAID in Section 20.2.7.

Network When the amount of traffic on the network is too great, or when the number of network collisions is large, network bottlenecks occur.

Each of the previous resources may affect other system resources. Additionally, an improvement in one resource may effect an improvement in other system resources. For example:

- procuring more main memory should result in less paging, which should help avoid CPU bottlenecks;
- more effective use of main memory may result in less disk I/O.

Summary Tuning is an activity that is never complete. Throughout the life of the system, it will be necessary to monitor performance, particularly to account for changes in the environment and user requirements. However, making a change to

one area of an operational system to improve performance may have an adverse effect on another area. For example, adding an index to a relation may improve the performance of one transaction, but it may adversely affect another, perhaps more important, transaction. Therefore, care must be taken when making changes to an operational system. If possible, test the changes either on a test database, or alternatively, when the system is not being fully used (such as outside of working hours).

Document tuning activity

The mechanisms used to tune the system should be fully documented, along with the reasons for tuning it in the chosen way. In particular, document the reasons for selecting one approach where many alternatives exist.



New requirement for *DreamHome*

As well as tuning the system to maintain optimal performance, it may also be necessary to handle changing requirements. For example, suppose that after some months as a fully operational database, several users of the *DreamHome* system raise two new requirements:

- (1) *Ability to hold pictures of the properties for rent, together with comments that describe the main features of the property.*

In Microsoft Office Access we are able to accommodate this request using OLE (Object Linking and Embedding) fields, which are used to store data such as Microsoft Word or Microsoft Excel documents, pictures, sound, and other types of binary data created in other programs. OLE objects can be linked to or embedded in a field in a Microsoft Office Access table and then displayed in a form or report.

To implement this new requirement, we restructure the *PropertyForRent* table to include:

- (a) a field called *picture* specified as an OLE data type; this field holds graphical images of properties, created by scanning photographs of the properties for rent and saving the images as BMP (bitmapped) graphic files;
- (b) a field called *comments* specified as a Memo data type, capable of storing lengthy text.

A form based on some fields of the *PropertyForRent* table, including the new fields, is shown in Figure 19.12. The main problem associated with the storage of graphic images is the large amount of disk space required to store the image files. We would therefore need to continue to monitor the performance of the *DreamHome* database to ensure that satisfying this new requirement does not compromise the system's performance.

- (2) *Ability to publish a report describing properties available for rent on the Web.*

This requirement can be accommodated in both Microsoft Office Access and Oracle, as both DBMSs provide many features for developing a Web application and publishing on the Internet. However, to use these features, we require a Web browser, such as Microsoft Internet Explorer or Mozilla Firefox, and a modem or other network connection to access the Internet. In Chapter 29, we describe in detail the technologies used in the integration of databases and the Web.

Properties

Property No PG21

Street 18 Dale Rd

City Glasgow

Type Detached

Rooms 5

Rent £600.00

Comment Newly decorated throughout. Well equipped kitchen. Close to local amenities.

Record: 1 of 2 No Filter Search

Figure 19.12 Form based on PropertyForRent table with new picture and comments fields.

- (3) *Enable property owners to access details of their properties and comments made by prospective clients on the Web.*

This requirement is similar to the previous requirement in terms of technology, but also requires that the database is restructured to allow for the storage of a property owner's login details, which include email and password. Once logged in, a property owner will be able to view only the details relating to their properties. (The Client relation restructured to store a property owner's login details is shown in Figure 4.3.)

Chapter Summary

- Formally, the term **denormalization** refers to a refinement to the relational schema such that the degree of normalization for a modified relation is less than the degree of at least one of the original relations. The term is also used more loosely to refer to situations in which two relations are combined into one new relation and the new relation is still normalized but contains more nulls than the original relations.
- Step 7 of physical database design includes a consideration of whether to denormalize the relational schema to improve performance. There may be circumstances in which it may be necessary to accept the loss of some of the benefits of a fully normalized design in favor of performance. This option should be considered only when it is estimated that the system will not be able to meet its performance requirements. As a rule of thumb, if

performance is unsatisfactory and a relation has a low update rate and a very high query rate, denormalization may be a viable option.

- The final step (Step 8) of physical database design is the ongoing process of monitoring and tuning the operational system to achieve maximum performance.
- One of the main objectives of physical database design is to store and access data in an efficient way. There are a number of factors that can be used to measure efficiency, including throughput, response time, and disk storage.
- To improve performance, it is necessary to be aware of how the following four basic hardware components interact and affect system performance: main memory, CPU, disk I/O, and network.

Review Questions

- 19.1 Describe the purpose of the main steps in the physical design methodology presented in this chapter.
- 19.2 Under what circumstances would you want to denormalize a logical data model? Use examples to illustrate your answer.
- 19.3 What factors can be used to measure efficiency?
- 19.4 Discuss how partitioning and denormalization improve query efficiency.
- 19.5 What are the benefits of partitioning a relation?

Exercises



- 19.6 Analyze the conceptual and logical model for the *DreamHome* case study presented in figures 19.1(a) and 19.1(b) respectively. Suggest areas that require either denormalization or partitioning. Make any necessary constructive assumptions to justify your case.

Selected Database Issues

Chapter	20	Security and Administration	559
Chapter	21	Professional, Legal, and Ethical Issues in Data Management	593
Chapter	22	Transaction Management	619
Chapter	23	Query Processing	679

Chapter Objectives

In this chapter you will learn:

- The scope of database security.
- Why database security is a serious concern for an organization.
- The types of threat that can affect a database system.
- How to protect a computer system using computer-based controls.
- The security measures provided by Microsoft Office Access and Oracle DBMSs.
- Approaches for securing a DBMS on the Web.
- The distinction between data administration and database administration.
- The purpose and tasks associated with data administration and database administration.

Data is a valuable resource that must be strictly controlled and managed, as must any corporate resource. Part or all of the corporate data may have strategic importance to an organization and should therefore be kept secure and confidential.

In Chapter 2 we discussed the database environment and, in particular, the typical functions and services of a Database Management System (DBMS). These functions and services include authorization services, such that a DBMS must furnish a mechanism to ensure that only authorized users can access the database. In other words, the DBMS must ensure that the database is secure. The term **security** refers to the protection of the database against unauthorized access, either intentional or accidental. Besides the services provided by the DBMS, discussions on database security could also include broader issues associated with securing the database and its environment. However, these issues are outside the scope of this book and the interested reader is referred to Pfleeger (2006).



Structure of this Chapter In Section 20.1 we discuss the scope of database security and examine the types of threat that may affect computer systems in general. In Section 20.2 we consider the range of computer-based controls that are available as countermeasures to these threats. In Sections 20.3 and 20.4 we describe the security measures provided by Microsoft Office Access 2010 DBMS and Oracle11g DBMS. In Section 20.5 we identify the security measures associated with DBMSs and the Web. We conclude in Section 20.6 with a discussion of the purpose and tasks associated with data administration and database administration within an organization. The examples used throughout this chapter are taken from the *DreamHome* case study described in Section 11.4 and Appendix A.

20.1 Database Security

In this section we describe the scope of database security and discuss why organizations must take seriously potential threats to their computer systems. We also identify the range of threats and their consequences on computer systems.

Database security

The mechanisms that protect the database against intentional or accidental threats.

Security considerations apply to not just the data held in a database: breaches of security may affect other parts of the system, which may in turn affect the database. Consequently, database security encompasses hardware, software, people, and data. To effectively implement security requires appropriate controls, which are defined in specific mission objectives for the system. This need for security, though often neglected or overlooked in the past, is now increasingly recognized by organizations. The reason for this turnaround is the increasing amounts of crucial corporate data being stored on computer and the acceptance that any loss or unavailability of this data could prove to be disastrous.

A database represents an essential corporate resource that should be properly secured using appropriate controls. We consider database security in relation to the following situations:

- theft and fraud;
- loss of confidentiality (secrecy);
- loss of privacy;
- loss of integrity;
- loss of availability.

These situations broadly represent areas in which the organization should seek to reduce risk, that is, the possibility of incurring loss or damage. In some situations, these areas are closely related such that an activity that leads to loss in one area may also lead to loss in another. In addition, events such as fraud or loss of privacy

may arise because of either intentional or unintentional acts, and do not necessarily result in any detectable changes to the database or the computer system.

Theft and fraud affect not only the database environment but also the entire organization. As it is people who perpetrate such activities, attention should focus on reducing the opportunities for this occurring. Theft and fraud do not necessarily alter data, as is the case for activities that result in either loss of confidentiality or loss of privacy.

Confidentiality refers to the need to maintain secrecy over data—usually only data that is critical to the organization—whereas privacy refers to the need to protect data about individuals. Breaches of security resulting in loss of confidentiality could, for instance, lead to loss of competitiveness, and loss of privacy could lead to legal action being taken against the organization.

Loss of data integrity results in invalid or corrupted data, which may seriously affect the operation of an organization. Many organizations are now seeking virtually continuous operation, the so-called 24/7 availability (that is, 24 hours a day, 7 days a week). Loss of availability means that the data, or the system, or both cannot be accessed, which can seriously affect an organization's financial performance. In some cases, events that cause a system to be unavailable may also cause data corruption.

Database security aims to minimize losses caused by anticipated events in a cost-effective manner without unduly constraining the users. In recent times, computer-based criminal activities have significantly increased and are forecast to continue to rise over the next few years.

20.1.1 Threats

Threat

Any situation or event, whether intentional or accidental, that may adversely affect a system and consequently the organization.

A threat may be caused by a situation or event involving a person, action, or circumstance that is likely to bring harm to an organization. The harm may be tangible, such as loss of hardware, software, or data, or intangible, such as loss of credibility or client confidence. The problem facing any organization is to identify all possible threats. Therefore, as a minimum, an organization should invest time and effort in identifying the most serious threats.

In the previous section we identified areas of loss that may result from intentional or unintentional activities. Although some types of threat can be either intentional or unintentional, the impact remains the same. Intentional threats involve people and may be perpetrated by both authorized users and unauthorized users, some of whom may be external to the organization.

Any threat must be viewed as a potential breach of security that, if successful, will have a certain impact. Table 20.1 presents examples of various types of threat, listed under the area on which they may have an impact. For example, “viewing and disclosing unauthorized data” as a threat may result in theft and fraud, loss of confidentiality, and loss of privacy for the organization.

The extent that an organization suffers as a result of a threat's succeeding depends upon a number of factors, such as the existence of countermeasures and contingency plans. For example, if a hardware failure occurs corrupting secondary storage, all processing activity must cease until the problem is resolved. The recovery will

TABLE 20.1 Examples of threats.

THREAT	THEFT AND FRAUD	LOSS OF CONFIDENTIALITY	LOSS OF PRIVACY	LOSS OF INTEGRITY	LOSS OF AVAILABILITY
Using another person's means of access	✓	✓	✓		
Unauthorized amendment or copying of data	✓			✓	
Program alteration	✓			✓	✓
Inadequate policies and procedures that allow a mix of confidential and normal output	✓	✓	✓		
Wire tapping	✓	✓	✓		
Illegal entry by hacker	✓	✓	✓		
Blackmail	✓	✓	✓		
Creating "trapdoor" into system	✓	✓	✓		
Theft of data, programs, and equipment	✓	✓	✓		✓
Failure of security mechanisms, giving greater access than normal		✓	✓	✓	
Staff shortages or strikes				✓	✓
Inadequate staff training		✓	✓	✓	✓
Viewing and disclosing unauthorized data	✓	✓	✓		
Electronic interference and radiation				✓	✓
Data corruption owing to power loss or surge				✓	✓
Fire (electrical fault, lightning strike, arson), flood, bomb				✓	✓
Physical damage to equipment				✓	✓
Breaking cables or disconnection of cables				✓	✓
Introduction of viruses				✓	✓

depend upon a number of factors, which include when the last backups were taken and the time needed to restore the system.

An organization needs to identify the types of threat that it may be subjected to and to initiate appropriate plans and countermeasures, bearing in mind the costs of implementing them. Obviously, it may not be cost-effective to spend considerable time, effort, and money on potential threats that may result only in

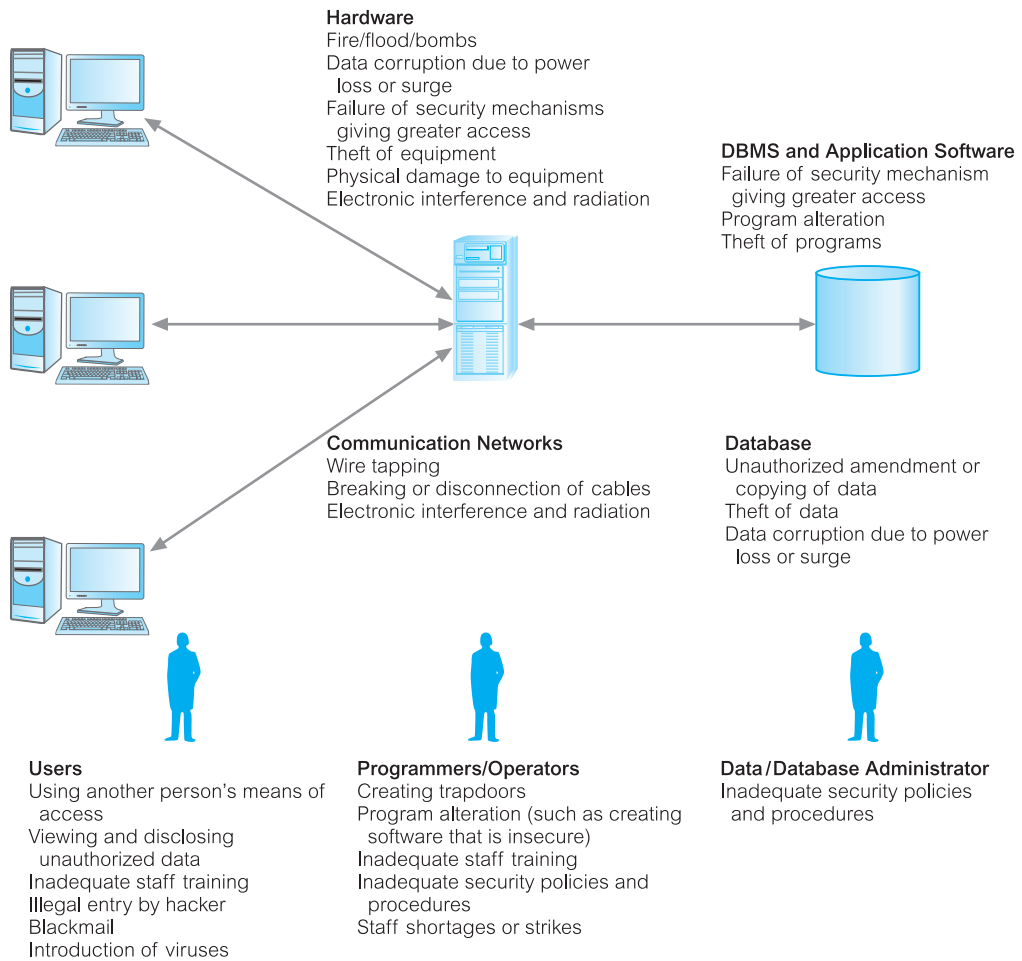


Figure 20.1 Summary of potential threats to computer systems.

minor inconvenience. The organization's business may also influence the types of threat that should be considered, some of which may be rare. However, rare events should be taken into account, particularly if their impact would be significant. A summary of the potential threats to computer systems is represented in Figure 20.1.

20.2 Countermeasures—Computer-Based Controls

The types of countermeasure to threats on computer systems range from physical controls to administrative procedures. Despite the range of computer-based controls that are available, it is worth noting that generally, the security of a DBMS is only as good as that of the operating system, owing to their close association. Representation of a typical multi-user computer environment is shown in Figure 20.2. In this section

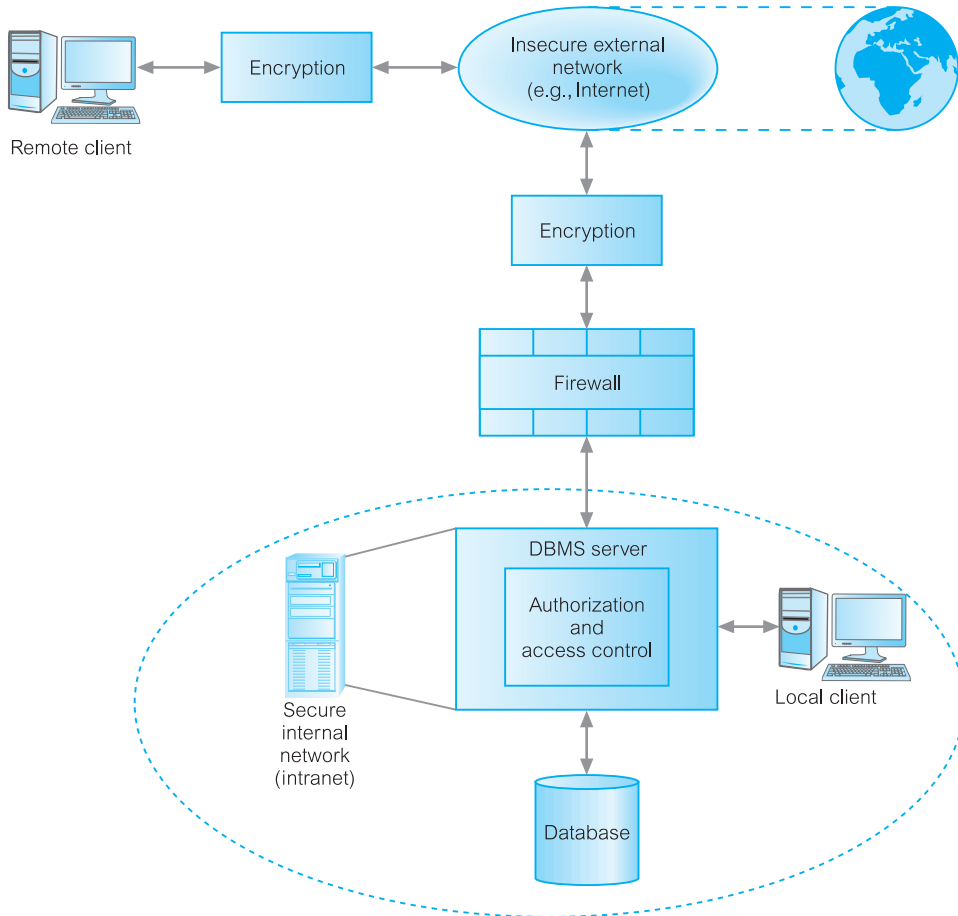


Figure 20.2 Representation of a typical multi-user computer environment.

we focus on the following computer-based security controls for a multi-user environment (some of which may not be available in the PC environment):

- authorization,
- access controls,
- views,
- backup and recovery,
- integrity,
- encryption, and
- RAID technology.

20.2.1 Authorization

Authorization

The granting of a right or privilege that enables a subject to have legitimate access to a system or a system's object.

Authorization controls can be built into the software and govern not only what system or object a specified user can access, but also what the user may do with it. The process of authorization involves authentication of subjects requesting access to objects, where “subject” represents a user or program and “object” represents a database table, view, procedure, trigger, or any other object that can be created within the system.

Authentication

A mechanism that determines whether a user is who he or she claims to be.

A system administrator is usually responsible for allowing users to have access to a computer system by creating individual user accounts. Each user is given a unique identifier, which is used by the operating system to determine who they are. Associated with each identifier is a password, chosen by the user and known to the operating system, which must be supplied to enable the operating system to verify (or authenticate) who the user claims to be.

This procedure allows authorized use of a computer system but does not necessarily authorize access to the DBMS or any associated application programs. A separate, similar procedure may have to be undertaken to give a user the right to use the DBMS. The responsibility to authorize use of the DBMS usually rests with the Database Administrator (DBA), who must also set up individual user accounts and passwords using the DBMS itself.

Some DBMSs maintain a list of valid user identifiers and associated passwords, which can be distinct from the operating system’s list. However, other DBMSs maintain a list whose entries are validated against the operating system’s list based on the current user’s login identifier. This prevents a user from logging on to the DBMS with one name, having already logged on to the operating system using a different name.

Cloud computing services describes the varied forms of computing software or hardware resources that are delivered over a network and accessed typically from a Web browser or mobile application (see Section 3.5). This technology underpins many Internet-based business and consumer products and services, and typically requires complex infrastructure that provide remote access to data, software, or computation. Cloud computing is increasingly being used by companies as a way of providing and accessing scalable software applications more conveniently, at lower risk, cost, and as a way of lowering the cost of overall investment in IT infrastructure.

For many Web- and cloud-based services, depending on the level of security, the process of authentication can typically be automated with or without computer human intervention for user account validation using email or other Internet services to ensure that individuals are who they say they are and the appropriate account is created. Online banking services, financial retail, other online retail, corporate applications products and server services, as well as e-government services follow varying combinations of account validation.

20.2.2 Access Controls

The typical way to provide access controls for a database system is based on the granting and revoking of privileges. A **privilege** allows a user to create or access (that is read, write, or modify) some database object (such as a relation, view, or

index) or to run certain DBMS utilities. Privileges are granted to users to accomplish the tasks required for their jobs. As excessive granting of unnecessary privileges can compromise security: a privilege should be granted to a user only if that user cannot accomplish his or her work without that privilege. A user who creates a database object such as a relation or a view automatically gets all privileges on that object. The DBMS subsequently keeps track of how these privileges are granted to other users, and possibly revoked, and ensures that at all times only users with necessary privileges can access an object.

Discretionary Access Control (DAC)

Most commercial DBMSs provide an approach to managing privileges that uses SQL called **Discretionary Access Control (DAC)**. The SQL standard supports DAC through the GRANT and REVOKE commands. The GRANT command gives privileges to users, and the REVOKE command takes away privileges. We discussed how the SQL standard supports discretionary access control in Section 7.6.



Discretionary access control, though effective, has certain weaknesses. In particular, an unauthorized user can trick an authorized user into disclosing sensitive data. For example, an unauthorized user such as an Assistant in the *DreamHome* case study can create a relation to capture new client details and give access privileges to an authorized user such as a Manager without their knowledge. The Assistant can then alter some application programs that the Manager uses to include some hidden instruction to copy sensitive data from the Client relation that only the Manager has access to, into the new relation created by the Assistant. The unauthorized user, namely the Assistant, now has a copy of the sensitive data, namely new clients of *DreamHome*, and to cover up his or her actions now modifies the altered application programs back to the original form.

Clearly, an additional security approach is required to remove such loopholes, and this requirement is met in an approach called Mandatory Access Control (MAC), which we discuss in detail next. Although discretionary access control is typically provided by most commercial DBMSs, only some also provide support for mandatory access control.

Mandatory Access Control (MAC)

Mandatory Access Control (MAC) is based on system-wide policies that cannot be changed by individual users. In this approach each database object is assigned a *security class* and each user is assigned a *clearance* for a security class, and *rules* are imposed on reading and writing of database objects by users. The DBMS determines whether a given user can read or write a given object based on certain rules that involve the security level of the object and the clearance of the user. These rules seek to ensure that sensitive data can never be passed on to another user without the necessary clearance. The SQL standard does not include support for MAC.

A popular model for MAC is called Bell–LaPadula model (Bell and LaPadula, 1974), which is described in terms of **objects** (such as relations, views, tuples, and attributes), **subjects** (such as users and programs), **security classes**, and **clearances**. Each database object is assigned a *security class*, and each subject is assigned a *clearance* for a security class. The security classes in a system are ordered, with a most secure class and a least secure class. For our discussion of the model, we assume that there are four classes: *top secret (TS)*, *secret (S)*, *confidential (C)*, and *unclassified (U)*, and

we denote the class of an object or subject A as *class* (A). Therefore for this system, $TS > S > C > U$, where $A > B$ means that class A data has a higher security level than class B data.

The Bell–LaPadula model imposes two restrictions on all reads and writes of database objects:

- (1) **Simple Security Property:** Subject S is allowed to read object O only if $\text{class}(S) \geq \text{class}(O)$. For example, a user with TS clearance can read a relation with C clearance, but a user with C clearance cannot read a relation with TS classification.
- (2) ***_Property:** Subject S is allowed to write object O only if $\text{class}(S) \leq \text{class}(O)$. For example, a user with S clearance can only write objects with S or TS classification.

If discretionary access controls are also specified, these rules represent additional restrictions. Thus, to read or write a database object, a user must have the necessary privileges provided through the SQL GRANT command (see Section 7.6) and the security classes of the user and the object must satisfy the restrictions given previously.

Multilevel Relations and Polyinstantiation

In order to apply mandatory access control policies in a relational DBMS, a security class must be assigned to each database object. The objects can be at the granularity of relations, tuples, or even individual attribute values. Assume that each tuple is assigned a security class. This situation leads to the concept of a **multilevel relation**, which is a relation that reveals different tuples to users with different security clearances.

For example, the *Client* relation with an additional attribute displaying the security class for each tuple is shown in Figure 20.3(a).

Users with S and TS clearance will see all tuples in the *Client* relation. However, a user with C clearance will see only the first two tuples and a user with U clearance will see no tuples at all. Assume that a user with clearance C wishes to enter a tuple (CR74, David, Sinclair) into the *Client* relation, where the primary key of the relation is *clientNo*. This insertion is disallowed, because it violates the primary key constraint (see Section 4.2.5) for this relation. However, the inability to insert this new tuple informs the user with clearance C that a tuple exists with a primary key value of CR74 at a higher security class than C . This compromises the security requirement that users should not be able to infer any information about objects that have a higher security classification.

This problem of inference can be solved by including the security classification attribute as part of the primary key for a relation. In the previous example, the insertion of the new tuple into the *Client* relation is allowed, and the relation instance is modified as shown in Figure 20.3(b). Users with clearance C see the first two tuples and the newly added tuple, but users with clearance S or TS see all five tuples. The result is a relation with two tuples with a *clientNo* of CR74, which can be confusing. This situation may be dealt with by assuming that the tuple with the higher classification takes priority over the other, or by revealing a single tuple only according to the user's clearance. The presence of data objects that appear to have different values to users with different clearances is called **polyinstantiation**.

Figure 20.3(a)

The Client relation with an additional attribute displaying the security class for each tuple.

clientNo	fName	lName	telNo	prefType	maxRent	securityClass
CR76	John	Kay	0207-774-5632	Flat	425	C
CR56	Aline	Stewart	0141-848-1825	Flat	350	C
CR74	Mike	Ritchie	01475-392178	House	750	S
CR62	Mary	Tregar	01224-196720	Flat	600	S

Figure 20.3(b)

The Client relation with two tuples displaying clientNo as CR74. The primary key for this relation is (clientNo, securityClass).

clientNo	fName	lName	telNo	prefType	maxRent	securityClass
CR76	John	Kay	0207-774-5632	Flat	425	C
CR56	Aline	Stewart	0141-848-1825	Flat	350	C
CR74	Mike	Ritchie	01475-392178	House	750	S
CR62	Mary	Tregar	01224-196720	Flat	600	S
CR74	David	Sinclair				C

Although mandatory access control does address a major weakness of discretionary access control, a major disadvantage of MAC is the rigidity of the MAC environment. For example, MAC policies are often established by database or systems administrators, and the classification mechanisms are sometimes considered to be inflexible.

20.2.3 Views

View

A view is the dynamic result of one or more relational operations operating on the base relations to produce another relation. A view is a *virtual relation* that does not actually exist in the database, but is produced upon request by a particular user, at the time of request.

The view mechanism provides a powerful and flexible security mechanism by hiding parts of the database from certain users. The user is not aware of the existence of any attributes or rows that are missing from the view. A view can be defined over several relations with a user being granted the appropriate privilege to use it, but not to use the base relations. In this way, using a view is more restrictive than simply having certain privileges granted to a user on the base relation(s). We discussed views in detail in Sections 4.4 and 7.4.

20.2.4 Backup and Recovery

Backup

The process of periodically copying of the database and log file (and possibly programs) to offline storage media.

A DBMS should provide backup facilities to assist with the recovery of a database following failure. It is always advisable to make backup copies of the database and log file at regular intervals and to ensure that the copies are in a secure location. In the event of a failure that renders the database unusable, the backup copy and the details captured in the log file are used to restore the database to the latest possible consistent state. A description of how a log file is used to restore a database is described in more detail in Section 22.3.3.

Journaling

The process of keeping and maintaining a log file (or journal) of all changes made to the database to enable recovery to be undertaken effectively in the event of a failure.

A DBMS should provide logging facilities, sometimes referred to as journaling, which keep track of the current state of transactions and database changes, to provide support for recovery procedures. The advantage of journaling is that in the event of a failure, the database can be recovered to its last known consistent state using a backup copy of the database and the information contained in the log file. If no journaling is enabled on a failed system, the only means of recovery is to restore the database using the latest backed up version of the database. However, without a log file, any changes made after the last backup to the database will be lost. The process of journaling is discussed in more detail in Section 22.3.3.

20.2.5 Integrity

Integrity constraints also contribute to maintaining a secure database system by preventing data from becoming invalid, and hence giving misleading or incorrect results. Integrity constraints were discussed in detail in Section 4.3.

20.2.6 Encryption

Encryption

The encoding of the data by a special algorithm that renders the data unreadable by any program without the decryption key.

If a database system holds particularly sensitive data, it may be deemed necessary to encode it as a precaution against possible external threats or attempts to access it. Some DBMSs provide an encryption facility for this purpose. The DBMS can access the data (after decoding it), although there is a degradation in performance because of the time taken to decode it. Encryption also protects data transmitted over communication lines. There are a number of techniques for encoding data to conceal the information; some are termed “irreversible” and others “reversible.” Irreversible techniques, as the name implies, do not permit the original data to be known. However, the data can be used to obtain valid statistical information. Reversible techniques are more commonly used. To transmit data securely over insecure networks requires the use of a **cryptosystem**, which includes:

- an *encryption key* to encrypt the data (plaintext);
- an *encryption algorithm* that with the encryption key transforms the plaintext into *ciphertext*;
- a *decryption key* to decrypt the ciphertext;
- a *decryption algorithm* that with the decryption key transforms the ciphertext back into plaintext.

One technique, called **symmetric encryption**, uses the same key for both encryption and decryption and relies on safe communication lines for exchanging the key. However, most users do not have access to a secure communication line, and to be really secure, the keys need to be as long as the message (Leiss, 1982). However, most working systems are based on user keys shorter than the message. One scheme

used for encryption is the **Data Encryption Standard (DES)**, which is a standard encryption algorithm developed by IBM. This scheme uses one key for both encryption and decryption, which must be kept secret, although the algorithm need not be. The algorithm transforms each 64-bit block of plaintext using a 56-bit key. The DES is not universally regarded as being very secure, and some authors maintain that a larger key is required. For example, a scheme called PGP (Pretty Good Privacy) uses a 128-bit symmetric algorithm for bulk encryption of the data it sends.

Keys with 64 bits are now probably breakable by major governments with special hardware, albeit at substantial cost. However, this technology will be within the reach of organized criminals, major organizations, and smaller governments in a few years. Although it is envisaged that keys with 80 bits will also become breakable in the future, it is probable that keys with 128 bits will remain unbeakable for the foreseeable future. The terms “strong authentication” and “weak authentication” are sometimes used to distinguish between algorithms that to all intents and purposes cannot be broken with existing technologies and knowledge (strong) from those that can be (weak).

Another type of cryptosystem uses different keys for encryption and decryption, and is referred to as **asymmetric encryption**. One example is **public key** cryptosystems, which use two keys, one of which is public and the other private. The encryption algorithm may also be public, so that anyone wishing to send a user a message can use the user’s publicly known key in conjunction with the algorithm to encrypt it. Only the owner of the private key can then decipher the message. Public key cryptosystems can also be used to send a “digital signature” with a message and prove that the message came from the person who claimed to have sent it. The most well known asymmetric encryption is **RSA** (the name is derived from the initials of the three designers of the algorithm).

Generally, symmetric algorithms are much faster to execute on a computer than those that are asymmetric. However, in practice, they are often used together, so that a public key algorithm is used to encrypt a randomly generated encryption key, and the random key is used to encrypt the actual message using a symmetric algorithm. We discuss encryption in the context of the Web in Section 20.5.

20.2.7 RAID (Redundant Array of Independent Disks)

The hardware that the DBMS is running on must be *fault-tolerant*, meaning that the DBMS should continue to operate even if one of the hardware components fails. This suggests having redundant components that can be seamlessly integrated into the working system whenever there is one or more component failures. The main hardware components that should be fault-tolerant include disk drives, disk controllers, CPU, power supplies, and cooling fans. Disk drives are the most vulnerable components, with the shortest times between failure of any of the hardware components.

One solution is the use of **Redundant Array of Independent Disks (RAID)** technology. RAID originally stood for *Redundant Array of Inexpensive Disks*, but more recently the “I” in RAID has come to stand for *Independent*. RAID works on having a large disk array comprising an arrangement of several independent disks that are organized to improve reliability and at the same time increase performance.

Performance is increased through *data striping*: the data is segmented into equal-size partitions (the *striping units*), which are transparently distributed across multiple disks. This gives the appearance of a single large, fast disk, although the data

is actually distributed across several smaller disks. Striping improves overall I/O performance by allowing multiple I/Os to be serviced in parallel. At the same time, data striping also balances the load among disks.

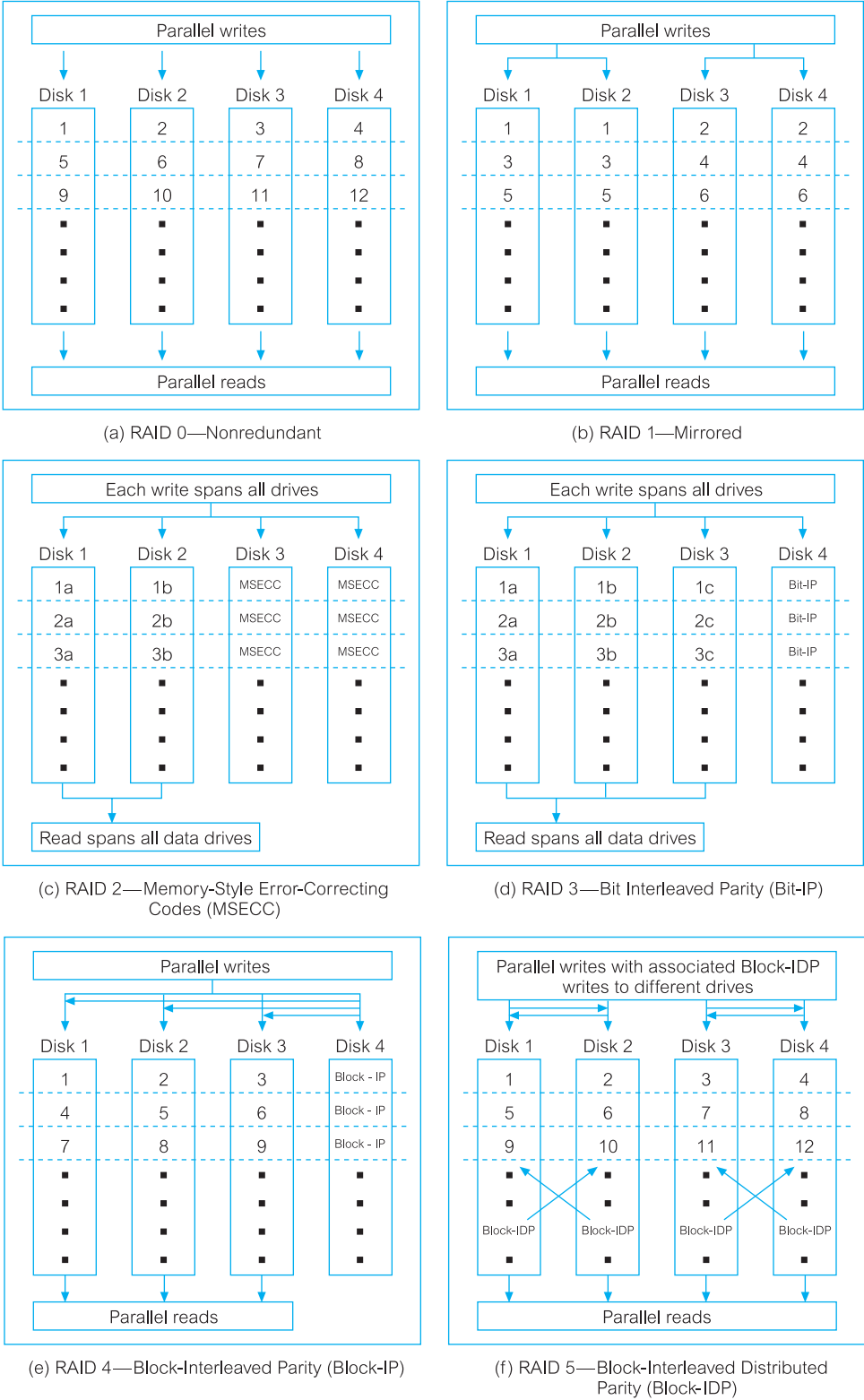
Reliability is improved through storing redundant information across the disks using a *parity* scheme or an *error-correcting* scheme, such as Reed-Solomon codes (see, for example, Pless, 1989). In a parity scheme, each byte may have a parity bit associated with it that records whether the number of bits in the byte that are set to 1 is even or odd. If the number of bits in the byte becomes corrupted, the new parity of the byte will not match the stored parity. Similarly, if the stored parity bit becomes corrupted, it will not match the data in the byte. Error-correcting schemes store two or more additional bits, and can reconstruct the original data if a single bit becomes corrupt. These schemes can be used through striping bytes across disks.

There are a number of different disk configurations with RAID, termed RAID *levels*. A brief description of each RAID level is given here, with a diagrammatic representation for each of the *main* levels in Figure 20.4. In this figure the numbers represent sequential data blocks and the letters indicate segments of a data block.

- RAID 0—Nonredundant. This level maintains no redundant data and therefore has the best write performance, as updates do not have to be replicated. Data striping is performed at the level of blocks. A diagrammatic representation of RAID 0 is shown in Figure 20.4(a).
- RAID 1—Mirrored. This level maintains (*mirrors*) two identical copies of the data across different disks. To maintain consistency in the presence of disk failure, writes may not be performed simultaneously. This is the most expensive storage solution. A diagrammatic representation of RAID 1 is shown in Figure 20.4(b).
- RAID 0+1—Nonredundant and Mirrored. This level combines striping and mirroring.
- RAID 2—Memory-Style Error-Correcting Codes. With this level, the striping unit is a single bit and Hamming codes are used as the redundancy scheme. A diagrammatic representation of RAID 2 is shown in Figure 20.4(c).
- RAID 3—Bit-Interleaved Parity. This level provides redundancy by storing parity information on a single disk in the array. This parity information can be used to recover the data on other disks should they fail. This level uses less storage space than RAID 1, but the parity disk can become a bottleneck. A diagrammatic representation of RAID 3 is shown in Figure 20.4(d).
- RAID 4—Block-Interleaved Parity. With this level, the striping unit is a disk block—a parity block is maintained on a separate disk for corresponding blocks from a number of other disks. If one of the disks fails, the parity block can be used with the corresponding blocks from the other disks to restore the blocks of the failed disk. A diagrammatic representation of RAID 4 is shown in Figure 20.4(e).
- RAID 5—Block-Interleaved Distributed Parity. This level uses parity data for redundancy in a similar way to RAID 3 but stripes the parity data across all the disks, similar to the way in which the source data is striped. This alleviates the bottleneck on the parity disk. A diagrammatic representation of RAID 5 is shown in Figure 20.4(f).
- RAID 6—P+Q Redundancy. This level is similar to RAID 5, but additional redundant data is maintained to protect against multiple disk failures. Error-correcting codes are used instead of parity.

Figure 20.4

RAID levels. The numbers represent sequential data blocks and the letters indicate segments of a data block.



Oracle, for example, recommends use of RAID 1 for the redo log files. For the database files, Oracle recommends either RAID 5, provided the write overhead is acceptable; otherwise, Oracle recommends either RAID 1 or RAID 0+1. A fuller discussion of RAID is outwith the scope of this book and the interested reader is referred to the papers by Chen and Patterson (1990) and Chen et al. (1994).

20.3 Security in Microsoft Office Access DBMS

In this section we provide an overview of the security measures provided by Microsoft Office Access 2010 DBMS. In Section 7.6 we described how SQL can be used to control access to a database through the SQL GRANT and REVOKE statements; however, Microsoft Office Access 2010 does not support these statements, but instead provides the following methods for securing a database:

- splitting the database;
- setting a password for the database;
- trusting (enabling) the disabled content in a database;
- packaging, signing, and deploying the database.

Splitting the database

The most secure way to protect data in a database is to store the database tables separately from the database application objects such as forms and reports. This action is referred to as “splitting” the database; Office Access 2010 provides a Database Splitter Wizard, available through the Access Database button of the Tools options in the Move Data section. Clicking the Access Database button displays the Database Splitter window shown in Figure 20.5.

The location of the backend database is specified and once copied to the new location, the backend database can be further protected by assigning a password as described in the following section.

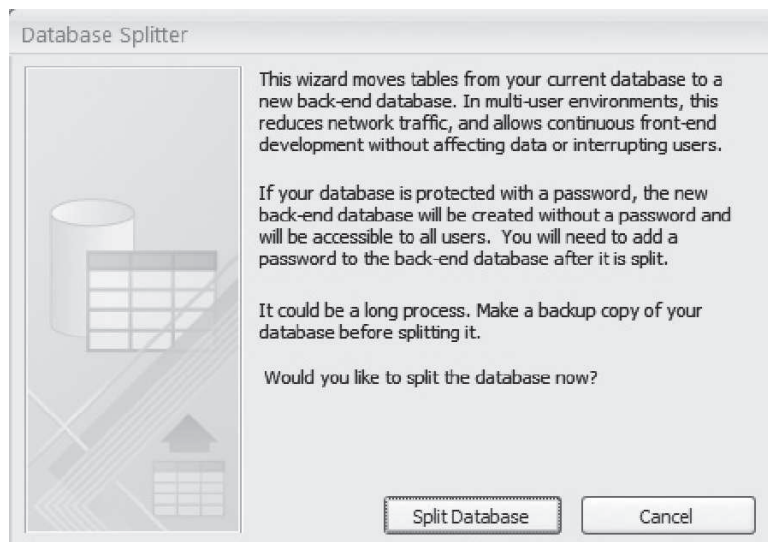


Figure 20.5
The Database Splitter Wizard window.

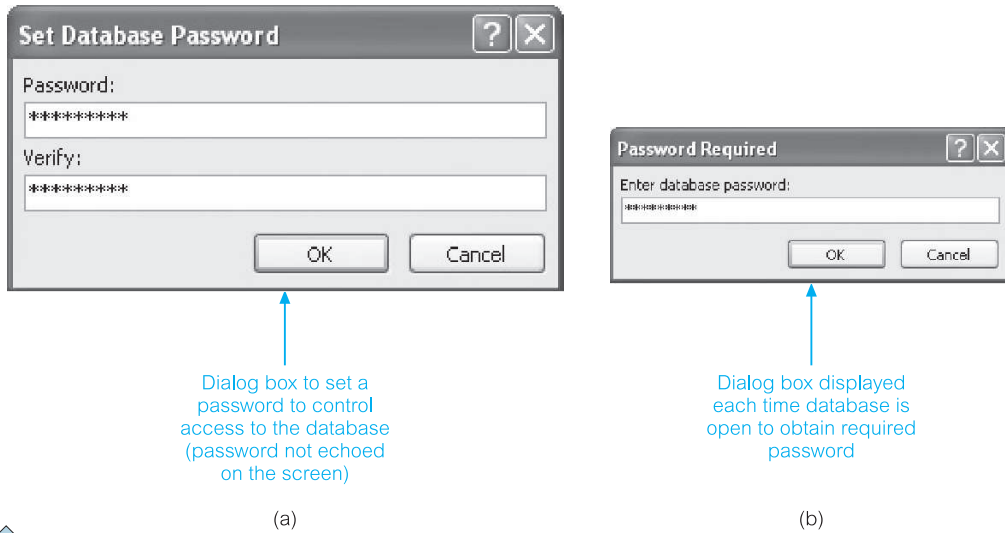


Figure 20.6 Securing the *DreamHome* database using a password: (a) the Set Database Password dialog box; (b) the Password Required dialog box shown at startup.

Setting a password for the database A simple way to secure a database is to set a password for opening the database. Setting a password is available through the Encrypt with Password option in the File/Info section. Once a password has been set, a dialog box requesting the password will be displayed when the database is opened. Only users who type the correct password will be allowed to open the database. This method is secure, as Microsoft Access encrypts the password so that it cannot be accessed by reading the database file directly. However, once a database is open, all the objects contained within the database are available to the user. Figure 20.6(a) shows the dialog box to set the password and Figure 20.6(b) shows the dialog box requesting the password whenever the database is opened.

Microsoft Access 2010 has improved encryption encoding for database passwords and integration with Microsoft SharePoint for more effective management and use of information tracking applications compared to previous versions.

Trusting (enabling) the disabled content in a database The Trust Center is a dialog box that can be used to trust (enable) the disabled content in a database. The Trust Center dialog box is shown in Figure 20.7.

The Trust Center can be used to create or change trusted locations and to set security options for Office Access 2010 databases. Those settings affect how new and existing databases behave when they are opened in that instance of Access. The Trust Center also contains logic for evaluating the components in a database and for determining whether the database is safe to open or whether the Trust Center should disable the database and let the user decide to enable it.

Packaging, signing, and deploying the database The Package-and-Sign feature places the database in an Access Deployment (.accdc) file, signs the package, and then places the code-signed package to the desired location. Users can then

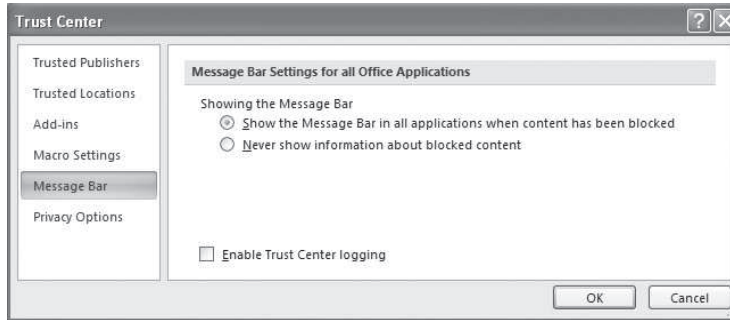


Figure 20.7
The Microsoft
Office Access
Trust Center.

extract the database from the package and work directly in the database (not in the package file). Packaging a database and signing the package are ways to convey trust. When the users receive the package, the signature confirms that the database has not been tampered with. If the users trust the author, they can enable the content.

An overview of Microsoft Office Access 2010 DBMS is provided in Appendix H (available on the Web site).

20.4 Security in Oracle DBMS

In Appendix H we provide an overview of Oracle11g DBMS. In this section we focus on the security measures provided by Oracle. In this section we examine how Oracle provides two types of security: system security and data security. As with Office Access, one form of system security used by Oracle is the standard user name and password mechanism, whereby a user has to provide a valid user name and password before access can be gained to the database, although the responsibility to authenticate users can be devolved to the operating system. Figure 20.8 illustrates the creation of a new user called Beech with password authentication set. Whenever user Beech tries to connect to the database, this user will be presented with a Connect or Log On dialog box similar to the one illustrated in Figure 20.9, prompting for a user name and password to access the specified database.

Privileges

As we discussed in Section 20.2.2, a **privilege** is a right to execute a particular type of SQL statement or to access another user's objects. Some examples of Oracle privileges include the right to:

- connect to the database (create a session);
- create a table;
- select rows from another user's table.

In Oracle, there are two distinct categories of privileges:

- system privileges;
- object privileges.

System privileges A **system privilege** is the right to perform a particular action or to perform an action on any schema objects of a particular type. For example,

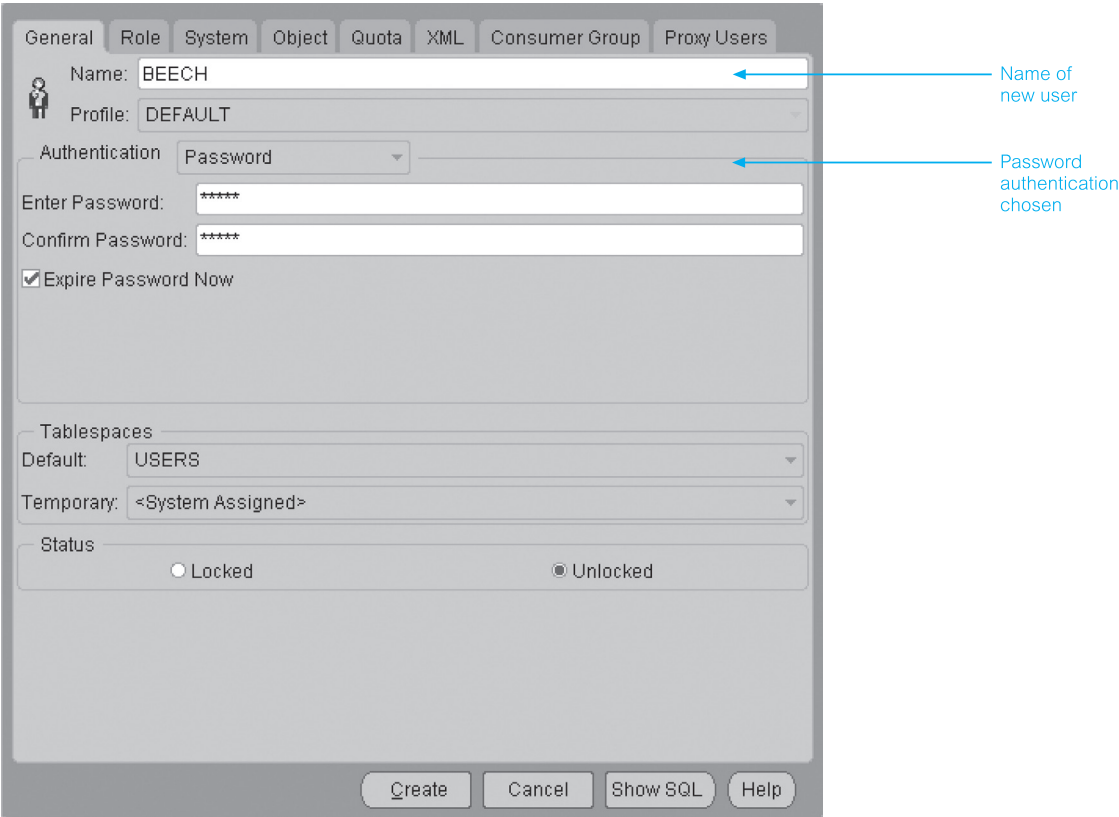
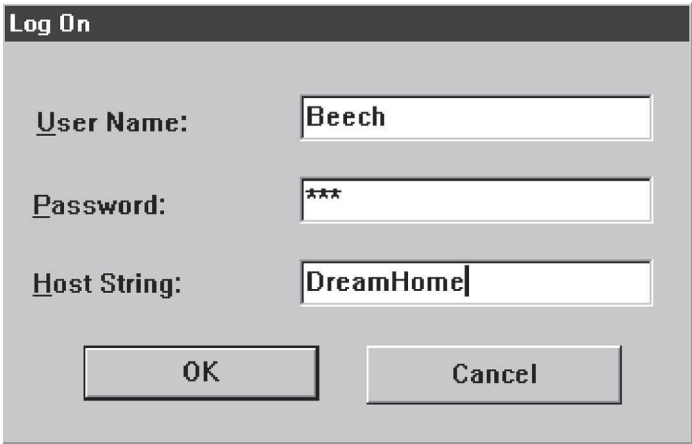


Figure 20.8 Creation of a new user called Beech with password authentication set.

Figure 20.9
Log On dialog box requesting user name, password, and the name of the database the user wishes to connect to.



the privileges to create tablespaces and to create users in a database are system privileges. There are more than eighty distinct system privileges in Oracle. System privileges are granted to, or revoked from, users and **roles** (discussed below) using either of the following:

- Grant System Privileges/Roles dialog box and Revoke System Privileges/Roles dialog box of the Oracle Security Manager;
- SQL GRANT and REVOKE statements (see Section 7.6).

However, only users who are granted a specific system privilege with the ADMIN OPTION or users with the GRANT ANY PRIVILEGE system privilege can grant or revoke system privileges.

Object privileges An **object privilege** is a privilege or right to perform a particular action on a specific table, view, sequence, procedure, function, or package. Different object privileges are available for different types of object. For example, the privilege to delete rows from the Staff table is an object privilege.

Some schema objects (such as clusters, indexes, and triggers) do not have associated object privileges; their use is controlled with system privileges. For example, to alter a cluster, a user must own the cluster or have the ALTER ANY CLUSTER system privilege.

A user automatically has all object privileges for schema objects contained in his or her schema. A user can grant any object privilege on any schema object he or she owns to any other user or role. If the grant includes the WITH GRANT OPTION (of the GRANT statement), the grantee can further grant the object privilege to other users; otherwise, the grantee can use the privilege but cannot grant it to other users. The object privileges for tables and views are shown in Table 20.2.

TABLE 20.2 What each object privilege allows a grantee to do with tables and views.

OBJECT PRIVILEGE	TABLE	VIEW
ALTER	Change the table definition with the ALTER TABLE statement.	N/A
DELETE	Remove rows from the table with the DELETE statement. Note: SELECT privilege on the table must be granted along with the DELETE privilege.	Remove rows from the view with the DELETE statement.
INDEX	Create an index on the table with the CREATE INDEX statement.	N/A
INSERT	Add new rows to the table with the INSERT statement.	Add new rows to the view with the INSERT statement.
REFERENCES	Create a constraint that refers to the table. Cannot grant this privilege to a role.	N/A
SELECT	Query the table with the SELECT statement.	Query the view with the SELECT statement.
UPDATE	Change data in the table with the UPDATE statement. Note: SELECT privilege on the table must be granted along with the UPDATE privilege.	Change data in the view with the UPDATE statement.

Roles A user can receive a privilege in two different ways:

- (1) Privileges can be explicitly granted to users. For example, a user can explicitly grant the privilege to insert rows into the `PropertyForRent` table to the user Beech:

GRANT INSERT ON `PropertyForRent` **TO** Beech;

- (2) Privileges can also be granted to a **role** (a named group of privileges), and then the role granted to one or more users. For example, a user can grant the privileges to select, insert, and update rows from the `PropertyForRent` table to the role named Assistant, which in turn can be granted to the user Beech. A user can have access to several roles, and several users can be assigned the same roles. Figure 20.10 illustrates the granting of these privileges to the role Assistant using the Oracle Security Manager.

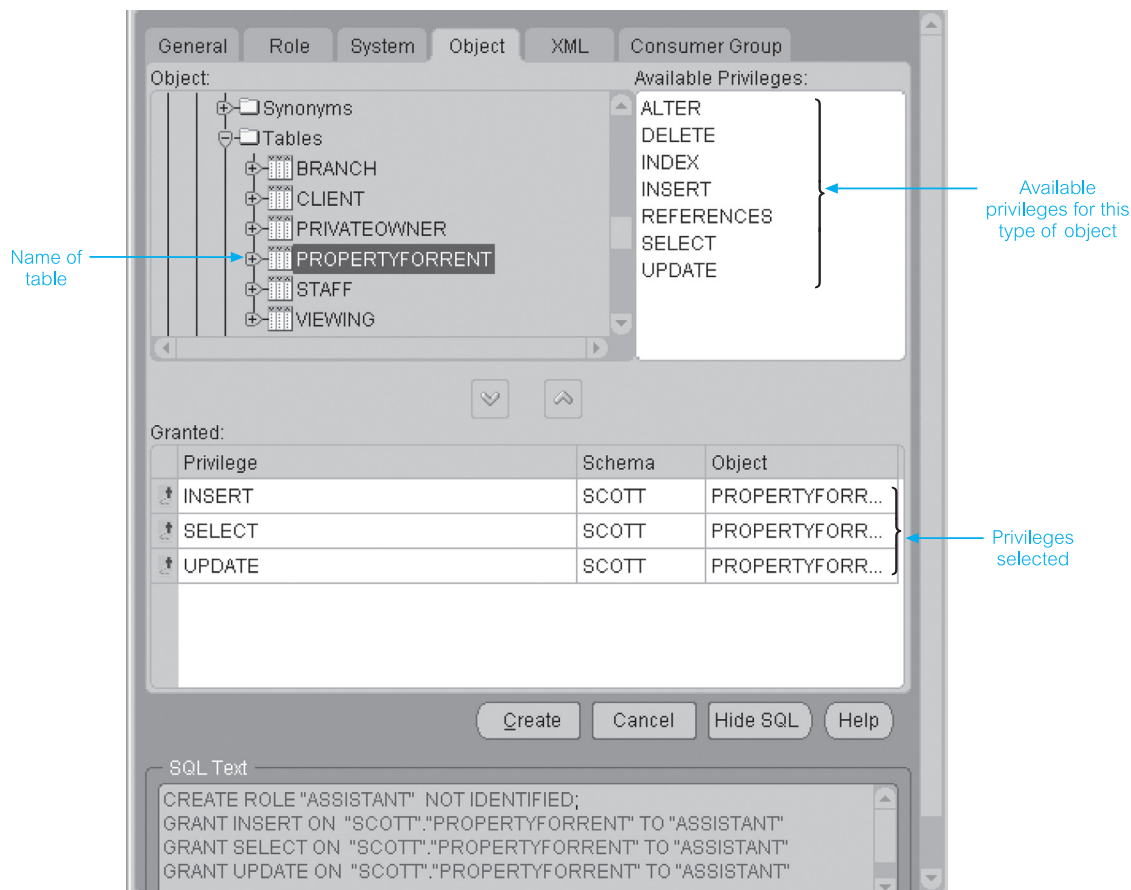


Figure 20.10 Setting the Insert, Select, and Update privileges on the `PropertyForRent` table to the role Assistant.

Because roles allow for easier and better management of privileges, privileges should normally be granted to roles and not to specific users.

20.5 DBMSs and Web Security

In Chapter 29 we provide a general overview of DBMSs on the Web. In this section we focus on how to make a DBMS secure on the Web. Those readers unfamiliar with the terms and technologies associated with DBMSs on the Web are advised to read Chapter 29 before reading this section.

Internet communication relies on TCP/IP as the underlying protocol. However, TCP/IP and HTTP were not designed with security in mind. Without special software, all Internet traffic travels “in the clear” and anyone who monitors traffic can read it. This form of attack is relatively easy to perpetrate using freely available “packet sniffing” software, as the Internet has traditionally been an open network. Consider, for example, the implications of credit card numbers being intercepted by unethical parties during transmission when customers use their cards to purchase products over the Internet. The challenge is to transmit and receive information over the Internet while ensuring that:

- it is inaccessible to anyone but the sender and receiver (privacy);
- it has not been changed during transmission (integrity);
- the receiver can be sure it came from the sender (authenticity);
- the sender can be sure the receiver is genuine (nonfabrication);
- the sender cannot deny he or she sent it (nonrepudiation).

However, protecting the transaction solves only part of the problem. Once the information has reached the Web server, it must also be protected there. With the three-tier architecture that is popular in a Web environment, we also have the complexity of ensuring secure access to, and of, the database. Today, most parts of such architecture can be secured, but it generally requires different products and mechanisms.

One other aspect of security that has to be addressed in the Web environment is that information transmitted to the client’s machine may have executable content. For example, HTML pages may contain ActiveX controls, JavaScript/VBScript, and/or one or more Java applets. Executable content can perform the following malicious actions, and measures need to be taken to prevent them:

- corrupt data or the execution state of programs;
- reformat complete disks;
- perform a total system shutdown;
- collect and download confidential data, such as files or passwords, to another site;
- usurp identity and impersonate the user or user’s computer to attack other targets on the network;
- lock up resources making them unavailable for legitimate users and programs;
- cause nonfatal but unwelcome effects, especially on output devices.

In earlier sections we identified general security mechanisms for database systems. However, the increasing accessibility of databases on the public Internet and private intranets requires a re-analysis and extension of these approaches.

In this section we address some of the issues associated with database security in these environments.

20.5.1 Proxy Servers

In a Web environment, a proxy server is a computer that sits between a Web browser and a Web server. It intercepts all requests to the Web server to determine whether it can fulfill the requests itself. If not, it forwards the requests to the Web server. Proxy servers have two main purposes: to improve performance and filter requests.

Improve performance

Because a proxy server saves the results of all requests for a certain amount of time, it can significantly improve performance for groups of users. For example, assume that user A and user B access the Web through a proxy server. First, user A requests a certain Web page and, slightly later, user B requests the same page. Instead of forwarding the request to the Web server where that page resides, the proxy server simply returns the cached page that it had already fetched for user A. Because the proxy server is often on the same network as the user, this is a much faster operation. Real proxy servers, such as those employed by Compuserve and America Online, can support thousands of users.

Filter requests

Proxy servers can also be used to filter requests. For example, an organization might use a proxy server to prevent its employees from accessing a specific set of Web sites.

20.5.2 Firewalls

The standard security advice is to ensure that Web servers are unconnected to any in-house networks and regularly backed up to recover from inevitable attacks. When the Web server must be connected to an internal network—for example, to access the company database—firewall technology can help to prevent unauthorized access, provided that it has been installed and maintained correctly.

A **firewall** is a system designed to prevent unauthorized access to or from a private network. Firewalls can be implemented as both hardware and software or a combination of both. They are frequently used to prevent unauthorized Internet users from accessing private networks connected to the Internet, especially intranets. All messages entering or leaving the intranet pass through the firewall, which examines each message and blocks those that do not meet the specified security criteria. There are several types of firewall technique:

- **Packet filter**, which looks at each packet entering or leaving the network and accepts or rejects it based on user-defined rules. Packet filtering is a fairly effective mechanism and transparent to users, but can be difficult to configure. In addition, it is susceptible to IP spoofing. (IP spoofing is a technique used to gain unauthorized access to computers in which the intruder sends messages to a computer with an IP address indicating that the message is coming from a trusted port.)

- **Application gateway**, which applies security mechanisms to specific applications, such as FTP and Telnet servers. This is a very effective mechanism, but can degrade performance.
- **Circuit-level gateway**, which applies security mechanisms when a TCP or UDP (User Datagram Protocol) connection is established. Once the connection has been made, packets can flow between the hosts without further checking.
- **Proxy server**, which intercepts all messages entering and leaving the network. The proxy server in effect hides the true network addresses.

In practice, many firewalls provide more than one of these techniques. A firewall is considered a first line of defense in protecting private information. For greater security, data can be encrypted, as discussed next and earlier in Section 20.2.6.

20.5.3 Message Digest Algorithms and Digital Signatures

A message digest algorithm, or one-way hash function, takes an arbitrarily sized string (the *message*) and generates a fixed-length string (the *digest* or *hash*). A digest has the following characteristics:

- it should be computationally infeasible to find another message that will generate the same digest;
- the digest does not reveal anything about the message.

A digital signature consists of two pieces of information: a string of bits that is computed from the data that is being “signed,” along with the private key of the individual or organization wishing the signature. The signature can be used to verify that the data comes from this individual or organization. Like a handwritten signature, a digital signature has many useful properties:

- its authenticity can be verified, using a computation based on the corresponding public key;
- it cannot be forged (assuming that the private key is kept secret);
- it is a function of the data signed and cannot be claimed to be the signature for any other data;
- the signed data cannot be changed; otherwise, the signature will no longer verify the data as being authentic.

Some digital signature algorithms use message digest algorithms for parts of their computations; others, for efficiency, compute the digest of a message and digitally sign the digest rather than signing the message itself.

20.5.4 Digital Certificates

A digital certificate is an attachment to an electronic message used for security purposes, most commonly to verify that a user sending a message is who he or she claims to be and to provide the receiver with the means to encode a reply.

An individual wishing to send an encrypted message applies for a digital certificate from a Certificate Authority (CA) such as VeriSign/Thawte, GeoTrust, or Comodo. The CA issues an encrypted digital certificate containing the applicant’s public key and a variety of other identification information. The CA makes

its own public key readily available through printed material or perhaps on the Internet.

The recipient of an encrypted message uses the CA's public key to decode the digital certificate attached to the message, verifies it as issued by the CA, and then obtains the sender's public key and identification information held within the certificate. With this information, the recipient can send an encrypted reply.

VeriSign developed the following classes for digital certificates:

- Class 1 for individuals (intended for email);
- Class 2 for organizations (for which proof of identity is required);
- Class 3 for servers/software signing (for which independent verification and checking of identity and authority is carried out by issuing a CA);
- Class 4 for online business transactions between computers;
- Class 5 for private organization or government security.

Nowadays, the most common use of certificates is for S-HTTP sites. A Web browser validates that a Secure Sockets Layer (SSL) Web server is authentic, so that the users can be assured that the interaction with the Web site has no eavesdroppers and that the site is what it claims to be. (S-HTTP and SSL are discussed shortly.)

Clearly, the CA's role in this process is critical, acting as a go-between for the two parties. In a large, distributed complex network like the Internet, this third-party trust model is necessary, as clients and servers may not have an established mutual trust yet both want to have a secure session. However, because each party trusts the CA, and because the CA is vouching for each party's identification and trustworthiness by signing their certificates, each party recognizes and implicitly trusts each other. The most widely used standard for digital certificates is X.509.

20.5.5 Kerberos

Kerberos is a server of secured user names and passwords (named after the three-headed monster in Greek mythology that guarded the gate of hell). The importance of Kerberos is that it provides one centralized security server for all data and resources on the network. Database access, login, authorization control, and other security features are centralized on trusted Kerberos servers. Kerberos has a similar function to that of a certificate server: to identify and validate a user. Security companies are currently investigating a merger of Kerberos and Certificate servers to provide a network-wide secure system.

20.5.6 Secure Sockets Layer and Secure HTTP

Many large Internet product developers agreed to use an encryption protocol known as Secure Sockets Layer (SSL) developed by Netscape for transmitting private documents over the Internet. SSL works by using a private key to encrypt data that is transferred over the SSL connection. Both Firefox and Internet Explorer support SSL, and many Web sites use this protocol to obtain confidential user information, such as credit card numbers. The protocol, layered between application-level protocols such as HTTP and the TCP/IP transport-level protocol, is designed to prevent eavesdropping, tampering, and message forgery. Because

SSL is layered under application-level protocols, it may be used for other application-level protocols such as FTP and NNTP.

Another protocol for transmitting data securely over the Web is Secure HTTP (S-HTTP), a modified version of the standard HTTP protocol. S-HTTP was developed by Enterprise Integration Technologies (EIT), which was acquired by Verifone, Inc., in 1995. Whereas SSL creates a secure connection between a client and a server, over which any amount of data can be sent securely, S-HTTP is designed to transmit individual messages securely. SSL and S-HTTP, therefore, can be seen as complementary rather than competing technologies. Both protocols have been submitted to the Internet Engineering Task Force (IETF) for approval as standards. By convention, Web pages that require an SSL connection start with **https:** instead of **http:**. Not all Web browsers and servers support SSL/S-HTTP.

Basically, these protocols allow the browser and server to authenticate one another and secure information that subsequently flows between them. Through the use of cryptographic techniques such as encryption, and digital signatures, these protocols:

- allow Web browsers and servers to authenticate each other;
- permit Web site owners to control access to particular servers, directories, files, or services;
- allow sensitive information (for example, credit card numbers) to be shared between browser and server, yet remain inaccessible to third parties;
- ensure that data exchanged between browser and server is reliable, that is, cannot be corrupted either accidentally or deliberately, without detection.

A key component in the establishment of secure Web sessions using the SSL or S-HTTP protocols is the digital certificate, discussed previously. Without authentic and trustworthy certificates, protocols like SSL and S-HTTP offer no security at all.

20.5.7 Secure Electronic Transactions and Secure Transaction Technology

The Secure Electronic Transactions (SET) protocol is an open, interoperable standard for processing credit card transactions over the Internet, created jointly by Netscape, Microsoft, Visa, Mastercard, GTE, SAIC, Terisa Systems, and VeriSign. SET's goal is to allow credit card transactions to be as simple and secure on the Internet as they are in retail stores. To address privacy concerns, the transaction is split in such a way that the merchant has access to information about what is being purchased, how much it costs, and whether the payment is approved, but no information on what payment method the customer is using. Similarly, the card issuer (for example, Visa) has access to the purchase price but no information on the type of merchandise involved.

Certificates are heavily used by SET, both for certifying a cardholder and for certifying that the merchant has a relationship with the financial institution. The mechanism is illustrated in Figure 20.11. Though both Microsoft and Visa International are major participants in the SET specifications, they currently provide the Secure Transaction Technology (STT) protocol, which has been designed

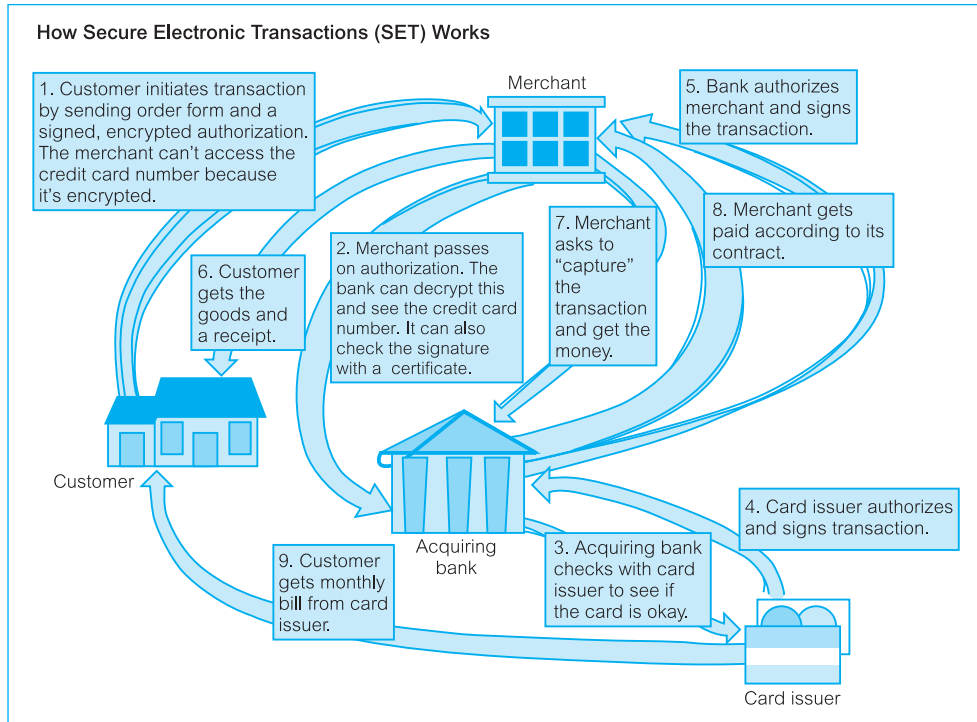


Figure 20.11 A SET transaction.

to handle secure bank payments over the Internet. STT uses DES encryption of information, RSA encryption of bankcard information, and strong authentication of all parties involved in the transaction.

20.5.8 Java Security

In Section 29.7 we introduce the Java language as an increasingly important language for Web development. Those readers unfamiliar with Java are advised to read Section 29.7 before reading this section.

Safety and security are integral parts of Java's design, with the "sandbox" ensuring that an untrusted and possibly malicious application cannot gain access to system resources. To implement this sandbox, three components are used: a class loader, a bytecode verifier, and a security manager. The safety features are provided by the Java language and the Java Virtual Machine (JVM), and enforced by the compiler and the runtime system; security is a policy that is built on top of this safety layer.

Two safety features of the Java language relate to strong typing and automatic garbage collection. In this section we look at two other features: the class loader and the bytecode verifier. To complete this section on Java security, we examine the JVM Security Manager.

The class loader

The class loader, as well as loading each required class and checking it is in the correct format, additionally checks whether the application/applet violates system security by allocating a *namespace*. Namespaces are hierarchical and allow the JVM to group classes based on where they originate (local or remote). A class loader never allows a class from a less protected namespace to replace a class from a more protected namespace. In this way, the file system's I/O primitives, which are defined in a local Java class, cannot be invoked or indeed overridden by classes from outside the local machine. An executing JVM allows multiple class loaders, each with its own namespace, to be active simultaneously. As browsers and Java applications can typically provide their own class loader, albeit based on a recommended template from Sun Microsystems, this may be viewed as a weakness in the security model. However, some argue that this is a strength of the language, allowing system administrators to implement their own (presumably tighter) security measures.

The bytecode verifier

Before the JVM will allow an application/applet to run, its code must be verified. The verifier assumes that all code is meant to crash or violate system security and performs a series of checks, including the execution of a theorem prover, to ensure that this is not the case. Typical checks include verifying that:

- compiled code is correctly formatted;
- internal stacks will not overflow/underflow;
- no “illegal” data conversions will occur (for example, integer to pointer)—this ensures that variables will not be granted access to restricted memory areas;
- bytecode instructions are appropriately typed;
- all class member accesses are valid.

The Security Manager

The Java security policy is application-specific. A Java application, such as a Java-enabled Web browser or a Web server, defines and implements its own security policy. Each of these applications implements its own Security Manager. A Java-enabled Web browser contains its own applet Security Manager, and any applets downloaded by this browser are subject to its policies. Generally, the Security Manager performs runtime verification of potentially “dangerous” methods, that is, methods that request I/O, network access, or attempt to define a new class loader. In general, downloaded applets are prevented from:

- reading and writing files on the client's file system. This also prevents applets storing persistent data (for example, a database) on the client side, although the data could be sent back to the host for storage;
- making network connections to machines other than the host that provided the compiled .class files. This is either the host where the HTML page came from, or the host specified in the CODEBASE parameter in the applet tag, with CODEBASE taking precedence;
- starting other programs on the client;

- loading libraries;
- defining method calls. Allowing an applet to define native method calls would give the applet direct access to the underlying operating system.

These restrictions apply to applets that are downloaded over the public Internet or company intranet. They do not apply to applets on the client's local disk and in a directory that is on the client's CLASSPATH. Local applets are loaded by the file system loader and, as well as being able to read and write files, are allowed to exit the virtual machine and are not passed through the bytecode verifier. The JDK (Java Development Kit) Appletviewer also slightly relaxes these restrictions, by letting the user define an explicit list of files that can be accessed by downloaded applets. In a similar way, Microsoft's Internet Explorer 4.0 introduced the concept of "zones," and some zones may be trusted and others untrusted. Java applets loaded from certain zones are able to read and write to files on the client's hard drive. The zones with which this is possible are customizable by the network administrators.

Enhanced applet security

The sandbox model was introduced with the first release of the Java applet API in January 1996. Although this model does generally protect systems from untrusted code obtained from the network, it does not address several other security and privacy issues. Authentication is needed to ensure that an applet comes from where it claims to have come from. Further, digitally signed and authenticated applets can then be raised to the status of trusted applets, and subsequently allowed to run with fewer security restrictions.

The Java Security API, available in JDK 1.1, contains APIs for digital signatures, message digests, key management, and encryption/decryption (subject to United States export control regulations). Work is in progress to define an infrastructure that allows flexible security policies for signed applets.

20.5.9 ActiveX Security

The ActiveX security model is considerably different from Java applets. Java achieves security by restricting the behavior of applets to a safe set of instructions. ActiveX, on the other hand, places no restrictions on what a control can do. Instead, each ActiveX control can be digitally signed by its author using a system called Authenticode™. The digital signatures are then certified by a CA. This security model places the responsibility for the computer's security on the user. Before the browser downloads an ActiveX control that has not been signed or has been certified by an unknown CA, it presents a dialog box warning the user that this action may not be safe. The user can then abort the transfer or continue and accept the consequences.

20.6 Data Administration and Database Administration

The Data Administration (DA) and Database Administrator (DBA) are responsible for managing and controlling the activities associated with the corporate data and the corporate database, respectively. The DA is more concerned with the early

stages of the lifecycle, from planning through to logical database design. In contrast, the DBA is more concerned with the later stages, from application/physical database design to operational maintenance. In this final section of the chapter, we discuss the purpose and tasks associated with data and database administration.

20.6.1 Data Administration

Data administration

The management of the data resource, which includes database planning, development, and maintenance of standards, policies and procedures, and conceptual and logical database design.

The DA is responsible for the corporate data resource, which includes noncomputerized data, and in practice is often concerned with managing the shared data of users or application areas of an organization. The DA has the primary responsibility of consulting with and advising senior managers and ensuring that the application of database technologies continues to support corporate objectives. In some enterprises, data administration is a distinct functional area; in others it may be combined with database administration. The tasks associated with data administration are described in Table 20.3.

TABLE 20.3 Data administration tasks.

Selecting appropriate productivity tools.
Assisting in the development of the corporate IT/IS and enterprise strategies.
Undertaking feasibility studies and planning for database development.
Developing a corporate data model.
Determining the organization's data requirements.
Setting data collection standards and establishing data formats.
Estimating volumes of data and likely growth.
Determining patterns and frequencies of data usage.
Determining data access requirements and safeguards for both legal and enterprise requirements.
Undertaking conceptual and logical database design.
Liaising with database administration staff and application developers to ensure applications meet all stated requirements.
Educating users on data standards and legal responsibilities.
Keeping up to date with IT/IS and enterprise developments.
Ensuring documentation is up to date and complete, including standards, policies, procedures, use of the data dictionary, and controls on end-users.
Managing the data dictionary.
Liaising with users to determine new requirements and to resolve difficulties over data access or performance.
Developing a security policy.

TABLE 20.4 Database administration tasks.

Evaluating and selecting DBMS products.
Undertaking physical database design.
Implementing a physical database design using a target DBMS.
Defining security and integrity constraints.
Liaising with database application developers.
Developing test strategies.
Training users.
Responsible for “signing off” on the implemented database system.
Monitoring system performance and tuning the database as appropriate.
Performing backups routinely.
Ensuring that recovery mechanisms and procedures are in place.
Ensuring that documentation is complete, including in-house produced material.
Keeping up to date with software and hardware developments and costs, and installing updates as necessary.

20.6.2 Database Administration

**Database
administration**

The management of the physical realization of a database system, which includes physical database design and implementation, setting security and integrity controls, monitoring system performance, and reorganizing the database, as necessary.

The database administration staff are more technically oriented than the data administration staff, requiring knowledge of specific DBMSs and the operating system environment. Although the primary responsibilities are centered on developing and maintaining systems using the DBMS software to its fullest extent, DBA staff also assist DA staff in other areas, as indicated in Table 20.4. The number of staff assigned to the database administration functional area varies, and is often determined by the size of the organization. The tasks of database administration are described in Table 20.4.

20.6.3 Comparison of Data and Database Administration

The preceding sections examined the purpose and tasks associated with data administration and database administration. In this final section we briefly contrast these functional areas. Table 20.5 summarizes the main task differences of data administration and database administration. Perhaps the most obvious difference lies in the nature of the work carried out. Data administration staff tend to be much more managerial, whereas the database administration staff tend to be more technical.

TABLE 20.5 Data administration and database administration—main task differences.

DATA ADMINISTRATION	DATABASE ADMINISTRATION
Involved in strategic IS planning	Evaluates new DBMSs
Determines long-term goals	Executes plans to achieve goals
Enforces standards, policies, and procedures	Enforces standards, policies, and procedures
Determines data requirements	Implements data requirements
Develops conceptual and logical database design	Develops logical and physical database design
Develops and maintains corporate data model	Implements physical database design
Coordinates system development	Monitors and controls database
Managerial orientation	Technical orientation
DBMS-independent	DBMS-dependent

Chapter Summary

- **Database security** is the mechanisms that protect the database against intentional or accidental threats. Database security is concerned with avoiding the following situations: theft and fraud, loss of confidentiality (secrecy), loss of privacy, loss of integrity, and loss of availability.
- A **threat** is any situation or event, whether intentional or accidental, that will adversely affect a system and consequently an organization.
- **Computer-based security controls** for the multiuser environment include: authorization, access controls, views, backup and recovery, integrity, encryption, and RAID technology.
- **Authorization** is the granting of a right or privilege that enables a subject to have legitimate access to a system or a system's object. **Authentication** is a mechanism that determines whether a user is who he or she claims to be.
- Most commercial DBMSs provide an approach called **Discretionary Access Control (DAC)**, which manages privileges using SQL. The SQL standard supports DAC through the GRANT and REVOKE commands. Some commercial DBMSs also provide an approach to access control called **Mandatory Access Control (MAC)**, which is based on system-wide policies that cannot be changed by individual users. In this approach each database object is assigned a *security class* and each user is assigned a *clearance* for a security class, and *rules* are imposed on reading and writing of database objects by users. The SQL standard does not include support for MAC.
- A **view** is the dynamic result of one or more relational operations operating on the base relations to produce another relation. A view is a **virtual relation** that does not actually exist in the database but is produced upon request by a particular user at the time of request. The view mechanism provides a powerful and flexible security mechanism by hiding parts of the database from certain users.
- **Backup** is the process of periodically taking a copy of the database and log file (and possibly programs) on to offline storage media. **Journaling** is the process of keeping and maintaining a log file (or journal) of all changes made to the database to enable recovery to be undertaken effectively in the event of a failure.
- **Integrity constraints** also contribute to maintaining a secure database system by preventing data from becoming invalid, and hence giving misleading or incorrect results.
- **Encryption** is the encoding of the data by a special algorithm that renders the data unreadable by any program without the decryption key.
- **Cloud computing** is the use of computing software or hardware resources that are delivered over a network and accessed typically from a Web browser or mobile application.

- **Microsoft Office Access DBMS** provides four methods to secure the database including splitting the database, setting a password, trusting (enabling) the disabled content of a database, packaging, signing, and deploying the database.
- **Oracle DBMS** provides two types of security measure: system security and data security. **System security** enables the setting of a password for opening a database, and **data security** provides user-level security, which can be used to limit the parts of a database that a user can read and update.
- The security measures associated with **DBMSs on the Web** include: proxy servers, firewalls, message digest algorithms and digital signatures, digital certificates, Kerberos, Secure Sockets Layer (SSL) and Secure HTTP (S-HTTP), Secure Electronic Transactions (SET) and Secure Transaction Technology (SST), Java security, and ActiveX security.
- **Data administration** is the management of the data resource, including database planning, development and maintenance of standards, policies and procedures, and conceptual and logical database design.
- **Database administration** is the management of the physical realization of a database system, including physical database design and implementation, setting security and integrity controls, monitoring system performance, and reorganizing the database as necessary.

Review Questions

- 20.1 Explain the purpose and scope of database security.
- 20.2 The scope of database security extends beyond just DBMS controls. Discuss the role of the database administrator in database security and recovery.
- 20.3 Explain the following in terms of providing security for a database:
 - (a) authorization;
 - (b) authentication;
 - (c) access controls;
 - (d) views;
 - (e) backup and recovery;
 - (f) integrity;
 - (g) encryption;
 - (h) RAID technology.
- 20.4 Distinguish between role and privilege in the context of object and systems privilege.
- 20.5 Describe the security challenges associated with the use of DBMSs on the web.
- 20.6 Describe the approaches for securing DBMSs on the Web.
- 20.7 Describe cloud computing services and the databases within their context.
- 20.8 Describe any specific security measures for databases in mobile applications and devices.
- 20.9 What tasks are associated with data administration?
- 20.10 What tasks are associated with database administration?

Exercises

- 20.11 Examine any DBMS used by your organization and identify the security measures provided.
- 20.12 Identify the types of security approach that are used by your organization to secure any DBMSs that are accessible over the Web.



- 20.13 Consider the *DreamHome* case study described in Chapter 11. List the potential threats that could occur and propose countermeasures to overcome them.
- 20.14 Consider the *Wellmeadows Hospital* case study described in Appendix B.3. List the potential threats that could occur and propose countermeasures to overcome them.
- 20.15 Investigate whether data administration and database administration exist as distinct functional areas within your organization. If identified, describe the organization, responsibilities, and tasks associated with each functional area.
- 20.16 Describe the use and potential uses of cloud computing services by your organization and list the potential threats, implications, and countermeasures.
- 20.17 Analyze the security features of three DBMSs and discuss the similarities and differences between them.
- 20.18 You are contracted to deploy database security for the university accommodation system. Describe how you will approach the project.

Professional, Legal, and Ethical Issues in Data Management

Chapter Objectives

In this chapter you will learn:

- How to define ethical, IT governance, and legal issues in IT.
- How to distinguish between legal, IT governance, and ethical issues and situations data/database administrators face.
- How new regulations are placing additional requirements and responsibilities on data/database administrators.
- How legislation such as the Sarbanes-Oxley Act and the Basel II accords affect data/database administration functions.
- Best practices for preparing for and supporting auditing, IT governance, and compliance functions.
- Intellectual property (IP) issues related to IT and data/database administration.

As discussed in Chapter 20, data and database administrators should be well versed in issues related to the day-to-day internal control over growing volumes of organizational data. Such managers, however, will increasingly find themselves delving deeply into the relatively unfamiliar territory of legal and ethical compliance. With the increased scrutiny of corporations brought on by the massive scandals emerging in the past several years (for example, Enron and WorldCom in the United States and Parmalat in Europe) have come new laws and regulations that will bring about significant changes in the way organizations operate. The rise in cases of identity theft has likewise brought increased scrutiny upon data management practices. In the context of this book, the impact of legal and ethical issues on the IT functions quite often revolves around the management of data. The aim of this chapter is therefore to define the underlying issues, to illustrate the data management context in which these issues present themselves, and to provide best practices for data and database administrators with respect to these issues.



Structure of this Chapter We begin this chapter by defining what constitutes legal and ethical issues in the context of IT in Section 21.1. In Section 21.2, we discuss regulations associated with controlling these issues and highlight the impact such regulations can have on data and database administration functions. In Section 21.3 we examine the steps towards developing an organization-wide policy and processes for legal and ethical behavior with relation to data management and IT governance. We use the *DreamHome* property rentals case study described in Section 11.4 to present a worked example of developing such a policy. We conclude this chapter with Section 21.4, which introduces some of the main concepts underlying intellectual property.

21.1 Defining Legal and Ethical Issues in IT

It is interesting to first consider why it is deemed necessary to include a chapter on legal, IT governance, and ethical issues in a book about database systems. The answer is simple: organizations around the world increasingly find themselves having to answer tough questions about the conduct and character of their employees and the manner in which their activities are carried out. At the same time, we need to develop knowledge of what constitutes professional and unprofessional behavior. Additionally, in order to ensure more transparency and effectiveness in IT resource and data management, organizations face increasing pressures to instill processes and policies that are in support of these efforts.

21.1.1 Defining Ethics in the Context of IT

In the past, IT workers in their roles supporting the organization may have felt largely shielded from the actions and deeds (or misdeeds, as the case may be) of their managers and executives. After all, the executives and managers define how the organization operates with IT staff simply implementing what they dictate. Today, however, this is no longer the case. The confluence of massive increases in storage and processing power, influence on the rest of the organization, and the increase in scrutiny by shareholders, watchdogs, the media, and quite possibly insiders, have placed IT staff in the limelight. It is not unusual, for example, for even a small eCommerce vendor to amass many terabytes (10^{12} bytes) of “clickstream”¹ data from users. Financial services firms deal with petabytes (10^{15} bytes) of financial transactions involving hundreds of millions of individuals and business entities. Within such a technology climate, IT departments are being asked to leverage these mountains of data to achieve business advantages. As was discussed in Chapter 20, individuals intent on identity theft and fraud (who could come from inside or

¹A clickstream is a web or application log that traces where and when a user clicks the links on a Web site or application.

outside of the organization) find these volumes of data extremely tempting. At the same time, overzealous managers may become tempted by the possibilities of business intelligence technologies whose usage can push against ethical boundaries. (We discuss business intelligence in Chapters 31 to 34). Needless to say, legislators and government regulations are having a hard time keeping up with the rapidly shifting technological landscape. In addition, new and emerging regulations require accurate data analysis to demonstrate compliance and conformity.

For the purposes of this chapter, we start with some basic definitions and discuss some example scenarios to illustrate best practices.

Ethics

A set of principles of correct conduct or a theory or a system of moral values.

There are many definitions of ethics and most seem quite distant from what the business or technology person faces on a daily basis. It might be helpful therefore to think of ethical behavior as “doing what is right” according to the standards of society. This, of course, begs the question “of whose society,” as what might be considered ethical behavior in one culture (country, religion, and ethnicity) might not be so in another. Tackling this particular debate is beyond the scope of this book.

21.1.2 The Difference Between Ethical and Legal Behavior

Another point of confusion may be the contrast between what is *ethical* and what is *legal*. We may think of laws as simply enforcing certain ethical behaviors. For example, most would agree that it is unethical to steal from another. Enough individuals seem capable of dismissing this aspect of “doing what is right” that in most societies laws must be enacted to enforce this particular ethical behavior. This line of thinking leads to two familiar ideas: what is ethical is legal and what is unethical is illegal. Many examples can be provided to support these claims.

This claim, however, brings up the question: Is all unethical behavior illegal? Consider a lonely database administrator for *DreamHome* who is using his or her administrative access privileges to query the *DreamHome* database for clients whose property rental patterns suggest that they are single. It may very well be the case that no laws are broken, while at the same time the company would consider this unethical behavior.

Another question that can be asked: Is all ethical behavior legal? Consider, for example, the U.S. Securities and Exchange Commission (SEC) Reg NMS (Regulation National Market System). This sweeping set of regulations aims to alter the way in which equities (stocks) are traded in U.S. markets. One aspect of Reg NMS is called the “order protection rule” and states that a trade cannot execute at one price if a better price can be found in a different “fast” exchange or market (a fast market being one capable of subsecond executions). For example, suppose that IBM shares are selling for \$80.15 with buyers available for 50,000 shares on the New York Stock Exchange (NYSE), but they are selling for \$80.14 with buyers for only 300 shares on Instinet, an electronic stock market. The NYSE is required under Reg NMS to route the shares to the market with the cheaper price. However, institutional investors such as mutual funds and retirement funds trading very large blocks of stock need to trade



on the market with the greatest liquidity, that is, with the most shares available to buy or sell, ensuring the fastest completion of their order, even if it might be a penny or two less advantageous at that moment. The reason is that if the market sees such a large order coming, the bid (or offer) will move away from the current price; that is, a large sell order will drive the price down. As a result, under Reg NMS, which was intended to ensure best price execution for all clients, changes what appears to be ethical for an institutional investor to be illegal for their broker or the exchange receiving the order, even on their client's instructions.

In essence, ethics precedes the law. Ethical codes of practice help determine whether specific laws should be introduced. Ethics fills the gap between the time when technology creates new problems and the time when laws are introduced.

21.1.3 Ethical Behavior in IT

A survey conducted by TechRepublic (techrepublic.com), an IT-oriented web portal maintained by CNET Networks, reported that 57% of the IT workers polled indicated they had been asked to do something “unethical” by their supervisors (Thornberry, 2002). Although the survey did not delve into specifics actions, clearly this is a troubling figure. Actions that most consider unethical (and also illegal) include installing unlicensed software (see Section 21.4.5), accessing personal information, and divulging trade secrets.

Another aspect of database technology concerns the use of data mining and data warehousing to aggregate and find associations and patterns among disparate data. Such business intelligence tools have evolved significantly over the past decade, and coupled with the tremendous increases in processing power and storage, afford even small and medium-sized businesses the ability to examine client behavior on an extremely fine-grain level.

There are many clearly ethical uses for such technology. For example, consider the case of a suspect who is placed near the scene of a crime by linking the suspect's car (identified by a video scan of the license plate) to the suspect's cash machine transaction. However, just as easily, we can think of many examples of situations that would violate ethical and privacy standards. For example, as a database administrator, imagine that you are asked to implement a schema for a loan application process that includes the applicant's race or national origin.

A growing number of daily events trigger data collection on a grand scale; for example, making a cell phone call, making a purchase with a credit or debit card, applying for a loan, passing through an automated toll booth or subway turnstile, driving around London, visiting a doctor or a pharmacy, or simply clicking on a link on a Web site all generate transaction logs that can potentially be correlated, joined, and otherwise mined for patterns that many business and government entities would take great interest in. The temptation to collect and mine data is made even greater as businesses are pressured to gain competitive advantage.

These examples illustrate legal behavior that many would consider unethical. The issue comes down to recognizing the fact that although advances in IT make these kinds of analysis possible, governments and legal communities have been slow to recognize the potential threats to privacy and thus the decision of whether these activities *should* be carried out is left up to the individual organization. Such decisions, in turn, necessarily depend upon the organization's culture and awareness

of what constitutes ethical behavior. In Section 21.3, we discuss best practices for developing a culture of ethical behavior among IT workers.

IT Governance

Corporate IT has become more sophisticated within organizations and, as highlighted earlier, the influence of data and IT resources on the rest of the organization and its associated business strategies has increased dramatically to become a vital function to support innovation, efficiencies and growth in most organizations over the past few decades. As a result of this, alignment of IT investment to business has become a major concern of board and executive management.

Governance became an important issue in the wake of large corporate scandals in 2002 that resulted in the adoption of practice and legislation for defining organizational objectives, rules and processes for monitoring financial and corporate performance to ensure that the objectives are transparently obtained. IT governance builds on priorities in accountability, decision rights, management and use of IT resources to ensure that organizational business objectives are met with IT. Additionally, IT governance must also inculcate legislative regulations and/or voluntary regulations, standards and best practice put in place to reduce illegal behavior, improve efficiencies, and risks of IT failures or vulnerabilities on other companies, national infrastructure, consumers and the natural environment. As a result of this, we provide the following widely accepted definition for IT governance (Weill and Ross, 2004):

IT Governance

Used for specifying the decision rights and accountability framework to encourage desirable behavior in the use of IT.

21.2 Legislation and Its Impact on the IT Function

As mentioned in the previous section, often the distinction between what is legal and what is ethical becomes blurred by the introduction of or amendments to laws and regulations that affect how data may be collected, processed, and distributed. In this section we discuss several recent regulations and discuss the impact such regulations have on data and database administration functions.

21.2.1 Securities and Exchange Commission (SEC) Regulation National Market System (NMS)

In the context of activities that appear ethical but are in fact illegal, we considered the SEC's Regulation NMS and the "order protection rule," under which an activity that is acceptable to one facet of the investment community (purchasing a large block of shares at an inferior price) was deemed illegal under the new regulation. As a result of this regulation, financial services firms are now required to collect market data so that they can demonstrate that a better price was indeed not available at the time the trade was executed. The data administrator for a financial services firm would thus need to be aware of this regulation and how it affects the

trading operations within the firm. Additional databases would need to be developed to link the trading activity to the market data and reporting tools would be required to generate confirmation reports and *ad hoc* reports to satisfy SEC inquiries.

21.2.2 The Sarbanes-Oxley Act, COBIT, and COSO

In the aftermath of major financial frauds allegedly carried out within companies such as Enron, WorldCom, Parmalat, and others, both the U.S. and European governments have put forth legislation to tighten requirements on how companies form their board of directors, interact with auditors, and report their financial statements. In the United States this legislation has taken the form of the Sarbanes-Oxley Act of 2002 (SOX), which also affects European companies who are listed on U.S. exchanges. Although the focus of SOX is on accounting and finance-related issues, data management and auditing of databases and applications have been thrust to the forefront, as companies must now certify the accuracy of their financial data. From a data administrator's perspective, SOX places increased requirements on the security and auditing of financial data, and this requirement has implications for how data is collected, processed, secured, and (possibly) reported, both internally and externally to the organization.

Internal controls

A set of rules that an organization adopts to ensure that policies and procedures are not violated, data is properly secured and reliable, and operations can be carried out efficiently.

In order to comply with SOX, companies must adopt a formal control framework for information management risks. Two leading frameworks are the “Control Objectives for Information and related Technology” (COBIT) and the “Committee of Sponsoring Organizations of the Treadway Commission” (COSO), as we now discuss.

COBIT was created in 1996 by the IT Governance Institute (ITGI) and has since gone through four revisions to its current form, COBIT 5, released in 2012 by the Information Systems Audit and Control Association (ISACA) and ITGI. The ISACA Web site for COBIT (www.isaca.org/cobit) explains:

COBIT 5 provides a comprehensive framework that assists enterprises in achieving their objectives for the governance and management of enterprise information and technology assets (IT). . . . COBIT 5 enables IT to be governed and managed in a holistic manner for the entire enterprise, taking in the full end-to-end business and IT functional areas of responsibility, considering the IT-related interests of internal and external stakeholders.

COBIT 5 consolidates and integrates COBIT 4.1, Val IT 2.0 and Risk IT frameworks, and draws from ISACA's IT Assurance Framework (ITAF) and the Business Model for Information Security (BMIS). It aligns with a number of frameworks and standards such as Information Technology Infrastructure Library (ITIL), International Organization for Standardization (ISO), Project Management Body of Knowledge (PMBOK), PRINCE2 and The Open Group Architecture Framework (TOGAF).

The COBIT 5 processes are split into governance and management areas. These two areas contain a total of five domains and 37 processes:

- **Governance of Enterprise IT**

- **Evaluate, Direct and Monitor (EDM):** *Governance* ensures that enterprise objectives are achieved by evaluating stakeholder needs, conditions and options; setting direction through prioritisation and decision making; and monitoring performance, compliance and progress against agreed-on direction and objectives. This domain consists of five processes:

EDM01	Ensure Governance Framework Setting and Maintenance
EDM02	Ensure Benefits Delivery
EDM03	Ensure Risk Optimization
EDM04	Ensure Resource Optimization
EDM05	Ensure Stakeholder Transparency

- **Management of Enterprise IT**

- **Align, Plan and Organize (APO):** APO covers the use of information and technology and how best it can be used in an organization to help achieve the organization's goals and objectives. It also highlights the organizational and infrastructural form IT is to take in order to achieve the optimal results and produce maximum benefits from its use. This domain consists of 13 processes:

APO01	Manage the IT Management Framework
APO02	Manage Strategy
APO03	Manage Enterprise Architecture
APO04	Manage Innovation
APO05	Manage Portfolio
APO06	Manage Budget and Costs
APO07	Manage Human Relations
APO08	Manage Relationships
APO09	Manage Service Agreements
APO10	Manage Suppliers
APO11	Manage Quality
APO12	Manage Risk
APO13	Manage Security

- **Build, Acquire and Implement (BAI):** BAI covers identifying IT requirements, acquiring the technology, and implementing it within the organization's current business processes. This domain consists of 10 processes:

BAI01	Manage Programs and Projects
BAI02	Manage Requirements Definition
BAI03	Manage Solutions Identification and Build
BAI04	Manage Availability and Capacity
BAI05	Manage Organizational Change Enablement
BAI06	Manage Changes
BAI07	Manage Changes Acceptance and Transitioning
BAI08	Manage Knowledge
BAI09	Manage Assets
BAI10	Manage Configuration

- **Deliver, Service, and Support (DSS):** DSS covers the delivery aspects of IT, such as the execution of the applications within the IT system and its results, and the support processes that enable the effective and efficient execution of these IT systems. This domain consists of six processes:

DSS01	Manage Operations
DSS02	Manage Service Requests and Incidents
DSS03	Manage Problems
DSS04	Manage Continuity
DSS05	Manage Security Services
DSS06	Manage Business Process Controls

- **Monitor, Evaluate, and Assess (MEA):** MEA deals with an organization's strategy in assessing its needs and whether or not the current IT system still meets the objectives for which it was designed and the controls necessary to comply with regulatory requirements. Monitoring also covers the issue of an independent assessment of the effectiveness of the IT system's ability to meet business objectives and the organization's control processes by internal and external auditors. This domain consists of three processes:

MEA01	Monitor, Evaluate, and Assess Performance and Conformance
MEA02	Monitor, Evaluate, and Assess the System of Internal Control
MEA03	Evaluate and Assess Compliance with External Requirements

On the other hand, the COSO framework focuses more narrowly on internal controls and consists of five major components, including:

- *Control environment*—establishes a culture of control, accountability, and ethical behavior.
- *Risk assessment*—evaluates the risks faced in carrying out the organization's objectives.
- *Control activities*—implements controls necessary to mitigate risks.

- *Information and communications*—specifies the paths of reporting and communication within an organization and between the organization and its trading partners.
- *Monitoring*—assessing the effectiveness of controls put in place.

Clearly, data administrators should be directly involved in each of these components. In large organizations, this would mean working closely with senior management to make them aware of the impact of implementing these controls from the IT perspective. Significant resources (new software, hardware, databases, training, and new personnel) will need to be lobbied for.

It is worth noting that although COBIT provides guidelines on policies and standards for organizations, it does not provide recommendations on implementing them within processes and designing supporting structures for the guidelines. This is something that organizations have to interpret for themselves.

21.2.3 The Health Insurance Portability and Accountability Act

The Health Insurance Portability and Accountability Act (HIPAA) of 1996 is administered by the Department of Health and Human Services in the United States and affects all providers of healthcare and health insurance. Provisions of the act are being rolled out in stages, with deadlines for implementation that depend upon the size and nature of the healthcare provider. The Act has several main provisions, of which the following five are of direct importance to database management:

- (1) *Privacy of patient information* (mandated by April 2003 for most organizations). Patients are now required to sign consent forms to allow their healthcare providers to share medical information with other providers and insurers. Data administrators must ensure that databases and systems track patient consent and allow or deny data transfers (either electronically or on paper) accordingly.
- (2) *Standardizing electronic health/medical records and transactions between healthcare organizations* (mandated by October 2003). A series of standards have been developed that cover typical healthcare transactions such as claims, enrollment, patient eligibility, payments. For data administrators, this has meant changing enterprise data models to include additional attributes and to work with electronic data interchange (EDI²) software vendors to ensure that healthcare data can be exchanged in a standard format between organizations.
- (3) *Establishing a nationally recognized identifier for employees to be used by all employee health plans* (July 2004 for most organizations). Such an identifier (which may not be the Social Security Number) will then be used in all subsequent transactions between healthcare organizations. Again, data administrators need to introduce changes to enterprise data models to include these alternative identifiers.
- (4) *Standards for the security of patient data and transactions involving this data* (April 2005 for most organizations). Patient data must be secured both within database systems as well as when transmitted between organizations. Failure to secure patient data can result in large fines. Database vendors such as Oracle and IBM

²EDI is the computer-to-computer exchange of business data over a network. Typical transactions include purchase orders, invoices, payments, and so on.

now offer tools to encrypt data within columns of database tables and provide facilities to enable fine-grained auditing of database transactions.

- (5) *Need for a nationally recognized identifier for healthcare organizations and individual providers.* The implications for this provision are similar to those for the standardized employee identifier discussed previously.

21.2.4 The European Union (EU) Directive on Data Protection of 1995

The official title of the EU's data protection directive is the "Directive 95/46/EC of the European Parliament and of the Council of 24 October 1995 on the protection of individuals with regard to the processing of personal data and on the free movement of such data" (OJEC, 1995). This directive, adopted by all EU members in 1995, spans 34 articles and is perhaps the most comprehensive of all similar directives or acts in the world today. An in-depth treatment of all aspects of the directive is beyond the scope of this book; however, some articles of the directive merit particular attention:

- Articles 6 and 7 consist of 11 requirements, of which 8 were used as a basis for the U.K.'s data protection principles described in the next section.
- Article 8 focuses on the processing of "personal data revealing racial or ethnic origin, political opinions, religious or philosophical beliefs, trade-union membership, and the processing of data concerning health or sex life." This activity is generally prohibited; however, 10 exceptions are noted, including that if the subject gives consent, the processor is ordered by law or does the work in accordance with its normal business functions, and so on.
- Articles 10, 11, and 12 address how data is collected and the rights of individuals to see their data and appeal for corrections.
- Articles 16 and 17 address the confidentiality and security measures taken while data is collected and processed.
- Articles 18 through 21 deal with how a processor notifies an EU member of its intention to process data, and situations under which the EU member will publicize the processing operations.

The EU Directive is founded on seven principles:

- *Notice:* subjects whose data is being collected should be given notice of such collection.
- *Purpose:* data collected should be used only for stated purposes and for no other purposes.
- *Consent:* personal data should not be disclosed or shared with third parties without consent from its subject.
- *Security:* once collected, personal data should be kept safe and secure from potential abuse, theft, or loss.
- *Disclosure:* subjects whose personal data is being collected should be informed as to the party or parties collecting such data.
- *Access:* subjects should be granted access to their personal data and allowed to correct any inaccuracies.
- *Accountability:* subjects should be able to hold personal data collectors accountable for adhering to all of these principles.

In this Directive, personal data means “any information relating to an identified or identifiable natural person (‘data subject’); an identifiable person is one who can be identified, directly or indirectly, in particular by reference to an identification number or to one or more factors specific to his physical, physiological, mental, economic, cultural, or social identity” (Article 2a). Data is considered personal when it allows anyone to link information to a specific person, even if the person or entity holding that data cannot make that link. Examples of such data include addresses, bank statements, and credit card numbers. Processing is also broadly defined and involves any manual or automatic operation on personal data, including its collection, recording, organization, storage, modification, retrieval, use, transmission, dissemination or publication, and even blocking, erasure, or destruction (Article 2b). The directive applies not only when the controller is based or operates within the EU, but whenever the controller uses equipment located inside the EU to process personal data. Thus, organizations from outside the EU who process personal data inside the EU must still comply with this directive.

In January 2012, the European Commission unveiled a draft legislative package to create a single European data protection law that will be applicable in all member states of the European Union. Proposed changes include:

- Applicability of the law for all non-EU companies without any establishment in the EU, provided that the processing of data is directed at EU residents.
- An obligatory opt-in consent mechanism, so that any processing of personal data will require clear information to be provided to concerned individuals as well as specific and explicit consent to be obtained from such individuals for the processing of their data.
- Making a safe transfer of data outside of the EU (including data in clouds) easier in the event that the parties involved commit themselves to binding corporate rules.
- New privacy rights, including data subject’s “right of portability” and the “right to be forgotten,” will be established in the EU. The former right will allow a transfer of all data from one provider to another upon request, for example, transfer of a social media profile, whereas the ‘right to be forgotten’ will allow people to erase the historical data.
- The processing of data of individuals under the age of 13 will normally require parental consent, which will make it more difficult for companies to conduct business aimed at minors.
- Companies must notify EU data protection authorities as well as the individuals whose data are concerned by any breaches of data protection regulations or data leaks within 24 hours of discovering the breach.
- Harsh sanctions where breaches of the unified EU data protection law occur, with penalties of up to 2% of a company’s worldwide turnover for severe data protection breaches.

21.2.5 The United Kingdom’s Data Protection Act of 1998

The intent of the United Kingdom’s Data Protection Act (DPA) of 1998 (OPSI, 1998) is to uphold eight data protection principles that are outlined in Table 21.1. These principles were borrowed from the 1995 EU Directive on Data Protection. Under this act, citizens have the right to request to inspect copies of data that

TABLE 21.1 U.K. Data Protection Act 1998 (OPSI, 1998)

(1)	Personal data shall be processed fairly and lawfully and, in particular, shall not be processed unless it is consented to or "necessary." The conditions under which processing is considered necessary are explicitly listed in Schedule 2 and Schedule 3 of the Act.
(2)	Personal data shall be obtained only for one or more specified and lawful purposes, and shall not be further processed in any manner incompatible with that purpose or those purposes.
(3)	Personal data shall be adequate, relevant, and not excessive in relation to the purpose or purposes for which they are processed.
(4)	Personal data shall be accurate and, where necessary, kept up to date.
(5)	Personal data processed for any purpose or purposes shall not be kept for longer than is necessary for that purpose or those purposes.
(6)	Personal data shall be processed in accordance with the rights of data subjects under this Act.
(7)	Appropriate technical and organizational measures shall be taken against unauthorized or unlawful processing of personal data and against accidental loss or destruction of, or damage to, personal data.
(8)	Personal data shall not be transferred to a country or territory outside the European Economic Area unless that country or territory ensures an adequate level of protection for the rights and freedoms of data subjects in relation to the processing of personal data.

any organization keeps about them and to request inaccuracies to be corrected. Organizations that collect and maintain such data must have clear policies regarding how to respond to requests to inspect data as well as requests to share data with other organizations. Such policies clearly need to be consistent with the law as well as the ethical standards of the organization.

21.2.6 Access to Information Laws

According to RighttoInfo.org, the organization that monitors and lobbies Right to Information laws, as of January 2012 over 90 countries had nationwide laws providing the right of, and procedures for, public access to government held information. The first country to establish an access to information law was Sweden in 1766, followed by Finland in 1951, the US in 1966 and the remaining countries to date over the following 50 years. The overarching driving force behind these laws similar to the DPA is to increase openness and accountability in public authorities. For some countries, an additional driving force is to reduce corruption and encourage formal practice or legal behaviors.

The UK established the Freedom of Information Act 2000 and Freedom of Information (Scotland) Act in 2002 (FOISA). This provides any member of the public the right to access the wealth of information held by public authorities, from hospitals and emergency services to local authorities, central government, and higher education institutions. The act has eight parts with several provisions within them and overlaps with the DPA 1998. The first two that provide the overview of the law are described in Table 21.2.

TABLE 21.2 Excerpt from UK Freedom of Information Act, 2000**Part 1 – Access to Information Held by Public Authorities**

- Provides for the general right of access to recorded information held by public authorities and specifies the conditions which need to be fulfilled before an authority is obliged to comply with a request;
- Describes the effect of the exemptions in Part II on the obligations under section 1;
- Provides for the Act to cover the bodies, persons, or office holders specified in Schedule 1 and publicly owned companies and includes a power to specify further public authorities for the purpose of the Act;
- Allows public authorities to charge fees in accordance with regulations made by the Secretary of State;
- Provides for time limits for complying with a request;
- Makes special provision relating to public records transferred to the Public Record Office, etc.;
- Requires public authorities to provide advice and assistance to applicants;
- Requires public authorities to state the basis for refusal of a request;
- Renames the Data Protection Commissioner and Data Protection Tribunal (with consequential amendments to other legislation being made in Schedule 2);
- Requires public authorities to adopt and maintain a publication scheme and to publish information in accordance with it.

Part 2 – Exemption Information

- Sets out the circumstances in which information is 'exempt information'.

Part 4 – Enforcement

- Enables an applicant who is not satisfied with the response by a public authority to a request for information to apply to the Commissioner for a decision on whether the authority has acted in accordance with the provisions of the Act. Subject to certain conditions, for example, the exhaustion of other means of complaint, the Commissioner is under a duty to reach a decision.
- Describes the investigative and enforcement powers of the Commissioner. It confirms that the Act does not give rise to any right of action against public authorities for breach of statutory duty. This part also provides for the circumstances in which a certificate may be issued by an accountable person in respect of a decision notice or enforcement notice issued by the Commissioner in respect of the disclosure of exempt information. The effect of such a certificate is that a public authority need not comply with the Commissioner's notice.

The UK Freedom of Information laws are enforced by the Information Commissioner and they make recommendations on a model publication scheme that the relevant organizations adopt or work to comply with. Depending on the request, and how accessible the information is from databases and records, compliance to these laws and their deadlines typically have potential administrative cost implications to an organization so, as a result, many choose to implement policies and procedures and provide training to staff as an integral part of their governance structures. This is often handled by a department and/or individual primarily responsible for legal compliance, information or data management, or a department also handling DPA requests. Where requests reflect a larger administrative threshold cost to extract, a decision is made as to whether to charge the requestor. However, in order to reduce some of the administration and costs to comply with

and FOIA or FOISA, many organizations include within their policies procedures for voluntarily publishing non-exempt information regularly, often to a Web site, accessible to the public or internally. Additionally, as a result, implications for recording, storing and managing data across the organization, in such a way that it can be made accessible at short notice with relatively low costs to comply with the Information Commissioner's deadlines, are also taken into consideration in the decision making structures, information, and data management policies.

21.2.7 International Banking—Basel II Accords

The *International Convergence of Capital Measurement and Capital Standards*, otherwise known as “Basel II,” is a 2004 revision to the 1998 Basel Capital Accord (Basel I) produced by the Basel Committee on Banking Supervision (BCBS). These are recommended policies that must be enacted into law in each individual country and monitored by the appropriate national regulators. In the U.S. this is the Federal Reserve Bank for the 10 largest internationally active U.S. banks, and the SEC for securities firms. These two regulators were created to level the playing field among globally competitive institutions and to set standards to minimize systemic risk in the world financial system. Institutions in the international banking system are interconnected in a number of ways through agreements, loans, and other credit and debt obligations. The fear has been that the failure of one large firm could cause defaults in countries and organizations far removed from the failing institution.

The Basel II framework consists of three main “pillars”:

- (1) *Minimum capital requirements.* Institutions must maintain sufficient capital (funds) given the level of risk inherent in their portfolio of assets (loans, securities, and so on). The measurement of risk has been revised and expanded to include:
 - (a) Credit risk—the risk that creditors will not be able to repay their debts (principle and interest).
 - (b) Market risk—the risk that all investments will decline in value as the entire market (or economy) declines, or due to industry- or firm-specific causes, including interest rate and currency risks.
 - (c) Interest rate risk—the risk that investment will lose value as a result of interest rate increases.
 - (d) Operational risk—the risk of loss as a result of poor internal controls, operations, systems, or human resources, or the risk of loss as a result of some external event (such as a natural disaster).
- (2) *Supervisory review process.* Management must understand and actively control the risks, have sufficient internal risk controls and timely reporting, including compensation plans that reward appropriate risk management behavior.
- (3) *Market discipline.* Institutions must publicly disclose information about their capital adequacy, risk exposures, and the processes by which they measure and mitigate risks.

In order to calculate market risks, firms must aggregate positions from all trading, loans (credit card, cars, homes, business, and so on), and financial operations at least daily and for trading in real time. Value at risk calculations need historical

data of one or two years or more for each asset to be able to calculate the variance—covariance matrices required by risk models, including Monte Carlo simulations. Credit risk models use external sources such as Standard & Poor's credit rating for public companies and large liquid instruments, but banks must still maintain significant data to execute their internal risk models. This includes credit histories, business financial data, loan officer reports, and so on. Assessing operational risk is even more data-intensive, as Basel II requires at least five years of data. Operational risk assessment requires analysis of high-frequency but low-value events and, more critically, high-value infrequent events, for which there is little statistical evidence upon which to base capital requirements. In the United States, and Europe, consortiums are being established where banks are sharing operational risk events so that each member has a base upon which to develop their internal models of operational risk. As with the Sarbanes-Oxley legislation, having effective internal controls in place also plays a large role in mitigating operation risk.

Following the financial crisis of 2008–9, BCBS significantly revised its existing capital adequacy guidelines, creating Basel III. The objective was “*to strengthen global capital and liquidity rules with the goal of promoting a more resilient banking sector.*” The G20 (finance ministers and central bank governors for 20 major economies) endorsed Basel III at the November 2010 summit in Seoul. While not all details have been finalized, the core principles have been agreed and a 2019 deadline has been established for institutions to complete implementation. As with Basel II, the quality of the data and meta-data financial institutions hold and exchange will be of paramount importance in implementing Basel III.

21.3 Establishing a Culture of Legal and Ethical Data Stewardship

The complexity and IT implications of the recent legislation discussed in the previous section raise issues that are vital to employees throughout an organization. Senior managers such as board members, presidents, chief information officers (CIOs), and data administrators are increasingly finding themselves liable for any violations of these laws. It is therefore mandatory that official policies be created and articulated to employees at all organizational levels. An obvious question arises: “Where do we start?” Some basic steps are outlined next.

21.3.1 Developing an Organization-Wide Policy for Legal and Ethical Behavior

First, it is important that the senior management team is aware of new legislation and changes in industry practice. An assessment of how these changes affect the organization is also a critical first step. Often issues will arise as a result of the growing interconnected nature of global business. For example, a firm may find that it is subject to the stricter laws of a foreign country in which it does business, and as a result may have to adjust its entire operations according to the most stringent standard.

Next, data administrators and CIOs need to assess how legislation affects the flow of data through the organization. Special attention must be paid to how data

is collected, stored, secured, and accessed by users who may be internal or external to the organization. Many of the security techniques discussed in Chapter 20 can be applied in this case. Following this, new or revised operating procedures must be documented and communicated to all affected parties. Again, an assessment may reveal additional employees or trading partners involved in working with sensitive data or processes that were previously overlooked.

Once explicit rules for conducting business within legal parameters have been developed, a similar set of ethical principles for the business should be developed. Often organizations already have a corporate-wide statement of ethics that can be used as a starting point. As already noted, any resulting policies must be documented and articulated to all employees in such a way that they come to understand the seriousness with which senior management takes these issues.

Finally, lapses in legal and ethical behavior must be dealt with swiftly and fairly, and within the guidelines made known to all employees. Such incidences can also serve to help refine policies and procedures going forward such that legal and ethical policy statements evolve over time to adapt to new business situations.

Another potential source of general guidelines can be an existing Codes of Ethics, Codes of Conduct, and/or Codes of Practice document that has been adopted by a professional IT-related society or organization. The next section discusses two such codes documents.

21.3.2 Professional Organizations and Codes of Ethics

Many professional organizations have a code of ethics that all members pledge to uphold. Perhaps the most comprehensive code of ethics for IT comes from the Association for Computing Machinery (ACM), an organization in existence since 1947 with more than 80,000 members worldwide (www.acm.org). The ACM Code of Ethics and Professional Conduct (ACM, 1992) consists of 24 statements of personal responsibility in four main categories:

- *Fundamental ethical considerations.* This category addresses eight areas:
 - Contribute to society and human well-being.
 - Avoid harm to others.
 - Be honest and trustworthy.
 - Be fair and take action not to discriminate.
 - Honor property rights, including copyrights and patent.
 - Give proper credit for intellectual property.
 - Respect the privacy of others.
 - Honor confidentiality.
- *Specific considerations of professional conduct.* This category addresses eight areas:
 - Strive to achieve the highest quality, effectiveness, and dignity in both the process and products of professional work.
 - Acquire and maintain professional competence.
 - Know and respect existing laws pertaining to professional work.
 - Accept and provide appropriate professional review.

- Give comprehensive and thorough evaluations of computer systems and their impacts, including analysis of possible risks.
- Honor contracts, agreements, and assigned responsibilities.
- Improve public understanding of computing and its consequences.
- Access computing and communication resources only when authorized to do so.
- *Considerations for individuals in leadership roles.* This category covers six areas:
 - Articulate social responsibilities of members of an organizational unit and encourage full acceptance of those responsibilities.
 - Manage personnel and resources to design and build information systems that enhance the quality of working life.
 - Acknowledge and support proper and authorized uses of an organization's computing and communication resources.
 - Ensure that users and those who will be affected by a system have their needs clearly articulated during the assessment and design of requirements; later, the system must be validated to meet requirements.
 - Articulate and support policies that protect the dignity of users and others affected by a computing system.
 - Create opportunities for members of the organization to learn the principles and limitations of computer systems.
- *Compliance with the code.* This final category addresses two main points:
 - Uphold and promote the principles of this code.
 - Treat violations of this code as inconsistent with membership in the ACM.

The British Computer Society (www.bcs.org) was founded in 1957 and currently has more than 50,000 members in 100 countries. The 2011 BCS Code of Conduct (BCS, 2011), which all BCS members agree to uphold, specifies conduct in four main areas:

(1) **Public Interest**

- (a) You shall have due regard for public health, privacy, security, and well-being of others and the environment.
- (b) You shall have due regard for the legitimate rights of third parties.
- (c) You shall conduct your professional activities without discrimination on the grounds of sex, sexual orientation, marital status, nationality, color, race, ethnic origin, religion, age or disability, or of any other condition or requirement.
- (d) You shall promote equal access to the benefits of IT and seek to promote the inclusion of all sectors in society wherever opportunities arise.

(2) **Professional Competence and Integrity**

- (a) You shall only undertake to do work or provide a service that is within your professional competence.
- (b) You shall **NOT** claim any level of competence that you do not possess.
- (c) You shall develop your professional knowledge, skills, and competence on a continuing basis, maintaining awareness of technological developments, procedures, and standards that are relevant to your field.

- (d) You shall ensure that you have the knowledge and understanding of Legislation and that you comply with such Legislation, in carrying out your professional responsibilities.
 - (e) You shall respect and value alternative viewpoints and seek, accept, and offer honest criticisms of work.
 - (f) You shall avoid injuring others, their property, reputation, or employment by false or malicious or negligent action or inaction.
 - (g) You shall reject and will not make any offer of bribery or unethical inducement.
- (3) **Duty to Relevant Authority**
- (a) You shall carry out your professional responsibilities with due care and diligence in accordance with the Relevant Authority's requirements while exercising your professional judgement at all times.
 - (b) You shall seek to avoid any situation that may give rise to a conflict of interest between you and your Relevant Authority.
 - (c) You shall accept professional responsibility for your work and for the work of colleagues who are defined in a given context as working under your supervision.
 - (d) You shall **NOT** disclose or authorize to be disclosed, or use for personal gain or to benefit a third party, confidential information except with the permission of your Relevant Authority, or as required by Legislation.
 - (e) You shall **NOT** misrepresent or withhold information on the performance of products, systems, or services (unless lawfully bound by a duty of confidentiality not to disclose such information), or take advantage of the lack of relevant knowledge or inexperience of others.
- (4) **Duty to the Profession**
- (a) You shall accept your personal duty to uphold the reputation of the profession and not take any action which could bring the profession into disrepute.
 - (b) You shall seek to improve professional standards through participation in their development, use, and enforcement.
 - (c) You shall uphold the reputation and good standing of BCS, the Chartered Institute for IT.
 - (d) You shall act with integrity and respect in your professional relationships with all members of BCS and with members of other professions with whom you work in a professional capacity.
 - (e) You shall notify BCS if convicted of a criminal offence or upon becoming bankrupt or disqualified as a Company Director and in each case give details of the relevant jurisdiction.
 - (f) You shall encourage and support fellow members in their professional development.

The ACM Code and BCS Code are similar in that both begin by establishing grounding in providing an overall benefit to society. From that point, performing one's professional duties to the highest possible standard and carrying out duties in a legal and ethical manner are paramount. Recognition of intellectual property rights (discussed next) and acknowledgement of sources, respecting privacy and confidentiality, and overall concern for public health, safety, and the environment

are also common themes. Both codes explicitly mention a member's duty to understand and comply with all relevant laws, regulations, and standards—something that is highly relevant, as discussed in this chapter. Both codes also mention duties to one's superiors as well as to the public at large.

It should not be surprising that the two major computer societies in the United States and United Kingdom share much common ground due to their common language, and general common ground regarding law and ethics. However, not all countries share the same societal values as the U.S. and U.K. Therefore, we can find situations in several countries where concepts such as an individual's right to privacy and antidiscrimination are not consistent with U.S. and U.K. norms.

These existing codes and others as cited by Lee (2006) can be used as a resource for organizations wishing to establish their own similar codes.

21.3.3 Developing an Organization-Wide Policy for Legal and Ethical Behavior for *DreamHome*



In this section, we outline the steps the *DreamHome* property rentals company might take to develop an organization-wide policy that addresses legal and ethical behavior. As a business, *DreamHome* interacts with private and business property owners, clients who wish to or who already rent property, and other organizations such as newspaper businesses on a daily basis. Some of the data *DreamHome* maintains, such as the amount a client pays to rent a particular property, could be considered sensitive information. In a similar fashion, a client's rental history and payment information are also considered very sensitive. *DreamHome*'s policy should therefore explicitly address:

- *Interactions between DreamHome staff and clients and business partners such as owners.* Critical points include:
 - Treating clients with respect (e.g., in email and over the phone).
 - Treating business partners with respect.
 - Taking special care to limit information disclosure to business partners, including the proper procedures for handling information requests.
- *The security of clients and other business data.* Critical points include:
 - Raising awareness of the sensitivity of a client's personal data, payment history, credit card numbers, and rental history.
 - Ensuring appropriate security measures are maintained to protect these sensitive data.
 - Proper procedures for handling data requests from:
 - internal employees (e.g., proposals for data mining or accessing sensitive client data);
 - owners (e.g., request to reset a password);
 - business partners (if such data sharing is allowed);
 - and possibly law enforcement (e.g., a request for a client's payment information or rental history).

- The use of company resources (hardware, software, Internet, and so on). Critical points include:
 - Computer hardware may not be removed from the premises without branch manager's approval.
 - Licensed computer software may not be copied, distributed, or otherwise used improperly.
 - Additional software may not be installed without IT department approval.
 - Internet resources may not be used for noncompany business.
- *Ramifications for violating the security and/or trust of clients and business partner data.* Critical points include:
 - All violations will be documented and presented before an oversight board consisting of representatives from different business areas as well as management levels. Serious violations will also be reported to appropriate authorities.
 - Willful or malicious violations will be met with dismissal and prosecution.
 - Other violations will result in sanctions commensurate with the seriousness of the violation as determined by the oversight board.



Finally, *DreamHome* should further establish a procedure by which the policy is reviewed annually and/or in the wake of any major violations of the policy or other incidents to ensure that the policy does not become outdated as the technology and business environment changes.

21.4 Intellectual Property

In this final section, we introduce some of the main concepts underlying **intellectual property** (sometimes referred to by the simple acronym **IP**). It is important that data and database administrators as well as business analysts and software developers recognize and understand the issues surrounding IP, to ensure both that their ideas can be protected and that other people's rights are not infringed. We start with a definition.

Intellectual property

The product of human creativity in the industrial, scientific, literary, and artistic fields.

Intellectual property includes inventions, inventive ideas, designs, patents and patent applications, discoveries, improvements, trademarks, designs and design rights (registered and unregistered), written work (including computer software), and know-how devised, developed, or written by an individual or set of individuals. IP generated through the course of employment legally belongs to the employer unless specifically agreed otherwise. In the same way that ownership of tangible products gives the owner rights, the ownership of intangible property attempts to provide similar rights to allow owners the rights of exclusivity to give away, license, or sell their intellectual property. Although the exclusive nature of intellectual property rights can seem strong, the strength of IP laws are tempered by limitations placed on their duration and/or scope, as well as the owner's freedom not to enforce their rights.

We can distinguish two types of IP:

Background IP	IP that exists before an activity takes place.
Foreground IP	IP that is generated during an activity.

A project may make use of background IP owned by someone other than the organization but in this case relevant contractual arrangements should be put in place with the owner of the IP. There are three main ways to protect **IP rights** (or **IPR**): patents, copyright, and trademarks, as we now discuss.

21.4.1 Patent

Patent

Provides an exclusive (legal) right for a set period of time to make, use, sell, or import an invention.

Patents are granted by a government when an individual or organization can demonstrate:

- the invention is *new*;
- the invention is in some way *useful*;
- the invention involves an *inventive step*.

In addition, one of the key considerations of the patent system is that the patent application must disclose how the invention works. This information is then disseminated to the public once a patent is issued, thereby increasing the wealth of public knowledge. Patents give effective protection for new technology that will lead to a product, composition, or process with significant long-term commercial gain. Note, however, that artistic creations, mathematical models, plans, schemes, or other purely mental processes cannot be patented.

21.4.2 Copyright

Copyright

Provides an exclusive (legal) right for a set period of time to reproduce and distribute a literary, musical, audiovisual, or other “work” of authorship.

Unlike a patent where rights are granted through a formal application process, copyright comes into effect as soon as what is “created” takes a fixed form (for example, in writing or in sound). Copyright covers not just work like books, articles, song lyrics, music CDs, videos, DVDs, and TV programs, but also computer software, databases, technical drawings and designs, and multimedia. Copyright holders can sell the rights to their works to individuals or organizations in return for payment, which are often referred to as *royalties*. There are some exceptions to copyright, so that some minor uses may not infringe copyright (for example, limited use for noncommercial research, private study, and teaching purposes).

Copyright also gives *moral rights* to be identified as the creator of certain kinds of material and to object to distortion or mutilation of it. Although copyright does

not require registration many countries allow for registration of works, for example, to identify titles of works or to serve as *prima facie* evidence in a court of law when copyright is infringed or disputed.

21.4.3 Trademark

Trademark

Provides an exclusive (legal) right to use a word, symbol, image, sound, or some other distinctive (in connection with certain goods or services) element that identifies the source of origin.

A third form of protection is the trademark. Generally, trademarks are intended to be associated with specific goods and services and as a result they assist consumers in identifying the nature and quality of the products they purchase. Like patents and copyright, a trademark gives the owner exclusive legal rights to use, license, or sell the goods and services for which it is registered. Like copyright, a trademark does not have to be registered, although registration may be advisable, as it can be expensive and time-consuming to take action under common law. For the *DreamHome* property rentals case study, the company may decide to trademark their business name.



21.4.4 Intellectual Property Rights Issues for Software

As noted earlier, it is important to understand IPR for a number of reasons:

- to understand your own right or your organization's right as a producer of original ideas and works;
- to recognize the value of original works;
- to understand the procedures for protecting and exploiting such work;
- to know the legal measures that can be used to defend against the illegal use of such work;
- to be fair and sensible about legitimate use of your work for nonprofit purposes.

In this section, we briefly discuss some of the issues related specifically to IPR and software.

Software and patentability

In the 1970s and 1980s, there were extensive discussions on whether patents or copyright should provide protection for computer software. These discussions resulted in a generally accepted principle that software should be protected by copyright, whereas apparatus using software should be protected by patent. However, this is less clear nowadays. Although the U.K. specifically excludes software from patent protection, there has been some latitude in the interpretation where software forms part of the overall machinery or industrial process. Therefore, an application to just patent a piece of software will be refused, but an application to patent some technical effect that is produced by a piece of software will be considered, subject to constraints discussed in Section 21.5.1. In the United States, patentability has been extended to cover what are termed "business methods" and many software patents

have been granted and more have been applied for, particularly with the growth of the Internet, mobile operating systems, applications, and e-commerce.

The introduction of smartphones and tablet computers in recent years has opened up a competitive arena for new innovations and dominance of operating systems and devices that has given rise to fierce and strategic battles by large multinational companies like Google, Apple, Samsung, Microsoft, Motorola, and HTC over their patents. Some of these disputes and applications have been controversial because, in addition to the objectives of protecting the innovation, the resultant court battles have been seen as aimed at tactically preventing or delaying sales of devices in certain regions and therefore market share. This represents just one example where the law faces new challenges in adapting to changes and innovations that are taking place in the environment to ensure legal and ethical behaviors in organizations when it comes to intellectual property.

Software and copyright

All software has one or more authors who assert the right to their intellectual property in what they have written. Copyright applies therefore to all software whether or not you have paid money for it, and the distribution and use of software is subject to a “license” that specifies the terms of use. The conditions that apply to a particular piece of software depend on a number of things, but in general there are four types of license:

- *Commercial software (perpetual use)*. In this case a fee is paid for the software and the license allows the software to be used for as long as you like (generally on one machine) and to make copies only for the purpose of backup if something goes wrong with the machine. Software can be transferred to a different machine, but it *must* be deleted from the old one first. In some cases a license may permit use on more than one machine, but this would be explicit in the license terms.
- *Commercial software (annual fee)*. This is similar to the perpetual use license, but a fee may be required for each year of continued use, and in most cases the software stops working unless the fee is paid and a new “license key” is issued by the supplier. Annual rental often applies to site licenses (in which once the fee is paid, the organization may use the software on as many machines as it likes) and to software on mainframe or server computers. Again, the license terms will be explicit as to what use is allowed.
- *Shareware*. Software made available initially for a free “trial” period. If after the initial period (for example, 30 days) we wish to continue using the software, we are asked to send a (usually small) fee to the author(s) of the software. In some cases the software enforces this by refusing to work after the trial period, but irrespective of this, by using the software you are accepting the license terms and are infringing the author’s copyright if we continue to use the software after the trial period. In return for the fee, a more up-to-date version of the software may be provided that does not constantly remind the user to register and pay for the software.
- *Freeware*. Software made available free for certain categories of use (such as education or personal use). There are two main types of freeware: software that is distributed without the source code preventing modification by users and open source software (OSS). The latter is usually issued under a license such as the GNU Public License (GPL) that specifies the terms and conditions of free

use. The main restrictions are that the software cannot be used for commercial purposes, although we are usually allowed to modify the software but are duty-bound to submit any improvements that we make to the author(s) so that they can incorporate them in future releases. A second restriction is that the text of the copyrighted GPL license itself be included with any redistribution.

Note that in neither of the first two cases are we permitted to attempt to modify or reverse-engineer the software or remove any copyright messages and the like. All software has license conditions, even software downloaded free from the Internet, and failure to comply with the license conditions is an infringement of copyright.

21.4.5 Intellectual Property Rights Issues for Data



Consideration must also be paid to data that an organization collects, processes, and possibly shares with its trading partners. In conjunction with senior management and legal counsel, data administrators must define and enforce policies that govern when data can be shared and in what ways it can be used within the organization. For example, consider the data that *DreamHome* maintains on its clients' rental habits. It is entirely possible that other retailers, target marketing firms, or even law enforcement would be interested in gaining access to detailed transaction histories of clients. For some businesses, sharing limited data (such as purchase patterns without revealing individual identities) or aggregated data may make sense from a revenue standpoint. In the event that a business case is made for sharing data, appropriate licensing of the data must be put into effect so that it is not "reshared" with other parties.

Although not exclusively a cloud computing problem, with its increasing use and transnational data storage, the ability to have complete control over this issue of access to personal or corporate IP has increased as a concern. For example, some governments have established laws that provide them with unrestricted access to corporate or individual private data. The USA Patriot Act 2001, established as a direct response to the terrorist attacks of September 11th, forces U.S. companies with local or foreign servers or companies with data controlled by a U.S. company, to release data on individuals—even if their local laws (for example EU) prevent them from doing so. Since a U.S. company and its subsidiaries operating in Europe are still subject to the U.S. Patriot Act, European customers that use their services are exposing themselves to U.S. laws. The U.S. and EU Swiss Safe Harbour framework was established to avoid this situation, however, there have been reports from some US companies operating in Europe that have admitted that this has proved ineffective.

Chapter Summary

- Recent failures of well-known organizations have lead to increased scrutiny of organizations. This scrutiny has been formalized in a number of acts of legislation in the United States and elsewhere.
- **Ethics** as defined as "doing what is right" within a given society or culture.

- **IT Governance** is used for specifying the decision rights and accountability framework to encourage desirable behavior in the use of IT.
- What constitutes legal behavior is most often aligned with ethical behavior, although this is not always the case.
- Most of the legislation discussed in this chapter was put into place to help diminish the possibility of unintended information disclosure.
- Most of the legislative acts discussed have at their core a mandate to protect the data of clients while at the same time increasing the reporting requirements of companies to official agencies. Both of these general items have at their core data management issues.
- **Internal controls** are a set of measures that an organization adopts to ensure that policies and procedures are not violated, data is properly secured and reliable, and operations can be carried out efficiently.
- Establishing a corporate-wide (and certainly IT-wide) awareness of security, privacy, and reporting as it relates to the data an organization collects and processes is a critical task, especially given the current regulatory environment.
- **Intellectual property (IP)** includes inventions, inventive ideas, designs, patents and patent applications, discoveries, improvements, trademarks, designs and design rights, written work, and know-how devised, developed, or written by an individual or set of individuals.
- **Background intellectual property** is IP that already exists before an activity takes place.
- **Foreground intellectual property** is IP that is generated during an activity.
- **Patent** provides an exclusive (legal) right for a set period of time to make, use, sell, or import an invention.
- **Copyright** provides an exclusive (legal) right for a set period of time to reproduce and distribute a literary, musical, audiovisual, or other “work” of authorship.
- **Trademark** provides an exclusive (legal) right to use a word, symbol, image, sound, or some other distinction element that identifies the source of origin in connection with certain goods or services another make, use, sell, or import an invention.



Review Questions

- 21.1 Explain the motives behind organizational ethics and legislations.
- 21.2 Describe business situations in which an individual's or businesses' behavior would be considered:
 - (a) illegal and unethical;
 - (b) legal but unethical;
 - (c) illegal but ethical.
- 21.3 Explain the role of the organizational code of conduct in relation to local and international legislation.
- 21.4 Describe the importance of IT governance and its relationship to legal and ethical practice in organizations.
- 21.5 Explain how an international business can be influenced by laws from a foreign land and how it may face liability from its customers for reasons beyond its control.
- 21.6 Explain the legal risks to an international business in data management.
- 21.7 Describe how an entity or individuals outside and within organizations can have legitimate power to access corporate data and, therefore, influence how it is managed.
- 21.8 Describe some of the legal challenges faced in protecting Intellectual property and promoting innovation with emerging technology.

Exercises



- 21.9 Suppose that you are a data administrator in a large European pharmaceutical manufacturer that has significant sales and marketing efforts in Europe, Japan, and the United States. What data management issues would you have to be most concerned with?
- 21.10 Suppose that you have just joined a large financial services company as the head of IT and are asked to create a formal code of ethics for IT. What steps would you take to research this task, and what resources would you consider?
- 21.11 Access Peter Neumann's "Inside Risk" article archives for the Communications of the ACM (visit www.csl.sri.com/users/neumann/insiderisks.html). Summarize, in a few paragraphs, a recent article from these archives dealing with legal and/or ethical issues related to IT.
- 21.12 Access the ACM Code of Ethics and Professional Conduct and the BCS Code of Conduct and Code of Good Practice. When comparing the two, discuss elements that are emphasized more (or less) in one code than another.
- 21.13 Access the Singapore Computer Society's Code of Conduct. (www.scs.org.sg/code_of_conduct.php). Compare this code with either the ACM or BCS code and note any differences.
- 21.14 Consider the *DreamHome* case study described in Chapter 11. Produce a report for the Director of the company outlining the legal and ethical issues that need to be considered and make any recommendations that you think are appropriate.
- 21.15 Consider the case studies described in Appendix B. Produce a report for each case study outlining the legal and ethical issues that need to be considered and make any recommendations you think appropriate.
- 21.16 Suppose you are the Chief Information Officer for a public University in Europe. What data management issues would you have to be concerned with and what governance policies and procedures would you have to consider putting in place to ensure you comply with legal standards?
- 21.17 Describe the IT governance mechanisms that could be implemented in your organization to improve how IT and data is managed.
- 21.18 Describe with examples how legal frameworks in place to support transnational management of data can prove ineffective in protecting the privacy of business or consumers.
- 21.19 Most of the legislations described in this chapter were triggered by a problematic situation facing the society. Analyze the situations in your country and identify problems that, in one way or another, forced the introduction of the available known legislation.
- 21.20 IT governance plays a vital role in ensuring accountability and decision rights. Proper management and use of IT resources helps organizations meet their objectives. Critically analyze how IT governance mechanisms are applied in an organization of your choice.

Chapter Objectives

In this chapter you will learn:

- The purpose of concurrency control.
- The purpose of database recovery.
- The function and importance of transactions.
- The properties of a transaction.
- The meaning of serializability and how it applies to concurrency control.
- How locks can be used to ensure serializability.
- How the two-phase locking (2PL) protocol works.
- The meaning of deadlock and how it can be resolved.
- How timestamps can be used to ensure serializability.
- How optimistic concurrency control techniques work.
- How different levels of locking may affect concurrency.
- Some causes of database failure.
- The purpose of the transaction log file.
- The purpose of checkpoints during transaction logging.
- How to recover following database failure.
- Alternative models for long duration transactions.
- How Oracle handles concurrency control and recovery.

In Chapter 2 we discussed the functions that a database management system (DBMS) should provide. Among these are three closely related functions that are intended to ensure that the database is reliable and remains in a consistent state: transaction support, concurrency control services, and recovery services. This reliability and consistency must be maintained in the presence of failures of both hardware and software components, and when multiple users are accessing the database. In this chapter we concentrate on these three functions.

Although each function can be discussed separately, they are mutually dependent. Both concurrency control and recovery are required to protect the database from data inconsistencies and data loss. Many DBMSs allow users to undertake simultaneous operations on the database. If these operations are not controlled, the accesses may interfere with one another and the database can become inconsistent. To overcome this, the DBMS implements a **concurrency control** protocol that prevents database accesses from interfering with one another.

Database recovery is the process of restoring the database to a correct state following a failure. The failure may be the result of a system crash due to hardware or software errors, a media failure, such as a head crash, or a software error in the application, such as a logical error in the program that is accessing the database. It may also be the result of unintentional or intentional corruption or destruction of data or facilities by system administrators or users. Whatever the underlying cause of the failure, the DBMS must be able to recover from the failure and restore the database to a consistent state.

Structure of this Chapter Central to an understanding of both concurrency control and recovery is the notion of a **transaction**, which we describe in Section 22.1. In Section 22.2 we discuss concurrency control and examine the protocols that can be used to prevent conflict. In Section 22.3 we discuss database recovery and examine the techniques that can be used to ensure the database remains in a consistent state in the presence of failures. In Section 22.4 we examine more advanced transaction models that have been proposed for transactions that are of a long duration (from hours to possibly even months) and have uncertain developments, so that some actions cannot be foreseen at the beginning. In Section 22.5 we examine how Oracle handles concurrency control and recovery.

In this chapter we consider transaction support, concurrency control, and recovery for a centralized DBMS, that is, a DBMS that consists of a single database. Later, in Chapter 25, we consider these services for a distributed DBMS, that is, a DBMS that consists of multiple logically related databases distributed across a network.

22.1 Transaction Support

Transaction

An action, or series of actions, carried out by a single user or application program, that reads or updates the contents of the database.

A transaction is treated as a **logical unit of work** on the database. It may be an entire program, a part of a program, or a single statement (for example, the SQL statement INSERT or UPDATE), and it may involve any number of operations on the

<pre> read(staffNo = x, salary) salary = salary * 1.1 write(staffNo = x, salary) </pre>	<pre> delete(staffNo = x) for all PropertyForRent records, pno begin read(propertyNo = pno, staffNo) if (staffNo = x) then begin staffNo = newStaffNo write(propertyNo = pno, staffNo) end end end </pre>
(a)	(b)

Figure 22.1 Example transactions.

database. In the database context, the execution of an application program can be thought of as one or more transactions with nondatabase processing taking place in between. To illustrate the concepts of a transaction, we examine two relations from the instance of the *DreamHome* rental database shown in Figure 4.3:

Staff	(staffNo , fName, lName, position, sex, DOB, salary, branchNo)
PropertyForRent	(propertyNo , street, city, postcode, type, rooms, rent, ownerNo, staffNo , branchNo)



A simple transaction against this database is to update the salary of a particular member of staff given the staff number, x . At a high level, we could write this transaction as shown in Figure 22.1(a). In this chapter we denote a database read or write operation on a data item x as $\text{read}(x)$ or $\text{write}(x)$. Additional qualifiers may be added as necessary; for example, in Figure 22.1(a), we have used the notation $\text{read}(\text{staffNo} = x, \text{salary})$ to indicate that we want to read the data item **salary** for the tuple with primary key value x . In this example, we have a **transaction** consisting of two database operations (read and write) and a nondatabase operation ($\text{salary} = \text{salary} * 1.1$).

A more complicated transaction is to delete the member of staff with a given staff number x , as shown in Figure 22.1(b). In this case, as well as having to delete the tuple in the **Staff** relation, we also need to find all the **PropertyForRent** tuples that this member of staff managed and reassign them to a different member of staff, **newStaffNo** say. If all these updates are not made, referential integrity will be lost and the database will be in an **inconsistent state**: a property will be managed by a member of staff who no longer exists in the database.

A transaction should always transform the database from one consistent state to another, although we accept that consistency may be violated while the transaction is in progress. For example, during the transaction in Figure 22.1(b), there may be some moment when one tuple of **PropertyForRent** contains the new **newStaffNo** value and another still contains the old one, x . However, at the end of the transaction, all necessary tuples should have the new **newStaffNo** value.

A transaction can have one of two outcomes. If it completes successfully, the transaction is said to have **committed** and the database reaches a new consistent state. On the other hand, if the transaction does not execute successfully, the

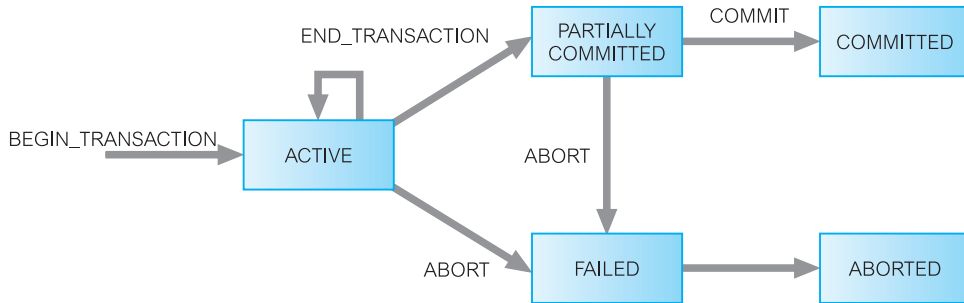


Figure 22.2 State transition diagram for a transaction.

transaction is **aborted**. If a transaction is aborted, the database must be restored to the consistent state it was in before the transaction started. Such a transaction is **rolled back** or **undone**. A committed transaction cannot be aborted. If we decide that the committed transaction was a mistake, we must perform another **compensating transaction** to reverse its effects (as we discuss in Section 22.4.2). However, an aborted transaction that is rolled back can be restarted later and, depending on the cause of the failure, may successfully execute and commit at that time.

The DBMS has no inherent way of knowing which updates are grouped together to form a single logical transaction. It must therefore provide a method to allow the user to indicate the boundaries of a transaction. The keywords `BEGIN TRANSACTION`, `COMMIT`, and `ROLLBACK` (or their equivalent[†]) are available in many data manipulation languages to delimit transactions. If these delimiters are not used, the entire program is usually regarded as a single transaction, with the DBMS automatically performing a `COMMIT` when the program terminates correctly and a `ROLLBACK` if it does not.

Figure 22.2 shows the state transition diagram for a transaction. Note that in addition to the obvious states of `ACTIVE`, `COMMITTED`, and `ABORTED`, there are two other states:

- **PARTIALLY COMMITTED**, which occurs after the final statement has been executed. At this point, it may be found that the transaction has violated serializability (see Section 22.2.2) or has violated an integrity constraint and the transaction has to be aborted. Alternatively, the system may fail and any data updated by the transaction may not have been safely recorded on secondary storage. In such cases, the transaction would go into the `FAILED` state and would have to be aborted. If the transaction has been successful, any updates can be safely recorded and the transaction can go to the `COMMITTED` state.
- **FAILED**, which occurs if the transaction cannot be committed or the transaction is aborted while in the `ACTIVE` state, perhaps due to the user aborting the

[†]With the ISO SQL standard, `BEGIN TRANSACTION` is implied by the first *transaction-initiating* SQL statement (see Section 7.5).

transaction or as a result of the concurrency control protocol aborting the transaction to ensure serializability.

22.1.1 Properties of Transactions

There are properties that all transactions should possess. The four basic, or so-called **ACID**, properties that define a transaction are (Haerder and Reuter, 1983):

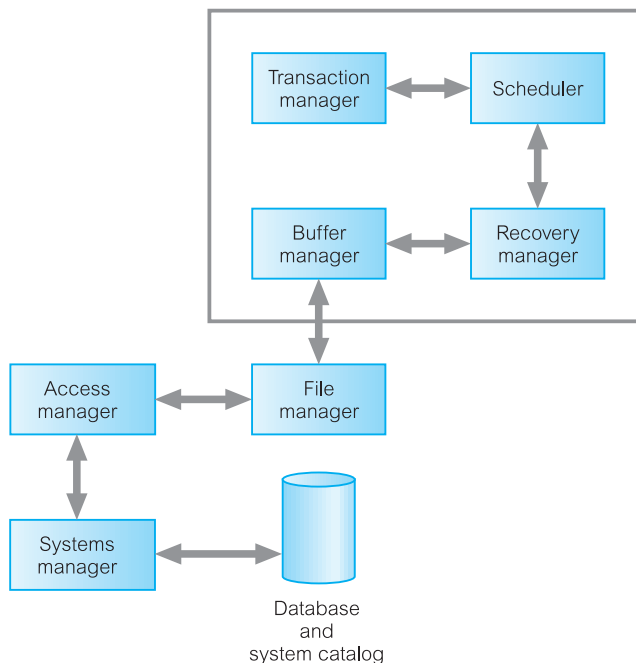
- *Atomicity*. The “all or nothing” property. A transaction is an indivisible unit that is either performed in its entirety or is not performed at all. It is the responsibility of the recovery subsystem of the DBMS to ensure atomicity.
- *Consistency*. A transaction must transform the database from one consistent state to another consistent state. It is the responsibility of both the DBMS and the application developers to ensure consistency. The DBMS can ensure consistency by enforcing all the constraints that have been specified on the database schema, such as integrity constraints. However, in itself this is insufficient to ensure consistency. For example, suppose that we have a transaction that is intended to transfer money from one bank account to another and the programmer makes an error in the transaction logic and debits one account but credits the wrong account; then the database is in an inconsistent state. However, the DBMS would not have been responsible for introducing this inconsistency and would have had no ability to detect the error.
- *Isolation*. Transactions execute independently of one another. In other words, the partial effects of incomplete transactions should not be visible to other transactions. It is the responsibility of the concurrency control subsystem to ensure isolation.
- *Durability*. The effects of a successfully completed (committed) transaction are permanently recorded in the database and must not be lost because of a subsequent failure. It is the responsibility of the recovery subsystem to ensure durability.

22.1.2 Database Architecture

In Chapter 3 we presented an architecture for a DBMS. Figure 22.3 represents an extract from Figure 3.21 identifying four high-level database modules that handle transactions, concurrency control, and recovery. The **transaction manager** coordinates transactions on behalf of application programs. It communicates with the **scheduler**, the module responsible for implementing a particular strategy for concurrency control. The scheduler is sometimes referred to as the **lock manager** if the concurrency control protocol is locking-based. The objective of the scheduler is to maximize concurrency without allowing concurrently executing transactions to interfere with one another, and so compromise the integrity or consistency of the database.

If a failure occurs during the transaction, then the database could be inconsistent. It is the task of the **recovery manager** to ensure that the database is restored to the state it was in before the start of the transaction, and therefore a consistent state. Finally, the **buffer manager** is responsible for the efficient transfer of data between disk storage and main memory.

Figure 22.3
DBMS
transaction
subsystem.



22.2 Concurrency Control

In this section we examine the problems that can arise with concurrent access and the techniques that can be employed to avoid these problems. We start with the following working definition of concurrency control.

Concurrency control

The process of managing simultaneous operations on the database without having them interfere with one another.

22.2.1 The Need for Concurrency Control

A major objective in developing a database is to enable many users to access shared data concurrently. Concurrent access is relatively easy if all users are only reading data, as there is no way that they can interfere with one another. However, when two or more users are accessing the database simultaneously and at least one is updating data, there may be interference that can result in inconsistencies.

This objective is similar to the objective of multi-user computer systems, which allow two or more programs (or transactions) to execute at the same time. For example, many systems have input/output (I/O) subsystems that can handle I/O operations independently, while the main central processing unit (CPU) performs other operations. Such systems can allow two or more transactions to execute simultaneously. The system begins executing the first transaction until it reaches an I/O operation. While the I/O is being performed, the CPU suspends the first transaction and executes commands from the second transaction. When the second transaction reaches an I/O operation, control then returns to the first transaction and its operations are resumed from the point at which it was suspended.

The first transaction continues until it again reaches another I/O operation. In this way, the operations of the two transactions are **interleaved** to achieve concurrent execution. In addition, **throughput**—the amount of work that is accomplished in a given time interval—is improved as the CPU is executing other transactions instead of being in an idle state waiting for I/O operations to complete.

However, although two transactions may be perfectly correct in themselves, the interleaving of operations in this way may produce an incorrect result, thus compromising the integrity and consistency of the database. We examine three examples of potential problems caused by concurrency: the **lost update problem**, the **uncommitted dependency problem**, and the **inconsistent analysis problem**. To illustrate these problems, we use a simple bank account relation that contains the *DreamHome* staff account balances. In this context, we are using the transaction as the *unit of concurrency control*.



EXAMPLE 22.1 The lost update problem

An apparently successfully completed update operation by one user can be overridden by another user. This is known as the **lost update problem** and is illustrated in Figure 22.4, in which transaction T_1 is executing concurrently with transaction T_2 . T_1 is withdrawing £10 from an account with balance bal_x , initially £100, and T_2 is depositing £100 into the same account. If these transactions are executed **serially**, one after the other with no interleaving of operations, the final balance would be £190 no matter which transaction is performed first.

Transactions T_1 and T_2 start at nearly the same time, and both read the balance as £100. T_2 increases bal_x by £100 to £200 and stores the update in the database. Meanwhile, transaction T_1 decrements its copy of bal_x by £10 to £90 and stores this value in the database, overwriting the previous update, and thereby “losing” the £100 previously added to the balance.

The loss of T_2 ’s update is avoided by preventing T_1 from reading the value of bal_x until after T_2 ’s update has been completed.

Time	T_1	T_2	bal_x
t_1		begin_transaction	100
t_2	begin_transaction	read(bal_x)	100
t_3	read(bal_x)	$bal_x = bal_x + 100$	100
t_4	$bal_x = bal_x - 10$	write(bal_x)	200
t_5	write(bal_x)	commit	90
t_6	commit		90

Figure 22.4 The lost update problem.

EXAMPLE 22.2 The uncommitted dependency (or dirty read) problem

The uncommitted dependency problem occurs when one transaction is allowed to see the intermediate results of another transaction before it has committed. Figure 22.5 shows an example of an uncommitted dependency that causes an error, using the same initial value for balance bal_x as in the previous example. Here, transaction T_4 updates bal_x to £200, but it aborts the transaction so that bal_x should be restored to its original value of £100. However, by this time transaction T_3 has read the new value of bal_x (£200)

Time	T ₃	T ₄	bal _x
t ₁		begin_transaction	100
t ₂		read(bal _x)	100
t ₃		bal _x = bal _x + 100	100
t ₄	begin_transaction	write(bal _x)	200
t ₅	read(bal _x)	:	200
t ₆	bal _x = bal _x - 10	rollback	100
t ₇	write(bal _x)		190
t ₈	commit		190

Figure 22.5 The uncommitted dependency problem.

and is using this value as the basis of the £10 reduction, giving a new incorrect balance of £190, instead of £90. The value of bal_x read by T₃ is called *dirty data*, giving rise to the alternative name, *the dirty read problem*.

The reason for the rollback is unimportant; it may be that the transaction was in error, perhaps crediting the wrong account. The effect is the assumption by T₃ that T₄'s update completed successfully, although the update was subsequently rolled back. This problem is avoided by preventing T₃ from reading bal_x until after the decision has been made to either commit or abort T₄'s effects.

The two problems in these examples concentrate on transactions that are updating the database and their interference may corrupt the database. However, transactions that only read the database can also produce inaccurate results if they are allowed to read partial results of incomplete transactions that are simultaneously updating the database. We illustrate this with the next example.

EXAMPLE 22.3 The inconsistent analysis problem

The problem of inconsistent analysis occurs when a transaction reads several values from the database but a second transaction updates some of them during the execution of the first. For example, a transaction that is summarizing data in a database (for example, totaling balances) will obtain inaccurate results if, while it is executing, other transactions are updating the database. One example is illustrated in Figure 22.6, in which a

Time	T ₅	T ₆	bal _x	bal _y	bal _z	sum
t ₁		begin_transaction	100	50	25	
t ₂	begin_transaction	sum = 0	100	50	25	0
t ₃	read(bal _x)	read(bal _x)	100	50	25	0
t ₄	bal _x = bal _x - 10	sum = sum + bal _x	100	50	25	100
t ₅	write(bal _x)	read(bal _y)	90	50	25	100
t ₆	read(bal _z)	sum = sum + bal _y	90	50	25	150
t ₇	bal _z = bal _z + 10		90	50	25	150
t ₈	write(bal _z)		90	50	35	150
t ₉	commit	read(bal _z)	90	50	35	150
t ₁₀		sum = sum + bal _z	90	50	35	185
t ₁₁		commit	90	50	35	185

Figure 22.6 The inconsistent analysis problem.

summary transaction T_6 is executing concurrently with transaction T_5 . Transaction T_6 is totaling the balances of account x (£100), account y (£50), and account z (£25). However, in the meantime, transaction T_5 has transferred £10 from bal_x to bal_z , so that T_6 now has the wrong result (£10 too high). This problem is avoided by preventing transaction T_6 from reading bal_x and bal_z until after T_5 has completed its updates.

Another problem can occur when a transaction T rereads a data item it has previously read but, in between, another transaction has modified it. Thus, T receives two different values for the same data item. This is sometimes referred to as a **nonrepeatable** (or **fuzzy**) **read**. A similar problem can occur if transaction T executes a query that retrieves a set of tuples from a relation satisfying a certain predicate, re-executes the query at a later time, but finds that the retrieved set contains an additional (**phantom**) tuple that has been inserted by another transaction in the meantime. This is sometimes referred to as a **phantom read**.

Before we discuss the main concurrency control techniques, we discuss some key concepts associated with serializability and recoverability of transactions.

22.2.2 Serializability and Recoverability

The objective of a concurrency control protocol is to schedule transactions in such a way as to avoid any interference between them and hence prevent the types of problem described in the previous section. One obvious solution is to allow only one transaction to execute at a time: one transaction is *committed* before the next transaction is allowed to *begin*. However, the aim of a multi-user DBMS is also to maximize the degree of concurrency or parallelism in the system, so that transactions that can execute without interfering with one another can run in parallel. For example, transactions that access different parts of the database can be scheduled together without interference. In this section, we examine serializability as a means of helping to identify those executions of transactions that are *guaranteed* to ensure consistency (Papadimitriou, 1979). First, we give some definitions.

Schedule

A sequence of the operations by a set of concurrent transactions that preserves the order of the operations in each of the individual transactions.

A transaction comprises a sequence of operations consisting of read and/or write actions to the database, followed by a commit or abort action. A schedule S consists of a sequence of the operations from a set of n transactions $T_1, T_2, \dots, T_n$, subject to the constraint that the order of operations for each transaction is preserved in the schedule. Thus, for each transaction T_i in schedule S , the order of the operations in T_i must be the same in schedule S .

Serial schedule

A schedule where the operations of each transaction are executed consecutively without any interleaved operations from other transactions.

In a serial schedule, the transactions are performed in serial order. For example, if we have two transactions T_1 and T_2 , serial order would be T_1 followed by T_2 , or T_2 followed by T_1 . Thus, in serial execution there is no interference between transactions, because only one is executing at any given time. However, there is no guarantee that the results of all serial executions of a given set of transactions will be identical. In banking, for example, it matters whether interest is calculated on an account before a large deposit is made or after.

Nonserial schedule

A schedule where the operations from a set of concurrent transactions are interleaved.

The problems described in Examples 22.1–22.3 resulted from the mismanagement of concurrency, which left the database in an inconsistent state in the first two examples and presented the user with the wrong result in the third. Serial execution prevents such problems from occurring. No matter which serial schedule is chosen, serial execution never leaves the database in an inconsistent state, so *every serial execution is considered correct*, although different results may be produced. The objective of **serializability** is to find nonserial schedules that allow transactions to execute concurrently without interfering with one another, and thereby produce a database state that could be produced by a serial execution.

If a set of transactions executes concurrently, we say that *the (nonserial) schedule is correct if it produces the same result as some serial execution*. Such a schedule is called **serializable**. To prevent inconsistency from transactions interfering with one another, it is essential to guarantee serializability of concurrent transactions. In serializability, the ordering of read and write operations is important:

- If two transactions only read a data item, they do not conflict and order is not important.
- If two transactions either read or write completely separate data items, they do not conflict and order is not important.
- If one transaction writes a data item and another either reads or writes the same data item, the order of execution is important.

Consider the schedule S_1 shown in Figure 22.7(a) containing operations from two concurrently executing transactions T_7 and T_8 . Because the write operation on bal_x in T_8 does not conflict with the subsequent read operation on bal_y in T_7 , we can change the order of these operations to produce the equivalent schedule S_2 shown in Figure 22.7(b). If we also now change the order of the following non-conflicting operations, we produce the equivalent serial schedule S_3 shown in Figure 22.7(c):

- Change the order of the $write(bal_x)$ of T_8 with the $write(bal_y)$ of T_7 .
- Change the order of the $read(bal_x)$ of T_8 with the $read(bal_y)$ of T_7 .
- Change the order of the $read(bal_x)$ of T_8 with the $write(bal_y)$ of T_7 .

Schedule S_3 is a serial schedule and, because S_1 and S_2 are equivalent to S_3 , S_1 and S_2 are serializable schedules.

Time	T ₇	T ₈	T ₇	T ₈	T ₇	T ₈
t ₁	begin_transaction		begin_transaction		begin_transaction	
t ₂	read(bal_x)		read(bal_x)		read(bal_x)	
t ₃	write(bal_x)		write(bal_x)		write(bal_x)	
t ₄		begin_transaction		begin_transaction		begin_transaction
t ₅		read(bal_x)		read(bal_x)		read(bal_x)
t ₆		write(bal_x)	read(bal_y)		commit	
t ₇	read(bal_y)			write(bal_x)		
t ₈	write(bal_y)		write(bal_y)			begin_transaction
t ₉	commit		commit			read(bal_x)
t ₁₀		read(bal_y)		read(bal_y)		write(bal_x)
t ₁₁		write(bal_y)		write(bal_y)		read(bal_y)
t ₁₂		commit		commit		write(bal_y)
						commit

(a) Schedule S₁(b) Schedule S₂(c) Schedule S₃

Figure 22.7 Equivalent schedules: (a) nonserial schedule S₁; (b) nonserial schedule S₂ equivalent to S₁; (c) serial schedule S₃, equivalent to S₁ and S₂.

This type of serializability is known as **conflict serializability**. A conflict serializable schedule orders any conflicting operations in the same way as some serial execution.

Testing for conflict serializability Under the **constrained write rule** (that is, a transaction updates a data item based on its old value, which is first read by the transaction), a **precedence** (or **serialization**) **graph** can be produced to test for conflict serializability. For a schedule S, a precedence graph is a directed graph $G = (N, E)$ that consists of a set of nodes N and a set of directed edges E, which is constructed as follows:

- Create a node for each transaction.
- Create a directed edge $T_i \rightarrow T_j$, if T_j reads the value of an item written by T_i .
- Create a directed edge $T_i \rightarrow T_j$, if T_j writes a value into an item after it has been read by T_i .
- Create a directed edge $T_i \rightarrow T_j$, if T_j writes a value into an item after it has been written by T_i .

If an edge $T_i \rightarrow T_j$ exists in the precedence graph for S, then in any serial schedule S' equivalent to S, T_i must appear before T_j . If the precedence graph contains a cycle, the schedule is not conflict serializable.

EXAMPLE 22.4 Nonconflict serializable schedule

Consider the two transactions shown in Figure 22.8. Transaction T₉ is transferring £100 from one account with balance **bal_x** to another account with balance **bal_y**, while T₁₀ is increasing the balance of these two accounts by 10%. The precedence graph for this schedule, shown in Figure 22.9, has a cycle and so is not conflict serializable.

Time	T ₉	T ₁₀
t ₁	begin_transaction	
t ₂	read(bal_x)	
t ₃	bal_x = bal_x + 100	
t ₄	write(bal_x)	begin_transaction
t ₅		read(bal_x)
t ₆		bal_x = bal_x * 1.1
t ₇		write(bal_x)
t ₈		read(bal_y)
t ₉		bal_y = bal_y * 1.1
t ₁₀		write(bal_y)
t ₁₁	read(bal_y)	commit
t ₁₂	bal_y = bal_y - 100	
t ₁₃	write(bal_y)	
t ₁₄	commit	

Figure 22.8 Two concurrent update transactions that are not conflict serializable.

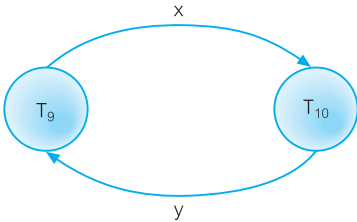


Figure 22.9 Precedence graph for Figure 22.8 showing a cycle, so schedule is not conflict serializable.

View serializability

There are several other types of serializability that offer less stringent definitions of schedule equivalence than that offered by conflict serializability. One less restrictive definition is called **view serializability**. Two schedules S_1 and S_2 consisting of the same operations from n transactions $T_1, T_2, \dots, T_n$ are view equivalent if the following three conditions hold:

- For each data item x , if transaction T_i reads the initial value of x in schedule S_1 , then transaction T_i must also read the initial value of x in schedule S_2 .
- For each read operation on data item x by transaction T_i in schedule S_1 , if the value read by T_i has been written by transaction T_j , then transaction T_i must also read the value of x produced by transaction T_j in schedule S_2 .
- For each data item x , if the last write operation on x was performed by transaction T_i in schedule S_1 , the same transaction must perform the final write on data item x in schedule S_2 .

A schedule is view serializable if it is view equivalent to a serial schedule. Every conflict serializable schedule is view serializable, although the converse is not true. For example, the schedule shown in Figure 22.10 is view serializable, although it is not conflict serializable. In this example, transactions T_{12} and T_{13} do not conform

Time	T ₁₁	T ₁₂	T ₁₃
t ₁	begin_transaction		
t ₂	read(bal _x)		
t ₃		begin_transaction	
t ₄		write(bal _x)	
t ₅		commit	
t ₆	write(bal _x)		
t ₇	commit		
t ₈			begin_transaction
t ₉			write(bal _x)
t ₁₀			commit

Figure 22.10 View serializable schedule that is not conflict serializable.

to the constrained write rule; in other words, they perform *blind writes*. It can be shown that any view serializable schedule that is not conflict serializable contains one or more blind writes.

Testing for view serializability Testing for view serializability is much more complex than testing for conflict serializability. In fact, it has been shown that testing for view serializability is NP-complete; thus, it is highly improbable that an efficient algorithm can be found (Papadimitriou, 1979). If we produce a (conflict serializable) precedence graph corresponding to the schedule given in Figure 22.10 we would get the graph shown in Figure 22.11. This graph contains a cycle indicating that the schedule is not conflict serializable; however, we know that it is view serializable, as it is equivalent to the serial schedule T₁₁ followed by T₁₂ followed by T₁₃. When we examine the rules for the precedence graph given previously, we can see that the edge T₁₂ → T₁₁ should not have been inserted into the graph, as the values of bal_x written by T₁₁ and T₁₂ were never used by any other transaction because of the blind writes.

As a result, to test for view serializability we need a method to decide whether an edge should be inserted into the precedence graph. The approach we take is to construct a *labeled precedence graph* for the schedule as follows:

- (1) Create a node for each transaction.
- (2) Create a node labeled T_{bw}. T_{bw} is a dummy transaction inserted at the beginning of the schedule containing a write operation for each data item accessed in the schedule.

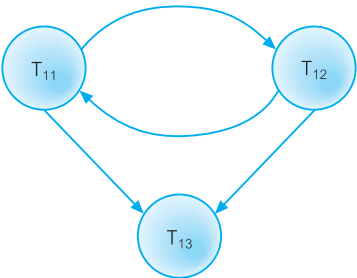


Figure 22.11
Precedence graph for the view serializable schedule of Figure 22.10.

- (3) Create a node labeled T_{fr} . T_{fr} is a dummy transaction added at the end of the schedule containing a read operation for each data item accessed in the schedule.
- (4) Create a directed edge $T_i \xrightarrow{0} T_j$ if T_j reads the value of an item written by T_i .
- (5) Remove all directed edges incident on transaction T_i for which there is no path from T_i to T_{fr} .
- (6) For each data item that T_j reads that has been written by T_i , and T_k writes ($T_k \neq T_{bw}$), then:
 - (a) If $T_i = T_{bw}$ and $T_j \neq T_{fr}$, then create a directed edge $T_i \xrightarrow{0} T_k$.
 - (b) If $T_i \neq T_{bw}$ and $T_j = T_{fr}$, then create a directed edge $T_k \xrightarrow{0} T_i$.
 - (c) If $T_i \neq T_{bw}$ and $T_j \neq T_{fr}$, then create a pair of directed edge $T_k \xrightarrow{x} T_i$, and $T_j \xrightarrow{x} T_k$, where x is a unique positive integer that has not been used for labeling an earlier directed edge. This rule is a more general case of the preceding two rules, indicating that if transaction T_i writes an item that T_j subsequently reads, then any transaction, T_k , that writes the same item must *either* precede T_i or succeed T_j .

Applying the first five rules to the schedule in Figure 22.10 produces the precedence graph shown in Figure 22.12(a). Applying rule 6(a), we add the edges $T_{11} \rightarrow T_{12}$ and $T_{11} \rightarrow T_{13}$, both labeled 0; applying rule 6(b), we add the edges $T_{11} \rightarrow T_{13}$ (which is already present) and $T_{12} \rightarrow T_{13}$, again both labeled 0. The final graph is shown in Figure 22.12(b).

Based on this labeled precedence graph, the test for view serializability is as follows:

- (1) If the graph contains no cycles, the schedule is view serializable.
- (2) The presence of a cycle, however, is not a sufficient condition to conclude that the schedule is not view serializable. The actual test is based on the observation that rule 6(c) generates m distinct directed edge pairs, resulting in 2^m different graphs containing just one edge from each pair. If *any* one of these graphs is acyclic, then the corresponding schedule is view serializable and the serializability order is determined by the topological sorting of the graph with the dummy transactions T_{bw} and T_{fr} removed.

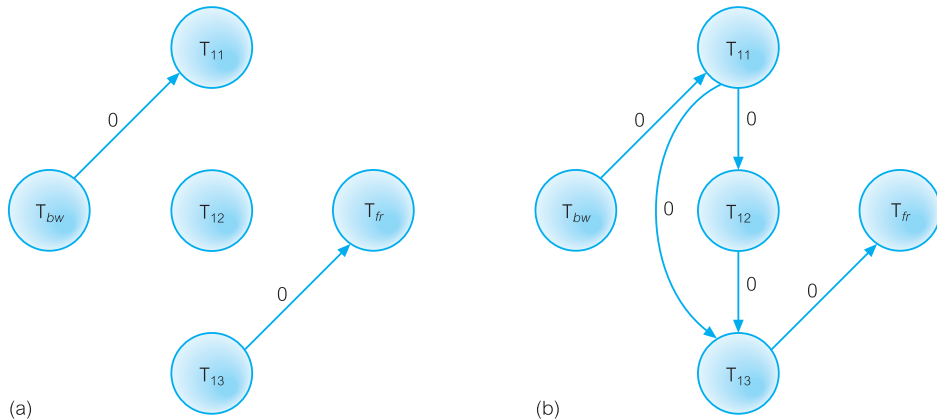


Figure 22.12 Labeled precedence graph for the view serializable schedule of Figure 22.10.

Time	T_{11}	T_{12}	T_{13A}
t_1	begin_transaction		
t_2	read(bal_x)		
t_3		begin_transaction	
t_4		write(bal_x)	
t_5		commit	
t_6			begin_transaction
t_7			read(bal_x)
t_8	write(bal_x)		
t_9	commit		
t_{10}			write(bal_x)
t_{11}			commit

Figure 22.13

A modified version of the schedule in Figure 22.10 containing an additional read operation.

Applying these tests to the graph in Figure 22.12(b), which is acyclic, we conclude that the schedule is view serializable. As another example, consider the slightly modified variant of the schedule of Figure 22.10 shown in Figure 22.13 containing an additional read operation in transaction T_{13A} . Applying the first five rules to this schedule produces the precedence graph shown in Figure 22.14(a). Applying rule 6(a), we add the edges $T_{11} \rightarrow T_{12}$ and $T_{11} \rightarrow T_{13A}$, both labeled 0; applying rule 6(b), we add the edges $T_{11} \rightarrow T_{13A}$ (which is already present) and $T_{12} \rightarrow T_{13A}$ (again already present), both labeled 0. Applying rule 6(c), we add the pair of edges $T_{11} \rightarrow T_{12}$ and $T_{13A} \rightarrow T_{11}$, this time both labeled 1. The final graph is shown in Figure 22.14(b). From this we can produce two different graphs containing only one edge, as shown in Figures 22.14(c) and 20.14(d). As Figure 22.14(c) is acyclic, we can conclude that this schedule is also view serializable (corresponding to the serial schedule $T_{11} \rightarrow T_{12} \rightarrow T_{13A}$).

In practice, a DBMS does not test for the serializability of a schedule. This would be impractical, as the interleaving of operations from concurrent transactions is determined by the operating system. Instead, the approach taken is to use protocols that are known to produce serializable schedules. We discuss such protocols in the next section.

Recoverability

Serializability identifies schedules that maintain the consistency of the database, assuming that none of the transactions in the schedule fails. An alternative perspective examines the *recoverability* of transactions within a schedule. If a transaction fails, the atomicity property requires that we undo the effects of the transaction. In addition, the durability property states that once a transaction commits, its changes cannot be undone (without running another, compensating, transaction). Consider again the two transactions shown in Figure 22.8, but instead of the commit operation at the end of transaction T_9 , assume that T_9 decides to roll back the effects of the transaction. T_{10} has read the update to bal_x performed by T_9 , and has itself updated bal_x and committed the change. Strictly speaking, we should undo transaction T_{10} , because it has used a value for bal_x that has been undone. However, the durability property does not allow this. In other words, this schedule is a *nonrecoverable schedule*, which should not be allowed. This leads to the definition of a recoverable schedule.

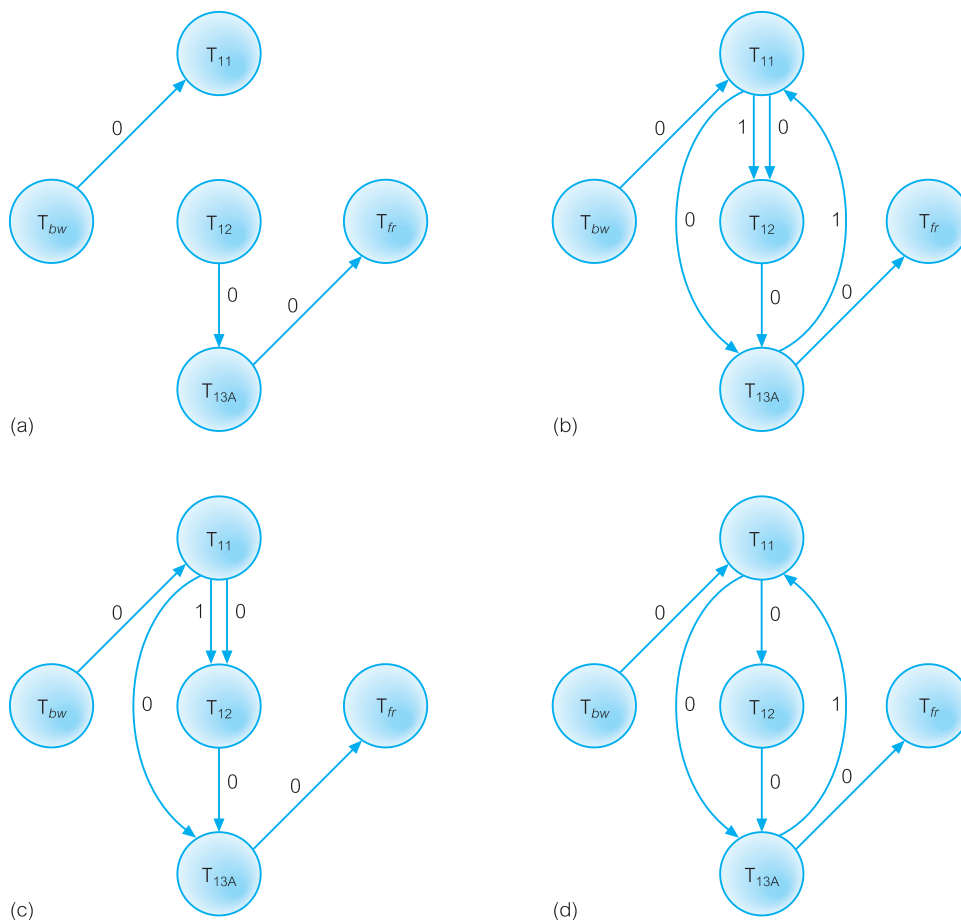


Figure 22.14 Labeled precedence graph for the view serializable schedule in Figure 22.13.

Recoverable schedule

A schedule in which for each pair of transactions T_i and T_j , if T_j reads a data item previously written by T_i , then the commit operation of T_i precedes the commit operation of T_j .

Concurrency control techniques

Serializability can be achieved in several ways; however, the two main concurrency control techniques that allow transactions to execute safely in parallel subject to certain constraints are called locking and timestamping.

Locking and timestamping are **conservative** (or **pessimistic**) approaches in that they cause transactions to be delayed in case they conflict with other transactions at some time in the future. **Optimistic** methods, as you will see later, are based on the premise that conflict is rare, so they allow transactions to proceed unsynchronized and check for conflicts only at the end, when a transaction commits. We discuss locking, timestamping, and optimistic concurrency control techniques in the following sections.

22.2.3 Locking Methods

Locking

A procedure used to control concurrent access to data. When one transaction is accessing the database, a lock may deny access to other transactions to prevent incorrect results.

Locking methods are the most widely used approach to ensure serializability of concurrent transactions. There are several variations, but all share the same fundamental characteristic, namely that a transaction must claim a **shared** (*read*) or **exclusive** (*write*) lock on a data item before the corresponding database read or write operation. The **lock** prevents another transaction from modifying the item or even reading it, in the case of an exclusive lock. The basic rules for locking are as follows.

Shared lock

If a transaction has a shared lock on a data item, it can read the item but not update it.

Exclusive lock

If a transaction has an exclusive lock on a data item, it can both read and update the item.

Data items of various sizes, ranging from the entire database down to a field, may be locked. The size of the item determines the fineness, or **granularity**, of the lock. The actual lock might be implemented by setting a bit in the data item to indicate that that portion of the database is locked, or by keeping a list of locked parts of the database, or by other means. We examine lock granularity further in Section 22.2.8. In the meantime, we continue to use the term “data item” to refer to the lock granularity.

Because read operations cannot conflict, it is permissible for more than one transaction to hold shared locks simultaneously on the same item. On the other hand, an exclusive lock gives a transaction exclusive access to that item. Thus, as long as a transaction holds the exclusive lock on the item, no other transactions can read or update that data item. Locks are used in the following way:

- Any transaction that needs to access a data item must first lock the item, requesting a shared lock for read-only access or an exclusive lock for both read and write access.
- If the item is not already locked by another transaction, the lock will be granted.
- If the item is currently locked, the DBMS determines whether the request is compatible with the existing lock. If a shared lock is requested on an item that already has a shared lock on it, the request will be granted; otherwise, the transaction must **wait** until the existing lock is released.
- A transaction continues to hold a lock until it explicitly releases it either during execution or when it terminates (aborts or commits). It is only when the exclusive lock has been released that the effects of the write operation will be made visible to other transactions.

In addition to these rules, some systems permit a transaction to issue a shared lock on an item and then later to **upgrade** the lock to an exclusive lock. This in effect allows a transaction to examine the data first and then decide whether it wishes to

update it. If upgrading is not supported, a transaction must hold exclusive locks on all data items that it may update at some time during the execution of the transaction, thereby potentially reducing the level of concurrency in the system. For the same reason, some systems also permit a transaction to issue an exclusive lock and then later to **downgrade** the lock to a shared lock.

Using locks in transactions, as described previously, does not guarantee serializability of schedules by themselves, as Example 22.5 shows.

EXAMPLE 22.5 Incorrect locking schedule

Consider again the two transactions shown in Figure 22.8. A valid schedule that may be employed using the previous locking rules is:

```
S = {write_lock(T9, balx), read(T9, balx), write(T9, balx), unlock(T9, balx),
      write_lock(T10, balx), read(T10, balx), write(T10, balx), unlock(T10, balx),
      write_lock(T10, baly), read(T10, baly), write(T10, baly), unlock(T10, baly),
      commit(T10), write_lock(T9, baly), read(T9, baly), write(T9, baly),
      unlock(T9, baly), commit(T9)}
```

If, prior to execution, $bal_x = 100$, $bal_y = 400$, the result should be $bal_x = 220$, $bal_y = 330$, if T_9 executes before T_{10} , or $bal_x = 210$ and $bal_y = 340$, if T_{10} executes before T_9 . However, the result of executing schedule S would give $bal_x = 220$ and $bal_y = 340$. (S is **not** a serializable schedule.)

The problem in this example is that the schedule releases the locks that are held by a transaction as soon as the associated read/write is executed and that lock item (say bal_x) no longer needs to be accessed. However, the transaction itself is locking other items (bal_y), after it releases its lock on bal_x . Although this may seem to allow greater concurrency, it permits transactions to interfere with one another, resulting in the loss of total isolation and atomicity.

To guarantee serializability, we must follow an additional protocol concerning the positioning of the lock and unlock operations in every transaction. The best-known protocol is **two-phase locking (2PL)**.

Two-phase locking (2PL)

2PL

A transaction follows the two-phase locking protocol if all locking operations precede the first unlock operation in the transaction.

According to the rules of this protocol, every transaction can be divided into two phases: first a **growing phase**, in which it acquires all the locks needed but cannot release any locks, and then a **shrinking phase**, in which it releases its locks but cannot acquire any new locks. There is no requirement that all locks be obtained simultaneously. Normally, the transaction acquires some locks, does some processing, and goes on to acquire additional locks as needed. However, it never releases any lock until it has reached a stage where no new locks are needed. The rules are:

- A transaction must acquire a lock on an item before operating on the item. The lock may be read or write, depending on the type of access needed.
- Once the transaction releases a lock, it can never acquire any new locks.

If upgrading of locks is allowed, upgrading can take place only during the growing phase and may require that the transaction wait until another transaction releases a shared lock on the item. Downgrading can take place only during the shrinking phase. We now look at how two-phase locking is used to resolve the three problems identified in Section 22.2.1.

EXAMPLE 22.6 Preventing the lost update problem using 2PL

A solution to the lost update problem is shown in Figure 22.15. To prevent the lost update problem occurring, T_2 first requests an exclusive lock on bal_x . It can then proceed to read the value of bal_x from the database, increment it by £100, and write the new value back to the database. When T_1 starts, it also requests an exclusive lock on bal_x . However, because the data item bal_x is currently exclusively locked by T_2 , the request is not immediately granted and T_1 has to **wait** until the lock is released by T_2 . This occurs only once the commit of T_2 has been completed.

Time	T_1	T_2	bal_x
t_1		begin_transaction	100
t_2	begin_transaction	write_lock(bal_x)	100
t_3	write_lock(bal_x)	read(bal_x)	100
t_4	WAIT	$bal_x = bal_x + 100$	100
t_5	WAIT	write(bal_x)	200
t_6	WAIT	commit/unlock(bal_x)	200
t_7	read(bal_x)		200
t_8	$bal_x = bal_x - 10$		200
t_9	write(bal_x)		190
t_{10}	commit/unlock(bal_x)		190

Figure 22.15 Preventing the lost update problem.

EXAMPLE 22.7 Preventing the uncommitted dependency problem using 2PL

A solution to the uncommitted dependency problem is shown in Figure 22.16. To prevent this problem occurring, T_4 first requests an exclusive lock on bal_x . It can then proceed to read the value of bal_x from the database, increment it by £100, and write the new value back to the database. When the rollback is executed, the updates of transaction T_4 are undone and the value of bal_x in the database is returned to its original value of £100. When T_3 starts, it also requests an exclusive lock on bal_x . However, because the data item bal_x is currently exclusively locked by T_4 , the request is not immediately granted and T_3 must wait until the lock is released by T_4 . This occurs only when the rollback of T_4 has been completed.

Time	T ₃	T ₄	bal _x
t ₁		begin_transaction	100
t ₂		write_lock(bal _x)	100
t ₃		read(bal _x)	100
t ₄	begin_transaction	bal _x = bal _x + 100	100
t ₅	write_lock(bal _x)	write(bal _x)	200
t ₆	WAIT	rollback/unlock(bal _x)	100
t ₇	read(bal _x)		100
t ₈	bal _x = bal _x - 10		100
t ₉	write(bal _x)		90
t ₁₀	commit/unlock(bal _x)		90

Figure 22.16 Preventing the uncommitted dependency problem.

EXAMPLE 22.8 Preventing the inconsistent analysis problem using 2PL

A solution to the inconsistent analysis problem is shown in Figure 22.17. To prevent this problem from occurring, T₅ must precede its reads by exclusive locks, and T₆ must precede its reads with shared locks. Therefore, when T₅ starts, it requests and obtains an exclusive lock on bal_x. Now, when T₆ tries to share lock bal_x, the request is not immediately granted and T₆ has to wait until the lock is released, which is when T₅ commits.

Time	T ₅	T ₆	bal _x	bal _y	bal _z	sum
t ₁		begin_transaction	100	50	25	
t ₂	begin_transaction	sum = 0	100	50	25	0
t ₃	write_lock(bal _x)		100	50	25	0
t ₄	read(bal _x)	read_lock(bal _x)	100	50	25	0
t ₅	bal _x = bal _x - 10	WAIT	100	50	25	0
t ₆	write(bal _x)	WAIT	90	50	25	0
t ₇	write_lock(bal _z)	WAIT	90	50	25	0
t ₈	read(bal _z)	WAIT	90	50	25	0
t ₉	bal _z = bal _z + 10	WAIT	90	50	25	0
t ₁₀	write(bal _z)	WAIT	90	50	35	0
t ₁₁	commit/unlock(bal _x , bal _z)	WAIT	90	50	35	0
t ₁₂		read(bal _x)	90	50	35	0
t ₁₃		sum = sum + bal _x	90	50	35	90
t ₁₄		read_lock(bal _y)	90	50	35	90
t ₁₅		read(bal _y)	90	50	35	90
t ₁₆		sum = sum + bal _y	90	50	35	140
t ₁₇		read_lock(bal _z)	90	50	35	140
t ₁₈		read(bal _z)	90	50	35	140
t ₁₉		sum = sum + bal _z	90	50	35	175
t ₂₀		commit/unlock(bal _x , bal _y , bal _z)	90	50	35	175

Figure 22.17 Preventing the inconsistent analysis problem.

It can be proved that if *every* transaction in a schedule follows the two-phase locking protocol, then the schedule is guaranteed to be conflict serializable (Eswaran *et al.*, 1976). However, although the two-phase locking protocol guarantees

serializability, problems can occur with the interpretation of when locks can be released, as the next example shows.

EXAMPLE 22.9 Cascading rollback

Consider a schedule consisting of the three transactions shown in Figure 22.18, which conforms to the two-phase locking protocol. Transaction T_{14} obtains an exclusive lock on bal_x and then updates it using bal_y , which has been obtained with a shared lock, and writes the value of bal_x back to the database before releasing the lock on bal_x . Transaction T_{15} then obtains an exclusive lock on bal_x , reads the value of bal_x from the database, updates it, and writes the new value back to the database before releasing the lock. Finally, T_{16} share locks bal_x and reads it from the database. By now, T_{14} has failed and has been rolled back. However, because T_{15} is dependent on T_{14} (it has read an item that has been updated by T_{14}), T_{15} must also be rolled back. Similarly, T_{16} is dependent on T_{15} , so it too must be rolled back. This situation, in which a single transaction leads to a series of rollbacks, is called **cascading rollback**.

Time	T_{14}	T_{15}	T_{16}
t_1	begin_transaction		
t_2	write_lock(bal_x)		
t_3	read(bal_x)		
t_4	read_lock(bal_y)		
t_5	read(bal_y)		
t_6	$bal_x = bal_y + bal_x$		
t_7	write(bal_x)		
t_8	unlock(bal_x)	begin_transaction	
t_9	:	write_lock(bal_x)	
t_{10}	:	read(bal_x)	
t_{11}	:	$bal_x = bal_x + 100$	
t_{12}	:	write(bal_x)	
t_{13}	:	unlock(bal_x)	
t_{14}	:	:	
t_{15}	rollback	:	
t_{16}		:	begin_transaction
t_{17}		:	read_lock(bal_x)
t_{18}		rollback	:
t_{19}			rollback

Figure 22.18 Cascading rollback with 2PL.

Cascading rollbacks are undesirable, because they potentially lead to the undoing of a significant amount of work. Clearly, it would be useful if we could design protocols that prevent cascading rollbacks. One way to achieve this with two-phase locking is to leave the release of *all* locks until the end of the transaction, as in the previous examples. In this way, the problem illustrated here would not occur, as T_{15} would not obtain its exclusive lock until after T_{14} had completed the rollback. This is called **rigorous 2PL**. It can be shown that with rigorous 2PL, transactions can be serialized in the order in which they commit. Another variant of 2PL, called **strict 2PL**, holds only *exclusive locks* until the end of the transaction. Most database systems implement one of these two variants of 2PL.

Another problem with two-phase locking, which applies to all locking-based schemes, is that it can cause **deadlock**, as transactions can wait for locks on data items. If two transactions wait for locks on items held by the other, deadlock will occur and the deadlock detection and recovery scheme described in the Section 22.2.4 will be needed. It is also possible for transactions to be in **livelock**, that is, left in a wait state indefinitely, unable to acquire any new locks, although the DBMS is not in deadlock. This can happen if the waiting algorithm for transactions is unfair and does not take account of the time that transactions have been waiting. To avoid livelock, a priority system can be used, whereby the longer a transaction has to wait, the higher its priority, for example, a *first-come-first-served* queue can be used for waiting transactions.

Concurrency control with index structures

Concurrency control for an index structure (see Appendix F) can be managed by treating each page of the index as a data item and applying the two-phase locking protocol described previously. However, because indexes are likely to be frequently accessed—particularly the higher levels of trees (as searching occurs from the root downwards)—this simple concurrency control strategy may lead to high lock contention. Therefore, a more efficient locking protocol is required for indexes. If we examine how tree-based indexes are traversed, we can make the following two observations:

- The search path starts from the root and moves down to the leaf nodes of the tree, but the search never moves back up the tree. Thus, once a lower-level node has been accessed, the higher-level nodes in that path will not be used again.
- When a new index value (a key and a pointer) is being inserted into a leaf node, then if the node is not full, the insertion will not cause changes to the higher-level nodes. This suggests that we have to exclusively lock only the leaf node in such a case, and exclusively lock higher-level nodes only if a node is full and must be split.

Based on these observations, we can derive the following locking strategy:

- For searches, obtain shared locks on nodes starting at the root and proceeding down wards along the required path. Release the lock on a (parent) node once a lock has been obtained on the child node.
- For insertions, a conservative approach would be to obtain exclusive locks on all nodes as we descend the tree to the leaf node to be modified. This ensures that a split in the leaf node can propagate all the way up the tree to the root. However, if a child node is not full, the lock on the parent node can be released. A more optimistic approach would be to obtain shared locks on all nodes as we descend to the leaf node to be modified, where we obtain an exclusive lock on the leaf node itself. If the leaf node has to split, we upgrade the shared lock on the parent node to an exclusive lock. If this node also has to split, we continue to upgrade the locks at the next higher level. In the majority of cases, a split is not required, making this a better approach.

The technique of locking a child node and releasing the lock on the parent node if possible is known as **lock-coupling** or **crabbing**. For further details on the performance of concurrency control algorithms for trees, the interested reader is referred to Srinivasan and Carey (1991).

Latches

DBMSs also support another type of lock called a **latch**, which is held for a much shorter duration than a normal lock. A latch can be used before a page is read from, or written to, disk to ensure that the operation is atomic. For example, a latch would be obtained to write a page from the database buffers to disk, the page would then be written to disk, and the latch immediately unset. As the latch is simply to prevent conflict for this type of access, latches do not need to conform to the normal concurrency control protocol such as two-phase locking.

22.2.4 Deadlock

Deadlock

An impasse that may result when two (or more) transactions are each waiting for locks to be released that are held by the other.

Figure 22.19 shows two transactions, T_{17} and T_{18} , that are deadlocked because each is waiting for the other to release a lock on an item it holds. At time t_2 , transaction T_{17} requests and obtains an exclusive lock on item bal_x and at time t_3 transaction T_{18} obtains an exclusive lock on item bal_y . Then at t_6 , T_{17} requests an exclusive lock on item bal_y . Because T_{18} holds a lock on bal_y , transaction T_{17} waits. Meanwhile, at time t_7 , T_{18} requests a lock on item bal_x , which is held by transaction T_{17} . Neither transaction can continue, because each is waiting for a lock it cannot obtain until the other completes. Once deadlock occurs, the applications involved cannot resolve the problem. Instead, the DBMS has to recognize that deadlock exists and break the deadlock in some way.

Unfortunately, there is only one way to break deadlock: abort one or more of the transactions. This usually involves undoing all the changes made by the aborted transaction(s). In Figure 22.19, we may decide to abort transaction T_{18} . Once this is complete, the locks held by transaction T_{18} are released and T_{17} is able to continue again. Deadlock should be transparent to the user, so the DBMS should automatically restart the aborted transaction(s). However, in practice the DBMS cannot restart the aborted transaction since it is unaware of the transaction logic even if it was aware of the transaction history (unless there is no user input in the transaction or the input is not a function of the database state).

Time	T_{17}	T_{18}
t_1	begin_transaction	
t_2	write_lock(bal_x)	begin_transaction
t_3	read(bal_x)	write_lock(bal_y)
t_4	$bal_x = bal_x - 10$	read(bal_y)
t_5	write(bal_x)	$bal_y = bal_y + 100$
t_6	write_lock(bal_y)	write(bal_y)
t_7	WAIT	write_lock(bal_x)
t_8	WAIT	WAIT
t_9	WAIT	WAIT
t_{10}	:	WAIT
t_{11}	:	:

Figure 22.19
Deadlock
between two
transactions.

There are three general techniques for handling deadlock: timeouts, deadlock prevention, and deadlock detection and recovery. With timeouts, the transaction that has requested a lock waits for at most a specified period of time. Using **deadlock prevention**, the DBMS looks ahead to determine if a transaction would cause deadlock, and never allows deadlock to occur. Using **deadlock detection and recovery**, the DBMS allows deadlock to occur but recognizes occurrences of deadlock and breaks them. Because it is more difficult to prevent deadlock than to use timeouts or test for deadlock and break it when it occurs, systems generally avoid the deadlock prevention method.

Timeouts

A simple approach to deadlock prevention is based on *lock timeouts*. In this approach, a transaction that requests a lock will wait for only a system-defined period of time. If the lock has not been granted within this period, the lock request times out. In this case, the DBMS assumes that the transaction may be deadlocked, even though it may not be, and it aborts and automatically restarts the transaction. This is a very simple and practical solution to deadlock prevention that is used by several commercial DBMSs.

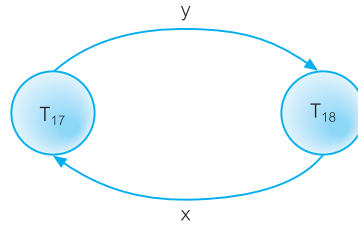
Deadlock prevention

Another possible approach to deadlock prevention is to order transactions using transaction timestamps, which we discuss in Section 22.2.5. Two algorithms have been proposed by Rosenkrantz *et al.* (1978). One algorithm, *Wait-Die*, allows only an older transaction to wait for a younger one, otherwise the transaction is aborted (*dies*) and restarted with the same timestamp, so that eventually it will become the oldest active transaction and will not die. The second algorithm, *Wound-Wait*, uses a symmetrical approach: only a younger transaction can wait for an older one. If an older transaction requests a lock held by a younger one, the younger one is aborted (*wounded*).

A variant of 2PL, called **conservative 2PL**, can also be used to prevent deadlock. In conservative 2PL, a transaction obtains all its locks when it begins, or it waits until all the locks are available. This protocol has the advantage that if lock contention is heavy, the time that locks are held is reduced because transactions are never blocked and therefore never have to wait for locks. On the other hand, if lock contention is low then locks are held longer under this protocol. Further, the overhead for setting locks is high, because all the locks must be obtained and released all at once. Thus, if a transaction fails to obtain one lock it must release all the current locks it has obtained and start the lock process again. From a practical perspective, a transaction may not know at the start which locks it may actually need and, therefore, may have to set more locks than is required. As a result, this protocol is not used in practice.

Deadlock detection

Deadlock detection is usually handled by the construction of a **wait-for graph (WFG)** that shows the transaction dependencies; that is, transaction T_i is dependent on T_j if transaction T_j holds the lock on a data item that T_i is waiting for. The WFG is a directed graph $G = (N, E)$ that consists of a set of nodes N and a set of directed edges E , which is constructed as follows:

**Figure 22.20**

WFG with a cycle showing deadlock between two transactions.

- Create a node for each transaction.
- Create a directed edge $T_i \rightarrow T_j$, if transaction T_i is waiting to lock an item that is currently locked by T_j .

Deadlock exists if and only if the WFG contains a cycle (Holt, 1972). Figure 22.20 shows the WFG for the transactions in Figure 22.19. Clearly, the graph has a cycle in it ($T_{17} \rightarrow T_{18} \rightarrow T_{17}$), so we can conclude that the system is in deadlock.

Frequency of deadlock detection

Because a cycle in the wait-for graph is a necessary and sufficient condition for deadlock to exist, the deadlock detection algorithm generates the WFG at regular intervals and examines it for a cycle. The choice of time interval between executions of the algorithm is important. If the interval chosen is too small, deadlock detection will add considerable overhead; if the interval is too large, deadlock may not be detected for a long period. Alternatively, a dynamic deadlock detection algorithm could start with an initial interval size. Each time no deadlock is detected, the detection interval could be increased, for example, to twice the previous interval, and each time deadlock is detected, the interval could be reduced, for example, to half the previous interval, subject to some upper and lower limits.

Recovery from deadlock detection

As we mentioned previously, once deadlock has been detected the DBMS needs to abort one or more of the transactions. There are several issues that need to be considered:

- (1) *Choice of deadlock victim.* In some circumstances, the choice of transactions to abort may be obvious. However, in other situations, the choice may not be so clear. In such cases, we would want to abort the transactions that incur the minimum costs. This may take into consideration:
 - (a) how long the transaction has been running (it may be better to abort a transaction that has just started rather than one that has been running for some time);
 - (b) how many data items have been updated by the transaction (it would be better to abort a transaction that has made little change to the database rather than one that has made significant changes to the database);
 - (c) how many data items the transaction is still to update (it would be better to abort a transaction that has many changes still to make to the database rather than one that has few changes to make). Unfortunately, this may not be something that the DBMS would necessarily know.

- (2) *How far to roll a transaction back.* Having decided to abort a particular transaction, we have to decide how far to roll the transaction back. Clearly, undoing all the changes made by a transaction is the simplest solution, although not necessarily the most efficient. It may be possible to resolve the deadlock by rolling back only part of the transaction.
- (3) *Avoiding starvation.* Starvation occurs when the same transaction is always chosen as the victim, and the transaction can never complete. Starvation is very similar to livelock, mentioned in Section 22.2.3, which occurs when the concurrency control protocol never selects a particular transaction that is waiting for a lock. The DBMS can avoid starvation by storing a count of the number of times a transaction has been selected as the victim and using a different selection criterion once this count reaches some upper limit.

22.2.5 Timestamping Methods

The use of locks, combined with the two-phase locking protocol, guarantees serializability of schedules. The order of transactions in the equivalent serial schedule is based on the order in which the transactions lock the items they require. If a transaction needs an item that is already locked, it may be forced to wait until the item is released. A different approach that also guarantees serializability uses transaction timestamps to order transaction execution for an equivalent serial schedule.

Timestamp methods for concurrency control are quite different from locking methods. No locks are involved and therefore there can be no deadlock. Locking methods generally prevent conflicts by making transactions wait. With timestamp methods, there is no waiting: transactions involved in conflict are simply rolled back and restarted.

Timestamp

A unique identifier created by the DBMS that indicates the relative starting time of a transaction.

Timestamps can be generated by simply using the system clock at the time the transaction started, or, more normally, by incrementing a logical counter every time a new transaction starts.

Timestamping

A concurrency control protocol that orders transactions in such a way that older transactions, transactions with *smaller* timestamps, get priority in the event of conflict.

With timestamping, if a transaction attempts to read or write a data item, then the read or write is only allowed to proceed if the *last update on that data item* was carried out by an older transaction. Otherwise, the transaction requesting the read/write is restarted and given a new timestamp. New timestamps must be assigned to restarted transactions to prevent their being continually aborted and restarted. Without new timestamps, a transaction with an old timestamp might not be able to commit owing to younger transactions having already committed.

In addition to timestamps for transactions, there are timestamps for data items. Each data item contains a **read_timestamp**, giving the timestamp of the last transaction to read the item, and a **write_timestamp**, giving the timestamp of the last transaction to write (update) the item. For a transaction T with timestamp $ts(T)$, the timestamp ordering protocol works as follows:

(1) *Transaction T issues a read(x).*

- (a) Transaction T asks to read an item (x) that has already been updated by a younger (later) transaction, that is, $ts(T) < write_timestamp(x)$. This means that an earlier transaction is trying to read a value of an item that has been updated by a later transaction. The earlier transaction is too late to read the previous outdated value, and any other values it has acquired are likely to be inconsistent with the updated value of the data item. In this situation, transaction T must be aborted and restarted with a new (later) timestamp.
- (b) Otherwise, $ts(T) \geq write_timestamp(x)$, and the read operation can proceed. We set $read_timestamp(x) = \max(ts(T), read_timestamp(x))$.

(2) *Transaction T issues a write(x).*

- (a) Transaction T asks to write an item (x) whose value has already been read by a younger transaction, that is $ts(T) < read_timestamp(x)$. This means that a later transaction is already using the current value of the item and it would be an error to update it now. This occurs when a transaction is late in doing a write and a younger transaction has already read the old value or written a new one. In this case, the only solution is to roll back transaction T and restart it using a later timestamp.
- (b) Transaction T asks to write an item (x) whose value has already been written by a younger transaction, that is $ts(T) < write_timestamp(x)$. This means that transaction T is attempting to write an obsolete value of data item x . Transaction T should be rolled back and restarted using a later timestamp.
- (c) Otherwise, the write operation can proceed. We set $write_timestamp(x) = ts(T)$.

This scheme, called **basic timestamp ordering**, guarantees that transactions are conflict serializable, and the results are equivalent to a serial schedule in which the transactions are executed in chronological order of the timestamps. In other words, the results will be as if all of transaction 1 were executed, then all of transaction 2, and so on, with no interleaving. However, basic timestamp ordering does not guarantee recoverable schedules. Before we show how these rules can be used to generate a schedule using timestamping, we first examine a slight variation to this protocol that provides greater concurrency.

Thomas's write rule

A modification to the basic timestamp ordering protocol that relaxes conflict serializability can be used to provide greater concurrency by rejecting obsolete write operations (Thomas, 1979). The extension, known as **Thomas's write rule**, modifies the checks for a write operation by transaction T as follows:

- (a) Transaction T asks to write an item (x) whose value has already been read by a younger transaction, that is, $ts(T) < read_timestamp(x)$. As before, roll back transaction T and restart it using a later timestamp.
- (b) Transaction T asks to write an item (x) whose value has already been written by a younger transaction, that is $ts(T) < write_timestamp(x)$. This means that a later transaction has already updated the value of the item, and the value that the older transaction is writing must be based on an obsolete value of the item. In this

case, the write operation can safely be ignored. This is sometimes known as the **ignore obsolete write rule**, and allows greater concurrency.

- (c) Otherwise, as before, the write operation can proceed. We set $\text{write_timestamp}(x) = \text{ts}(T)$.

The use of Thomas’s write rule allows schedules to be generated that would not have been possible under the other concurrency protocols discussed in this section. For example, the schedule shown in Figure 22.10 is not conflict serializable: the write operation on bal_x by transaction T_{11} following the write by T_{12} would be rejected, and T_{11} would need to be rolled back and restarted with a new time-stamp. In contrast, using Thomas’s write rule this view serializable schedule would be valid without requiring any transactions to be rolled back.

We examine another timestamping protocol that is based on the existence of multiple versions of each data item in the next section.

EXAMPLE 22.10 Basic timestamp ordering

Three transactions are executing concurrently, as illustrated in Figure 22.21. Transaction T_{19} has a timestamp of $\text{ts}(T_{19})$, T_{20} has a timestamp of $\text{ts}(T_{20})$, and T_{21} has a timestamp of $\text{ts}(T_{21})$, such that $\text{ts}(T_{19}) < \text{ts}(T_{20}) < \text{ts}(T_{21})$. At time t_8 , the write by transaction T_{20} violates the first write rule and so T_{20} is aborted and restarted at time t_{14} . Also at time t_{14} , the write by transaction T_{19} can safely be ignored using the ignore obsolete write rule, as it would have been overwritten by the write of transaction T_{21} at time t_{12} .

Time	Op	T_{19}	T_{20}	T_{21}
t_1		begin_transaction		
t_2	read(bal_x)	read(bal_x)		
t_3	$\text{bal}_x = \text{bal}_x + 10$	$\text{bal}_x = \text{bal}_x + 10$		
t_4	write(bal_x)	write(bal_x)	begin_transaction	
t_5	read(bal_y)		read(bal_y)	
t_6	$\text{bal}_y = \text{bal}_y + 20$		$\text{bal}_y = \text{bal}_y + 20$	begin_transaction
t_7	read(bal_y)			read(bal_y)
t_8	write(bal_y)		write(bal_y) [†]	
t_9	$\text{bal}_y = \text{bal}_y + 30$			$\text{bal}_y = \text{bal}_y + 30$
t_{10}	write(bal_y)			write(bal_y)
t_{11}	$\text{bal}_z = 100$			$\text{bal}_z = 100$
t_{12}	write(bal_z)			write(bal_z)
t_{13}	$\text{bal}_z = 50$	$\text{bal}_z = 50$		commit
t_{14}	write(bal_z)	write(bal_z) [‡]	begin_transaction	
t_{15}	read(bal_y)	commit	read(bal_y)	
t_{16}	$\text{bal}_y = \text{bal}_y + 20$		$\text{bal}_y = \text{bal}_y + 20$	
t_{17}	write(bal_y)		write(bal_y)	
t_{18}			commit	

[†] At time t_8 , the write by transaction T_{20} violates the first timestamping write rule described previously and therefore is aborted and restarted at time t_{14} .

[‡] At time t_{14} , the write by transaction T_{19} can safely be ignored using the ignore obsolete write rule, as it would have been overwritten by the write of transaction T_{21} at time t_{12} .

Figure 22.21 Timestamping example.

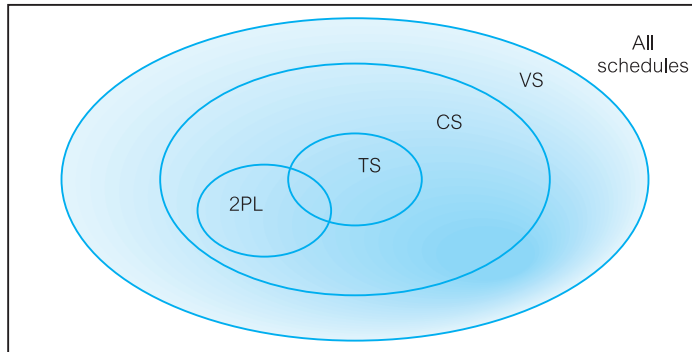


Figure 22.22
Comparison
of conflict
serializability (CS),
view serializability
(VS), two-phase
locking (2PL), and
timestamping (TS).

Comparison of methods

Figure 22.22 illustrates the relationship between conflict serializability (CS), view serializability (VS), two-phase locking (2PL), and timestamping (TS). As can be seen, view serializability encompasses the other three methods, conflict serializability encompasses 2PL and timestamping, and 2PL and timestamping overlap. Note that in the last case that there are schedules common to both 2PL and time-stamping, but there are also schedules that can be produced by 2PL but not time-stamping, and vice versa.

22.2.6 Multiversion Timestamp Ordering

Versioning of data can also be used to increase concurrency, as different users may work concurrently on different versions of the same object instead of having to wait for each others' transactions to complete. In the event that the work appears faulty at any stage, it should be possible to roll back the work to some valid state. Versions have been used as an alternative to the nested and multilevel concurrency control protocols that we discuss in Section 22.4 (for example, see Beech and Mahbod, 1988; Chou and Kim, 1986, 1988). In this section we briefly examine one concurrency control scheme that uses versions to increase concurrency based on timestamps (Reed, 1978; 1983). In Section 22.5 we briefly discuss how Oracle uses this scheme for concurrency control.

The basic timestamp ordering protocol discussed in the previous section assumes that only one version of a data item exists, and so only one transaction can access a data item at a time. This restriction can be relaxed if we allow multiple transactions to read and write different versions of the same data item, and ensure that each transaction sees a consistent set of versions for all the data items it accesses. In multiversion concurrency control, each write operation creates a new version of a data item while retaining the old version. When a transaction attempts to read a data item, the system selects the correct version of the data item according to the timestamp of the requesting transaction.

For each data item x , we assume that the database holds n versions $x_1, x_2, \dots, x_n$. For each version i , the system stores three values:

- the value of version x_i ;
- $\text{read_timestamp}(x_i)$, which is the largest timestamp of all transactions that have successfully read version x_i ;

- `write_timestamp(x)`, which is the timestamp of the transaction that created version x_i .

Let $ts(T)$ be the timestamp of the current transaction. The multiversion timestamp ordering protocol uses the following two rules to ensure serializability:

- (1) *Transaction T issues a write(x).* If transaction T wishes to write data item x , we must ensure that the data item has not been read already by some other transaction T_j such that $ts(T) < ts(T_j)$. If we allow transaction T to perform this write operation, then for serializability its change should be seen by T_j , but clearly T_j , which has already read the value, will not see T 's change.

Thus, if version x_j has the largest write timestamp of data item x that is *less than or equal to* $ts(T)$ (that is, $write_timestamp(x_j) \leq ts(T)$) and $read_timestamp(x_j) > ts(T)$, transaction T must be aborted and restarted with a new timestamp. Otherwise, we create a new version x_i of x and $setread_timestamp(x_i) = write_timestamp(x_i) = ts(T)$.

- (2) *Transaction T issues a read(x).* If transaction T wishes to read data item x , we must return the version x_j that has the largest write timestamp of data item x that is *less than or equal to* $ts(T)$. In other words, return x_j such that $write_timestamp(x_j) \leq ts(T)$. Set the value of $read_timestamp(x_j) = \max(ts(T), read_timestamp(x_j))$. Note that with this protocol a read operation never fails.

Versions can be deleted once they are no longer required. To determine whether a version is required, we find the timestamp of the oldest transaction in the system. Then, for any two versions x_i and x_j of data item x with write timestamps less than this oldest timestamp, we can delete the older version.

22.2.7 Optimistic Techniques

In some environments, conflicts between transactions are rare and the additional processing required by locking or timestamping protocols is unnecessary for many of the transactions. **Optimistic techniques** are based on the assumption that conflict is rare and that it is more efficient to allow transactions to proceed without imposing delays to ensure serializability (Kung and Robinson, 1981). When a transaction wishes to commit, a check is performed to determine whether conflict has occurred. If there has been a conflict, the transaction must be rolled back and restarted. Because the premise is that conflict occurs very infrequently, rollback will be rare. The overhead involved in restarting a transaction may be considerable, as it effectively means redoing the entire transaction. This could be tolerated only if it happened very infrequently, in which case the majority of transactions will be processed without being subjected to any delays. These techniques potentially allow greater concurrency than traditional protocols, as no locking is required.

There are two or three phases to an optimistic concurrency control protocol, depending on whether it is a read-only or an update transaction:

- *Read phase.* This extends from the start of the transaction until immediately before the commit. The transaction reads the values of all data items it needs from the database and stores them in local variables. Updates are applied to a local copy of the data, not to the database itself.

- *Validation phase.* This follows the read phase. Checks are performed to ensure that serializability is not violated if the transaction updates are applied to the database. For a read-only transaction, this consists of checking whether the data values read are still the current values for the corresponding data items. If no interference occurred, the transaction is committed. If interference occurred, the transaction is aborted and restarted. For a transaction that has updates, validation consists of determining whether the current transaction leaves the database in a consistent state, with serializability maintained. If not, the transaction is aborted and needs to be restarted.
- *Write phase.* This follows the successful validation phase for update transactions. During this phase, the updates made to the local copy are applied to the database.

The validation phase examines the reads and writes of transactions that may cause interference. Each transaction T is assigned a timestamp at the start of its execution, $start(T)$; one at the start of its validation phase, $validation(T)$; and one at its finish time, $finish(T)$, including its write phase, if any. To pass the validation test, one of the following must be true:

- (1) All transactions S with earlier timestamps must have finished before transaction T started; that is, $finish(S) < start(T)$.
- (2) If transaction T starts before an earlier one S finishes, then:
 - (a) the set of data items written by the earlier transaction are not the ones read by the current transaction; *and*
 - (b) the earlier transaction completes its write phase before the current transaction enters its validation phase, that is, $start(T) < finish(S) < validation(T)$.

Rule 2(a) guarantees that the writes of an earlier transaction are not read by the current transaction; rule 2(b) guarantees that the writes are done serially, ensuring no conflict.

Although optimistic techniques are very efficient when there are few conflicts, they can result in the rollback of individual transactions. Note that the rollback involves only a local copy of the data, so there are no cascading rollbacks, because the writes have not actually reached the database. However, if the aborted transaction is of a long duration, valuable processing time will be lost, because the transaction must be restarted. If rollback occurs often, it is an indication that the optimistic method is a poor choice for concurrency control in that particular environment.

22.2.8 Granularity of Data Items

Granularity

The size of data items chosen as the *unit of protection* by a concurrency control protocol.

All the concurrency control protocols that we have discussed assume that the database consists of a number of “data items,” without explicitly defining the term. Typically, a data item is chosen to be one of the following, ranging from coarse to fine, where fine granularity refers to small item sizes and coarse granularity refers to large item sizes:

- the entire database;
- a file;

- a page (sometimes called an area or database space—a section of physical disk in which relations are stored);
- a record;
- a field value of a record.

The size or granularity of the data item that can be locked in a single operation has a significant effect on the overall performance of the concurrency control algorithm. However, there are several tradeoffs that have to be considered in choosing the data item size. We discuss these tradeoffs in the context of locking, although similar arguments can be made for other concurrency control techniques.

Consider a transaction that updates a single tuple of a relation. The concurrency control algorithm might allow the transaction to lock only that single tuple, in which case the granule size for locking is a single record. On the other hand, it might lock the entire database, in which case the granule size is the entire database. In the second case, the granularity would prevent any other transactions from executing until the lock is released. This would clearly be undesirable. On the other hand, if a transaction updates 95% of the records in a file, then it would be more efficient to allow it to lock the entire file rather than to force it to lock each record separately. However, escalating the granularity from field or record to file may increase the likelihood of deadlock occurring.

Thus, the coarser the data item size, the lower the degree of concurrency permitted. On the other hand, the finer the item size, the more locking information that needs to be stored. The best item size depends upon the nature of the transactions. If a typical transaction accesses a small number of records, it is advantageous to have the data item granularity at the record level. On the other hand, if a transaction typically accesses many records of the same file, it may be better to have page or file granularity so that the transaction considers all those records as one (or a few) data items.

Some techniques have been proposed that have dynamic data item sizes. With these techniques, depending on the types of transaction that are currently executing, the data item size may be changed to the granularity that best suits these transactions. Ideally, the DBMS should support mixed granularity with record, page, and file-level locking. Some systems automatically upgrade locks from record or page to file if a particular transaction is locking more than a certain percentage of the records or pages in the file.

Hierarchy of granularity

We could represent the granularity of locks in a hierarchical structure where each node represents data items of different sizes, as shown in Figure 22.23. Here, the root node represents the entire database, the level 1 nodes represent files, the level 2 nodes represent pages, the level 3 nodes represent records, and the level 4 leaves represent individual fields. Whenever a node is locked, all its descendants are also locked. For example, if a transaction locks a page, $Page_2$, all its records ($Record_1$ and $Record_2$) as well as all their fields ($Field_1$ and $Field_2$) are also locked. If another transaction requests an incompatible lock on the *same* node, the DBMS clearly knows that the lock cannot be granted.

If another transaction requests a lock on any of the *descendants* of the locked node, the DBMS checks the hierarchical path from the root to the requested node